

RFE 2.0 Interface

for S32R41

Rev.0.8.10/16.12.2024

© 2024 NXP Semiconductors

1 Module Index	1
1.1 Software Modules	1
2 Data Structure Index	3
2.1 Data Structures	3
3 Module Documentation	5
3.1 RFE Abstract 2.0	5
3.1.1 Detailed Description	5
3.2 RFE Abstract 2.0 API	5
3.2.1 Detailed Description	6
3.3 RFE Abstract 2.0 External Software Dependencies	6
3.3.1 Detailed Description	6
3.4 RFE Abstract 2.0 Firmware API	6
3.4.1 Detailed Description	7
3.5 RFE Abstract 2.0 Constants	7
3.5.1 Detailed Description	8
3.5.2 Macro Definition Documentation	8
3.5.2.1 CDD_RFE_CFG_HALT_ON_ERROR	9
3.5.2.2 CDD_RFE_CONFIGURE	9
3.5.2.3 CDD_RFE_DEV_ERROR_DETECT	9
3.5.2.4 CDD_RFE_E_CMD_LENGTH	9
3.5.2.5 CDD_RFE_E_CMD_NOT_IMPLEMENTD	9
3.5.2.6 CDD_RFE_E_FUSA_ERROR	10
3.5.2.7 CDD_RFE_E_FW_ERROR	10
3.5.2.8 CDD_RFE_E_LIMIT_EXCEED	10
3.5.2.9 CDD_RFE_E_PARAM_POINTER	10
3.5.2.10 CDD_RFE_E_PARAM_SIZE	10
3.5.2.11 CDD_RFE_E_REDUNDANCY_CHECK_FAILED	10
3.5.2.12 CDD_RFE_E_RESPONSE_ERROR	11
3.5.2.13 CDD_RFE_E_SETUP_FAILED	11
3.5.2.14 CDD_RFE_E_UNRESPONSIVE	11
3.5.2.15 CDD_RFE_GET_DATA	11
3.5.2.16 CDD_RFE_HALT_ON_ERROR	11
3.5.2.17 CDD_RFE_SET_DATA	12
3.5.2.18 CDD_RFE_SID_CONFIGURE	12
3.5.2.19 CDD_RFE_SID_GET_ERROR	12
3.5.2.20 CDD_RFE_SID_GET_VERSION_INFO	12
3.5.2.21 CDD_RFE_SID_GETFUSAERRORS	12
3.5.2.22 CDD_RFE_SID_GETNEXTRADARCYCLESTARTTIME	12
3.5.2.23 CDD_RFE_SID_GETRADARCYCLECOUNT	12
3.5.2.24 CDD_RFE_SID_GETSTATE	13
3.5.2.25 CDD_RFE_SID_GETTIME	13

3.5.2.26 CDD_RFE_SID_GETVERSION	13
3.5.2.27 CDD_RFE_SID_INIT	13
3.5.2.28 CDD_RFE_SID_INVALID	13
3.5.2.29 CDD_RFE_SID_MONITORREAD	13
3.5.2.30 CDD_RFE_SID_RADARCYCLESTART	13
3.5.2.31 CDD_RFE_SID_RADARCYCLESTOP	14
3.5.2.32 CDD_RFE_SID_SETNEXTRADARCYCLESTARTTIME	14
3.5.2.33 CDD_RFE_SID_TESTCONTINUOUSWAVETRANSMISSIONSTART	14
3.5.2.34 CDD_RFE_SID_TESTCONTINUOUSWAVETRANSMISSIONSTOP	14
3.5.2.35 CDD_RFE_SID_TESTSETPARAM	14
3.5.2.36 CDD_RFE_SID_UPDATEBEGIN	14
3.5.2.37 CDD_RFE_SID_UPDATEDYNAMICTABLE	14
3.5.2.38 CDD_RFE_SID_UPDATEPARAM	14
3.5.2.39 CDD_RFE_SID_UPDATEPUSH	15
3.5.2.40 CDD_RFE_SINGLE_MANAGEMENT_THREADS	15
3.5.2.41 CDD_RFE_START_EXECUTION	15
3.5.2.42 CDD_RFE_STOP_EXECUTION	15
3.5.2.43 CDD_RFE_VERSION_INFO_API	15
3.5.2.44 CDD_RFE_VERSION_INFO_CHECK	15
3.5.2.45 CDD_RFE_ZERO	15
3.5.2.46 REG_MAX_COUNT_TO_SEND	16
3.5.2.47 RFE_ADC_REG_NUMBER	16
3.5.2.48 RFE_ATB_VOLTAGE_TABLE_NUMBER	16
3.5.2.49 RFE_CAFC_REG_NUMBER	16
3.5.2.50 RFE_CC_REG_NUMBER	16
3.5.2.51 RFE_CSI2_REG_NUMBER	16
3.5.2.52 RFE_GB_REG_NUMBER	16
3.5.2.53 RFE_GLDO_REG_NUMBER	17
3.5.2.54 RFE_ISM_REG_NUMBER	17
3.5.2.55 RFE_LDVS_REG_NUMBER	17
3.5.2.56 RFE_LOI_REG_NUMBER	17
3.5.2.57 RFE_MCLK_REG_NUMBER	17
3.5.2.58 RFE_OTP_REG_NUMBER	17
3.5.2.59 RFE_OTP_TABLE_NUMBER	17
3.5.2.60 RFE_RX_REG_NUMBER	18
3.5.2.61 RFE_SC_REG_NUMBER	18
3.5.2.62 RFE_SSBMOD_REG_NUMBER	18
3.5.2.63 RFE_TE_REG_NUMBER	18
3.5.2.64 RFE_TX_REG_NUMBER	18
3.5.3 Variable Documentation	18
3.5.3.1 gsCddRfeLoopExit	18
3.6 RFE Abstract 2.0 Interface	18

3.6.1 Detailed Description	21
3.6.2 Typedef Documentation	22
3.6.2.1 Cdd_RfeApiIRQDataType	22
3.6.2.2 Cdd_RfeApiWaitForInterruptDataType	22
3.6.2.3 Cdd_RfeCfgParamType	23
3.6.2.4 Cdd_RfeCfgSectionType	23
3.6.2.5 Cdd_RfeChirpProfileIndexType	23
3.6.2.6 Cdd_RfeCoreIdDataType	23
3.6.2.7 Cdd_RfeDynParamsSelectType	23
3.6.2.8 Cdd_RfeErrorType	23
3.6.2.9 Cdd_RfeEventsIRQDataType	23
3.6.2.10 Cdd_RfeFusaFaultType	24
3.6.2.11 Cdd_RfeMonitorSelectType	24
3.6.2.12 Cdd_RfeMonitorValuesType	24
3.6.2.13 Cdd_RfeParamUpdateType	24
3.6.2.14 Cdd_RfeRadarCycleCountType	24
3.6.2.15 Cdd_RfeRxBistZeroHourRefDataType	24
3.6.2.16 Cdd_RfeStateType	25
3.6.2.17 Cdd_RfeSysMemAddressType	25
3.6.2.18 Cdd_RfeTestParamType	25
3.6.2.19 Cdd_RfeVersionType	25
3.6.3 Enumeration Type Documentation	25
3.6.3.1 CDD_RfeFrontEndHierarchyType	25
3.6.3.2 CDD_RfeFrontEndIdMaskType	26
3.6.3.3 rfe_bcdModuleId_t	26
3.6.3.4 rfe_moduleDumpSelect_t	27
3.6.4 Function Documentation	27
3.6.4.1 CDD_Rfe_Configure()	28
3.6.4.2 CDD_Rfe_ConfigureInterrupt()	29
3.6.4.3 CDD_Rfe_GetBistZeroHourReferenceData()	30
3.6.4.4 CDD_Rfe_GetBlobValue()	31
3.6.4.5 CDD_Rfe_GetError()	31
3.6.4.6 CDD_Rfe_GetFrontEnd()	32
3.6.4.7 CDD_Rfe_GetFuSaFaults()	32
3.6.4.8 CDD_Rfe_GetFuSaFaultStatistics()	33
3.6.4.9 CDD_Rfe_GetNextRadarCycleStartTime()	34
3.6.4.10 CDD_Rfe_GetRadarCycleCount()	35
3.6.4.11 CDD_Rfe_GetRegisterDump()	36
3.6.4.12 CDD_Rfe_GetState()	36
3.6.4.13 CDD_Rfe_GetTime()	37
3.6.4.14 CDD_Rfe_GetVersionInfo()	37
3.6.4.15 CDD_Rfe_Init()	38

3.6.4.16 CDD_Rfe_MonitorRead()	38
3.6.4.17 CDD_Rfe_RadarCycleStart()	39
3.6.4.18 CDD_Rfe_RadarCycleStop()	40
3.6.4.19 CDD_Rfe_SetBlobValue()	41
3.6.4.20 CDD_Rfe_SetFrontEnd()	41
3.6.4.21 CDD_Rfe_SetNextRadarCycleStartTime()	42
3.6.4.22 CDD_Rfe_SwHwGetVersion()	43
3.6.4.23 CDD_Rfe_TestContinuousWaveTransmissionStart()	44
3.6.4.24 CDD_Rfe_TestContinuousWaveTransmissionStop()	44
3.6.4.25 CDD_Rfe_TestGetInternalError()	45
3.6.4.26 CDD_Rfe_TestSetParam()	46
3.6.4.27 CDD_Rfe_UpdateBegin()	46
3.6.4.28 CDD_Rfe_UpdateDynamicTable()	47
3.6.4.29 CDD_Rfe_UpdateParam()	48
3.6.4.30 CDD_Rfe_UpdatePush()	48
3.7 RFE Abstract 2.0 - Low Level Constants	49
3.7.1 Detailed Description	51
3.7.2 Macro Definition Documentation	51
3.7.2.1 RFE_API_IRQ_ALL	51
3.7.2.2 RFE_API_IRQ_MAX_NUM	51
3.7.2.3 RFE_CMD_CLIENT_FUSA_FAULT_BYTE_COUNT	51
3.7.2.4 RFE_CMD_CLIENT_UPDATE_SECTION_DYNAMIC_TABLE	51
3.7.2.5 RFE_DATA_OUT_DEST_CSI2	52
3.7.2.6 RFE_DATA_OUT_DEST_PACKET_PROCESSOR	52
3.7.2.7 RFE_DYN_PARAMS_SELECT_ALL	52
3.7.2.8 RFE_DYN_PARAMS_SELECT_CHIRP_FREQUENCY_DRIFT_STEPS	52
3.7.2.9 RFE_DYN_PARAMS_SELECT_CHIRP_PROFILE	52
3.7.2.10 RFE_DYN_PARAMS_SELECT_CIT_TICKS	52
3.7.2.11 RFE_DYN_PARAMS_SELECT_DWELL_TIME_TICKS	52
3.7.2.12 RFE_DYN_PARAMS_SELECT_TX1_PHASE_ROTATION	53
3.7.2.13 RFE_DYN_PARAMS_SELECT_TX2_PHASE_ROTATION	53
3.7.2.14 RFE_DYN_PARAMS_SELECT_TX3_PHASE_ROTATION	53
3.7.2.15 RFE_DYN_PARAMS_SELECT_TX4_PHASE_ROTATION	53
3.7.2.16 RFE_DYN_PARAMS_SELECT_TX_TRANSMISSION_ENABLE	53
3.7.2.17 RFE_ERROR_RFE_INTERNAL_ERROR_BIT	53
3.7.2.18 RFE_EVENTS_IRQ_ALL	53
3.7.2.19 RFE_FUSA_R1_FAULT_COUNT_MAX	54
3.7.2.20 RFE_FUSA_R1_R2_FAULT_COUNT_MAX	54
3.7.2.21 RFE_FUSA_R1_R2_MASK_UNMASK_FAULT_BYTE_COUNT	54
3.7.2.22 RFE_FUSA_R2_FAULT_COUNT_MAX	54
3.7.2.23 RFE_KELVIN_TO_CELSIUS	54
3.7.2.24 RFE_MAX_SUPPORTED_DEVICES	54

3.7.2.25 RFE_MONITOR_SELECT_ALL	54
3.7.2.26 RFE_MONITOR_SELECT_PDC_CLIPPING_COUNT	55
3.7.2.27 RFE_MONITOR_SELECT_RX_SAT_COUNT_STAGE_1I	55
3.7.2.28 RFE_MONITOR_SELECT_TEMP_AFTER_CHIRP_SEQ	55
3.7.2.29 RFE_MONITOR_SELECT_TEMP_BEFORE_CHIRP_SEQ	55
3.7.2.30 RFE_MONITOR_SELECT_TEMP_IMMEDIATELY	55
3.7.2.31 RFE_MONITOR_SELECT_TX_POWER	55
3.7.2.32 RFE_NS_PER_TIME_TICK	55
3.7.2.33 RFE_RX_BIST_COMBINATIONS	56
3.7.2.34 RFE_RX_CLIPPING_RESET	56
3.7.2.35 RFE_RX_COUNT	56
3.7.2.36 RFE_RX_SELECT_1	56
3.7.2.37 RFE_RX_SELECT_2	56
3.7.2.38 RFE_RX_SELECT_3	56
3.7.2.39 RFE_RX_SELECT_4	56
3.7.2.40 RFE_RX_SELECT_ALL	56
3.7.2.41 RFE_RX_SELECT_NONE	57
3.7.2.42 RFE_RX_STAGE	57
3.7.2.43 RFE_TEMPERATURE_1_DEG_CELSIUS	57
3.7.2.44 RFE_TEMPERATURE_SENSOR_COUNT	57
3.7.2.45 RFE_TIME_TICKS_PER_US	57
3.7.2.46 RFE_TX_BIST_COMBINATIONS	57
3.7.2.47 RFE_TX_POWER_ERR	57
3.7.2.48 RFE_TX_SELECT_1	58
3.7.2.49 RFE_TX_SELECT_2	58
3.7.2.50 RFE_TX_SELECT_3	58
3.7.2.51 RFE_TX_SELECT_4	58
3.7.2.52 RFE_TX_SELECT_ALL	58
3.7.2.53 RFE_TX_SELECT_NONE	58
3.7.2.54 RFE_UPDATE_COUNT_MAX	58
3.8 RFE Abstract 2.0 - Low Level Interface	58
3.8.1 Detailed Description	80
3.8.2 Typedef Documentation	81
3.8.2.1 rfe_apiWaitForInterrupt_f	81
3.8.2.2 rfe_coreId_t	81
3.8.2.3 rfe_dataOutDest_t	81
3.8.2.4 rfe_dynParamsSelect_t	81
3.8.2.5 rfe_monitorSelect_t	81
3.8.2.6 rfe_rxSelect_t	81
3.8.2.7 rfe_sysMemAddress_t	82
3.8.2.8 rfe_temperature_t	82
3.8.2.9 rfe_txSelect_t	82

3.8.3 Enumeration Type Documentation	82
3.8.3.1 rfe_apilRQ_t	82
3.8.3.2 rfe_bistInterval_t	83
3.8.3.3 rfe_chirpPIIVco_t	84
3.8.3.4 rfe_chirpProfileIndex_t	84
3.8.3.5 rfe_chirpSequenceIndex_t	84
3.8.3.6 rfe_chirpSlopeDirection_t	85
3.8.3.7 rfe_coreId_et	85
3.8.3.8 rfe_error_t	86
3.8.3.9 rfe_eventsIRQ_t	100
3.8.3.10 rfe_fusaFault_R1_t	100
3.8.3.11 rfe_fuSaFault_t	102
3.8.3.12 rfe_fuSaFaultMaskIndex_t	105
3.8.3.13 rfe_paramUpdate_t	106
3.8.3.14 rfe_powerMode_t	109
3.8.3.15 rfe_PRPCGenMode_e	109
3.8.3.16 rfe_rxClippingReset_t	110
3.8.3.17 rfe_rxGain_t	110
3.8.3.18 rfe_rxHpfCutOffFrequency_t	111
3.8.3.19 rfe_rxIndex_t	111
3.8.3.20 rfe_rxLpfCutOffFrequency_t	111
3.8.3.21 rfe_rxSaturationThreshold_t	112
3.8.3.22 rfe_rxStage_t	112
3.8.3.23 rfe_state_t	113
3.8.3.24 rfe_temperatureSensorIndex_t	114
3.8.3.25 rfe_testParam_t	114
3.8.3.26 rfe_txIndex_t	114
3.8.3.27 rfe_txPowerErrorFlag_t	115
3.8.3.28 rfe_virtualChannel_t	115
3.8.3.29 rfeCfg_param_t	115
3.8.3.30 rfeCfg_section_t	119
3.8.3.31 rfeCfg_size_t	120
3.8.3.32 rfeCmdClient_sharedParameter_t	120
3.8.3.33 rfeSwBist_InjectToneMode_t	121
3.8.3.34 rfeTef82XXRfBistTest_t	121
3.8.4 Function Documentation	121
3.8.4.1 rfe_configure()	121
3.8.4.2 rfe_configureInterrupt()	123
3.8.4.3 rfe_getBistZeroHourReferenceData()	124
3.8.4.4 rfe_getFrontEnd()	125
3.8.4.5 rfe_getFuSaFaults()	125
3.8.4.6 rfe_getFuSaFaultStatistics()	126

3.8.4.7 rfe_getNextRadarCycleStartTime()	128
3.8.4.8 rfe_getRadarCycleCount()	128
3.8.4.9 rfe_getRegisterDump()	129
3.8.4.10 rfe_getState()	130
3.8.4.11 rfe_getTime()	130
3.8.4.12 rfe_getVersion()	131
3.8.4.13 rfe_monitorRead()	132
3.8.4.14 rfe_radarCycleStart()	133
3.8.4.15 rfe_radarCycleStop()	134
3.8.4.16 rfe_setFrontEnd()	134
3.8.4.17 rfe_setNextRadarCycleStartTime()	135
3.8.4.18 rfe_sync()	136
3.8.4.19 rfe_testContinuousWaveTransmissionStart()	137
3.8.4.20 rfe_testContinuousWaveTransmissionStop()	137
3.8.4.21 rfe_testGetInternalError()	138
3.8.4.22 rfe_testSetParam()	139
3.8.4.23 rfe_updateBegin()	139
3.8.4.24 rfe_updateDynamicTable()	140
3.8.4.25 rfe_updateParam()	141
3.8.4.26 rfe_updatePush()	141
3.8.4.27 rfeCmdClient_call()	142
3.8.4.28 rfeCmdClient_checkNotBusy()	143
3.8.4.29 rfeCmdClient_cmdParamAddBool()	143
3.8.4.30 rfeCmdClient_cmdParamAddBuffer()	144
3.8.4.31 rfeCmdClient_cmdParamAddInt16()	145
3.8.4.32 rfeCmdClient_cmdParamAddInt32()	145
3.8.4.33 rfeCmdClient_cmdParamAddInt8()	146
3.8.4.34 rfeCmdClient_cmdParamAddPointer()	146
3.8.4.35 rfeCmdClient_cmdParamAddUint16()	147
3.8.4.36 rfeCmdClient_cmdParamAddUint32()	148
3.8.4.37 rfeCmdClient_cmdParamAddUint8()	148
3.8.4.38 rfeCmdClient_cmdParamStart()	149
3.8.4.39 rfeCmdClient_cmdRespGetBool()	149
3.8.4.40 rfeCmdClient_cmdRespGetBuf()	150
3.8.4.41 rfeCmdClient_cmdRespGetInt16()	150
3.8.4.42 rfeCmdClient_cmdRespGetInt32()	151
3.8.4.43 rfeCmdClient_cmdRespGetInt8()	152
3.8.4.44 rfeCmdClient_cmdRespGetUint16()	152
3.8.4.45 rfeCmdClient_cmdRespGetUint32()	153
3.8.4.46 rfeCmdClient_cmdRespGetUint8()	153
3.8.4.47 rfeCmdClient_configureInterrupt()	154
3.8.4.48 rfeCmdClient_getCurrentResplen()	154

3.8.4.49 rfeCmdClient_init()	155
3.8.4.50 rfeCmdClient_sharedDataRead()	155
3.8.4.51 rfeCrc_calBuf()	156
3.9 RFE Abstract 2.0 - Front-End configuration structure : BLOB	156
3.9.1 Detailed Description	156
3.10 RFE Abstract 2.0 BLOB - Constants	156
3.10.1 Detailed Description	159
3.10.2 metadata	159
3.10.2.1 chirpSequenceConfigCount	159
3.10.2.2 chirpProfileCount	159
3.10.2.3 majorVersion	160
3.10.2.4 minorVersion	160
3.10.2.5 patchVersion	160
3.10.2.6 hashVersion	160
3.10.3 general	160
3.10.3.1 powerMode	160
3.10.3.2 pdcDecimationFilter	161
3.10.3.3 jumpbackTimeTicks	161
3.10.3.4 txCalibration	161
3.10.4 monitorAndSafety	161
3.10.4.1 rxSatCountResetEveryChirpSequence	162
3.10.4.2 rxSaturationThreshold	162
3.10.4.3 rxSaturationCountLimit	162
3.10.4.4 adcClippingCountResetEveryChirpSequence	162
3.10.4.5 frequencyForBist	162
3.10.4.6 zeroHrRefForRxPhaseDiffLna	162
3.10.4.7 rxPhaseDiffLnaThresholdTolerance	163
3.10.4.8 zeroHourReferenceForRxGainDiffLna	163
3.10.4.9 rxGainDiffLnaThresholdTolerance	163
3.10.4.10 zeroHrRefForRxPhaseDiffMixer	163
3.10.4.11 rxPhaseDiffMixerThresholdTolerance	163
3.10.4.12 zeroHourReferenceForRxGainDiffMixer	163
3.10.4.13 rxGainDiffMixerThresholdTolerance	163
3.10.4.14 injectTestTone	163
3.10.4.15 checkBbd_loi	164
3.10.4.16 checkBbd_tx	164
3.10.4.17 checkBbd_rx	164
3.10.4.18 fuSaFaultMask	164
3.10.5 radarCycle	164
3.10.5.1 radarCycleDuration	165
3.10.5.2 chirpSequenceCount	165
3.10.5.3 chirpSequence	165

3.10.5.4 chirpSequenceStartTimeOffset	165
3.10.5.5 bistInterval	165
3.10.6 chirpSequenceConfig	166
3.10.6.1 chirpCount	167
3.10.6.2 chirpProfileSequence	167
3.10.6.3 chirpProfileSequenceLength	167
3.10.6.4 chirpProfileRepeatCount	167
3.10.6.5 dynamicUpdatesEnabled	168
3.10.6.6 dynamicTableIndex	168
3.10.6.7 txPhaseRotation_ddma	168
3.10.6.8 txPhaseRotation_ddma_tx	168
3.10.6.9 txPhaseRotation_ddma_tx1_init	168
3.10.6.10 txPhaseRotation_ddma_tx1_update	169
3.10.6.11 txPhaseRotation_ddma_tx1_mode	169
3.10.6.12 rxEnable	169
3.10.6.13 txCalibration_thresholdHigh	169
3.10.6.14 txCalibration_thresholdLow	169
3.10.7 chirpProfile	169
3.10.7.1 effectiveSamplingFrequency	170
3.10.7.2 chirpIntervalTimeTicks	170
3.10.7.3 dwellTimeTicks	171
3.10.7.4 settleTimeTicks	171
3.10.7.5 acquisitionTimeTicks	171
3.10.7.6 resetTimeTicks	171
3.10.7.7 centerFrequency	171
3.10.7.8 effectiveChirpBandwidth	171
3.10.7.9 chirpPIIVcoSelect	171
3.10.7.10 chirpSlopeDirection	171
3.10.7.11 chirpPIILoopFilterBandwidth	172
3.10.7.12 txTransmissionEnable	172
3.10.7.13 txPower	172
3.10.7.14 txPhaseRotation	172
3.10.7.15 chirpFrequencyDriftSteps	172
3.10.7.16 rxGain	173
3.10.7.17 rxHpfCutOffFrequency	173
3.10.7.18 rxLpfCutOffFrequency	173
3.10.7.19 virtualChannel	173
3.10.8 Macro Definition Documentation	173
3.10.8.1 RFE_CFG_MAX_DATA_LENGTH	174
3.10.8.2 RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_0	174
3.10.8.3 RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_1	174
3.10.8.4 RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_2	174

3.10.8.5 RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_3	174
3.10.8.6 RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_4	174
3.10.8.7 RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_5	174
3.10.8.8 RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_6	175
3.10.8.9 RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_7	175
3.10.8.10 RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_0	175
3.10.8.11 RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_1	175
3.10.8.12 RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_2	175
3.10.8.13 RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_3	175
3.10.8.14 RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_4	175
3.10.8.15 RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_5	176
3.10.8.16 RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_6	176
3.10.8.17 RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_7	176
3.10.8.18 RFE_CFG_OFFSET_SECTION_CRC	176
3.10.8.19 RFE_CFG_OFFSET_SECTION_GENERAL	176
3.10.8.20 RFE_CFG_OFFSET_SECTION_METADATA	176
3.10.8.21 RFE_CFG_OFFSET_SECTION_MONITOR_AND_SAFETY	176
3.10.8.22 RFE_CFG_OFFSET_SECTION_RADAR_CYCLE	177
3.10.8.23 RFE_CFG_PARAM_OFFSET_MSK	177
3.10.8.24 RFE_CFG_PARAM_OFFSET_SHF	177
3.10.8.25 RFE_CFG_PARAM_SECTION_CHIRP_PROFILE	177
3.10.8.26 RFE_CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG	177
3.10.8.27 RFE_CFG_PARAM_SECTION_CRC	177
3.10.8.28 RFE_CFG_PARAM_SECTION_GENERAL	177
3.10.8.29 RFE_CFG_PARAM_SECTION_METADATA	178
3.10.8.30 RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY	178
3.10.8.31 RFE_CFG_PARAM_SECTION_MSK	178
3.10.8.32 RFE_CFG_PARAM_SECTION_RADAR_CYCLE	178
3.10.8.33 RFE_CFG_PARAM_SIZE_16_BIT	178
3.10.8.34 RFE_CFG_PARAM_SIZE_32_BIT	178
3.10.8.35 RFE_CFG_PARAM_SIZE_32_BIT_FLOAT	178
3.10.8.36 RFE_CFG_PARAM_SIZE_8_BIT	179
3.10.8.37 RFE_CFG_PARAM_SIZE_MSK	179
3.10.8.38 RFE_CFG_SIZE_SECTION_CHIRP_PROFILE	179
3.10.8.39 RFE_CFG_SIZE_SECTION_CHIRP_SEQUENCE	179
3.10.8.40 RFE_CFG_SIZE_SECTION_CRC	179
3.10.8.41 RFE_CFG_SIZE_SECTION_GENERAL	179
3.10.8.42 RFE_CFG_SIZE_SECTION_METADATA	179
3.10.8.43 RFE_CFG_SIZE_SECTION_MONITOR_AND_SAFETY	179
3.10.8.44 RFE_CFG_SIZE_SECTION_RADAR_CYCLE	180
3.10.8.45 RFE_CFG_SIZE_TOTAL	180
3.10.8.46 RFE_CHIRP_PROFILE_SEQUENCE_LENGTH_MAX	180

3.10.8.47 RFE_CHIRP_PROFILES_MAX	180
3.10.8.48 RFE_CHIRP_SEQUENCE_CONFIGS_MAX	180
3.10.8.49 RFE_CHIRP_SEQUENCES_PER_RADAR_CYCLE_MAX	180
3.10.8.50 RFE_FUSA_FAULT_MASK_PARAM_COUNT	180
3.10.8.51 RFE_SW_CFG_MNGR_CHIRP_PROFILE_PARAMS	181
3.10.8.52 RFE_SW_CFG_MNGR_CHIRP_SEQUENCE_PARAMS	181
3.10.8.53 RFE_SW_CFG_MNGR_GENRAL_PARAMS	181
3.10.8.54 RFE_SW_CFG_MNGR_MONITOR_AND_SAFETY_PARAMS	181
3.10.8.55 RFE_SW_CFG_MNGR_NO_OF_SECTION	181
3.10.8.56 RFE_SW_CFG_MNGR_NON_UPDATABLE_PARAMS	181
3.10.8.57 RFE_SW_CFG_MNGR_RADAR_CYCLE_PARAMS	181
3.10.8.58 RFE_SW_CFG_MNGR_UPDATABLE_PARAMS	182
3.10.8.59 RFE_TX_COUNT	182
3.10.9 Typedef Documentation	182
3.10.9.1 rfe_phaseRotation_t	182
3.10.10 Enumeration Type Documentation	182
3.10.10.1 rfe_chirpPllLoopFilterBandwidth_t	182
3.10.10.2 rfe_effectiveSamplingFrequency_t	183
3.10.10.3 rfe_txTransmissionReferenceTime_t	184
3.11 RFE Abstract 2.0 BLOB - Low Level Interface	184
3.11.1 Detailed Description	190
3.11.2 Function Documentation	190
3.11.2.1 rfeCfg_calcAndWriteCrc()	190
3.11.2.2 rfeCfg_CheckBlobCompatibility()	191
3.11.2.3 rfeCfg_chirpProfile_getAcquisitionTimeTicks()	191
3.11.2.4 rfeCfg_chirpProfile_getCenterFrequency()	191
3.11.2.5 rfeCfg_chirpProfile_getChirpIntervalTimeTicks()	192
3.11.2.6 rfeCfg_chirpProfile_getChirpPllLoopFilterBandwidth()	192
3.11.2.7 rfeCfg_chirpProfile_getChirpPllVco()	193
3.11.2.8 rfeCfg_chirpProfile_getChirpSlopeDirection()	193
3.11.2.9 rfeCfg_chirpProfile_getDwellTimeTicks()	193
3.11.2.10 rfeCfg_chirpProfile_getEffectiveChirpBandwidth()	194
3.11.2.11 rfeCfg_chirpProfile_getEffectiveSamplingFrequency()	194
3.11.2.12 rfeCfg_chirpProfile_getResetTimeTicks()	195
3.11.2.13 rfeCfg_chirpProfile_getRxGain()	195
3.11.2.14 rfeCfg_chirpProfile_getRxHpfCutOffFrequency()	196
3.11.2.15 rfeCfg_chirpProfile_getRxLpfCutOffFrequency()	196
3.11.2.16 rfeCfg_chirpProfile_getSettleTimeTicks()	196
3.11.2.17 rfeCfg_chirpProfile_getTxPhaseRotation()	197
3.11.2.18 rfeCfg_chirpProfile_getTxPower()	197
3.11.2.19 rfeCfg_chirpProfile_getTxTransmissionEnable()	198
3.11.2.20 rfeCfg_chirpProfile_getTxTransmissionReferenceTime()	198

3.11.2.21 rfeCfg_chirpProfile_getTxTransmissionTimeOffset()	199
3.11.2.22 rfeCfg_chirpProfile_getVirtualChannel()	199
3.11.2.23 rfeCfg_chirpProfile_setAcquisitionTimeTicks()	199
3.11.2.24 rfeCfg_chirpProfile_setCenterFrequency()	200
3.11.2.25 rfeCfg_chirpProfile_setChirpIntervalTimeTicks()	200
3.11.2.26 rfeCfg_chirpProfile_setChirpPllLoopFilterBandwidth()	201
3.11.2.27 rfeCfg_chirpProfile_setChirpPllVco()	201
3.11.2.28 rfeCfg_chirpProfile_setChirpSlopeDirection()	201
3.11.2.29 rfeCfg_chirpProfile_setDwellTimeTicks()	202
3.11.2.30 rfeCfg_chirpProfile_setEffectiveChirpBandwidth()	202
3.11.2.31 rfeCfg_chirpProfile_setEffectiveSamplingFrequency()	203
3.11.2.32 rfeCfg_chirpProfile_setResetTimeTicks()	203
3.11.2.33 rfeCfg_chirpProfile_setRxGain()	203
3.11.2.34 rfeCfg_chirpProfile_setRxHpfCutOffFrequency()	204
3.11.2.35 rfeCfg_chirpProfile_setRxLpfCutOffFrequency()	204
3.11.2.36 rfeCfg_chirpProfile_setSettleTimeTicks()	205
3.11.2.37 rfeCfg_chirpProfile_setTxPhaseRotation()	205
3.11.2.38 rfeCfg_chirpProfile_setTxPower()	205
3.11.2.39 rfeCfg_chirpProfile_setTxTransmissionEnable()	206
3.11.2.40 rfeCfg_chirpProfile_setTxTransmissionReferenceTime()	206
3.11.2.41 rfeCfg_chirpProfile_setTxTransmissionTimeOffset()	207
3.11.2.42 rfeCfg_chirpProfile_setVirtualChannel()	207
3.11.2.43 rfeCfg_chirpSequence_getChirpCount()	207
3.11.2.44 rfeCfg_chirpSequence_getChirpFrequencyDriftSteps()	208
3.11.2.45 rfeCfg_chirpSequence_getChirpProfileRepeatCount()	208
3.11.2.46 rfeCfg_chirpSequence_getChirpProfileSequence()	209
3.11.2.47 rfeCfg_chirpSequence_getChirpProfileSequenceLength()	209
3.11.2.48 rfeCfg_chirpSequence_getDynamicTableIndex()	209
3.11.2.49 rfeCfg_chirpSequence_getDynamicUpdatesEnabled()	210
3.11.2.50 rfeCfg_chirpSequence_getRxEnable()	210
3.11.2.51 rfeCfg_chirpSequence_setChirpCount()	211
3.11.2.52 rfeCfg_chirpSequence_setChirpProfileRepeatCount()	211
3.11.2.53 rfeCfg_chirpSequence_setChirpProfileSequence()	211
3.11.2.54 rfeCfg_chirpSequence_setChirpProfileSequenceLength()	212
3.11.2.55 rfeCfg_chirpSequence_setDynamicTableIndex()	213
3.11.2.56 rfeCfg_chirpSequence_setDynamicUpdatesEnabled()	213
3.11.2.57 rfeCfg_chirpSequence_setRxEnable()	214
3.11.2.58 rfeCfg_general_getJumpbackTimeTicks()	214
3.11.2.59 rfeCfg_general_getPdcBitwidth()	214
3.11.2.60 rfeCfg_general_getPowerMode()	215
3.11.2.61 rfeCfg_general_getTxCalibrationMode()	215
3.11.2.62 rfeCfg_general_setPowerMode()	215

3.11.2.63 rfeCfg_metadata_getChirpProfileCount()	216
3.11.2.64 rfeCfg_metadata_getChirpSequenceConfigCount()	216
3.11.2.65 rfeCfg_monitorAndSafety_getAdcClippingCountResetEveryChirpSequence()	217
3.11.2.66 rfeCfg_monitorAndSafety_getBist()	217
3.11.2.67 rfeCfg_monitorAndSafety_getFrequencyForBist()	218
3.11.2.68 rfeCfg_monitorAndSafety_getFuSaFaultMask()	218
3.11.2.69 rfeCfg_monitorAndSafety_getInjectTestTone()	219
3.11.2.70 rfeCfg_monitorAndSafety_getLoiBBD()	219
3.11.2.71 rfeCfg_monitorAndSafety_getRxBBD()	219
3.11.2.72 rfeCfg_monitorAndSafety_getRxSatCountResetEveryChirpSequence()	220
3.11.2.73 rfeCfg_monitorAndSafety_getRxSaturationCountLimitStage()	220
3.11.2.74 rfeCfg_monitorAndSafety_getRxSaturationThresholdStage()	221
3.11.2.75 rfeCfg_monitorAndSafety_getTxBBD()	221
3.11.2.76 rfeCfg_monitorAndSafety_setBist()	222
3.11.2.77 rfeCfg_monitorAndSafety_setFrequencyForBist()	222
3.11.2.78 rfeCfg_monitorAndSafety_setInjectTestTone()	223
3.11.2.79 rfeCfg_monitorAndSafety_setLoiBBD()	223
3.11.2.80 rfeCfg_radarCycle_getBistInterval()	223
3.11.2.81 rfeCfg_radarCycle_getChirpSequence()	224
3.11.2.82 rfeCfg_radarCycle_getChirpSequenceCount()	224
3.11.2.83 rfeCfg_radarCycle_getChirpSequenceStartTimeOffset()	225
3.11.2.84 rfeCfg_radarCycle_getRadarCycleDuration()	225
3.11.2.85 rfeCfg_radarCycle_setRadarCycleDuration()	226
3.11.2.86 rfeCfg_read()	226
3.11.2.87 rfeCfg_readB()	226
3.11.2.88 rfeCfg_readDW()	227
3.11.2.89 rfeCfg_readW()	227
3.11.2.90 rfeCfg_sequence_getFinalPcGenMode()	228
3.11.2.91 rfeCfg_sequence_getPhaseRotationDdmaMode()	228
3.11.2.92 rfeCfg_sequence_getPhaseRotationTxUseDdma()	228
3.11.2.93 rfeCfg_sequence_getTxCalibrationThresholdHigh()	228
3.11.2.94 rfeCfg_sequence_getTxCalibrationThresholdLow()	229
3.11.2.95 rfeCfg_sequence_getTxDdmaInitPhase()	229
3.11.2.96 rfeCfg_sequence_getTxDdmaUpdatePhase()	229
3.11.2.97 rfeCfg_sequence_setPhaseRotationDdmaMode()	230
3.11.2.98 rfeCfg_sequence_setTxDdmaInitPhase()	231
3.11.2.99 rfeCfg_sequence_setTxDdmaUpdatePhase()	231
3.11.2.100 rfeCfg_sequence_setTxFinalPcGenMode()	232
3.11.2.101 rfeCfg_util_chirpProfileIndex()	232
3.11.2.102 rfeCfg_util_chirpProfileSequence()	232
3.11.2.103 rfeCfg_util_chirpSequenceConfigIndex()	233
3.11.2.104 rfeCfg_util_radarCycleChirpSequence()	233

3.11.2.105 rfeCfg_write()	234
3.12 RFE Abstract 2.0 - IPC Interface	234
3.12.1 Detailed Description	235
3.12.2 Function Documentation	235
3.12.2.1 rfelpc_clientCmdBuffAquire()	235
3.12.2.2 rfelpc_clientCmdTx()	235
3.12.2.3 rfelpc_clientRspWait()	235
3.12.2.4 rfelpc_clientStsPoll()	236
3.12.2.5 rfelpc_clientStsWait()	236
3.12.2.6 rfelpc_srvCmdPoll()	237
3.12.2.7 rfelpc_srvRspBuffAcquire()	237
3.12.2.8 rfelpc_srvRspTx()	237
3.12.2.9 rfelpc_srvStsBuffAcquire()	238
3.12.2.10 rfelpc_srvStsTx()	238
3.13 RFE Abstract 2.0 - Peripheral Access Interface	238
3.13.1 Detailed Description	239
3.14 RFE Abstract 2.0 - GPIO Interface	239
3.14.1 Detailed Description	239
3.14.2 Enumeration Type Documentation	240
3.14.2.1 TEF82XX_ExtPinID_e	240
3.14.3 Function Documentation	240
3.14.3.1 RfeHwGpioGet()	240
3.14.3.2 RfeHwGpioInit()	241
3.14.3.3 RfeHwGpioSet()	241
3.15 RFE Abstract 2.0 - SPI Interface	242
3.15.1 Detailed Description	242
3.15.2 Function Documentation	242
3.15.2.1 RfeHwSpiInitialize()	242
3.15.2.2 RfeHwSpiTransfer()	242
3.16 RFE Abstract 2.0 - Timers Interface	244
3.16.1 Detailed Description	244
3.16.2 Function Documentation	244
3.16.2.1 RfeHwDelayUs()	244
3.16.2.2 RfeHwHeartBeatTimerGetValue()	245
3.16.2.3 RfeHwTimersInit()	245
3.17 RFE Abstract 2.0 - Interrupt Interface	245
3.17.1 Detailed Description	246
3.17.2 Function Documentation	246
3.17.2.1 RfeHwRaiseIRQ()	246
3.17.2.2 RfeHwSetIRQ()	246
3.18 RFE Abstract 2.0 - Debug Interface	246
3.18.1 Detailed Description	247

3.18.2 Function Documentation	247
3.18.2.1 RfeDbgPrintMsg()	247
3.18.2.2 RfelnitDebug()	247
3.19 RFE Abstract 2.0 - Firmware Constants	248
3.19.1 Detailed Description	248
3.19.2 Macro Definition Documentation	248
3.19.2.1 RFE_SW_MAIN_FSM_MAIN_STATE_COUNT	248
3.20 RFE Abstract 2.0 - Firmware Interface	248
3.20.1 Detailed Description	249
3.20.2 Enumeration Type Documentation	249
3.20.2.1 rfeFrontEnd_Configuration_t	249
3.20.2.2 rfeSwMainFsm_mainState_t	249
3.20.3 Function Documentation	249
3.20.3.1 rfeMainFsm()	250
3.20.3.2 rfeSetFrontendsConfiguration()	250
3.20.3.3 rfeSetSharedAdresses()	251
4 Data Structure Documentation	253
4.1 ATB_ADCVoltages_t Struct Reference	253
4.1.1 Detailed Description	253
4.1.2 Field Documentation	253
4.1.2.1 ADC_NODE_1V1_SAFETY_SUPPLY	253
4.1.2.2 ADC_NODE_1V1_SUPPLY	253
4.1.2.3 ADC_NODE_1V8_SUPPLY	254
4.2 ATB_ATBVoltages_t Struct Reference	254
4.2.1 Detailed Description	254
4.2.2 Field Documentation	254
4.2.2.1 ATB_NODE_VDDA_1V1	254
4.2.2.2 ATB_NODE_VDDA_1V8	254
4.2.2.3 ATB_NODE_VDDD_SAFETY_1V1	254
4.3 ATB_CAFCVoltages_t Struct Reference	255
4.3.1 Detailed Description	255
4.3.2 Field Documentation	255
4.3.2.1 CAFC_NODE_LDO_CPPFD_VDD	255
4.3.2.2 CAFC_NODE_LDO_DIVIDER_VDD	255
4.3.2.3 CAFC_NODE_LDO_VCO_VDD	255
4.3.2.4 CAFC_NODE_VDD_PLL_1V1_FILTERED	256
4.3.2.5 CAFC_NODE_VDD_PLL_1V8	256
4.3.2.6 CAFC_NODE_VDD_VCO_1V8	256
4.4 ATB_GBIASVoltages_t Struct Reference	256
4.4.1 Detailed Description	256
4.4.2 Field Documentation	256

4.4.2.1 GBIAS_NODE_VDDA_GLOBAL_BIAS_1V8	256
4.5 ATB_GLDOVoltages_t Struct Reference	257
4.5.1 Detailed Description	257
4.5.2 Field Documentation	257
4.5.2.1 GLDO_NODE_VDDA_LDO_1V1	257
4.5.2.2 GLDO_NODE_VDDA_LDO_1V8	257
4.5.2.3 GLDO_NODE_VDDA_LDO_3V3	257
4.5.2.4 GLDO_NODE_VDDD_SAFETY_1V1	257
4.6 ATB_LOIVoltages_t Struct Reference	258
4.6.1 Detailed Description	258
4.6.2 Field Documentation	258
4.6.2.1 LOI_NODE_VDDA_1V8	258
4.6.2.2 LOI_NODE_VDDA_INTERFACE_1V1	258
4.6.2.3 LOI_NODE_VDDD_SAFETY_1V1	258
4.7 ATB_MCLKVoltages_t Struct Reference	258
4.7.1 Detailed Description	259
4.7.2 Field Documentation	259
4.7.2.1 MCLK_NODE_VDDA_1V1	259
4.7.2.2 MCLK_NODE_VDDA_1V8	259
4.7.2.3 MCLK_NODE_VDDD_SAFETY_1V1	259
4.8 ATB_MIPIVoltages_t Struct Reference	259
4.8.1 Detailed Description	260
4.8.2 Field Documentation	260
4.8.2.1 MIPI_NODE_SUPPLY1V1_CSI2	260
4.8.2.2 MIPI_NODE_SUPPLY1V1_SER	260
4.8.2.3 MIPI_NODE_SUPPLY1V8	260
4.8.2.4 MIPI_NODE_SUPPLY_MIPI_3v3	260
4.8.2.5 MIPI_NODE_SUPPLYIO_1v8_3v3	260
4.9 ATB_RXVoltages_t Struct Reference	260
4.9.1 Detailed Description	261
4.9.2 Field Documentation	261
4.9.2.1 RX_NODE_VDDA_1V1	261
4.9.2.2 RX_NODE_VDDA_1V8	261
4.9.2.3 RX_NODE_VDDD_1V1	261
4.10 ATB_SSBVoltages_t Struct Reference	261
4.10.1 Detailed Description	262
4.10.2 Field Documentation	262
4.10.2.1 SSB_NODE_VDDA_1V1	262
4.10.2.2 SSB_NODE_VDDA_1V8	262
4.10.2.3 SSB_NODE_VDDD_1V1	262
4.11 ATB_TXVoltages_t Struct Reference	262
4.11.1 Detailed Description	262

4.11.2 Field Documentation	263
4.11.2.1 TX_NODE_VDDA_1V1	263
4.11.2.2 TX_NODE_VDDA_1V8	263
4.11.2.3 TX_NODE_VDDD_1V1_FS	263
4.12 rfe_monitorValues_t Struct Reference	263
4.12.1 Detailed Description	263
4.12.2 Field Documentation	263
4.12.2.1 pdcClippingCount	264
4.12.2.2 rxSaturationClippingCount	264
4.12.2.3 temperature_afterChirpSequence	264
4.12.2.4 temperature_beforeChirpSequence	264
4.12.2.5 temperature_immediately	264
4.12.2.6 txPower	264
4.13 rfe_radarCycleCount_t Struct Reference	264
4.13.1 Detailed Description	265
4.13.2 Field Documentation	265
4.13.2.1 chirpSequenceCount	265
4.13.2.2 radarCycleCount	265
4.14 rfe_registerDumpValues_t Struct Reference	265
4.14.1 Detailed Description	265
4.14.2 Field Documentation	266
4.14.2.1 frontEndId	266
4.14.2.2 moduleId	266
4.14.2.3 registerAddress	266
4.14.2.4 value	266
4.15 rfe_tef82xxVoltages_t Struct Reference	266
4.15.1 Detailed Description	267
4.15.2 Field Documentation	267
4.15.2.1 ADC12Voltages	267
4.15.2.2 ADC34Voltages	267
4.15.2.3 ATBVoltages	267
4.15.2.4 CAFCVoltages	267
4.15.2.5 GBIASVoltages	267
4.15.2.6 GLDOVoltages	267
4.15.2.7 LOIVoltages	268
4.15.2.8 MCLKVoltages	268
4.15.2.9 MIPIVoltages	268
4.15.2.10 RX1Voltages	268
4.15.2.11 RX2Voltages	268
4.15.2.12 RX3Voltages	268
4.15.2.13 RX4Voltages	268
4.15.2.14 SSBVoltages	268

4.15.2.15 TX1Voltages	269
4.15.2.16 TX2Voltages	269
4.15.2.17 TX3Voltages	269
4.16 rfe_version_t Struct Reference	269
4.16.1 Detailed Description	269
4.16.2 Field Documentation	269
4.16.2.1 fwHash	270
4.16.2.2 fwVariant	270
4.16.2.3 fwVersionMajor	270
4.16.2.4 fwVersionMinor	270
4.16.2.5 fwVersionPatch	270
4.16.2.6 fwVersionReleased	270
4.16.2.7 hwType	270
4.16.2.8 hwUniqueld	271
4.16.2.9 hwVariant	271
4.16.2.10 hwVersion	271
4.17 rfeCfg_rxBistReferenceData_t Struct Reference	271
4.17.1 Detailed Description	271
4.17.2 Field Documentation	271
4.17.2.1 refMag	272
4.17.2.2 refPh	272
4.18 rfeCmdClient_update_t Struct Reference	272
4.18.1 Detailed Description	272
4.18.2 Field Documentation	272
4.18.2.1 param	272
4.18.2.2 section	272
4.18.2.3 value	272

Chapter 1

Module Index

1.1 Software Modules

Here is a list of all modules:

RFE Abstract 2.0	5
RFE Abstract 2.0 API	5
RFE Abstract 2.0 Constants	7
RFE Abstract 2.0 Interface	18
RFE Abstract 2.0 - Low Level Constants	49
RFE Abstract 2.0 - Low Level Interface	58
RFE Abstract 2.0 - Front-End configuration structure : BLOB	156
RFE Abstract 2.0 BLOB - Constants	156
RFE Abstract 2.0 BLOB - Low Level Interface	184
RFE Abstract 2.0 External Software Dependencies	6
RFE Abstract 2.0 - IPC Interface	234
RFE Abstract 2.0 - Peripheral Access Interface	238
RFE Abstract 2.0 - GPIO Interface	239
RFE Abstract 2.0 - SPI Interface	242
RFE Abstract 2.0 - Timers Interface	244
RFE Abstract 2.0 - Interrupt Interface	245
RFE Abstract 2.0 - Debug Interface	246
RFE Abstract 2.0 Firmware API	6
RFE Abstract 2.0 - Firmware Constants	248
RFE Abstract 2.0 - Firmware Interface	248

Chapter 2

Data Structure Index

2.1 Data Structures

Here are the data structures with brief descriptions:

ATB_ADCVoltages_t	253
ATB_ATBVoltages_t	254
ATB_CAFCVoltages_t	255
ATB_GBIASVoltages_t	256
ATB_GLDOVoltages_t	257
ATB_LOIVoltages_t	258
ATB_MCLKVoltages_t	258
ATB_MIPIVoltages_t	259
ATB_RXVoltages_t	260
ATB_SSBVoltages_t	261
ATB_TXVoltages_t	262
rfe_monitorValues_t	263
rfe_radarCycleCount_t	264
rfe_registerDumpValues_t	265
rfe_tef82xxVoltages_t	266
rfe_version_t	269
rfeCfg_rxBistReferenceData_t	
Struct to hold value of RX Bist reference data	271
rfeCmdClient_update_t	272

Chapter 3

Module Documentation

3.1 RFE Abstract 2.0

Abstraction module to setup and communicate with the attached Front-Ends.

Modules

- [RFE Abstract 2.0 API](#)
- [RFE Abstract 2.0 External Software Dependencies](#)
- [RFE Abstract 2.0 Firmware API](#)

3.1.1 Detailed Description

Abstraction module to setup and communicate with the attached Front-Ends.

This software module is part of "Radar SDK" deliverable. It provides two sub-modules - namely: *RFE Firmware* & *RFE API* - both of which are essential in the process of communicating with the connected Radar Front-Ends. A more detailed architectural description will be given in the `rfe_abstraction_ug`.

3.2 RFE Abstract 2.0 API

Modules

- [RFE Abstract 2.0 Constants](#)
Defines the API constants used by the RFE Abstract 2.0 API.
- [RFE Abstract 2.0 Interface](#)
RFE Abstract 2.0 specific API structures and functions.
- [RFE Abstract 2.0 - Low Level Constants](#)
Defines the low level constants used by the RFE Abstract 2.0 API.
- [RFE Abstract 2.0 - Low Level Interface](#)
RFE Abstract 2.0 specific low level structures and functions.
- [RFE Abstract 2.0 - Front-End configuration structure : BLOB](#)
RFE Abstract 2.0 constants for the specific structure for Front-End configuration.

3.2.1 Detailed Description

RFE Abstract 2.0 API is an API for radar application development. The characteristics of this API are:

- Does not have OS dependency as it uses no OS services for accessing a message buffer (SRAM).
- Developed and validated together with RFE-Fw to ensure compatibility with the RFE front end (TEF82XX Front End).
- Reduces complexity of application software development by taking care of all functional configurability of HW for the customer to realize radar module.
- Minimum set of abstract APIs for full RFE control by application.
- Supports complex use cases, for example two chirp sequences in one radar cycle, reconfiguration of chirp profile, phase coding, timing adjustment of frame start.
- Offer to the customer two level interface access :
 - high level access - AUTOSAR interface, the recommended level to use
 - low level access - direct access to the rfe_ like functions, which communicate with the RFE-Fw/M7-1 core

3.3 RFE Abstract 2.0 External Software Dependencies

Modules

- [RFE Abstract 2.0 - IPC Interface](#)
Defines the external function API required by RFE Abstract 2.0 for driver and firmware process communication.
- [RFE Abstract 2.0 - Peripheral Access Interface](#)
Defines the external function API required by RFE Abstract 2.0 to access different peripherals:

3.3.1 Detailed Description

RFE Abstract 2.0 External Software Dependencies are APIs used by the RFE Abstract 2.0 both Firmware and driver. These functions need to be implemented/provided by the user at application level. The characteristics of these APIs are:

- Provide mechanisms to communicate with S32R41-M7-1 core (RFE-Fw application).
- Provide abstract abstraction layer of general-purpose input/output (GPIO).
- Provide abstract abstraction layer of Serial Peripheral Interface (SPI) initialization.
- Provide abstract abstraction layer for system timers.
- Provide abstract abstraction layer for interrupts.

3.4 RFE Abstract 2.0 Firmware API

Modules

- [RFE Abstract 2.0 - Firmware Constants](#)
Defines the constants used to initialize the firmware finite state machine(FSM)
- [RFE Abstract 2.0 - Firmware Interface](#)
RFE Abstract 2.0 firmware finite state machine(FSM) specific structures and functions.

3.4.1 Detailed Description

RFE Abstract 2.0 Firmware Finit State Machine

- Firmware Finite State Machine is interfacing with application on one side and on the other it manages the RFE front end(TEF82XX Front End)

3.5 RFE Abstract 2.0 Constants

Defines the API constants used by the RFE Abstract 2.0 API.

Macros

- #define CDD_RFE_DEV_ERROR_DETECT STD_OFF
- #define CDD_RFE_VERSION_INFO_API STD_ON
- #define CDD_RFE_SINGLE_MANAGEMENT_THREADS STD_ON
- #define CDD_RFE_CFG_HALT_ON_ERROR STD_OFF
- #define CDD_RFE_E_UNRESPONSIVE 1u
- #define CDD_RFE_E_FUSA_ERROR 2u
- #define CDD_RFE_E_FW_ERROR 3u
- #define CDD_RFE_HALT_ON_ERROR
- #define CDD_RFE_ZERO (uint32)0U
- #define CDD_RFE_SID_INIT (0x00U)
- *API IDs.*
- #define CDD_RFE_SID_CONFIGURE (0x01U)
- #define CDD_RFE_SID_RADARCYCLESTART (0x02U)
- #define CDD_RFE_SID_RADARCYCLESTOP (0x03U)
- #define CDD_RFE_SID_GETSTATE (0x04U)
- #define CDD_RFE_SID_GETRADARCYCLECOUNT (0x05U)
- #define CDD_RFE_SID_GETFUSAERRORS (0x06U)
- #define CDD_RFE_SID_GETTIME (0x07U)
- #define CDD_RFE_SID_GETVERSION (0x08U)
- #define CDD_RFE_SID_MONITORREAD (0x09U)
- #define CDD_RFE_SID_GETNEXTRADARCYCLESTARTTIME (0x0AU)
- #define CDD_RFE_SID_SETNEXTRADARCYCLESTARTTIME (0x0BU)
- #define CDD_RFE_SID_UPDATEBEGIN (0x0CU)
- #define CDD_RFE_SID_UPDATEPARAM (0x0DU)
- #define CDD_RFE_SID_UPDATEDYNAMICTABLE (0x0EU)
- #define CDD_RFE_SID_UPDATEPUSH (0x13U)
- #define CDD_RFE_SID_TESTCONTINUOUSWAVETRANSMISSIONSTART (0x14U)
- #define CDD_RFE_SID_TESTCONTINUOUSWAVETRANSMISSIONSTOP (0x0FU)
- #define CDD_RFE_SID_TESTSETPARAM (0x10U)
- #define CDD_RFE_SID_GET_VERSION_INFO (0x11U)
- #define CDD_RFE_SID_GET_ERROR (0x12U)
- #define CDD_RFE_SID_INVALID (0xFFU)
- #define CDD_RFE_E_PARAM_POINTER ((uint8)0x01U)
- *API request called with an invalid parameter (Nullpointer or invalid parameter). rfe_error_api_invalidArgument← Value_e, rfe_error_api_cfgInvalidParameterSize_rfeDriverOnly_e,.*
- #define CDD_RFE_E_PARAM_SIZE ((uint8)0x02U)
- #define CDD_RFE_E_LIMIT_EXCEED ((uint8)0x03U)

- API request called with invalid parameter (invalid value). `rfe_error_api_invalidConfigurationSize_e`, `rfe_error_api_maxUpdateCountExceeded_e`, `rfe_error_api_noUpdatesBuffered_e`, `rfe_error_sharedData_invalidIndex_e`,.
- #define `CDD_RFE_E_REDUNDANCY_CHECK_FAILED` ((uint8)0x04U)
API request with Redundancy Check Error . `rfe_error_sharedData_redundancyCheckFailed_e`,.
- #define `CDD_RFE_E_SETUP_FAILED` ((uint8)0x05U)
API request with of RFE Driver Initialization failed. `rfe_error_api_unsynchronized_e`.
- #define `CDD_RFE_E_RESPONSE_ERROR` ((uint8)0x06U)
API request with invalid response, CRC and response length. `rfe_error_cmdInterface_invalidResponse_e`, `rfe_error_cmdInterface_invalidCrc_e`, `rfe_error_cmdInterface_invalidRspLength_e`,.
- #define `CDD_RFE_E_CMD_LENGTH` ((uint8)0x07U)
API request with Invalid Command Length. `rfe_error_cmdInterface_invalidCmdLength_e`,.
- #define `CDD_RFE_E_CMD_NOT_IMPLEMENTD` ((uint8)0x08U)
API request for a function not implemented in RFE-Fw side. `rfe_error_api_function_not_implemented_e`,.
- #define `CDD_RFE_VERSION_INFO_CHECK` 0u
- #define `CDD_RFE_CONFIGURE` 1u
- #define `CDD_RFE_START_EXECUTION` 2u
- #define `CDD_RFE_STOP_EXECUTION` 3u
- #define `CDD_RFE_GET_DATA` 4u
- #define `CDD_RFE_SET_DATA` 5u
- #define `REG_MAX_COUNT_TO_SEND` 58U
- #define `RFE_CC_REG_NUMBER` 68U
- #define `RFE_LDVS_REG_NUMBER` 46U
- #define `RFE_CSI2_REG_NUMBER` 71U
- #define `RFE_TE_REG_NUMBER` 71U
- #define `RFE_SC_REG_NUMBER` 70U
- #define `RFE_CAFRC_REG_NUMBER` 137U
- #define `RFE_ADC_REG_NUMBER` 118U
- #define `RFE_GB_REG_NUMBER` 46U
- #define `RFE_LOI_REG_NUMBER` 32U
- #define `RFE_MCLK_REG_NUMBER` 68U
- #define `RFE_RX_REG_NUMBER` 72U
- #define `RFE_TX_REG_NUMBER` 124U
- #define `RFE_SSBMOD_REG_NUMBER` 24U
- #define `RFE_GLDO_REG_NUMBER` 38U
- #define `RFE_OTP_REG_NUMBER` 19U
- #define `RFE_ISM_REG_NUMBER` 89U
- #define `RFE_OTP_TABLE_NUMBER` 64U
- #define `RFE_ATB_VOLTAGE_TABLE_NUMBER` 55U

Variables

- volatile bool `gsCddRfeLoopExit`

3.5.1 Detailed Description

Defines the API constants used by the RFE Abstract 2.0 API.

3.5.2 Macro Definition Documentation

3.5.2.1 CDD_RFE_CFG_HALT_ON_ERROR

```
#define CDD_RFE_CFG_HALT_ON_ERROR STD_OFF
```

Pre-processor switch to define the application behavior - to stop or not after an error is detected.

3.5.2.2 CDD_RFE_CONFIGURE

```
#define CDD_RFE_CONFIGURE 1u
```

Error at configuration time

3.5.2.3 CDD_RFE_DEV_ERROR_DETECT

```
#define CDD_RFE_DEV_ERROR_DETECT STD_OFF
```

The CRC computation can be done only using software method. This is a fact imposed by :

- the necessity to use a single 32 bits CRC polynomial for all CRC implementation
- the Configurator is based on java implementation is not able to use the same polynomial as the S32R41 hardware has implemented Pre-processor switch to enable/disable development error detection.

3.5.2.4 CDD_RFE_E_CMD_LENGTH

```
#define CDD_RFE_E_CMD_LENGTH ((uint8)0x07U)
```

API request with Invalid Command Length. `rfe_error_cmdInterface_invalidCmdLength_e,`

3.5.2.5 CDD_RFE_E_CMD_NOT_IMPLEMENTD

```
#define CDD_RFE_E_CMD_NOT_IMPLEMENTD ((uint8)0x08U)
```

API request for a function not implemented in RFE-Fw side. `rfe_error_api_function_not_implemented_e,`

3.5.2.6 CDD_RFE_E_FUSA_ERROR

```
#define CDD_RFE_E_FUSA_ERROR 2u
```

FuSa error reported

3.5.2.7 CDD_RFE_E_FW_ERROR

```
#define CDD_RFE_E_FW_ERROR 3u
```

Error reported by the firmware

3.5.2.8 CDD_RFE_E_LIMIT_EXCEED

```
#define CDD_RFE_E_LIMIT_EXCEED ((uint8)0x03U)
```

API request called with invalid parameter (invalid value). rfe_error_api_invalidConfigurationSize_e, rfe_error_api_maxUpdateCountExceeded_e, rfe_error_api_noUpdatesBuffered_e, rfe_error_sharedData_invalidIndex_e,.

3.5.2.9 CDD_RFE_E_PARAM_POINTER

```
#define CDD_RFE_E_PARAM_POINTER ((uint8)0x01U)
```

API request called with an invalid parameter (Nullpointer or invalid parameter). rfe_error_api_invalidArgumentValue_e, rfe_error_api_cfgInvalidParameterSize_rfeDriverOnly_e,.

3.5.2.10 CDD_RFE_E_PARAM_SIZE

```
#define CDD_RFE_E_PARAM_SIZE ((uint8)0x02U)
```

3.5.2.11 CDD_RFE_E_REDUNDANCY_CHECK_FAILED

```
#define CDD_RFE_E_REDUNDANCY_CHECK_FAILED ((uint8)0x04U)
```

API request with Redundancy Check Error . rfe_error_sharedData_redundancyCheckFailed_e,.

3.5.2.12 CDD_RFE_E_RESPONSE_ERROR

```
#define CDD_RFE_E_RESPONSE_ERROR ((uint8)0x06U)
```

API request with invalid response, CRC and response length. `rfe_error_cmdInterface_invalidResponse_e`, `rfe_error_cmdInterface_invalidCrc_e`, `rfe_error_cmdInterface_invalidRspLength_e`.

3.5.2.13 CDD_RFE_E_SETUP_FAILED

```
#define CDD_RFE_E_SETUP_FAILED ((uint8)0x05U)
```

API request with of RFE Driver Initialization failed. `rfe_error_api_unsynchronized_e`.

3.5.2.14 CDD_RFE_E_UNRESPONSIVE

```
#define CDD_RFE_E_UNRESPONSIVE 1u
```

The Time-Out get for Command Server response

3.5.2.15 CDD_RFE_GET_DATA

```
#define CDD_RFE_GET_DATA 4u
```

Error when ask for data from RFE-Fw

3.5.2.16 CDD_RFE_HALT_ON_ERROR

```
#define CDD_RFE_HALT_ON_ERROR
```

Value:

```
while (gsCddRfeLoopExit != true) \
{ \
; /* empty loop */ \
}
```

Infinite loop if error detected and the option is to detect development errors. To exit the loop at debug time is possible to use `gsCddRfeLoopExit` variable

3.5.2.17 CDD_RFE_SET_DATA

```
#define CDD_RFE_SET_DATA 5u
```

Error when ask for data set

3.5.2.18 CDD_RFE_SID_CONFIGURE

```
#define CDD_RFE_SID_CONFIGURE (0x01U)
```

3.5.2.19 CDD_RFE_SID_GET_ERROR

```
#define CDD_RFE_SID_GET_ERROR (0x12U)
```

3.5.2.20 CDD_RFE_SID_GET_VERSION_INFO

```
#define CDD_RFE_SID_GET_VERSION_INFO (0x11U)
```

3.5.2.21 CDD_RFE_SID_GETFUSAERRORS

```
#define CDD_RFE_SID_GETFUSAERRORS (0x06U)
```

3.5.2.22 CDD_RFE_SID_GETNEXTRADARCYCLESTARTTIME

```
#define CDD_RFE_SID_GETNEXTRADARCYCLESTARTTIME (0x0AU)
```

3.5.2.23 CDD_RFE_SID_GETRADARCYCLECOUNT

```
#define CDD_RFE_SID_GETRADARCYCLECOUNT (0x05U)
```


3.5.2.24 CDD_RFE_SID_GETSTATE

```
#define CDD_RFE_SID_GETSTATE (0x04U)
```

3.5.2.25 CDD_RFE_SID_GETTIME

```
#define CDD_RFE_SID_GETTIME (0x07U)
```

3.5.2.26 CDD_RFE_SID_GETVERSION

```
#define CDD_RFE_SID_GETVERSION (0x08U)
```

3.5.2.27 CDD_RFE_SID_INIT

```
#define CDD_RFE_SID_INIT (0x00U)
```

API IDs.

3.5.2.28 CDD_RFE_SID_INVALID

```
#define CDD_RFE_SID_INVALID (0xFFU)
```

3.5.2.29 CDD_RFE_SID_MONITORREAD

```
#define CDD_RFE_SID_MONITORREAD (0x09U)
```

3.5.2.30 CDD_RFE_SID_RADARCYCLESTART

```
#define CDD_RFE_SID_RADARCYCLESTART (0x02U)
```

3.5.2.31 CDD_RFE_SID_RADARCYCLESTOP

```
#define CDD_RFE_SID_RADARCYCLESTOP (0x03U)
```

3.5.2.32 CDD_RFE_SID_SETNEXTRADARCYCLESTARTTIME

```
#define CDD_RFE_SID_SETNEXTRADARCYCLESTARTTIME (0x0BU)
```

3.5.2.33 CDD_RFE_SID_TESTCONTINUOUSWAVETRANSMISSIONSTART

```
#define CDD_RFE_SID_TESTCONTINUOUSWAVETRANSMISSIONSTART (0x14U)
```

3.5.2.34 CDD_RFE_SID_TESTCONTINUOUSWAVETRANSMISSIONSTOP

```
#define CDD_RFE_SID_TESTCONTINUOUSWAVETRANSMISSIONSTOP (0x0FU)
```

3.5.2.35 CDD_RFE_SID_TESTSETPARAM

```
#define CDD_RFE_SID_TESTSETPARAM (0x10U)
```

3.5.2.36 CDD_RFE_SID_UPDATEBEGIN

```
#define CDD_RFE_SID_UPDATEBEGIN (0x0CU)
```

3.5.2.37 CDD_RFE_SID_UPDATEDYNAMICTABLE

```
#define CDD_RFE_SID_UPDATEDYNAMICTABLE (0x0EU)
```

3.5.2.38 CDD_RFE_SID_UPDATEPARAM

```
#define CDD_RFE_SID_UPDATEPARAM (0x0DU)
```

3.5.2.39 CDD_RFE_SID_UPDATEPUSH

```
#define CDD_RFE_SID_UPDATEPUSH (0x13U)
```

3.5.2.40 CDD_RFE_SINGLE_MANAGEMENT_THREADS

```
#define CDD_RFE_SINGLE_MANAGEMENT_THREADS STD_ON
```

Pre-processor switch to define single management thread or multiple management threads. If single thread (which is the normal approach) - there are no necessary exclusive areas for the driver

3.5.2.41 CDD_RFE_START_EXECUTION

```
#define CDD_RFE_START_EXECUTION 2u
```

Error when ask for execution start

3.5.2.42 CDD_RFE_STOP_EXECUTION

```
#define CDD_RFE_STOP_EXECUTION 3u
```

Error when ask for execution start

3.5.2.43 CDD_RFE_VERSION_INFO_API

```
#define CDD_RFE_VERSION_INFO_API STD_ON
```

Pre-processor switch to enable/disable version info report.

3.5.2.44 CDD_RFE_VERSION_INFO_CHECK

```
#define CDD_RFE_VERSION_INFO_CHECK 0u
```

Error at VersionInfo call

3.5.2.45 CDD_RFE_ZERO

```
#define CDD_RFE_ZERO (uint32)0U
```

Zero value

3.5.2.46 REG_MAX_COUNT_TO_SEND

```
#define REG_MAX_COUNT_TO_SEND 58U
```

3.5.2.47 RFE_ADC_REG_NUMBER

```
#define RFE_ADC_REG_NUMBER 118U
```

Number ADC12 (0x07) / ADC34 (0x08) registers.

3.5.2.48 RFE_ATB_VOLTAGE_TABLE_NUMBER

```
#define RFE_ATB_VOLTAGE_TABLE_NUMBER 55U
```

Number ATB voltage Table

3.5.2.49 RFE_CAFC_REG_NUMBER

```
#define RFE_CAFC_REG_NUMBER 137U
```

Number Chirp AFC (0x06) registers.

3.5.2.50 RFE_CC_REG_NUMBER

```
#define RFE_CC_REG_NUMBER 68U
```

Number Central Control (0x00) registers.

3.5.2.51 RFE_CSI2_REG_NUMBER

```
#define RFE_CSI2_REG_NUMBER 71U
```

Number CSI2 Module (0x02) registers.

3.5.2.52 RFE_GB_REG_NUMBER

```
#define RFE_GB_REG_NUMBER 46U
```

Number Global Bias (0x09) registers.

3.5.2.53 RFE_GLDO_REG_NUMBER

```
#define RFE_GLDO_REG_NUMBER 38U
```

Number Global LDO (0x15) registers.

3.5.2.54 RFE_ISM_REG_NUMBER

```
#define RFE_ISM_REG_NUMBER 89U
```

Number ISM Module (0x18) registers.

3.5.2.55 RFE_LDVS_REG_NUMBER

```
#define RFE_LDVS_REG_NUMBER 46U
```

Number LVDS Module (0x01) registers.

3.5.2.56 RFE_LOI_REG_NUMBER

```
#define RFE_LOI_REG_NUMBER 32U
```

Number LO Interface (0x0A) registers.

3.5.2.57 RFE_MCLK_REG_NUMBER

```
#define RFE_MCLK_REG_NUMBER 68U
```

Number Master Clock (0x0C) registers.

3.5.2.58 RFE_OTP_REG_NUMBER

```
#define RFE_OTP_REG_NUMBER 19U
```

Number OTP (0x17) registers.

3.5.2.59 RFE_OTP_TABLE_NUMBER

```
#define RFE_OTP_TABLE_NUMBER 64U
```

Number ISM Table

3.5.2.60 RFE_RX_REG_NUMBER

```
#define RFE_RX_REG_NUMBER 72U
```

Number RX1 (0x0D) / RX2 (0x0E) / RX3 (0x0F) / RX4 (0x10) registers.

3.5.2.61 RFE_SC_REG_NUMBER

```
#define RFE_SC_REG_NUMBER 70U
```

Number Sweep Control (0x05) registers.

3.5.2.62 RFE_SSBMOD_REG_NUMBER

```
#define RFE_SSBMOD_REG_NUMBER 24U
```

Number SSBMOD (0x14) registers.

3.5.2.63 RFE_TE_REG_NUMBER

```
#define RFE_TE_REG_NUMBER 71U
```

Number Timing Engine (0x04) registers.

3.5.2.64 RFE_TX_REG_NUMBER

```
#define RFE_TX_REG_NUMBER 124U
```

Number TX1 (0x11) / TX2 (0x12) / TX3 (0x13) registers.

3.5.3 Variable Documentation

3.5.3.1 gsCddRfeLoopExit

```
volatile bool gsCddRfeLoopExit [extern]
```

Global variable to allow exit from [CDD_RFE_HALT_ON_ERROR](#) loop

3.6 RFE Abstract 2.0 Interface

RFE Abstract 2.0 specific API structures and functions.

Data Structures

- struct [rfe_registerDumpValues_t](#)
- struct [ATB_ADCVoltages_t](#)
- struct [ATB_ATBVoltages_t](#)
- struct [ATB_CAFCVoltages_t](#)
- struct [ATB_GBIASVoltages_t](#)
- struct [ATB_GLDOVoltages_t](#)
- struct [ATB_LOIVoltages_t](#)
- struct [ATB_MCLKVoltages_t](#)
- struct [ATB_MIPIVoltages_t](#)
- struct [ATB_RXVoltages_t](#)
- struct [ATB_SSBVoltages_t](#)
- struct [ATB_TXVoltages_t](#)
- struct [rfe_tef82xxVoltages_t](#)

Typedefs

- typedef [rfe_sysMemAddress_t](#) Cdd_RfeSysMemAddressType
- typedef [rfe_fuSaFault_t](#) Cdd_RfeFusaFaultType
- typedef [rfe_error_t](#) Cdd_RfeErrorType
- typedef [rfe_monitorSelect_t](#) Cdd_RfeMonitorSelectType
- typedef [rfe_monitorValues_t](#) Cdd_RfeMonitorValuesType
- typedef [rfeCfg_section_t](#) Cdd_RfeCfgSectionType
- typedef [rfeCfg_param_t](#) Cdd_RfeCfgParamType
- typedef [rfe_paramUpdate_t](#) Cdd_RfeParamUpdateType
- typedef [rfe_dynParamsSelect_t](#) Cdd_RfeDynParamsSelectType
- typedef [rfe_chirpProfileIndex_t](#) Cdd_RfeChirpProfileIndexType
- typedef [rfe_testParam_t](#) Cdd_RfeTestParamType
- typedef [rfe_radarCycleCount_t](#) Cdd_RfeRadarCycleCountType
- typedef [rfe_state_t](#) Cdd_RfeStateType
- typedef [rfe_version_t](#) Cdd_RfeVersionType
- typedef [rfeCfg_rxBistReferenceData_t](#) Cdd_RfeRxBistZeroHourRefDataType
- typedef [rfe_coreId_t](#) Cdd_RfeCoreIdDataType
- typedef [rfe_eventsIRQ_t](#) Cdd_RfeEventsIRQDataType
- typedef [rfe_apilIRQ_t](#) Cdd_RfeApiIRQDataType
- typedef [rfe_apiWaitForInterrupt_f](#) Cdd_RfeApiWaitForInterruptDataType

Enumerations

- enum [rfe_moduleDumpSelect_t](#) {
[rfe_registerDump_CC](#) = 0x00000001UL , [rfe_registerDump_LDVS](#) = 0x00000002UL , [rfe_registerDump_CSI2](#)
= 0x00000004UL , [rfe_registerDump_TE](#) = 0x00000010UL ,
[rfe_registerDump_SC](#) = 0x00000020UL , [rfe_registerDump_CAFC](#) = 0x00000040UL , [rfe_registerDump_ADC12](#)
= 0x00000080UL , [rfe_registerDump_ADC34](#) = 0x00000100UL ,
[rfe_registerDump_GB](#) = 0x00000200UL , [rfe_registerDump_LOI](#) = 0x00000400UL , [rfe_registerDump_MCLK](#)
= 0x00001000UL , [rfe_registerDump_RX1](#) = 0x00002000UL ,
[rfe_registerDump_RX2](#) = 0x00004000UL , [rfe_registerDump_RX3](#) = 0x00008000UL , [rfe_registerDump_RX4](#)
= 0x00010000UL , [rfe_registerDump_TX1](#) = 0x00020000UL ,
[rfe_registerDump_TX2](#) = 0x00040000UL , [rfe_registerDump_TX3](#) = 0x00080000UL , [rfe_registerDump_SSBMOD](#)
= 0x00100000UL , [rfe_registerDump_GLDO](#) = 0x00200000UL ,
[rfe_registerDump_OTP](#) = 0x00800000UL , [rfe_registerDump_ISM](#) = 0x01000000UL , [rfe_registerDump_OTP_TABLE](#)
= 0x02000000UL , [rfe_registerDump_ATB_VOLTAGE_READOUT](#) = 0x04000000UL }

- enum `rfe_bcdModuleId_t` {
`rfe_bcd_cc_e` = 0x00U , `rfe_bcd_lvds_e` = 0x01U , `rfe_bcd_csi2_e` = 0x02U , `rfe_bcd_te_e` = 0x04U ,
`rfe_bcd_sc_e` = 0x05U , `rfe_bcd_cafc_e` = 0x06U , `rfe_bcd_adc12_e` = 0x07U , `rfe_bcd_adc34_e` = 0x08U ,
`rfe_bcd_gb_e` = 0x09U , `rfe_bcd_loi_e` = 0x0aU , `rfe_bcd_mclk_e` = 0x0cU , `rfe_bcd_rx1_e` = 0x0dU ,
`rfe_bcd_rx2_e` = 0x0eU , `rfe_bcd_rx3_e` = 0x0fU , `rfe_bcd_rx4_e` = 0x10U , `rfe_bcd_tx1_e` = 0x11U ,
`rfe_bcd_tx2_e` = 0x12U , `rfe_bcd_tx3_e` = 0x13U , `rfe_bcd_ssbmod_e` = 0x14U , `rfe_bcd_gldo_e` = 0x15U ,
`rfe_bcd_otp_e` = 0x17U , `rfe_bcd_ism_e` = 0x18U , `rfe_bcd_otp_table_e` = 0x19U , `rfe_bcd_atb_voltage_e` =
0x1aU }
- enum `CDD_RfeFrontEndHierarchyType` {
`CDD_RFE_FRONT_END_LEADER` , `CDD_RFE_FRONT_END_FOLLOWER_0` , `CDD_RFE_FRONT_END_FOLLOWER_1`
, `CDD_RFE_FRONT_END_FOLLOWER_2` ,
`CDD_RFE_FRONT_END_FOLLOWER_MAX` }
- enum `CDD_RfeFrontEndIdMaskType` { `CDD_RFE_FRONT_END_MASK_LEADER` = 0x01 , `CDD_RFE_FRONT_END_MASK`
= 0x02 , `CDD_RFE_FRONT_END_MASK_FOLLOWER_1` = 0x04 , `CDD_RFE_FRONT_END_MASK_FOLLOWER_2`
= 0x08 }

Functions

- void `CDD_Rfe_Init` (void)
This function synchronizes the Rfe_Driver with the RFE-Fw.
- Std_ReturnType `CDD_Rfe_Configure` (uint8 *configPtr, const `Cdd_RfeSysMemAddressType` dynamic↔
TableAddress)
This function fully configures the RFE-Fw.
- Std_ReturnType `CDD_Rfe_RadarCycleStart` (uint16 radarCycleCount, boolean isScheduled, uint32 start↔
Time)
This function starts the configured radar cycles.
- uint16 `CDD_Rfe_RadarCycleStop` (void)
This function stops the active radar cycles.
- `Cdd_RfeStateType` `CDD_Rfe_GetState` (void)
This function returns the RFE-Fw state.
- `Cdd_RfeRadarCycleCountType` `CDD_Rfe_GetRadarCycleCount` (void)
This function returns the radar cycle and chirp sequence count.
- Std_ReturnType `CDD_Rfe_GetFuSaFaults` (uint8 *fuSaR1R2FaultListPtr)
This function provides the RFE-Fw FuSa faults which are set in Front End FCCU.
- Std_ReturnType `CDD_Rfe_GetFuSaFaultStatistics` (uint16 *r1FaultPromotedToR2Ptr, `Cdd_RfeRadarCycleCountType`
*radarCycleCountPtr, uint8 *fuSaR1FaultCountListPtr)
This function provides the fault statistics for R1 faults.
- Std_ReturnType `CDD_Rfe_GetBistZeroHourReferenceData` (`Cdd_RfeRxBistZeroHourRefDataType` *rx↔
ReferenceDataPtr)
This function returns RX BIST measurement details to be used as zero hour reference data.
- uint32 `CDD_Rfe_GetTime` (void)
This function returns the RFE-Fw time.
- `Cdd_RfeVersionType` `CDD_Rfe_SwHwGetVersion` (void)
This function returns version and variant details of the RFE-Fw.
- `Cdd_RfeRadarCycleCountType` `CDD_Rfe_MonitorRead` (`Cdd_RfeMonitorSelectType` monitorSelect,
`Cdd_RfeMonitorValuesType` *monitorValuesPtr)
This function will read-out one or more RFE monitors.
- uint32 `CDD_Rfe_GetNextRadarCycleStartTime` (uint16 *radarCycleIndexPtr)
This function returns the scheduled absolute start time of the next radar cycle.
- uint16 `CDD_Rfe_SetNextRadarCycleStartTime` (uint32 startTime)
This function updates the start time of the next radar cycle.
- void `CDD_Rfe_UpdateBegin` (void)

- This function begins a RFE-Fw configuration update operation.*
- Std_ReturnType [CDD_Rfe_UpdateParam](#) ([Cdd_RfeCfgSectionType](#) section, [Cdd_RfeParamUpdateType](#) param, uint32 value)
- This functions buffers one of multiple configuration parameter set operations.*
- Std_ReturnType [CDD_Rfe_UpdateDynamicTable](#) (uint8 dynamicTableIndex, [Cdd_RfeDynParamsSelectType](#) paramsSelect)
- This functions buffers a dynamic table update.*
- uint16 [CDD_Rfe_UpdatePush](#) (void)
- This function pushes all the buffered update operations jointly to RFE-Fw.*
- Std_ReturnType [CDD_Rfe_TestContinuousWaveTransmissionStart](#) ([Cdd_RfeChirpProfileIndexType](#) profileIndex)
- This function start continuous wave transmission.*
- Std_ReturnType [CDD_Rfe_TestContinuousWaveTransmissionStop](#) (void)
- This function stops continuous wave transmission.*
- Std_ReturnType [CDD_Rfe_TestSetParam](#) ([Cdd_RfeTestParamType](#) testParam, uint32 value)
- This function sets a test parameter.*
- [Cdd_RfeErrorType](#) [CDD_Rfe_GetError](#) (void)
- This function provides the error occurred.*
- Std_ReturnType [CDD_Rfe_TestGetInternalError](#) (void)
- This function fetches the internal software error code from M7_1 and stores it.*
- Std_ReturnType [CDD_Rfe_GetBlobValue](#) (uint8 *configPtr, [Cdd_RfeCfgSectionType](#) section, [Cdd_RfeCfgParamType](#) param, uint32 *valuePtr)
- Function to get the specified value from the blob.*
- Std_ReturnType [CDD_Rfe_SetBlobValue](#) (uint8 *configPtr, [Cdd_RfeCfgSectionType](#) section, [Cdd_RfeCfgParamType](#) param, uint32 value)
- Function to get the specified value from the blob.*
- void [CDD_Rfe_GetVersionInfo](#) (Std_VersionInfoType *versionInfoPtr)
- Service for passing the version information through a pointer *versionInfoPtr.*
- Std_ReturnType [CDD_Rfe_SetFrontEnd](#) ([CDD_RfeFrontEndHierarchyType](#) frontEndPos)
- Function to change the associated Front-End "subject" in a cascaded setup.*
- Std_ReturnType [CDD_Rfe_GetFrontEnd](#) ([CDD_RfeFrontEndHierarchyType](#) *frontEndPosPtr)
- Function to get the current associated Front-End "subject" in the RFE-Fw, in a cascaded setup.*
- Std_ReturnType [CDD_Rfe_GetRegisterDump](#) ([CDD_RfeFrontEndIdMaskType](#) frontEndSelect, [rfe_moduleDumpSelect_t](#) moduleSelect, [rfe_registerDumpValues_t](#) *dumpArrayPtr, uint32_t *dumpArrayLenPtr)
- Debug function to get specific Front-End register values.*
- Std_ReturnType [CDD_Rfe_ConfigureInterrupt](#) ([Cdd_RfeCoreIdDataType](#) coreId, [Cdd_RfeEventsIRQDataType](#) events, [Cdd_RfeApiIRQDataType](#) rfeAPIs, [Cdd_RfeApiWaitForInterruptDataType](#) apiWaitForInterrupt)
- This function configure the RFE firmware interrupt.*

3.6.1 Detailed Description

RFE Abstract 2.0 specific API structures and functions.

Note

The RFE API interface provides three kinds of functions, with different Front-End usage:

- **Functions which apply globally to the RFE-Firmware software.**
- **Functions which apply to a single, selected Radar Front-End only.** For a Cascaded solution, these functions need to be preceded by a call to apply to the Radar Front-End previously selected by the call to [CDD_Rfe_SetFrontEnd](#). Normally, they need to be repeated for each Radar Front-End.

- **Functions which can be applied to multiple Front-Ends simultaneously.** These functions have a first argument `CDD_RfeFrontEndIdMaskType < frontendSelect >`, which defines a mask to select one or more Radar Front-Ends.
 - `CDD_Rfe_ConfigureInterrupt`
 - `CDD_Rfe_RadarCycleStart`
 - `CDD_Rfe_RadarCycleStop`
 - `CDD_Rfe_SwHwGetVersion`
 - `CDD_Rfe_GetVersionInfo`
 - `CDD_Rfe_GetError`
 - `CDD_Rfe_GetState`
 - `CDD_Rfe_GetTime`
 - `CDD_Rfe_GetRadarCycleCount`
 - `CDD_Rfe_GetNextRadarCycleStartTime`
 - `CDD_Rfe_TestContinuousWaveTransmissionStart`
 - `CDD_Rfe_TestContinuousWaveTransmissionStop`
 - `CDD_Rfe_TestGetInternalError`
- Functions which apply globally to the RFE-Firmware software.
 - `CDD_Rfe_ConfigureInterrupt`
 - `CDD_Rfe_RadarCycleStart`
 - `CDD_Rfe_RadarCycleStop`
 - `CDD_Rfe_SwHwGetVersion`
 - `CDD_Rfe_GetVersionInfo`
 - `CDD_Rfe_GetError`
 - `CDD_Rfe_GetState`
 - `CDD_Rfe_GetTime`
 - `CDD_Rfe_GetRadarCycleCount`
 - `CDD_Rfe_GetNextRadarCycleStartTime`
 - `CDD_Rfe_TestContinuousWaveTransmissionStart`
 - `CDD_Rfe_TestContinuousWaveTransmissionStop`
 - `CDD_Rfe_TestGetInternalError`
- Functions which apply to a single, selected Radar Front-End only.
 - `CDD_Rfe_Configure`
 - `CDD_Rfe_GetFuSaFaults`
 - `CDD_Rfe_GetBistZeroHourReferenceData`
 - `CDD_Rfe_MonitorRead`
 - `CDD_Rfe_UpdateBegin`
 - `CDD_Rfe_UpdateParam`
 - `CDD_Rfe_UpdateDynamicTable`
 - `CDD_Rfe_UpdatePush`
- Functions applicable to multiple Front-Ends simultaneously:
 - `CDD_Rfe_GetRegisterDump`

3.6.2 Typedef Documentation

3.6.2.1 Cdd_RfeApiIRQDataType

```
typedef rfe_apiIRQ_t Cdd_RfeApiIRQDataType
```

This type defines which RFE APIs shall trigger an IRQ from RFE firmware upon completion

3.6.2.2 Cdd_RfeApiWaitForInterruptDataType

```
typedef rfe_apiWaitForInterrupt_f Cdd_RfeApiWaitForInterruptDataType
```

This type defines RFE API wait for interrupt function pointer.

3.6.2.3 Cdd_RfeCfgParamType

```
typedef rfeCfg_param_t Cdd_RfeCfgParamType
```

The defined sections.

3.6.2.4 Cdd_RfeCfgSectionType

```
typedef rfeCfg_section_t Cdd_RfeCfgSectionType
```

The defined sections.

3.6.2.5 Cdd_RfeChirpProfileIndexType

```
typedef rfe_chirpProfileIndex_t Cdd_RfeChirpProfileIndexType
```

This enumerated type defines the chirp profile indices.

3.6.2.6 Cdd_RfeCoreIdDataType

```
typedef rfe_coreId_t Cdd_RfeCoreIdDataType
```

This enumerated type defines which core will receive IRQs from RFE firmware

Note

Core Id 0: appM7 core Core Id 1: a53 core

3.6.2.7 Cdd_RfeDynParamsSelectType

```
typedef rfe_dynParamsSelect_t Cdd_RfeDynParamsSelectType
```

This type defines a selection of dynamic parameters.

3.6.2.8 Cdd_RfeErrorType

```
typedef rfe_error_t Cdd_RfeErrorType
```

API error codes.

3.6.2.9 Cdd_RfeEventsIRQDataType

```
typedef rfe_eventsIRQ_t Cdd_RfeEventsIRQDataType
```

This type defines which events shall trigger an IRQ from RFE firmware

3.6.2.10 Cdd_RfeFusaFaultType

```
typedef rfe_fuSaFault_t Cdd_RfeFusaFaultType
```

This enumeration data type is used for FuSa faults.

- In RFE Configuration, the enumeration must be used as bitwise index to mask or unmask the corresponding fault bit.
- In [CDD_Rfe_GetFuSaFaults\(\)](#), the enumeration must be used as bitwise index to get the fault status for the corresponding fault.
- In [CDD_Rfe_GetFuSaFaultStatistics\(\)](#), the enumeration must be used as bitwise index to get the fault count for the corresponding R1 fault.

3.6.2.11 Cdd_RfeMonitorSelectType

```
typedef rfe_monitorSelect_t Cdd_RfeMonitorSelectType
```

This types defines a selection of RFE monitors.

3.6.2.12 Cdd_RfeMonitorValuesType

```
typedef rfe_monitorValues_t Cdd_RfeMonitorValuesType
```

This structure is a container for all RFE monitors to be read via [rfe_monitorRead\(\)](#).

3.6.2.13 Cdd_RfeParamUpdateType

```
typedef rfe_paramUpdate_t Cdd_RfeParamUpdateType
```

This enumerated types defines the RFE parameters that can be updated.

3.6.2.14 Cdd_RfeRadarCycleCountType

```
typedef rfe_radarCycleCount_t Cdd_RfeRadarCycleCountType
```

This structure contains the radar cycle and chirp sequence count. It can be retrieved via [rfe_getRadarCycleCount\(\)](#).

3.6.2.15 Cdd_RfeRxBistZeroHourRefDataType

```
typedef rfeCfg_rxBistReferenceData_t Cdd_RfeRxBistZeroHourRefDataType
```

This structure contains the zero hour reference data for rxPhase difference, Gain difference It can be retrieved via [CDD_Rfe_GetBistZeroHourReferenceData\(\)](#).

3.6.2.16 Cdd_RfeStateType

```
typedef rfe_state_t Cdd_RfeStateType
```

This type defines the RFE states. RFE state can be retrieved via [rfe_getState\(\)](#).

3.6.2.17 Cdd_RfeSysMemAddressType

```
typedef rfe_sysMemAddress_t Cdd_RfeSysMemAddressType
```

This type defines an address to system memory.

3.6.2.18 Cdd_RfeTestParamType

```
typedef rfe_testParam_t Cdd_RfeTestParamType
```

This enumerated type defines the RFE test parameters. Used by [rfe_testSetParam\(\)](#).

3.6.2.19 Cdd_RfeVersionType

```
typedef rfe_version_t Cdd_RfeVersionType
```

This structure contains the version and variant details of the rfe:

- Hardware and firmware variants

3.6.3 Enumeration Type Documentation

3.6.3.1 CDD_RfeFrontEndHierarchyType

```
enum CDD_RfeFrontEndHierarchyType
```

This enumeration must be used to define the Front-End usage. It must be used only for 'cascaded' setup, for 'stand-alone' Front-End it is meaningless.

Enumerator

CDD_RFE_FRONT_END_LEADER	
CDD_RFE_FRONT_END_FOLLOWER_0	
CDD_RFE_FRONT_END_FOLLOWER_1	
CDD_RFE_FRONT_END_FOLLOWER_2	
CDD_RFE_FRONT_END_FOLLOWER_MAX	

3.6.3.2 CDD_RfeFrontEndIdMaskType

enum `CDD_RfeFrontEndIdMaskType`

This enumeration must be used to define the Front-End usage if the Front-End is defined as mask and not as enum. It must be used only for 'cascaded' setup, for 'stand-alone' Front-End it is meaningless. This approach is allowing to define more than one Front-End using a single value (as a sum of masks).

Enumerator

CDD_RFE_FRONT_END_MASK_LEADER	
CDD_RFE_FRONT_END_MASK_FOLLOWER↔ _0	
CDD_RFE_FRONT_END_MASK_FOLLOWER↔ _1	
CDD_RFE_FRONT_END_MASK_FOLLOWER↔ _2	

3.6.3.3 rfe_bcdModuleId_t

enum `rfe_bcdModuleId_t`

This type defines the BCD module Id.

Enumerator

rfe_bcd_cc_e	
rfe_bcd_lvds_e	
rfe_bcd_csi2_e	
rfe_bcd_te_e	
rfe_bcd_sc_e	
rfe_bcd_cafc_e	
rfe_bcd_adc12_e	
rfe_bcd_adc34_e	
rfe_bcd_gb_e	
rfe_bcd_loi_e	
rfe_bcd_mclk_e	
rfe_bcd_rx1_e	
rfe_bcd_rx2_e	
rfe_bcd_rx3_e	
rfe_bcd_rx4_e	
rfe_bcd_tx1_e	
rfe_bcd_tx2_e	
rfe_bcd_tx3_e	
rfe_bcd_ssbmod_e	
rfe_bcd_gldo_e	

Enumerator

rfe_bcd_otp_e	
rfe_bcd_ism_e	
rfe_bcd_otp_table_e	
rfe_bcd_atb_voltage_e	

3.6.3.4 rfe_moduleDumpSelect_t

```
enum rfe_moduleDumpSelect_t
```

This types defines a selection of BBC register dump.

Enumerator

rfe_registerDump_CC	
rfe_registerDump_LDVS	
rfe_registerDump_CSI2	
rfe_registerDump_TE	
rfe_registerDump_SC	
rfe_registerDump_CAFD	
rfe_registerDump_ADC12	
rfe_registerDump_ADC34	
rfe_registerDump_GB	
rfe_registerDump_LOI	
rfe_registerDump_MCLK	
rfe_registerDump_RX1	
rfe_registerDump_RX2	
rfe_registerDump_RX3	
rfe_registerDump_RX4	
rfe_registerDump_TX1	
rfe_registerDump_TX2	
rfe_registerDump_TX3	
rfe_registerDump_SSBMOD	
rfe_registerDump_GLDO	
rfe_registerDump_OTP	
rfe_registerDump_ISM	
rfe_registerDump_OTP_TABLE	
rfe_registerDump_ATB_VOLTAGE_READOUT	

3.6.4 Function Documentation

3.6.4.1 CDD_Rfe_Configure()

```
Std_ReturnType CDD_Rfe_Configure (
    uint8 * configPtr,
    const Cdd_RfeSysMemAddressType dynamicTableAddress )
```

This function fully configures the RFE-Fw.

This function copies the provided configuration and dynamic tables to the RFE-Fw and verifies them, after which the API function returns. Asynchronously, the RFE-Fw configures the Radar Front End (TEF82XX for now) and performs the configuration calibrations. This function returns after RFE-Fw processed the API call and configuration. This function must be called for each Front-End having as parameters the appropriate blob and dynamic table(s).

This function can sets one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- [rfe_error_api_invalidArgumentValue_e](#)
- [rfe_error_api_invalidConfigurationSize_e](#)
- [rfe_error_api_invalidConfigurationParameterValue_e](#)
- [rfe_error_api_invalidDynamicTableSize_e](#)
- [rfe_error_api_invalidDynamicTableValue_e](#)
- [rfe_error_api_invalidRadarCycleTimeSchedule_e](#)
- Any [rfe_error_api_configParamCrossCheck_dwellSettleTimeTooShort_e](#) ... [rfe_error_api_configParamCrossCheck_unconfigured_e](#)
- Any [rfe_error_cmdInterface_invalidCmdId_e](#) ... [rfe_error_cmdInterface_invalidConfigCmdSequencing_e](#).
- [rfe_error_sharedData_redundancyCheckFailed_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Precondition

The RFE-Fw must be in [rfe_state_initialized_e](#) state, or in [rfe_state_configured_e](#) state in case of full reconfiguration.

Parameters

in	<i>configPtr</i>	- Pointer to Configuration Data Memory (see RFE Abstract 2.0 - Front-End configuration structure : BLOB)
in	<i>dynamicTableAddress</i>	- Address of the dynamic tables in system memory. The dynamic tables created by the RFE configuration generator need to be loaded or copied to this address in system memory before calling this API function. Set to 0 when dynamic table is not used.

Returns

Std_ReturnType - E_OK / E_NOT_OK

Postcondition

RFE-Fw state becomes `rfe_state_busy_e` and RFE-Fw performs configuration and configuration calibration asynchronously. RFE-Fw state becomes `rfe_state_configured_e` only after all Front-End(s) are configured.

3.6.4.2 CDD_Rfe_ConfigureInterrupt()

```
Std_ReturnType CDD_Rfe_ConfigureInterrupt (
    Cdd_RfeCoreIdDataType coreId,
    Cdd_RfeEventsIRQDataType events,
    Cdd_RfeApiIRQDataType rfeAPIs,
    Cdd_RfeApiWaitForInterruptDataType apiWaitForInterrupt )
```

This function configure the RFE firmware interrupt.

`rfe_configureInterrupt()` configures which conditions will raise an IRQ to `coreId`, these conditions are:

- RFE state changes
- Radar Cycle count changes
- Chirp Sequence count changes
- Specific RFE API responses from the rfe firmware

The `apiWaitForInterrupt` function will be called while the driver is waiting for an RFE API response, this function shall return whenever a RFE firmware interrupt is received.

This is a user registered call back function where wait for interrupt or functionality to give control to Operating System can be implemented as per customer need.

This function may be called multiple times on a single RFE API call. During this call only APIs that do not interact with the firmware are allowed.

Note

`apiWaitForInterrupt` IS NOT an interrupt handler, the application code is responsible for setting up and handle incoming RFE interrupts.

Parameters

in	<i>coreId</i>	- Which core should the RFE interrupt be sent.
in	<i>events</i>	- Bitmask of events that will trigger an interrupt.
in	<i>rfeAPIs</i>	- Bitmask of RFE-APIs responses that will trigger an interrupt.
in	<i>apiWaitForInterrupt</i>	- API wait for interrupt function pointer. This is a user registered call back function where wait for interrupt or functionality to give control to Operating System can be implemented as per customer need.

Returns

Std_ReturnType

3.6.4.3 CDD_Rfe_GetBistZeroHourReferenceData()

```
Std_ReturnType CDD_Rfe_GetBistZeroHourReferenceData (
    Cdd_RfeRxBistZeroHourRefDataType * rxReferenceDataPtr )
```

This function returns RX BIST measurement details to be used as zero hour reference data.

The RFE-Fw performs the RX phase difference and RX gain difference measurements upon this function call. The measured data is provided along with information on the frequency, power and RX test tone configuration (refer [rfeCfg_param_monitorAndSafety_injectTestTone_e](#)) used for this measurement. This data must be stored by Application and be used as zero hour reference (to be passed via RFE-Fw configuration BLOB, refer [rfeCfg_section_monitorAndSafety_e](#)).

This function can return one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_rfeFuSaFault_e](#)

Precondition

The RFE-Fw must be in [rfe_state_configured_e](#).

Parameters

out	<i>rxReferenceDataPtr</i>	- pointer to structure containing zero hour RX bist reference data.
-----	---------------------------	---

Returns

Std_ReturnType - E_OK / E_NOT_OK

Remarks

- The configuration that changed due to execution of [CDD_Rfe_GetBistZeroHourReferenceData\(\)](#) will be restored to configuration that existed before the function call.
- The threshold values ([rfeCfg_section_monitorAndSafety_e](#)) to be used along with zero hour data are determined after characterization.

Postcondition

This function does not result in change of RFE-Fw state.

3.6.4.4 CDD_Rfe_GetBlobValue()

```
Std_ReturnType CDD_Rfe_GetBlobValue (
    uint8 * configPtr,
    Cdd_RfeCfgSectionType section,
    Cdd_RfeCfgParamType param,
    uint32 * valuePtr )
```

Function to get the specified value from the blob.

The function identifies the value to be returned, according to the provided parameters. An uint32 value will be returned, for any dimension of the required value.

Parameters

in	<i>configPtr</i>	- pointer to the blob array
in	<i>section</i>	- blob section
in	<i>param</i>	- section parameter
in	<i>valuePtr</i>	- pointer to the receiving address

3.6.4.5 CDD_Rfe_GetError()

```
Cdd_RfeErrorType CDD_Rfe_GetError (
    void )
```

This function provides the error occurred.

This is the first RFE-Fw function to be called. The first call to this function initiates the control core <-> RFE-Fw synchronization protocol and always fails. This function shall be called until it does not return [rfe_error_api_unresponsive_e](#) error anymore, which indicates that the synchronization protocol was executed successfully.

[CDD_Rfe_GetError\(\)](#) can then return one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- [rfe_error_api_invalidArgumentValue_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_sharedData_redundancyCheckFailed_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Precondition

The RFE-Fw must be powered up and intialized.

Returns

Cdd_RfeErrorType

3.6.4.6 CDD_Rfe_GetFrontEnd()

```
Std_ReturnType CDD_Rfe_GetFrontEnd (
    CDD_RfeFrontEndHierarchyType * frontEndPosPtr )
```

Function to get the current associated Front-End "subject" in the RFE-Fw, in a cascaded setup.

The function must be used only in a cascaded setup. If used in a non-cascaded setup, the result will be LEADER for each request. The possible errors are :

- unresponsive
- not synchronized
- busy

Parameters

in	*frontEndPosPtr	- pointer to a CDD_RfeFrontEndHierarchyType variable to receive the used Front-End
----	-----------------	--

3.6.4.7 CDD_Rfe_GetFuSaFaults()

```
Std_ReturnType CDD_Rfe_GetFuSaFaults (
    uint8 * fuSaR1R2FaultListPtr )
```

This function provides the RFE-Fw FuSa faults which are set in Front End FCCU.

This function has to be used when a FuSa fault occurs, which is indicated by the ERROR_N signal and can also be observed by [CDD_Rfe_GetState\(\)](#) returning [rfe_state_fuSaFault_e](#).

This function can return one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- [rfe_error_api_invalidArgumentValue_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_rfeFuSaFault_e](#)

Precondition

The RFE-Fw must be in one of the following states:

- [rfe_state_initialized_e](#)
- [rfe_state_configured_e](#)
- [rfe_state_radarCycleIdle_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Parameters

out	<i>fuSaR1R2FaultListPtr</i>	- Pointer to an array, where this function will store the list of fusa faults. The size of this array must be RFE_FUSA_R1_R2_MASK_UNMASK_FAULT_BYTE_COUNT bytes. The enumeration rfe_fuSaFault_t must be used as bit index to determine the fault from the pFuSaR1R2FaultList e.g., if byte index 3 of pFuSaR1R2FaultList has the value b0010000, it means bit position 5 of fuSaFaultMask3 is set, i.e. rfe_fuSaFault_sr47_lo_level_max_rx1_e fault is set.
-----	-----------------------------	---

Returns

Std_ReturnType - E_OK / E_NOT_OK

Postcondition

This function does not result in change of RFE-Fw state.

3.6.4.8 CDD_Rfe_GetFuSaFaultStatistics()

```
Std_ReturnType CDD_Rfe_GetFuSaFaultStatistics (
    uint16 * r1FaultPromotedToR2Ptr,
    Cdd_RfeRadarCycleCountType * radarCycleCountPtr,
    uint8 * fuSaR1FaultCountListPtr )
```

This function provides the fault statistics for R1 faults.

This function provides list of fault occurrence count for each R1 fault. Whenever a R1 fault is detected, the corresponding counter is incremented. Each counter can reach up to a maximum value of 0xFF and stays at this level until reset. The counter is reset as part of RFE initialization upon:

- Functional reset.
- Destructive reset.
- Power on reset.

The **pFuSaR1FaultCountList** is of fixed size **RFE_FUSA_R1_FAULT_COUNT_MAX** and does not decrease even when R1 fault is promoted to R2 (refer **rfeCfg_param_monitorAndSafety_thresholdValueToPromoteR1Faults_e**). The **pR1FaultPromotedToR2Since** will indicate which R1 fault was promoted to R2.

This function returns after RFE-Fw processed the API call.

This function can return one of following errors:

- **rfe_error_api_unsynchronized_e**
- **rfe_error_api_operationNotAllowed_e**
- **rfe_error_api_busy_e**
- **rfe_error_api_invalidArgumentValue_e**
- Any *cmdInterface* error of **rfe_error_t**.
- **rfe_error_rfeFuSaFault_e**

Precondition

The RFE-Fw must be in one of the following states:

- [rfe_state_initialized_e](#)
- [rfe_state_configured_e](#)
- [rfe_state_radarCycleIdle_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Parameters

out	<i>r1FaultPromotedToR2Ptr</i>	- Indicates the R1 fault which is promoted to R2.
out	<i>radarCycleCountPtr</i>	- Pointer to structure which provides Radar cycle count and Chirp sequence count. The count in pRadarCycleCount is updated whenever the fault occurrence count is updated. If there is no active radar cycle, the radar cycle count and chirp sequence count value will be 0x00 each.
out	<i>fuSaR1FaultCountListPtr</i>	- Pointer to an array containing fault occurrence count for each R1 fault. The size of this array should be RFE_FUSA_R1_FAULT_COUNT_MAX bytes. The <i>rfe_fuSaFault_t</i> up to index value <i>rfe_fuSaFault_R1_sm11_bist_tx4_sw_e</i> bytes, must be used to index the list to determine the fault occurrence count for the corresponding fault e.g., the value in pFuSaCountList [<i>rfe_fuSaFault_R1_sm11_bist_tx4_sw_e</i>] provides the fault occurrence count for <i>rfe_fuSaFault_R1_sm11_bist_tx4_sw_e</i> .

Returns

Std_ReturnType - E_OK / E_NOT_OK

Postcondition

This function does not result in change of RFE-Fw state.

Remarks

1. The functions [CDD_Rfe_GetFuSaFaults\(\)](#), [CDD_Rfe_RadarCycleStart\(\)](#) and [CDD_Rfe_RadarCycleStop\(\)](#) do not affect the counter values.
2. The [rfeCfg_param_monitorAndSafety_thresholdValueToPromoteR1Faults_e](#) has no relation to the counter used in [CDD_Rfe_GetFuSaFaultStatistics\(\)](#).

3.6.4.9 CDD_Rfe_GetNextRadarCycleStartTime()

```
uint32 CDD_Rfe_GetNextRadarCycleStartTime (
    uint16 * radarCycleIndexPtr )
```

This function returns the scheduled absolute start time of the next radar cycle.

The returned time is according to the RFE-Fw time. This function returns after RFE-Fw processed the API call.

This function can sets one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- [rfe_error_api_invalidArgumentValue_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_sharedData_redundancyCheckFailed_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Precondition

The RFE-Fw must be in [rfe_state_radarCycleIdle_e](#) state.

Parameters

out	<i>*radarCycleIndexPtr</i>	- Pointer to index of the next radar cycle of which the start time is returned.
-----	----------------------------	---

Returns

uint32 - Absolute start time of the next scheduled radar cycle in 25 [ns] resolution.

3.6.4.10 CDD_Rfe_GetRadarCycleCount()

```
Cdd_RfeRadarCycleCountType CDD_Rfe_GetRadarCycleCount (
    void )
```

This function returns the radar cycle and chirp sequence count.

It returns the number radar cycles and chirp sequences that have been completed since *Cdd_Rfe_radarCycleStart()*. It can be used to keep track of the radar cycle or chirp sequence execution. This function does not interact with RFE-Fw, it returns information from a shared memory location.

This function can sets one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_sharedData_redundancyCheckFailed_e](#)

Precondition

The RFE-Fw must be in [rfe_state_radarCycleIdle_e](#) state.

Returns

Cdd_RfeRadarCycleCountType - Structure that contains the radar cycle and chirp sequence count.

3.6.4.11 CDD_Rfe_GetRegisterDump()

```
Std_ReturnType CDD_Rfe_GetRegisterDump (
    CDD_RfeFrontEndIdMaskType frontEndSelect,
    rfe_moduleDumpSelect_t moduleSelect,
    rfe_registerDumpValues_t * dumpArrayPtr,
    uint32_t * dumpArrayLenPtr )
```

Debug function to get specific Front-End register values.

The primary usage of this function is to return the current values of the Front-End registers. The read-out is performed on demand (when called), when the radar cycle is stopped. The order of processing is :

- loop over Front-Ends, in following order : Leader, Follower_0 [, Follower_1,...]
- loop over modules (order described here: [rfe_bcdModuleId_t](#))

Front-end modules supply voltages can be also acquired with this function by adding [rfe_registerDump_ATB_VOLTAGE_READOUT](#) to the moduleSelect mask.

The order of returned module supply voltages (measured with the ATB) is identical to the order of modules and voltages described in the [rfe_tef82xxVoltages_t](#) structure. Possible errors that can be obtained calling the [CDD_Rfe_GetError](#) function and are :

- unresponsive
- not synchronized
- busy

Parameters

in	<i>frontEndSelect</i>	- front-end selection mask
in	<i>moduleSelect</i>	- module selection mask
in	<i>dumpArrayPtr</i>	- pointer to the returned values array of rfe_registerDumpValues_t type
in, out	<i>dumpArrayLenPtr</i>	- in : pointer to the returned values array maximum length • out : pointer to the exact number of returned values

3.6.4.12 CDD_Rfe_GetState()

```
Cdd_RfeStateType CDD_Rfe_GetState (
    void )
```

This function returns the RFE-Fw state.

This function does not interact with RFE-Fw, it returns state information from a shared memory location.

This function can sets one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_sharedData_redundancyCheckFailed_e](#)

Returns

Cdd_RfeStateType - The RFE-Fw state

3.6.4.13 CDD_Rfe_GetTime()

```
uint32 CDD_Rfe_GetTime (
    void )
```

This function returns the RFE-Fw time.

It returns the absolute RFE-Fw time since RFE-Fw initialization in 25[ns] resolution. The counter wrap around time is equal to $2^{32} * 25$ ns, which is approximately 107 seconds. This function returns after RFE-Fw processed the API call.

This function can return one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_sharedData_redundancyCheckFailed_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Precondition

The RFE-Fw must be in one of the following states:

- [rfe_state_initialized_e](#)
- [rfe_state_configured_e](#)
- [rfe_state_radarCycleIdle_e](#)

Returns

uint32_t - Absolute RFE-Fw time since initialization in 25 [ns] resolution.

3.6.4.14 CDD_Rfe_GetVersionInfo()

```
void CDD_Rfe_GetVersionInfo (
    Std_VersionInfoType * versionInfoPtr )
```

Service for passing the version information through a pointer *versionInfoPtr.

Service for passing the Information through a pointer "*versionInfoPtr".

Parameters

in	<i>*versionInfoPtr</i>	--> pointer to an address where all the version information needs to be copied.
----	------------------------	---

3.6.4.15 CDD_Rfe_Init()

```
void CDD_Rfe_Init (
    void )
```

This function synchronizes the Rfe_Driver with the RFE-Fw.

This is the first RFE Abstract 2.0 function to be called. The first call to this function initiates the control core <-> RFE-Fw synchronization protocol and always fails. This function shall be called until it does not sets [rfe_error_api_unresponsive_e](#) error anymore, which indicates that the synchronization protocol was executed successfully.

This function can sets one of following errors:

- [rfe_error_api_unresponsive_e](#)
- [rfe_error_sharedData_redundancyCheckFailed_e](#)

Precondition

The RFE-Fw must be powered up and intialized.

Postcondition

RFE-Fw in [rfe_state_initialized_e](#) state

3.6.4.16 CDD_Rfe_MonitorRead()

```
Cdd_RfeRadarCycleCountType CDD_Rfe_MonitorRead (
    Cdd_RfeMonitorSelectType monitorSelect,
    Cdd_RfeMonitorValuesType * monitorValuesPtr )
```

This function will read-out one or more RFE monitors.

Monitors to be read are selected by `monitorSelect`.

Current implementation supports reading of temperature sensors. [RFE_MONITOR_SELECT_TX_POWER](#) and [RFE_MONITOR_SELECT_PDC_CLIPPING_COUNT](#) of `monitorSelect` are not implemented. API user will receive [rfe_error_api_operationNotAllowed_e](#), if these options are passed.

[RFE_MONITOR_SELECT_ALL](#) bits of `monitorSelect`, is to read values from all monitors supported by API. Since current implementation supports only temperature sensors, other sensor values of the passed structure should be discarded.

This function returns after RFE-Fw processed the API call.

This function can sets one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- [rfe_error_api_invalidArgumentValue_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_sharedData_redundancyCheckFailed_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Precondition

The RFE-Fw must be in one of the following states:

- [rfe_state_initialized_e](#)
- [rfe_state_configured_e](#)
- [rfe_state_radarCycleIdle_e](#) ([RFE_MONITOR_SELECT_TEMP_IMMEDIATELY](#) bit of **monitorSelect** must be '0')
- [rfe_state_testContinuousWaveTransmission_e](#)

Parameters

in	<i>monitorSelect</i>	- Selection of the RFE-Fw monitors to read-out
out	<i>*monitorValuesPtr</i>	- The monitor values will be written to this pointer

Returns

Cdd_RfeRadarCycleCountType - The radar cycle and chirp sequence count, indicating when the monitors were read.

3.6.4.17 CDD_Rfe_RadarCycleStart()

```
Std_ReturnType CDD_Rfe_RadarCycleStart (
    uint16 radarCycleCount,
    boolean isScheduled,
    uint32 startTime )
```

This function starts the configured radar cycles.

The configured radar cycles are started at **startTime** or as soon as possible depending on the **isScheduled** parameter. The radar cycle start can be scheduled from 10 us up to 1000 ms in the future (on reception of the API call by the RFE-Fw). This function returns after RFE-Fw processed the API call.

This function can sets one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)

- [rfe_error_api_busy_e](#)
- [rfe_error_api_invalidArgumentValue_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_sharedData_redundancyCheckFailed_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Precondition

The RFE-Fw must be in [rfe_state_configured_e](#) state.

Parameters

in	<i>radarCycleCount</i>	- Number of radar cycles to start. A value of 0 indicates an infinite number of radar cycles, i.e. until stopped by <i>Cdd_Rfe_radarCycleStop()</i> .
in	<i>isScheduled</i>	- If true , radar cycle starts at startTime . If false, radar cycle starts as soon as possible.
in	<i>startTime</i>	- The absolute start time in 25 [ns] resolution of the chirp sequence according to the RFE-Fw time.

Returns

Std_ReturnType - E_OK / E_NOT_OK

Postcondition

When the API call is accepted and radar cycle is started, RFE-Fw state will become [rfe_state_busy_e](#) during the the calibration, BIST and chirp sequences. During the radar cycle idle windows the state is [rfe_state_radarCycleIdle_e](#). RFE-Fw state returns to [rfe_state_configured_e](#), when all radar cycles are completed.

3.6.4.18 CDD_Rfe_RadarCycleStop()

```
uint16 CDD_Rfe_RadarCycleStop (
    void )
```

This function stops the active radar cycles.

The active radar cycle will not be finished, but it is stopped as soon as the command has been received. Pending parameters update will be cancelled. It can be called, for example, when the application needs to do a full reconfiguration or needs to go to a lower power mode or for any external event where active radar cycle needs to be stopped. This function returns after RFE-Fw processed the API call.

This function can sets one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)

- [rfe_error_api_busy_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_sharedData_redundancyCheckFailed_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Precondition

The RFE-Fw state must be [rfe_state_radarCycleIdle_e](#).

Returns

uint16 - Index of the active radar cycle at which radar cycle was stopped.

Postcondition

Radar cycle is stopped and RFE-Fw state returns to [rfe_state_configured_e](#).

3.6.4.19 CDD_Rfe_SetBlobValue()

```
Std_ReturnType CDD_Rfe_SetBlobValue (
    uint8 * configPtr,
    Cdd_RfeCfgSectionType section,
    Cdd_RfeCfgParamType param,
    uint32 value )
```

Function to get the specified value from the blob.

The function identifies the value to be set, according to the provided parameters. The provided value is truncated to the real blob value dimension.

Parameters

in	<i>configPtr</i>	- pointer to the blob array
in	<i>section</i>	- blob section
in	<i>param</i>	- section parameter
in	<i>value</i>	- the value to be set

3.6.4.20 CDD_Rfe_SetFrontEnd()

```
Std_ReturnType CDD_Rfe_SetFrontEnd (
    CDD_RfeFrontEndHierarchyType frontEndPos )
```

Function to change the associated Front-End "subject" in a cascaded setup.

The function must be used only in a cascaded setup. If used in a non-cascaded mode the result will be : SUCCESS for LEADER request and error for FOLLOWER request. The reported error will be detailed using Det mechanism. The possible errors are :

- unresponsive
- not synchronized
- busy
- invalid argument - only this is a specific error for this operation request, returned from RFE-Fw

Parameters

in	<i>frontEndPos</i>	- the Front-End to be used as conversation subject between the two cores
----	--------------------	--

3.6.4.21 CDD_Rfe_SetNextRadarCycleStartTime()

```
uint16 CDD_Rfe_SetNextRadarCycleStartTime (
    uint32 startTime )
```

This function updates the start time of the next radar cycle.

Cdd_Rfe_radarCycleStart() starts one or multiple radar cycles immediately or at a schedule time, with a certain period ([rfeCfg_param_radarCycle_radarCycleDuration_e](#)), as per the configuration. This function updates the absolute start time of the next radar cycle according to the RFE time. That radar cycle will thereby be delayed or pulled in. Subsequent radar cycles will shift in time accordingly to maintain the configured [rfeCfg_param_radarCycle_radarCycleDuration_e](#). This functionality can be used to synchronize multiple radar sensors in time. The next radar cycle start time can be delayed up to 100 ms and should not conflict with the current radar cycle time schedule (in case of pull in). This function returns after RFE-Fw processed the API call.

This function can sets one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- [rfe_error_api_invalidArgumentValue_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_sharedData_redundancyCheckFailed_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Precondition

The RFE-Fw must be in [rfe_state_radarCycleIdle_e](#) state.

Parameters

in	<i>startTime</i>	- Absolute start time of the next radar cycle in 25 [ns] resolution.
----	------------------	--

Returns

uint16 - Index of the next radar cycle of which the start time is set.

Postcondition

Next radar cycle starts at the given start time.

3.6.4.22 CDD_Rfe_SwHwGetVersion()

```
Cdd_RfeVersionType CDD_Rfe_SwHwGetVersion (  
    void )
```

This function returns version and variant details of the RFE-Fw.

This function returns after RFE-Fw processed the API call.

This function can sets one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_sharedData_redundancyCheckFailed_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Precondition

The RFE must be in one of the following states:

- [rfe_state_initialized_e](#)
- [rfe_state_configured_e](#)
- [rfe_state_radarCycleIdle_e](#)

Returns

Cdd_RfeVersionType - Structure containing RFE-Fw variant and version details.

3.6.4.23 CDD_Rfe_TestContinuousWaveTransmissionStart()

```
Std_ReturnType CDD_Rfe_TestContinuousWaveTransmissionStart (
    Cdd_RfeChirpProfileIndexType profileIndex )
```

This function start continuous wave transmission.

It uses the RFE-Fw profile as configured via Cdd_Rfe_configure() as specified by `profileIndex`. The frequency is kept at the configured center frequency. This function returns after RFE-Fw processed the API call.

This function can sets one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- [rfe_error_api_invalidArgumentValue_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_sharedData_redundancyCheckFailed_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Precondition

RFE-Fw must be in [rfe_state_configured_e](#) state.

Parameters

in	profileIndex	- Index of the chirp profile to be used for continuous wave transmission.
----	------------------------------	---

Returns

Std_ReturnType - E_OK / E_NOT_OK

Postcondition

RFE-Fw goes to [rfe_state_testContinuousWaveTransmission_e](#) state.

3.6.4.24 CDD_Rfe_TestContinuousWaveTransmissionStop()

```
Std_ReturnType CDD_Rfe_TestContinuousWaveTransmissionStop (
    void )
```

This function stops continuous wave transmission.

Continuous wave transmission can be started via [CDD_Rfe_TestContinuousWaveTransmissionStart\(\)](#). This function returns after RFE-Fw processed the API call.

This function can sets one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_sharedData_redundancyCheckFailed_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Precondition

RFE-Fw must be in [rfe_state_testContinuousWaveTransmission_e](#) state.

Returns

Std_ReturnType - E_OK / E_NOT_OK

Postcondition

RFE-Fw goes to [rfe_state_configured_e](#) state.

3.6.4.25 CDD_Rfe_TestGetInternalError()

```
Std_ReturnType CDD_Rfe_TestGetInternalError (  
    void )
```

This function fetches the internal software error code from M7_1 and stores it.

This function fetches the internal software error code from M7_1 and stores it. This function returns whether RFE has accepted or rejected the API call. API user should invoke [CDD_Rfe_GetError\(\)](#) to get the internal software error stored by this API for further usage.

Precondition

The RFE must be in one of the following states:

- [rfe_state_initialized_e](#)
- [rfe_state_configured_e](#)
- [rfe_state_radarCycleIdle_e](#)
- [rfe_state_fuSaFault_e](#)

Returns

Std_ReturnType - E_OK / E_NOT_OK

3.6.4.26 CDD_Rfe_TestSetParam()

```
Std_ReturnType CDD_Rfe_TestSetParam (
    Cdd_RfeTestParamType testParam,
    uint32 value )
```

This function sets a test parameter.

This function returns after RFE-Fw processed the API call.

This function can sets one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- [rfe_error_api_invalidArgumentValue_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_sharedData_redundancyCheckFailed_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Precondition

The RFE-Fw must be in one of the following states:

- [rfe_state_initialized_e](#)
- [rfe_state_configured_e](#)

Parameters

in	<i>testParam</i>	- The test parameter to set
in	<i>value</i>	- The value to set

Returns

Std_ReturnType - E_OK / E_NOT_OK

3.6.4.27 CDD_Rfe_UpdateBegin()

```
void CDD_Rfe_UpdateBegin (
    void )
```

This function begins a RFE-Fw configuration update operation.

When the RFE-Fw Configuration (as set via [CDD_Rfe_Configure\(\)](#)) is required to be updated then this is the first function to be called. This function will clear the buffered RFE-Fw configuration. This function does not interact with RFE-Fw, it buffers data locally. This function does not return any error.

Postcondition

The functions [CDD_Rfe_UpdateParam\(\)](#) and/or [CDD_Rfe_UpdateDynamicTable\(\)](#) must be called after this function to buffer the configuration and then push the configuration into RFE-Fw via the configuration update via [CDD_Rfe_UpdatePush\(\)](#).

3.6.4.28 CDD_Rfe_UpdateDynamicTable()

```
Std_ReturnType CDD_Rfe_UpdateDynamicTable (
    uint8 dynamicTableIndex,
    Cdd_RfeDynParamsSelectType paramsSelect )
```

This functions buffers a dynamic table update.

It buffers an update of the dynamic table in the RFE-Fw after the initial configuration. It indicates to the RFE-Fw that a full dynamic table or part of it (i.e. selected dynamic parameters), located at the dynamicTableAddress in system memory (as configured via [Cdd_Rfe_configure\(\)](#)), has been modified and needs to be updated in the RFE-Fw. This update operation will be jointly pushed, with other buffered update operations, when calling [Cdd_Rfe_updatePush\(\)](#). A maximum number of [RFE_UPDATE_COUNT_MAX](#) updates operations are supported. This function does not interact with RFE-Fw, it buffers data locally.

This function can sets one of following errors:

- [rfe_error_api_maxUpdateCountExceeded_e](#)

Precondition

Call [Cdd_Rfe_updateBegin\(\)](#).

Parameters

in	<i>dynamicTableIndex</i>	- The index of the dynamic table to update.
in	<i>paramsSelect</i>	- The selection of dynamic parameters to update. Use RFE_DYN_PARAMS_SELECT_ALL to update the full table.

Returns

Std_ReturnType - E_OK / E_NOT_OK

Postcondition

Push all buffered update operations to RFE-Fw by calling [Cdd_Rfe_updatePush\(\)](#).

3.6.4.29 CDD_Rfe_UpdateParam()

```
Std_ReturnType CDD_Rfe_UpdateParam (
    Cdd_RfeCfgSectionType section,
    Cdd_RfeParamUpdateType param,
    uint32 value )
```

This functions buffers one of multiple configuration parameter set operations.

It buffers an update of the RFE-Fw configuration as was set via Cdd_Rfe_configure(). This parameter set operation will be jointly pushed with other buffered update operations when calling Cdd_Rfe_UpdatePush(). A maximum number of [RFE_UPDATE_COUNT_MAX](#) updates operations are supported for each push. This function does not interact with RFE-Fw, it buffers data locally.

This function can sets one of following errors:

- [rfe_error_api_maxUpdateCountExceeded_e](#)

Precondition

Call Cdd_Rfe_updateBegin().

Parameters

in	<i>section</i>	- The configuration section to set
in	<i>param</i>	- The configuration parameter to set
in	<i>value</i>	- The value to set

Returns

Std_ReturnType - E_OK / E_NOT_OK

Postcondition

Push all buffered update operations to RFE-Fw by calling Cdd_Rfe_updatePush().

3.6.4.30 CDD_Rfe_UpdatePush()

```
uint16 CDD_Rfe_UpdatePush (
    void )
```

This function pushes all the buffered update operations jointly to RFE-Fw.

Precondition

In a **Cascaded** configuration, before calling this function set (using [CDD_Rfe_SetFrontEnd](#) or be sure it is the correct one) the Front-End to receive the update. Only one Front-End will receive the update at a time. If necessary to proceed to the same update on many Front-Ends, after calling [CDD_Rfe_UpdatePush](#), set the new Front-End to receive the same update using [CDD_Rfe_SetFrontEnd](#) and call again [CDD_Rfe_UpdatePush](#).

Update-operations can be buffered by [CDD_Rfe_UpdateParam\(\)](#) or [CDD_Rfe_UpdateDynamicTable\(\)](#). It updates the RFE-Fw configuration as was set via [CDD_Rfe_Configure\(\)](#). The RFE-Fw will first check that the parameter values are within the supported range. If so, the parameters will be jointly applied for the next radar cycle. The next radar cycle index will be returned by this API function. Updates are not allowed when the last radar cycle is active. When there is no radar cycle active, the updates will be applied immediately within the API function. An error is returned if the parameters are not within the accepted range. This function returns after RFE-Fw processed the API call.

This function can sets one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- [rfe_error_api_noUpdatesBuffered_e](#)
- [rfe_error_api_maxUpdateCountExceeded_e](#)
- [rfe_error_api_updateAlreadyPending_e](#)
- [rfe_error_api_invalidParameterForUpdate_e](#)
- [rfe_error_api_dynamicTablesNotConfigured_e](#)
- [rfe_error_api_cfgSectionMismatch_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_sharedData_redundancyCheckFailed_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Precondition

RFE-Fw must be in [rfe_state_configured_e](#) or [rfe_state_radarCycleIdle_e](#) state.

Returns

uint16 - The index of the next radar cycle at which the updates will be applied. If radar cycle is not active, 0 will be returned.

3.7 RFE Abstract 2.0 - Low Level Constants

Defines the low level constants used by the RFE Abstract 2.0 API.

Macros

- #define `RFE_CMD_CLIENT_UPDATE_SECTION_DYNAMIC_TABLE` (0xFFFFFUL)
- #define `RFE_CMD_CLIENT_FUSA_FAULT_BYTE_COUNT` (12UL)
- #define `RFE_ERROR_RFE_INTERNAL_ERROR_BIT` (1UL<<24u)
- #define `RFE_FUSA_R1_FAULT_COUNT_MAX` (29UL) /* TODO: Exact count needs to be updated once R1/R2 Fault classification is complete */
- #define `RFE_FUSA_R2_FAULT_COUNT_MAX` (12UL) /* TODO: Exact count needs to be updated once R1/R2 Fault classification is complete */
- #define `RFE_FUSA_R1_R2_FAULT_COUNT_MAX` (RFE_FUSA_R1_FAULT_COUNT_MAX + RFE_FUSA_R2_FAULT_COUNT_MAX)
- #define `RFE_FUSA_R1_R2_MASK_UNMASK_FAULT_BYTE_COUNT` (12UL)
- #define `RFE_UPDATE_COUNT_MAX` (16UL)
- #define `RFE_TEMPERATURE_SENSOR_COUNT` (4UL)
- #define `RFE_TX_BIST_COMBINATIONS` (3UL)
- #define `RFE_MONITOR_SELECT_RX_SAT_COUNT_STAGE_1I` ((rfe_monitorSelect_t) 1UL<<0U)
- #define `RFE_MONITOR_SELECT_PDC_CLIPPING_COUNT` ((rfe_monitorSelect_t)(1UL<<4U))
- #define `RFE_MONITOR_SELECT_TEMP_BEFORE_CHIRP_SEQ` ((rfe_monitorSelect_t)(1UL<<5U))
- #define `RFE_MONITOR_SELECT_TEMP_AFTER_CHIRP_SEQ` ((rfe_monitorSelect_t)(1UL<<6U))
- #define `RFE_MONITOR_SELECT_TEMP_IMMEDIATELY` ((rfe_monitorSelect_t)(1UL<<7U))
- #define `RFE_MONITOR_SELECT_TX_POWER` ((rfe_monitorSelect_t)(1UL<<8U))
- #define `RFE_MONITOR_SELECT_ALL` ((rfe_monitorSelect_t)0x1FFU)
- #define `RFE_DYN_PARAMS_SELECT_CHIRP_PROFILE` ((rfe_dynParamsSelect_t) (1UL<<0U))
- #define `RFE_DYN_PARAMS_SELECT_TX1_PHASE_ROTATION` ((rfe_dynParamsSelect_t) (1UL<<1U))
- #define `RFE_DYN_PARAMS_SELECT_TX2_PHASE_ROTATION` ((rfe_dynParamsSelect_t) (1UL<<2U))
- #define `RFE_DYN_PARAMS_SELECT_TX3_PHASE_ROTATION` ((rfe_dynParamsSelect_t) (1UL<<3U))
- #define `RFE_DYN_PARAMS_SELECT_TX4_PHASE_ROTATION` ((rfe_dynParamsSelect_t) (1UL<<4U))
- #define `RFE_DYN_PARAMS_SELECT_TX_TRANSMISSION_ENABLE` ((rfe_dynParamsSelect_t) (1UL<<5U))
- #define `RFE_DYN_PARAMS_SELECT_DWELL_TIME_TICKS` ((rfe_dynParamsSelect_t) (1UL<<6U))
- #define `RFE_DYN_PARAMS_SELECT_CIT_TICKS` ((rfe_dynParamsSelect_t) (1UL<<7U))
- #define `RFE_DYN_PARAMS_SELECT_CHIRP_FREQUENCY_DRIFT_STEPS` ((rfe_dynParamsSelect_t) (1UL<<8U))
- #define `RFE_DYN_PARAMS_SELECT_ALL` ((rfe_dynParamsSelect_t) (0x1FFUL))
- #define `RFE_RX_COUNT` (4UL)
- #define `RFE_RX_STAGE` (2UL)
- #define `RFE_RX_CLIPPING_RESET` (5UL)
- #define `RFE_TX_POWER_ERR` (4UL)
- #define `RFE_RX_BIST_COMBINATIONS` (3UL)
- #define `RFE_NS_PER_TIME_TICK` (25UL)
- #define `RFE_TIME_TICKS_PER_US` (40UL)
- #define `RFE_MAX_SUPPORTED_DEVICES` (2U)
- #define `RFE_RX_SELECT_NONE` ((rfe_rxSelect_t) 0x0UL)
- #define `RFE_RX_SELECT_1` ((rfe_rxSelect_t) (1UL<<0U))
- #define `RFE_RX_SELECT_2` ((rfe_rxSelect_t) (1UL<<1U))
- #define `RFE_RX_SELECT_3` ((rfe_rxSelect_t) (1UL<<2U))
- #define `RFE_RX_SELECT_4` ((rfe_rxSelect_t) (1UL<<3U))
- #define `RFE_RX_SELECT_ALL` ((rfe_rxSelect_t) 0xFUL)
- #define `RFE_TX_SELECT_NONE` ((rfe_txSelect_t) 0x0UL)
- #define `RFE_TX_SELECT_1` ((rfe_txSelect_t) (1UL<<0))
- #define `RFE_TX_SELECT_2` ((rfe_txSelect_t) (1UL<<1))
- #define `RFE_TX_SELECT_3` ((rfe_txSelect_t) (1UL<<2))
- #define `RFE_TX_SELECT_4` ((rfe_txSelect_t) (1UL<<3))
- #define `RFE_TX_SELECT_ALL` ((rfe_txSelect_t) 0xFUL)
- #define `RFE_DATA_OUT_DEST_PACKET_PROCESSOR` ((rfe_dataOutDest_t)(1UL<<0u))
- #define `RFE_DATA_OUT_DEST_CSI2` ((rfe_dataOutDest_t)(1UL<<1u))
- #define `RFE_API_IRQ_MAX_NUM` (19UL)

- `#define RFE_TEMPERATURE_1_DEG_CELSIUS ((rfe_temperature_t)(1UL<<6U))`
- `#define RFE_KELVIN_TO_CELSIUS(celsius) ((celsius) - 273.0f)`
- `#define RFE_EVENTS_IRQ_ALL ((uint8_t)RFE_EVENTS_IRQ_STATE_CHANGE | (uint8_t)RFE_EVENTS_IRQ_RADAR_CYCLE | (uint8_t)RFE_EVENTS_IRQ_CHIRP_SEQUENCE_CHANGE)`
- `#define RFE_API_IRQ_ALL`

3.7.1 Detailed Description

Defines the low level constants used by the RFE Abstract 2.0 API.

3.7.2 Macro Definition Documentation

3.7.2.1 RFE_API_IRQ_ALL

```
#define RFE_API_IRQ_ALL
```

Value:

```
( (uint32_t)RFE_API_IRQ_CONFIGURE | (uint32_t)RFE_API_IRQ_RADAR_CYCLE_START |
(uint32_t)RFE_API_IRQ_RADAR_CYCLE_STOP | \
(uint32_t)RFE_API_IRQ_GET_FUSA_FAULTS | (uint32_t)RFE_API_IRQ_GET_TIME |
(uint32_t)RFE_API_IRQ_GET_VERSION | (uint32_t)RFE_API_IRQ_MONITOR_READ | \
(uint32_t)RFE_API_IRQ_GET_NEXT_RADAR_CYCLE_START_TIME |
(uint32_t)RFE_API_IRQ_SET_NEXT_RADAR_CYCLE_START_TIME | (uint32_t)RFE_API_IRQ_UPDATE_PUSH | \
(uint32_t)RFE_API_IRQ_CONTINUOUS_WAVE_TRANSMISSION_START |
(uint32_t)RFE_API_IRQ_CONTINUOUS_WAVE_TRANSMISSION_STOP | (uint32_t)RFE_API_IRQ_TEST_SET_PARAM | \
(uint32_t)RFE_API_IRQ_GET_FUSA_FAULT_STATISTICS |
(uint32_t)RFE_API_IRQ_GET_BIST_ZERO_HOUR_REFERENCE_DATA |
(uint32_t)RFE_API_IRQ_TEST_GET_INTERNAL_ERROR | \
(uint32_t)RFE_API_IRQ_SET_FE_ID | (uint32_t)RFE_API_IRQ_GET_FE_ID | (uint32_t)RFE_API_IRQ_REG_DUMP
)
```

This bit enables IRQ on all Rfe APIs completion

3.7.2.2 RFE_API_IRQ_MAX_NUM

```
#define RFE_API_IRQ_MAX_NUM (19UL)
```

Thus value define the maximum number of API interrupt requests

3.7.2.3 RFE_CMD_CLIENT_FUSA_FAULT_BYTE_COUNT

```
#define RFE_CMD_CLIENT_FUSA_FAULT_BYTE_COUNT (12UL)
```

This macro defines the number of FuSa fault bytes

3.7.2.4 RFE_CMD_CLIENT_UPDATE_SECTION_DYNAMIC_TABLE

```
#define RFE_CMD_CLIENT_UPDATE_SECTION_DYNAMIC_TABLE (0xFFFFFUL)
```

This macro defines the section value for the dynamic table, that is used in the `rfe_updateDynamicTable()` API function

3.7.2.5 RFE_DATA_OUT_DEST_CSI2

```
#define RFE_DATA_OUT_DEST_CSI2 ((rfe_dataOutDest_t) (1UL<<1u))
```

This bit enables data output over CSI2.

3.7.2.6 RFE_DATA_OUT_DEST_PACKET_PROCESSOR

```
#define RFE_DATA_OUT_DEST_PACKET_PROCESSOR ((rfe_dataOutDest_t) (1UL<<0u))
```

This bit enables data output to packet processor.

3.7.2.7 RFE_DYN_PARAMS_SELECT_ALL

```
#define RFE_DYN_PARAMS_SELECT_ALL ((rfe_dynParamsSelect_t) (0x1FFUL))
```

Selects all the above dynamic parameters

3.7.2.8 RFE_DYN_PARAMS_SELECT_CHIRP_FREQUENCY_DRIFT_STEPS

```
#define RFE_DYN_PARAMS_SELECT_CHIRP_FREQUENCY_DRIFT_STEPS ((rfe_dynParamsSelect_t) (1UL<<8U))
```

Selects chirp frequency drift as dynamic parameter

3.7.2.9 RFE_DYN_PARAMS_SELECT_CHIRP_PROFILE

```
#define RFE_DYN_PARAMS_SELECT_CHIRP_PROFILE ((rfe_dynParamsSelect_t) (1UL<<0U))
```

Selects chirp profile as dynamic parameter, will overwrite [chirpProfileSequence](#) of the configuration blob.

3.7.2.10 RFE_DYN_PARAMS_SELECT_CIT_TICKS

```
#define RFE_DYN_PARAMS_SELECT_CIT_TICKS ((rfe_dynParamsSelect_t) (1UL<<7U))
```

Selects chirp interval time as dynamic parameter, will overwrite [chirpIntervalTimeTicks](#) of the config blob.

3.7.2.11 RFE_DYN_PARAMS_SELECT_DWELL_TIME_TICKS

```
#define RFE_DYN_PARAMS_SELECT_DWELL_TIME_TICKS ((rfe_dynParamsSelect_t) (1UL<<6U))
```

Selects chirp dwell time as dynamic parameter, will over-write [dwellTimeTicks](#) of the configuration blob.

3.7.2.12 RFE_DYN_PARAMS_SELECT_TX1_PHASE_ROTATION

```
#define RFE_DYN_PARAMS_SELECT_TX1_PHASE_ROTATION ((rfe_dynParamsSelect_t) (1UL<<1U))
```

Selects Tx1 phase rotation as dynamic parameter, will overwrite [txPhaseRotation](#) of the configuration blob.

3.7.2.13 RFE_DYN_PARAMS_SELECT_TX2_PHASE_ROTATION

```
#define RFE_DYN_PARAMS_SELECT_TX2_PHASE_ROTATION ((rfe_dynParamsSelect_t) (1UL<<2U))
```

Selects Tx2 phase rotation as dynamic parameter, will overwrite [txPhaseRotation](#) of the configuration blob.

3.7.2.14 RFE_DYN_PARAMS_SELECT_TX3_PHASE_ROTATION

```
#define RFE_DYN_PARAMS_SELECT_TX3_PHASE_ROTATION ((rfe_dynParamsSelect_t) (1UL<<3U))
```

Selects Tx3 phase rotation as dynamic parameter, will overwrite [txPhaseRotation](#) of the configuration blob.

3.7.2.15 RFE_DYN_PARAMS_SELECT_TX4_PHASE_ROTATION

```
#define RFE_DYN_PARAMS_SELECT_TX4_PHASE_ROTATION ((rfe_dynParamsSelect_t) (1UL<<4U))
```

Selects Tx4 phase rotation as dynamic parameter, will overwrite [txPhaseRotation](#) of the configuration blob.

3.7.2.16 RFE_DYN_PARAMS_SELECT_TX_TRANSMISSION_ENABLE

```
#define RFE_DYN_PARAMS_SELECT_TX_TRANSMISSION_ENABLE ((rfe_dynParamsSelect_t) (1UL<<5U))
```

Selects Tx transmission enable as dynamic parameter, will overwrite [txTransmissionEnable](#) of the config blob.

3.7.2.17 RFE_ERROR_RFE_INTERNAL_ERROR_BIT

```
#define RFE_ERROR_RFE_INTERNAL_ERROR_BIT (1UL<<24u)
```

This macro defines the start of RFE-Fw Internal Errors.

3.7.2.18 RFE_EVENTS_IRQ_ALL

```
#define RFE_EVENTS_IRQ_ALL ((uint8_t)RFE_EVENTS_IRQ_STATE_CHANGE | (uint8_t)RFE_EVENTS_IRQ_RADAR_CYCLE_CHANGE  
| (uint8_t)RFE_EVENTS_IRQ_CHIRP_SEQUENCE_CHANGE)
```

This bitmask enable all events IRQs

3.7.2.19 RFE_FUSA_R1_FAULT_COUNT_MAX

```
#define RFE_FUSA_R1_FAULT_COUNT_MAX (29UL) /* TODO: Exact count needs to be updated once R1/R2  
Fault classification is complete */
```

This macro defines the maximum number of R1 FUSA errors.

3.7.2.20 RFE_FUSA_R1_R2_FAULT_COUNT_MAX

```
#define RFE_FUSA_R1_R2_FAULT_COUNT_MAX (RFE_FUSA_R1_FAULT_COUNT_MAX + RFE_FUSA_R2_FAULT_COUNT_MAX)
```

This macro defines the maximum number of R1, R2 FUSA faults.

3.7.2.21 RFE_FUSA_R1_R2_MASK_UNMASK_FAULT_BYTE_COUNT

```
#define RFE_FUSA_R1_R2_MASK_UNMASK_FAULT_BYTE_COUNT (12UL)
```

This macro defines the number of bytes used in RFE-FW Configuration BLOB for masking or unmasking the R1, R2 FuSa Faults of total 12 bytes.

3.7.2.22 RFE_FUSA_R2_FAULT_COUNT_MAX

```
#define RFE_FUSA_R2_FAULT_COUNT_MAX (12UL) /* TODO: Exact count needs to be updated once R1/R2  
Fault classification is complete */
```

This macro defines the maximum number of R2 FUSA faults.

3.7.2.23 RFE_KELVIN_TO_CELSIUS

```
#define RFE_KELVIN_TO_CELSIUS(  
    celsius ) ((celsius) - 273.0f)
```

This macro converts kelvin to celsius

3.7.2.24 RFE_MAX_SUPPORTED_DEVICES

```
#define RFE_MAX_SUPPORTED_DEVICES ( 2U )
```

This macro defines the maximum number of frontends supported by SOC.

3.7.2.25 RFE_MONITOR_SELECT_ALL

```
#define RFE_MONITOR_SELECT_ALL ((rfe_monitorSelect_t)0x1FFU)
```

This value selects all monitors.

3.7.2.26 RFE_MONITOR_SELECT_PDC_CLIPPING_COUNT

```
#define RFE_MONITOR_SELECT_PDC_CLIPPING_COUNT ( (rfe_monitorSelect_t) (1UL<<4U) )
```

This bit selects the pdc clipping count off all enabled pdc for monitor read-out.

3.7.2.27 RFE_MONITOR_SELECT_RX_SAT_COUNT_STAGE_1I

```
#define RFE_MONITOR_SELECT_RX_SAT_COUNT_STAGE_1I ( ( rfe_monitorSelect_t ) 1UL << 0U )
```

Monitor select - complete definitions This bit selects the rx stage1 in-phase saturation count of all enabled rx for monitor read-out.

3.7.2.28 RFE_MONITOR_SELECT_TEMP_AFTER_CHIRP_SEQ

```
#define RFE_MONITOR_SELECT_TEMP_AFTER_CHIRP_SEQ ( (rfe_monitorSelect_t) (1UL<<6U) )
```

This bit selects the temperature after the chirp sequence of all temp sensors for monitor read-out.

3.7.2.29 RFE_MONITOR_SELECT_TEMP_BEFORE_CHIRP_SEQ

```
#define RFE_MONITOR_SELECT_TEMP_BEFORE_CHIRP_SEQ ( (rfe_monitorSelect_t) (1UL<<5U) )
```

This bit selects the temperature before the chirp sequence of all temp sensors for monitor read-out.

3.7.2.30 RFE_MONITOR_SELECT_TEMP_IMMEDIATELY

```
#define RFE_MONITOR_SELECT_TEMP_IMMEDIATELY ( (rfe_monitorSelect_t) (1UL<<7U) )
```

This bit selects the current temperature of all temp sensors for monitor read-out.

3.7.2.31 RFE_MONITOR_SELECT_TX_POWER

```
#define RFE_MONITOR_SELECT_TX_POWER ( (rfe_monitorSelect_t) (1UL<<8U) )
```

This bit selects the transmitter power for monitor read-out.

3.7.2.32 RFE_NS_PER_TIME_TICK

```
#define RFE_NS_PER_TIME_TICK ( 25UL )
```

This macro defines the number of nanoseconds that corresponds to a single time tick.

3.7.2.33 RFE_RX_BIST_COMBINATIONS

```
#define RFE_RX_BIST_COMBINATIONS (3UL)
```

This macro defines the RX BIST combinations: RX1-RX2, RX1-RX3 and RX1-RX4

3.7.2.34 RFE_RX_CLIPPING_RESET

```
#define RFE_RX_CLIPPING_RESET (5UL)
```

This macro defines the rx clipping reset counter. [rfe_rxClippingReset_t](#) defines the stage indices.

3.7.2.35 RFE_RX_COUNT

```
#define RFE_RX_COUNT (4UL)
```

This macro defines the number of receivers. [rfe_rxIndex_t](#) defines the receiver indices.

3.7.2.36 RFE_RX_SELECT_1

```
#define RFE_RX_SELECT_1 ( ( rfe_rxSelect_t ) ( 1UL<<0U ) )
```

This bit selects receiver 1.

3.7.2.37 RFE_RX_SELECT_2

```
#define RFE_RX_SELECT_2 ( ( rfe_rxSelect_t ) ( 1UL<<1U ) )
```

This bit selects receiver 2.

3.7.2.38 RFE_RX_SELECT_3

```
#define RFE_RX_SELECT_3 ( ( rfe_rxSelect_t ) ( 1UL<<2U ) )
```

This bit selects receiver 3.

3.7.2.39 RFE_RX_SELECT_4

```
#define RFE_RX_SELECT_4 ( ( rfe_rxSelect_t ) ( 1UL<<3U ) )
```

This bit selects receiver 4.

3.7.2.40 RFE_RX_SELECT_ALL

```
#define RFE_RX_SELECT_ALL ( ( rfe_rxSelect_t ) 0xFUL )
```

This value selects all 4 receivers.

3.7.2.41 RFE_RX_SELECT_NONE

```
#define RFE_RX_SELECT_NONE ( ( rfe_rxSelect_t ) 0x0UL )
```

This value selects no receivers.

3.7.2.42 RFE_RX_STAGE

```
#define RFE_RX_STAGE (2UL)
```

This macro defines the rx stages. [rfe_rxStage_t](#) defines the stage indices.

3.7.2.43 RFE_TEMPERATURE_1_DEG_CELSIUS

```
#define RFE_TEMPERATURE_1_DEG_CELSIUS ((rfe_temperature_t) (1UL<<6U))
```

This macro defines 1 degree celsius.

3.7.2.44 RFE_TEMPERATURE_SENSOR_COUNT

```
#define RFE_TEMPERATURE_SENSOR_COUNT (4UL)
```

This macro defines the number of temperature sensors. [rfe_temperatureSensorIndex_t](#) defines the temperature sensors.

3.7.2.45 RFE_TIME_TICKS_PER_US

```
#define RFE_TIME_TICKS_PER_US ( 40UL )
```

This macro defines the number of ticks per microsecond.

3.7.2.46 RFE_TX_BIST_COMBINATIONS

```
#define RFE_TX_BIST_COMBINATIONS (3UL)
```

This macro defines the TX BIST combinations: TX1-TX2, TX2-TX3 and TX3-TX4

3.7.2.47 RFE_TX_POWER_ERR

```
#define RFE_TX_POWER_ERR (4UL)
```

This macro defines the tx power error. [rfe_txPowerErrorFlag_t](#) defines the stage indices.

3.7.2.48 RFE_TX_SELECT_1

```
#define RFE_TX_SELECT_1 ( ( rfe_txSelect_t ) ( 1UL<<0 ) )
```

This bit selects transmitter 1.

3.7.2.49 RFE_TX_SELECT_2

```
#define RFE_TX_SELECT_2 ( ( rfe_txSelect_t ) ( 1UL<<1 ) )
```

This bit selects transmitter 2.

3.7.2.50 RFE_TX_SELECT_3

```
#define RFE_TX_SELECT_3 ( ( rfe_txSelect_t ) ( 1UL<<2 ) )
```

This bit selects transmitter 3.

3.7.2.51 RFE_TX_SELECT_4

```
#define RFE_TX_SELECT_4 ( ( rfe_txSelect_t ) ( 1UL<<3 ) )
```

This bit selects transmitter 4.

3.7.2.52 RFE_TX_SELECT_ALL

```
#define RFE_TX_SELECT_ALL ( ( rfe_txSelect_t ) 0xFUL )
```

This value selects all 4 transmitters.

3.7.2.53 RFE_TX_SELECT_NONE

```
#define RFE_TX_SELECT_NONE ( ( rfe_txSelect_t ) 0x0UL )
```

This value selects no transmitters.

3.7.2.54 RFE_UPDATE_COUNT_MAX

```
#define RFE_UPDATE_COUNT_MAX (16UL)
```

This macro defines the number of updates that can be buffered by [rfe_updateParam\(\)](#) and [rfe_updateDynamicTable\(\)](#) and pushed jointly via [rfe_updatePush\(\)](#).

3.8 RFE Abstract 2.0 - Low Level Interface

RFE Abstract 2.0 specific low level structures and functions.

Data Structures

- struct [rfeCmdClient_update_t](#)
- struct [rfe_radarCycleCount_t](#)
- struct [rfe_monitorValues_t](#)
- struct [rfe_version_t](#)

Typedefs

- typedef uint8_t [rfe_rxSelect_t](#)
- typedef uint16_t [rfe_dynParamsSelect_t](#)
- typedef uint8_t [rfe_txSelect_t](#)
- typedef uint8_t [rfe_dataOutDest_t](#)
- typedef int16_t [rfe_temperature_t](#)
- typedef uint8_t * [rfe_sysMemAddress_t](#)
- typedef uint16_t [rfe_monitorSelect_t](#)
- typedef enum [rfe_coreId_et](#) [rfe_coreId_t](#)
- typedef void(* [rfe_apiWaitForInterrupt_f](#)) (void)

Enumerations

- enum [rfeCmdClient_sharedParameter_t](#) { [rfeCmdClient_sharedParameter_state_e](#), [rfeCmdClient_sharedParameter_radarCycleCount_e](#), [rfeCmdClient_sharedParameter_chirpSequenceCount_e](#), [rfeCmdClient_sharedParameter_fw_ready_e](#) }
- enum [rfe_chirpSequenceIndex_t](#) { [rfe_chirpSequenceIndex_0_e](#), [rfe_chirpSequenceIndex_1_e](#), [rfe_chirpSequenceIndex_2_e](#), [rfe_chirpSequenceIndex_3_e](#), [rfe_chirpSequenceIndex_4_e](#), [rfe_chirpSequenceIndex_5_e](#), [rfe_chirpSequenceIndex_6_e](#), [rfe_chirpSequenceIndex_7_e](#), [rfe_chirpSequenceIndex_invalid_e](#) }
- enum [rfeCfg_param_t](#) { [rfeCfg_param_metadata_chirpSequenceConfigCount_e](#) = (uint16_t)(RFE_CFG_PARAM_SECTION_METADATA | (RFE_CFG_PARAM_SIZE_8_BIT | 0UL)), [rfeCfg_param_metadata_chirpProfileCount_e](#) = (uint16_t)(RFE_CFG_PARAM_SECTION_METADATA | (RFE_CFG_PARAM_SIZE_8_BIT | 1UL)), [rfeCfg_param_metadata_majorVersion_e](#) = (uint16_t)(RFE_CFG_PARAM_SECTION_METADATA | (RFE_CFG_PARAM_SIZE_8_BIT | 2UL)), [rfeCfg_param_metadata_minorVersion_e](#) = (uint16_t)(RFE_CFG_PARAM_SECTION_METADATA | (RFE_CFG_PARAM_SIZE_8_BIT | 3UL)), [rfeCfg_param_metadata_patchVersion_e](#) = (uint16_t)(RFE_CFG_PARAM_SECTION_METADATA | (RFE_CFG_PARAM_SIZE_8_BIT | 4UL)), [rfeCfg_param_metadata_hashVersion_e](#) = (uint16_t)(RFE_CFG_PARAM_SECTION_METADATA | (RFE_CFG_PARAM_SIZE_8_BIT | 5UL)), [rfeCfg_param_general_powerMode_e](#) = (uint16_t)(RFE_CFG_PARAM_SECTION_GENERAL | (RFE_CFG_PARAM_SIZE_8_BIT | 0UL)), [rfeCfg_param_general_radarCycleStartSignalGpio_e](#) = (uint16_t)(RFE_CFG_PARAM_SECTION_GENERAL | (RFE_CFG_PARAM_SIZE_8_BIT | 1UL)), [rfeCfg_param_general_chirpSequenceActiveSignalGpio_e](#) = (uint16_t)(RFE_CFG_PARAM_SECTION_GENERAL | (RFE_CFG_PARAM_SIZE_8_BIT | 2UL)), [rfeCfg_param_general_clkIoConfig_e](#) = (uint16_t)(RFE_CFG_PARAM_SECTION_GENERAL | (RFE_CFG_PARAM_SIZE_8_BIT | 3UL)), [rfeCfg_param_general_pdcDecimationFilter_e](#) = (uint16_t)(RFE_CFG_PARAM_SECTION_GENERAL | (RFE_CFG_PARAM_SIZE_8_BIT | 4UL)), [rfeCfg_param_general_dataOutConfig_e](#) = (uint16_t)(RFE_CFG_PARAM_SECTION_GENERAL | (RFE_CFG_PARAM_SIZE_8_BIT | 5UL)), [rfeCfg_param_general_metadataPacketSwContent_e](#) = (uint16_t)(RFE_CFG_PARAM_SECTION_GENERAL | (RFE_CFG_PARAM_SIZE_8_BIT | 6UL)), [rfeCfg_param_general_metadataPacketVirtualChannel_e](#) = (uint16_t)(RFE_CFG_PARAM_SECTION_GENERAL | (RFE_CFG_PARAM_SIZE_8_BIT | 7UL)), [rfeCfg_param_general_jumpbackTimeTicks_e](#) = (uint16_t)(RFE_CFG_PARAM_SECTION_GENERAL | (RFE_CFG_PARAM_SIZE_8_BIT | 8UL)), [rfeCfg_param_general_txCalibration_mode_e](#) = (uint16_t)(RFE_CFG_PARAM_SECTION_GENERAL | (RFE_CFG_PARAM_SIZE_8_BIT | 9UL)),

```

rfeCfg_param_monitorAndSafety_rxSatCountResetEveryChirpSequence_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | (RFE_CFG_PARAM_SIZE_8_BIT | 0UL)), rfeCfg_param_monitorAndSafety_rxSatCountResetEveryChirpSequence_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | (RFE_CFG_PARAM_SIZE_8_BIT | 1UL)), rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage1_rx2_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | (RFE_CFG_PARAM_SIZE_8_BIT | 2UL)), rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage1_rx3_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | (RFE_CFG_PARAM_SIZE_8_BIT | 3UL)), rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage1_rx4_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | (RFE_CFG_PARAM_SIZE_8_BIT | 4UL)), rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage2_rx2_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | (RFE_CFG_PARAM_SIZE_8_BIT | 6UL)), rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage2_rx3_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | (RFE_CFG_PARAM_SIZE_8_BIT | 7UL)), rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage2_rx4_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | (RFE_CFG_PARAM_SIZE_8_BIT | 8UL)), rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage1_rx2_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | (RFE_CFG_PARAM_SIZE_32_BIT | 9UL)), rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage1_rx3_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | (RFE_CFG_PARAM_SIZE_32_BIT | 17UL)), rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage1_rx4_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | (RFE_CFG_PARAM_SIZE_32_BIT | 21UL)), rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage2_rx2_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | (RFE_CFG_PARAM_SIZE_32_BIT | 29UL)), rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage2_rx3_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | (RFE_CFG_PARAM_SIZE_32_BIT | 33UL)), rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage2_rx4_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | (RFE_CFG_PARAM_SIZE_32_BIT | 37UL)), rfeCfg_param_monitorAndSafety_adcClippingCountLimit_adc1_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | (RFE_CFG_PARAM_SIZE_32_BIT | 42UL)), rfeCfg_param_monitorAndSafety_adcClippingCountLimit_adc2_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | (RFE_CFG_PARAM_SIZE_32_BIT | 46UL)), rfeCfg_param_monitorAndSafety_adcClippingCountLimit_adc3_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | (RFE_CFG_PARAM_SIZE_32_BIT | 50UL)), rfeCfg_param_monitorAndSafety_adcClippingCountLimit_adc4_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | (RFE_CFG_PARAM_SIZE_32_BIT | 54UL)), rfeCfg_param_monitorAndSafety_temperatureSensorThresholdHigh_tx12_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_32_BIT | 58UL), rfeCfg_param_monitorAndSafety_temperatureSensorThresholdHigh_tx3_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_32_BIT | 62UL), rfeCfg_param_monitorAndSafety_temperatureSensorThresholdHigh_xo_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_32_BIT | 66UL), rfeCfg_param_monitorAndSafety_temperatureSensorThresholdOver_tx3_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_32_BIT | 74UL), rfeCfg_param_monitorAndSafety_temperatureSensorThresholdOver_xo_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_32_BIT | 78UL), rfeCfg_param_monitorAndSafety_temperatureSensorThresholdUnder_tx12_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_32_BIT | 82UL), rfeCfg_param_monitorAndSafety_temperatureSensorThresholdUnder_xo_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_32_BIT | 90UL), rfeCfg_param_monitorAndSafety_thresholdValueToPromoteR1Faults_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_8_BIT | 94UL), rfeCfg_param_monitorAndSafety_frequencyForBist_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_32_BIT | 95UL), rfeCfg_param_monitorAndSafety_zeroHrRefForRxPha

```



```

= (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_16_
_BIT | 99UL) , rfeCfg_param_monitorAndSafety_zeroHrRefForRxPhaseDiffLna_rx1_rx3_e = (uint16_
t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_16_BIT | 101UL)
, rfeCfg_param_monitorAndSafety_zeroHrRefForRxPhaseDiffLna_rx1_rx4_e = (uint16_t)(RFE_CFG_
PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_16_BIT | 103UL) ,
rfeCfg_param_monitorAndSafety_rxPhaseDiffLnaThresholdTolerance_e = (uint16_t)(RFE_CFG_PARAM_
SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_16_BIT | 105UL) , rfeCfg_param_monitorAndSafety_zeroHr
= (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_16_
_BIT | 107UL) , rfeCfg_param_monitorAndSafety_zeroHrRefForRxGainDiffLna_rx1_rx3_e = (uint16_
t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_16_BIT | 109UL)
, rfeCfg_param_monitorAndSafety_zeroHrRefForRxGainDiffLna_rx1_rx4_e = (uint16_t)(RFE_CFG_
PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_16_BIT | 111UL) ,
rfeCfg_param_monitorAndSafety_rxGainDiffLnaThresholdTolerance_e = (uint16_t)(RFE_CFG_PARAM_
SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_16_BIT | 113UL) , rfeCfg_param_monitorAndSafety_zeroHr
= (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_16_
_BIT | 115UL) , rfeCfg_param_monitorAndSafety_zeroHrRefForRxPhaseDiffMixer_rx1_rx3_e = (uint16_
t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_16_BIT | 117UL)
, rfeCfg_param_monitorAndSafety_zeroHrRefForRxPhaseDiffMixer_rx1_rx4_e = (uint16_t)(RFE_CFG_
PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_16_BIT | 119UL) ,
rfeCfg_param_monitorAndSafety_rxPhaseDiffMixerThresholdTolerance_e = (uint16_t)(RFE_CFG_PARAM_
_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_16_BIT | 121UL) , rfeCfg_param_monitorAndSafety_zeroH
= (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_16_
_BIT | 123UL) , rfeCfg_param_monitorAndSafety_zeroHrRefForRxGainDiffMixer_rx1_rx3_e = (uint16_
t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_16_BIT | 125UL)
, rfeCfg_param_monitorAndSafety_zeroHrRefForRxGainDiffMixer_rx1_rx4_e = (uint16_t)(RFE_CFG_
PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_16_BIT | 127UL) ,
rfeCfg_param_monitorAndSafety_rxGainDiffMixerThresholdTolerance_e = (uint16_t)(RFE_CFG_PARAM_
SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_16_BIT | 129UL) , rfeCfg_param_monitorAndSafety_injectTe
= (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_8_BIT |
131UL) , rfeCfg_param_monitorAndSafety_checkBbd_loi_e = (uint16_t)(RFE_CFG_PARAM_SECTION_
MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_8_BIT | 132UL) , rfeCfg_param_monitorAndSafety_checkBbd_tx_e
= (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_8_BIT |
133UL) ,
rfeCfg_param_monitorAndSafety_checkBbd_rx_e = (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_
_AND_SAFETY | RFE_CFG_PARAM_SIZE_8_BIT | 134UL) , rfeCfg_param_monitorAndSafety_ppdThreshold_profile0_e
= (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_
32_BIT | 135UL) , rfeCfg_param_monitorAndSafety_ppdThreshold_profile1_e = (uint16_t)(RFE_
CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_32_BIT | 139UL) ,
rfeCfg_param_monitorAndSafety_ppdThreshold_profile2_e = (uint16_t)(RFE_CFG_PARAM_SECTION_
_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_32_BIT | 143UL) ,
rfeCfg_param_monitorAndSafety_ppdThreshold_profile3_e = (uint16_t)(RFE_CFG_PARAM_SECTION_
MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_32_BIT | 147UL) , rfeCfg_param_monitorAndSafety_ppdThreshold_pro
= (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_
32_BIT | 151UL) , rfeCfg_param_monitorAndSafety_ppdThreshold_profile5_e = (uint16_t)(RFE_
CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_32_BIT | 155UL) ,
rfeCfg_param_monitorAndSafety_ppdThreshold_profile6_e = (uint16_t)(RFE_CFG_PARAM_SECTION_
_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_32_BIT | 159UL) ,
rfeCfg_param_monitorAndSafety_ppdThreshold_profile7_e = (uint16_t)(RFE_CFG_PARAM_SECTION_
MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_32_BIT | 163UL) , rfeCfg_param_monitorAndSafety_fuSaFaultMask_0_
= (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_8_
_BIT | 167UL) , rfeCfg_param_monitorAndSafety_fuSaFaultMask_1_e = (uint16_t)(RFE_CFG_PARAM_
SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_8_BIT | 168UL) , rfeCfg_param_monitorAndSafety_fuSaFau
= (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_8_BIT |
169UL) ,
rfeCfg_param_monitorAndSafety_fuSaFaultMask_3_e = (uint16_t)(RFE_CFG_PARAM_SECTION_
MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_8_BIT | 170UL) , rfeCfg_param_monitorAndSafety_fuSaFaultMask_4_e
= (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_8_
_BIT | 171UL) , rfeCfg_param_monitorAndSafety_fuSaFaultMask_5_e = (uint16_t)(RFE_CFG_PARAM_

```

```

SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_8_BIT | 172UL), rfeCfg_param_monitorAndSafety_fuSaFaultMask_7_e
= (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_8_BIT |
173UL),
rfeCfg_param_monitorAndSafety_fuSaFaultMask_7_e = (uint16_t)(RFE_CFG_PARAM_SECTION_↵
MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_8_BIT | 174UL), rfeCfg_param_monitorAndSafety_fuSaFaultMask_8_e
= (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_8_↵
_BIT | 175UL), rfeCfg_param_monitorAndSafety_fuSaFaultMask_9_e = (uint16_t)(RFE_CFG_PARAM_↵
SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_8_BIT | 176UL), rfeCfg_param_monitorAndSafety_fuSaFaultMask_10_e
= (uint16_t)(RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_8_BIT |
177UL),
rfeCfg_param_monitorAndSafety_fuSaFaultMask_11_e = (uint16_t)(RFE_CFG_PARAM_SECTION_↵
MONITOR_AND_SAFETY | RFE_CFG_PARAM_SIZE_8_BIT | 178UL), rfeCfg_param_radarCycle_radarCycleDuration_e
= (uint16_t)(RFE_CFG_PARAM_SECTION_RADAR_CYCLE | RFE_CFG_PARAM_SIZE_32_BIT | 0UL),
rfeCfg_param_radarCycle_chirpSequenceCount_e = (uint16_t)(RFE_CFG_PARAM_SECTION_RADAR_↵
_CYCLE | RFE_CFG_PARAM_SIZE_8_BIT | 4UL), rfeCfg_param_radarCycle_chirpSequence_0_e =
(uint16_t)(RFE_CFG_PARAM_SECTION_RADAR_CYCLE | RFE_CFG_PARAM_SIZE_8_BIT | 5UL),
rfeCfg_param_radarCycle_chirpSequence_1_e = (uint16_t)(RFE_CFG_PARAM_SECTION_RADAR_↵
_CYCLE | RFE_CFG_PARAM_SIZE_8_BIT | 6UL), rfeCfg_param_radarCycle_chirpSequence_2_e =
(uint16_t)(RFE_CFG_PARAM_SECTION_RADAR_CYCLE | RFE_CFG_PARAM_SIZE_8_BIT | 7UL),
rfeCfg_param_radarCycle_chirpSequence_3_e = (uint16_t)(RFE_CFG_PARAM_SECTION_RADAR_↵
_CYCLE | RFE_CFG_PARAM_SIZE_8_BIT | 8UL), rfeCfg_param_radarCycle_chirpSequence_4_e =
(uint16_t)(RFE_CFG_PARAM_SECTION_RADAR_CYCLE | RFE_CFG_PARAM_SIZE_8_BIT | 9UL),
rfeCfg_param_radarCycle_chirpSequence_5_e = (uint16_t)(RFE_CFG_PARAM_SECTION_RADAR_↵
_CYCLE | RFE_CFG_PARAM_SIZE_8_BIT | 10UL), rfeCfg_param_radarCycle_chirpSequence_6_e =
(uint16_t)(RFE_CFG_PARAM_SECTION_RADAR_CYCLE | RFE_CFG_PARAM_SIZE_8_BIT | 11UL),
rfeCfg_param_radarCycle_chirpSequence_7_e = (uint16_t)(RFE_CFG_PARAM_SECTION_RADAR_↵
_CYCLE | RFE_CFG_PARAM_SIZE_8_BIT | 12UL), rfeCfg_param_radarCycle_chirpSequenceStartTimeOffset_0_e
= (uint16_t)(RFE_CFG_PARAM_SECTION_RADAR_CYCLE | RFE_CFG_PARAM_SIZE_32_BIT | 13UL),
rfeCfg_param_radarCycle_chirpSequenceStartTimeOffset_1_e = (uint16_t)(RFE_CFG_PARAM_SECTION_↵
_RADAR_CYCLE | RFE_CFG_PARAM_SIZE_32_BIT | 17UL), rfeCfg_param_radarCycle_chirpSequenceStartTimeOffset_2_e
= (uint16_t)(RFE_CFG_PARAM_SECTION_RADAR_CYCLE | RFE_CFG_PARAM_SIZE_32_BIT | 21UL),
rfeCfg_param_radarCycle_chirpSequenceStartTimeOffset_3_e = (uint16_t)(RFE_CFG_PARAM_↵
SECTION_RADAR_CYCLE | RFE_CFG_PARAM_SIZE_32_BIT | 25UL), rfeCfg_param_radarCycle_chirpSequenceStartTimeOffset_4_e
= (uint16_t)(RFE_CFG_PARAM_SECTION_RADAR_CYCLE | RFE_CFG_PARAM_SIZE_32_BIT | 29UL),
rfeCfg_param_radarCycle_chirpSequenceStartTimeOffset_5_e = (uint16_t)(RFE_CFG_PARAM_SECTION_↵
_RADAR_CYCLE | RFE_CFG_PARAM_SIZE_32_BIT | 33UL), rfeCfg_param_radarCycle_chirpSequenceStartTimeOffset_6_e
= (uint16_t)(RFE_CFG_PARAM_SECTION_RADAR_CYCLE | RFE_CFG_PARAM_SIZE_32_BIT | 37UL),
rfeCfg_param_radarCycle_chirpSequenceStartTimeOffset_7_e = (uint16_t)(RFE_CFG_PARAM_↵
SECTION_RADAR_CYCLE | RFE_CFG_PARAM_SIZE_32_BIT | 41UL), rfeCfg_param_radarCycle_recalibrateProfileDependent_0_e
= (uint16_t)(RFE_CFG_PARAM_SECTION_RADAR_CYCLE | RFE_CFG_PARAM_SIZE_8_BIT | 45UL),
rfeCfg_param_radarCycle_recalibrateProfileDependent_1_e = (uint16_t)(RFE_CFG_PARAM_SECTION_↵
RADAR_CYCLE | RFE_CFG_PARAM_SIZE_8_BIT | 46UL), rfeCfg_param_radarCycle_recalibrateProfileDependent_2_e
= (uint16_t)(RFE_CFG_PARAM_SECTION_RADAR_CYCLE | RFE_CFG_PARAM_SIZE_8_BIT | 47UL),
rfeCfg_param_radarCycle_recalibrateProfileDependent_3_e = (uint16_t)(RFE_CFG_PARAM_SECTION_↵
RADAR_CYCLE | RFE_CFG_PARAM_SIZE_8_BIT | 48UL), rfeCfg_param_radarCycle_recalibrateProfileDependent_4_e
= (uint16_t)(RFE_CFG_PARAM_SECTION_RADAR_CYCLE | RFE_CFG_PARAM_SIZE_8_BIT | 49UL),
rfeCfg_param_radarCycle_recalibrateProfileDependent_5_e = (uint16_t)(RFE_CFG_PARAM_SECTION_↵
RADAR_CYCLE | RFE_CFG_PARAM_SIZE_8_BIT | 50UL), rfeCfg_param_radarCycle_recalibrateProfileDependent_6_e
= (uint16_t)(RFE_CFG_PARAM_SECTION_RADAR_CYCLE | RFE_CFG_PARAM_SIZE_8_BIT | 51UL),
rfeCfg_param_radarCycle_recalibrateProfileDependent_7_e = (uint16_t)(RFE_CFG_PARAM_SECTION_↵
RADAR_CYCLE | RFE_CFG_PARAM_SIZE_8_BIT | 52UL), rfeCfg_param_radarCycle_recalibrateProfileIndependent_e
= (uint16_t)(RFE_CFG_PARAM_SECTION_RADAR_CYCLE | RFE_CFG_PARAM_SIZE_8_BIT | 53UL),
rfeCfg_param_radarCycle_bistInterval_e = (uint16_t)(RFE_CFG_PARAM_SECTION_RADAR_CYCLE |
RFE_CFG_PARAM_SIZE_8_BIT | 54UL), rfeCfg_param_chirpSequence_chirpCount_e = (uint16_t)(RFE_↵
_CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_SIZE_16_BIT | 0UL),
rfeCfg_param_chirpSequence_chirpProfileSequence_0_e = (uint16_t)(RFE_CFG_PARAM_SECTION_↵
CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_SIZE_8_BIT | 2UL), rfeCfg_param_chirpSequence_chirpProfileSequence_1_e
= (uint16_t)(RFE_CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_SIZE_8_↵

```

```

_BIT | 3UL) ,
rfeCfg_param_chirpSequence_chirpProfileSequence_2_e = (uint16_t)(RFE_CFG_PARAM_SECTION_↵
CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_SIZE_8_BIT | 4UL) , rfeCfg_param_chirpSequence_chirpProfileSequence
= (uint16_t)(RFE_CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_↵
SIZE_8_BIT | 5UL) , rfeCfg_param_chirpSequence_chirpProfileSequence_4_e = (uint16_t)(RFE_↵
CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_SIZE_8_BIT | 6UL) ,
rfeCfg_param_chirpSequence_chirpProfileSequence_5_e = (uint16_t)(RFE_CFG_PARAM_SECTION_↵
CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_SIZE_8_BIT | 7UL) ,
rfeCfg_param_chirpSequence_chirpProfileSequence_6_e = (uint16_t)(RFE_CFG_PARAM_SECTION_↵
CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_SIZE_8_BIT | 8UL) , rfeCfg_param_chirpSequence_chirpProfileSequence
= (uint16_t)(RFE_CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_SIZE_↵
_8_BIT | 9UL) , rfeCfg_param_chirpSequence_chirpProfileSequenceLength_e = (uint16_t)(RFE_↵
CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_SIZE_8_BIT | 10UL) ,
rfeCfg_param_chirpSequence_chirpProfileRepeatCount_e = (uint16_t)(RFE_CFG_PARAM_SECTION_↵
_CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_SIZE_8_BIT | 11UL) ,
rfeCfg_param_chirpSequence_dynamicUpdatesEnabled_e = (uint16_t)(RFE_CFG_PARAM_SECTION_↵
CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_SIZE_8_BIT | 12UL) , rfeCfg_param_chirpSequence_dynamicTableIndex
= (uint16_t)(RFE_CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_↵
SIZE_8_BIT | 13UL) , rfeCfg_param_chirpSequence_txPhaseRotation_ddma_e = (uint16_t)(RFE_↵
CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_SIZE_8_BIT | 14UL) ,
rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx_e = (uint16_t)(RFE_CFG_PARAM_SECTION_↵
CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_SIZE_8_BIT | 15UL) ,
rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx1_init_e = (uint16_t)(RFE_CFG_PARAM_↵
SECTION_CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_SIZE_8_BIT | 16UL) , rfeCfg_param_chirpSequence_txPhase
= (uint16_t)(RFE_CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_SIZE_↵
_8_BIT | 17UL) , rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx3_init_e = (uint16_t)(RFE_↵
_CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_SIZE_8_BIT | 18UL) ,
rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx1_update_e = (uint16_t)(RFE_CFG_PARAM_↵
SECTION_CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_SIZE_8_BIT | 19UL) ,
rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx2_update_e = (uint16_t)(RFE_CFG_PARAM_↵
SECTION_CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_SIZE_8_BIT | 20UL) , rfeCfg_param_chirpSequence_txPhase
= (uint16_t)(RFE_CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_SIZE_↵
_8_BIT | 21UL) , rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx1_mode_e = (uint16_t)(RFE_↵
_CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_SIZE_8_BIT | 22UL) ,
rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx2_mode_e = (uint16_t)(RFE_CFG_PARAM_↵
SECTION_CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_SIZE_8_BIT | 23UL) ,
rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx3_mode_e = (uint16_t)(RFE_CFG_PARAM_↵
SECTION_CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_SIZE_8_BIT | 24UL) , rfeCfg_param_chirpSequence_rxEnable
= (uint16_t)(RFE_CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_↵
SIZE_8_BIT | 25UL) , rfeCfg_param_chirpSequence_pdcNotchFilterEnable_e = (uint16_t)(RFE_↵
CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_SIZE_8_BIT | 26UL) ,
rfeCfg_param_chirpSequence_txCalibration_thresholdHigh_e = (uint16_t)(RFE_CFG_PARAM_SECTION_↵
_CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_SIZE_16_BIT | 27UL) ,
rfeCfg_param_chirpSequence_txCalibration_thresholdLow_e = (uint16_t)(RFE_CFG_PARAM_SECTION_↵
_CHIRP_SEQUENCE_CONFIG | RFE_CFG_PARAM_SIZE_16_BIT | 29UL) , rfeCfg_param_chirpProfile_effectiveSamplingFre
= (uint16_t)(RFE_CFG_PARAM_SECTION_CHIRP_PROFILE | RFE_CFG_PARAM_SIZE_8_BIT | 0UL) ,
rfeCfg_param_chirpProfile_chirpIntervalTimeTicks_e = (uint16_t)(RFE_CFG_PARAM_SECTION_CHIRP_↵
_PROFILE | RFE_CFG_PARAM_SIZE_32_BIT | 1UL) , rfeCfg_param_chirpProfile_dwellTimeTicks_e =
(uint16_t)(RFE_CFG_PARAM_SECTION_CHIRP_PROFILE | RFE_CFG_PARAM_SIZE_16_BIT | 5UL) ,
rfeCfg_param_chirpProfile_settleTimeTicks_e = (uint16_t)(RFE_CFG_PARAM_SECTION_CHIRP_↵
PROFILE | RFE_CFG_PARAM_SIZE_16_BIT | 7UL) , rfeCfg_param_chirpProfile_acquisitionTimeTicks_e
= (uint16_t)(RFE_CFG_PARAM_SECTION_CHIRP_PROFILE | RFE_CFG_PARAM_SIZE_16_BIT | 9UL) ,
rfeCfg_param_chirpProfile_resetTimeTicks_e = (uint16_t)(RFE_CFG_PARAM_SECTION_CHIRP_PROFILE
| RFE_CFG_PARAM_SIZE_16_BIT | 11UL) , rfeCfg_param_chirpProfile_centerFrequency_e = (uint16_↵
t)(RFE_CFG_PARAM_SECTION_CHIRP_PROFILE | RFE_CFG_PARAM_SIZE_32_BIT | 13UL) ,
rfeCfg_param_chirpProfile_effectiveChirpBandwidth_e = (uint16_t)(RFE_CFG_PARAM_SECTION_↵
CHIRP_PROFILE | RFE_CFG_PARAM_SIZE_32_BIT | 17UL) , rfeCfg_param_chirpProfile_chirpPIIVcoSelect_e
= (uint16_t)(RFE_CFG_PARAM_SECTION_CHIRP_PROFILE | RFE_CFG_PARAM_SIZE_8_BIT | 21UL)

```

```

, rfeCfg_param_chirpProfile_chirpSlopeDirection_e = (uint16_t)(RFE_CFG_PARAM_SECTION_CHIRP_↵
PROFILE | RFE_CFG_PARAM_SIZE_8_BIT | 22UL), rfeCfg_param_chirpProfile_chirpPllLoopFilterBandwidth_e
= (uint16_t)(RFE_CFG_PARAM_SECTION_CHIRP_PROFILE | RFE_CFG_PARAM_SIZE_8_BIT | 23UL),
rfeCfg_param_chirpProfile_txTransmissionEnable_e = (uint16_t)(RFE_CFG_PARAM_SECTION_CHIRP_↵
PROFILE | RFE_CFG_PARAM_SIZE_8_BIT | 24UL), rfeCfg_param_chirpProfile_txPower_e = (uint16_↵
_t)(RFE_CFG_PARAM_SECTION_CHIRP_PROFILE | RFE_CFG_PARAM_SIZE_16_BIT | 25UL),
rfeCfg_param_chirpProfile_txPhaseRotation_1_e = (uint16_t)(RFE_CFG_PARAM_SECTION_CHIRP_↵
PROFILE | RFE_CFG_PARAM_SIZE_8_BIT | 27UL), rfeCfg_param_chirpProfile_txPhaseRotation_2_e =
(uint16_t)(RFE_CFG_PARAM_SECTION_CHIRP_PROFILE | RFE_CFG_PARAM_SIZE_8_BIT | 28UL),
rfeCfg_param_chirpProfile_txPhaseRotation_3_e = (uint16_t)(RFE_CFG_PARAM_SECTION_CHIRP_↵
PROFILE | RFE_CFG_PARAM_SIZE_8_BIT | 29UL), rfeCfg_param_chirpProfile_txTransmissionReferenceTime_e
= (uint16_t)(RFE_CFG_PARAM_SECTION_CHIRP_PROFILE | RFE_CFG_PARAM_SIZE_8_BIT | 30UL),
rfeCfg_param_chirpProfile_txTransmissionTimeOffset_e = (uint16_t)(RFE_CFG_PARAM_SECTION_↵
CHIRP_PROFILE | RFE_CFG_PARAM_SIZE_16_BIT | 31UL), rfeCfg_param_chirpProfile_chirpFrequencyDriftSteps_e
= (uint16_t)(RFE_CFG_PARAM_SECTION_CHIRP_PROFILE | RFE_CFG_PARAM_SIZE_32_BIT | 33UL),
rfeCfg_param_chirpProfile_rxGain_e = (uint16_t)(RFE_CFG_PARAM_SECTION_CHIRP_PROFILE | RFE_↵
_CFG_PARAM_SIZE_8_BIT | 37UL), rfeCfg_param_chirpProfile_rxHpfCutOffFrequency_e = (uint16_↵
_t)(RFE_CFG_PARAM_SECTION_CHIRP_PROFILE | RFE_CFG_PARAM_SIZE_8_BIT | 38UL),
rfeCfg_param_chirpProfile_rxLpfCutOffFrequency_e = (uint16_t)(RFE_CFG_PARAM_SECTION_CHIRP_↵
_PROFILE | RFE_CFG_PARAM_SIZE_8_BIT | 39UL), rfeCfg_param_chirpProfile_virtualChannel_e =
(uint16_t)(RFE_CFG_PARAM_SECTION_CHIRP_PROFILE | RFE_CFG_PARAM_SIZE_8_BIT | 40UL),
rfeCfg_param_metadata_crc = (uint16_t)(RFE_CFG_PARAM_SECTION_CRC | (RFE_CFG_PARAM_↵
SIZE_32_BIT | 0UL)) }
• enum rfeCfg_section_t {
rfeCfg_section_metadata_e = RFE_CFG_PARAM_SECTION_METADATA | RFE_CFG_OFFSET_↵
SECTION_METADATA, rfeCfg_section_general_e = RFE_CFG_PARAM_SECTION_GENERAL | RFE_↵
_CFG_OFFSET_SECTION_GENERAL, rfeCfg_section_monitorAndSafety_e = RFE_CFG_PARAM_↵
_SECTION_MONITOR_AND_SAFETY | RFE_CFG_OFFSET_SECTION_MONITOR_AND_SAFETY,
rfeCfg_section_radarCycle_e = RFE_CFG_PARAM_SECTION_RADAR_CYCLE | RFE_CFG_OFFSET_↵
SECTION_RADAR_CYCLE,
rfeCfg_section_chirpSequenceConfig_0_e = RFE_CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG
| RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_0, rfeCfg_section_chirpProfile_0_e = RFE_↵
_CFG_PARAM_SECTION_CHIRP_PROFILE | RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_0,
rfeCfg_section_chirpSequenceConfig_1_e = RFE_CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG
| RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_1, rfeCfg_section_chirpProfile_1_e = RFE_CFG_↵
_PARAM_SECTION_CHIRP_PROFILE | RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_1,
rfeCfg_section_chirpSequenceConfig_2_e = RFE_CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG
| RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_2, rfeCfg_section_chirpProfile_2_e = RFE_↵
_CFG_PARAM_SECTION_CHIRP_PROFILE | RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_2,
rfeCfg_section_chirpSequenceConfig_3_e = RFE_CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG
| RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_3, rfeCfg_section_chirpProfile_3_e = RFE_CFG_↵
_PARAM_SECTION_CHIRP_PROFILE | RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_3,
rfeCfg_section_chirpSequenceConfig_4_e = RFE_CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG
| RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_4, rfeCfg_section_chirpProfile_4_e = RFE_↵
_CFG_PARAM_SECTION_CHIRP_PROFILE | RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_4,
rfeCfg_section_chirpSequenceConfig_5_e = RFE_CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG
| RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_5, rfeCfg_section_chirpProfile_5_e = RFE_CFG_↵
_PARAM_SECTION_CHIRP_PROFILE | RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_5,
rfeCfg_section_chirpSequenceConfig_6_e = RFE_CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG
| RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_6, rfeCfg_section_chirpProfile_6_e = RFE_↵
_CFG_PARAM_SECTION_CHIRP_PROFILE | RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_6,
rfeCfg_section_chirpSequenceConfig_7_e = RFE_CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG
| RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_7, rfeCfg_section_chirpProfile_7_e = RFE_CFG_↵
_PARAM_SECTION_CHIRP_PROFILE | RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_7,
rfeCfg_section_crc = RFE_CFG_PARAM_SECTION_CRC | RFE_CFG_OFFSET_SECTION_CRC }
• enum rfeCfg_size_t { rfeCfg_size_8_bit_e = 0U, rfeCfg_size_16_bit_e, rfeCfg_size_32_bit_e,
rfeCfg_size_invalid_e }
• enum rfe_paramUpdate_t {

```



```

rfe_paramUpdate_general_powerMode_e , rfe_paramUpdate_general_radarCycleStartSignalGpio_e ,
rfe_paramUpdate_general_chirpSequenceActiveSignalGpio_e , rfe_paramUpdate_general_clkloConfig_e
,
rfe_paramUpdate_general_pdcDecimationFilter_e , rfe_paramUpdate_general_dataOutConfig_e , rfe_paramUpdate_general_r
, rfe_paramUpdate_general_metadataPacketVirtualChannel_e ,
rfe_paramUpdate_general_jumpbackTimeTicks_e , rfe_paramUpdate_general_txCalibration_mode_e ,
rfe_paramUpdate_monitorAndSafety_rxSatCountResetEveryChirpSequence_e , rfe_paramUpdate_monitorAndSafety_rxSatur
,
rfe_paramUpdate_monitorAndSafety_rxSaturationThresholdStage1_rx2_e , rfe_paramUpdate_monitorAndSafety_rxSaturation
, rfe_paramUpdate_monitorAndSafety_rxSaturationThresholdStage1_rx4_e , rfe_paramUpdate_monitorAndSafety_rxSaturation
,
rfe_paramUpdate_monitorAndSafety_rxSaturationThresholdStage2_rx2_e , rfe_paramUpdate_monitorAndSafety_rxSaturation
, rfe_paramUpdate_monitorAndSafety_rxSaturationThresholdStage2_rx4_e , rfe_paramUpdate_monitorAndSafety_rxSaturation
,
rfe_paramUpdate_monitorAndSafety_rxSaturationCountLimitStage1_rx2_e , rfe_paramUpdate_monitorAndSafety_rxSaturation
, rfe_paramUpdate_monitorAndSafety_rxSaturationCountLimitStage1_rx4_e , rfe_paramUpdate_monitorAndSafety_rxSaturation
,
rfe_paramUpdate_monitorAndSafety_rxSaturationCountLimitStage2_rx2_e , rfe_paramUpdate_monitorAndSafety_rxSaturation
, rfe_paramUpdate_monitorAndSafety_rxSaturationCountLimitStage2_rx4_e , rfe_paramUpdate_monitorAndSafety_adcClippin
,
rfe_paramUpdate_monitorAndSafety_adcClippingCountLimit_adc1_e , rfe_paramUpdate_monitorAndSafety_adcClippingCount
, rfe_paramUpdate_monitorAndSafety_adcClippingCountLimit_adc3_e , rfe_paramUpdate_monitorAndSafety_adcClippingCou
,
rfe_paramUpdate_monitorAndSafety_temperatureSensorThresholdHigh_tx12_e , rfe_paramUpdate_monitorAndSafety_tempe
, rfe_paramUpdate_monitorAndSafety_temperatureSensorThresholdHigh_xo_e , rfe_paramUpdate_monitorAndSafety_temper
,
rfe_paramUpdate_monitorAndSafety_temperatureSensorThresholdOver_tx3_e , rfe_paramUpdate_monitorAndSafety_tempera
, rfe_paramUpdate_monitorAndSafety_temperatureSensorThresholdUnder_tx12_e , rfe_paramUpdate_monitorAndSafety_tem
,
rfe_paramUpdate_monitorAndSafety_temperatureSensorThresholdUnder_xo_e , rfe_paramUpdate_monitorAndSafety_thresho
, rfe_paramUpdate_monitorAndSafety_frequencyForBist_e , rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxPhaseDiffL
,
rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxPhaseDiffLna_rx1_rx3_e , rfe_paramUpdate_monitorAndSafety_zeroHrRef
, rfe_paramUpdate_monitorAndSafety_rxPhaseDiffLnaThresholdTolerance_e , rfe_paramUpdate_monitorAndSafety_zeroHrRef
,
rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxGainDiffLna_rx1_rx3_e , rfe_paramUpdate_monitorAndSafety_zeroHrRef
, rfe_paramUpdate_monitorAndSafety_rxGainDiffLnaThresholdTolerance_e , rfe_paramUpdate_monitorAndSafety_zeroHrRef
,
rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxPhaseDiffMixer_rx1_rx3_e , rfe_paramUpdate_monitorAndSafety_zeroH
, rfe_paramUpdate_monitorAndSafety_rxPhaseDiffMixerThresholdTolerance_e , rfe_paramUpdate_monitorAndSafety_zeroHrF
,
rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxGainDiffMixer_rx1_rx3_e , rfe_paramUpdate_monitorAndSafety_zeroHrF
, rfe_paramUpdate_monitorAndSafety_rxGainDiffMixerThresholdTolerance_e , rfe_paramUpdate_monitorAndSafety_injectTest
,
rfe_paramUpdate_monitorAndSafety_checkBbd_loi_e , rfe_paramUpdate_monitorAndSafety_checkBbd_tx_e
, rfe_paramUpdate_monitorAndSafety_checkBbd_rx_e , rfe_paramUpdate_monitorAndSafety_ppdThreshold_profile0_e
,
rfe_paramUpdate_monitorAndSafety_fuSaFaultMask_0_e , rfe_paramUpdate_monitorAndSafety_fuSaFaultMask_1_e
, rfe_paramUpdate_monitorAndSafety_fuSaFaultMask_2_e , rfe_paramUpdate_monitorAndSafety_fuSaFaultMask_3_e
,
rfe_paramUpdate_monitorAndSafety_fuSaFaultMask_4_e , rfe_paramUpdate_monitorAndSafety_fuSaFaultMask_5_e
, rfe_paramUpdate_monitorAndSafety_fuSaFaultMask_6_e , rfe_paramUpdate_monitorAndSafety_fuSaFaultMask_7_e
,
rfe_paramUpdate_monitorAndSafety_fuSaFaultMask_8_e , rfe_paramUpdate_monitorAndSafety_fuSaFaultMask_9_e
, rfe_paramUpdate_monitorAndSafety_fuSaFaultMask_10_e , rfe_paramUpdate_monitorAndSafety_fuSaFaultMask_11_e
,
rfe_paramUpdate_radarCycle_radarCycleDuration_e , rfe_paramUpdate_radarCycle_chirpSequenceCount_e
, rfe_paramUpdate_radarCycle_chirpSequence_0_e , rfe_paramUpdate_radarCycle_chirpSequence_1_e ,

```

```

rfe_paramUpdate_radarCycle_chirpSequence_2_e , rfe_paramUpdate_radarCycle_chirpSequence_3_e ,
rfe_paramUpdate_radarCycle_chirpSequence_4_e , rfe_paramUpdate_radarCycle_chirpSequence_5_e ,
rfe_paramUpdate_radarCycle_chirpSequence_6_e , rfe_paramUpdate_radarCycle_chirpSequence_7_e ,
rfe_paramUpdate_radarCycle_chirpSequenceStartTimeOffset_0_e , rfe_paramUpdate_radarCycle_chirpSequenceStartTimeC
,
rfe_paramUpdate_radarCycle_chirpSequenceStartTimeOffset_2_e , rfe_paramUpdate_radarCycle_chirpSequenceStartTimeC
, rfe_paramUpdate_radarCycle_chirpSequenceStartTimeOffset_4_e , rfe_paramUpdate_radarCycle_chirpSequenceStartTimeC
,
rfe_paramUpdate_radarCycle_chirpSequenceStartTimeOffset_6_e , rfe_paramUpdate_radarCycle_chirpSequenceStartTimeC
, rfe_paramUpdate_radarCycle_recalibrateProfileDependent_0_e , rfe_paramUpdate_radarCycle_recalibrateProfileDependent
,
rfe_paramUpdate_radarCycle_recalibrateProfileDependent_2_e , rfe_paramUpdate_radarCycle_recalibrateProfileDependent
, rfe_paramUpdate_radarCycle_recalibrateProfileDependent_4_e , rfe_paramUpdate_radarCycle_recalibrateProfileDependent
,
rfe_paramUpdate_radarCycle_recalibrateProfileDependent_6_e , rfe_paramUpdate_radarCycle_recalibrateProfileDependent
, rfe_paramUpdate_radarCycle_recalibrateProfileIndependent_e , rfe_paramUpdate_radarCycle_bistInterval_e
,
rfe_paramUpdate_chirpSequence_chirpCount_e , rfe_paramUpdate_chirpSequence_chirpProfileSequence_0_e
, rfe_paramUpdate_chirpSequence_chirpProfileSequence_1_e , rfe_paramUpdate_chirpSequence_chirpProfileSequence_2_e
,
rfe_paramUpdate_chirpSequence_chirpProfileSequence_3_e , rfe_paramUpdate_chirpSequence_chirpProfileSequence_4_e
, rfe_paramUpdate_chirpSequence_chirpProfileSequence_5_e , rfe_paramUpdate_chirpSequence_chirpProfileSequence_6_e
,
rfe_paramUpdate_chirpSequence_chirpProfileSequence_7_e , rfe_paramUpdate_chirpSequence_chirpProfileSequenceLength_e
, rfe_paramUpdate_chirpSequence_dynamicUpdatesEnabled_e , rfe_paramUpdate_chirpSequence_dynamicTableIndex_e
,
rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_e , rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_tx_0_e
, rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_tx1_init_e , rfe_paramUpdate_chirpSequence_txPhaseRotation_c
,
rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_tx3_init_e , rfe_paramUpdate_chirpSequence_txPhaseRotation_d
, rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_tx2_update_e , rfe_paramUpdate_chirpSequence_txPhaseRotati
,
rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_tx1_mode_e , rfe_paramUpdate_chirpSequence_txPhaseRotation
, rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_tx3_mode_e , rfe_paramUpdate_chirpSequence_rxEnable_e
,
rfe_paramUpdate_chirpSequence_pdcNotchFilterEnable_e , rfe_paramUpdate_chirpSequence_txCalibration_thresholdHigh_e
, rfe_paramUpdate_chirpSequence_txCalibration_thresholdLow_e , rfe_paramUpdate_chirpProfile_effectiveSamplingFrequency_e
,
rfe_paramUpdate_chirpProfile_chirpIntervalTimeTicks_e , rfe_paramUpdate_chirpProfile_dwellTimeTicks_e
, rfe_paramUpdate_chirpProfile_settleTimeTicks_e , rfe_paramUpdate_chirpProfile_acquisitionTimeTicks_e
,
rfe_paramUpdate_chirpProfile_resetTimeTicks_e , rfe_paramUpdate_chirpProfile_centerFrequency_e ,
rfe_paramUpdate_chirpProfile_effectiveChirpBandwidth_e , rfe_paramUpdate_chirpProfile_chirpPIIVcoSelect_e
,
rfe_paramUpdate_chirpProfile_chirpSlopeDirection_e , rfe_paramUpdate_chirpProfile_chirpPIILoopFilterBandwidth_e
, rfe_paramUpdate_chirpProfile_txTransmissionEnable_e , rfe_paramUpdate_chirpProfile_txPower_e ,
rfe_paramUpdate_chirpProfile_txPhaseRotation_1_e , rfe_paramUpdate_chirpProfile_txPhaseRotation_2_e
, rfe_paramUpdate_chirpProfile_txPhaseRotation_3_e , rfe_paramUpdate_chirpProfile_txTransmissionReferenceTime_e
,
rfe_paramUpdate_chirpProfile_txTransmissionTimeOffset_e , rfe_paramUpdate_chirpProfile_chirpFrequencyDriftSteps_e
, rfe_paramUpdate_chirpProfile_rxGain_e , rfe_paramUpdate_chirpProfile_rxHpfCutOffFrequency_e ,
rfe_paramUpdate_chirpProfile_rxLpfCutOffFrequency_e , rfe_paramUpdate_chirpProfile_virtualChannel_e }
• enum rfe_error_t {
rfe_error_none_e = 0UL , rfe_error_api_unresponsive_e = 1UL , rfe_error_api_unsynchronized_e = 2UL ,
rfe_error_api_operationNotAllowed_e = 3UL ,
rfe_error_api_busy_e = 4UL , rfe_error_api_invalidArgumentValue_e = 5UL , rfe_error_api_invalidConfigurationSize_e
= 6UL , rfe_error_api_invalidConfigurationParameterValue_e = 7UL ,
rfe_error_api_invalidDynamicTableSize_e = 8UL , rfe_error_api_invalidDynamicTableValue_e = 9UL ,

```

```

rfe_error_api_noUpdatesBuffered_e = 10UL , rfe_error_api_maxUpdateCountExceeded_e = 11UL ,
rfe_error_api_updateAlreadyPending_e = 12UL , rfe_error_api_invalidParameterForUpdate_e = 13UL ,
rfe_error_api_dynamicTablesNotConfigured_e = 14UL , rfe_error_api_cfgSectionMismatch_external_e =
15UL ,
rfe_error_api_cfgInvalidParameterSize_rfeDriverOnly_e = 16UL , rfe_error_api_invalidRadarCycleTimeSchedule_e
= 17UL , rfe_error_api_configParamCrossCheck_dwellSettleTimeTooShort_e = 19UL , rfe_error_api_configParamCrossCheck_
= 20UL ,
rfe_error_api_configParamCrossCheck_chirpIntervalTimeTooShort_e = 21UL , rfe_error_api_configParamCrossCheck_unconfi
= 22UL , rfe_error_api_configParamCrossCheck_unconfiguredChirpProfile_e = 23UL , rfe_error_api_configParamCrossCheck_
= 24UL ,
rfe_error_api_invalidConfigurationVersion_e = 25UL , rfe_error_api_invalidFrontendId_e = 26UL ,
rfe_error_api_function_not_implemented_e = 27UL , rfe_error_api_fsm_was_started_e = 28UL ,
rfe_error_api_invalid_num_frontends_e = 29UL , rfe_error_api_invalid_frontends_conf_e = 30UL ,
rfe_error_api_blob_crc_wrong_e = 31UL , rfe_error_cmdInterface_invalidCmdId_e = 100UL ,
rfe_error_cmdInterface_invalidCmdLength_e = 101UL , rfe_error_cmdInterface_invalidRspLength_e = 102UL
, rfe_error_cmdInterface_invalidCrc_e = 103UL , rfe_error_cmdInterface_invalidResponse_e = 104UL ,
rfe_error_cmdInterface_invalidConfigCmdSequencing_e = 105UL , rfe_error_sharedData_invalidIndex_e =
200UL , rfe_error_sharedData_redundancyCheckFailed_e = 201UL , rfe_error_rfeFuSaFault_e = RFE_↵
ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_undefined_e = 1UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_rfeInErrorState_e =
9UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_invalidPointer_e = 10UL | RFE_ERROR_↵
_RFE_INTERNAL_ERROR_BIT , rfe_error_parameterOutOfRange_e = 12UL | RFE_ERROR_RFE_↵
INTERNAL_ERROR_BIT ,
rfe_error_saturationInOperation_e = 13UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_invalidSamplesNumber_e
= 14UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_sharedMemoryAddresOverlapping_e =
15UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_addressOutOfValidSharedMemory_e =
16UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_unknown_ip_e = 17UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_1_e = 18UL |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_2_e = 19UL | RFE_ERROR_RFE_INTERNAL_↵
ERROR_BIT , rfe_error_3_e = 20UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_cmdServerInternal_no_cmd_received_e = 21UL | RFE_ERROR_RFE_INTERNAL_ERROR_↵
_BIT , rfe_error_cmdServerDft_invalidParam_e = 22UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT
, rfe_error_cmdServerInternal_undefined_cmd_handler_e = 23UL | RFE_ERROR_RFE_INTERNAL_↵
ERROR_BIT , rfe_error_cmdServerInternal_out_of_seq_e = 24UL | RFE_ERROR_RFE_INTERNAL_↵
ERROR_BIT ,
rfe_error_cmdServerInternal_resplen_wrong_e = 26UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT
, rfe_error_cmdServerInternal_cmdbuf_overflow_e = 27UL | RFE_ERROR_RFE_INTERNAL_ERROR_↵
_BIT , rfe_error_utils_memcpy_range_error_e = 29UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_notAnExternalErrorCode_e = 31UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_uninitialized_e = 32UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_invalidShiftingParam_e
= 33UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_invalidFractionBits_e = 34UL | RFE_↵
_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_nullPointer_e = 35UL | RFE_ERROR_RFE_↵
INTERNAL_ERROR_BIT ,
rfe_error_invalidValue_e = 36UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_notYetImplemented_e
= 37UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_invalidFlexIOConfig_e = 38UL | RFE_↵
ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_invalidRfeAccessCsrConfig_e = 39UL | RFE_ERROR_↵
_RFE_INTERNAL_ERROR_BIT ,
rfe_error_fit1ProcedureFailed_e = 40UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_fit2ProcedureFailed_e
= 41UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_fic1ProcedureFailed_e = 42UL | RFE_↵
ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_fic2ProcedureFailed_e = 43UL | RFE_ERROR_RFE_↵
INTERNAL_ERROR_BIT ,
rfe_error_noSpaceLeftInSharedMemory_e = 44UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT
, rfe_error_histogramWrongXAxisLimit_e = 45UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_histogramWrongOutputSize_e = 46UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_invalidADCNumberOfBits_e = 47UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_fftSamplesCountNot2Multiple_e = 48UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_fftNumberOfAveragesOutOfBounds_e = 49UL | RFE_ERROR_RFE_INTERNAL_ERROR_↵
_BIT , rfe_error_fftInvalidOutputSelected_e = 50UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT

```

```

, rfe_error_fftSamplesCountAndNumberOfAveragesNotMultiples_e = 51UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_fftNotSupported_e = 52UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_dftSamplesCountNot2Multiple_e
= 53UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_dftNumberOfAveragesOutOfBounds_e
= 54UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_dftInvalidOutputSelected_e = 55UL |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_dftSamplesCountAndNumberOfAveragesNotMultiples_e = 56UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_dftNotSupported_e = 57UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_dftOnlySingleBinSupported_e = 58UL |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_dftRealOnlySupported_e = 59UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_api_cfgInvalidParameterSize_e = 60UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_param_invalidParam_e = 61UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_param_invalidValue_e
= 62UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_api_cfgSectionMismatch_e = 63UL |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_timerInUse_e = 69UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_boundedWaitResourcesExhausted_e
= 70UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_chirpPIIVcoCalibrationFailed_e = 71UL |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_atbInvalidAtbMaster_e = 72UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_atbInvalidTrimmingSelectedResistor_e = 73UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_atbInvalidOperationalMode_e = 74UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_atbModeOfOperationNotSet_e = 75UL |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_atbCurrentMeasurementWithoutResistors_e = 76UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_atbNotSupported_e = 77UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_atbCrossSwitchNotEnabled_e
= 78UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_atbBandGapNotStarted_e = 79UL |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_atbBandGapInvalidVout_e = 80UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_atbInvalidPullDownInput_e = 81UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_atbInvalidAdcInputDiff_e
= 82UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_atbInvalidAdcSelSingleMode_e = 83UL |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_atbInvalidAdcSingleModeGround_e = 84UL |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_atbAdcNotInSingleMode_e = 85UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_atbAdcNotConfigured_e
= 86UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_atbAdcInvalidAveragingSamples_e =
87UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_hwPitNotInitialized_e = 88UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_boundedWaitNotValidResource_e = 89UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_error_api_TimerInUse_e = 90UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_error_api_boundedWaitResourcesExhausted_e
= 91UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_rx1SaturationStatusReadingFailure_e =
92UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_rx2SaturationStatusReadingFailure_e = 93UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_rx3SaturationStatusReadingFailure_e = 94UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_rx4SaturationStatusReadingFailure_e = 95UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_txInvalidSubcomponent_e = 100UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_txInvalidPowerProfile_e = 101UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_txGainBuf2AExceedRange_e
= 102UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_txGainBuf2BCExceedRange_e = 103UL |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_txPaGainCExceedRange_e = 104UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_txInvalidCalibrationFSMState_e = 105UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_txInvalidAdcConnection_e = 106UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_txIndexInvalid_e
= 107UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_txPRPhaseExceedRange_e = 108UL |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_dynamicProgrammingDuringAcquisition_e = 109UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_pcdIndexInvalid_e = 118UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_pcdSamplingFrequencyInvalid_e = 119UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_pcdProfileIndexInvalid_e = 120UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_pcdPhaseCorrectionInvalid_e = 121UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_pcdSelectedInputNotImplemented_e = 122UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_pcdCorrelatorIntegrationTimeInvalid_e = 123UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,

```



```

rfe_error_distrGroupDoesNotExist_e = 124UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_invalidRadarCycleTiming_e = 125UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_mainFsm_eventAlready
= 126UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_radarCycleFsm_stateEndTimeExceeded_e
= 127UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_mainFsm_eventNotAllowed_e = 128UL
| RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_chirpStartedTooEarly_e = 129UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_mainFsm_chirpStartInte
= 130UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_mainFsm_chirpSequenceEndInterruptTimeout_e
= 131UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_mainFsm_chirpAcquisitionStartInterruptTimeout_e
= 132UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_mainFsm_mcgenAdpllNotLocked_e = 133UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT
, rfe_error_tSenseInvalidModeOfOperation_e = 135UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_tSenseInvalidResolutionTime_e = 136UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT
, rfe_error_tSenseConversionInProgress_e = 137UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_tSenseInvalidOverUnderThresholds_e = 138UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT
, rfe_error_tSenseInvalidWarningThreshold_e = 139UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_tSenseInvalidHysteresisThreshold_e = 140UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_tSenseDivisionByZero_e = 141UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_rxAdcModuleIdUnidentified_e = 142UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_rxAdcInputUnidentified_e = 143UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_chirpPllLoopfilterC1No
= 144UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_chirpPllLoopfilterC2FastNotOk_e =
145UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_chirpPllLoopfilterC2SlowNotOk_e = 146UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_chirpPllLoopfilterC3FastNotOk_e = 147UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_chirpPllLoopfilterC3SlowNotOk_e = 148UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_chirpPllLoopfilterR1NotOk_e = 149UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_chirpPllLoopfilterR2NotOk_e = 150UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_chirpPllLoopfilterCpNotOk_e = 151UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_atbInvalidAtbAdc_e = 152UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_atbInvalidAtbAdcGainMode
= 153UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_chirpPllBandgapNotOk_e = 154UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_chirpPllHvLdoVcoNotO
= 155UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_chirpPllHvLdoCpNotOk_e = 156UL
| RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_chirpPllPdivLdoNotOk_e = 157UL | RFE_
ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_chirpPllLvLdoNotOk_e = 158UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_rxCalibrationTargetInvalid
= 159UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_rxCalibrationIfStageInvalid_e = 160UL |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_mcgen0v9AnaNotOk_e = 161UL | RFE_ERROR_
_RFE_INTERNAL_ERROR_BIT ,
rfe_error_mcgenXoDetPllClkError_e = 162UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_mcgenAdpllOutVddOv
= 163UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_mcgenAdpllLockDetNotLocked_e =
164UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_initRtmErrorReported = 165UL | RFE_
ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_initMcgen0v9AnaNotOk_e = 166UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_rxTargetSettingsCombi
= 167UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_didt_test_failure_e = 168UL | RFE_
ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_testBufferOverflow_e = 169UL | RFE_ERROR_RFE_
INTERNAL_ERROR_BIT ,
rfe_error_chirpPllVcoCalAmpOutOfRange_e = 170UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_lpuartNotActive_e = 191UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_txPrCalibrationTimeout_e
= 192UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_registerComparisonFailed_e = 200UL |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_spiTransactionError_e = 201UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_spiTransactionTimeout_e
= 202UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_hwRegister_lock_e = 203UL | RFE_
ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_startup_seq_failed_e = 204UL | RFE_ERROR_RFE_
INTERNAL_ERROR_BIT ,
rfe_error_csi2PllLockTimeOut_e = 205UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_csi2PllcalibrationTimeOut
= 206UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_fcc_e = 207UL | RFE_ERROR_RFE_
_INTERNAL_ERROR_BIT , rfe_error_ppdVoutAboveMax_e = 208UL | RFE_ERROR_RFE_INTERNAL_
ERROR_BIT ,
rfe_error_initLldoPdcPowerNotOk_e = 209UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_atbInvalidInputSource

```

```

= 500UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_atbAdcNoCalibrated_e = 501UL | RFE_
_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bistZeroHourDataNotGenerated_e = 600UL | RFE_
ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_rxbistParameterOutOfRange_e = 610UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_wdmaIncorrectStreamSelected_e = 700UL | RFE_ERROR_RFE_INTERNAL_ERROR_
BIT , rfe_error_wdmaBufferOutOfBounds_e = 701UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_wdmaBufferAddressNotAligned_e = 702UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_wdmaBurstLengthOutOfBounds_e = 703UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_wdmaWrongValueForaxiBurstModeField_e = 704UL | RFE_ERROR_RFE_INTERNAL_ERROR_
_BIT , rfe_error_wdmaBufferAddressNotSet_e = 705UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_wdmaBufferLengthNotSet_e = 706UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_wdmaBufferOverwriteModeNotSet_e = 707UL | RFE_ERROR_RFE_INTERNAL_ERROR_
BIT , rfe_error_wdmaAxiBurstLengthNotSet_e = 708UL | RFE_ERROR_RFE_INTERNAL_ERROR_
BIT , rfe_error_wdmaAxiBurstModeNotSet_e = 709UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_wdmaIncorrectInterruptSourceSelected_e = 710UL | RFE_ERROR_RFE_INTERNAL_ERROR_
BIT ,
rfe_error_wdmaIncorrectNumberOfStreamsSelected_e = 711UL | RFE_ERROR_RFE_INTERNAL_
ERROR_BIT , rfe_error_rxIfCalibrationInvalidRcCode_e = 800UL | RFE_ERROR_RFE_INTERNAL_
ERROR_BIT , rfe_error_rxInvalidRcCode_e = 801UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_rxbistInvalidSsbCode_e = 802UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_rxIfCalibrationInvalidChannel = 803UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_rxIfCalibrationFailed = 804UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_rxIfCalibrationInvalidSettings
= 805UL | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_inputOutOfRange = 0x50001U |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_funcNotExist = 0x50002U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_failToInjectCrcErr
= 0x50003U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_wrongInterface = 0x50004U |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_ipDisabled = 0x50005U | RFE_ERROR_
RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_sys_ipHasNoAtbConnection = 0x51001U | RFE_ERROR_RFE_INTERNAL_ERROR_
BIT , rfe_error_bc_sys_cannotFindCafcDacCode = 0x51002U | RFE_ERROR_RFE_INTERNAL_
ERROR_BIT , rfe_error_bc_sys_adcInvalid = 0x51005U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_sys_fileError = 0x51006U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_sys_jsonStringNoExisted = 0x51007U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_sys_txTempReadTimeout = 0x5100AU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_sys_mclkIntegrityFailed = 0x5100BU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_sys_wrongAppconfig = 0x5100CU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_sys_invalidInput = 0x5100DU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_sys_wrongProfileNo
= 0x5100FU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_sys_errorRecoveryFailed =
0x51010U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_sys_cafcIntegrityCheckLevel =
0x51011U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_sys_cafcIntegrityCheckUnlock = 0x51012U | RFE_ERROR_RFE_INTERNAL_ERROR_
_BIT , rfe_error_bc_sys_wrongDataInterfaceConfig = 0x51014U | RFE_ERROR_RFE_INTERNAL_
ERROR_BIT , rfe_error_bc_sys_error_N = 0x51016U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_sys_invalidProfile = 0x51017U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_sys_faultInjectionFail = 0x51018U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_sys_wrongMidN
= 0x51019U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_sys_spiAccessCheckFail =
0x5101AU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_sys_cafcTempReadTimeout =
0x5101BU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_sys_slaveWrongAppConfig = 0x51026U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_sys_subBandIncOutOfRange = 0x5105CU | RFE_ERROR_RFE_INTERNAL_ERROR_
_BIT , rfe_error_bc_sys_ptrAddrNull = 0x51061U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_sys_aafcAutoNotSupported = 0x51062U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_sys_rawNumSampleInvalid = 0x51063U | RFE_ERROR_RFE_INTERNAL_ERROR_
BIT , rfe_error_bc_sys_invalidResetTime = 0x51064U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_sys_totalBwOutOfRange = 0x51065U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_sys_cafcIntegrityCheckFailed = 0x51066U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT
,
rfe_error_bc_sys_restoreChirpGlobalFailed = 0x51067U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT

```

```

, rfe_error_bc_sys_regCrcIsModuleErrorSet = 0x51068U | RFE_ERROR_RFE_INTERNAL_ERROR_↵
BIT , rfe_error_bc_sys_regCrcTriggerFailed = 0x51069U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_sys_txCtrlEnableWriteCheckBase = 0x51070U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT
,
rfe_error_bc_sys_tx1CtrlEnableWriteCheck = 0x51071U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_sys_tx2CtrlEnableWriteCheck = 0x51072U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_sys_tx1Tx2CtrlEnableWriteCheck = 0x51073U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT
, rfe_error_bc_sys_tx3CtrlEnableWriteCheck = 0x51074U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_sys_tx1Tx3CtrlEnableWriteCheck = 0x51075U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT
, rfe_error_bc_sys_tx2Tx3CtrlEnableWriteCheck = 0x51076U | RFE_ERROR_RFE_INTERNAL_ERROR_↵
_BIT , rfe_error_bc_sys_tx1Tx2Tx3CtrlEnableWriteCheck = 0x51077U | RFE_ERROR_RFE_INTERNAL_↵
_ERROR_BIT , rfe_error_bc_sys_loiInBallBreakDetected = 0x51078U | RFE_ERROR_RFE_INTERNAL_↵
_ERROR_BIT ,
rfe_error_bc_sys_loiOutBallBreakDetected = 0x51079U | RFE_ERROR_RFE_INTERNAL_ERROR_↵
_BIT , rfe_error_bc_algDummyError = 0x52001U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_spiWriteReadCrcError = 0x53001U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_spiWriteCheckFail = 0x53002U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_readFail = 0x53003U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_ccSysConfNotSet
= 0x60001U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_ccPtrAddrNull = 0x60002U |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_lvdsInvalidInput = 0x61001U | RFE_ERROR_↵
_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_lvdsPtrAddrNull = 0x61002U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_lvdsSamplesShiftIn
= 0x61003U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_lvdsClkEdgeInvalid = 0x61004U
| RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_lvdsAdcClkFromInvalid = 0x61005U | RFE_↵
_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_lvdsFreqCounterDeviation = 0x61006U | RFE_ERROR_RFE_INTERNAL_ERROR_↵
BIT , rfe_error_bc_lvdsCountPeriodInvalid = 0x61007U | RFE_ERROR_RFE_INTERNAL_ERROR_↵
BIT , rfe_error_bc_lvdsFreqCountTimeout = 0x61008U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_lvdsrestoreFreqCounter = 0x61009U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_mipiDeviceReadyTimeout = 0x62001U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT
, rfe_error_bc_mipiFreqCountTimeout = 0x62002U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_mipiFreqCntOutOfRange = 0x62003U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_mipiPtrAddrNull = 0x62004U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_mipiRestoreFreqCounter = 0x62005U | RFE_ERROR_RFE_INTERNAL_ERROR_↵
BIT , rfe_error_bc_tePrSafetyDelay = 0x64001U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_tePrCalibDelay = 0x64002U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_teRxActiveDelay
= 0x64003U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_teHpFResetDelay = 0x64004U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_teDcPowOnDelay
= 0x64005U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_teDcSafetyDelay = 0x64006U |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_teDdmaSetting = 0x64007U | RFE_ERROR_↵
_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_teDdmaMode = 0x64008U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_teJumpBack
= 0x64009U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_teTxRxGrpDelay = 0x6400AU |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_teFineGrpDelay = 0x6400BU | RFE_ERROR_↵
_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_teProfileList = 0x6400CU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_tePrCalTimeout
= 0x6400DU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_teBiasDelay = 0x6400EU |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_teInputNull = 0x6400FU | RFE_ERROR_↵
RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_tePrfRepeatCnt = 0x64010U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_teChirpSeqInterval
= 0x64011U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_teSettleTime = 0x64012U |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_teOutputSampRate = 0x64013U | RFE_↵
_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_teChirpInterval = 0x64014U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_teVirtualChannel
= 0x64015U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_teDwllTime = 0x64016U | RFE_↵
_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_teDelayFromRefpt = 0x64017U | RFE_ERROR_↵
RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_teFreqCntTimeout = 0x64018U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_teChirpSequence

```

```

= 0x64019U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_teChirpTrgMode = 0x6401AU |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_tePhaseOutOfRange = 0x6401EU | RFE_↵
ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_teHpfResetWrongTriggerPoint = 0x6401FU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_teTxCalEnableWriteCheckBase = 0x64020U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT
, rfe_error_bc_teTx1CalEnableWriteCheck = 0x64021U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_teTx2CalEnableWriteCheck = 0x64022U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_teTx1Tx2CalEnableWriteCheck = 0x64023U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT
, rfe_error_bc_teTx3CalEnableWriteCheck = 0x64024U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_teTx1Tx3CalEnableWriteCheck = 0x64025U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_teTx2Tx3CalEnableWriteCheck = 0x64026U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_teTx1Tx2Tx3CalEnableWriteCheck = 0x64027U | RFE_ERROR_RFE_INTERNAL_ERROR_↵
_BIT , rfe_error_bc_teChirpEnableWriteCheckFailed = 0x64028U | RFE_ERROR_RFE_INTERNAL_↵
_ERROR_BIT , rfe_error_bc_scProfile = 0x65001U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_scOperatingFreqOutOfRange = 0x65002U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT
,
rfe_error_bc_scPtrAddrNull = 0x65003U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_scInputFreqDriftInvalid
= 0x65004U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_scUserBwOutOfRange =
0x65005U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_scRampTimeOutOfRange =
0x65006U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_scResetTimeOutOfRange = 0x65007U | RFE_ERROR_RFE_INTERNAL_ERROR_↵
BIT , rfe_error_bc_cafcFreqCntTimeout = 0x66001U | RFE_ERROR_RFE_INTERNAL_ERROR_↵
BIT , rfe_error_bc_cafcSbOutOfRange = 0x66002U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_cafcInputOutOfRange = 0x66003U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_cafcBwOutOfRange = 0x66004U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_cafcConfigLoopF
= 0x66005U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_cafcLoopBwOutOfRange =
0x66006U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_cafcPtrAddrNull = 0x66007U |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_cafcWrongProfileId = 0x66008U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_cafcAafcKvCodVo
= 0x66009U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_cafcVcoCalibAafcFail =
0x6600AU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_cafcVcoCalibKvCodVdtFail =
0x6600BU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_cafcNoStoreInProfileSelected = 0x6600CU | RFE_ERROR_RFE_INTERNAL_ERROR_↵
BIT , rfe_error_bc_cafcDvdtCountWrong = 0x6600DU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT
, rfe_error_bc_cafcKvcoCountStatusErr = 0x6600EU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_cafcWrongKvco = 0x66010U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_cafcWrongAcquisitionDuration = 0x66011U | RFE_ERROR_RFE_INTERNAL_ERROR_↵
_BIT , rfe_error_bc_adcCalibrationFailed = 0x67001u | RFE_ERROR_RFE_INTERNAL_ERROR_↵
BIT , rfe_error_bc_adcCalibrationTimeout = 0x67002U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_adcCalibrationCheckNotDone = 0x67003U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_gbiasDummy = 0x69001U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_loiLutGainCodeOutOfR
= 0x6A003U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_loiCalBusy = 0x6A005U | RFE_↵
_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_loiCalError = 0x6A006U | RFE_ERROR_RFE_↵
INTERNAL_ERROR_BIT ,
rfe_error_bc_loiPtrAddrNull = 0x6A007U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_mclkPlIcalibrationFaile
= 0x6C003U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_mclkPlIcalibrationNotFinished
= 0x6C004U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_mclkMspcCalibrationError =
0x6C006U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_mclkMspcTimeout = 0x6C007U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_mclkPtrAddrNull
= 0x6C00AU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_rxInputOutOfRange = 0x6_↵
D001U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_rxCalBusy = 0x6D002U | RFE_↵
ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_rx1CalBusy = 0x6D003U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_rx2CalBusy
= 0x6D004U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_rx3CalBusy = 0x6D005U |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_rx4CalBusy = 0x6D006U | RFE_ERROR_↵
RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_rxCalError = 0x6D007U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_rx1CalError
= 0x6D008U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_rx2CalError = 0x6D009U |

```

```

RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_rx1Rx2CalError = 0x6D00AU | RFE_ERROR_↵
_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_rx3CalError = 0x6D00BU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_rx1Rx3CalError
= 0x6D00CU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_rx2Rx3CalError = 0x6D00DU
| RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_rx1Rx2Rx3CalError = 0x6D00EU | RFE_↵
ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_rx4CalError = 0x6D00FU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_rx1Rx4CalError
= 0x6D010U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_rx2Rx4CalError = 0x6D011U
| RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_rx1Rx2Rx4CalError = 0x6D012U | RFE_↵
ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_rx3Rx4CalError = 0x6D013U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_rx1Rx3Rx4CalError
= 0x6D014U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_rx2Rx3Rx4CalError = 0x6D015U
| RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_rx1Rx2Rx3Rx4CalError = 0x6D016U | RFE_↵
_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_rxPrtAddrNull = 0x6D017U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_rxWrngCalTargetOffset
= 0x6D018U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_rxFusaEnableWriteCheckBase
= 0x6D020U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_rx1FusaEnableWritecheck =
0x6D021U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_rx2FusaEnableWritecheck = 0x6D022U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_rx1Rx2FusaEnableWritecheck = 0x6D023U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT
, rfe_error_bc_rx3FusaEnableWritecheck = 0x6D024U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_rx1Rx3FusaEnableWritecheck = 0x6D025U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_rx2Rx3FusaEnableWritecheck = 0x6D026U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_rx1Rx2Rx3FusaEnableWritecheck = 0x6D027U | RFE_ERROR_RFE_INTERNAL_ERROR_↵
BIT , rfe_error_bc_rx4FusaEnableWritecheck = 0x6D028U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_rx1Rx4FusaEnableWritecheck = 0x6D029U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_rx2Rx4FusaEnableWritecheck = 0x6D02AU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_rx1Rx2Rx4FusaEnableWritecheck = 0x6D02BU | RFE_ERROR_RFE_INTERNAL_ERROR_↵
BIT , rfe_error_bc_rx3Rx4FusaEnableWritecheck = 0x6D02CU | RFE_ERROR_RFE_INTERNAL_ERROR_↵
_BIT , rfe_error_bc_rx1Rx3Rx4FusaEnableWritecheck = 0x6D02DU | RFE_ERROR_RFE_INTERNAL_↵
ERROR_BIT ,
rfe_error_bc_rx2Rx3Rx4FusaEnableWritecheck = 0x6D02EU | RFE_ERROR_RFE_INTERNAL_ERROR_↵
_BIT , rfe_error_bc_rx1Rx2Rx3Rx4FusaEnableWritecheck = 0x6D02FU | RFE_ERROR_RFE_INTERNAL_↵
_ERROR_BIT , rfe_error_bc_rxBbdEnableWritecheckBase = 0x6D030U | RFE_ERROR_RFE_INTERNAL_↵
_ERROR_BIT , rfe_error_bc_rx1BbdEnableWritecheck = 0x6D031U | RFE_ERROR_RFE_INTERNAL_↵
ERROR_BIT ,
rfe_error_bc_rx2BbdEnableWritecheck = 0x6D032U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_rx1Rx2BbdEnableWritecheck = 0x6D033U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT
, rfe_error_bc_rx3BbdEnableWritecheck = 0x6D034U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_rx1Rx3BbdEnableWritecheck = 0x6D035U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_rx2Rx3BbdEnableWritecheck = 0x6D036U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_rx1Rx2Rx3BbdEnableWritecheck = 0x6D037U | RFE_ERROR_RFE_INTERNAL_ERROR_↵
BIT , rfe_error_bc_rx4BbdEnableWritecheck = 0x6D038U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_rx1Rx4BbdEnableWritecheck = 0x6D039U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_rx2Rx4BbdEnableWritecheck = 0x6D03AU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_rx1Rx2Rx4BbdEnableWritecheck = 0x6D03BU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT
, rfe_error_bc_rx3Rx4BbdEnableWritecheck = 0x6D03CU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_rx1Rx3Rx4BbdEnableWritecheck = 0x6D03DU | RFE_ERROR_RFE_INTERNAL_ERROR_↵
BIT ,
rfe_error_bc_rx2Rx3Rx4BbdEnableWritecheck = 0x6D03EU | RFE_ERROR_RFE_INTERNAL_ERROR_↵
BIT , rfe_error_bc_rx1Rx2Rx3Rx4BbdEnableWritecheck = 0x6D03FU | RFE_ERROR_RFE_INTERNAL_↵
ERROR_BIT , rfe_error_bc_txInputOutOfRange = 0x71001U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT
, rfe_error_bc_txProcAdcTimeout = 0x71002U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_txSensorFitFail = 0x71003U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_txCalBusy
= 0x71004U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_tx1CalBusy = 0x71005U | RFE_↵
_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_tx2CalBusy = 0x71006U | RFE_ERROR_RFE_↵
INTERNAL_ERROR_BIT ,
rfe_error_bc_tx3CalBusy = 0x71007U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_txCalError

```



```

= 0x71008U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_tx1CalError = 0x71009U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_tx2CalError = 0x7100AU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_tx1Tx2CalError = 0x7100BU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_tx3CalError = 0x7100CU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_tx1Tx3CalError = 0x7100DU |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_tx2Tx3CalError = 0x7100EU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_tx1Tx2Tx3CalError = 0x7100FU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_txSafetyBusy = 0x71010U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_tx1SafetyBusy = 0x71011U |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_tx2SafetyBusy = 0x71012U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_tx3SafetyBusy = 0x71013U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_txSafetyError = 0x71014U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_tx1SafetyError = 0x71015U |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_tx2SafetyError = 0x71016U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_tx1Tx2SafetyError = 0x71017U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_tx3SafetyError = 0x71018U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_tx1Tx3SafetyError = 0x71019U |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_tx2Tx3SafetyError = 0x7101AU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_tx1Tx2Tx3SafetyError = 0x7101BU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_txPtrAddrNull = 0x7101CU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_txNoStoreInProfileSelected = 0x7101DU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_txPoutVgaReduceTxNotSupported
= 0x7101EU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_txRfMinCodeOutOfRange = 0x7101FU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_txFusaEnableWriteCheckBase = 0x71020U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_tx1FusaEnableWriteCheck = 0x71021U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_tx2FusaEnableWriteCheck = 0x71022U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_tx1Tx2FusaEnableWriteCheck = 0x71023U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_tx3FusaEnableWriteCheck = 0x71024U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_tx1Tx3FusaEnableWriteCheck = 0x71025U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_tx2Tx3FusaEnableWriteCheck = 0x71026U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_tx1Tx2Tx3FusaEnableWriteCheck = 0x71027U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_txCalibBiasCurrWriteFailed = 0x71028U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_txCalibTimeout = 0x71029U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_txBbdEnableWritecheckBase = 0x71030U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
,
rfe_error_bc_tx1BbdEnableWritecheck = 0x71031U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_tx2BbdEnableWritecheck = 0x71032U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_tx1tx2BbdEnableWritecheck = 0x71033U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_tx3BbdEnableWritecheck = 0x71034U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_tx1Tx3BbdEnableWritecheck = 0x71035U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_tx2Tx3BbdEnableWritecheck = 0x71036U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_tx1Tx2Tx3BbdEnableWritecheck = 0x71037U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_ssbSignalNotExistOnSelectedAtb = 0x74001U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_ssbInputOutOfRange = 0x74002U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_gldoSnsError = 0x75001U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_gldoSnsForceError = 0x75002U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_gldoSnsResetError = 0x75003U
| RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_atbBistAdcTimeout = 0x76001U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_atbWrongAtbNoSns = 0x76002U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_otpFailToRead = 0x77001U |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_otpCrcCheckFailure = 0x77002U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_otplsDisabled = 0x77003U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_otpOtplsNotLoaded = 0x77004U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_otpAllMemDisabled = 0x77006U
| RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_ismInvalidInput = 0x078001U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_ismPtrAddrNull = 0x078002U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_ismStartupSeqConfig = 0x078003U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT

```

```

= 0x078003U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_ismInputModeToggleTestFailed
= 0x078004U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_ismOutputModeToggleTestFailed
= 0x078005U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_ismFitErrorN_High = 0x078006U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_ismFitResetState
= 0x078007U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_ismFitResetErrorFailed =
0x078008U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_ismFitForceStateChangeFailed =
0x078009U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_ismFitForceErrorFailed = 0x07800AU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_ismFitFailed = 0x07800BU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_ismFitStatusFailed
= 0x07800CU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_bc_ismFitTxStatusFailed =
0x07800DU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_ismFitRxStatusFailed = 0x07800EU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_ismFitAdcStatusFailed = 0x07800FU | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_ismFitChirpStatusFailed = 0x078010U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_ismFitMclkLolsmOtpCcStatusFailed = 0x078011U | RFE_ERROR_RFE_INTERNAL_
ERROR_BIT ,
rfe_error_bc_ismFitSsbSerGbGldoAAtbStatusFailed = 0x078012U | RFE_ERROR_RFE_INTERNAL_
ERROR_BIT , rfe_error_bc_ismCalBusy = 0x078013U | RFE_ERROR_RFE_INTERNAL_ERROR_
_BIT , rfe_error_bc_ismCalError = 0x078014U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_ismCalSuccess = 0x078015U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_bc_ismDisableLbToggleTestFailed = 0x078016U | RFE_ERROR_RFE_INTERNAL_ERROR_
_BIT , rfe_error_sys_spi_wrg_unit_id = 0x07900AU | RFE_ERROR_RFE_INTERNAL_ERROR_
_BIT , rfe_error_sys_spi_invalid_len = 0x079014U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_sys_timer_cm7_init = 0x079001U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_sys_FE_chip_es2_not_supported = 0x079002U | RFE_ERROR_RFE_INTERNAL_ERROR_
_BIT , rfe_error_ic_wrg_input_params_e = 0x7a000U | RFE_ERROR_RFE_INTERNAL_ERROR_
_BIT , rfe_error_rxbist_frequency_fail = 0x7B000U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_rxbist_missing_signal = 0x7B001U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT ,
rfe_error_rxbist_lna_gain_fail_e = 0x7B002U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_rxbist_lna_phase_fail_
= 0x7B003U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_rxbist_mixer_gain_fail_e = 0x7B004U | RFE_ERROR_RFE_INTERNAL_ERROR_BIT , rfe_error_rxbist_mixer_phase_fail_e = 0x7B005U |
RFE_ERROR_RFE_INTERNAL_ERROR_BIT }
• enum rfe_fuSaFault_t {
rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_tx3_e = 0x0000U , rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_tx2_e
= 0x0001U , rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_tx1_e = 0x0002U , rfe_fuSaFault_R2_sr72_pr_tx3_e
= 0x0003U ,
rfe_fuSaFault_R2_sr72_pr_tx2_e = 0x0004U , rfe_fuSaFault_R2_sr72_pr_tx1_e = 0x0005U , rfe_fuSaFault_R2_sr65_sr67_sup
= 0x0006U , rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_tx2_e = 0x0007U ,
rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_tx1_e = 0x0100U , rfe_fuSaFault_R2_sr69_rf_level_max_tx3_e
= 0x0101U , rfe_fuSaFault_R2_sr69_rf_level_max_tx2_e = 0x0102U , rfe_fuSaFault_R2_sr69_rf_level_max_tx1_e
= 0x0103U ,
rfe_fuSaFault_R2_sr69_rf_level_min_tx3_e = 0x0104U , rfe_fuSaFault_R2_sr69_rf_level_min_tx2_e =
0x0105U , rfe_fuSaFault_R2_sr69_rf_level_min_tx1_e = 0x0106U , rfe_fuSaFault_R2_sr27_bb_tx3_e =
0x0107U ,
rfe_fuSaFault_R2_sr27_bb_tx2_e = 0x0200U , rfe_fuSaFault_R2_sr27_bb_tx1_e = 0x0201U , rfe_fuSaFault_R2_sr65_sr67_sup
= 0x0202U , rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_rx3_e = 0x0203U ,
rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_rx2_e = 0x0204U , rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_rx1_e
= 0x0205U , rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_rx4_e = 0x0206U , rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_
= 0x0207U ,
rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_rx2_e = 0x0300U , rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_rx1_e
= 0x0301U , rfe_fuSaFault_R2_sr47_lo_level_max_rx4_e = 0x0302U , rfe_fuSaFault_R2_sr47_lo_level_max_rx3_e
= 0x0303U ,
rfe_fuSaFault_R2_sr47_lo_level_max_rx2_e = 0x0304U , rfe_fuSaFault_R2_sr47_lo_level_max_rx1_e =
0x0305U , rfe_fuSaFault_R2_sr47_lo_level_min_rx4_e = 0x0306U , rfe_fuSaFault_R2_sr47_lo_level_min_rx3_e
= 0x0307U ,
rfe_fuSaFault_R2_sr47_lo_level_min_rx2_e = 0x0400U , rfe_fuSaFault_R2_sr47_lo_level_min_rx1_e =
0x0401U , rfe_fuSaFault_R2_sr9_bb_rx4_e = 0x0402U , rfe_fuSaFault_R2_sr9_bb_rx3_e = 0x0403U ,
rfe_fuSaFault_R2_sr9_bb_rx2_e = 0x0404U , rfe_fuSaFault_R2_sr9_bb_rx1_e = 0x0405U , rfe_fuSaFault_R2_sr46_aafc_cal_t

```

```

= 0x0406U , rfe_fuSaFault_R2_sr46_aafc_chirp_e = 0x0407U ,
rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_chirp_e = 0x0500U , rfe_fuSaFault_R2_sr65_sr67_supply_low_vco_1v8_chirp_e
= 0x0501U , rfe_fuSaFault_R2_sr65_sr67_supply_low_pll_1v8_chirp_e = 0x0502U , rfe_fuSaFault_R2_sr7_interface_stuck_chirp_e
= 0x0503U ,
rfe_fuSaFault_R2_sr42_vco_freq_480_chirp_e = 0x0504U , rfe_fuSaFault_R2_sr45_level_max_chirp_e =
0x0505U , rfe_fuSaFault_R2_sr45_level_min_chirp_e = 0x0506U , rfe_fuSaFault_R2_sr43_unlock_chirp_e
= 0x0507U ,
rfe_fuSaFault_R2_sr7_lock_step_chirp_e = 0x0600U , rfe_fuSaFault_R2_sr48_level_max_loi_e = 0x0601U
, rfe_fuSaFault_R2_sr48_level_min_loi_e = 0x0602U , rfe_fuSaFault_R2_sr19_rd_spi_crc_e = 0x0603U ,
rfe_fuSaFault_R2_sr19_wr_spi_crc_e = 0x0604U , rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_loi_e =
0x0605U , rfe_fuSaFault_R2_sr17_bb_lo_in_loi_e = 0x0606U , rfe_fuSaFault_R2_sr18_bb_lo_out_loi_e =
0x0607U ,
rfe_fuSaFault_R2_sr11_xo_no_clock_mclk_e = 0x0700U , rfe_fuSaFault_R2_sr36_mspc_mclk_e = 0x0701U
, rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_mclk_e = 0x0702U , rfe_fuSaFault_R2_sr39_dig_freq_mclk_e
= 0x0703U ,
rfe_fuSaFault_R2_sr35_pll_lock_mclk_e = 0x0704U , rfe_fuSaFault_R2_sr57_supply_high_1v1_gldo_e =
0x0705U , rfe_fuSaFault_R2_sr59_supply_low_1v1_gldo_e = 0x0706U , rfe_fuSaFault_R2_sr61_supply_high_1v8_gldo_e
= 0x0707U ,
rfe_fuSaFault_R2_sr63_supply_low_1v8_gldo_e = 0x0800U , rfe_fuSaFault_R2_sr65_67_supply_low_1v8_ser_e
= 0x0801U , rfe_fuSaFault_R2_sr65_67_supply_low_1v1_csi2_e = 0x0802U , rfe_fuSaFault_R2_sr65_67_supply_low_1v1_ser_e
= 0x0803U ,
rfe_fuSaFault_R2_sr65_67_supply_low_1v8_gbias_e = 0x0804U , rfe_fuSaFault_R2_sr65_67_supply_low_1v8_ssb_e
= 0x0805U , rfe_fuSaFault_R2_sr65_67_supply_low_1v8_adc34_e = 0x0806U , rfe_fuSaFault_R2_sr65_67_supply_low_1v8_a
= 0x0807U ,
rfe_fuSaFault_R2_sr34_cal_adc4_e = 0x0900U , rfe_fuSaFault_R2_sr34_cal_adc3_e = 0x0901U ,
rfe_fuSaFault_R2_sr34_cal_adc2_e = 0x0902U , rfe_fuSaFault_R2_sr34_cal_adc1_e = 0x0903U ,
rfe_fuSaFault_R2_sr3_reg_crc_e = 0x0904U , rfe_fuSaFault_R2_sr31_bist_lna_gaindiff_sw_e = 0x905U ,
rfe_fuSaFault_R2_sr31_bist_lna_phasediff_sw_e = 0x906U , rfe_fuSaFault_R2_sr31_bist_mixer_gaindiff_sw_e
= 0x907U ,
rfe_fuSaFault_R2_sr31_bist_mixer_phasediff_sw_e = 0xA00U , rfe_fuSaFault_R2_sr44_cafc_integrity_level_sw_e
= 0xA01U , rfe_fuSaFault_R2_sr44_cafc_integrity_unlock_sw_e = 0xA02U , rfe_fuSaFault_R2_sr44_cafc_integrity_sw_e
= 0xA03U ,
rfe_fuSaFault_R2_sr70_tx_integrity_sw_e = 0xA04U , rfe_fuSaFault_R2_sr72_tx_pr_cal_sw_e = 0xA05U ,
rfe_fuSaFault_R2_sr72_tx1_pr_cal_sw_e = 0xA06U , rfe_fuSaFault_R2_sr72_tx2_pr_cal_sw_e = 0xA07U ,
rfe_fuSaFault_R2_sr37_pll_level_mclk_e = 0xB00U , rfe_fuSaFault_R2_sr72_tx3_pr_cal_sw_e = 0xB01U ,
rfe_fuSaFault_R2_sr14_disable_lb_toggle_test_sw_e = 0xB02U , rfe_fuSaFault_R2_sr13_input_mode_toggle_test_sw_e
= 0xB03U ,
rfe_generic_sw_e = 0xB04U }
• enum rfe_fusaFault_R1_t {
rfe_fuSaFault_R1_tempErrorFlagTx1_e = 0x0000U , rfe_fuSaFault_R1_tempErrorFlagTx2_e = 0x0001U
, rfe_fuSaFault_R1_tempErrorFlagTx3_e = 0x0002U , rfe_fuSaFault_R1_rfLevelLowErrorFlagTx1_e =
0x0003U ,
rfe_fuSaFault_R1_rfLevelLowErrorFlagTx2_e = 0x0004U , rfe_fuSaFault_R1_rfLevelLowErrorFlagTx3_e =
0x0005U , rfe_fuSaFault_R1_rfLevelHighErrorFlagTx1_e = 0x0006U , rfe_fuSaFault_R1_rfLevelHighErrorFlagTx2_e
= 0x0007U ,
rfe_fuSaFault_R1_rfLevelHighErrorFlagTx3_e = 0x0008U , rfe_fuSaFault_R1_loLevelLowErrorFlagRx1_e =
0x0009U , rfe_fuSaFault_R1_loLevelLowErrorFlagRx2_e = 0x000AU , rfe_fuSaFault_R1_loLevelLowErrorFlagRx3_e
= 0x000BU ,
rfe_fuSaFault_R1_loLevelLowErrorFlagRx4_e = 0x000CU , rfe_fuSaFault_R1_loLevelHighErrorFlagRx1_e =
0x000DU , rfe_fuSaFault_R1_loLevelHighErrorFlagRx2_e = 0x000EU , rfe_fuSaFault_R1_loLevelHighErrorFlagRx3_e
= 0x000FU ,
rfe_fuSaFault_R1_loLevelHighErrorFlagRx4_e = 0x0010U , rfe_fuSaFault_R1_tempErrorFlagChirp_e =
0x0011U , rfe_fuSaFault_R1_vtuneLowErrorFlagChirp_e = 0x0012U , rfe_fuSaFault_R1_vtuneHighErrorFlagChirp_e
= 0x0013U ,
rfe_fuSaFault_R1_levelLowErrorFlagChirp_e = 0x0014U , rfe_fuSaFault_R1_levelHighErrorFlagChirp_e =
0x0015U , rfe_fuSaFault_R1_pllFreqCalErrorFlagMc_e = 0x0016U , rfe_fuSaFault_R1_loiLevelLowErrorFlagInterface_e
= 0x0017U ,
rfe_fuSaFault_R1_loiLevelHighErrorFlagInterface_e = 0x0018U , rfe_fuSaFault_R1_hardClippingFlagPersistentAdc1_e

```



```

= 0x0019U , rfe_fuSaFault_R1_hardClippingFlagPersistentAdc2_e = 0x001AU , rfe_fuSaFault_R1_hardClippingFlagPersistentAdc3_e = 0x001BU ,
rfe_fuSaFault_R1_hardClippingFlagPersistentAdc4_e = 0x001CU , rfe_fuSaFault_R1_size_e = 0x001DU }
• enum rfe_powerMode_t { rfe_powerMode_inactive_e , rfe_powerMode_active_e }
• enum rfe_bistInterval_t { rfe_bistInterval_everyRadarCycle_e = 0U , rfe_bistInterval_onceIn2RadarCycles_e = 1U , rfe_bistInterval_onceIn4RadarCycles_e = 2U , rfe_bistInterval_none_e = 3U }
• enum rfeSwBist_InjectToneMode_t { TEF82XX_both_Ina_mixer = 0 , TEF82XX_only_mixer , TEF82XX_alternate_Ina_mixer , InjectToneMode_Invalid }
    Structure to hold the RX BIST test modes.
• enum rfe_temperatureSensorIndex_t { rfe_temperatureSensorIndex_tx1_e , rfe_temperatureSensorIndex_tx2_e , rfe_temperatureSensorIndex_tx3_e , rfe_temperatureSensorIndex_chirp_e }
• enum rfe_chirpProfileIndex_t { rfe_chirpProfileIndex_0_e , rfe_chirpProfileIndex_1_e , rfe_chirpProfileIndex_2_e , rfe_chirpProfileIndex_3_e , rfe_chirpProfileIndex_4_e , rfe_chirpProfileIndex_5_e , rfe_chirpProfileIndex_6_e , rfe_chirpProfileIndex_7_e , rfe_chirpProfileIndex_invalid_e }
• enum rfe_fuSaFaultMaskIndex_t { rfe_fuSaFaultMaskIndex_0_e , rfe_fuSaFaultMaskIndex_1_e , rfe_fuSaFaultMaskIndex_2_e , rfe_fuSaFaultMaskIndex_3_e , rfe_fuSaFaultMaskIndex_4_e , rfe_fuSaFaultMaskIndex_5_e , rfe_fuSaFaultMaskIndex_6_e , rfe_fuSaFaultMaskIndex_7_e , rfe_fuSaFaultMaskIndex_8_e , rfe_fuSaFaultMaskIndex_9_e , rfe_fuSaFaultMaskIndex_10_e , rfe_fuSaFaultMaskIndex_11_e }
• enum rfe_txIndex_t { rfe_txIndex_1_e , rfe_txIndex_2_e , rfe_txIndex_3_e , rfe_txIndex_4_e , rfe_txIndex_invalid_e }
• enum rfe_PRPCGenMode_e { rfe_PR_NO_CHANGE_e = 0x00u , rfe_PR_REPLACE_MSB_WITH_QPSK = 0x01u , rfe_PR_ADD_QPSK_TO_MSB = 0x02u , rfe_PR_INVALID }
• enum rfe_rxIndex_t { rfe_rxIndex_1_e , rfe_rxIndex_2_e , rfe_rxIndex_3_e , rfe_rxIndex_4_e }
• enum rfe_rxStage_t { rfe_rxStage_1_e , rfe_rxStage_2_e }
• enum rfe_rxSaturationThreshold_t { rfe_rxSaturationThreshold_min13_05dB_e , rfe_rxSaturationThreshold_min12_18dB_e , rfe_rxSaturationThreshold_min11_31dB_e , rfe_rxSaturationThreshold_min10_44dB_e , rfe_rxSaturationThreshold_min9_57dB_e , rfe_rxSaturationThreshold_min8_70dB_e , rfe_rxSaturationThreshold_min7_83dB_e , rfe_rxSaturationThreshold_min6_96dB_e , rfe_rxSaturationThreshold_min5_22dB_e , rfe_rxSaturationThreshold_min4_35dB_e , rfe_rxSaturationThreshold_min3_48dB_e , rfe_rxSaturationThreshold_min2_61dB_e , rfe_rxSaturationThreshold_min1_74dB_e , rfe_rxSaturationThreshold_min0_87dB_e , rfe_rxSaturationThreshold_0dB_e }
• enum rfe_rxClippingReset_t { rfe_rx1Reset_e , rfe_rx2Reset_e , rfe_rx3Reset_e , rfe_rx4Reset_e , rfe_rx1234Reset_e }
• enum rfe_txPowerErrorFlag_t { rfe_txPowerMinErrorFlag , rfe_txPowerLowErrorFlag , rfe_txPowerHighErrorFlag , rfe_txPowerMaxErrorFlag }
• enum rfe_rxGain_t { rfe_rxGain_25dB_e , rfe_rxGain_28dB_e , rfe_rxGain_31dB_e , rfe_rxGain_34dB_e , rfe_rxGain_37dB_e , rfe_rxGain_40dB_e , rfe_rxGain_43dB_e , rfe_rxGain_46dB_e , rfe_rxGain_invalid_e }
• enum rfe_rxHpfCutOffFrequency_t { rfe_rxHpfCutOffFrequency_200kHz_e , rfe_rxHpfCutOffFrequency_300kHz_e , rfe_rxHpfCutOffFrequency_400kHz_e , rfe_rxHpfCutOffFrequency_800kHz_e , rfe_rxHpfCutOffFrequency_1600kHz_e , rfe_rxHpfCutOffFrequency_3200kHz_e , rfe_rxHpfCutOffFrequency_6400kHz_e , rfe_rxHpfCutOffFrequency_invalid_e }
• enum rfe_rxLpfCutOffFrequency_t { rfe_rxLpfCutOffFrequency_12P5MHz_e , rfe_rxLpfCutOffFrequency_15MHz_e , rfe_rxLpfCutOffFrequency_20MHz_e , rfe_rxLpfCutOffFrequency_25MHz_e , rfe_rxLpfCutOffFrequency_40MHz_e , rfe_rxLpfCutOffFrequency_invalid_e }

```

- enum `rfe_chirpSlopeDirection_t` { `rfe_chirpSlopeDirection_falling_e`, `rfe_chirpSlopeDirection_rising_e` }
 - enum `rfe_chirpPllVco_t` { `rfe_chirpPllVco_1GHz_e`, `rfe_chirpPllVco_2GHz_e`, `rfe_chirpPllVco_4GHz_e` }
 - enum `rfe_virtualChannel_t` { `rfe_virtualChannel_0_e`, `rfe_virtualChannel_1_e`, `rfe_virtualChannel_2_e`, `rfe_virtualChannel_3_e`, `rfe_virtualChannel_invalid_e` }
 - enum `rfeTef82XXRfBistTest_t` { `TEF82XX_RFBIST_MIXER_TEST` = 0U, `TEF82XX_RFBIST_LNA_TEST`, `TEF82XX_RFBIST_TESTS` }
- RF BIST Test Type.*
- enum `rfe_state_t` { `rfe_state_busy_e`, `rfe_state_initialized_e`, `rfe_state_configured_e`, `rfe_state_radarCycleIdle_e`, `rfe_state_testContinuousWaveTransmission_e`, `rfe_state_fuSaFaultRecovery_e`, `rfe_state_fuSaFault_e` }
 - enum `rfe_testParam_t` { `rfe_testParam_outputDataTestPattern_e`, `rfe_testParam_keepTxTransmissionEnabled_e`, `rfe_testParam_chirpPllTestPinEnable_e`, `rfe_testParam_enableClockRetuning_e` }
 - enum `rfe_coreId_t` { `rfe_coreId_none_e` = 0U, `rfe_coreId_appM7_e`, `rfe_coreId_a53_e`, `rfe_coreId_max` }
 - enum `rfe_eventsIRQ_t` { `RFE_EVENTS_IRQ_NONE` = 0U, `RFE_EVENTS_IRQ_STATE_CHANGE` = (1U << 0U), `RFE_EVENTS_IRQ_RADAR_CYCLE_START` = (1U << 1U), `RFE_EVENTS_IRQ_CHIRP_SEQUENCE_CHANGE` = (1U << 2U), `RFE_EVENTS_IRQ_MAX` = (1U << 3U) }
 - enum `rfe_apiIRQ_t` { `RFE_API_IRQ_NONE` = 0UL, `RFE_API_IRQ_CONFIGURE` = (1UL << 0UL), `RFE_API_IRQ_RADAR_CYCLE_START` = (1UL << 1UL), `RFE_API_IRQ_RADAR_CYCLE_STOP` = (1UL << 2UL), `RFE_API_IRQ_GET_FUSA_FAULTS` = (1UL << 3UL), `RFE_API_IRQ_GET_TIME` = (1UL << 4UL), `RFE_API_IRQ_GET_VERSION` = (1UL << 5UL), `RFE_API_IRQ_MONITOR_READ` = (1UL << 6UL), `RFE_API_IRQ_GET_NEXT_RADAR_CYCLE_START_TIME` = (1UL << 7UL), `RFE_API_IRQ_SET_NEXT_RADAR_CYCLE_START_TIME` = (1UL << 8UL), `RFE_API_IRQ_UPDATE_PUSH` = (1UL << 9UL), `RFE_API_IRQ_CONTINUOUS_WAVE_TRANSMISSION_START` = (1UL << 10UL), `RFE_API_IRQ_CONTINUOUS_WAVE_TRANSMISSION_STOP` = (1UL << 11UL), `RFE_API_IRQ_TEST_SET_PARAM` = (1UL << 12UL), `RFE_API_IRQ_GET_FUSA_FAULT_STATISTICS` = (1UL << 13UL), `RFE_API_IRQ_GET_BIST_ZERO_HITS` = (1UL << 14UL), `RFE_API_IRQ_TEST_GET_INTERNAL_ERROR` = (1UL << 15UL), `RFE_API_IRQ_SET_FE_ID` = (1UL << 16UL), `RFE_API_IRQ_GET_FE_ID` = (1UL << 17UL), `RFE_API_IRQ_REG_DUMP` = (1UL << 18UL), `RFE_API_IRQ_MAX` = (1UL << RFE_API_IRQ_MAX_NUM) }

Functions

- void `rfeCmdClient_init` (void *pCmdIfBase, volatile `rfe_error_t` *rfe___error___pointer)
Initialize the RFE-Fw command client.
- void `rfeCmdClient_call` (rfeCmdServer_cmdId_t cmdId, volatile `rfe_error_t` *rfe___error___pointer)
Calls a remote command.
- void `rfeCmdClient_cmdParamStart` (void)
Starts the preparation for the call of a remote RFE-Fw function.
- void `rfeCmdClient_cmdParamAddBuffer` (const void *pInput, uint32_t byteCount, volatile `rfe_error_t` *rfe___error___pointer)
Adds an arbitrary block of data as parameter to a function.
- static void `rfeCmdClient_cmdParamAddUInt8` (uint8_t value, volatile `rfe_error_t` *rfe___error___pointer)
Adds an uint8_t value as parameter to a remote function.
- static void `rfeCmdClient_cmdParamAddUInt16` (uint16_t value, volatile `rfe_error_t` *rfe___error___pointer)
Adds an uint16_t value as parameter to a remote function.
- static void `rfeCmdClient_cmdParamAddUInt32` (uint32_t value, volatile `rfe_error_t` *rfe___error___pointer)
Adds an uint32_t value as parameter to a remote function.
- static void `rfeCmdClient_cmdParamAddInt8` (int8_t value, volatile `rfe_error_t` *rfe___error___pointer)
Adds an int8_t value as parameter to a remote function.
- static void `rfeCmdClient_cmdParamAddInt16` (int16_t value, volatile `rfe_error_t` *rfe___error___pointer)

- Adds an int16_t value as parameter to a remote function.*

 - static void `rfeCmdClient_cmdParamAddInt32` (int32_t value, volatile `rfe_error_t` *rfe___error___pointer)

Adds an int32_t value as parameter to a remote function.

 - static void `rfeCmdClient_cmdParamAddBool` (bool value, volatile `rfe_error_t` *rfe___error___pointer)

Adds a bool value as parameter to a remote function.

 - void `rfeCmdClient_cmdParamAddPointer` (void *pValue, volatile `rfe_error_t` *rfe___error___pointer)

Adds a pointer value as parameter to a remote function.

 - void `rfeCmdClient_cmdRespGetBuf` (void *pOutput, uint32_t byteCount, volatile `rfe_error_t` *rfe___error___pointer)

Reads an arbitrary block of data from the response buffer.

 - uint8_t `rfeCmdClient_cmdRespGetUint8` (volatile `rfe_error_t` *rfe___error___pointer)

Reads an uint8 value from the response buffer.

 - uint16_t `rfeCmdClient_cmdRespGetUint16` (volatile `rfe_error_t` *rfe___error___pointer)

Reads an uint16 value from the response buffer.

 - uint32_t `rfeCmdClient_cmdRespGetUint32` (volatile `rfe_error_t` *rfe___error___pointer)

Reads an uint32 value from the response buffer.

 - int8_t `rfeCmdClient_cmdRespGetInt8` (volatile `rfe_error_t` *rfe___error___pointer)

Reads an int8 value from the response buffer.

 - int16_t `rfeCmdClient_cmdRespGetInt16` (volatile `rfe_error_t` *rfe___error___pointer)

Reads an int16 value from the response buffer.

 - int32_t `rfeCmdClient_cmdRespGetInt32` (volatile `rfe_error_t` *rfe___error___pointer)

Reads an int32 value from the response buffer.

 - bool `rfeCmdClient_cmdRespGetBool` (volatile `rfe_error_t` *rfe___error___pointer)

Reads a bool value from the response buffer.

 - uint16_t `rfeCmdClient_sharedDataRead` (uint32_t idx, volatile `rfe_error_t` *rfe___error___pointer)

Reads a SharedData location.

 - uint16_t `rfeCmdClient_getCurrentRespLen` (void)

Return the length of the last received answer from the Fw.

 - void `rfeCmdClient_checkNotBusy` (volatile `rfe_error_t` *rfe___error___pointer)

This function checks whether RFE-Fw is not busy.

 - void `rfeCmdClient_configureInterrupt` (`rfe_apiIRQ_t` rfeAPIs, `rfe_apiWaitForInterrupt_f` apiWaitForInterrupt)

Configure Fw interrupt parameters The function align the internal API driver parameters to the application request.

 - void `rfe_sync` (volatile `rfe_error_t` *rfe___error___pointer)

This function synchronizes the rfeDriver on M7-0 core with the RFE-Fw running on M7-1 core.

 - void `rfe_configure` (uint8_t *pConfig, `rfe_sysMemAddress_t` dynamicTableAddress, volatile `rfe_error_t` *rfe___error___pointer)

This function fully configures the RFE-Fw.

 - void `rfe_radarCycleStart` (uint16_t radarCycleCount, bool isScheduled, uint32_t startTime, volatile `rfe_error_t` *rfe___error___pointer)

This function starts the configured radar cycles.

 - uint16_t `rfe_radarCycleStop` (volatile `rfe_error_t` *rfe___error___pointer)

This function stops the active radar cycles.

 - `rfe_state_t` `rfe_getState` (volatile `rfe_error_t` *rfe___error___pointer)

This function returns the RFE-Fw state.

 - `rfe_radarCycleCount_t` `rfe_getRadarCycleCount` (volatile `rfe_error_t` *rfe___error___pointer)

This function returns the radar cycle and chirp sequence count.

 - void `rfe_getFuSaFaults` (uint8_t *pFuSaR1R2FaultList, volatile `rfe_error_t` *rfe___error___pointer)

This function provides the RFE-Fw FuSa faults which are set in RFE-Fw FCCU.

 - void `rfe_getFuSaFaultStatistics` (uint16_t *pR1FaultPromotedToR2, `rfe_radarCycleCount_t` *pRadarCycleCount, uint8_t *pFuSaR1FaultCountList, volatile `rfe_error_t` *rfe___error___pointer)

This function provides the fault statistics for R1 faults.

- void [rfe_getBistZeroHourReferenceData](#) ([rfeCfg_rxBistReferenceData_t](#) *pRxReferenceData, volatile [rfe_error_t](#) *rfe___error___pointer)
This function returns RX BIST measurement details to be used as zero hour reference data.
- uint32_t [rfe_getTime](#) (volatile [rfe_error_t](#) *rfe___error___pointer)
This function returns the RFE-Fw time.
- [rfe_version_t](#) [rfe_getVersion](#) (volatile [rfe_error_t](#) *rfe___error___pointer)
This function returns version and variant details of the RFE-Fw hardware and firmware.
- [rfe_radarCycleCount_t](#) [rfe_monitorRead](#) ([rfe_monitorSelect_t](#) monitorSelect, [rfe_monitorValues_t](#) *pMonitorValues, volatile [rfe_error_t](#) *rfe___error___pointer)
This function will read-out one or more RFE-Fw monitors.
- uint32_t [rfe_getNextRadarCycleStartTime](#) (uint16_t *pRadarCycleIndex, volatile [rfe_error_t](#) *rfe___error___pointer)
This function returns the scheduled absolute start time of the next radar cycle.
- uint16_t [rfe_setNextRadarCycleStartTime](#) (uint32_t startTime, volatile [rfe_error_t](#) *rfe___error___pointer)
This function updates the start time of the next radar cycle.
- void [rfe_updateBegin](#) (void)
This function begins a RFE configuration update operation.
- void [rfe_updateParam](#) ([rfeCfg_section_t](#) section, [rfe_paramUpdate_t](#) param, uint32_t value, volatile [rfe_error_t](#) *rfe___error___pointer)
This function start continuous wave transmission.
- void [rfe_updateDynamicTable](#) (uint8_t dynamicTableIndex, [rfe_dynParamsSelect_t](#) paramsSelect, volatile [rfe_error_t](#) *rfe___error___pointer)
This function stops continuous wave transmission.
- uint16_t [rfe_updatePush](#) (volatile [rfe_error_t](#) *rfe___error___pointer)
This function sets a test parameter.
- void [rfe_testContinuousWaveTransmissionStart](#) ([rfe_chirpProfileIndex_t](#) profileIndex, volatile [rfe_error_t](#) *rfe___error___pointer)
This function start continuous wave transmission.
- void [rfe_testContinuousWaveTransmissionStop](#) (volatile [rfe_error_t](#) *rfe___error___pointer)
This function stops continuous wave transmission.
- void [rfe_testSetParam](#) ([rfe_testParam_t](#) testParam, uint32_t value, volatile [rfe_error_t](#) *rfe___error___pointer)
This function sets a test parameter.
- uint32_t [rfe_testGetInternalError](#) (volatile [rfe_error_t](#) *rfe___error___pointer)
This function provides the Internal error occurred.
- void [rfe_getRegisterDump](#) (uint8_t frontEndPos, [rfe_moduleDumpSelect_t](#) moduleSelection, [rfe_registerDumpValues_t](#) *dumpArrayPtr, uint32_t *dumpArrayPtrLen, volatile [rfe_error_t](#) *rfe___error___pointer)
Function to get the complete registry status on one or many front-end(s)
- void [rfe_setFrontEnd](#) (uint8_t frontEndPos, volatile [rfe_error_t](#) *rfe___error___pointer)
Function to change the associated Front-End "subject" in a cascaded setup.
- uint8_t [rfe_getFrontEnd](#) (volatile [rfe_error_t](#) *rfe___error___pointer)
Function to get the associated Front-End "subject" in a cascaded setup.
- void [rfe_configureInterrupt](#) ([rfe_coreId_t](#) coreId, [rfe_eventsIRQ_t](#) events, [rfe_apiIRQ_t](#) rfeAPIs, [rfe_apiWaitForInterrupt_t](#) apiWaitForInterrupt, volatile [rfe_error_t](#) *rfe___error___pointer)
This function configure the RFE firmware interrupt.
- uint32_t [rfeCrc_calBuf](#) (void *bufPtr, uint32_t len)

3.8.1 Detailed Description

RFE Abstract 2.0 specific low level structures and functions.

This enumerated type defines indices for the chirp sequence configuration array.

3.8.2 Typedef Documentation

3.8.2.1 rfe_apiWaitForInterrupt_f

```
typedef void(* rfe_apiWaitForInterrupt_f) (void)
```

This type defines RFE API wait for interrupt function pointer.

3.8.2.2 rfe_coreId_t

```
typedef enum rfe_coreId_et rfe_coreId_t
```

This enumerated type defines which core will receive IRQs from RFE firmware

Note

Core Id 0: appM7 core Core Id 1: a53 core

3.8.2.3 rfe_dataOutDest_t

```
typedef uint8_t rfe_dataOutDest_t
```

This type defines the data out destinations of the packer.

3.8.2.4 rfe_dynParamsSelect_t

```
typedef uint16_t rfe_dynParamsSelect_t
```

These defines cover a selection of dynamic parameters.

3.8.2.5 rfe_monitorSelect_t

```
typedef uint16_t rfe_monitorSelect_t
```

This types defines a selection of RFE-Fw monitors.

3.8.2.6 rfe_rxSelect_t

```
typedef uint8_t rfe_rxSelect_t
```

3.8.2.7 rfe_sysMemAddress_t

```
typedef uint8_t* rfe_sysMemAddress_t
```

This type defines an address to system memory.

3.8.2.8 rfe_temperature_t

```
typedef int16_t rfe_temperature_t
```

This type defines temperature as SQ6 in degrees Celsius. SQ6 is a signed (2's complement) fixed point number format with 6 fractional bits. E.g. 1 degrees Celsius = 0x0040.

3.8.2.9 rfe_txSelect_t

```
typedef uint8_t rfe_txSelect_t
```

This type defines a selection of transmitters.

3.8.3 Enumeration Type Documentation

3.8.3.1 rfe_apiIRQ_t

```
enum rfe_apiIRQ_t
```

This type defines which RFE APIs shall trigger an IRQ from RFE firmware upon completion

Enumerator

RFE_API_IRQ_NONE	
RFE_API_IRQ_CONFIGURE	This bitmask disables all commands completion IRQs
RFE_API_IRQ_RADAR_CYCLE_START	This bit enables IRQ on rfe_configure completion
RFE_API_IRQ_RADAR_CYCLE_STOP	This bit enables IRQ on rfe_radarCycleStart completion
RFE_API_IRQ_GET_FUSA_FAULTS	This bit enables IRQ on rfe_radarCycleStop completion
RFE_API_IRQ_GET_TIME	This bit enables IRQ on rfe_getFuSaFaults completion
RFE_API_IRQ_GET_VERSION	This bit enables IRQ on rfe_getTime completion
RFE_API_IRQ_MONITOR_READ	This bit enables IRQ on rfe_getVersion completion

Enumerator

RFE_API_IRQ_GET_NEXT_RADAR_CYCLE_↔ START_TIME	This bit enables IRQ on rfe_monitorRead completion
RFE_API_IRQ_SET_NEXT_RADAR_CYCLE_↔ START_TIME	This bit enables IRQ on rfe_getNextRadarCycleStartTime completion
RFE_API_IRQ_UPDATE_PUSH	This bit enables IRQ on rfe_setNextRadarCycleStartTime completion
RFE_API_IRQ_CONTINUOUS_WAVE_↔ TRANSMISSION_START	This bit enables IRQ on rfe_updatePush completion
RFE_API_IRQ_CONTINUOUS_WAVE_↔ TRANSMISSION_STOP	This bit enables IRQ on rfe_continuousWaveTransmissionStart completion
RFE_API_IRQ_TEST_SET_PARAM	This bit enables IRQ on rfe_continuousWaveTransmissionStop completion
RFE_API_IRQ_GET_FUSA_FAULT_STATISTICS	This bit enables IRQ on rfe_testSetParam completion
RFE_API_IRQ_GET_BIST_ZERO_HOUR_↔ REFERENCE_DATA	This bit enables IRQ on rfe_getFuSaFaultStatistics completion
RFE_API_IRQ_TEST_GET_INTERNAL_ERROR	This bit enables IRQ on rfe_getBistZeroHourReferenceData completion
RFE_API_IRQ_SET_FE_ID	This bit enables IRQ on rfe_testGetInternalError completion
RFE_API_IRQ_GET_FE_ID	This bit enables IRQ on rfe_setFrontEnd completion
RFE_API_IRQ_REG_DUMP	This bit enables IRQ on rfe_getFrontEnd completion
RFE_API_IRQ_MAX	This bit enables IRQ on rfe_registerDump completion

3.8.3.2 rfe_bistInterval_t

```
enum rfe_bistInterval_t
```

This enumerated type defines the BIST interval.

Enumerator

rfe_bistInterval_everyRadarCycle_e	Perform BIST after the last chirp sequence in every radar cycle.
rfe_bistInterval_onceIn2RadarCycles↔ _e	Perform BIST after the last chirp sequence once in two radar cycles.
rfe_bistInterval_onceIn4RadarCycles↔ _e	Perform BIST after the last chirp sequence once in four radar cycles.
rfe_bistInterval_none_e	Do not perform BIST in radar Cycle.

3.8.3.3 rfe_chirpPllVco_t

```
enum rfe_chirpPllVco_t
```

This enumerated type defines chirp PLL voltage-controlled oscillators.

Enumerator

rfe_chirpPllVco_1GHz↔ _e	
rfe_chirpPllVco_2GHz↔ _e	
rfe_chirpPllVco_4GHz↔ _e	

3.8.3.4 rfe_chirpProfileIndex_t

```
enum rfe_chirpProfileIndex_t
```

This enumerated type defines the chirp profile indices.

Enumerator

rfe_chirpProfileIndex_0_e	
rfe_chirpProfileIndex_1_e	
rfe_chirpProfileIndex_2_e	
rfe_chirpProfileIndex_3_e	
rfe_chirpProfileIndex_4_e	
rfe_chirpProfileIndex_5_e	
rfe_chirpProfileIndex_6_e	
rfe_chirpProfileIndex_7_e	
rfe_chirpProfileIndex_invalid↔ _e	

3.8.3.5 rfe_chirpSequenceIndex_t

```
enum rfe_chirpSequenceIndex_t
```

Enumerator

rfe_chirpSequenceIndex_0_e	
rfe_chirpSequenceIndex_1_e	

Enumerator

rfe_chirpSequenceIndex_2_e	
rfe_chirpSequenceIndex_3_e	
rfe_chirpSequenceIndex_4_e	
rfe_chirpSequenceIndex_5_e	
rfe_chirpSequenceIndex_6_e	
rfe_chirpSequenceIndex_7_e	
rfe_chirpSequenceIndex_invalid↔ _e	

3.8.3.6 rfe_chirpSlopeDirection_t

```
enum rfe_chirpSlopeDirection_t
```

This enumerated type defines slope directions of a chirp.

Enumerator

rfe_chirpSlopeDirection_falling↔ _e	
rfe_chirpSlopeDirection_rising_e	

3.8.3.7 rfe_coreId_et

```
enum rfe_coreId_et
```

This enumerated type defines which core will receive IRQs from RFE firmware

Note

Core Id 0: appM7 core Core Id 1: a53 core

Enumerator

rfe_coreId_none_e	
rfe_coreId_appM7↔ _e	
rfe_coreId_a53_e	
rfe_coreId_max	

3.8.3.8 rfe_error_t

enum `rfe_error_t`

All defined return error codes.

Enumerator

<code>rfe_error_none_e</code>	No error occurred
<code>rfe_error_api_unresponsive_e</code>	RFE-Fw is unresponsive.
<code>rfe_error_api_unsynchronized_e</code>	Operation is not allowed if the <code>rfeDriver</code> is not synchronized with the RFE-Fw.
<code>rfe_error_api_operationNotAllowed_e</code>	Operation is not allowed in this state.
<code>rfe_error_api_busy_e</code>	RFE-Fw is busy.
<code>rfe_error_api_invalidArgumentValue_e</code>	API function argument has an invalid value.
<code>rfe_error_api_invalidConfigurationSize_e</code>	The configuration blob has an invalid size.
<code>rfe_error_api_invalidConfigurationParameterValue_e</code>	The configuration blob contains a parameter that has an invalid value.
<code>rfe_error_api_invalidDynamicTableSize_e</code>	The dynamic table blob has an invalid size.
<code>rfe_error_api_invalidDynamicTableValue_e</code>	The dynamic table contains an entry that has an invalid value.
<code>rfe_error_api_noUpdatesBuffered_e</code>	No updates were buffered using <code>rfe_updateParam()</code> or <code>rfe_updateDynamicTable()</code> .
<code>rfe_error_api_maxUpdateCountExceeded_e</code>	Update count is exceeding <code>RFE_UPDATE_COUNT_MAX</code> .
<code>rfe_error_api_updateAlreadyPending_e</code>	An update is already pending. Only 1 update per radar cycle is supported.
<code>rfe_error_api_invalidParameterForUpdate_e</code>	Invalid parameter provided for update.
<code>rfe_error_api_dynamicTablesNotConfigured_e</code>	Dynamic tables have not been configured.
<code>rfe_error_api_cfgSectionMismatch_external_e</code>	Given <code>rfeCfg_section_t</code> section is not matching with the given parameter.
<code>rfe_error_api_cfgInvalidParameterSize_rfeDriver↔ Only_e</code>	Given <code>rfeCfg_param_t</code> is containing an invalid parameter size.
<code>rfe_error_api_invalidRadarCycleTimeSchedule_e</code>	The configured radar cycle time schedule is invalid.
<code>rfe_error_api_configParamCrossCheck_dwellSettle↔ TimeTooShort_e</code>	Configuration parameter cross-check failed: The sum of <code>dwellTimeTicks</code> and <code>settleTimeTicks</code> of all chirp profiles used in <code>chirpProfileSequence</code> should be larger than 120 ticks (i.e. 3 microseconds) if <code>dynamicUpdatesEnabled</code> is true.
<code>rfe_error_api_configParamCrossCheck_acq↔ JumpbackResetTimeTooShort_e</code>	Configuration parameter cross-check failed: The sum of <code>acquisitionTimeTicks</code> , <code>jumpbackTimeTicks</code> and <code>resetTimeTicks</code> of all chirp profiles used in <code>chirpProfileSequence</code> should be larger than 200 ticks (i.e. 5 microseconds) if <code>dynamicUpdatesEnabled</code> is true.
<code>rfe_error_api_configParamCrossCheck_chirp↔ IntervalTimeTooShort_e</code>	Configuration parameter cross-check failed: <code>chirpIntervalTimeTicks</code> should be larger than the sum of <code>dwellTimeTicks</code> , <code>settleTimeTicks</code> , <code>acquisitionTimeTicks</code> , <code>jumpbackTimeTicks</code> , and <code>resetTimeTicks</code> .
<code>rfe_error_api_configParamCrossCheck_↔ unconfiguredChirpSequence_e</code>	Configuration parameter cross-check failed: chirp sequences contains a chirp sequence that has not been configured.

Enumerator

rfe_error_api_configParamCrossCheck_↔ unconfiguredChirpProfile_e	Configuration parameter cross-check failed: chirpProfileSequence contains a chirp profile that has not been configured.
rfe_error_api_configParamCrossCheck_↔ unconfiguredDynamicTable_e	Configuration parameter cross-check failed: dynamicTableIndex refers to a dynamic table that has not been configured.
rfe_error_api_invalidConfigurationVersion_e	The configuration blob version is incompatible with the RFE-Fw
rfe_error_api_invalidFrontendId_e	Front End ID is invalid
rfe_error_api_function_not_implemented_e	API function not implemented yet to be removed after all functions will be implemented
rfe_error_api_fsm_was_started_e	Number of connected front ends cannot be set after the FSM start
rfe_error_api_invalid_num_frontends_e	Invalid number of connected front ends
rfe_error_api_invalid_frontends_conf_e	Invalid front ends configuration, e.g. multiple single TEF82xx
rfe_error_api_blob_crc_wrong_e	The blob has a wrong CRC value and could be corrupted
rfe_error_cmdInterface_invalidCmdId_e	Invalid command id.
rfe_error_cmdInterface_invalidCmdLength_e	Invalid command length.
rfe_error_cmdInterface_invalidRspLength_e	Invalid command response Length.
rfe_error_cmdInterface_invalidCrc_e	Command has an invalid CRC.
rfe_error_cmdInterface_invalidResponse_e	Response is not matching command.
rfe_error_cmdInterface_invalidConfigCmd↔ Sequencing_e	Invalid sequencing of configuration commands.
rfe_error_sharedData_invalidIndex_e	Invalid shared data index.
rfe_error_sharedData_redundancyCheckFailed_e	Shared data redundancy check failed.
rfe_error_rfeFuSaFault_e	RFE FuSa fault occurred, check rfe_getFuSaFault()
rfe_error_undefined_e	Undefined error
rfe_error_rfeInErrorState_e	
rfe_error_invalidPointer_e	Error due to invalid pointer
rfe_error_parameterOutOfRange_e	Error because parameter is out of allowed range
rfe_error_saturationInOperation_e	Error due to a saturation in an arithmetic operation.
rfe_error_invalidSamplesNumber_e	Error due to an invalid number of samples (not greater than zero)
rfe_error_sharedMemoryAdresOverlapping_e	
rfe_error_addressOutOfValidSharedMemory_e	
rfe_error_unknown_ip_e	Unknwon IP selected.
rfe_error_1_e	Error ...
rfe_error_2_e	Error ...
rfe_error_3_e	Error ...
rfe_error_cmdServerInternal_no_cmd_received_e	A command was tried to be handled, but no command is available.
rfe_error_cmdServerDft_invalidParam_e	An invalid parameter was received over the command interface.
rfe_error_cmdServerInternal_undefined_cmd_↔ handler_e	An undefined command was received.

Enumerator

rfe_error_cmdServerInternal_out_of_seq_e	A command counter out of sequence was received. This indicates loss of a command.
rfe_error_cmdServerInternal_resplen_wrong_e	A function implementation returned an unexpected amount of result data.
rfe_error_cmdServerInternal_cmdbuf_overflow_e	A function made the command or response buffer overflow.
rfe_error_utils_memcpy_range_error_e	memcpy got dangerous pointers provided
rfe_error_notAnExternalErrorCode_e	
rfe_error_uninitialized_e	Error because operation is not allowed when rfe is uninitialized
rfe_error_invalidShiftingParam_e	Error indicating that value for arithmetic shift is invalid
rfe_error_invalidFractionBits_e	
rfe_error_nullPointer_e	
rfe_error_invalidValue_e	
rfe_error_notYetImplemented_e	Error because not yet implemented
rfe_error_invalidFlexIOConfig_e	Error because of invalid FlexIO configuration
rfe_error_invalidRfeAccessCsrConfig_e	Error because of invalid RFE Access CSR configuration
rfe_error_fit1ProcedureFailed_e	Error because of failed Fault Injection Test FIT 1 procedure
rfe_error_fit2ProcedureFailed_e	Error because of failed Fault Injection Test FIT 2 procedure
rfe_error_fic1ProcedureFailed_e	Error because of failed Functional Integrity Check FIC 1 procedure
rfe_error_fic2ProcedureFailed_e	Error because of failed Functional Integrity Check FIC 2 procedure
rfe_error_noSpaceLeftinSharedMemory_e	Error indicating that input vectors size do not fit in shared memory
rfe_error_histogramWrongXAxisLimit_e	Error indicating that the max and min for x axis in histogram function are wrong min > max
rfe_error_histogramWrongOutputSize_e	Error indicating that the length of histogram output vector is not an integer
rfe_error_invalidADCNumberOfBits_e	Error indicating that configured bits for ADC are not supported
rfe_error_fftSamplesCountNot2Multiple_e	Error indicating that number of samples given to FFT are not multiple of 2
rfe_error_fftNumberOfAveragesOutOfBounds_e	Error indicating that the number of averages for DFT/FFT is not in a valid range
rfe_error_fftInvalidOutputSelected_e	Error indicating that the selected output for FFT is invalid
rfe_error_fftSamplesCountAndNumberOfAveragesNotMultiples_e	Error indicating that FFT samples count is not divisible by number of average
rfe_error_fftNotSupported_e	Error indicating the FFT output selected is not supported for the input setting selected
rfe_error_dftSamplesCountNot2Multiple_e	Error indicating that number of samples given to DFT are not multiple of 2
rfe_error_dftNumberOfAveragesOutOfBounds_e	Error indicating that the number of averages for DFT is not in a valid range
rfe_error_dftInvalidOutputSelected_e	Error indicating that the selected output for DFT is invalid

Enumerator

rfe_error_dftSamplesCountAndNumberOfAverages↔ NotMultiples_e	Error indicating that FFT samples count is not divisible by number of average
rfe_error_dftNotSupported_e	Error indicating the DFT selected output option is not supported
rfe_error_dftOnlySingleBinSupported_e	Error indicating that dft configuration was for more than one single bin output and this is not supported
rfe_error_dftRealOnlySupported_e	Error indicating that dft configuration was set for complex input and that is not supported
rfe_error_api_cfgInvalidParameterSize_e	Given rfeCfg_param_t is containing an invalid parameter size. This error is treated as internal error, it has the <i>api</i> prefix because it is used in <i>rfeCfg</i> which exist in <i>rfeDriver</i> and in <i>rfeFw</i>
rfe_error_param_invalidParam_e	
rfe_error_param_invalidValue_e	
rfe_error_api_cfgSectionMismatch_e	Given rfeCfg_section_t section is not matching with the given parameter. This error is treated as internal error, it has the <i>api</i> prefix because it is used in <i>rfeCfg</i> which exist in <i>rfeDriver</i> and in <i>rfeFw</i>
rfe_error_timerInUse_e	Error because timer was not stopped (implicit resource release) before being re-used
rfe_error_boundedWaitResourcesExhausted_e	Error because maximum nesting level for WaitDo's reached
rfe_error_chirpPlIvcoCalibrationFailed_e	Error indicating that chirp PLL VCO calibration fails
rfe_error_atbInvalidAtbMaster_e	Error indicating that selected atb bus is invalid
rfe_error_atbInvalidTrimmingSelectedResistor_e	Error indicating that atb selected trimming resistor is invalid
rfe_error_atbInvalidOperationalMode_e	Error indicating that operational mode for atb is invalid
rfe_error_atbModeOfOperationNotSet_e	Error indicating that atb operational mode is not set so now switches will be enabled out of default
rfe_error_atbCurrentMeasurementWithout↔ Resistors_e	Error indicating that atb is trying to measure current without the trimming resistors enabled
rfe_error_atbNotSupported_e	Error indicating that the feature is not supported or implemented
rfe_error_atbCrossSwitchNotEnabled_e	Error indicating that ATB is being set in mode to measure external voltage but the operational mode is not set accordingly to enable that
rfe_error_atbBandGapNotStarted_e	Error indicating that ATB BAnd GAP in charged of providing voltage reference for ATB ADC's failed
rfe_error_atbBandGapInvalidVout_e	Error indicating that BandGap output voltage set is not supported by ATB HW and therefore invalid
rfe_error_atbInvalidPullDownInput_e	Error indicating that the input selected to configure the pull down resistors in ATB is not applicable, since resistors should not be enable for that input
rfe_error_atbInvalidAdcInputDiffMode_e	Error indicating that selected input in differential mode towards the ATB ADC's is wrong
rfe_error_atbInvalidAdcSelSingleMode_e	Error indicating that selected adc input in single mode is wrong
rfe_error_atbInvalidAdcSingleModeGround_e	Error indicating that selected input in single mode towards the ATB ADC's is wrong
rfe_error_atbAdcNotInSingleMode_e	Error indicating that adc ground is trying to be set when ADC is not working in single mode

Enumerator

rfe_error_atbAdcNotConfiguredToSample_e	Error indicating that adc was not configured properly to sample
rfe_error_atbAdcInvalidAveragingSamples_e	Error indicating that the number of samples selected for the ATB ADC are incorrect or not supported
rfe_error_hwPitNotInitialized_e	Error because pit timer not initialized
rfe_error_boundedWaitNotValidResource_e	Error because not a valid resource for this function
rfeError_error_api_TimerInUse_e	Error indicating ??
rfeError_error_api_boundedWaitResourcesExhausted_e	Error because maximum nesting level for WaitDo's reached
rfe_error_rx1SaturationStatusReadingFailure_e	Error indicating reading of rx1 saturation status fails
rfe_error_rx2SaturationStatusReadingFailure_e	Error indicating reading of rx2 saturation status fails
rfe_error_rx3SaturationStatusReadingFailure_e	Error indicating reading of rx3 saturation status fails
rfe_error_rx4SaturationStatusReadingFailure_e	Error indicating reading of rx4 saturation status fails
rfe_error_txInvalidSubcomponent_e	Error indicating that the chosen submodule for Tx is invalid
rfe_error_txInvalidPowerProfile_e	Error indicating that selected power profile for Tx is invalid
rfe_error_txGainBuf2AExceedRange_e	Error indicating that selected gain for PR BUF2A exceeds allowed range
rfe_error_txGainBuf2BCExceedRange_e	Error indicating that selected gain for PR BUF2BC exceeds allowed range
rfe_error_txPaGainCExceedRange_e	Error indicating that selected gain for PA current exceeds allowed range
rfe_error_txInvalidCalibrationFSMState_e	Error indicating that selected calibration FSM state to skip is invalid
rfe_error_txInvalidAdcConnection_e	Error indicating that selected connection in Tx ADC is not valid
rfe_error_txIndexInvalid_e	Error indicating that the tx index is not valid
rfe_error_txPRPhaseExceedRange_e	Error indicating that the phase value force into the PR exceeds allowed range
rfe_error_dynamicProgrammingDuringAcquisition_e	Error indicating that dynamic programming was not finished before chirp acquisition
rfe_error_pcdIndexInvalid_e	Error indicating that selected PDC index is invalid
rfe_error_pcdSamplingFrequencyInvalid_e	Error indicating that selected sampling rate for PDC is invalid
rfe_error_pcdProfileIndexInvalid_e	Error indicating that selected profile for PDC is invalid
rfe_error_pcdPhaseCorrectionInvalid_e	Error indicating that set phase correction for I or Q component exceeds its numeric range
rfe_error_pcdSlectedInputNotImplemented_e	Error indicating that selected input via INP_SEL is not supported, but it might be if implemented
rfe_error_pcdCorrelatorIntegrationTimeInvalid_e	Error indicating that set correlation integration time exceeds range
rfe_error_distrGroupDoesNotExist_e	Error indicating that rfeSwCfgMgr distribution group does not exist
rfe_error_invalidRadarCycleTiming_e	Configured radar cycle timing is invalid
rfe_error_mainFsm_eventAlreadyActive_e	Event is already active, main and radar cycle fsm each support only 1 event active at the same time
rfe_error_radarCycleFsm_stateEndTimeExceeded_e	A radar cycle state exceeded the scheduled end time
rfe_error_mainFsm_eventNotAllowed_e	Triggered main fsm event is not allowed in the current state

Enumerator

rfe_error_chirpStartedTooEarly_e	Chirp started earlier than expected, RFE FW was still busy
rfe_error_mainFsm_chirpStartInterruptTimeout_e	Chirp start interrupt was not received before timeout
rfe_error_mainFsm_chirpSequenceEndInterrupt← Timeout_e	Chirp sequence end interrupt was not received before timeout
rfe_error_mainFsm_chirpAcquisitionStartInterrupt← Timeout_e	Chirp acquisition start interrupt was not received before timeout
rfe_error_mainFsm_mcgenAdpllNotLocked_e	Mcgen ADPLL not in lock when chirp sequence started
rfe_error_tSenseInvalidModeOfOperation_e	Error indicating that selected temperature sensor is invalid
rfe_error_tSenseInvalidResolutionTime_e	Error indicating that temperature selected resolution is invalid
rfe_error_tSenseConversionInProgress_e	Error indicating that a temperature query was carried out when the conversion was not finished
rfe_error_tSenseInvalidOverUndeThresholds_e	Error indicating that temperature threshold either for over-temperature and under-temperature is invalid
rfe_error_tSenseInvalidWarningThreshold_e	Error indicating that warning temperature threshold is invalid
rfe_error_tSenseInvalidHysteresisThreshold_e	Error indicating that hysteresis temperature threshold is invalid
rfe_error_tSenseDivisionByZero_e	Error indicating that converting temperature via raw ADC value issued a division by zero
rfe_error_rxAdcModuleIdUnidentified_e	Error indicating that rfeHwRxAdc_adcModuleId_t is unidentified
rfe_error_rxAdcInputUnidentified_e	Error indicating that rfeHwRxAdc_inputI_rx_e or rfeHwRxAdc_inputQ_rx_e is unidentified
rfe_error_chirpPllLoopfilterC1NotOk_e	Error indicating that chirp pll loop filter C1 value is not valid
rfe_error_chirpPllLoopfilterC2FastNotOk_e	Error indicating that chirp pll loop filter C2Fast value is not valid
rfe_error_chirpPllLoopfilterC2SlowNotOk_e	Error indicating that chirp pll loop filter C2Slow value is not valid
rfe_error_chirpPllLoopfilterC3FastNotOk_e	Error indicating that chirp pll loop filter C3Fast value is not valid
rfe_error_chirpPllLoopfilterC3SlowNotOk_e	Error indicating that chirp pll loop filter C3Slow value is not valid
rfe_error_chirpPllLoopfilterR1NotOk_e	Error indicating that chirp pll loop filter R1 value is not valid
rfe_error_chirpPllLoopfilterR2NotOk_e	Error indicating that chirp pll loop filter R2 value is not valid
rfe_error_chirpPllLoopfilterCpNotOk_e	Error indicating that chirp pll loop filter charge pump value is not valid
rfe_error_atbInvalidAtbAdc_e	Error indicating invalid ATB ADC selection
rfe_error_atbInvalidAtbAdcGainMode_e	Error indicating invalid ATB ADC gain mode selection
rfe_error_chirpPllBandgapNotOk_e	Error indicating that chirp pll bandgap is not functional
rfe_error_chirpPllHvLdoVcoNotOk_e	Error indicating that chirp pll VCO HVLDO is not functional
rfe_error_chirpPllHvLdoCpNotOk_e	Error indicating that chirp pll CP HVLDO is not functional
rfe_error_chirpPllPdivLdoNotOk_e	Error indicating that chirp pll PDIV LDO is not functional
rfe_error_chirpPllLvLdoNotOk_e	Error indicating that chirp pll LVLDO is not functional

Enumerator

rfe_error_rxCalibrationTargetInvalid_e	Error indicating that rx calibration target is invalid
rfe_error_rxCalibrationIfStageInvalid_e	Error indicating that rx calibration target is invalid
rfe_error_mcgen0v9AnaNotOk_e	Error indicating that Mcgen 0v9 ana signal is not functional
rfe_error_mcgenXoDetPllClkError_e	Error indicating that Mcgen XO det clock is not functional
rfe_error_mcgenAdpllOutVdd0v9DigNotOk_e	Error indicating that Mcgen Ad pll out vdd 0v9 dig signal is not functional
rfe_error_mcgenAdpllLockDetNotLocked_e	Error indicating that Mcgen ADPLL is not locked
rfe_error_initRtmErrorReported	Error indicating RTM error from ISM is reported This error is only used during RFE initialization, FCCU error outside of init will not trigger a SW Error
rfe_error_initMcgen0v9AnaNotOk_e	Error indicating that Mcgen 0v9 ana signal is not functional
rfe_error_rxTargetSettingsCombinationsIsNotSupported_e	Error indicating that RX profile target combination is not supported
rfe_error_didt_test_failure_e	didt test fails
rfe_error_testBufferOverflow_e	Test buffer overflow
rfe_error_chirpPllVcoCalAmpOutOfRange_e	Error indicating that chirp pll VCO calibration amplitude level is out of range
rfe_error_lpUartNotActive_e	Error LpUart not active
rfe_error_txPrCalibrationTimeout_e	TX PR calibration took longer than the reserved time budget
rfe_error_registerComparisonFailed_e	
rfe_error_spiTransactionError_e	
rfe_error_spiTransactionTimeout_e	
rfe_error_HwRegister_lock_e	Access to register for a write operation is locked in Hardware.
rfe_error_startup_seq_failed_e	Startup sequence is not configured as described in the ISM/FCCU Init Sequence
rfe_error_csi2PllLockTimeOut_e	PLL lock failed
rfe_error_csi2PllcalibrationTimeOut_e	PLL PHY calibration failed
rfe_error_fcc_e	FCCU error occurred (does not override active SW error, so FCCU error was first)
rfe_error_ppdVoutAboveMax_e	Error indicating that PPD VOUT is above Max value
rfe_error_initLldoPdcPowerNotOk_e	Error indicating that LLDO PDC power not okay
rfe_error_atbInvalidInputSource_e	Error indicating that ATB input source selected is invalid
rfe_error_atbAdcNoCalibrated_e	Error indicating that ATB ADC is not calibrated and therefore unreliable to be used
rfe_error_bistZeroHourDataNotGenerated_e	
rfe_error_rxbistParameterOutOfRange_e	BIST: Error indication RX Bist parameter is out of range
rfe_error_wdmaIncorrectStreamSelected_e	WDMA: Error indicating that an invalid stream is selected for WDMA operation
rfe_error_wdmaBufferOutOfBounds_e	WDMA: Error indicating that an invalid address is selected for buffer in WDMA operation
rfe_error_wdmaBufferAddressNotAligned_e	WDMA: Error indicating that the selected buffer address is not aligned with the bit-width of the AXI bus
rfe_error_wdmaBurstLengthOutOfBounds_e	WDMA: Error indicating that the selected burst-length is out of allowed bounds as per AXI standard

Enumerator

rfe_error_wdmaWrongValueForaxiBurstModeField_e	WDMA: Error indicating that the incorrect value is passed for AXI burst length field
rfe_error_wdmaBufferAddressNotSet_e	WDMA: Error indicating that the buffer address is not set prior to triggering a stream on OR clearing the destination buffer
rfe_error_wdmaBufferLengthNotSet_e	WDMA: Error indicating that the buffer length (or size) is not set prior to triggering a stream on OR clearing the destination buffer
rfe_error_wdmaBufferOverwriteModeNotSet_e	WDMA: Error indicating that the buffer overwrite mode is not set prior to triggering a stream on
rfe_error_wdmaAxiBurstLengthNotSet_e	WDMA: Error indicating that the AXI burst length is not set prior to triggering a stream on
rfe_error_wdmaAxiBurstModeNotSet_e	WDMA: Error indicating that the AXI burst mode is not set prior to triggering a stream on
rfe_error_wdmaIncorrectInterruptSourceSelected_e	WDMA: Error indicating that an incorrect interrupt source has been selected
rfe_error_wdmaIncorrectNumberOfStreamsSelected_e	WDMA: Error indicating that an incorrect number of streams has been selected
rfe_error_rxIfCalibrationInvalidRcCode_e	error if invalid code for R and C banks is selected
rfe_error_rxInvalidRcCode_e	error if invalid code for R and C banks is selected within RX driver API
rfe_error_rxbistInvalidSsbCode_e	error if invalid code for RXBIST SSB control is selected within RX BIST API
rfe_error_rxIfCalibrationInvalidChannel	error indicating that selected channel for RX IF calibration is wrong
rfe_error_rxIfCalibrationFailed	error indicating that rx if calibration has failed
rfe_error_rxIfCalibrationInvalidSettings	error indicating that rx if calibration settings are invalid
rfe_error_bc_inputOutOfRange	Input parameters out of range.
rfe_error_bc_funcNotExist	Access to a not-existing function.
rfe_error_bc_failToInjectCrcErr	CRC error injection failure.
rfe_error_bc_wrongInterface	Serializer mode other than MIPI/LVDS selected.
rfe_error_bc_ipDisabled	Access to an OTP disabled IP.
rfe_error_bc_sys_ipHasNoAtbConnection	
rfe_error_bc_sys_cannotFindCafcDacCode	
rfe_error_bc_sys_adcInvalid	
rfe_error_bc_sys_fileError	
rfe_error_bc_sys_jsonStringNoExisted	
rfe_error_bc_sys_txTempReadTimeout	Failed reading temperature data of TX after multiple retries.
rfe_error_bc_sys_mclkIntegrityFailed	MCLK for serializer(LVDS/MIPI) is not within range. Expected MCLK is 480MHz with a minor drift of +/- 500KHz.
rfe_error_bc_sys_wrongAppconfig	
rfe_error_bc_sys_invalidInput	Invalid CRC trigger event is given as input.
rfe_error_bc_sys_wrongProfileNoRecovery	
rfe_error_bc_sys_errorRecoveryFailed	

Enumerator

rfe_error_bc_sys_cafcIntegrityCheckLevel	This error is related to CAFC sensor integrity check where known errors are injected and checked whether corresponding CAFC errors are triggered. This error is reported when VCOLevelMinErr is FALSE while TRUE is expected.
rfe_error_bc_sys_cafcIntegrityCheckUnlock	This error is related to CAFC sensor integrity check where known errors are injected and checked whether corresponding CAFC errors are triggered. This error is reported when PLLUnlockErr is FALSE while TRUE is expected.
rfe_error_bc_sys_wrongDataInterfaceConfig	Wrong data interface configuration. LVDS or MIPI interface need to be set in App configuration before initializing data interface.
rfe_error_bc_sys_error_N	ERROR_N Pin should be high at the end of successful startup sequence completion. This error code indicates there are some errors and ERROR_N Pin is low.
rfe_error_bc_sys_invalidProfile	
rfe_error_bc_sys_faultInjectionFail	
rfe_error_bc_sys_wrongMidNoRecovery	
rfe_error_bc_sys_spiAccessCheckFail	
rfe_error_bc_sys_cafcTempReadTimeout	Failed reading temperature data of CAFC after multiple retries.
rfe_error_bc_sys_slaveWrongAppConfig	CAFC module has to be disabled for slave device. This error occur if the CAFC module is enable for slave device.
rfe_error_bc_sys_subBandIncOutOfRange	
rfe_error_bc_sys_ptrAddrNull	
rfe_error_bc_sys_aaafcAutoNotSupported	
rfe_error_bc_sys_rawNumSampleInvalid	RawNumSample is the chirp active or acquisition time. This is affected by the decimation factor. The error means RawNumSample is not a factor of decimation or zero.
rfe_error_bc_sys_invalidResetTime	Reset time should be less or equal to chirp active time.
rfe_error_bc_sys_totalBwOutOfRange	Calculated total (full ramp) bandwidth = BW TSettle + BW Acquisition + BW TJumpback, is going out of range for corresponding VCO bandwidth.
rfe_error_bc_sys_cafcIntegrityCheckFailed	CAFC integrity check failure due to PLL unlock and/or VCO level min error, API is not able to clear the unlock and/or VCO level min error.
rfe_error_bc_sys_restoreChirpGlobalFailed	Restoring of CHIRP_GLOBAL_CONTROL register failed while CAFC integrity check.
rfe_error_bc_sys_regCrclsmModuleErrorSet	ISM register CRC error is reported, even after clearing the error injection.
rfe_error_bc_sys_regCrcTriggerFailed	
rfe_error_bc_sys_txCtrlEnableWriteCheckBase	Write check failed while setting defaults for TX_CONTROL_ENABLE register.
rfe_error_bc_sys_tx1CtrlEnableWriteCheck	
rfe_error_bc_sys_tx2CtrlEnableWriteCheck	
rfe_error_bc_sys_tx1Tx2CtrlEnableWriteCheck	
rfe_error_bc_sys_tx3CtrlEnableWriteCheck	

Enumerator

rfe_error_bc_sys_tx1Tx3CtrlEnableWriteCheck	
rfe_error_bc_sys_tx2Tx3CtrlEnableWriteCheck	
rfe_error_bc_sys_tx1Tx2Tx3CtrlEnableWriteCheck	
rfe_error_bc_sys_loiInBallBreakDetected	LOI Ball Break error.
rfe_error_bc_sys_loiOutBallBreakDetected	
rfe_error_bc_algDummyError	
rfe_error_bc_spiWriteReadCrcError	
rfe_error_bc_spiWriteCheckFail	
rfe_error_bc_readFail	
rfe_error_bc_ccSysConfNotSet	
rfe_error_bc_ccPtrAddrNull	
rfe_error_bc_lvdsInvalidInput	LVDS test mode setting is incorrect.
rfe_error_bc_lvdsPtrAddrNull	
rfe_error_bc_lvdsSamplesShiftInvalid	
rfe_error_bc_lvdsClkEdgeInvalid	
rfe_error_bc_lvdsAdcClkFromInvalid	
rfe_error_bc_lvdsFreqCounterDeviation	
rfe_error_bc_lvdsCountPeriodInvalid	
rfe_error_bc_lvdsFreqCountTimeout	Busy wait read of Frequency Counter register failed after max retries.
rfe_error_bc_lvdsrestoreFreqCounter	Restoring of the frequency counter register failed.
rfe_error_bc_mipiDeviceReadyTimeout	
rfe_error_bc_mipiFreqCountTimeout	Busy wait read of Frequency Counter register failed after max retries.
rfe_error_bc_mipiFreqCntOutOfRange	CountPeriod Period in us for doing the frequency measurement is out of range.
rfe_error_bc_mipiPtrAddrNull	
rfe_error_bc_mipiRestoreFreqCounter	Restoring of the frequency counter register failed.
rfe_error_bc_tePrSafetyDelay	
rfe_error_bc_tePrCalibDelay	
rfe_error_bc_teRxActiveDelay	
rfe_error_bc_teHpfResetDelay	Invalid HPF Reset Release Delay from reference point.
rfe_error_bc_teDcPowOnDelay	
rfe_error_bc_teDcSafetyDelay	Invalid safety monitor delay from reference point.
rfe_error_bc_teDdmaSetting	
rfe_error_bc_teDdmaMode	
rfe_error_bc_teJumpBack	
rfe_error_bc_teTxRxGrpDelay	Either TX or RX group delay is greater than DC power on delay.
rfe_error_bc_teFineGrpDelay	
rfe_error_bc_teProfileList	Invalid Profile or profile combination.
rfe_error_bc_tePrCalTimeout	
rfe_error_bc_teBiasDelay	
rfe_error_bc_teInputNull	
rfe_error_bc_tePrfRepeatCnt	Invalid profile repeat count.
rfe_error_bc_teChirpSeqInterval	
rfe_error_bc_teSettleTime	

Enumerator

rfe_error_bc_teOutputSampRate	
rfe_error_bc_teChirpInterval	
rfe_error_bc_teVirtualChannel	
rfe_error_bc_teDwllTime	
rfe_error_bc_teDelayFromRefpt	Invalid Transmission ON delay from timing reference point.
rfe_error_bc_teFreqCntTimeout	Polling for the completion of chirp sequence failed after multiple retries.
rfe_error_bc_teChirpSequence	
rfe_error_bc_teChirpTrgMode	
rfe_error_bc_tePhaseOutOfRange	
rfe_error_bc_teHpfResetWrongTriggerPoint	
rfe_error_bc_teTxCalEnableWriteCheckBase	Write check failed while setting defaults for TX_CAL_ENABLE register.
rfe_error_bc_teTx1CalEnableWriteCheck	
rfe_error_bc_teTx2CalEnableWriteCheck	
rfe_error_bc_teTx1Tx2CalEnableWriteCheck	
rfe_error_bc_teTx3CalEnableWriteCheck	
rfe_error_bc_teTx1Tx3CalEnableWriteCheck	
rfe_error_bc_teTx2Tx3CalEnableWriteCheck	
rfe_error_bc_teTx1Tx2Tx3CalEnableWriteCheck	
rfe_error_bc_teChirpEnableWriteCheckFailed	Write check failed while setting chirp enable.
rfe_error_bc_scProfile	
rfe_error_bc_scOperatingFreqOutOfRange	
rfe_error_bc_scPtrAddrNull	
rfe_error_bc_scInputFreqDriftInvalid	Frequency Drift value greater than the chirp bandwidth.
rfe_error_bc_scUserBwOutOfRange	Bandwidth is outside the range what hardware is capable of.
rfe_error_bc_scRampTimeOutOfRange	Ramp time outside the practical range.
rfe_error_bc_scResetTimeOutOfRange	Reset time outside the practical range.
rfe_error_bc_cafcFreqCntTimeout	
rfe_error_bc_cafcSbOutOfRange	
rfe_error_bc_cafcInputOutOfRange	
rfe_error_bc_cafcBwOutOfRange	Wrong VCO selected.
rfe_error_bc_cafcConfigLoopFilterFail	Compensated slice number is 0, which is invalid.
rfe_error_bc_cafcLoopBwOutOfRange	
rfe_error_bc_cafcPtrAddrNull	
rfe_error_bc_cafcWrongProfileid	
rfe_error_bc_cafcAafcKvCodVdtTimeout	Busy status for aafc, kvco and dvdt.
rfe_error_bc_cafcVcoCalibAafcFail	
rfe_error_bc_cafcVcoCalibKvCodVdtFail	
rfe_error_bc_cafcNoStoreInProfileSelected	
rfe_error_bc_cafcDvdtCountWrong	
rfe_error_bc_cafcKvcoCountStatusErr	
rfe_error_bc_cafcWrongKvco	
rfe_error_bc_cafcWrongAcquisitionDuration	
rfe_error_bc_adcCalibrationFailed	ADC calibration failure.

Enumerator

rfe_error_bc_adcCalibrationTimeout	ADC calibration timeout failure.
rfe_error_bc_adcCalibarationCheckNotDone	ADC calibration Check not done.
rfe_error_bc_gbiasDummy	
rfe_error_bc_loiLutGainCodeOutOfRange	LO LUT Gain code out of range.
rfe_error_bc_loiCalBusy	LO Gain cal busy.
rfe_error_bc_loiCalError	LO Gain cal error.
rfe_error_bc_loiPtrAddrNull	
rfe_error_bc_mclkPllCalibrationFailed	
rfe_error_bc_mclkPllCalibrationNotFinished	
rfe_error_bc_mclkMspcCalibrationError	
rfe_error_bc_mclkMspcTimeout	
rfe_error_bc_mclkPtrAddrNull	
rfe_error_bc_rxInputOutOfRange	
rfe_error_bc_rxCalBusy	
rfe_error_bc_rx1CalBusy	
rfe_error_bc_rx2CalBusy	
rfe_error_bc_rx3CalBusy	
rfe_error_bc_rx4CalBusy	
rfe_error_bc_rxCalError	
rfe_error_bc_rx1CalError	
rfe_error_bc_rx2CalError	
rfe_error_bc_rx1Rx2CalError	
rfe_error_bc_rx3CalError	
rfe_error_bc_rx1Rx3CalError	
rfe_error_bc_rx2Rx3CalError	
rfe_error_bc_rx1Rx2Rx3CalError	
rfe_error_bc_rx4CalError	
rfe_error_bc_rx1Rx4CalError	
rfe_error_bc_rx2Rx4CalError	
rfe_error_bc_rx1Rx2Rx4CalError	
rfe_error_bc_rx3Rx4CalError	
rfe_error_bc_rx1Rx3Rx4CalError	
rfe_error_bc_rx2Rx3Rx4CalError	
rfe_error_bc_rx1Rx2Rx3Rx4CalError	
rfe_error_bc_rxPrtAddrNull	
rfe_error_bc_rxWrngCalTargetOffset	
rfe_error_bc_rxFusaEnableWriteCheckBase	Write check failed while enabling Functional Safety Mon Level for RX.
rfe_error_bc_rx1FusaEnableWritecheck	
rfe_error_bc_rx2FusaEnableWritecheck	
rfe_error_bc_rx1Rx2FusaEnableWritecheck	
rfe_error_bc_rx3FusaEnableWritecheck	
rfe_error_bc_rx1Rx3FusaEnableWritecheck	
rfe_error_bc_rx2Rx3FusaEnableWritecheck	
rfe_error_bc_rx1Rx2Rx3FusaEnableWritecheck	
rfe_error_bc_rx4FusaEnableWritecheck	
rfe_error_bc_rx1Rx4FusaEnableWritecheck	
rfe_error_bc_rx2Rx4FusaEnableWritecheck	
rfe_error_bc_rx1Rx2Rx4FusaEnableWritecheck	

Enumerator

rfe_error_bc_rx3Rx4FusaEnableWritecheck	
rfe_error_bc_rx1Rx3Rx4FusaEnableWritecheck	
rfe_error_bc_rx2Rx3Rx4FusaEnableWritecheck	
rfe_error_bc_rx1Rx2Rx3Rx4FusaEnableWritecheck	
rfe_error_bc_rxBbdEnableWritecheckBase	Write check failed for Rx BBD Enable.
rfe_error_bc_rx1BbdEnableWritecheck	
rfe_error_bc_rx2BbdEnableWritecheck	
rfe_error_bc_rx1Rx2BbdEnableWritecheck	
rfe_error_bc_rx3BbdEnableWritecheck	
rfe_error_bc_rx1Rx3BbdEnableWritecheck	
rfe_error_bc_rx2Rx3BbdEnableWritecheck	
rfe_error_bc_rx1Rx2Rx3BbdEnableWritecheck	
rfe_error_bc_rx4BbdEnableWritecheck	
rfe_error_bc_rx1Rx4BbdEnableWritecheck	
rfe_error_bc_rx2Rx4BbdEnableWritecheck	
rfe_error_bc_rx1Rx2Rx4BbdEnableWritecheck	
rfe_error_bc_rx3Rx4BbdEnableWritecheck	
rfe_error_bc_rx1Rx3Rx4BbdEnableWritecheck	
rfe_error_bc_rx2Rx3Rx4BbdEnableWritecheck	
rfe_error_bc_rx1Rx2Rx3Rx4BbdEnableWritecheck	
rfe_error_bc_txInputOutOfRange	
rfe_error_bc_txPrcAdcTimeout	
rfe_error_bc_txSensorFitFail	
rfe_error_bc_txCalBusy	
rfe_error_bc_tx1CalBusy	
rfe_error_bc_tx2CalBusy	
rfe_error_bc_tx3CalBusy	
rfe_error_bc_txCalError	
rfe_error_bc_tx1CalError	
rfe_error_bc_tx2CalError	
rfe_error_bc_tx1Tx2CalError	
rfe_error_bc_tx3CalError	
rfe_error_bc_tx1Tx3CalError	
rfe_error_bc_tx2Tx3CalError	
rfe_error_bc_tx1Tx2Tx3CalError	
rfe_error_bc_txSafetyBusy	
rfe_error_bc_tx1SafetyBusy	
rfe_error_bc_tx2SafetyBusy	
rfe_error_bc_tx3SafetyBusy	
rfe_error_bc_txSafetyError	
rfe_error_bc_tx1SafetyError	
rfe_error_bc_tx2SafetyError	
rfe_error_bc_tx1Tx2SafetyError	
rfe_error_bc_tx3SafetyError	
rfe_error_bc_tx1Tx3SafetyError	
rfe_error_bc_tx2Tx3SafetyError	
rfe_error_bc_tx1Tx2Tx3SafetyError	
rfe_error_bc_txPtrAddrNull	

Enumerator

rfe_error_bc_txNoStoreInProfileSelected	
rfe_error_bc_txPoutVgaReduceTxNotSupported	
rfe_error_bc_txRfMinCodeOutOfRange	
rfe_error_bc_txFusaEnableWriteCheckBase	Write check failed for while enabling Functional Safety Mon Level enabling for TX.
rfe_error_bc_tx1FusaEnableWriteCheck	
rfe_error_bc_tx2FusaEnableWriteCheck	
rfe_error_bc_tx1Tx2FusaEnableWriteCheck	
rfe_error_bc_tx3FusaEnableWriteCheck	
rfe_error_bc_tx1Tx3FusaEnableWriteCheck	
rfe_error_bc_tx2Tx3FusaEnableWriteCheck	
rfe_error_bc_tx1Tx2Tx3FusaEnableWriteCheck	
rfe_error_bc_txCalibBiasCurrWriteFailed	If the TX calibration is not complete, setting bias current will not succeed.
rfe_error_bc_txCalibTimeout	TX calibration failed with timeout.
rfe_error_bc_txBbdEnableWritecheckBase	Write check failed while enabling BBD for TX.
rfe_error_bc_tx1BbdEnableWritecheck	
rfe_error_bc_tx2BbdEnableWritecheck	
rfe_error_bc_tx1tx2BbdEnableWritecheck	
rfe_error_bc_tx3BbdEnableWritecheck	
rfe_error_bc_tx1Tx3BbdEnableWritecheck	
rfe_error_bc_tx2Tx3BbdEnableWritecheck	
rfe_error_bc_tx1Tx2Tx3BbdEnableWritecheck	
rfe_error_bc_ssbSignalNotExistOnSelectedAtb	Connected signal node to a wrong ATB.
rfe_error_bc_ssbInputOutOfRange	Input out of range.
rfe_error_bc_gldoSnsError	
rfe_error_bc_gldoSnsForceError	
rfe_error_bc_gldoSnsResetError	
rfe_error_bc_atbBistAdcTimeout	BISTADC data collection timeout.
rfe_error_bc_atbWrongAtbNoSelected	selected a wrong ATB
rfe_error_bc_otpFailToRead	Fail to read OTP table.
rfe_error_bc_otpCrcCheckFailure	CRC check failure.
rfe_error_bc_otplplsDisabled	IP is disabled.
rfe_error_bc_otpOtplsNotLoaded	OTP is not loaded.
rfe_error_bc_otpAllMemDisabled	
rfe_error_bc_ismInvalidInput	
rfe_error_bc_ismPtrAddrNull	
rfe_error_bc_ismStartupSeqConfigFailed	
rfe_error_bc_ismInputModeToggleTestFailed	
rfe_error_bc_ismOutputModeToggleTestFailed	
rfe_error_bc_ismFitErrorN_High	
rfe_error_bc_ismFitResetStateChangeFailed	
rfe_error_bc_ismFitResetErrorFailed	
rfe_error_bc_ismFitForceStateChangeFailed	
rfe_error_bc_ismFitForceErrorFailed	
rfe_error_bc_ismFitFailed	
rfe_error_bc_ismFitStatusFailed	
rfe_error_bc_ismFitTxStatusFailed	

Enumerator

rfe_error_bc_ismFitRxStatusFailed	
rfe_error_bc_ismFitAdcStatusFailed	
rfe_error_bc_ismFitChirpStatusFailed	
rfe_error_bc_ismFitMclkLoIsMotpCcStatusFailed	
rfe_error_bc_ismFitSsbSerGbGldoAAtbStatusFailed	
rfe_error_bc_ismCalBusy	
rfe_error_bc_ismCalError	
rfe_error_bc_ismCalSuccess	
rfe_error_bc_ismDisableLbToggleTestFailed	
rfe_error_sys_spi_wrg_unit_id	
rfe_error_sys_spi_invalid_len	
rfe_error_sys_timer_cm7_init	
rfe_error_sys_FE_chip_es2_not_supported	
rfe_error_ic_wrg_input_params_e	wrong input parameter(s), usually NULL pointer
rfe_error_rxbist_frequency_fail	
rfe_error_rxbist_missing_signal	
rfe_error_rxbist_lna_gain_fail_e	
rfe_error_rxbist_lna_phase_fail_e	
rfe_error_rxbist_mixer_gain_fail_e	
rfe_error_rxbist_mixer_phase_fail_e	

3.8.3.9 rfe_eventsIRQ_t

```
enum rfe_eventsIRQ_t
```

This type defines which events shall trigger an IRQ from RFE firmware

Enumerator

RFE_EVENTS_IRQ_NONE	
RFE_EVENTS_IRQ_STATE_CHANGE	Using only this bitmask, all events IRQs are disabled
RFE_EVENTS_IRQ_RADAR_CYCLE_CHANGE	This bit enables IRQ on state change
RFE_EVENTS_IRQ_CHIRP_SEQUENCE_CHANGE	This bit enables IRQ on radar cycle change
RFE_EVENTS_IRQ_MAX	This bit enables IRQ on chirp sequence change

3.8.3.10 rfe_fusaFault_R1_t

```
enum rfe_fusaFault_R1_t
```


Enumerator

rfe_fuSaFault_R1_tempErrorFlagTx1_e	TX1 Temperature is too high if operation continue
rfe_fuSaFault_R1_tempErrorFlagTx2_e	TX2 Temperature is too high if operation continue
rfe_fuSaFault_R1_tempErrorFlagTx3_e	TX3 Temperature is too high if operation continue
rfe_fuSaFault_R1_rfLevelLowErrorFlagTx1_e	TX1 RF level reached below a programmed low level limits
rfe_fuSaFault_R1_rfLevelLowErrorFlagTx2_e	TX2 RF level reached below a programmed low level limits
rfe_fuSaFault_R1_rfLevelLowErrorFlagTx3_e	TX3 RF level reached below a programmed low level limits
rfe_fuSaFault_R1_rfLevelHighErrorFlagTx1_e	TX1 RF level reached beyond a programmed high level limits
rfe_fuSaFault_R1_rfLevelHighErrorFlagTx2_e	TX2 RF level reached beyond a programmed high level limits
rfe_fuSaFault_R1_rfLevelHighErrorFlagTx3_e	TX3 RF level reached beyond a programmed high level limits
rfe_fuSaFault_R1_loLevelLowErrorFlagRx1_e	RX1 LO level reached below a programmed low level limits
rfe_fuSaFault_R1_loLevelLowErrorFlagRx2_e	RX2 LO level reached below a programmed low level limits
rfe_fuSaFault_R1_loLevelLowErrorFlagRx3_e	RX3 LO level reached below a programmed low level limits
rfe_fuSaFault_R1_loLevelLowErrorFlagRx4_e	RX4 LO level reached below a programmed low level limits
rfe_fuSaFault_R1_loLevelHighErrorFlagRx1_e	RX1 LO level reached beyond a programmed high level limits
rfe_fuSaFault_R1_loLevelHighErrorFlagRx2_e	RX2 LO level reached beyond a programmed high level limits
rfe_fuSaFault_R1_loLevelHighErrorFlagRx3_e	RX3 LO level reached beyond a programmed high level limits
rfe_fuSaFault_R1_loLevelHighErrorFlagRx4_e	RX4 LO level reached beyond a programmed high level limits
rfe_fuSaFault_R1_tempErrorFlagChirp_e	Chirp temperature is too high if operation continue
rfe_fuSaFault_R1_vtuneLowErrorFlagChirp_e	Chirp Vtune level reached below a programmed low level limits
rfe_fuSaFault_R1_vtuneHighErrorFlagChirp_e	Chirp Vtune level reached above a programmed high level limits
rfe_fuSaFault_R1_levelLowErrorFlagChirp_e	Chirp VCO level reached below a programmed low level limits
rfe_fuSaFault_R1_levelHighErrorFlagChirp_e	Chirp VCO level reached above a programmed high level limits
rfe_fuSaFault_R1_pllFreqCalErrorFlagMc_e	Main pll (4.8GHz) calibration issues
rfe_fuSaFault_R1_loiLevelLowErrorFlagInterface_e	Level low flag mask for LOI
rfe_fuSaFault_R1_loiLevelHighErrorFlagInterface_e	Level high flag mask for LOI
rfe_fuSaFault_R1_hardClippingFlagPersistentAdc1↔ _e	Status monitoring flag mask ADC1
rfe_fuSaFault_R1_hardClippingFlagPersistentAdc2↔ _e	Status monitoring flag mask ADC2
rfe_fuSaFault_R1_hardClippingFlagPersistentAdc3↔ _e	Status monitoring flag mask ADC3
rfe_fuSaFault_R1_hardClippingFlagPersistentAdc4↔ _e	Status monitoring flag mask ADC4
rfe_fusaFault_R1_size_e	

3.8.3.11 rfe_fuSaFault_t

enum [rfe_fuSaFault_t](#)

In cascaded mode when report an error need to do (`rfe_error_t || (fe index << 8)`) ex: `rfe_error_sys_FE_chip_es2_not_supported || (1 << 8)`

In cascaded mode when check a reported error need to do:

- `((rfe_error_t & 0x00000F00U) >> 8)` to see which fe reported the error
- `(rfe_error_t & 0xFFFFF0FFU)` to see the error ex: for this error `0x079102U`
 - `0x079102U & 0x00000F00U >> 8 = 0x000001U => fe_1`
 - `0x079102U & 0xFFFFF0FFU = 0x079002U => rfe_error_sys_FE_chip_es2_not_supported`

Or you can use the masks like : `rfe_error_fe_0 = 0x000000U`, `rfe_error_fe_1 = 0x000100U`, `rfe_error_fe_2 = 0x000200U`, ... This enumeration data type is used for FuSa faults

- In RFE Configuration ([fuSaFaultMask](#)), the enumeration must be used as bitwise index to mask or unmask the corresponding fault.
- In [rfe_getFuSaFaults\(\)](#), the enumeration must be used as bitwise index to get the fault status for the corresponding fault.
- The MSB represent byte index and LSB represent bitwise index.
- Bitwise index = 0 to 7
- Byte index = 0 to 11
- for example this error "`rfe_fuSaFault_sr65_sr67_supply_low_1v8_rx3_e = 0x0203U`" will be find in byte index 2 fusa fault array at bit 3

Enumerator

<code>rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_tx3_e</code>	TX3 supply low error (1V1)
<code>rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_tx2_e</code>	TX2 supply low error (1V1)
<code>rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_tx1_e</code>	TX1 supply low error (1V1)
<code>rfe_fuSaFault_R2_sr72_pr_tx3_e</code>	Phase rotator error for TX3
<code>rfe_fuSaFault_R2_sr72_pr_tx2_e</code>	Phase rotator error for TX2
<code>rfe_fuSaFault_R2_sr72_pr_tx1_e</code>	Phase rotator error for TX1
<code>rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_tx3_e</code>	TX3 supply low error (1V8)
<code>rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_tx2_e</code>	TX2 supply low error (1V8)
<code>rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_tx1_e</code>	TX1 supply low error (1V8)
<code>rfe_fuSaFault_R2_sr69_rf_level_max_tx3_e</code>	Max level monitoring error indicating TX3 RF level reached beyond max limit

Enumerator

rfe_fuSaFault_R2_sr69_rf_level_max_tx2_e	Max level monitoring error indicating TX2 RF level reached beyond max limit
rfe_fuSaFault_R2_sr69_rf_level_max_tx1_e	Max level monitoring error indicating TX1 RF level reached beyond max limit
rfe_fuSaFault_R2_sr69_rf_level_min_tx3_e	Min level monitoring error indicating TX3 RF level reached below min limit
rfe_fuSaFault_R2_sr69_rf_level_min_tx2_e	Min level monitoring error indicating TX2 RF level reached below min limit
rfe_fuSaFault_R2_sr69_rf_level_min_tx1_e	Min level monitoring error indicating TX1 RF level reached below min limit
rfe_fuSaFault_R2_sr27_bb_tx3_e	RF connectivity to external antenna port error for TX3
rfe_fuSaFault_R2_sr27_bb_tx2_e	RF connectivity to external antenna port error for TX2
rfe_fuSaFault_R2_sr27_bb_tx1_e	RF connectivity to external antenna port error for TX1
rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_rx4_e	Supply 1v8 low error RX4
rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_rx3_e	Supply 1v8 low error RX3
rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_rx2_e	Supply 1v8 low error RX2
rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_rx1_e	Supply 1v8 low error RX1
rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_rx4_e	Supply 1v1 low error RX4
rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_rx3_e	Supply 1v1 low error RX3
rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_rx2_e	Supply 1v1 low error RX2
rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_rx1_e	Supply 1v1 low error RX1
rfe_fuSaFault_R2_sr47_lo_level_max_rx4_e	Max Level monitoring error flag indicating RX4 LO level reached beyond max limit
rfe_fuSaFault_R2_sr47_lo_level_max_rx3_e	Max Level monitoring error flag indicating RX3 LO level reached beyond max limit
rfe_fuSaFault_R2_sr47_lo_level_max_rx2_e	Max Level monitoring error flag indicating RX2 LO level reached beyond max limit
rfe_fuSaFault_R2_sr47_lo_level_max_rx1_e	Max Level monitoring error flag indicating RX1 LO level reached beyond max limit
rfe_fuSaFault_R2_sr47_lo_level_min_rx4_e	Min Level monitoring error flag indicating RX4 LO level reached below min limit
rfe_fuSaFault_R2_sr47_lo_level_min_rx3_e	Min Level monitoring error flag indicating RX3 LO level reached below min limit
rfe_fuSaFault_R2_sr47_lo_level_min_rx2_e	Min Level monitoring error flag indicating RX2 LO level reached below min limit
rfe_fuSaFault_R2_sr47_lo_level_min_rx1_e	Min Level monitoring error flag indicating RX1 LO level reached below min limit
rfe_fuSaFault_R2_sr9_bb_rx4_e	RF connectivity to external antenna port error for RX4
rfe_fuSaFault_R2_sr9_bb_rx3_e	RF connectivity to external antenna port error for RX3
rfe_fuSaFault_R2_sr9_bb_rx2_e	RF connectivity to external antenna port error for RX2
rfe_fuSaFault_R2_sr9_bb_rx1_e	RF connectivity to external antenna port error for RX1
rfe_fuSaFault_R2_sr46_aafc_cal_timeout_chirp_e	AAFC calibration timeout error for chirp digital
rfe_fuSaFault_R2_sr46_aafc_chirp_e	AAFC calibration error for chirp digital

Enumerator

rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_↔ chirp_e	Supply 1V1 low error for chirp digital
rfe_fuSaFault_R2_sr65_sr67_supply_low_vco_1v8_↔ _chirp_e	Supply 1V8 low error for chirp VCO
rfe_fuSaFault_R2_sr65_sr67_supply_low_pll_1v8_↔ chirp_e	Supply 1V8 low error for chirp PLL
rfe_fuSaFault_R2_sr7_interface_stuck_chirp_e	Stuck on chirp_seq_active, safety_monitoring, window_active detected
rfe_fuSaFault_R2_sr42_vco_freq_480_chirp_e	Error indicating VCO ouput frequency to the chirp digital logic is beyond programmable limit
rfe_fuSaFault_R2_sr45_level_max_chirp_e	Max Level monitoring error flag indicating VCO output level reached beyond max limit
rfe_fuSaFault_R2_sr45_level_min_chirp_e	Min Level monitoring error flag indicating VCO output level reached below min limit
rfe_fuSaFault_R2_sr43_unlock_chirp_e	Error indicating chirp PLL in unlock condition
rfe_fuSaFault_R2_sr7_lock_step_chirp_e	Error indicating chirp digital logic in stuck condition or transient logic failure
rfe_fuSaFault_R2_sr48_level_max_loi_e	Max Level monitoring error flag indicating LOI level reached beyond max limit
rfe_fuSaFault_R2_sr48_level_min_loi_e	Min Level monitoring error flag indicating LOI level reached below min limit
rfe_fuSaFault_R2_sr19_rd_spi_crc_e	SPI CRC read error
rfe_fuSaFault_R2_sr19_wr_spi_crc_e	SPI CRC write error
rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_loi_e	Supply low error from LO interface
rfe_fuSaFault_R2_sr17_bb_lo_in_loi_e	RF connectivity issues in LO-in interface
rfe_fuSaFault_R2_sr18_bb_lo_out_loi_e	RF connectivity issues in LO-out interface
rfe_fuSaFault_R2_sr11_xo_no_clock_mclk_e	Clock failure (no clock error) from master clock
rfe_fuSaFault_R2_sr36_mspc_mclk_e	Leader-Follower phase calibration error
rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_↔ mclk_e	Supply error form master clock (1V8)
rfe_fuSaFault_R2_sr39_dig_freq_mclk_e	Error indicating master clock digital ouput frequency not in range
rfe_fuSaFault_R2_sr35_pll_lock_mclk_e	Error indicating Pll in unlock condition
rfe_fuSaFault_R2_sr57_supply_high_1v1_gldo_e	Global 1V1 LDO supply high sensor error
rfe_fuSaFault_R2_sr59_supply_low_1v1_gldo_e	Global 1V1 LDO supply low sensor error
rfe_fuSaFault_R2_sr61_supply_high_1v8_gldo_e	Global 1V8 LDO supply high sensor error
rfe_fuSaFault_R2_sr63_supply_low_1v8_gldo_e	Global 1V8 LDO supply low sensor error
rfe_fuSaFault_R2_sr65_67_supply_low_1v8_ser_e	Supply 1V8 low sensor error for serializer
rfe_fuSaFault_R2_sr65_67_supply_low_1v1_csi2_e	Supply 1V1 low sensor error for serializer CSI2
rfe_fuSaFault_R2_sr65_67_supply_low_1v1_ser_e	Supply 1V1 low sensor error for serializer digital and LVDS
rfe_fuSaFault_R2_sr65_67_supply_low_1v8_gbias_↔ _e	Supply 1V8 low sensor error for global bias
rfe_fuSaFault_R2_sr65_67_supply_low_1v8_ssb_e	Supply 1V8 low sensor error for SSB MOD
rfe_fuSaFault_R2_sr65_67_supply_low_1v8_↔ adc34_e	Supply low (1v8) for ADC34
rfe_fuSaFault_R2_sr65_67_supply_low_1v8_↔ adc12_e	Supply low (1v8) for ADC12
rfe_fuSaFault_R2_sr34_cal_adc4_e	Calibration error for ADC4

Enumerator

rfe_fuSaFault_R2_sr34_cal_adc3_e	Calibration error for ADC3
rfe_fuSaFault_R2_sr34_cal_adc2_e	Calibration error for ADC2
rfe_fuSaFault_R2_sr34_cal_adc1_e	Calibration error for ADC1
rfe_fuSaFault_R2_sr3_reg_crc_e	Register CRC error
rfe_fuSaFault_R2_sr31_bist_lna_gaindiff_sw_e	RFBIST LNA gain test error
rfe_fuSaFault_R2_sr31_bist_lna_phasediff_sw_e	RFBIST LNA phase test error
rfe_fuSaFault_R2_sr31_bist_mixer_gaindiff_sw_e	RFBIST mixer gain test error
rfe_fuSaFault_R2_sr31_bist_mixer_phasediff_sw_e	RFBIST mixer phase test error
rfe_fuSaFault_R2_sr44_cafc_integrity_level_sw_e	CAFC level detector integrity check error
rfe_fuSaFault_R2_sr44_cafc_integrity_unlock_sw_e	CAFC unlock detector integrity check error
rfe_fuSaFault_R2_sr44_cafc_integrity_sw_e	CAFC integrity check error
rfe_fuSaFault_R2_sr70_tx_integrity_sw_e	TX integrity check error
rfe_fuSaFault_R2_sr72_tx_pr_cal_sw_e	TX phase rotator calibration check error
rfe_fuSaFault_R2_sr72_tx1_pr_cal_sw_e	TX1 phase rotator calibration check error
rfe_fuSaFault_R2_sr72_tx2_pr_cal_sw_e	TX2 phase rotator calibration check error
rfe_fuSaFault_R2_sr37_pll_level_mclk_e	PLL /VCO level is too low or too high
rfe_fuSaFault_R2_sr72_tx3_pr_cal_sw_e	TX3 phase rotator calibration check error
rfe_fuSaFault_R2_sr14_disable_lb_toggle_test_sw↔ _e	Disable loopback toggle test
rfe_fuSaFault_R2_sr13_input_mode_toggle_test↔ sw_e	Input mode toggle test
rfe_generic_sw_e	SW generic error

3.8.3.12 rfe_fuSaFaultMaskIndex_t

```
enum rfe_fuSaFaultMaskIndex_t
```

This enumerated type defines indices for the FuSa fault mask configuration array.

Enumerator

rfe_fuSaFaultMaskIndex_0_e	
rfe_fuSaFaultMaskIndex_1_e	
rfe_fuSaFaultMaskIndex_2_e	
rfe_fuSaFaultMaskIndex_3_e	
rfe_fuSaFaultMaskIndex_4_e	
rfe_fuSaFaultMaskIndex_5_e	
rfe_fuSaFaultMaskIndex_6_e	
rfe_fuSaFaultMaskIndex_7_e	
rfe_fuSaFaultMaskIndex_8_e	
rfe_fuSaFaultMaskIndex_9_e	
rfe_fuSaFaultMaskIndex_10↔ _e	
rfe_fuSaFaultMaskIndex_11↔ _e	

3.8.3.13 rfe_paramUpdate_t

enum `rfe_paramUpdate_t`

This enumerated types defines the RFE-Fw parameters that can be updated.

Enumerator

<code>rfe_paramUpdate_general_powerMode_e</code>	
<code>rfe_paramUpdate_general_radarCycleStartSignalGpio_e</code>	
<code>rfe_paramUpdate_general_chirpSequenceActiveSignalGpio_e</code>	
<code>rfe_paramUpdate_general_clkIoConfig_e</code>	
<code>rfe_paramUpdate_general_pdcDecimationFilter_e</code>	
<code>rfe_paramUpdate_general_dataOutConfig_e</code>	
<code>rfe_paramUpdate_general_metadataPacketSwContent_e</code>	
<code>rfe_paramUpdate_general_metadataPacketVirtualChannel_e</code>	
<code>rfe_paramUpdate_general_jumpbackTimeTicks_e</code>	
<code>rfe_paramUpdate_general_txCalibration_mode_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_rxSatCountResetEveryChirpSequence_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_rxSaturationThresholdStage1_rx1_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_rxSaturationThresholdStage1_rx2_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_rxSaturationThresholdStage1_rx3_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_rxSaturationThresholdStage1_rx4_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_rxSaturationThresholdStage2_rx1_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_rxSaturationThresholdStage2_rx2_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_rxSaturationThresholdStage2_rx3_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_rxSaturationThresholdStage2_rx4_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_rxSaturationCountLimitStage1_rx1_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_rxSaturationCountLimitStage1_rx2_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_rxSaturationCountLimitStage1_rx3_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_rxSaturationCountLimitStage1_rx4_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_rxSaturationCountLimitStage2_rx1_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_rxSaturationCountLimitStage2_rx2_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_rxSaturationCountLimitStage2_rx3_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_rxSaturationCountLimitStage2_rx4_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_adcClippingCountResetEveryChirpSequence_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_adcClippingCountLimit_adc1_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_adcClippingCountLimit_adc2_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_adcClippingCountLimit_adc3_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_adcClippingCountLimit_adc4_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_temperatureSensorThresholdHigh_tx12_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_temperatureSensorThresholdHigh_tx3_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_temperatureSensorThresholdHigh_xo_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_temperatureSensorThresholdOver_tx12_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_temperatureSensorThresholdOver_tx3_e</code>	
<code>rfe_paramUpdate_monitorAndSafety_temperatureSensorThresholdOver_xo_e</code>	

Enumerator

rfe_paramUpdate_monitorAndSafety_temperatureSensorThresholdUnder_tx12_e	
rfe_paramUpdate_monitorAndSafety_temperatureSensorThresholdUnder_tx3_e	
rfe_paramUpdate_monitorAndSafety_temperatureSensorThresholdUnder_xo_e	
rfe_paramUpdate_monitorAndSafety_thresholdValueToPromoteR1Faults_e	
rfe_paramUpdate_monitorAndSafety_frequencyForBist_e	
rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxPhaseDiffLna_rx1_rx2_e	
rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxPhaseDiffLna_rx1_rx3_e	
rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxPhaseDiffLna_rx1_rx4_e	
rfe_paramUpdate_monitorAndSafety_rxPhaseDiffLnaThresholdTolerance_e	
rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxGainDiffLna_rx1_rx2_e	
rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxGainDiffLna_rx1_rx3_e	
rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxGainDiffLna_rx1_rx4_e	
rfe_paramUpdate_monitorAndSafety_rxGainDiffLnaThresholdTolerance_e	
rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxPhaseDiffMixer_rx1_rx2_e	
rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxPhaseDiffMixer_rx1_rx3_e	
rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxPhaseDiffMixer_rx1_rx4_e	
rfe_paramUpdate_monitorAndSafety_rxPhaseDiffMixerThresholdTolerance_e	
rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxGainDiffMixer_rx1_rx2_e	
rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxGainDiffMixer_rx1_rx3_e	
rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxGainDiffMixer_rx1_rx4_e	
rfe_paramUpdate_monitorAndSafety_rxGainDiffMixerThresholdTolerance_e	
rfe_paramUpdate_monitorAndSafety_injectTestTone_e	
rfe_paramUpdate_monitorAndSafety_checkBbd_loi_e	
rfe_paramUpdate_monitorAndSafety_checkBbd_tx_e	
rfe_paramUpdate_monitorAndSafety_checkBbd_rx_e	
rfe_paramUpdate_monitorAndSafety_ppdThreshold_profile0_e	
rfe_paramUpdate_monitorAndSafety_fuSaFaultMask_0_e	
rfe_paramUpdate_monitorAndSafety_fuSaFaultMask_1_e	
rfe_paramUpdate_monitorAndSafety_fuSaFaultMask_2_e	
rfe_paramUpdate_monitorAndSafety_fuSaFaultMask_3_e	
rfe_paramUpdate_monitorAndSafety_fuSaFaultMask_4_e	
rfe_paramUpdate_monitorAndSafety_fuSaFaultMask_5_e	
rfe_paramUpdate_monitorAndSafety_fuSaFaultMask_6_e	
rfe_paramUpdate_monitorAndSafety_fuSaFaultMask_7_e	
rfe_paramUpdate_monitorAndSafety_fuSaFaultMask_8_e	
rfe_paramUpdate_monitorAndSafety_fuSaFaultMask_9_e	
rfe_paramUpdate_monitorAndSafety_fuSaFaultMask_10_e	
rfe_paramUpdate_monitorAndSafety_fuSaFaultMask_11_e	
rfe_paramUpdate_radarCycle_radarCycleDuration_e	
rfe_paramUpdate_radarCycle_chirpSequenceCount_e	
rfe_paramUpdate_radarCycle_chirpSequence_0_e	
rfe_paramUpdate_radarCycle_chirpSequence_1_e	
rfe_paramUpdate_radarCycle_chirpSequence_2_e	
rfe_paramUpdate_radarCycle_chirpSequence_3_e	
rfe_paramUpdate_radarCycle_chirpSequence_4_e	
rfe_paramUpdate_radarCycle_chirpSequence_5_e	

Enumerator

rfe_paramUpdate_radarCycle_chirpSequence_6_e	
rfe_paramUpdate_radarCycle_chirpSequence_7_e	
rfe_paramUpdate_radarCycle_chirpSequenceStartTimeOffset_0_e	
rfe_paramUpdate_radarCycle_chirpSequenceStartTimeOffset_1_e	
rfe_paramUpdate_radarCycle_chirpSequenceStartTimeOffset_2_e	
rfe_paramUpdate_radarCycle_chirpSequenceStartTimeOffset_3_e	
rfe_paramUpdate_radarCycle_chirpSequenceStartTimeOffset_4_e	
rfe_paramUpdate_radarCycle_chirpSequenceStartTimeOffset_5_e	
rfe_paramUpdate_radarCycle_chirpSequenceStartTimeOffset_6_e	
rfe_paramUpdate_radarCycle_chirpSequenceStartTimeOffset_7_e	
rfe_paramUpdate_radarCycle_recalibrateProfileDependent_0_e	
rfe_paramUpdate_radarCycle_recalibrateProfileDependent_1_e	
rfe_paramUpdate_radarCycle_recalibrateProfileDependent_2_e	
rfe_paramUpdate_radarCycle_recalibrateProfileDependent_3_e	
rfe_paramUpdate_radarCycle_recalibrateProfileDependent_4_e	
rfe_paramUpdate_radarCycle_recalibrateProfileDependent_5_e	
rfe_paramUpdate_radarCycle_recalibrateProfileDependent_6_e	
rfe_paramUpdate_radarCycle_recalibrateProfileDependent_7_e	
rfe_paramUpdate_radarCycle_recalibrateProfileIndependent_e	
rfe_paramUpdate_radarCycle_bistInterval_e	
rfe_paramUpdate_chirpSequence_chirpCount_e	
rfe_paramUpdate_chirpSequence_chirpProfileSequence_0_e	
rfe_paramUpdate_chirpSequence_chirpProfileSequence_1_e	
rfe_paramUpdate_chirpSequence_chirpProfileSequence_2_e	
rfe_paramUpdate_chirpSequence_chirpProfileSequence_3_e	
rfe_paramUpdate_chirpSequence_chirpProfileSequence_4_e	
rfe_paramUpdate_chirpSequence_chirpProfileSequence_5_e	
rfe_paramUpdate_chirpSequence_chirpProfileSequence_6_e	
rfe_paramUpdate_chirpSequence_chirpProfileSequence_7_e	
rfe_paramUpdate_chirpSequence_chirpProfileSequenceLength_e	
rfe_paramUpdate_chirpSequence_dynamicUpdatesEnabled_e	
rfe_paramUpdate_chirpSequence_dynamicTableIndex_e	
rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_e	
rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_tx_e	
rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_tx1_init_e	
rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_tx2_init_e	
rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_tx3_init_e	
rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_tx1_update_e	
rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_tx2_update_e	
rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_tx3_update_e	
rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_tx1_mode_e	
rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_tx2_mode_e	
rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_tx3_mode_e	
rfe_paramUpdate_chirpSequence_rxEnable_e	
rfe_paramUpdate_chirpSequence_pdcNotchFilterEnable_e	
rfe_paramUpdate_chirpSequence_txCalibration_thresholdHigh_e	

Enumerator

rfe_paramUpdate_chirpSequence_txCalibration_thresholdLow_e	
rfe_paramUpdate_chirpProfile_effectiveSamplingFrequency_e	
rfe_paramUpdate_chirpProfile_chirpIntervalTimeTicks_e	
rfe_paramUpdate_chirpProfile_dwellTimeTicks_e	
rfe_paramUpdate_chirpProfile_settleTimeTicks_e	
rfe_paramUpdate_chirpProfile_acquisitionTimeTicks_e	
rfe_paramUpdate_chirpProfile_resetTimeTicks_e	
rfe_paramUpdate_chirpProfile_centerFrequency_e	
rfe_paramUpdate_chirpProfile_effectiveChirpBandwidth_e	
rfe_paramUpdate_chirpProfile_chirpPllVcoSelect_e	
rfe_paramUpdate_chirpProfile_chirpSlopeDirection_e	
rfe_paramUpdate_chirpProfile_chirpPllLoopFilterBandwidth_e	
rfe_paramUpdate_chirpProfile_txTransmissionEnable_e	
rfe_paramUpdate_chirpProfile_txPower_e	
rfe_paramUpdate_chirpProfile_txPhaseRotation_1_e	
rfe_paramUpdate_chirpProfile_txPhaseRotation_2_e	
rfe_paramUpdate_chirpProfile_txPhaseRotation_3_e	
rfe_paramUpdate_chirpProfile_txTransmissionReferenceTime_e	
rfe_paramUpdate_chirpProfile_txTransmissionTimeOffset_e	
rfe_paramUpdate_chirpProfile_chirpFrequencyDriftSteps_e	
rfe_paramUpdate_chirpProfile_rxGain_e	
rfe_paramUpdate_chirpProfile_rxHpfCutOffFrequency_e	
rfe_paramUpdate_chirpProfile_rxLpfCutOffFrequency_e	
rfe_paramUpdate_chirpProfile_virtualChannel_e	

3.8.3.14 rfe_powerMode_t

```
enum rfe_powerMode_t
```

This enumerated type defines the RFE power modes.

Enumerator

rfe_powerMode_inactive↔ _e	Disable RFE chirp power mode.
rfe_powerMode_active_e	Puts RFE in active chirp power mode.

3.8.3.15 rfe_PRPCGenMode_e

```
enum rfe_PRPCGenMode_e
```

This enumerated type defines the phase rotator final pc gen mode.

Enumerator

rfe_PR_NO_CHANGE_e	No change to initial phase code.
rfe_PR_REPLACE_MSB_WITH_QPSK	Replace two MSB bits of initial phase code with the QPSK[1:0] bits.
rfe_PR_ADD_QPSK_TO_MSB	Add the QPSK[1:0] bits to two MSB bits of initial phase code.
rfe_PR_INVALID	

3.8.3.16 rfe_rxClippingReset_t

```
enum rfe_rxClippingReset_t
```

This enumerated type defines the rx clipping reset counter.

Enumerator

rfe_rx1Reset_e	
rfe_rx2Reset_e	
rfe_rx3Reset_e	
rfe_rx4Reset_e	
rfe_rx1234Reset↔ _e	

3.8.3.17 rfe_rxGain_t

```
enum rfe_rxGain_t
```

This enumerated type defines the receiver gains.

Enumerator

rfe_rxGain_25dB_e	
rfe_rxGain_28dB_e	
rfe_rxGain_31dB_e	
rfe_rxGain_34dB_e	
rfe_rxGain_37dB_e	
rfe_rxGain_40dB_e	
rfe_rxGain_43dB_e	
rfe_rxGain_46dB_e	
rfe_rxGain_invalid↔ _e	

3.8.3.18 rfe_rxHpfCutOffFrequency_t

```
enum rfe_rxHpfCutOffFrequency_t
```

This enumerated type defines the receiver high-pass filter cut-off frequencies.

Enumerator

rfe_rxHpfCutOffFrequency_200kHz_e	
rfe_rxHpfCutOffFrequency_300kHz_e	
rfe_rxHpfCutOffFrequency_400kHz_e	
rfe_rxHpfCutOffFrequency_800kHz_e	
rfe_rxHpfCutOffFrequency_1600kHz↔ _e	
rfe_rxHpfCutOffFrequency_3200kHz↔ _e	
rfe_rxHpfCutOffFrequency_6400kHz↔ _e	
rfe_rxHpfCutOffFrequency_invalid_e	

3.8.3.19 rfe_rxIndex_t

```
enum rfe_rxIndex_t
```

This enumerated type defines the indices of the receivers.

Enumerator

rfe_rxIndex_1↔ _e	
rfe_rxIndex_2↔ _e	
rfe_rxIndex_3↔ _e	
rfe_rxIndex_4↔ _e	

3.8.3.20 rfe_rxLpfCutOffFrequency_t

```
enum rfe_rxLpfCutOffFrequency_t
```

This enumerated type defines the receiver low-pass filter cut-off frequencies.

Enumerator

rfe_rxLpfCutOffFrequency_12P5MHz↔ _e	
rfe_rxLpfCutOffFrequency_15MHz_e	
rfe_rxLpfCutOffFrequency_20MHz_e	
rfe_rxLpfCutOffFrequency_25MHz_e	
rfe_rxLpfCutOffFrequency_40MHz_e	
rfe_rxLpfCutOffFrequency_invalid_e	

3.8.3.21 rfe_rxSaturationThreshold_t

```
enum rfe_rxSaturationThreshold_t
```

This enumerated type defines the rx saturation threshold.

Enumerator

rfe_rxSaturationThreshold_min13_05dB↔ _e	
rfe_rxSaturationThreshold_min12_18dB↔ _e	
rfe_rxSaturationThreshold_min11_31dB↔ _e	
rfe_rxSaturationThreshold_min10_44dB↔ _e	
rfe_rxSaturationThreshold_min9_57dB_e	
rfe_rxSaturationThreshold_min8_70dB_e	
rfe_rxSaturationThreshold_min7_83dB_e	
rfe_rxSaturationThreshold_min6_96dB_e	
rfe_rxSaturationThreshold_min6_09dB_e	
rfe_rxSaturationThreshold_min5_22dB_e	
rfe_rxSaturationThreshold_min4_35dB_e	
rfe_rxSaturationThreshold_min3_48dB_e	
rfe_rxSaturationThreshold_min2_61dB_e	
rfe_rxSaturationThreshold_min1_74dB_e	
rfe_rxSaturationThreshold_min0_87dB_e	
rfe_rxSaturationThreshold_0dB_e	

3.8.3.22 rfe_rxStage_t

```
enum rfe_rxStage_t
```

This enumerated type defines the rx stages.

Enumerator

rfe_rxStage_1↔ _e	
rfe_rxStage_2↔ _e	

3.8.3.23 rfe_state_t

```
enum rfe_state_t
```

This type defines the RFE-Fw states. RFE-Fw state can be retrieved via [rfe_getState\(\)](#).

Enumerator

rfe_state_busy_e	RFE-Fw is busy and not accepting any commands.
rfe_state_initialized_e	RFE-Fw is initialized and ready to be configured via rfe_configure() .
rfe_state_configured_e	RFE-Fw is configured and ready to take any command, such as rfe_radarCycleStart() .
rfe_state_radarCycleIdle_e	RFE-Fw is in the idle period of the radar cycle and accepting limited commands, such as: • rfe_updatePush() • rfe_radarCycleStop() • rfe_getNextRadarCycleStartTime() • rfe_setNextRadarCycleStartTime() • rfe_getTime() • rfe_monitorRead()
rfe_state_testContinuousWaveTransmission↔ _e	RFE-Fw is transmitting a continuous wave signal. and accepting limited commands, such as: • rfe_testContinuousWaveTransmissionStop() • rfe_getTime() • rfe_monitorRead()
rfe_state_fuSaFaultRecovery_e	A FuSa fault has occurred in the RFE-Fw, RFE-Fw is busy. RFE-Fw parses the fault and attempts fault recovery for R1 faults.
rfe_state_fuSaFault_e	A FuSa fault has occurred in the RFE, call rfe_getFuSaFaults() to retrieve the FuSa fault.

3.8.3.24 rfe_temperatureSensorIndex_t

enum [rfe_temperatureSensorIndex_t](#)

This enumerated type defines the temperature sensors.

Enumerator

rfe_temperatureSensorIndex_tx1_e	The temperature sensor located near transmitter 1.
rfe_temperatureSensorIndex_tx2_e	The temperature sensor located near transmitter 2.
rfe_temperatureSensorIndex_tx3_e	The temperature sensor located near transmitter 3.
rfe_temperatureSensorIndex_chirp↔ _e	The temperature sensor located near the chirp module.

3.8.3.25 rfe_testParam_t

enum [rfe_testParam_t](#)

This enumerated type defines the RFE-Fw test parameters. Used by [rfe_testSetParam\(\)](#).

Enumerator

rfe_testParam_outputDataTestPattern_e	This test is not implemented yet.
rfe_testParam_keepTxTransmissionEnabled↔ _e	This test is not implemented yet.
rfe_testParam_chirpPllTestPinEnable_e	This test is not implemented yet.
rfe_testParam_enableClockRetuning_e	This test is not implemented yet.

3.8.3.26 rfe_txIndex_t

enum [rfe_txIndex_t](#)

This enumerated type defines the indices of the transmitters.

Enumerator

rfe_txIndex_1_e	
rfe_txIndex_2_e	
rfe_txIndex_3_e	
rfe_txIndex_4_e	
rfe_txIndex_invalid↔ _e	

3.8.3.27 rfe_txPowerErrorFlag_t

```
enum rfe_txPowerErrorFlag_t
```

Enumerator

rfe_txPowerMinErrorFlag	
rfe_txPowerLowErrorFlag	
rfe_txPowerHighErrorFlag	
rfe_txPowerMaxErrorFlag	

3.8.3.28 rfe_virtualChannel_t

```
enum rfe_virtualChannel_t
```

This enumerated type defines the virtual channels.

Enumerator

rfe_virtualChannel_0_e	
rfe_virtualChannel_1_e	
rfe_virtualChannel_2_e	
rfe_virtualChannel_3_e	
rfe_virtualChannel_invalid↔ _e	

3.8.3.29 rfeCfg_param_t

```
enum rfeCfg_param_t
```

The defined parameters.

Enumerator

rfeCfg_param_metadata_chirpSequenceConfigCount_e	
rfeCfg_param_metadata_chirpProfileCount_e	
rfeCfg_param_metadata_majorVersion_e	
rfeCfg_param_metadata_minorVersion_e	
rfeCfg_param_metadata_patchVersion_e	
rfeCfg_param_metadata_hashVersion_e	
rfeCfg_param_general_powerMode_e	

Enumerator

rfeCfg_param_general_radarCycleStartSignalGpio_e	
rfeCfg_param_general_chirpSequenceActiveSignalGpio_e	
rfeCfg_param_general_clkloConfig_e	
rfeCfg_param_general_pdcDecimationFilter_e	
rfeCfg_param_general_dataOutConfig_e	
rfeCfg_param_general_metadataPacketSwContent_e	
rfeCfg_param_general_metadataPacketVirtualChannel_e	
rfeCfg_param_general_jumpbackTimeTicks_e	
rfeCfg_param_general_txCalibration_mode_e	
rfeCfg_param_monitorAndSafety_rxSatCountResetEveryChirpSequence_e	
rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage1_rx1_e	
rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage1_rx2_e	
rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage1_rx3_e	
rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage1_rx4_e	
rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage2_rx1_e	
rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage2_rx2_e	
rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage2_rx3_e	
rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage2_rx4_e	
rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage1_rx1_e	
rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage1_rx2_e	
rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage1_rx3_e	
rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage1_rx4_e	
rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage2_rx1_e	
rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage2_rx2_e	
rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage2_rx3_e	
rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage2_rx4_e	
rfeCfg_param_monitorAndSafety_adcClippingCountResetEveryChirpSequence_e	
rfeCfg_param_monitorAndSafety_adcClippingCountLimit_adc1_e	
rfeCfg_param_monitorAndSafety_adcClippingCountLimit_adc2_e	
rfeCfg_param_monitorAndSafety_adcClippingCountLimit_adc3_e	
rfeCfg_param_monitorAndSafety_adcClippingCountLimit_adc4_e	
rfeCfg_param_monitorAndSafety_temperatureSensorThresholdHigh_tx12_e	
rfeCfg_param_monitorAndSafety_temperatureSensorThresholdHigh_tx3_e	
rfeCfg_param_monitorAndSafety_temperatureSensorThresholdHigh_xo_e	
rfeCfg_param_monitorAndSafety_temperatureSensorThresholdOver_tx12_e	
rfeCfg_param_monitorAndSafety_temperatureSensorThresholdOver_tx3_e	
rfeCfg_param_monitorAndSafety_temperatureSensorThresholdOver_xo_e	
rfeCfg_param_monitorAndSafety_temperatureSensorThresholdUnder_tx12_e	
rfeCfg_param_monitorAndSafety_temperatureSensorThresholdUnder_tx3_e	
rfeCfg_param_monitorAndSafety_temperatureSensorThresholdUnder_xo_e	
rfeCfg_param_monitorAndSafety_thresholdValueToPromoteR1Faults_e	
rfeCfg_param_monitorAndSafety_frequencyForBist_e	
rfeCfg_param_monitorAndSafety_zeroHrRefForRxPhaseDiffLna_rx1_rx2_e	
rfeCfg_param_monitorAndSafety_zeroHrRefForRxPhaseDiffLna_rx1_rx3_e	
rfeCfg_param_monitorAndSafety_zeroHrRefForRxPhaseDiffLna_rx1_rx4_e	

Enumerator

rfeCfg_param_monitorAndSafety_rxPhaseDiffLnaThresholdTolerance_e	
rfeCfg_param_monitorAndSafety_zeroHrRefForRxGainDiffLna_rx1_rx2_e	
rfeCfg_param_monitorAndSafety_zeroHrRefForRxGainDiffLna_rx1_rx3_e	
rfeCfg_param_monitorAndSafety_zeroHrRefForRxGainDiffLna_rx1_rx4_e	
rfeCfg_param_monitorAndSafety_rxGainDiffLnaThresholdTolerance_e	
rfeCfg_param_monitorAndSafety_zeroHrRefForRxPhaseDiffMixer_rx1_rx2_e	
rfeCfg_param_monitorAndSafety_zeroHrRefForRxPhaseDiffMixer_rx1_rx3_e	
rfeCfg_param_monitorAndSafety_zeroHrRefForRxPhaseDiffMixer_rx1_rx4_e	
rfeCfg_param_monitorAndSafety_rxPhaseDiffMixerThresholdTolerance_e	
rfeCfg_param_monitorAndSafety_zeroHrRefForRxGainDiffMixer_rx1_rx2_e	
rfeCfg_param_monitorAndSafety_zeroHrRefForRxGainDiffMixer_rx1_rx3_e	
rfeCfg_param_monitorAndSafety_zeroHrRefForRxGainDiffMixer_rx1_rx4_e	
rfeCfg_param_monitorAndSafety_rxGainDiffMixerThresholdTolerance_e	
rfeCfg_param_monitorAndSafety_injectTestTone_e	
rfeCfg_param_monitorAndSafety_checkBbd_loi_e	
rfeCfg_param_monitorAndSafety_checkBbd_tx_e	
rfeCfg_param_monitorAndSafety_checkBbd_rx_e	
rfeCfg_param_monitorAndSafety_ppdThreshold_profile0_e	
rfeCfg_param_monitorAndSafety_ppdThreshold_profile1_e	
rfeCfg_param_monitorAndSafety_ppdThreshold_profile2_e	
rfeCfg_param_monitorAndSafety_ppdThreshold_profile3_e	
rfeCfg_param_monitorAndSafety_ppdThreshold_profile4_e	
rfeCfg_param_monitorAndSafety_ppdThreshold_profile5_e	
rfeCfg_param_monitorAndSafety_ppdThreshold_profile6_e	
rfeCfg_param_monitorAndSafety_ppdThreshold_profile7_e	
rfeCfg_param_monitorAndSafety_fuSaFaultMask_0_e	
rfeCfg_param_monitorAndSafety_fuSaFaultMask_1_e	
rfeCfg_param_monitorAndSafety_fuSaFaultMask_2_e	
rfeCfg_param_monitorAndSafety_fuSaFaultMask_3_e	
rfeCfg_param_monitorAndSafety_fuSaFaultMask_4_e	
rfeCfg_param_monitorAndSafety_fuSaFaultMask_5_e	
rfeCfg_param_monitorAndSafety_fuSaFaultMask_6_e	
rfeCfg_param_monitorAndSafety_fuSaFaultMask_7_e	
rfeCfg_param_monitorAndSafety_fuSaFaultMask_8_e	
rfeCfg_param_monitorAndSafety_fuSaFaultMask_9_e	
rfeCfg_param_monitorAndSafety_fuSaFaultMask_10_e	
rfeCfg_param_monitorAndSafety_fuSaFaultMask_11_e	
rfeCfg_param_radarCycle_radarCycleDuration_e	
rfeCfg_param_radarCycle_chirpSequenceCount_e	
rfeCfg_param_radarCycle_chirpSequence_0_e	
rfeCfg_param_radarCycle_chirpSequence_1_e	
rfeCfg_param_radarCycle_chirpSequence_2_e	
rfeCfg_param_radarCycle_chirpSequence_3_e	
rfeCfg_param_radarCycle_chirpSequence_4_e	
rfeCfg_param_radarCycle_chirpSequence_5_e	
rfeCfg_param_radarCycle_chirpSequence_6_e	

Enumerator

rfeCfg_param_radarCycle_chirpSequence_7_e	
rfeCfg_param_radarCycle_chirpSequenceStartTimeOffset_0_e	
rfeCfg_param_radarCycle_chirpSequenceStartTimeOffset_1_e	
rfeCfg_param_radarCycle_chirpSequenceStartTimeOffset_2_e	
rfeCfg_param_radarCycle_chirpSequenceStartTimeOffset_3_e	
rfeCfg_param_radarCycle_chirpSequenceStartTimeOffset_4_e	
rfeCfg_param_radarCycle_chirpSequenceStartTimeOffset_5_e	
rfeCfg_param_radarCycle_chirpSequenceStartTimeOffset_6_e	
rfeCfg_param_radarCycle_chirpSequenceStartTimeOffset_7_e	
rfeCfg_param_radarCycle_recalibrateProfileDependent_0_e	
rfeCfg_param_radarCycle_recalibrateProfileDependent_1_e	
rfeCfg_param_radarCycle_recalibrateProfileDependent_2_e	
rfeCfg_param_radarCycle_recalibrateProfileDependent_3_e	
rfeCfg_param_radarCycle_recalibrateProfileDependent_4_e	
rfeCfg_param_radarCycle_recalibrateProfileDependent_5_e	
rfeCfg_param_radarCycle_recalibrateProfileDependent_6_e	
rfeCfg_param_radarCycle_recalibrateProfileDependent_7_e	
rfeCfg_param_radarCycle_recalibrateProfileIndependent_e	
rfeCfg_param_radarCycle_bistInterval_e	
rfeCfg_param_chirpSequence_chirpCount_e	
rfeCfg_param_chirpSequence_chirpProfileSequence_0_e	
rfeCfg_param_chirpSequence_chirpProfileSequence_1_e	
rfeCfg_param_chirpSequence_chirpProfileSequence_2_e	
rfeCfg_param_chirpSequence_chirpProfileSequence_3_e	
rfeCfg_param_chirpSequence_chirpProfileSequence_4_e	
rfeCfg_param_chirpSequence_chirpProfileSequence_5_e	
rfeCfg_param_chirpSequence_chirpProfileSequence_6_e	
rfeCfg_param_chirpSequence_chirpProfileSequence_7_e	
rfeCfg_param_chirpSequence_chirpProfileSequenceLength_e	
rfeCfg_param_chirpSequence_chirpProfileRepeatCount_e	
rfeCfg_param_chirpSequence_dynamicUpdatesEnabled_e	
rfeCfg_param_chirpSequence_dynamicTableIndex_e	
rfeCfg_param_chirpSequence_txPhaseRotation_ddma_e	
rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx_e	
rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx1_init_e	
rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx2_init_e	
rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx3_init_e	
rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx1_update_e	
rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx2_update_e	
rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx3_update_e	
rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx1_mode_e	
rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx2_mode_e	
rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx3_mode_e	
rfeCfg_param_chirpSequence_rxEnable_e	
rfeCfg_param_chirpSequence_pdcNotchFilterEnable_e	
rfeCfg_param_chirpSequence_txCalibration_thresholdHigh_e	

Enumerator

rfeCfg_param_chirpSequence_txCalibration_thresholdLow_e	
rfeCfg_param_chirpProfile_effectiveSamplingFrequency_e	
rfeCfg_param_chirpProfile_chirpIntervalTimeTicks_e	
rfeCfg_param_chirpProfile_dwellTimeTicks_e	
rfeCfg_param_chirpProfile_settleTimeTicks_e	
rfeCfg_param_chirpProfile_acquisitionTimeTicks_e	
rfeCfg_param_chirpProfile_resetTimeTicks_e	
rfeCfg_param_chirpProfile_centerFrequency_e	
rfeCfg_param_chirpProfile_effectiveChirpBandwidth_e	
rfeCfg_param_chirpProfile_chirpPllVcoSelect_e	
rfeCfg_param_chirpProfile_chirpSlopeDirection_e	
rfeCfg_param_chirpProfile_chirpPllLoopFilterBandwidth_e	
rfeCfg_param_chirpProfile_txTransmissionEnable_e	
rfeCfg_param_chirpProfile_txPower_e	
rfeCfg_param_chirpProfile_txPhaseRotation_1_e	
rfeCfg_param_chirpProfile_txPhaseRotation_2_e	
rfeCfg_param_chirpProfile_txPhaseRotation_3_e	
rfeCfg_param_chirpProfile_txTransmissionReferenceTime_e	
rfeCfg_param_chirpProfile_txTransmissionTimeOffset_e	
rfeCfg_param_chirpProfile_chirpFrequencyDriftSteps_e	
rfeCfg_param_chirpProfile_rxGain_e	
rfeCfg_param_chirpProfile_rxHpfCutOffFrequency_e	
rfeCfg_param_chirpProfile_rxLpfCutOffFrequency_e	
rfeCfg_param_chirpProfile_virtualChannel_e	
rfeCfg_param_metadata_crc	

3.8.3.30 rfeCfg_section_t

```
enum rfeCfg_section_t
```

The defined sections.

Enumerator

rfeCfg_section_metadata_e	
rfeCfg_section_general_e	
rfeCfg_section_monitorAndSafety_e	
rfeCfg_section_radarCycle_e	
rfeCfg_section_chirpSequenceConfig_0↔ _e	
rfeCfg_section_chirpProfile_0_e	
rfeCfg_section_chirpSequenceConfig_1↔ _e	
rfeCfg_section_chirpProfile_1_e	

Enumerator

rfeCfg_section_chirpSequenceConfig_2↔ _e	
rfeCfg_section_chirpProfile_2_e	
rfeCfg_section_chirpSequenceConfig_3↔ _e	
rfeCfg_section_chirpProfile_3_e	
rfeCfg_section_chirpSequenceConfig_4↔ _e	
rfeCfg_section_chirpProfile_4_e	
rfeCfg_section_chirpSequenceConfig_5↔ _e	
rfeCfg_section_chirpProfile_5_e	
rfeCfg_section_chirpSequenceConfig_6↔ _e	
rfeCfg_section_chirpProfile_6_e	
rfeCfg_section_chirpSequenceConfig_7↔ _e	
rfeCfg_section_chirpProfile_7_e	
rfeCfg_section_crc	

3.8.3.31 rfeCfg_size_t

```
enum rfeCfg_size_t
```

Size of the configuration data item.

Enumerator

rfeCfg_size_8_bit_e	8 bit data size.
rfeCfg_size_16_bit↔ _e	16 bit data size.
rfeCfg_size_32_bit↔ _e	32 bit data size.
rfeCfg_size_invalid↔ _e	invalide bit data size.

3.8.3.32 rfeCmdClient_sharedParameter_t

```
enum rfeCmdClient_sharedParameter_t
```

This structure defines a shared parameter type

Enumerator

rfeCmdClient_sharedParameter_state_e	
rfeCmdClient_sharedParameter_radarCycleCount_e	
rfeCmdClient_sharedParameter_chirpSequenceCount_e	
rfeCmdClient_sharedParameter_fw_ready_e	

3.8.3.33 rfeSwBist_InjectToneMode_t

enum rfeSwBist_InjectToneMode_t

Structure to hold the RX BIST test modes.

Enumerator

TEF82XX_both_lna_mixer	
TEF82XX_only_mixer	
TEF82XX_alternate_lna_mixer	
InjectToneMode_Invalid	

3.8.3.34 rfeTef82XXRfBistTest_t

enum rfeTef82XXRfBistTest_t

RF BIST Test Type.

Select TEF82XX_RFBIST_MIXER_TEST to inject the RFBIST signal at the mixer input. Alternatively, use TEF82XX_RFBIST_LNA_TEST to inject the RFBIST signal at the LNA input.

Enumerator

TEF82XX_RFBIST_MIXER_TEST	
TEF82XX_RFBIST_LNA_TEST	
TEF82XX_RFBIST_TESTS	

3.8.4 Function Documentation

3.8.4.1 rfe_configure()

```
void rfe_configure (
    uint8_t * pConfig,
```

```
rfe_sysMemAddress_t dynamicTableAddress,
volatile rfe_error_t * rfe___error___pointer )
```

This function fully configures the RFE-Fw.

This function copies the provided configuration and dynamic tables to the RFE-Fw and verifies the response. This function returns only after RFE-Fw has processed the command and responded. Asynchronously, after this command is processed, the RFE-Fw performs the configuration and calibrations.

If error occur, CDD_Rfe_GetError must return one of :

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- [rfe_error_api_invalidArgumentValue_e](#)
- [rfe_error_api_invalidConfigurationSize_e](#)
- [rfe_error_api_invalidConfigurationParameterValue_e](#)
- [rfe_error_api_invalidDynamicTableSize_e](#)
- [rfe_error_api_invalidDynamicTableValue_e](#)
- [rfe_error_api_invalidRadarCycleTimeSchedule_e](#)
- Any [rfe_error_api_configParamCrossCheck_...](#) error of [rfe_error_t](#).
- Any [rfe_error_cmdInterface_...](#) error of [rfe_error_t](#).
- [rfe_error_sharedData_redundancyCheckFailed_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Precondition

The RFE-Fw must be in [rfe_state_initialized_e](#) state, or in [rfe_state_configured_e](#) state.

Parameters

in	<i>pConfig</i>	- Pointer to Configuration Data Memory (see RFE Abstract 2.0 - Front-End configuration structure : BLOB)
in	<i>dynamicTableAddress</i>	- Address of the dynamic tables in system memory. The dynamic tables created by the rfe configuration generator need to be loaded or copied to this address in system memory before calling this API function. Set to NULL when dynamic table is not used.
in, out	<i>rfe___error___pointer</i>	- Error handling parameter: On success <code>#*rfe___error___pointer == rfe_error_none_e</code> is true, On failure <code>#*rfe___error___pointer != rfe_error_none_e</code> is true

Postcondition

RFE-Fw state becomes [rfe_state_busy_e](#) and RFE-Fw performs configuration and calibrations asynchronously. When done, RFE-Fw state becomes [rfe_state_configured_e](#).

3.8.4.2 rfe_configureInterrupt()

```
void rfe_configureInterrupt (
    rfe_coreId_t coreId,
    rfe_eventsIRQ_t events,
    rfe_apiIRQ_t rfeAPIs,
    rfe_apiWaitForInterrupt_f apiWaitForInterrupt,
    volatile rfe_error_t * rfe___error___pointer )
```

This function configure the RFE firmware interrupt.

`rfe_configureInterrupt()` configures which conditions will raise an IRQ to `coreId`, these conditions are:

- RFE state changes
- Radar Cycle count changes
- Chirp Sequence count changes
- Specific RFE API responses from the firmware

The `apiWaitForInterrupt` function will be called while the driver is waiting for an RFE API response, this function shall return whenever a RFE firmware interrupt is received.

This is a user registered call back function where wait for interrupt or functionality to give control to Operating System can be implemented as per customer need.

This function may be called multiple times on a single RFE API call. During this call only APIs that do not interact with the firmware are allowed.

This function can return one of following errors:

- `rfe_error_api_unsynchronized_e`
- `rfe_error_api_operationNotAllowed_e`
- `rfe_error_api_busy_e`
- `rfe_error_api_invalidArgumentValue_e`
- Any `cmdInterface` error of `rfe_error_t`.
- `rfe_error_rfeFuSaFault_e`

Note

`apiWaitForInterrupt` IS NOT an interrupt handler, the application code is responsible for setting up and handle incoming RFE interrupts.

Precondition

The RFE must be in one of the following states:

- `rfe_state_initialized_e`
- `rfe_state_configured_e`
- `rfe_state_radarCycleIdle_e`
- `rfe_error_rfeFuSaFault_e`

Parameters

in	<i>coreId</i>	- Which core should the RFE interrupt be sent.
in	<i>events</i>	- Bitmask of events that will trigger an interrupt.
in	<i>rfeAPIs</i>	- Bitmask of RFE-APIs responses that will trigger an interrupt.
in	<i>apiWaitForInterrupt</i>	- API wait for interrupt function pointer. This is a user registered call back function where wait for interrupt or functionality to give control to Operating System can be implemented as per customer need.
in, out	<i>rfe__error__pointer</i>	- Error handling parameter

3.8.4.3 rfe_getBistZeroHourReferenceData()

```
void rfe_getBistZeroHourReferenceData (
    rfeCfg_rxBistReferenceData_t * pRxReferenceData,
    volatile rfe_error_t * rfe__error__pointer )
```

This function returns RX BIST measurement details to be used as zero hour reference data.

The RFE-Fw performs the RX phase difference and RX gain difference measurements upon this function call.

This function can return one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_rfeFuSaFault_e](#)

Precondition

The RFE-Fw must be in [rfe_state_configured_e](#).

Parameters

out	<i>pRxReferenceData</i>	- pointer to structure containing zero hour RX bist reference data.
in, out	<i>rfe__error__pointer</i>	- Error handling parameter: On success <code>#*rfe__error__pointer == rfe_error_none_e</code> is true, On failure <code>#*rfe__error__pointer != rfe_error_none_e</code> is true

Remarks

1. The configuration that changed due to execution of [CDD_Rfe_GetBistZeroHourReferenceData\(\)](#) will be restored to configuration that existed before the function call.
2. The threshold values ([monitorAndSafety](#)) to be used along with zero hour data are determined after characterization.

Postcondition

This function does not result in change of RFE-Fw state.

3.8.4.4 rfe_getFrontEnd()

```
uint8_t rfe_getFrontEnd (
    volatile rfe_error_t * rfe__error__pointer )
```

Function to get the associated Front-End "subject" in a cascaded setup.

The function must be used only in a cascaded setup, else an error will be returned. The possible errors are :

- unresponsive
- not synchronized
- busy
- operation not allowed - only this is a specific error for this operation request, returned from RFE-Fw

Parameters

in, out	<i>rfe__error__pointer</i>	- Error handling parameter: On success <code>#*rfe__error__pointer == rfe_error_none_e</code> is true, On failure <code>#*rfe__error__pointer != rfe_error_none_e</code> is true
---------	----------------------------	--

3.8.4.5 rfe_getFuSaFaults()

```
void rfe_getFuSaFaults (
    uint8_t * pFuSaR1R2FaultList,
    volatile rfe_error_t * rfe__error__pointer )
```

This function provides the RFE-Fw FuSa faults which are set in RFE-Fw FCCU.

This function has to be used when a FuSa fault occurs, which is indicated by the ERROR_N signal and can also be observed by [CDD_Rfe_GetState\(\)](#) returning [rfe_state_fuSaFault_e](#).

This function can return one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- [rfe_error_api_invalidArgumentValue_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_rfeFuSaFault_e](#)

Precondition

The RFE-Fw must be in one of the following states:

- [rfe_state_initialized_e](#)
- [rfe_state_configured_e](#)
- [rfe_state_radarCycleIdle_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Parameters

out	<i>pFuSaR1R2FaultList</i>	- Pointer to an array, where this function will store the list of fusa faults. The size of this array must be RFE_FUSA_R1_R2_MASK_UNMASK_FAULT_BYTE_COUNT bytes. The enumeration rfe_fuSaFault_t must be used as bit index to determine the fault from the pFuSaR1R2FaultList e.g., if byte index 3 of pFuSaR1R2FaultList has the value b0010000, it means bit position 5 of fuSaFaultMask3 is set, i.e rfe_fuSaFault_sr47_lo_level_max_rx1_e fault is set.
in, out	<i>rfe___error___pointer</i>	- Error handling parameter: On success #*rfe___error___pointer == rfe_error_none_e is true, On failure #*rfe___error___pointer != rfe_error_none_e is true

Postcondition

This function does not result in change of RFE-Fw state.

3.8.4.6 rfe_getFuSaFaultStatistics()

```
void rfe_getFuSaFaultStatistics (
    uint16_t * pR1FaultPromotedToR2,
    rfe_radarCycleCount_t * pRadarCycleCount,
    uint8_t * pFuSaR1FaultCountList,
    volatile rfe_error_t * rfe___error___pointer )
```

This function provides the fault statistics for R1 faults.

This function provides list of fault occurrence count for each R1 fault. Whenever a R1 fault is detected, the corresponding counter is incremented. Each counter can reach up to a maximum value of 0xFF and stays at this level until reset. The counter is reset as part of RFE-Fw initialization upon

- Functional reset.
- Destructive reset.
- Power on reset.

The **pFuSaR1FaultCountList** is of fixed size **RFE_FUSA_R1_FAULT_COUNT_MAX** and does not decrease even when R1 fault is promoted to R2 (refer [rfeCfg_param_monitorAndSafety_thresholdValueToPromoteR1Faults_e](#)). The **pR1FaultPromotedToR2Since** will indicate which R1 fault was promoted to R2.

This function returns after RFE-Fw has accepted or rejected the call.

This function can return one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- [rfe_error_api_invalidArgumentValue_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_rfeFuSaFault_e](#)

Precondition

The RFE-Fw must be in one of the following states:

- [rfe_state_initialized_e](#)
- [rfe_state_configured_e](#)
- [rfe_state_radarCycleIdle_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Parameters

out	<i>pR1FaultPromotedToR2</i>	- Indicates the R1 fault which is promoted to R2.
out	<i>pRadarCycleCount</i>	- Pointer to structure which provides Radar cycle count and Chirp sequence count. The count in pRadarCycleCount is updated whenever the fault occurrence count is updated. If there is no active radar cycle, the radar cycle count and chirp sequence count value will be 0x00 each.
out	<i>pFuSaR1FaultCountList</i>	- Pointer to an array containing fault occurrence count for each R1 fault. The size of this array should be RFE_FUSA_R1_FAULT_COUNT_MAX bytes. The <i>rfe_fuSaFault_t</i> up to index value <i>rfe_fuSaFault_R1_sm11_bist_tx4_sw_e</i> bytes, must be used to index the list to determine the fault occurrence count for the corresponding fault e.g., the value in pFuSaCountList [<i>rfe_fuSaFault_R1_sm11_bist_tx4_sw_e</i>] provides the fault occurrence count for <i>rfe_fuSaFault_R1_sm11_bist_tx4_sw_e</i> .
in, out	<i>rfe__error__pointer</i>	- Error handling parameter: On success <i>#*rfe__error__pointer == rfe_error_none_e</i> is true, On failure <i>#*rfe__error__pointer != rfe_error_none_e</i> is true

Postcondition

This function does not result in change of RFE-Fw state.

Remarks

1. The functions [CDD_Rfe_GetFuSaFaults\(\)](#), [CDD_Rfe_RadarCycleStart\(\)](#) and [CDD_Rfe_RadarCycleStop\(\)](#) do not affect the counter values.
2. The [rfeCfg_param_monitorAndSafety_thresholdValueToPromoteR1Faults_e](#) has no relation to the counter used in [CDD_Rfe_GetFuSaFaultStatistics\(\)](#).

3.8.4.7 rfe_getNextRadarCycleStartTime()

```
uint32_t rfe_getNextRadarCycleStartTime (
    uint16_t * pRadarCycleIndex,
    volatile rfe_error_t * rfe___error___pointer )
```

This function returns the scheduled absolute start time of the next radar cycle.

The returned time is according to the RFE-Fw time. This function returns after RFE-Fw has accepted or rejected the call.

This function can return one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- [rfe_error_api_invalidArgumentValue_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_sharedData_redundancyCheckFailed_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Precondition

The RFE-Fw must be in [rfe_state_radarCycleIdle_e](#) state.

Parameters

out	<i>*pRadarCycleIndex</i>	- Pointer to index of the next radar cycle of which the start time is returned.
in, out	<i>rfe___error___pointer</i>	- Error handling parameter: On success <i>#*rfe___error___pointer == rfe_error_none_e</i> is true, On failure <i>#*rfe___error___pointer != rfe_error_none_e</i> is true

Returns

uint32 - Absolute start time of the next scheduled radar cycle in 25 [ns] resolution.

3.8.4.8 rfe_getRadarCycleCount()

```
rfe_radarCycleCount_t rfe_getRadarCycleCount (
    volatile rfe_error_t * rfe___error___pointer )
```

This function returns the radar cycle and chirp sequence count.

It returns the number radar cycles and chirp sequences completed since *Cdd_Rfe_radarCycleStart()*. It can be used to keep track of the radar cycle or chirp sequence execution. This function does not interact with the RFE-Fw, instead it returns information from a shared memory location.

This function can return one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_sharedData_redundancyCheckFailed_e](#)

Parameters

in, out	<i>rfe___error___pointer</i>	- Error handling parameter: On success <code>#*rfe___error___pointer == rfe_error_none_e</code> is true, On failure <code>#*rfe___error___pointer != rfe_error_none_e</code> is true
---------	------------------------------	--

Returns

`Cdd_RfeRadarCycleCountType` - Structure that contains the radar cycle and chirp sequence count.

3.8.4.9 rfe_getRegisterDump()

```
void rfe_getRegisterDump (
    uint8_t frontEndPos,
    rfe_moduleDumpSelect_t moduleSelection,
    rfe_registerDumpValues_t * dumpArrayPtr,
    uint32_t * dumpArrayPtrLen,
    volatile rfe_error_t * rfe___error___pointer )
```

Function to get the complete registry status on one or many front-end(s)

The function must be used only when the radar cycle is stopped else an error will be reported. The returned values will be dumped in the result array pointed by `dumpArrayPtr`. For the moment this function doesn't have a CDD equivalent. It is the user duty to have enough space for the received data, please be aware of the :

- [rfe_registerDumpValues_t](#) structure length
- the expected number of registers to be dump for the requested module(s) found in `rfe_register_dump.h` as `RFE_..._REG_NUMBER`

Parameters

in	<i>frontEndPos</i>	- the Front-End to be used for registry dump
in	<i>moduleSelection</i>	- one or more modules to get the registry values
in	<i>dumpArrayPtr</i>	- pointer to the data area to save the received data
in, out	<i>dumpArrayPtrLen</i>	- input - pointer to data array maximum length; output - the pointer receive the exact number of values returned
in, out	<i>rfe___error___pointer</i>	- Error handling parameter: On success <code>#*rfe___error___pointer == rfe_error_none_e</code> is true, On failure <code>#*rfe___error___pointer != rfe_error_none_e</code> is true

3.8.4.10 rfe_getState()

```
rfe_state_t rfe_getState (
    volatile rfe_error_t * rfe___error___pointer )
```

This function returns the RFE-Fw state.

This function does not interact with the RFE-Fw, returning the state information from the shared memory.

This function can return one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_sharedData_redundancyCheckFailed_e](#)

Parameters

in, out	<i>rfe___error___pointer</i>	- Error handling parameter: On success <code>#*rfe___error___pointer == rfe_error_none_e</code> is true, On failure <code>#*rfe___error___pointer != rfe_error_none_e</code> is true
---------	------------------------------	--

Returns

`Cdd_RfeStateType` - The RFE-Fw state

3.8.4.11 rfe_getTime()

```
uint32_t rfe_getTime (
    volatile rfe_error_t * rfe___error___pointer )
```

This function returns the RFE-Fw time.

It returns the absolute RFE-Fw time since the initialization in 25[ns] resolution. The counter wrap around time is equal to $2^{32} * 25$ ns, which is approximately 107 seconds. This function returns after RFE-Fw has accepted or rejected the call.

This function can return one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_sharedData_redundancyCheckFailed_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Precondition

The RFE-Fw must be in one of the following states:

- [rfe_state_initialized_e](#)
- [rfe_state_configured_e](#)
- [rfe_state_radarCycleIdle_e](#)

Parameters

in, out	<i>rfe__error__pointer</i>	- Error handling parameter: On success <code>#*rfe__error__pointer == rfe_error_none_e</code> is true, On failure <code>#*rfe__error__pointer != rfe_error_none_e</code> is true
---------	----------------------------	--

Returns

uint32_t - Absolute RFE-Fw time since initialization in 25 [ns] resolution.

3.8.4.12 rfe_getVersion()

```
rfe_version_t rfe_getVersion (
    volatile rfe_error_t * rfe__error__pointer )
```

This function returns version and variant details of the RFE-Fw hardware and firmware.

This function returns after RFE-Fw has accepted or rejected the call.

This function can return one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_sharedData_redundancyCheckFailed_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Precondition

The RFE-Fw must be in one of the following states:

- [rfe_state_initialized_e](#)
- [rfe_state_configured_e](#)
- [rfe_state_radarCycleIdle_e](#)

Parameters

in, out	<i>rfe__error__pointer</i>	- Error handling parameter: On success <code>#*rfe__error__pointer == rfe_error_none_e</code> is true, On failure <code>#*rfe__error__pointer != rfe_error_none_e</code> is true
---------	----------------------------	--

Returns

Cdd_RfeVersionType - Structure containing RFE-Fw variant and version details.

3.8.4.13 rfe_monitorRead()

```
rfe_radarCycleCount_t rfe_monitorRead (
    rfe_monitorSelect_t monitorSelect,
    rfe_monitorValues_t * pMonitorValues,
    volatile rfe_error_t * rfe___error___pointer )
```

This function will read-out one or more RFE-Fw monitors.

Monitors to be read are selected by `monitorSelect`.

This function returns after RFE-Fw has accepted or rejected the API call.

This function can return one of following errors:

- `rfe_error_api_unsynchronized_e`
- `rfe_error_api_operationNotAllowed_e`
- `rfe_error_api_busy_e`
- `rfe_error_api_invalidArgumentValue_e`
- Any `cmdInterface` error of `rfe_error_t`.
- `rfe_error_sharedData_redundancyCheckFailed_e`
- `rfe_error_rfeFuSaFault_e`

Precondition

The RFE-Fw must be in one of the following states:

- `rfe_state_initialized_e`
- `rfe_state_configured_e`
- `rfe_state_radarCycleIdle_e` (`RFE_MONITOR_SELECT_TEMP_IMMEDIATELY` bit of `monitorSelect` must be '0')
- `rfe_state_testContinuousWaveTransmission_e`

Parameters

in	<code>monitorSelect</code>	- Selection of the RFE-Fw monitors to read-out
out	<code>*pMonitorValues</code>	- The monitor values will be written to this pointer
in, out	<code>rfe___error___pointer</code>	- Error handling parameter: On success <code>#*rfe___error___pointer == rfe_error_none_e</code> is true, On failure <code>#*rfe___error___pointer != rfe_error_none_e</code> is true

Returns

`Cdd_RfeRadarCycleCountType` - The radar cycle and chirp sequence count, indicating when the monitors were read.

3.8.4.14 rfe_radarCycleStart()

```
void rfe_radarCycleStart (
    uint16_t radarCycleCount,
    bool isScheduled,
    uint32_t startTime,
    volatile rfe_error_t * rfe___error___pointer )
```

This function starts the configured radar cycles.

The configured radar cycles are started at `startTime` or as soon as possible depending on the `isScheduled` parameter. The radar cycle start can be scheduled from 10 us up to 1000 ms in the future (time interval started after the reception of the API call by the RFE-Fw). This function returns only after RFE-Fw has processed the command and responded.

This function can return one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- [rfe_error_api_invalidArgumentValue_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_sharedData_redundancyCheckFailed_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Precondition

The RFE-Fw must be in [rfe_state_configured_e](#) state.

Parameters

in	<i>radarCycleCount</i>	- Number of radar cycles to start. 0 indicates an infinite number of radar cycles, until stopped by <code>Cdd_Rfe_radarCycleStop()</code> .
in	<i>isScheduled</i>	- If true, radar cycle starts at <code>startTime</code> . If false, radar cycle starts asap.
in	<i>startTime</i>	- The absolute start time delay in 25 [ns] resolution, according to the RFE-Fw time.
in, out	<i>rfe___error___pointer</i>	- Error handling parameter: On success <code>*rfe___error___pointer == rfe_error_none_e</code> is true, On failure <code>*rfe___error___pointer != rfe_error_none_e</code> is true

Postcondition

When the API call is accepted and radar cycle is started, RFE-Fw state will become [rfe_state_busy_e](#) during the the calibration, BIST and chirp sequences. During the radar cycle idle windows the state is [rfe_state_radarCycleIdle_e](#). RFE-Fw state returns to [rfe_state_configured_e](#), when all radar cycles are completed.

3.8.4.15 rfe_radarCycleStop()

```
uint16_t rfe_radarCycleStop (
    volatile rfe_error_t * rfe__error__pointer )
```

This function stops the active radar cycles.

The active radar cycle will not be finished, but it is stopped as soon as the command has been received. Pending parameters update will be cancelled. It can be called, for example, when the application needs to do a full reconfiguration or needs to go to a lower power mode or for any external event where active radar cycle needs to be stopped. It returns after RFE-Fw has accepted or rejected the call.

This function can return one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_sharedData_redundancyCheckFailed_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Precondition

The RFE-Fw state must be [rfe_state_radarCycleIdle_e](#).

Parameters

in, out	<i>rfe__error__pointer</i>	- Error handling parameter: On success <code>#*rfe__error__pointer == rfe_error_none_e</code> is true, On failure <code>#*rfe__error__pointer != rfe_error_none_e</code> is true
---------	----------------------------	--

Returns

uint16 - Index of the active radar cycle at which radar cycle was stopped.

Postcondition

Radar cycle is stopped and RFE-Fw state returns to [rfe_state_configured_e](#).

3.8.4.16 rfe_setFrontEnd()

```
void rfe_setFrontEnd (
    uint8_t frontEndPos,
    volatile rfe_error_t * rfe__error__pointer )
```

Function to change the associated Front-End "subject" in a cascaded setup.

The function must be used only in a cascaded setup, else an error will be returned. The possible errors are :

- unresponsive
- not synchronized
- busy
- operation not allowed - only this is a specific error for this operation request, returned from RFE-Fw
- invalid argument

Parameters

in	<i>frontEndPos</i>	- the Front-End to be used as conversation subject between the two cores
in, out	<i>rfe__error__pointer</i>	- Error handling parameter: On success <code>#*rfe__error__pointer == rfe_error_none_e</code> is true, On failure <code>#*rfe__error__pointer != rfe_error_none_e</code> is true

3.8.4.17 `rfe_setNextRadarCycleStartTime()`

```
uint16_t rfe_setNextRadarCycleStartTime (
    uint32_t startTime,
    volatile rfe_error_t * rfe__error__pointer )
```

This function updates the start time of the next radar cycle.

`Cdd_Rfe_radarCycleStart()` starts one or multiple radar cycles immediately or at a schedule time, with a certain period ([radarCycleDuration](#)), as per the configuration. This function updates the absolute start time of the next radar cycle according to the RFE-Fw time. That radar cycle will thereby be delayed or pulled in. Subsequent radar cycles will shift in time accordingly to maintain the configured [radarCycleDuration](#). This functionality can be used to synchronize multiple radar sensors in time. The next radar cycle start time can be delayed up to 100 ms and should not conflict with the current radar cycle time schedule (in case of pull in). This function returns after RFE-Fw has accepted or rejected the API call.

This function can return one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- [rfe_error_api_invalidArgumentValue_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_sharedData_redundancyCheckFailed_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Precondition

The RFE-Fw must be in [rfe_state_radarCycleIdle_e](#) state.

Parameters

in	<i>startTime</i>	- Absolute start time of the next radar cycle in 25 [ns] resolution.
in, out	<i>rfe__error__pointer</i>	- Error handling parameter: On success <code>#*rfe__error__pointer == rfe_error_none_e</code> is true, On failure <code>#*rfe__error__pointer != rfe_error_none_e</code> is true

Returns

uint16 - Index of the next radar cycle of which the start time is set.

Postcondition

Next radar cycle starts at the given start time.

3.8.4.18 rfe_sync()

```
void rfe_sync (
    volatile rfe_error_t * rfe__error__pointer )
```

This function synchronizes the rfeDriver on M7-0 core with the RFE-Fw running on M7-1 core.

This is the first RFE Driver function to be called. The first call to this function initiates the control core <-> RFE-Fw synchronization protocol and always fails. This function shall be called until the `rfe_error_api_unresponsive_e` error is not set anymore, which indicates that the synchronization protocol was executed successfully. The error is get using `CDD_Rfe_GetError()` funtion.

This function can set one of following errors:

- `rfe_error_api_unresponsive_e`
- `rfe_error_sharedData_redundancyCheckFailed_e`

Precondition

The RFE-Fw (M7-1 core) must be started.

Parameters

in, out	<i>rfe__error__pointer</i>	- Error handling parameter: On success <code>#*rfe__error__pointer == rfe_error_none_e</code> is true, On failure <code>#*rfe__error__pointer != rfe_error_none_e</code> is true
---------	----------------------------	--

Postcondition

RFE-Fw will be in `rfe_state_initialized_e` state after the correct initialization.

3.8.4.19 rfe_testContinuousWaveTransmissionStart()

```
void rfe_testContinuousWaveTransmissionStart (
    rfe_chirpProfileIndex_t profileIndex,
    volatile rfe_error_t * rfe___error___pointer )
```

This function start continuous wave transmission.

It uses the RFE-Fw profile as configured via [rfe_configure\(\)](#) as specified by `profileIndex`. The effective chirp bandwidth of this profile is neglected. The frequency is kept at the configured center frequency. This function returns after RFE-Fw has accepted or rejected the call.

This function can return one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- [rfe_error_api_invalidArgumentValue_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_sharedData_redundancyCheckFailed_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Precondition

RFE must be in [rfe_state_configured_e](#) state.

Parameters

in	<i>profileIndex</i>	- Index of the chirp profile to be used for continuous wave transmission.
in, out	<i>rfe___error___pointer</i>	- Error handling parameter: On success <code>#*rfe___error___pointer == rfe_error_none_e</code> is true, On failure <code>#*rfe___error___pointer != rfe_error_none_e</code> is true

Postcondition

RFE-Fw goes to [rfe_state_testContinuousWaveTransmission_e](#) state.

3.8.4.20 rfe_testContinuousWaveTransmissionStop()

```
void rfe_testContinuousWaveTransmissionStop (
    volatile rfe_error_t * rfe___error___pointer )
```

This function stops continuous wave transmission.

Continuous wave transmission can be started via [rfe_testContinuousWaveTransmissionStart\(\)](#). This function returns after RFE-Fw has accepted or rejected the API call.

This function can return one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_sharedData_redundancyCheckFailed_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Precondition

RFE-Fw must be in [rfe_state_testContinuousWaveTransmission_e](#) state.

Parameters

in, out	<i>rfe__error__pointer</i>	- Error handling parameter: On success <code>#*rfe__error__pointer == rfe_error_none_e</code> is true, On failure <code>#*rfe__error__pointer != rfe_error_none_e</code> is true
---------	----------------------------	--

Postcondition

RFE-Fw goes to [rfe_state_configured_e](#) state.

3.8.4.21 [rfe_testGetInternalError\(\)](#)

```
uint32_t rfe_testGetInternalError (
    volatile rfe\_error\_t * rfe__error__pointer )
```

This function provides the Internal error occurred.

This function returns internal error code. This function returns whether RFE has accepted or rejected the API call.

This function can return one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- [rfe_error_api_invalidArgumentValue_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_sharedData_redundancyCheckFailed_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Returns

the error

3.8.4.22 rfe_testSetParam()

```
void rfe_testSetParam (
    rfe_testParam_t testParam,
    uint32_t value,
    volatile rfe_error_t * rfe___error___pointer )
```

This function sets a test parameter.

This function returns whether RFE-Fw has accepted or rejected the API call.

This function can return one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- [rfe_error_api_invalidArgumentValue_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_sharedData_redundancyCheckFailed_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Precondition

The RFE-Fw must be in one of the following states:

- [rfe_state_initialized_e](#)
- [rfe_state_configured_e](#)

Parameters

in	<i>testParam</i>	- The test parameter to set
in	<i>value</i>	- The value to set
in, out	<i>rfe___error___pointer</i>	- Error handling parameter: On success <code>#*rfe___error___pointer == rfe_error_none_e</code> is true, On failure <code>#*rfe___error___pointer != rfe_error_none_e</code> is true

3.8.4.23 rfe_updateBegin()

```
void rfe_updateBegin (
    void )
```

This function begins a RFE configuration update operation.

When the RFE Configuration (as set via `Cdd_Rfe_configure()`) is required to be updated then this is the first function to be called. This function will clear the buffered RFE configuration. This function does not interact with the RFE, instead it buffers data locally. This function does not return any error.

Postcondition

The functions `Cdd_Rfe_updateParam()` and/or `Cdd_Rfe_updateDynamicTable()` must be called after this function to buffer the configuration and then push the configuration into RFE FW via the configuration update via `Cdd_Rfe_updatePush()`.

3.8.4.24 rfe_updateDynamicTable()

```
void rfe_updateDynamicTable (
    uint8_t dynamicTableIndex,
    rfe_dynParamsSelect_t paramsSelect,
    volatile rfe_error_t * rfe___error___pointer )
```

This function stops continuous wave transmission.

Continuous wave transmission can be started via `rfe_testContinuousWaveTransmissionStart()`. This function returns whether RFE-Fw has accepted or rejected the call.

This function can sets one of following errors:

- `rfe_error_api_unsynchronized_e`
- `rfe_error_api_operationNotAllowed_e`
- `rfe_error_api_busy_e`
- Any `cmdInterface` error of `rfe_error_t`.
- `rfe_error_sharedData_redundancyCheckFailed_e`
- `rfe_error_rfeFuSaFault_e`

Parameters

in	<i>dynamicTableIndex</i>	- the index of the dynamic table to be updated
in	<i>paramsSelect</i>	- the parameters selection
in, out	<i>rfe___error___pointer</i>	- Error handling parameter: On success <code>#*rfe___error___pointer == rfe_error_none_e</code> is true, On failure <code>#*rfe___error___pointer != rfe_error_none_e</code> is true

Precondition

RFE must be in `rfe_state_testContinuousWaveTransmission_e` state.

Postcondition

RFE goes to `rfe_state_configured_e` state.

3.8.4.25 rfe_updateParam()

```
void rfe_updateParam (
    rfeCfg_section_t section,
    rfe_paramUpdate_t param,
    uint32_t value,
    volatile rfe_error_t * rfe___error___pointer )
```

This function start continuous wave transmission.

It uses the RFE-Fw profile as configured via [rfe_configure\(\)](#) as specified by `profileIndex`. The effective chirp bandwidth of this profile is neglected. The frequency is kept at the configured center frequency. This function returns after RFE-Fw has accepted or rejected the call.

This function can sets one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- [rfe_error_api_invalidArgumentValue_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_sharedData_redundancyCheckFailed_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Precondition

RFE-Fw must be in [rfe_state_configured_e](#) state.

Parameters

in	<i>section</i>	- the section mask, as in rfeCfg_section_tenum
in	<i>param</i>	- index of the parameter to be updated, as in rfe_paramUpdate_t
in	<i>value</i>	- the new value of the parameter
in, out	<i>rfe___error___pointer</i>	- Error handling parameter: On success <code>#*rfe___error___pointer == rfe_error_none_e</code> is true, On failure <code>#*rfe___error___pointer != rfe_error_none_e</code> is true

Postcondition

RFE goes to [rfe_state_testContinuousWaveTransmission_e](#) state.

3.8.4.26 rfe_updatePush()

```
uint16_t rfe_updatePush (
    volatile rfe_error_t * rfe___error___pointer )
```

This function sets a test parameter.

This function returns whether RFE-Fw has accepted or rejected the call.

This function can sets one of following errors:

- [rfe_error_api_unsynchronized_e](#)
- [rfe_error_api_operationNotAllowed_e](#)
- [rfe_error_api_busy_e](#)
- [rfe_error_api_invalidArgumentValue_e](#)
- Any *cmdInterface* error of [rfe_error_t](#).
- [rfe_error_sharedData_redundancyCheckFailed_e](#)
- [rfe_error_rfeFuSaFault_e](#)

Precondition

The RFE-Fw must be in one of the following states:

- [rfe_state_initialized_e](#)
- [rfe_state_configured_e](#)

Parameters

<i>in, out</i>	<i>rfe__error__pointer</i>	- Error handling parameter: On success <code>#*rfe__error__pointer == rfe_error_none_e</code> is true, On failure <code>#*rfe__error__pointer != rfe_error_none_e</code> is true
----------------	----------------------------	--

Returns

the `radarCycleCount` reported by the firmware

3.8.4.27 rfeCmdClient_call()

```
void rfeCmdClient_call (
    rfeCmdServer_cmdId_t cmdId,
    volatile rfe\_error\_t * rfe__error__pointer )
```

Calls a remote command.

This function will trigger a command execution on RFE-Fw. Before calling this function, the parameters of the function have to be added using the command client functions starting with the function name `rfeCmdClient_cmd`↔`ParamAdd` (e.g. `rfeCmdClient_cmdParamAddUInt8`).

[rfeCmdClient_call\(\)](#) will return once the RFE-Fw has finished executing the command and written the response data to shared memory (i.e. blocking in behavior). In case the function executed successfully, response data can be read using functions starting with the name `rfeCmdClient_cmdRespGet` (e.g. `rfeCmdClient_cmdRespGetUInt8`).

The Error code will contain errors from rfe command client layer, rfe command server layer and the remotely executed API function.

Precondition

The command interface is initialized successfully using the function [rfeCmdClient_init\(\)](#). The function [rfeCmdClient_cmdParamStart\(\)](#) was called before this function. If the remote function requires parameters they have to be added too.

Parameters

in	<i>cmdId</i>	A command ID matching the command that should be executed (see rfeCmdServer_cmdId_t definition)
in, out	<i>rfe__error__pointer</i>	Error handling parameter: • rfe_error_none_e = No error occurred. • Any other value = An error occurred.

Postcondition

If successful: The command finished executing on RFE-Fw. In case the function returns response values they can be read using the [rfeCmdClient_cmdRespGet*\(\)](#) functions.

3.8.4.28 rfeCmdClient_checkNotBusy()

```
void rfeCmdClient_checkNotBusy (
    volatile rfe\_error\_t * rfe__error__pointer )
```

This function checks whether RFE-Fw is not busy.

If RFE-Fw is busy, an error is set to the error pointer.

Parameters

in, out	<i>rfe__error__pointer</i>	- Error handling parameter: On success <i>*rfe__error__pointer</i> is not changed, On failure <i>*rfe__error__pointer</i> == rfe_error_api_busy_e , or it get the error from reading the shared memory space
---------	----------------------------	--

3.8.4.29 rfeCmdClient_cmdParamAddBool()

```
static void rfeCmdClient_cmdParamAddBool (
    bool value,
    volatile rfe\_error\_t * rfe__error__pointer ) [inline], [static]
```

Adds a bool value as parameter to a remote function.

Note: A bool value is internally written as 8 bit value with 0 for false and 1 for true. The function is not blocking.

Precondition

Function [rfeCmdClient_cmdParamStart\(\)](#) was called.

Parameters

in	<i>value</i>	The value that should be added as parameter.
in, out	<i>rfe__error__pointer</i>	Error handling parameter: rfe_error_none_e = The parameter was added successfully. rfe_error_rfeFuSaFault_e = Too many parameters added to the buffer or this function was called without calling rfeCmdClient_cmdParamStart() as first step of command call preparation.

Postcondition

Other required parameters can be added or the function can be called using the function [rfeCmdClient_call\(\)](#).

3.8.4.30 rfeCmdClient_cmdParamAddBuffer()

```
void rfeCmdClient_cmdParamAddBuffer (
    const void * pInput,
    uint32_t byteCount,
    volatile rfe\_error\_t * rfe__error__pointer )
```

Adds an arbitrary block of data as parameter to a function.

Note: This function is non blocking.

Precondition

Function [rfeCmdClient_cmdParamStart\(\)](#) was called.

Parameters

in	<i>pInput</i>	A pointer to the buffer containing the data that should be added as parameter.
in	<i>byteCount</i>	The count of bytes that should be added from the buffer.
in, out	<i>rfe__error__pointer</i>	Error handling parameter: rfe_error_none_e = The parameter was added successfully. rfe_error_rfeFuSaFault_e = Too many parameters added to the buffer or this function was called without calling rfeCmdClient_cmdParamStart() as first step of command call preparation.

Postcondition

Other required parameters can be added or the function can be called using the function [rfeCmdClient_call\(\)](#).

3.8.4.31 rfeCmdClient_cmdParamAddInt16()

```
static void rfeCmdClient_cmdParamAddInt16 (
    int16_t value,
    volatile rfe_error_t * rfe__error__pointer ) [inline], [static]
```

Adds an int16_t value as parameter to a remote function.

Note: This function is non blocking.

Parameters

in	value	The value that should be added as parameter.
in, out	rfe__error__pointer	Error handling parameter: - rfe_error_none_e = The parameter was added successfully. - rfe_error_rfeFuSaFault_e = Too many parameters added to the buffer or this function was called without calling rfeCmdClient_cmdParamStart() as first step of command call preparation.

Postcondition

Other required parameters can be added or the function can be called using the function rfeCmdClient_call().

3.8.4.32 rfeCmdClient_cmdParamAddInt32()

```
static void rfeCmdClient_cmdParamAddInt32 (
    int32_t value,
    volatile rfe_error_t * rfe__error__pointer ) [inline], [static]
```

Adds an int32_t value as parameter to a remote function.

Note: This function is non blocking.

Precondition

Function rfeCmdClient_cmdParamStart() was called.

Parameters

in	value	The value that should be added as parameter.
in, out	rfe__error__pointer	Error handling parameter: - rfe_error_none_e = The parameter was added successfully. - rfe_error_rfeFuSaFault_e = Too many parameters added to the buffer or this function was called without calling rfeCmdClient_cmdParamStart() as first step of command call preparation.

Postcondition

Other required parameters can be added or the function can be called using the function [rfeCmdClient_call\(\)](#).

3.8.4.33 rfeCmdClient_cmdParamAddInt8()

```
static void rfeCmdClient_cmdParamAddInt8 (
    int8_t value,
    volatile rfe_error_t * rfe__error__pointer ) [inline], [static]
```

Adds an int8_t value as parameter to a remote function.

Note: This function is non blocking.

Precondition

Function [rfeCmdClient_cmdParamStart\(\)](#) was called.

Parameters

in	value	The value that should be added as parameter.
in, out	rfe__error__pointer	Error handling parameter: rfe_error_none_e = The parameter was added successfully. rfe_error_rfeFuSaFault_e = Too many parameters added to the buffer or this function was called without calling rfeCmdClient_cmdParamStart() as first step of command call preparation.

Postcondition

Other required parameters can be added or the function can be called using the function [rfeCmdClient_call\(\)](#).

3.8.4.34 rfeCmdClient_cmdParamAddPointer()

```
void rfeCmdClient_cmdParamAddPointer (
    void * pValue,
    volatile rfe_error_t * rfe__error__pointer )
```

Adds a pointer value as parameter to a remote function.

Note: The pointer is transported as 32 bit pointer as the SRAM for R41 is placed below 0x100000000. Only physical pointer is used and this is for further compatibility. The function is not blocking.

Precondition

Function [rfeCmdClient_cmdParamStart\(\)](#) was called.

Parameters

in	<i>pValue</i>	The pointer value that should be added as parameter.
in, out	<i>rfe__error__pointer</i>	Error handling parameter: rfe_error_none_e = The parameter was added successfully. rfe_error_rfeFuSaFault_e = Too many parameters added to the buffer or this function was called without calling rfeCmdClient_cmdParamStart() as first step of command call preparation.

Postcondition

Other required parameters can be added or the function can be called using the function [rfeCmdClient_call\(\)](#).

3.8.4.35 rfeCmdClient_cmdParamAddUint16()

```
static void rfeCmdClient_cmdParamAddUint16 (
    uint16_t value,
    volatile rfe\_error\_t * rfe__error__pointer ) [inline], [static]
```

Adds an uint16_t value as parameter to a remote function.

Note: This function is non blocking.

Precondition

Function [rfeCmdClient_cmdParamStart\(\)](#) was called.

Parameters

in	<i>value</i>	The value that should be added as parameter.
in, out	<i>rfe__error__pointer</i>	Error handling parameter: rfe_error_none_e = The parameter was added successfully. rfe_error_rfeFuSaFault_e = Too many parameters added to the buffer or this function was called without calling rfeCmdClient_cmdParamStart() as first step of command call preparation.

Postcondition

Other required parameters can be added or the function can be called using the function [rfeCmdClient_call\(\)](#).

3.8.4.36 rfeCmdClient_cmdParamAddUint32()

```
static void rfeCmdClient_cmdParamAddUint32 (
    uint32_t value,
    volatile rfe_error_t * rfe___error___pointer ) [inline], [static]
```

Adds an uint32_t value as parameter to a remote function.

Note: This function is non blocking.

Precondition

Function [rfeCmdClient_cmdParamStart\(\)](#) was called.

Parameters

in	value	The value that should be added as parameter.
in, out	rfe___error___pointer	Error handling parameter: • rfe_error_none_e = The parameter was added successfully. • rfe_error_rfeFuSaFault_e = Too many parameters added to the buffer or this function was called without calling rfeCmdClient_cmdParamStart() as first step of command call preparation.

Postcondition

Other required parameters can be added or the function can be called using the function [rfeCmdClient_call\(\)](#).

3.8.4.37 rfeCmdClient_cmdParamAddUint8()

```
static void rfeCmdClient_cmdParamAddUint8 (
    uint8_t value,
    volatile rfe_error_t * rfe___error___pointer ) [inline], [static]
```

Adds an uint8_t value as parameter to a remote function.

Note: This function is non blocking.

Precondition

Function [rfeCmdClient_cmdParamStart\(\)](#) was called.

Parameters

in	value	The value that should be added as parameter.
in, out	rfe___error___pointer	Error handling parameter: • rfe_error_none_e = The parameter was added successfully. • rfe_error_rfeFuSaFault_e = Too many parameters added to the buffer or this function was called without calling rfeCmdClient_cmdParamStart() as first step of command call preparation.

Postcondition

Other required parameters can be added or the function can be called using the function [rfeCmdClient_call\(\)](#).

3.8.4.38 rfeCmdClient_cmdParamStart()

```
void rfeCmdClient_cmdParamStart (
    void )
```

Starts the preparation for the call of a remote RFE-Fw function.

When calling functions on RFE-Fw, this function has to be executed as first step to prepare the remote function call. After calling this function the required parameters can be added to the function call.

Precondition

The command interface is initialized successfully using the function [rfeCmdClient_init\(\)](#).

Postcondition

Function parameters can be added and the remote function called using [rfeCmdClient_call\(\)](#)

3.8.4.39 rfeCmdClient_cmdRespGetBool()

```
bool rfeCmdClient_cmdRespGetBool (
    volatile rfe\_error\_t * rfe__error__pointer )
```

Reads a bool value from the response buffer.

Note: a bool is read as 8 bit value. A 0 will translate to false, any other value to true. The function is not blocking.

Precondition

Function [rfeCmdClient_cmdCall\(\)](#) was called without error returned and the remote function added responses.

Parameters

<i>in, out</i>	<i>rfe__error__pointer</i>	Error handling parameter: rfe_error_none_e = The parameter was added successfully. rfe_error_rfeFuSaFault_e = Read length error.
----------------	----------------------------	--

Postcondition

Other response values can be read.

Returns

The value read from the response buffer.

3.8.4.40 rfeCmdClient_cmdRespGetBuf()

```
void rfeCmdClient_cmdRespGetBuf (
    void * pOutput,
    uint32_t byteCount,
    volatile rfe_error_t * rfe___error___pointer )
```

Reads an arbitrary block of data from the response buffer.

This function can be used to read an array of data from the response buffer. The function is not blocking.

Precondition

Function `rfeCmdClient_cmdCall()` was called without error returned and the remote function added responses.

Parameters

in	<i>pOutput</i>	Pointer to the memory where the read data should be written to.
in	<i>byteCount</i>	The count of bytes that should be read from response buffer and written to *pOutput.
in, out	<i>rfe___error___pointer</i>	Error handling parameter: • <code>rfe_error_none_e</code> = The parameter was added successfully. • <code>rfe_error_rfeFuSaFault_e</code> = Read length error.

Postcondition

Other response values can be read.

3.8.4.41 rfeCmdClient_cmdRespGetInt16()

```
int16_t rfeCmdClient_cmdRespGetInt16 (
    volatile rfe_error_t * rfe___error___pointer )
```

Reads an int16 value from the response buffer.

This function is not blocking.

Precondition

Function `rfeCmdClient_cmdCall()` was called without error returned and the remote function added responses.

Parameters

<code>in, out</code>	<code>rfe___error___pointer</code>	Error handling parameter: • <code>rfe_error_none_e</code> = The parameter was added successfully. • <code>rfe_error_rfeFuSaFault_e</code> = Read length error.
----------------------	------------------------------------	---

Postcondition

Other response values can be read.

Returns

The value read from the response buffer.

3.8.4.42 rfeCmdClient_cmdRespGetInt32()

```
int32_t rfeCmdClient_cmdRespGetInt32 (
    volatile rfe_error_t * rfe___error___pointer )
```

Reads an int32 value from the response buffer.

This function is not blocking.

Precondition

Function `rfeCmdClient_cmdCall()` was called without error returned and the remote function added responses.

Parameters

<code>in, out</code>	<code>rfe___error___pointer</code>	Error handling parameter: • <code>rfe_error_none_e</code> = The parameter was added successfully. • <code>rfe_error_rfeFuSaFault_e</code> = Read length error.
----------------------	------------------------------------	---

Postcondition

Other response values can be read.

Returns

The value read from the response buffer.

3.8.4.43 rfeCmdClient_cmdRespGetInt8()

```
int8_t rfeCmdClient_cmdRespGetInt8 (
    volatile rfe_error_t * rfe___error___pointer )
```

Reads an int8 value from the response buffer.

This function is not blocking.

Precondition

Function rfeCmdClient_cmdCall() was called without error returned and the remote function added responses.

Parameters

in, out	<i>rfe___error___pointer</i>	Error handling parameter: • rfe_error_none_e = The parameter was added successfully. • rfe_error_rfeFuSaFault_e = Read length error.
---------	------------------------------	---

Postcondition

Other response values can be read.

Returns

The value read from the response buffer.

3.8.4.44 rfeCmdClient_cmdRespGetUint16()

```
uint16_t rfeCmdClient_cmdRespGetUint16 (
    volatile rfe_error_t * rfe___error___pointer )
```

Reads an uint16 value from the response buffer.

This function is not blocking.

Precondition

Function rfeCmdClient_cmdCall() was called without error returned and the remote function added responses.

Parameters

in, out	<i>rfe___error___pointer</i>	Error handling parameter: • rfe_error_none_e = The parameter was added successfully. • rfe_error_rfeFuSaFault_e = Read length error.
---------	------------------------------	---

Postcondition

Other response values can be read.

Returns

The value read from the response buffer.

3.8.4.45 rfeCmdClient_cmdRespGetUint32()

```
uint32_t rfeCmdClient_cmdRespGetUint32 (
    volatile rfe_error_t * rfe__error__pointer )
```

Reads an uint32 value from the response buffer.

This function is not blocking.

Precondition

Function rfeCmdClient_cmdCall() was called without error returned and the remote function added responses.

Parameters

in, out	rfe__error__pointer	Error handling parameter: - rfe_error_none_e = The parameter was added successfully. - rfe_error_rfeFuSaFault_e = Read length error.
---------	---------------------	--

Postcondition

Other response values can be read.

Returns

The value read from the response buffer.

3.8.4.46 rfeCmdClient_cmdRespGetUint8()

```
uint8_t rfeCmdClient_cmdRespGetUint8 (
    volatile rfe_error_t * rfe__error__pointer )
```

Reads an uint8 value from the response buffer.

This function is not blocking.

Precondition

Function rfeCmdClient_cmdCall() was called without error returned and the remote function added responses.

Parameters

<code>in, out</code>	<code>rfe___error___pointer</code>	Error handling parameter: • <code>rfe_error_none_e</code> = The parameter was added successfully. • <code>rfe_error_rfeFuSaFault_e</code> = Read length error.
----------------------	------------------------------------	---

Postcondition

Other response values can be read.

Returns

The value read from the response buffer.

3.8.4.47 rfeCmdClient_configureInterrupt()

```
void rfeCmdClient_configureInterrupt (
    rfe_apiIRQ_t rfeAPIs,
    rfe_apiWaitForInterrupt_f apiWaitForInterrupt )
```

Configure Fw interrupt parameters The function align the internal API driver parameters to the application request.

Parameters

<code>in</code>	<code>rfeAPIs</code>	Bitmask of API functions that will trigger an interrupt.
<code>in</code>	<code>apiWaitForInterrupt</code>	Callback to be used if at least one API functions will trigger interrupt.

3.8.4.48 rfeCmdClient_getCurrentRespLen()

```
uint16_t rfeCmdClient_getCurrentRespLen (
    void )
```

Return the length of the last received answer from the Fw.

This is the only function which can inform the application about the length of the last received response.

Precondition

`rfe_sync()` was called successfully

Returns

Returns the read value from the sharedData.

3.8.4.49 rfeCmdClient_init()

```
void rfeCmdClient_init (
    void * pCmdIfBase,
    volatile rfe_error_t * rfe___error___pointer )
```

Initialize the RFE-Fw command client.

The initialization function initializes the command client. This initialization does mainly align on the shared memory address of the command response interface. If the RFE-Fw command server is not initialized when calling this function, it returns immediately the error code `rfe_error_api_unresponsive_e`. Calling this function before the RFE-Fw command server is running, will not lead to undefined behavior, but for correct operation of the command client this function has to be executed once without an error.

Pseudocode for successful initialization independent of RFE-Fw command server state:

```
do { rfeCmdClient_init( <a shared memory address>, rfe___error___pointer); } while(rfe_error_api_unresponsive_e == *rfe___error___pointer); if ( rfe_error_none_e == *rfe___error___pointer ) { ... (successfully initialized the RFE-Fw command client. Communication with RFE-Fw is enabled!) }
```

Precondition

ECC protection of shared memory has to be initialized

Parameters

in	<i>pCmdIfBase</i>	A pointer to the start of the command response memory region. The memory region has to be <code>RFE_CMD_IF_CMDRSPLN*2*4</code> bytes long. The memory region has to be uncached.
in, out	<i>rfe___error___pointer</i>	Error handling parameter: <code>rfe_error_none_e</code> = No error occurred. <code>rfe_error_api_unresponsive_e</code> = RFE-Fw command server do not respond. The function has to be called again until success.

Postcondition

If the function executed with an error code `rfe_error_none_e`, the command client is ready to communicate with RFE-Fw.

3.8.4.50 rfeCmdClient_sharedDataRead()

```
uint16_t rfeCmdClient_sharedDataRead (
    uint32_t idx,
    volatile rfe_error_t * rfe___error___pointer )
```

Reads a SharedData location.

SharedData is a set of register like values, which is written by RFE-M7-0 and can be read from the client. The values are 16-bit unsigned integers, but stored as 32 Bit values internally

Precondition

`rfe_sync()` was called successfully

Parameters

<code>in</code>	<code>idx</code>	The sharedData index to be read
<code>in, out</code>	<code>rfe___error___pointer</code>	Error handling parameter

Returns

Returns the read value from the sharedData.

3.8.4.51 rfeCrc_calBuf()

```
uint32_t rfeCrc_calBuf (
    void * bufPtr,
    uint32_t len )
```

3.9 RFE Abstract 2.0 - Front-End configuration structure : BLOB

RFE Abstract 2.0 constants for the specific structure for Front-End configuration.

Modules

- [RFE Abstract 2.0 BLOB - Constants](#)

RFE Abstract 2.0 constants for the specific structure for Front-End configuration.

- [RFE Abstract 2.0 BLOB - Low Level Interface](#)

RFE Abstract 2.0 low-level interfaces. These interfaces are sets of get()/set() function pairs for retrieving or updating data to/from the in-memory blob, as well as some helper functions.

3.9.1 Detailed Description

RFE Abstract 2.0 constants for the specific structure for Front-End configuration.

3.10 RFE Abstract 2.0 BLOB - Constants

RFE Abstract 2.0 constants for the specific structure for Front-End configuration.

Macros

- #define RFE_CFG_MAX_DATA_LENGTH (RFE_CMD_IF_MAXDATLEN - 2UL)
- #define RFE_CFG_SIZE_SECTION_METADATA (6UL)
- #define RFE_CFG_SIZE_SECTION_GENERAL (10UL)
- #define RFE_CFG_SIZE_SECTION_MONITOR_AND_SAFETY (179UL)
- #define RFE_CFG_SIZE_SECTION_RADAR_CYCLE (55UL)
- #define RFE_CFG_SIZE_SECTION_CHIRP_SEQUENCE (31UL)
- #define RFE_CFG_SIZE_SECTION_CHIRP_PROFILE (41UL)
- #define RFE_CFG_SIZE_SECTION_CRC (4UL)
- #define RFE_CFG_OFFSET_SECTION_METADATA ((uint16_t)(0x000UL + 0x000UL))
- #define RFE_CFG_OFFSET_SECTION_GENERAL ((uint16_t)(RFE_CFG_OFFSET_SECTION_METADATA + RFE_CFG_SIZE_SECTION_METADATA))
- #define RFE_CFG_OFFSET_SECTION_MONITOR_AND_SAFETY ((uint16_t)(RFE_CFG_OFFSET_SECTION_GENERAL + RFE_CFG_SIZE_SECTION_GENERAL))
- #define RFE_CFG_OFFSET_SECTION_RADAR_CYCLE ((uint16_t)(RFE_CFG_OFFSET_SECTION_MONITOR_AND_SAFETY + RFE_CFG_SIZE_SECTION_MONITOR_AND_SAFETY))
- #define RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_0 ((uint16_t)(RFE_CFG_OFFSET_SECTION_RADAR_CYCLE + RFE_CFG_SIZE_SECTION_RADAR_CYCLE))
- #define RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_0 ((uint16_t)(RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_0 + RFE_CFG_SIZE_SECTION_CHIRP_SEQUENCE))
- #define RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_1 ((uint16_t)(RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_0 + RFE_CFG_SIZE_SECTION_CHIRP_PROFILE))
- #define RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_1 ((uint16_t)(RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_1 + RFE_CFG_SIZE_SECTION_CHIRP_SEQUENCE))
- #define RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_2 ((uint16_t)(RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_1 + RFE_CFG_SIZE_SECTION_CHIRP_PROFILE))
- #define RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_2 ((uint16_t)(RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_2 + RFE_CFG_SIZE_SECTION_CHIRP_SEQUENCE))
- #define RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_3 ((uint16_t)(RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_2 + RFE_CFG_SIZE_SECTION_CHIRP_PROFILE))
- #define RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_3 ((uint16_t)(RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_3 + RFE_CFG_SIZE_SECTION_CHIRP_SEQUENCE))
- #define RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_4 ((uint16_t)(RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_3 + RFE_CFG_SIZE_SECTION_CHIRP_PROFILE))
- #define RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_4 ((uint16_t)(RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_4 + RFE_CFG_SIZE_SECTION_CHIRP_SEQUENCE))
- #define RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_5 ((uint16_t)(RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_4 + RFE_CFG_SIZE_SECTION_CHIRP_PROFILE))
- #define RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_5 ((uint16_t)(RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_5 + RFE_CFG_SIZE_SECTION_CHIRP_SEQUENCE))
- #define RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_6 ((uint16_t)(RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_5 + RFE_CFG_SIZE_SECTION_CHIRP_PROFILE))
- #define RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_6 ((uint16_t)(RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_6 + RFE_CFG_SIZE_SECTION_CHIRP_SEQUENCE))
- #define RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_7 ((uint16_t)(RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_6 + RFE_CFG_SIZE_SECTION_CHIRP_PROFILE))
- #define RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_7 ((uint16_t)(RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_7 + RFE_CFG_SIZE_SECTION_CHIRP_SEQUENCE))
- #define RFE_CFG_OFFSET_SECTION_CRC ((uint16_t)(RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_7 + RFE_CFG_SIZE_SECTION_CHIRP_PROFILE))
- #define RFE_CFG_SIZE_TOTAL (RFE_CFG_OFFSET_SECTION_CRC + RFE_CFG_SIZE_SECTION_CRC)
- #define RFE_CFG_PARAM_SECTION_MSK ((uint16_t)((1UL<<15) | ((1UL<<14) | (1UL<<13))))
- #define RFE_CFG_PARAM_SIZE_MSK ((uint16_t)((1UL<<12) | (1UL<<11)))
- #define RFE_CFG_PARAM_OFFSET_MSK ((uint16_t)(0x07ffUL))
- #define RFE_CFG_PARAM_SECTION_METADATA ((uint16_t)((0UL<<15) | ((0UL<<14) | (0UL<<13))))

- `#define RFE_CFG_PARAM_SECTION_GENERAL ((uint16_t)((0UL<<15) | ((0UL<<14) | (1UL<<13))))`
- `#define RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY ((uint16_t)((0UL<<15) | ((1UL<<14) | (0UL<<13))))`
- `#define RFE_CFG_PARAM_SECTION_RADAR_CYCLE ((uint16_t)((0UL<<15) | ((1UL<<14) | (1UL<<13))))`
- `#define RFE_CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG ((uint16_t)((1UL<<15) | ((0UL<<14) | (0UL<<13))))`
- `#define RFE_CFG_PARAM_SECTION_CHIRP_PROFILE ((uint16_t)((1UL<<15) | ((0UL<<14) | (1UL<<13))))`
- `#define RFE_CFG_PARAM_SECTION_CRC ((uint16_t)((1UL<<15) | ((1UL<<14) | (1UL<<13))))`
- `#define RFE_CFG_PARAM_SIZE_8_BIT ((uint16_t)((0UL<<12) | (0UL<<11)))`
- `#define RFE_CFG_PARAM_SIZE_16_BIT ((uint16_t)((0UL<<12) | (1UL<<11)))`
- `#define RFE_CFG_PARAM_SIZE_32_BIT ((uint16_t)((1UL<<12) | (0UL<<11)))`
- `#define RFE_CFG_PARAM_SIZE_32_BIT_FLOAT ((uint16_t)((1UL<<12) | (1UL<<11)))`
- `#define RFE_CHIRP_SEQUENCE_CONFIGS_MAX (8UL)`
- `#define RFE_CFG_PARAM_OFFSET_SHF ((uint8_t)(0UL))`
- `#define RFE_CHIRP_PROFILE_SEQUENCE_LENGTH_MAX (8UL)`
- `#define RFE_CHIRP_PROFILES_MAX (8UL)`
- `#define RFE_TX_COUNT (3UL)`
- `#define RFE_CHIRP_SEQUENCES_PER_RADAR_CYCLE_MAX (8UL)`
- `#define RFE_FUSA_FAULT_MASK_PARAM_COUNT (12UL)`
- `#define RFE_SW_CFG_MNGR_GENRAL_PARAMS ( (uint16_t) 19UL )`
- `#define RFE_SW_CFG_MNGR_MONITOR_AND_SAFETY_PARAMS ( (uint16_t) 83UL )`
- `#define RFE_SW_CFG_MNGR_RADAR_CYCLE_PARAMS ( (uint16_t) 28UL )`
- `#define RFE_SW_CFG_MNGR_CHIRP_SEQUENCE_PARAMS ( (uint16_t) 21UL )`
- `#define RFE_SW_CFG_MNGR_CHIRP_PROFILE_PARAMS ( (uint16_t) 22UL )`
- `#define RFE_SW_CFG_MNGR_NON_UPDATABLE_PARAMS ( (uint16_t) 10UL )`
- `#define RFE_SW_CFG_MNGR_NO_OF_SECTION ( (uint16_t) 6UL )`
- `#define RFE_SW_CFG_MNGR_UPDATABLE_PARAMS ( (uint16_t) 156UL )`

Typedefs

- `typedef uint8_t rfe_phaseRotation_t`

Enumerations

- `enum rfe_txTransmissionReferenceTime_t { rfe_txTransmissionReferenceTime_startOfDwellTime_e , rfe_txTransmissionReferenceTime_startOfSettleTime_e }`
- `enum rfe_chirpPllLoopFilterBandwidth_t { rfe_chirpPllLoopFilterBandwidth_200kHz_e = 0U , rfe_chirpPllLoopFilterBandwidth_250kHz_e = 1U , rfe_chirpPllLoopFilterBandwidth_300kHz_e = 2U , rfe_chirpPllLoopFilterBandwidth_350kHz_e = 3U , rfe_chirpPllLoopFilterBandwidth_400kHz_e = 4U , rfe_chirpPllLoopFilterBandwidth_450kHz_e = 5U , rfe_chirpPllLoopFilterBandwidth_500kHz_e = 6U , rfe_chirpPllLoopFilterBandwidth_550kHz_e = 7U , rfe_chirpPllLoopFilterBandwidth_600kHz_e = 8U , rfe_chirpPllLoopFilterBandwidth_650kHz_e = 9U , rfe_chirpPllLoopFilterBandwidth_700kHz_e = 10U , rfe_chirpPllLoopFilterBandwidth_750kHz_e = 11U , rfe_chirpPllLoopFilterBandwidth_800kHz_e = 12U , rfe_chirpPllLoopFilterBandwidth_850kHz_e = 13U , rfe_chirpPllLoopFilterBandwidth_900kHz_e = 14U , rfe_chirpPllLoopFilterBandwidth_950kHz_e = 15U , rfe_chirpPllLoopFilterBandwidth_1000kHz_e = 16U , rfe_chirpPllLoopFilterBandwidth_1050kHz_e = 17U , rfe_chirpPllLoopFilterBandwidth_1100kHz_e = 18U , rfe_chirpPllLoopFilterBandwidth_1150kHz_e = 19U , rfe_chirpPllLoopFilterBandwidth_1200kHz_e = 20U , rfe_chirpPllLoopFilterBandwidth_1250kHz_e = 21U , rfe_chirpPllLoopFilterBandwidth_1300kHz_e = 22U , rfe_chirpPllLoopFilterBandwidth_1350kHz_e = 23U , rfe_chirpPllLoopFilterBandwidth_1400kHz_e = 24U , rfe_chirpPllLoopFilterBandwidth_1450kHz_e = 25U , rfe_chirpPllLoopFilterBandwidth_1500kHz_e = 26U , rfe_chirpPllLoopFilterBandwidth_1550kHz_e = 27U , rfe_chirpPllLoopFilterBandwidth_1600kHz_e = 28U , rfe_chirpPllLoopFilterBandwidth_1650kHz_e = 29U , rfe_chirpPllLoopFilterBandwidth_1700kHz_e = 30U , rfe_chirpPllLoopFilterBandwidth_invalid_e }`
- `enum rfe_effectiveSamplingFrequency_t { rfe_effectiveSamplingFrequency_40MHz_e , rfe_effectiveSamplingFrequency_20MHz_e , rfe_effectiveSamplingFrequency_10MHz_e , rfe_effectiveSamplingFrequency_5MHz_e , rfe_effectiveSamplingFrequency_invalid_e }`

3.10.1 Detailed Description

RFE Abstract 2.0 constants for the specific structure for Front-End configuration.

Here we describe the RFE configuration binary object (blob) which is to be provided as input to [rfe_configure\(\)](#) API function. The blob consists of multiple memory sections as shown in the below table. The RFE Configuration BLOB can be created by the RFE configuration generator. The functions in [rfe_blob_access.h](#) are used to retrieve or update the blob configuration data at run time. After local manipulation, (i.e. updating the blob in the platform memory) the configuration needs to be reloaded to the RFE by calling [rfe_configure\(\)](#). This will reconfigure and calibrate the full RFE, which takes in the order of tens of milliseconds depending on the amount of chirp profiles used. To update a subset of RFE parameters in a timely manner (tens of microseconds) it is recommend to use the [rfe_update.h](#) functions. These updates can even take place when the radar cycle is active, during the radar cycle idle time.

Section	Multiplicity	Description
metadata	1	Contains the number of sequences, chirp profiles and tool version
general	1	Contains the general parameters
monitorAndSafety	1	Contains the monitor and safety parameters
radarCycle	1	Contains the radar cycle parameters
chirpSequenceConfig	RFE_CHIRP_SEQUENCE_CONFIGS_MAX	Contains the chirp sequence parameters for each chirp sequence configuration
chirpProfile	RFE_CHIRP_PROFILES_MAX	Contains the chirp profile parameters for each profile

3.10.2 metadata

This section describes the metadata section of the RFE Configuration BLOB.

Parameter	Type	Min	Max	Set	Get
chirpSequenceConfigCount	uint8↔ _t	1	RFE_CHIRP_SEQUENCE_CONFIGS_MAX	—	rfeCfg_metadata_getChirpSequenceConfigCount()
chirpProfileCount	uint8↔ _t	1	RFE_CHIRP_PROFILES_MAX	—	rfeCfg_metadata_getChirpProfileCount()
majorVersion	uint8↔ _t	0	0	—	—
minorVersion	uint8↔ _t	8	8	—	—
patchVersion	uint8↔ _t	13	13	—	—

3.10.2.1 chirpSequenceConfigCount

The number of chirp **sequence** configurations that will be configured on the Front-End.

3.10.2.2 chirpProfileCount

The number of chirp **profile** configurations that will be stored on the Front-End.

3.10.2.3 majorVersion

The major version number of the binary RFE configuration format.

3.10.2.4 minorVersion

The minor version number of the binary RFE configuration format.

3.10.2.5 patchVersion

The patch version number of the binary RFE configuration format.

3.10.2.6 hashVersion

The hash version number of the binary RFE configuration format.

3.10.3 general

This section describes the general section of the RFE Configuration BLOB.

Parameter	Type	Min	Max	Set	Get
powerMode	uint8↔ _t	0	1	rfeCfg_general_setPowerMode()	rfeCfg_general_getPowerMode()
pdcDecimationFilter	uint8↔ _t	0	1	—	rfeCfg_general_getPdcBitwidth()
jumpbackTimeTicks	uint8↔ _t	1	254	—	rfeCfg_general_getJumpbackTimeTicks()
txCalibration (optional)	uint8↔ _t	0	1	—	rfeCfg_general_getTxCalibrationMode()

3.10.3.1 powerMode

Selects the powerMode of the rfe:

- 0: lower power
- 1: active

Warning

If a sequence uses multiple profiles with different TXs enabled (see [txTransmissionEnable](#)), this setting must be configured to "lower power" otherwise a lock-step error would be triggered from ISM.

3.10.3.2 pdcDecimationFilter

Selects the decimation filter of the programmable decimation chain. This option reduces the effective sampling rate of the ADC as follows:

- 0: **Narrow** bandwidth with a passband response up to $0.4 \times f_{\text{EFFECTIVE_SAMPLE_RATE}}$.
- 1: **Wide** bandwidth with a passband response up to $0.5 \times f_{\text{EFFECTIVE_SAMPLE_RATE}}$.

Remarks

For a detailed description, please refer to the Reference Manual (RM00227), chapter "Programmable decimation chain".

3.10.3.3 jumpbackTimeTicks

The chirp jumpback time for all chirps in 25 [ns] resolution.

3.10.3.4 txCalibration

Configures whether the TX antennas will perform calibration with **normal radiation levels**, or **significantly reduced radiation levels**.

In case of calibration with *reduced* transmission radiation, the [txCalibration_thresholdHigh](#) and [txCalibration_thresholdLow](#) must also be set.

Remarks

Default value: 1 (i.e. **with transmission**, if omitted from XML when generating)

3.10.4 monitorAndSafety

This section describes the monitorAndSafety section of the RFE Configuration BLOB.

Parameter	Type	Min	Max	Set	Get
rxSatCountResetEveryChirpSequence	uint8_t	0	1	—	rfeCfg_monitorAndSafety_getRxSatCountResetEveryChirpSequence()
rxSaturationThreshold	uint8_t	0	0x0F	—	rfeCfg_monitorAndSafety_getRxSaturationThreshold()
rxSaturationCountLimit	uint16_t	0	0x3FFFFFF	—	rfeCfg_monitorAndSafety_getRxSaturationCountLimit()
adcClippingCountResetEveryChirpSequence	bool	0	1	—	rfeCfg_monitorAndSafety_getAdcClippingCountResetEveryChirpSequence()
frequencyForBist	uint32_t	75000000	82000000	rfeCfg_monitorAndSafety_setFrequencyForBist()	rfeCfg_monitorAndSafety_getFrequencyForBist()
zeroHrRefForRxPhaseDiffLnaThreshold	int32_t	-1800	1800	rfeCfg_monitorAndSafety_setZeroHrRefForRxPhaseDiffLnaThreshold()	rfeCfg_monitorAndSafety_getZeroHrRefForRxPhaseDiffLnaThreshold()
rxPhaseDiffLnaThreshold	uint32_t	0	400	rfeCfg_monitorAndSafety_setRxPhaseDiffLnaThreshold()	rfeCfg_monitorAndSafety_getRxPhaseDiffLnaThreshold()
zeroHourReferenceForRxGainDiffLna	int32_t	400	400	rfeCfg_monitorAndSafety_setZeroHourReferenceForRxGainDiffLna()	rfeCfg_monitorAndSafety_getZeroHourReferenceForRxGainDiffLna()
rxGainDiffLnaThreshold	uint32_t	0	400	rfeCfg_monitorAndSafety_setRxGainDiffLnaThreshold()	rfeCfg_monitorAndSafety_getRxGainDiffLnaThreshold()

Parameter	Type	Min	Max	Set	Get
zeroHrRefForRxPhaseDiffLna	float32_t	-1800	1800	rfeCfg_monitorAndSafety_setBist()	rfeCfg_monitorAndSafety_getBist()
rxPhaseDiffMixerThresholdLna	float32_t	0	400	rfeCfg_monitorAndSafety_setBist()	rfeCfg_monitorAndSafety_getBist()
zeroHourReferenceForRxGainDiffMixer	float32_t	400	400	rfeCfg_monitorAndSafety_setBist()	rfeCfg_monitorAndSafety_getBist()
rxGainDiffMixerThresholdLna	float32_t	0	400	rfeCfg_monitorAndSafety_setBist()	rfeCfg_monitorAndSafety_getBist()
injectTestTone	bool	0	2	rfeCfg_monitorAndSafety_setInjectTestTone()	rfeCfg_monitorAndSafety_getInjectTestTone()
checkBbd_loi	uint8_t	0	31	rfeCfg_monitorAndSafety_setClippingBBD()	rfeCfg_monitorAndSafety_getLoiBBD()
checkBbd_tx	uint8_t	0	7	—	rfeCfg_monitorAndSafety_getTxBBD()
checkBbd_rx	uint8_t	0	7	—	rfeCfg_monitorAndSafety_getRxBBD()
fuSaFaultMask	uint8_t	0	0xFF	—	rfeCfg_monitorAndSafety_getFuSaFaultMask()

3.10.4.1 rxSatCountResetEveryChirpSequence

Reset the Rx saturation error counter after each Chirp Sequence.

3.10.4.2 rxSaturationThreshold

ADC output level that triggers the clipping detector for each RX antenna.

3.10.4.3 rxSaturationCountLimit

The number of ADC clipping events, that triggers an error.

3.10.4.4 adcClippingCountResetEveryChirpSequence

If **enabled**, the clipping counter is reset every chirp sequence.

If **disabled**, read out of the monitors via [rfe_monitorRead\(\)](#) will also reset these counters.

3.10.4.5 frequencyForBist

The center frequency in kHz, to be used for RX BIST.

3.10.4.6 zeroHrRefForRxPhaseDiffLna

Reference signal phase difference of all RX channels against the 1st RX (in degrees, for LNA input. See [injectTestTone](#)).

Remarks

1. The RX BIST is for the fixed combination RX1-RX2, RX1-RX3 and RX1-RX4.
2. The RX BIST will be executed for enabled RX channels for the combination mentioned above.
3. The RX BIST will be skipped if RX1 is not enabled.

3.10.4.7 rxPhaseDiffLnaThresholdTolerance

The allowed difference between the *measured RX to RX phase difference* and the *zero hour data* (in degrees, for LNA input. See [injectTestTone](#)). If the measured value exceeds the threshold for any of the measured combinations (RX1-RX2, RX1-RX3 and RX1-RX4), the RX BIST will be marked as failed and ERROR_N will be asserted.

3.10.4.8 zeroHourReferenceForRxGainDiffLna

The reference RX-RX gain difference which will be compared with measured values during RX BIST (in dB, for LNA input. See [injectTestTone](#)).

3.10.4.9 rxGainDiffLnaThresholdTolerance

The allowed difference between the *measured RX to RX gain difference* and the *zero hour data* (in dB, for LNA input. See [injectTestTone](#)). If the measured value exceeds the threshold for any of the measured combinations (RX1-RX2, RX1-RX3 and RX1-RX4), the RX BIST will be marked as failed and ERROR_N will be asserted.

Remarks

1. The RX BIST is for the fixed combination RX1-RX2, RX1-RX3 and RX1-RX4.
2. The RX BIST will be executed for enabled RX channels for the combination mentioned above.
3. The RX BIST will be skipped if RX1 is not enabled.

3.10.4.10 zeroHrRefForRxPhaseDiffMixer

Same as [zeroHrRefForRxPhaseDiffLna](#) but for **Mixer** input instead of LNA. (See [injectTestTone](#))

3.10.4.11 rxPhaseDiffMixerThresholdTolerance

Same as [rxPhaseDiffLnaThresholdTolerance](#) but for **Mixer** input instead of LNA. (See [injectTestTone](#))

3.10.4.12 zeroHourReferenceForRxGainDiffMixer

Same as [zeroHourReferenceForRxGainDiffLna](#) but for **Mixer** input instead of LNA. (See [injectTestTone](#))

3.10.4.13 rxGainDiffMixerThresholdTolerance

Same as [rxGainDiffLnaThresholdTolerance](#) but for **Mixer** input instead of LNA. (See [injectTestTone](#))

3.10.4.14 injectTestTone

Configures the injection point of the test tone during RX BIST. When **injectTestTone** is set to 0, the test tone is injected twice every RX BIST sequence, once before **LNA** & once before **Mixer**. when **injectTestTone** is set to 1, the test tone is injected every RX BIST sequence only before Mixer. when **injectTestTone** is set to 2, injection point of the test tone repeatedly alternates between **before LNA** and **before Mixer** with each RX BIST sequence.

3.10.4.15 checkBbd_loi

Configures the following settings, bitwise encoded:

- 0001 0000 : **Enables**/Disables the Ball Break detector for LO Interface.
- 0000 1000 : **High**/Normal current sensitivity for the LO-out Ball Break detector.
- 0000 0100 : **Grounded**/Not Grounded Ball Break detector **balun**, for the LO-out.
- 0000 0010 : **High**/Normal current sensitivity for the LO-in Ball Break detector.
- 0000 0001 : **Grounded**/Not Grounded Ball Break detector **balun**, for the LO-in.

3.10.4.16 checkBbd_tx

Configures the following settings, bitwise encoded:

- 0000 0100 : **Enables**/Disables the Ball Break detector for TX antennas.
- 0000 0010 : **High**/Normal current sensitivity for the TX antennas Ball Break detector.
- 0000 0001 : **Grounded**/Not Grounded Ball Break detector **balun**, for the TX antennas Ball.

3.10.4.17 checkBbd_rx

Configures the following settings, bitwise encoded:

- 0000 0100 : **Enables**/Disables the Ball Break detector for RX antennas.
- 0000 0010 : **High**/Normal current sensitivity for the RX antennas Ball Break detector.
- 0000 0001 : **Grounded**/Not Grounded Ball Break detector **balun**, for the RX antennas Ball.

3.10.4.18 fuSaFaultMask

Functional safety faults to be masked/unmasked. Masked faults do not assert the ERROR_N signal.

The enumeration from [rfe_fuSaFault_t](#) must be used as bitwise index to mask or unmask the corresponding fault. To mask, set the desired bit to '1', to unmask, set to '0'.

Remarks

- NXP does not recommend the user to mask the faults, as they will not be detected, which may violate the safety goal.
- [rfe_generic_sw_e](#) - is a generic software error that is raised by any [rfe_error_t](#) triggered, therefore it cannot be masked! It is kept however in this list for visibility.

3.10.5 radarCycle

This section describes the radarCycle section of the RFE Configuration BLOB.

Parameter	Type	Min	Max	Set	Get
radarCycleDuration	uint32↔ _t	400000	20000000	rfeCfg_radarCycle_setRadarCycleDuration()	rfeCfg_radarCycle_getRadarCycleDuration()
chirpSequenceCount	uint8↔ _t	1	8	—	rfeCfg_radarCycle_getChirpSequenceCount()
chirpSequence	uint8↔ _t	0	7	—	rfeCfg_radarCycle_getChirpSequence()
chirpSequenceStartTimeOffset	uint32↔ _t	0	20000000	—	rfeCfg_radarCycle_getChirpSequenceStartTimeOffset()
bistInterval	uint8↔ _t	0	3	—	rfeCfg_radarCycle_getBistInterval()

3.10.5.1 radarCycleDuration

Radar cycle duration in 25 [ns] resolution.

3.10.5.2 chirpSequenceCount

Number of chirp sequences configured for the radar cycle.

3.10.5.3 chirpSequence

Contains the chirp sequence configurations, in the order they were configured in the XML at generation time. Based on [chirpSequenceCount](#), how many of the [rfeCfg_param_radarCycle_chirpSequence_0_e](#) --> [rfeCfg_param_radarCycle_chirpSequence_7_e](#) contain a configuration that is intended to be used in the radar cycle.

3.10.5.4 chirpSequenceStartTimeOffset

The chirp sequence time offset in 25 [ns] resolution with respect to the start time of the radar cycle. The offset is defined as the time difference between the start of the dwell time of the first chirp and the start time of the radar cycle.

If using [powerMode](#) = 1, then an additional DC_Power_on_Delay value in the order of hundreds of microseconds will be added before the dwell time of the first chirp. The start time of chirp sequence 0 is fixed at design time. For the first sequence in the radar cycle, the time checking is disabled because the init chirp sequence and recalibration may take longer, otherwise the timing will not be optimal for the radar system.

3.10.5.5 bistInterval

The interval to perform RFE built-in-self-tests. The selections are:

- 0 : No BIST.
- 1 : Every radar cycle. (recommended)
- 2 : Once in two radar cycles.
- 3 : Once in four radar cycles. The rationale is that RX BIST must be executed once per Fault Handling Time Interval (FHTI). FHTI is associated with 1 radar cycle in radar safety concept. Doing it less often than 1 radar cycle is not recommended. For more information on RX BIST and FHTI relation please refer to Functional Safety Manual.

3.10.6 chirpSequenceConfig

This section describes the RFE_CHIRP_SEQUENCE_CONFIGS_MAX chirpSequenceConfig sections of the RFE Configuration BLOB.

Parameter	Type	Min	Max	Set	Get
chirpCount	uint16_t	1	4096	rfeCfg_chirpSequence_setChirpCount()	rfeCfg_chirpSequence_getChirpCount()
chirpProfileSequence	uint8_t	0	7	rfeCfg_chirpSequence_setChirpProfileSequence()	rfeCfg_chirpSequence_getChirpProfileSequence()
chirpProfileSequenceLength	uint8_t	1	8	rfeCfg_chirpSequence_setChirpProfileSequenceLength()	rfeCfg_chirpSequence_getChirpProfileSequenceLength()
chirpProfileRepeatCount	uint8_t	1	127	rfeCfg_chirpSequence_setChirpProfileRepeatCount()	rfeCfg_chirpSequence_getChirpProfileRepeatCount()
dynamicUpdatesEnabled	bool	0	1	rfeCfg_chirpSequence_setDynamicUpdatesEnabled()	rfeCfg_chirpSequence_getDynamicUpdatesEnabled()
dynamicTableIndex	uint8_t	0	7	rfeCfg_chirpSequence_setDynamicTableIndex()	rfeCfg_chirpSequence_getDynamicTableIndex()
txPhaseRotation_ddma	uint8_t	0	3	rfeCfg_sequence_setPhaseRotationDdmaMode()	rfeCfg_sequence_getPhaseRotationDdmaMode()
txPhaseRotation_ddma_tx	uint8_t	0	7	—	rfeCfg_sequence_getPhaseRotationTxUse()
txPhaseRotation_ddma_tx1	uint8_t	0	63	rfeCfg_sequence_setTxDdmaInitPhase()	rfeCfg_sequence_getTxDdmaInitPhase()
txPhaseRotation_ddma_tx1update	uint8_t	0	63	rfeCfg_sequence_setTxDdmaUpdatePhase()	rfeCfg_sequence_getTxDdmaUpdatePhase()
txPhaseRotation_ddma_tx1update	uint8_t	0	2	rfeCfg_sequence_setTxFinalPcGenMode()	rfeCfg_sequence_getFinalPcGenMode()
rxEnable	rfe_rx↔ Select_t	0x0	0xF	rfeCfg_chirpSequence_setRxEnable()	rfeCfg_chirpSequence_getRxEnable()
txCalibration_thresholdHigh (optional)	uint16_t	0	400	—	rfeCfg_sequence_getTxCalibrationThresholdHigh()
txCalibration_thresholdLow (optional)	uint16_t	0	400	—	rfeCfg_sequence_getTxCalibrationThresholdLow()

3.10.6.1 chirpCount

The number of chirps within the chirp sequence.

3.10.6.2 chirpProfileSequence

The list of profiles that are used for the current sequence, and their order. Based on the `chirpProfileSequenceLength`, determines how many profiles should be considered, and how many should be ignored, from the list `rfeCfg_param_chirpSequence_chirpProfileSequence_0_e --> rfeCfg_param_chirpSequence_chirpProfileSequence_7_e`.

3.10.6.3 chirpProfileSequenceLength

The number of chirp profiles, that are configured for the current sequence.

This parameter determines how many `rfeCfg_param_chirpSequence_chirpProfileSequence_0_e --> rfeCfg_param_chirpSequence_7_e` should be used, and which should be ignored.

3.10.6.4 chirpProfileRepeatCount

Configures the number of times a profile will be repeated within the chirp sequence, before switching to the next profile in the list (e.g. using `profileRepeatCount = 3`, and `chirpProfileSequence = "0 1 2"` will result in the following profiles being used: 0, 0, 0, 1, 1, 1, 2, 2, 2. Otherwise, with `profileRepeatCount = 1` -> 0, 1, 2, 0, 1, 2, 0, 1, 2.

- Example 1: Chirp sequence: 0-0-0-0-0-0-..
`chirpProfileSequence = {rfe_chirpProfileIndex_0_e};`
`chirpProfileRepeatCount = 1;`
- Example 2: Chirp sequence: 1-1-1-1-1-1-..
`chirpProfileSequence = {rfe_chirpProfileIndex_1_e};`
`chirpProfileRepeatCount = 1;`

- Example 3: Interleaved chirp sequence 0-0-0-1-1-1-2-2-2..
`chirpProfileSequence = {rfe_chirpProfileIndex_0_e, rfe_chirpProfileIndex_1_e, rfe_chirpProfileIndex_2_e};`
`chirpProfileRepeatCount = 3;`
- Example 4: Interleaved chirp sequence 0-1-2-0-1-2-0-1-2..
`chirpProfileSequence = {rfe_chirpProfileIndex_0_e, rfe_chirpProfileIndex_1_e, rfe_chirpProfileIndex_2_e};`
`chirpProfileRepeatCount = 1;`

3.10.6.5 dynamicUpdatesEnabled

Sets whether the dynamic updates, i.e. parameter updates per chirp, are enabled or disabled.

3.10.6.6 dynamicTableIndex

The index of the dynamic table to be used for the chirp sequence. This table contains the values for each dynamic parameter for each chirp.

3.10.6.7 txPhaseRotation_ddma

Configures the DDMA control mode. Values are:

- 0: disabled
- 1: functional mode
- 2: test mode (phase code step is incremented during chirp @ 40 MHz)
- 3: test no chirp DDMA test mode with non-chirp mode (phase code step is incremented during chirp @ 40 MHz)

3.10.6.8 txPhaseRotation_ddma_tx

- 0000 0001 (TX1): 0 -> Phase code is taken from Profile | 1 -> Phase code is taken from DDMA generator.
- 0000 0010 (TX2): 0 -> Phase code is taken from Profile | 1 -> Phase code is taken from DDMA generator.
- 0000 0100 (TX3): 0 -> Phase code is taken from Profile | 1 -> Phase code is taken from DDMA generator.

3.10.6.9 txPhaseRotation_ddma_tx1_init

Configures the PR DDMA initial phase for TX 1. (in steps of 5.625 degrees).

Remarks

A similar parameter exists for TX2 & TX3.

3.10.6.10 txPhaseRotation_ddma_tx1_update

Configures the PR DDMA phase increment for TX 1. (in steps of 5.625 degrees).

Remarks

A similar parameter exists for TX2 & TX3.

3.10.6.11 txPhaseRotation_ddma_tx1_mode

Configures how the phase code is generated for TX 1. Values are:

- 0: No change to phase code.
- 1: Replace two MSB bits of phase code with the QPSK[1:0] bits
- 2: Add the QPSK[1:0] bits to two MSB bits of initial phase code

Remarks

A similar parameter exists for TX2 & TX3.

3.10.6.12 rxEnable

Selection of receivers that are enabled during the chirp sequence.

- 0000 0001 : RX1 **Enabled**/Disabled
- 0000 0010 : RX2 **Enabled**/Disabled
- 0000 0100 : RX3 **Enabled**/Disabled
- 0000 1000 : RX4 **Enabled**/Disabled

3.10.6.13 txCalibration_thresholdHigh

Configures the threshold that once reached, would cause the TX antenna output power to be decreased. Values coming from the blob are in dB, multiplied by 100 to account for 2 digits. [0.01 .. 4] This setting is only valid when [txCalibration](#) is set to 0.

Remarks

TEF82xx takes values in steps of 0.29dB. Any value within the accepted range can be used but the value that ends up on the front-end will be the closest multiple of 0.29dB, relative to the value that was used when generating the blob.

3.10.6.14 txCalibration_thresholdLow

Configures the threshold that once reached, would cause the TX antenna output power to be increased. Values coming from the blob are in dB, multiplied by 100 to account for 2 digits. [0.01 .. 4] This setting is only valid when [txCalibration](#) is set to 0.

Remarks

TEF82xx takes values in steps of 0.29dB. Any value within the accepted range can be used but the value that ends up on the front-end will be the closest multiple of 0.29dB, relative to the value that was used when generating the blob.

3.10.7 chirpProfile

This section describes the RFE_CHIRP_PROFILES_MAX chirpProfile sections of the RFE Configuration BLOB.

Parameter	Type	Min	Max	Set	Get
effectiveSamplingFrequency	uint8↔ _t	0	3	rfeCfg_chirpProfile_setEffectiveSamplingFrequency()	rfeCfg_chirpProfile_getEffectiveSamplingFrequency()
chirpIntervalTimeTicks	uint32↔ _t	400	4194303	rfeCfg_chirpProfile_setChirpIntervalTimeTicks()	rfeCfg_chirpProfile_getChirpIntervalTimeTicks()
dwellTimeTicks	uint16↔ _t	40	4094	rfeCfg_chirpProfile_setDwellTimeTicks()	rfeCfg_chirpProfile_getDwellTimeTicks()
settleTimeTicks	uint16↔ _t	80	4094	rfeCfg_chirpProfile_setSettleTimeTicks()	rfeCfg_chirpProfile_getSettleTimeTicks()
acquisitionTimeTicks	uint16↔ _t	8	8192	rfeCfg_chirpProfile_setAcquisitionTimeTicks()	rfeCfg_chirpProfile_getAcquisitionTimeTicks()
resetTimeTicks	uint16↔ _t	30	4094	rfeCfg_chirpProfile_setResetTimeTicks()	rfeCfg_chirpProfile_getResetTimeTicks()
centerFrequency	uint32↔ _t	76000000	81000000	rfeCfg_chirpProfile_setCenterFrequency()	rfeCfg_chirpProfile_getCenterFrequency()
effectiveChirpBandwidth	uint32↔ _t	0	4000000	rfeCfg_chirpProfile_setEffectiveChirpBandwidth()	rfeCfg_chirpProfile_getEffectiveChirpBandwidth()
chirpPIIVcoSelect	uint8↔ _t	0	2	rfeCfg_chirpProfile_setChirpPIIVco()	rfeCfg_chirpProfile_getChirpPIIVco()
chirpSlopeDirection	uint8↔ _t	0	1	rfeCfg_chirpProfile_setChirpSlopeDirection()	rfeCfg_chirpProfile_getChirpSlopeDirection()
chirpPIILoopFilterBandwidth	uint8↔ _t	0	30	rfeCfg_chirpProfile_setChirpPIILoopFilterBandwidth()	rfeCfg_chirpProfile_getChirpPIILoopFilterBandwidth()
txTransmissionEnable	uint8↔ _t	0	7	rfeCfg_chirpProfile_setTxTransmissionEnable()	rfeCfg_chirpProfile_getTxTransmissionEnable()
txPower	int16↔ _t	30	150	rfeCfg_chirpProfile_setTxPower()	rfeCfg_chirpProfile_getTxPower()
txPhaseRotation	uint8↔ _t	0	63	rfeCfg_chirpProfile_setTxPhaseRotation()	rfeCfg_chirpProfile_getTxPhaseRotation()
chirpFrequencyDriftSteps	int32↔ _t	0	932068	—	rfeCfg_chirpSequence_getChirpFrequencyDriftSteps()
rxGain	uint8↔ _t	1	7	rfeCfg_chirpProfile_setRxGain()	rfeCfg_chirpProfile_getRxGain()
rxHpfCutOffFrequency	uint8↔ _t	0	6	rfeCfg_chirpProfile_setRxHpfCutOffFrequency()	rfeCfg_chirpProfile_getRxHpfCutOffFrequency()
rxLpfCutOffFrequency	uint8↔ _t	0	4	rfeCfg_chirpProfile_setRxLpfCutOffFrequency()	rfeCfg_chirpProfile_getRxLpfCutOffFrequency()
virtualChannel	uint8↔ _t	0	2	rfeCfg_chirpProfile_setVirtualChannel()	rfeCfg_chirpProfile_getVirtualChannel()

3.10.7.1 [effectiveSamplingFrequency](#)

The effective sampling frequency of the chirp. Here 'effective' indicates that this is the sampling frequency after decimation, i.e. of the output samples.

3.10.7.2 [chirpIntervalTimeTicks](#)

The chirp interval time in 25 [ns] resolution. It is the time between the start of subsequent chirps and this parameter can be used to introduce additional idle time between chirps. The value for this parameter must be equal or larger than the sum of its components: [dwellTimeTicks](#), [settleTimeTicks](#), [acquisitionTimeTicks](#), [jumpbackTimeTicks](#), and [resetTimeTicks](#).

3.10.7.3 dwellTimeTicks

The chirp dwell time in 25 [ns] resolution.

3.10.7.4 settleTimeTicks

The chirp settle time in 25 [ns] resolution.

3.10.7.5 acquisitionTimeTicks

The chirp acquisition time in 25 [ns] resolution.

3.10.7.6 resetTimeTicks

The chirp reset time in 25 [ns] resolution.

3.10.7.7 centerFrequency

The center frequency of the chirp in [kHz].

3.10.7.8 effectiveChirpBandwidth

The effective bandwidth in [kHz]. This is the bandwidth used during the chirp acquisition. A bandwidth of 0 results in a 'chirp' with constant frequency. The actual effective chirp bandwidth is rounded to a multiple of [acquisitionTimeTicks](#) * RFE_CHIRP_FREQUENCY_DRIFT_STEPS_HZ.

3.10.7.9 chirpPIIVcoSelect

Selection of the VCO that the chirp PLL uses to generate the chirp:

- 0: 1GHz VCO
- 1: 2GHz VCO
- 2: 4GHz VCO

3.10.7.10 chirpSlopeDirection

Selection of the direction of the chirp slope: rising or falling.

3.10.7.11 chirpPllLoopFilterBandwidth

Selects the chirp PLL loop filter bandwidth.

See [rfe_chirpPllLoopFilterBandwidth_t](#) for all the values.

Attention

Values that can be selected, depend on the VCO parameter from above. In order to achieve the highest recommended chirp steepness per VCO, the values below are suggested, which also ensure the lowest phase noise:

- For 1GHz VCO -> 200kHz | 250kHz | 300kHz
- For 2GHz VCO -> 250kHz | 300kHz
- For 4GHz VCO -> 300kHz | 350kHz

3.10.7.12 txTransmissionEnable

Selection of transmitters that transmit the chirp.

- 0000 0001 : TX1 **Enabled**/Disabled
- 0000 0010 : TX2 **Enabled**/Disabled
- 0000 0100 : TX3 **Enabled**/Disabled

Warning

If a Sequence uses multiple profiles, that has different TXs enabled, then [powerMode](#) must be configured to **0** ("lower power") otherwise a lock-step error would be triggered from ISM.

3.10.7.13 txPower

Output power of the transmitters in the steps of 0.1 [dBm]. The value coming from the blob is multiplied by 10, to account for the 1 decimal.

3.10.7.14 txPhaseRotation

Phase rotation for each transmitter (in steps of 5.625 degrees).

3.10.7.15 chirpFrequencyDriftSteps

The frequency of the chirp (i.e., start & end frequency of the chirp frequency sweep) increments (positive) or decrements (negative) by $x * 42.9153$ Hz each chirp, after chirp 1. A value of 0 will disable the drift. Values other than 0 are not supported, when the chirp sequence consists of chirps with different Center Frequencies (i.e. center-frequency-kHz)

Remarks

When [chirpFrequencyDriftSteps](#) is enabled (i.e. value != 0), if using different [centerFrequency](#) in the same sequence, the first [centerFrequency](#) value will be used, and the rest will be ignored. Front-End will issue no warning for such case.

3.10.7.16 rxGain

The gain of the receivers. Values are:

- 1: 27dB
- 2: 30dB
- 3: 33dB
- 4: 36dB
- 5: 39dB
- 6: 42dB
- 7: 45dB

3.10.7.17 rxHpfCutOffFrequency

The cut-off frequency of receivers' high-pass filters:

- 0: 100 kHz
- 1: 200 kHz
- 2: 300 kHz
- 3: 400 kHz
- 4: 800 kHz
- 5: 1600 kHz
- 6: 3200 kHz

3.10.7.18 rxLpfCutOffFrequency

The cut-off frequency of receivers' low-pass filters.

- 0: 15 MHz
- 1: 20 MHz
- 2: 25 MHz
- 3: 40 MHz

3.10.7.19 virtualChannel

The virtual channel applied on data for the chirps using this profile.

3.10.8 Macro Definition Documentation

3.10.8.1 RFE_CFG_MAX_DATA_LENGTH

```
#define RFE_CFG_MAX_DATA_LENGTH (RFE_CMD_IF_MAXDATLEN - 2UL)
```

The size (in bytes) of the metadata section of the configuration data memory.

3.10.8.2 RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_0

```
#define RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_0 ((uint16_t) (RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_0  
+ RFE_CFG_SIZE_SECTION_CHIRP_SEQUENCE))
```

The offset (in bytes) of the chirp profile 0 section of the configuration data memory.

3.10.8.3 RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_1

```
#define RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_1 ((uint16_t) (RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_1  
+ RFE_CFG_SIZE_SECTION_CHIRP_SEQUENCE))
```

The offset (in bytes) of the chirp profile 1 section of the configuration data memory.

3.10.8.4 RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_2

```
#define RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_2 ((uint16_t) (RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_2  
+ RFE_CFG_SIZE_SECTION_CHIRP_SEQUENCE))
```

The offset (in bytes) of the chirp profile 2 section of the configuration data memory.

3.10.8.5 RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_3

```
#define RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_3 ((uint16_t) (RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_3  
+ RFE_CFG_SIZE_SECTION_CHIRP_SEQUENCE))
```

The offset (in bytes) of the chirp profile 3 section of the configuration data memory.

3.10.8.6 RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_4

```
#define RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_4 ((uint16_t) (RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_4  
+ RFE_CFG_SIZE_SECTION_CHIRP_SEQUENCE))
```

The offset (in bytes) of the chirp profile 4 section of the configuration data memory.

3.10.8.7 RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_5

```
#define RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_5 ((uint16_t) (RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_5  
+ RFE_CFG_SIZE_SECTION_CHIRP_SEQUENCE))
```

The offset (in bytes) of the chirp profile 5 section of the configuration data memory.

3.10.8.8 RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_6

```
#define RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_6 ((uint16_t) (RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_6  
+ RFE_CFG_SIZE_SECTION_CHIRP_SEQUENCE))
```

The offset (in bytes) of the chirp profile 6 section of the configuration data memory.

3.10.8.9 RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_7

```
#define RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_7 ((uint16_t) (RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_7  
+ RFE_CFG_SIZE_SECTION_CHIRP_SEQUENCE))
```

The offset (in bytes) of the chirp profile 7 section of the configuration data memory.

3.10.8.10 RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_0

```
#define RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_0 ((uint16_t) (RFE_CFG_OFFSET_SECTION_RADAR_CYCLE  
+ RFE_CFG_SIZE_SECTION_RADAR_CYCLE))
```

The offset (in bytes) of the chirp sequence 0 section of the configuration data memory.

3.10.8.11 RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_1

```
#define RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_1 ((uint16_t) (RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_0  
+ RFE_CFG_SIZE_SECTION_CHIRP_PROFILE))
```

The offset (in bytes) of the chirp sequence 1 section of the configuration data memory.

3.10.8.12 RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_2

```
#define RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_2 ((uint16_t) (RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_1  
+ RFE_CFG_SIZE_SECTION_CHIRP_PROFILE))
```

The offset (in bytes) of the chirp sequence 2 section of the configuration data memory.

3.10.8.13 RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_3

```
#define RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_3 ((uint16_t) (RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_2  
+ RFE_CFG_SIZE_SECTION_CHIRP_PROFILE))
```

The offset (in bytes) of the chirp sequence 3 section of the configuration data memory.

3.10.8.14 RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_4

```
#define RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_4 ((uint16_t) (RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_3  
+ RFE_CFG_SIZE_SECTION_CHIRP_PROFILE))
```

The offset (in bytes) of the chirp sequence 4 section of the configuration data memory.

3.10.8.15 RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_5

```
#define RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_5 ((uint16_t) (RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_4  
+ RFE_CFG_SIZE_SECTION_CHIRP_PROFILE))
```

The offset (in bytes) of the chirp sequence 5 section of the configuration data memory.

3.10.8.16 RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_6

```
#define RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_6 ((uint16_t) (RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_5  
+ RFE_CFG_SIZE_SECTION_CHIRP_PROFILE))
```

The offset (in bytes) of the chirp sequence 6 section of the configuration data memory.

3.10.8.17 RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_7

```
#define RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_7 ((uint16_t) (RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_6  
+ RFE_CFG_SIZE_SECTION_CHIRP_PROFILE))
```

The offset (in bytes) of the chirp sequence 7 section of the configuration data memory.

3.10.8.18 RFE_CFG_OFFSET_SECTION_CRC

```
#define RFE_CFG_OFFSET_SECTION_CRC ((uint16_t) (RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_7 + RFE_CFG_SIZE_SECTION_CHIRP_PROFILE))
```

The offset (in bytes) of the chirp profile 7 section of the configuration data memory.

3.10.8.19 RFE_CFG_OFFSET_SECTION_GENERAL

```
#define RFE_CFG_OFFSET_SECTION_GENERAL ((uint16_t) (RFE_CFG_OFFSET_SECTION_METADATA + RFE_CFG_SIZE_SECTION_METADATA))
```

The offset (in bytes) of the general section of the configuration data memory.

3.10.8.20 RFE_CFG_OFFSET_SECTION_METADATA

```
#define RFE_CFG_OFFSET_SECTION_METADATA ((uint16_t) (0x0000UL + 0x0000UL))
```

The offset (in bytes) of the metadata section of the configuration data memory.

3.10.8.21 RFE_CFG_OFFSET_SECTION_MONITOR_AND_SAFETY

```
#define RFE_CFG_OFFSET_SECTION_MONITOR_AND_SAFETY ((uint16_t) (RFE_CFG_OFFSET_SECTION_GENERAL +  
RFE_CFG_SIZE_SECTION_GENERAL))
```

The offset (in bytes) of the monitor and safety section of the configuration data memory.

3.10.8.22 RFE_CFG_OFFSET_SECTION_RADAR_CYCLE

```
#define RFE_CFG_OFFSET_SECTION_RADAR_CYCLE ((uint16_t)(RFE_CFG_OFFSET_SECTION_MONITOR_AND_SAFETY  
+ RFE_CFG_SIZE_SECTION_MONITOR_AND_SAFETY))
```

The offset (in bytes) of the radar cycle section of the configuration data memory.

3.10.8.23 RFE_CFG_PARAM_OFFSET_MSK

```
#define RFE_CFG_PARAM_OFFSET_MSK ((uint16_t)(0x07ffUL))
```

The mask for offset part of a configuration parameter.

3.10.8.24 RFE_CFG_PARAM_OFFSET_SHF

```
#define RFE_CFG_PARAM_OFFSET_SHF ((uint8_t)(0UL))
```

The shift for offset part of a configuration parameter.

3.10.8.25 RFE_CFG_PARAM_SECTION_CHIRP_PROFILE

```
#define RFE_CFG_PARAM_SECTION_CHIRP_PROFILE ((uint16_t)((1UL<<15) | ((0UL<<14) | (1UL<<13))))
```

The encoding of the chirp profile section part into a configuration parameter.

3.10.8.26 RFE_CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG

```
#define RFE_CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG ((uint16_t)((1UL<<15) | ((0UL<<14) | (0↵  
UL<<13))))
```

The encoding of the chirp sequence section part into a configuration parameter.

3.10.8.27 RFE_CFG_PARAM_SECTION_CRC

```
#define RFE_CFG_PARAM_SECTION_CRC ((uint16_t)((1UL<<15) | ((1UL<<14) | (1UL<<13))))
```

The encoding of the chirp profile section part into a configuration parameter.

3.10.8.28 RFE_CFG_PARAM_SECTION_GENERAL

```
#define RFE_CFG_PARAM_SECTION_GENERAL ((uint16_t)((0UL<<15) | ((0UL<<14) | (1UL<<13))))
```

The encoding of the general section part into a configuration parameter.

3.10.8.29 RFE_CFG_PARAM_SECTION_METADATA

```
#define RFE_CFG_PARAM_SECTION_METADATA ((uint16_t)((0UL<<15) | ((0UL<<14) | (0UL<<13))))
```

The encoding of the metadata section part into a configuration parameter.

3.10.8.30 RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY

```
#define RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY ((uint16_t)((0UL<<15) | ((1UL<<14) | (0UL<<13))))
```

The encoding of the monitor and safety section part into a configuration parameter.

3.10.8.31 RFE_CFG_PARAM_SECTION_MSK

```
#define RFE_CFG_PARAM_SECTION_MSK ((uint16_t)((1UL<<15) | ((1UL<<14) | (1UL<<13))))
```

The mask for section part of a configuration parameter.

3.10.8.32 RFE_CFG_PARAM_SECTION_RADAR_CYCLE

```
#define RFE_CFG_PARAM_SECTION_RADAR_CYCLE ((uint16_t)((0UL<<15) | ((1UL<<14) | (1UL<<13))))
```

The encoding of the radar cycle section part into a configuration parameter.

3.10.8.33 RFE_CFG_PARAM_SIZE_16_BIT

```
#define RFE_CFG_PARAM_SIZE_16_BIT ((uint16_t)((0UL<<12) | (1UL<<11)))
```

The encoding of the size part into a configuration parameter for a 16 bit value.

3.10.8.34 RFE_CFG_PARAM_SIZE_32_BIT

```
#define RFE_CFG_PARAM_SIZE_32_BIT ((uint16_t)((1UL<<12) | (0UL<<11)))
```

The encoding of the size part into a configuration parameter for a 32 bit value.

3.10.8.35 RFE_CFG_PARAM_SIZE_32_BIT_FLOAT

```
#define RFE_CFG_PARAM_SIZE_32_BIT_FLOAT ((uint16_t)((1UL<<12) | (1UL<<11)))
```

The encoding of the size part into a configuration parameter for a 32 bit float value.

3.10.8.36 RFE_CFG_PARAM_SIZE_8_BIT

```
#define RFE_CFG_PARAM_SIZE_8_BIT ((uint16_t)((0UL<<12) | (0UL<<11)))
```

The encoding of the size part into a configuration parameter for an 8 bit value.

3.10.8.37 RFE_CFG_PARAM_SIZE_MSK

```
#define RFE_CFG_PARAM_SIZE_MSK ((uint16_t)((1UL<<12) | (1UL<<11)))
```

The mask for size part of a configuration parameter.

3.10.8.38 RFE_CFG_SIZE_SECTION_CHIRP_PROFILE

```
#define RFE_CFG_SIZE_SECTION_CHIRP_PROFILE (41UL)
```

The size (in bytes) of a chirp profile section of the configuration data memory.

3.10.8.39 RFE_CFG_SIZE_SECTION_CHIRP_SEQUENCE

```
#define RFE_CFG_SIZE_SECTION_CHIRP_SEQUENCE (31UL)
```

The size (in bytes) of a chirp sequence section of the configuration data memory.

3.10.8.40 RFE_CFG_SIZE_SECTION_CRC

```
#define RFE_CFG_SIZE_SECTION_CRC (4UL)
```

The size (in bytes) of the CRC section of the configuration data memory.

3.10.8.41 RFE_CFG_SIZE_SECTION_GENERAL

```
#define RFE_CFG_SIZE_SECTION_GENERAL (10UL)
```

The size (in bytes) of the general section of the configuration data memory.

3.10.8.42 RFE_CFG_SIZE_SECTION_METADATA

```
#define RFE_CFG_SIZE_SECTION_METADATA (6UL)
```

The size (in bytes) of the metadata section of the configuration data memory.

3.10.8.43 RFE_CFG_SIZE_SECTION_MONITOR_AND_SAFETY

```
#define RFE_CFG_SIZE_SECTION_MONITOR_AND_SAFETY (179UL)
```

The size (in bytes) of the monitor and safety section of the configuration data memory.

3.10.8.44 RFE_CFG_SIZE_SECTION_RADAR_CYCLE

```
#define RFE_CFG_SIZE_SECTION_RADAR_CYCLE (55UL)
```

The size (in bytes) of the radar cycle section of the configuration data memory.

3.10.8.45 RFE_CFG_SIZE_TOTAL

```
#define RFE_CFG_SIZE_TOTAL (RFE_CFG_OFFSET_SECTION_CRC + RFE_CFG_SIZE_SECTION_CRC)
```

The total size (in bytes) of the configuration data memory.

3.10.8.46 RFE_CHIRP_PROFILE_SEQUENCE_LENGTH_MAX

```
#define RFE_CHIRP_PROFILE_SEQUENCE_LENGTH_MAX (8UL)
```

This macro defines the maximum chirp profile sequence length.

3.10.8.47 RFE_CHIRP_PROFILES_MAX

```
#define RFE_CHIRP_PROFILES_MAX (8UL)
```

This macro defines the maximum number of configurable profiles. [rfe_chirpProfileIndex_t](#) defines the chirp profile indices.

3.10.8.48 RFE_CHIRP_SEQUENCE_CONFIGS_MAX

```
#define RFE_CHIRP_SEQUENCE_CONFIGS_MAX (8UL)
```

This macro defines the maximum number of chirp sequence configs. [rfe_chirpSequenceIndex_t](#) defines the chirp sequence config indices.

3.10.8.49 RFE_CHIRP_SEQUENCES_PER_RADAR_CYCLE_MAX

```
#define RFE_CHIRP_SEQUENCES_PER_RADAR_CYCLE_MAX (8UL)
```

This macro defines the maximum supported chirp sequences per radar cycle.

3.10.8.50 RFE_FUSA_FAULT_MASK_PARAM_COUNT

```
#define RFE_FUSA_FAULT_MASK_PARAM_COUNT (12UL)
```

This macro defines the number of bytes used in RFE Configuration BLOB for masking or unmasking the FuSa Faults of total 12 bytes.

3.10.8.51 RFE_SW_CFG_MNGR_CHIRP_PROFILE_PARAMS

```
#define RFE_SW_CFG_MNGR_CHIRP_PROFILE_PARAMS ( (uint16_t) 22UL )
```

This macro defines the size of the chirp profile parameters

3.10.8.52 RFE_SW_CFG_MNGR_CHIRP_SEQUENCE_PARAMS

```
#define RFE_SW_CFG_MNGR_CHIRP_SEQUENCE_PARAMS ( (uint16_t) 21UL )
```

This macro defines the size of the chirp sequence parameters

3.10.8.53 RFE_SW_CFG_MNGR_GENRAL_PARAMS

```
#define RFE_SW_CFG_MNGR_GENRAL_PARAMS ( (uint16_t) 19UL )
```

This macro defines the total number of configuration parameters. This macro defines the size of the general parameters

3.10.8.54 RFE_SW_CFG_MNGR_MONITOR_AND_SAFETY_PARAMS

```
#define RFE_SW_CFG_MNGR_MONITOR_AND_SAFETY_PARAMS ( (uint16_t) 83UL )
```

This macro defines the size of the monitor and safety parameters

3.10.8.55 RFE_SW_CFG_MNGR_NO_OF_SECTION

```
#define RFE_SW_CFG_MNGR_NO_OF_SECTION ( (uint16_t) 6UL )
```

This macro defines the number of sections

3.10.8.56 RFE_SW_CFG_MNGR_NON_UPDATABLE_PARAMS

```
#define RFE_SW_CFG_MNGR_NON_UPDATABLE_PARAMS ( (uint16_t) 10UL )
```

This macro defines the size of the non updatable parameters

3.10.8.57 RFE_SW_CFG_MNGR_RADAR_CYCLE_PARAMS

```
#define RFE_SW_CFG_MNGR_RADAR_CYCLE_PARAMS ( (uint16_t) 28UL )
```

This macro defines the size of the radar cycle parameters

3.10.8.58 RFE_SW_CFG_MNGR_UPDATABLE_PARAMS

```
#define RFE_SW_CFG_MNGR_UPDATABLE_PARAMS ( (uint16_t) 156UL )
```

This macro defines the number of updatable parameters

3.10.8.59 RFE_TX_COUNT

```
#define RFE_TX_COUNT (3UL)
```

This macro defines the number of transmitters. [rfe_txIndex_t](#) defines the transmitter indices.

3.10.9 Typedef Documentation

3.10.9.1 rfe_phaseRotation_t

```
typedef uint8_t rfe_phaseRotation_t
```

This type defines the phase rotation as Q7 for 360 degrees, 45 deg = 0x10.

3.10.10 Enumeration Type Documentation

3.10.10.1 rfe_chirpPllLoopFilterBandwidth_t

```
enum rfe_chirpPllLoopFilterBandwidth_t
```

This enumerated type defines the chirp PLL loop filter bandwidths.

Enumerator

rfe_chirpPllLoopFilterBandwidth_200kHz_e	
rfe_chirpPllLoopFilterBandwidth_250kHz_e	
rfe_chirpPllLoopFilterBandwidth_300kHz_e	
rfe_chirpPllLoopFilterBandwidth_350kHz_e	
rfe_chirpPllLoopFilterBandwidth_400kHz_e	
rfe_chirpPllLoopFilterBandwidth_450kHz_e	
rfe_chirpPllLoopFilterBandwidth_500kHz_e	
rfe_chirpPllLoopFilterBandwidth_550kHz_e	
rfe_chirpPllLoopFilterBandwidth_600kHz_e	
rfe_chirpPllLoopFilterBandwidth_650kHz_e	
rfe_chirpPllLoopFilterBandwidth_700kHz_e	
rfe_chirpPllLoopFilterBandwidth_750kHz_e	

Enumerator

rfe_chirpPllLoopFilterBandwidth_800kHz_e	
rfe_chirpPllLoopFilterBandwidth_850kHz_e	
rfe_chirpPllLoopFilterBandwidth_900kHz_e	
rfe_chirpPllLoopFilterBandwidth_950kHz_e	
rfe_chirpPllLoopFilterBandwidth_1000kHz↔ _e	
rfe_chirpPllLoopFilterBandwidth_1050kHz↔ _e	
rfe_chirpPllLoopFilterBandwidth_1100kHz↔ _e	
rfe_chirpPllLoopFilterBandwidth_1150kHz↔ _e	
rfe_chirpPllLoopFilterBandwidth_1200kHz↔ _e	
rfe_chirpPllLoopFilterBandwidth_1250kHz↔ _e	
rfe_chirpPllLoopFilterBandwidth_1300kHz↔ _e	
rfe_chirpPllLoopFilterBandwidth_1350kHz↔ _e	
rfe_chirpPllLoopFilterBandwidth_1400kHz↔ _e	
rfe_chirpPllLoopFilterBandwidth_1450kHz↔ _e	
rfe_chirpPllLoopFilterBandwidth_1500kHz↔ _e	
rfe_chirpPllLoopFilterBandwidth_1550kHz↔ _e	
rfe_chirpPllLoopFilterBandwidth_1600kHz↔ _e	
rfe_chirpPllLoopFilterBandwidth_1650kHz↔ _e	
rfe_chirpPllLoopFilterBandwidth_1700kHz↔ _e	
rfe_chirpPllLoopFilterBandwidth_invalid_e	

3.10.10.2 rfe_effectiveSamplingFrequency_t

```
enum rfe_effectiveSamplingFrequency_t
```

This enumerated type defines the effective sampling frequency of the chirp. Effective means the sampling frequency after decimation, i.e. of the output samples.

Enumerator

rfe_effectiveSamplingFrequency_40MHz↔ _e	
rfe_effectiveSamplingFrequency_20MHz↔ _e	

Enumerator

rfe_effectiveSamplingFrequency_10MHz↔ _e	
rfe_effectiveSamplingFrequency_5MHz_e	
rfe_effectiveSamplingFrequency_invalid_e	

3.10.10.3 rfe_txTransmissionReferenceTime_t

```
enum rfe_txTransmissionReferenceTime_t
```

This enumerated type defines the time reference for the tx transmission enable and setting of the phase rotation.

Enumerator

rfe_txTransmissionReferenceTime_startOfDwellTime↔ _e	
rfe_txTransmissionReferenceTime_startOfSettleTime↔ _e	

3.11 RFE Abstract 2.0 BLOB - Low Level Interface

RFE Abstract 2.0 low-level interfaces. These interfaces are sets of get()/set() function pairs for retrieving or updating data to/from the in-memory blob, as well as some helper functions.

Data Structures

- struct [rfeCfg_rxBistReferenceData_t](#)
Struct to hold value of RX Bist reference data.

Functions

- void [rfeCfg_chirpSequence_setChirpCount](#) (uint8_t *pConfig, [rfe_chirpSequenceIndex_t](#) chirpSequence↔ ConfigIndex, uint16_t chirpCount, [rfe_error_t](#) *rfe___error___pointer)
Sets a different [chirpCount](#) for the selected sequence.
- uint16_t [rfeCfg_chirpSequence_getChirpCount](#) (const uint8_t *pConfig, [rfe_chirpSequenceIndex_t](#) chirp↔ SequenceConfigIndex, [rfe_error_t](#) *rfe___error___pointer)
Gets the current [chirpCount](#) for the selected sequence.
- void [rfeCfg_chirpSequence_setChirpProfileSequenceLength](#) (uint8_t *pConfig, [rfe_chirpSequenceIndex_t](#) chirpSequenceConfigIndex, uint8_t chirpProfileSequenceLength, [rfe_error_t](#) *rfe___error___pointer)
Sets the number of profiles used for the selected sequence (see [chirpProfileSequenceLength](#)). Useful for adding or removing profiles that will be used in the selected sequence.
- uint8_t [rfeCfg_chirpSequence_getChirpProfileSequenceLength](#) (const uint8_t *pConfig, [rfe_chirpSequenceIndex_t](#) chirpSequenceConfigIndex, [rfe_error_t](#) *rfe___error___pointer)

Gets the number of profiles used for the selected sequence (see [chirpProfileSequenceLength](#)).

- void [rfeCfg_chirpSequence_setChirpProfileRepeatCount](#) (uint8_t *pConfig, [rfe_chirpSequenceIndex_t](#) chirpSequenceConfigIndex, uint8_t chirpProfileRepeatCount, [rfe_error_t](#) *rfe___error___pointer)

Set the [chirpProfileRepeatCount](#) for the selected sequence.

- uint8_t [rfeCfg_chirpSequence_getChirpProfileRepeatCount](#) (const uint8_t *pConfig, [rfe_chirpSequenceIndex_t](#) chirpSequenceConfigIndex, [rfe_error_t](#) *rfe___error___pointer)

Get the [chirpProfileRepeatCount](#) for the selected sequence.

- void [rfeCfg_chirpSequence_setDynamicUpdatesEnabled](#) (uint8_t *pConfig, [rfe_chirpSequenceIndex_t](#) chirpSequenceConfigIndex, bool dynamicUpdatesEnabled, [rfe_error_t](#) *rfe___error___pointer)

Sets whether the dynamic updates are enabled or disabled for the selected sequence. (see [dynamicUpdatesEnabled](#))

- bool [rfeCfg_chirpSequence_getDynamicUpdatesEnabled](#) (const uint8_t *pConfig, [rfe_chirpSequenceIndex_t](#) chirpSequenceConfigIndex, [rfe_error_t](#) *rfe___error___pointer)

Get the existing setting for [dynamicUpdatesEnabled](#).

- void [rfeCfg_chirpSequence_setDynamicTableIndex](#) (uint8_t *pConfig, [rfe_chirpSequenceIndex_t](#) chirpSequenceConfigIndex, uint8_t dynamicTableIndex, [rfe_error_t](#) *rfe___error___pointer)

Set the [dynamicTableIndex](#) for the selected sequence.

- uint8_t [rfeCfg_chirpSequence_getDynamicTableIndex](#) (const uint8_t *pConfig, [rfe_chirpSequenceIndex_t](#) chirpSequenceConfigIndex, [rfe_error_t](#) *rfe___error___pointer)

Get the [dynamicTableIndex](#) for the selected sequence.

- void [rfeCfg_chirpSequence_setRxEnable](#) (uint8_t *pConfig, [rfe_chirpSequenceIndex_t](#) chirpSequenceConfigIndex, [rfe_rxSelect_t](#) rxEnable, [rfe_error_t](#) *rfe___error___pointer)

Sets which RX antennas to be enabled/disabled for the selected sequence (see [rxEnable](#))

- uint8_t [rfeCfg_chirpSequence_getRxEnable](#) (const uint8_t *pConfig, [rfe_chirpSequenceIndex_t](#) chirpSequenceConfigIndex, [rfe_error_t](#) *rfe___error___pointer)

Gets which RX antennas are enabled/disabled for the selected sequence (see [rxEnable](#))

- void [rfeCfg_chirpSequence_setChirpProfileSequence](#) (uint8_t *pConfig, [rfe_chirpSequenceIndex_t](#) chirpSequenceConfigIndex, uint8_t chirpProfileSequenceIndex, [rfe_chirpProfileIndex_t](#) chirpProfile, [rfe_error_t](#) *rfe___error___pointer)

Sets which profiles to be used in the selected sequence.

- uint8_t [rfeCfg_chirpSequence_getChirpProfileSequence](#) (const uint8_t *pConfig, [rfe_chirpSequenceIndex_t](#) chirpSequenceConfigIndex, uint8_t chirpProfileSequenceIndex, [rfe_error_t](#) *rfe___error___pointer)

Gets which profiles are used in the selected sequence.

- void [rfeCfg_chirpProfile_setEffectiveSamplingFrequency](#) (uint8_t *pConfig, [rfe_chirpProfileIndex_t](#) chirpProfileIndex, [rfe_effectiveSamplingFrequency_t](#) effectiveSamplingFrequency, [rfe_error_t](#) *rfe___error___pointer)

Set a new value for [effectiveSamplingFrequency](#) in the selected profile.

- uint8_t [rfeCfg_chirpProfile_getEffectiveSamplingFrequency](#) (const uint8_t *pConfig, [rfe_chirpProfileIndex_t](#) chirpProfileIndex, [rfe_error_t](#) *rfe___error___pointer)

Get the current [effectiveSamplingFrequency](#) value from the selected profile.

- void [rfeCfg_chirpProfile_setChirpIntervalTimeTicks](#) (uint8_t *pConfig, [rfe_chirpProfileIndex_t](#) chirpProfileIndex, uint32_t chirpIntervalTimeTicks, [rfe_error_t](#) *rfe___error___pointer)

Set a new value for [chirpIntervalTimeTicks](#) in the selected profile.

- uint32_t [rfeCfg_chirpProfile_getChirpIntervalTimeTicks](#) (const uint8_t *pConfig, [rfe_chirpProfileIndex_t](#) chirpProfileIndex, [rfe_error_t](#) *rfe___error___pointer)

Get the current [chirpIntervalTimeTicks](#) from the selected profile.

- void [rfeCfg_chirpProfile_setDwellTimeTicks](#) (uint8_t *pConfig, [rfe_chirpProfileIndex_t](#) chirpProfileIndex, uint16_t dwellTimeTicks, [rfe_error_t](#) *rfe___error___pointer)

Set a new value for [dwellTimeTicks](#) in the selected profile.

- uint16_t [rfeCfg_chirpProfile_getDwellTimeTicks](#) (const uint8_t *pConfig, [rfe_chirpProfileIndex_t](#) chirpProfileIndex, [rfe_error_t](#) *rfe___error___pointer)

Get the current [dwellTimeTicks](#) from the selected profile.

- void [rfeCfg_chirpProfile_setSettleTimeTicks](#) (uint8_t *pConfig, [rfe_chirpProfileIndex_t](#) chirpProfileIndex, uint16_t settleTimeTicks, [rfe_error_t](#) *rfe___error___pointer)

Set a new value for *settleTimeTicks* in the selected profile.

- `uint16_t rfeCfg_chirpProfile_getSettleTimeTicks (const uint8_t *pConfig, rfe_chirpProfileIndex_t chirpProfileIndex, rfe_error_t *rfe___error___pointer)`

Get the current *settleTimeTicks* from the selected profile.

- `void rfeCfg_chirpProfile_setAcquisitionTimeTicks (uint8_t *pConfig, rfe_chirpProfileIndex_t chirpProfileIndex, uint16_t acquisitionTimeTicks, rfe_error_t *rfe___error___pointer)`

Set a new value for *acquisitionTimeTicks* in the selected profile.

- `uint16_t rfeCfg_chirpProfile_getAcquisitionTimeTicks (const uint8_t *pConfig, rfe_chirpProfileIndex_t chirpProfileIndex, rfe_error_t *rfe___error___pointer)`

Get the current *acquisitionTimeTicks* from the selected profile.

- `void rfeCfg_chirpProfile_setResetTimeTicks (uint8_t *pConfig, rfe_chirpProfileIndex_t chirpProfileIndex, uint16_t resetTimeTicks, rfe_error_t *rfe___error___pointer)`

Set a new value for *resetTimeTicks* in the selected profile.

- `uint16_t rfeCfg_chirpProfile_getResetTimeTicks (const uint8_t *pConfig, rfe_chirpProfileIndex_t chirpProfileIndex, rfe_error_t *rfe___error___pointer)`

Get the current *resetTimeTicks* from the selected profile.

- `void rfeCfg_chirpProfile_setCenterFrequency (uint8_t *pConfig, rfe_chirpProfileIndex_t chirpProfileIndex, uint32_t centerFrequency, rfe_error_t *rfe___error___pointer)`

Set a new value for *centerFrequency* in the selected profile.

- `uint32_t rfeCfg_chirpProfile_getCenterFrequency (const uint8_t *pConfig, rfe_chirpProfileIndex_t chirpProfileIndex, rfe_error_t *rfe___error___pointer)`

Get the current *centerFrequency* from the selected profile.

- `void rfeCfg_chirpProfile_setEffectiveChirpBandwidth (uint8_t *pConfig, rfe_chirpProfileIndex_t chirpProfileIndex, uint32_t effectiveChirpBandwidth, rfe_error_t *rfe___error___pointer)`

Set a new value for *effectiveChirpBandwidth* in the selected profile.

- `uint32_t rfeCfg_chirpProfile_getEffectiveChirpBandwidth (const uint8_t *pConfig, rfe_chirpProfileIndex_t chirpProfileIndex, rfe_error_t *rfe___error___pointer)`

Get the current *effectiveChirpBandwidth* from the selected profile.

- `void rfeCfg_chirpProfile_setChirpPllVco (uint8_t *pConfig, rfe_chirpProfileIndex_t chirpProfileIndex, rfe_chirpPllVco_t chirpPllVcoSelect, rfe_error_t *rfe___error___pointer)`

Set a new value for *chirpPllVcoSelect* in the selected profile.

- `uint8_t rfeCfg_chirpProfile_getChirpPllVco (const uint8_t *pConfig, rfe_chirpProfileIndex_t chirpProfileIndex, rfe_error_t *rfe___error___pointer)`

Get the current *chirpPllVcoSelect* from the selected profile.

- `void rfeCfg_chirpProfile_setChirpSlopeDirection (uint8_t *pConfig, rfe_chirpProfileIndex_t chirpProfileIndex, rfe_chirpSlopeDirection_t chirpSlopeDirection, rfe_error_t *rfe___error___pointer)`

Set a new value for *chirpSlopeDirection* in the selected profile.

- `uint8_t rfeCfg_chirpProfile_getChirpSlopeDirection (const uint8_t *pConfig, rfe_chirpProfileIndex_t chirpProfileIndex, rfe_error_t *rfe___error___pointer)`

Get the current *chirpSlopeDirection* from the selected profile.

- `void rfeCfg_chirpProfile_setChirpPllLoopFilterBandwidth (uint8_t *pConfig, rfe_chirpProfileIndex_t chirpProfileIndex, rfe_chirpPllLoopFilterBandwidth_t chirpPllLoopFilterBandwidth, rfe_error_t *rfe___error___pointer)`

Set a new value for *chirpPllLoopFilterBandwidth* in the selected profile.

- `uint8_t rfeCfg_chirpProfile_getChirpPllLoopFilterBandwidth (const uint8_t *pConfig, rfe_chirpProfileIndex_t chirpProfileIndex, rfe_error_t *rfe___error___pointer)`

Get the current *chirpPllLoopFilterBandwidth* from the selected profile.

- `void rfeCfg_chirpProfile_setTxTransmissionEnable (uint8_t *pConfig, rfe_chirpProfileIndex_t chirpProfileIndex, rfe_txSelect_t txTransmissionEnable, rfe_error_t *rfe___error___pointer)`

Set a new value for *txTransmissionEnable* in the selected profile.

- `uint8_t rfeCfg_chirpProfile_getTxTransmissionEnable (const uint8_t *pConfig, rfe_chirpProfileIndex_t chirpProfileIndex, rfe_error_t *rfe___error___pointer)`

Get the current *txTransmissionEnable* from the selected profile.

- void `rfeCfg_chirpProfile_setTxPower` (uint8_t *pConfig, `rfe_chirpProfileIndex_t` chirpProfileIndex, int16_t txPower, `rfe_error_t` *rfe_error_pointer)
Set a new value for txPower in the selected profile.
- int16_t `rfeCfg_chirpProfile_getTxPower` (const uint8_t *pConfig, `rfe_chirpProfileIndex_t` chirpProfileIndex, `rfe_error_t` *rfe_error_pointer)
Get the current txPower from the selected profile.
- void `rfeCfg_chirpProfile_setTxPhaseRotation` (uint8_t *pConfig, `rfe_chirpProfileIndex_t` chirpProfileIndex, `rfe_txIndex_t` tx, `rfe_phaseRotation_t` txPhaseRotation, `rfe_error_t` *rfe_error_pointer)
Set a new value for txPhaseRotation in the selected profile.
- uint8_t `rfeCfg_chirpProfile_getTxPhaseRotation` (const uint8_t *pConfig, `rfe_chirpProfileIndex_t` chirpProfileIndex, `rfe_txIndex_t` tx, `rfe_error_t` *rfe_error_pointer)
Get the current txPhaseRotation from the selected profile.
- void `rfeCfg_chirpProfile_setTxTransmissionReferenceTime` (uint8_t *pConfig, `rfe_chirpProfileIndex_t` chirpProfileIndex, `rfe_txTransmissionReferenceTime_t` txTransmissionReferenceTime, `rfe_error_t` *rfe_error_pointer)
Set a new value for Tx Transmission Reference Time.
- uint8_t `rfeCfg_chirpProfile_getTxTransmissionReferenceTime` (const uint8_t *pConfig, `rfe_chirpProfileIndex_t` chirpProfileIndex, `rfe_error_t` *rfe_error_pointer)
Get the current Tx Transmission Reference Time.
- void `rfeCfg_chirpProfile_setTxTransmissionTimeOffset` (uint8_t *pConfig, `rfe_chirpProfileIndex_t` chirpProfileIndex, uint16_t txTransmissionTimeOffset, `rfe_error_t` *rfe_error_pointer)
Set a new value for Tx Transmission Time Offset.
- uint16_t `rfeCfg_chirpProfile_getTxTransmissionTimeOffset` (const uint8_t *pConfig, `rfe_chirpProfileIndex_t` chirpProfileIndex, `rfe_error_t` *rfe_error_pointer)
Get the current Tx Transmission Time Offset.
- void `rfeCfg_chirpProfile_setRxGain` (uint8_t *pConfig, `rfe_chirpProfileIndex_t` chirpProfileIndex, `rfe_rxGain_t` rxGain, `rfe_error_t` *rfe_error_pointer)
Set a new value for rxGain in the selected profile.
- uint8_t `rfeCfg_chirpProfile_getRxGain` (const uint8_t *pConfig, `rfe_chirpProfileIndex_t` chirpProfileIndex, `rfe_error_t` *rfe_error_pointer)
Get the current rxGain from the selected profile.
- void `rfeCfg_chirpProfile_setRxHpfCutOffFrequency` (uint8_t *pConfig, `rfe_chirpProfileIndex_t` chirpProfileIndex, `rfe_rxHpfCutOffFrequency_t` rxHpfCutOffFrequency, `rfe_error_t` *rfe_error_pointer)
Set a new value for rxHpfCutOffFrequency in the selected profile.
- uint8_t `rfeCfg_chirpProfile_getRxHpfCutOffFrequency` (const uint8_t *pConfig, `rfe_chirpProfileIndex_t` chirpProfileIndex, `rfe_error_t` *rfe_error_pointer)
Get the current rxHpfCutOffFrequency from the selected profile.
- void `rfeCfg_chirpProfile_setRxLpfCutOffFrequency` (uint8_t *pConfig, `rfe_chirpProfileIndex_t` chirpProfileIndex, `rfe_rxLpfCutOffFrequency_t` rxLpfCutOffFrequency, `rfe_error_t` *rfe_error_pointer)
Set a new value for rxLpfCutOffFrequency in the selected profile.
- uint8_t `rfeCfg_chirpProfile_getRxLpfCutOffFrequency` (const uint8_t *pConfig, `rfe_chirpProfileIndex_t` chirpProfileIndex, `rfe_error_t` *rfe_error_pointer)
Get the current rxLpfCutOffFrequency from the selected profile.
- void `rfeCfg_chirpProfile_setVirtualChannel` (uint8_t *pConfig, `rfe_chirpProfileIndex_t` chirpProfileIndex, `rfe_virtualChannel_t` virtualChannel, `rfe_error_t` *rfe_error_pointer)
Set a new value for virtualChannel in the selected profile.
- uint8_t `rfeCfg_chirpProfile_getVirtualChannel` (const uint8_t *pConfig, `rfe_chirpProfileIndex_t` chirpProfileIndex, `rfe_error_t` *rfe_error_pointer)
Get the current virtualChannel from the selected profile.
- void `rfeCfg_radarCycle_setRadarCycleDuration` (uint8_t *pConfig, uint32_t radarCycleDuration, `rfe_error_t` *rfe_error_pointer)
Set a new value for radarCycleDuration.

- `uint32_t rfeCfg_radarCycle_getRadarCycleDuration` (const `uint8_t *pConfig`, `rfe_error_t *rfe___error___`↔
pointer)
 Get the current `radarCycleDuration`.
- `void rfeCfg_write` (`uint8_t *pConfig`, `rfeCfg_section_t` section, `rfeCfg_param_t` param, `uint32_t` value,
`rfe_error_t *rfe___error___`pointer)
 This function writes a value to the configuration blob, according to the provided parameters. The provided value must be a `uint32_t` value, but the written value is according to the size of the parameter definition (see [RFE Abstract 2.0 BLOB - Constants](#)).
- `uint8_t rfeCfg_readB` (const `uint8_t *pConfig`, `rfeCfg_section_t` section, `rfeCfg_param_t` param, `rfe_error_t *rfe___error___`pointer)
 This function reads a `uint8_t` value from the configuration.
- `uint16_t rfeCfg_readW` (const `uint8_t *pConfig`, `rfeCfg_section_t` section, `rfeCfg_param_t` param, `rfe_error_t *rfe___error___`pointer)
 This function reads a `uint16_t` value from the configuration.
- `uint32_t rfeCfg_readDW` (const `uint8_t *pConfig`, `rfeCfg_section_t` section, `rfeCfg_param_t` param, `rfe_error_t *rfe___error___`pointer)
 This function reads a `uint32_t` value from the configuration.
- `uint32_t rfeCfg_read` (const `uint8_t *pConfig`, `rfeCfg_section_t` section, `rfeCfg_param_t` param, `rfe_error_t *rfe___error___`pointer)
 This function reads a value from the configuration.
- `static uint8_t rfeCfg_radarCycle_getChirpSequenceCount` (const `uint8_t *pConfig`, `rfe_error_t *rfe___error___`↔
____pointer)
 Gets the `chirpSequenceCount`. This is useful in combination with `rfeCfg_radarCycle_getChirpSequence`, for determining how many sequence configurations are utilized in the configured Radar Cycle and in what order they are used.
- `uint8_t rfeCfg_radarCycle_getChirpSequence` (const `uint8_t *pConfig`, `uint8_t` chirpSequenceIndex, `rfe_error_t *rfe___error___`pointer)
 This function gets the sequence configuration, that is used at the selected index.
- `uint32_t rfeCfg_radarCycle_getChirpSequenceStartTimeOffset` (const `uint8_t *pConfig`, `uint8_t` chirp↔
SequenceIndex, `rfe_error_t *rfe___error___`pointer)
 This function gets the corresponding `chirpSequenceStartTimeOffset`.
- `static uint8_t rfeCfg_metadata_getChirpSequenceConfigCount` (const `uint8_t *pConfig`, `rfe_error_t *rfe___`↔
____error____pointer)
 This function gets the `chirpSequenceConfigCount`.
- `static uint8_t rfeCfg_metadata_getChirpProfileCount` (const `uint8_t *pConfig`, `rfe_error_t *rfe___error___`↔
____pointer)
 This function gets the `chirpProfileCount`.
- `void rfeCfg_general_setPowerMode` (`uint8_t *pConfig`, `rfe_powerMode_t` powerMode, `rfe_error_t *rfe___`↔
____error____pointer)
 This function sets the `powerMode`.
- `uint8_t rfeCfg_general_getPowerMode` (const `uint8_t *pConfig`, `rfe_error_t *rfe___error___`pointer)
 This function gets the `powerMode`.
- `uint8_t rfeCfg_general_getTxCalibrationMode` (const `uint8_t *pConfig`, `rfe_error_t *rfe___error___`pointer)
- `static uint8_t rfeCfg_general_getJumpbackTimeTicks` (const `uint8_t *pConfig`, `rfe_error_t *rfe___error___`↔
____pointer)
 This function gets the `jumpbackTimeTicks`.
- `uint8_t rfeCfg_monitorAndSafety_getFuSaFaultMask` (const `uint8_t *pConfig`, `rfe_fuSaFaultMaskIndex_t` fu↔
SaFaultMaskIndex, `rfe_error_t *rfe___error___`pointer)
 This function gets the `fuSaFaultMask`.
- `void rfeCfg_monitorAndSafety_setInjectTestTone` (`uint8_t *pConfig`, `rfeSwBist_InjectToneMode_t` injectTest↔
ToneMode, `rfe_error_t *rfe___error___`pointer)
 This function sets the `injectTestTone`.
- `uint8_t rfeCfg_monitorAndSafety_getInjectTestTone` (const `uint8_t *pConfig`, `rfe_error_t *rfe___error___`↔
____pointer)

This function gets the *injectTestTone*.

- void `rfeCfg_monitorAndSafety_setBist` (uint8_t *pConfig, `rfeCfg_param_t` param, int16_t value, `rfe_error_t` *rfe___error___pointer)

This function updates values, for any of the following parameters, in the in-memory blob:

- int16_t `rfeCfg_monitorAndSafety_getBist` (const uint8_t *pConfig, `rfeCfg_param_t` param, `rfe_error_t` *rfe___error___pointer)

This function gets the following values, from the in-memory blob:

- static uint8_t `rfeCfg_radarCycle_getBistInterval` (const uint8_t *pConfig, `rfe_error_t` *rfe___error___pointer)

This function gets the *bistInterval*.

- int32_t `rfeCfg_chirpSequence_getChirpFrequencyDriftSteps` (const uint8_t *pConfig, `rfe_chirpProfileIndex_t` chirpProfileIndex, `rfe_error_t` *rfe___error___pointer)

This function gets the *chirpFrequencyDriftSteps*.

- uint8_t `rfeCfg_sequence_getPhaseRotationDdmaMode` (const uint8_t *pConfig, `rfe_chirpSequenceIndex_t` chirpSequenceConfigIndex, `rfe_error_t` *rfe___error___pointer)
- uint8_t `rfeCfg_sequence_getFinalPcGenMode` (const uint8_t *pConfig, `rfe_chirpSequenceIndex_t` chirpSequenceConfigIndex, `rfe_txIndex_t` tx, `rfe_error_t` *rfe___error___pointer)
- uint8_t `rfeCfg_sequence_getPhaseRotationTxUseDdma` (const uint8_t *pConfig, `rfe_chirpSequenceIndex_t` chirpSequenceConfigIndex, `rfe_error_t` *rfe___error___pointer)
- uint8_t `rfeCfg_sequence_getTxDdmaInitPhase` (const uint8_t *pConfig, `rfe_chirpSequenceIndex_t` chirpSequenceConfigIndex, `rfe_txIndex_t` tx, `rfe_error_t` *rfe___error___pointer)
- uint8_t `rfeCfg_sequence_getTxDdmaUpdatePhase` (const uint8_t *pConfig, `rfe_chirpSequenceIndex_t` chirpSequenceConfigIndex, `rfe_txIndex_t` tx, `rfe_error_t` *rfe___error___pointer)
- uint8_t `rfeCfg_monitorAndSafety_getLoiBBD` (const uint8_t *pConfig, `rfe_error_t` *rfe___error___pointer)

This function gets the *checkBbd_loi*.

- void `rfeCfg_monitorAndSafety_setLoiBBD` (const uint8_t *pConfig, uint8_t val, `rfe_error_t` *rfe___error___pointer)

This function sets the *checkBbd_loi*.

- uint8_t `rfeCfg_monitorAndSafety_getTxBBD` (const uint8_t *pConfig, `rfe_error_t` *rfe___error___pointer)

This function gets the *checkBbd_tx*.

- uint8_t `rfeCfg_monitorAndSafety_getRxBBD` (const uint8_t *pConfig, `rfe_error_t` *rfe___error___pointer)

This function gets the *checkBbd_rx*.

- uint8_t `rfeCfg_monitorAndSafety_getRxSaturationThresholdStage` (const uint8_t *pConfig, uint8_t rxStage, uint8_t rxIndex, `rfe_error_t` *rfe___error___pointer)

This function gets the *rxSaturationThreshold*.

- uint16_t `rfeCfg_monitorAndSafety_getRxSaturationCountLimitStage` (const uint8_t *pConfig, uint8_t rxStage, uint8_t rxIndex, `rfe_error_t` *rfe___error___pointer)

This function gets the *rxSaturationCountLimit*.

- bool `rfeCfg_monitorAndSafety_getRxSatCountResetEveryChirpSequence` (const uint8_t *pConfig, `rfe_error_t` *rfe___error___pointer)

This function gets the *rxSatCountResetEveryChirpSequence*.

- bool `rfeCfg_monitorAndSafety_getAdcClippingCountResetEveryChirpSequence` (const uint8_t *pConfig, `rfe_error_t` *rfe___error___pointer)

This function gets the *adcClippingCountResetEveryChirpSequence*.

- void `rfeCfg_sequence_setTxDdmaUpdatePhase` (uint8_t *pConfig, `rfe_chirpSequenceIndex_t` chirpSequenceConfigIndex, `rfe_txIndex_t` tx, `rfe_phaseRotation_t` txDdmaUpdatePhase, `rfe_error_t` *rfe___error___pointer)

Set a new value for *txPhaseRotation_ddma_tx1_update* for the selected sequence.

- `rfeCfg_section_t` `rfeCfg_util_chirpSequenceConfigIndex` (`rfe_chirpSequenceIndex_t` chirpSequenceIndex, `rfe_error_t` *rfe___error___pointer)

Maps chirp sequence config index to the corresponding section.

- void `rfeCfg_sequence_setTxDdmaInitPhase` (uint8_t *pConfig, `rfe_chirpSequenceIndex_t` chirpSequenceConfigIndex, `rfe_txIndex_t` tx, `rfe_phaseRotation_t` txDdmaInitPhase, `rfe_error_t` *rfe___error___pointer)

Set a new value for *txPhaseRotation_ddma_tx1_init* for the selected sequence.

- void [rfeCfg_sequence_setPhaseRotationDdmaMode](#) (uint8_t *pConfig, [rfe_chirpSequenceIndex_t](#) chirpSequenceConfigIndex, uint8_t enable, [rfe_error_t](#) *rfe___error___pointer)
Set [txPhaseRotation_ddma](#) for the selected sequence.
- void [rfeCfg_sequence_setTxFinalPcGenMode](#) (uint8_t *pConfig, [rfe_chirpSequenceIndex_t](#) chirpSequenceConfigIndex, [rfe_txIndex_t](#) tx, [rfe_PRPCGenMode_e](#) txFinalPcGenMode, [rfe_error_t](#) *rfe___error___pointer)
Set a new value for [txPhaseRotation_ddma_tx1_mode](#) for the selected sequence.
- uint16_t [rfeCfg_sequence_getTxCalibrationThresholdHigh](#) (const uint8_t *pConfig, [rfe_chirpSequenceIndex_t](#) chirpSequenceConfigIndex, [rfe_error_t](#) *rfe___error___pointer)
Get the [txCalibration_thresholdHigh](#) for the selected sequence.
- uint16_t [rfeCfg_sequence_getTxCalibrationThresholdLow](#) (const uint8_t *pConfig, [rfe_chirpSequenceIndex_t](#) chirpSequenceConfigIndex, [rfe_error_t](#) *rfe___error___pointer)
Get the [txCalibration_thresholdLow](#) for the selected sequence.
- [rfeCfg_param_t](#) [rfeCfg_util_radarCycleChirpSequence](#) (uint8_t index, [rfe_error_t](#) *rfe___error___pointer)
Maps radar cycle chirp sequence index to the corresponding param.
- [rfeCfg_param_t](#) [rfeCfg_util_chirpProfileSequence](#) (uint8_t index, [rfe_error_t](#) *rfe___error___pointer)
Maps chirp profile sequence index to the corresponding param.
- [rfeCfg_section_t](#) [rfeCfg_util_chirpProfileIndex](#) ([rfe_chirpProfileIndex_t](#) chirpProfileIndex, [rfe_error_t](#) *rfe___error___pointer)
Writes a configuration item.
- void [rfeCfg_monitorAndSafety_setFrequencyForBist](#) (uint8_t *pConfig, uint32_t frequencyForBist, [rfe_error_t](#) *rfe___error___pointer)
This function sets the [frequencyForBist](#).
- uint32_t [rfeCfg_monitorAndSafety_getFrequencyForBist](#) (const uint8_t *pConfig, [rfe_error_t](#) *rfe___error___pointer)
This function gets the [frequencyForBist](#).
- void [rfeCfg_CheckBlobCompatibility](#) (void *configPtr, volatile [rfe_error_t](#) *rfe___error___pointer)
Check the blob version compatibility with the API version.
- bool [rfeCfg_general_getPdcBitwidth](#) (const uint8_t *pConfig, [rfe_error_t](#) *rfe___error___pointer)
This function gets the [pdcDecimationFilter](#) stored in the Configuration Data Memory.
- void [rfeCfg_calcAndWriteCrc](#) (void *pConfig, uint32_t len)

3.11.1 Detailed Description

RFE Abstract 2.0 low-level interfaces. These interfaces are sets of get()/set() function pairs for retrieving or updating data to/from the in-memory blob, as well as some helper functions.

3.11.2 Function Documentation

3.11.2.1 [rfeCfg_calcAndWriteCrc\(\)](#)

```
void rfeCfg_calcAndWriteCrc (
    void * pConfig,
    uint32_t len )
```

3.11.2.2 rfeCfg_CheckBlobCompatibility()

```
void rfeCfg_CheckBlobCompatibility (
    void * configPtr,
    volatile rfe_error_t * rfe__error__pointer )
```

Check the blob version compatibility with the API version.

Parameters

in	configPtr	Pointer to the blob
in, out	rfe__error__pointer	Error handling parameter:

3.11.2.3 rfeCfg_chirpProfile_getAcquisitionTimeTicks()

```
uint16_t rfeCfg_chirpProfile_getAcquisitionTimeTicks (
    const uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_error_t * rfe__error__pointer )
```

Get the current [acquisitionTimeTicks](#) from the selected profile.

Parameters

pConfig	= pointer to the blob configuration
chirpProfileIndex	= the chirp profile index
rfe__error__pointer	= pointer to the error variable

Returns

The existing Acquisition Time Ticks

3.11.2.4 rfeCfg_chirpProfile_getCenterFrequency()

```
uint32_t rfeCfg_chirpProfile_getCenterFrequency (
    const uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_error_t * rfe__error__pointer )
```

Get the current [centerFrequency](#) from the selected profile.

Parameters

pConfig	= pointer to the blob configuration
chirpProfileIndex	= the chirp profile index
rfe__error__pointer	= pointer to the error variable

Returns

The existing Center Frequency

3.11.2.5 rfeCfg_chirpProfile_getChirpIntervalTimeTicks()

```
uint32_t rfeCfg_chirpProfile_getChirpIntervalTimeTicks (
    const uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_error_t * rfe__error__pointer )
```

Get the current [chirpIntervalTimeTicks](#) from the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>rfe__error__pointer</i>	= pointer to the error variable

Returns

The existing Chirp Interval Time Ticks

3.11.2.6 rfeCfg_chirpProfile_getChirpPllLoopFilterBandwidth()

```
uint8_t rfeCfg_chirpProfile_getChirpPllLoopFilterBandwidth (
    const uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_error_t * rfe__error__pointer )
```

Get the current [chirpPllLoopFilterBandwidth](#) from the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>rfe__error__pointer</i>	= pointer to the error variable

Returns

The existing Chirp Pll Loop Filter Bandwidth

3.11.2.7 rfeCfg_chirpProfile_getChirpPllVco()

```
uint8_t rfeCfg_chirpProfile_getChirpPllVco (
    const uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_error_t * rfe__error__pointer )
```

Get the current /ref chirpPllVcoSelect from the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>rfe__error__pointer</i>	= pointer to the error variable

Returns

The existing Chirp Pll Vco

3.11.2.8 rfeCfg_chirpProfile_getChirpSlopeDirection()

```
uint8_t rfeCfg_chirpProfile_getChirpSlopeDirection (
    const uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_error_t * rfe__error__pointer )
```

Get the current chirpSlopeDirection from the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>rfe__error__pointer</i>	= pointer to the error variable

Returns

The existing Chirp Slope Direction

3.11.2.9 rfeCfg_chirpProfile_getDwellTimeTicks()

```
uint16_t rfeCfg_chirpProfile_getDwellTimeTicks (
    const uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_error_t * rfe__error__pointer )
```

Get the current dwellTimeTicks from the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>rfe__error__pointer</i>	= pointer to the error variable

Returns

The existing Dwell Time Ticks

3.11.2.10 rfeCfg_chirpProfile_getEffectiveChirpBandwidth()

```
uint32_t rfeCfg_chirpProfile_getEffectiveChirpBandwidth (
    const uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_error_t * rfe__error__pointer )
```

Get the current [effectiveChirpBandwidth](#) from the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>rfe__error__pointer</i>	= pointer to the error variable

Returns

The existing Effective Chirp Bandwidth

3.11.2.11 rfeCfg_chirpProfile_getEffectiveSamplingFrequency()

```
uint8_t rfeCfg_chirpProfile_getEffectiveSamplingFrequency (
    const uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_error_t * rfe__error__pointer )
```

Get the current [effectiveSamplingFrequency](#) value from the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>rfe__error__pointer</i>	= pointer to the error variable

Returns

The existing Effective Sampling Frequency

3.11.2.12 rfeCfg_chirpProfile_getResetTimeTicks()

```
uint16_t rfeCfg_chirpProfile_getResetTimeTicks (
    const uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_error_t * rfe__error__pointer )
```

Get the current [resetTimeTicks](#) from the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>rfe__error__pointer</i>	= pointer to the error variable

Returns

The existing Reset Time Ticks

3.11.2.13 rfeCfg_chirpProfile_getRxGain()

```
uint8_t rfeCfg_chirpProfile_getRxGain (
    const uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_error_t * rfe__error__pointer )
```

Get the current [rxGain](#) from the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>rfe__error__pointer</i>	= pointer to the error variable

Returns

The existing Rx Gain

3.11.2.14 rfeCfg_chirpProfile_getRxHpfcutOffFrequency()

```
uint8_t rfeCfg_chirpProfile_getRxHpfcutOffFrequency (
    const uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_error_t * rfe__error__pointer )
```

Get the current [rxHpfcutOffFrequency](#) from the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>rfe__error__pointer</i>	= pointer to the error variable

Returns

The existing Rx Hpfcut Off Frequency

3.11.2.15 rfeCfg_chirpProfile_getRxLpfcutOffFrequency()

```
uint8_t rfeCfg_chirpProfile_getRxLpfcutOffFrequency (
    const uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_error_t * rfe__error__pointer )
```

Get the current [rxLpfcutOffFrequency](#) from the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>rfe__error__pointer</i>	= pointer to the error variable

Returns

The existing Rx Lpfcut Off Frequency

3.11.2.16 rfeCfg_chirpProfile_getSettleTimeTicks()

```
uint16_t rfeCfg_chirpProfile_getSettleTimeTicks (
    const uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_error_t * rfe__error__pointer )
```

Get the current [settleTimeTicks](#) from the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>rfe__error__pointer</i>	= pointer to the error variable

Returns

The existing Settle Time Ticks

3.11.2.17 rfeCfg_chirpProfile_getTxPhaseRotation()

```
uint8_t rfeCfg_chirpProfile_getTxPhaseRotation (
    const uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_txIndex_t tx,
    rfe_error_t * rfe__error__pointer )
```

Get the current [txPhaseRotation](#) from the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>tx</i>	= the index value for Tx Phase Rotation
<i>rfe__error__pointer</i>	= pointer to the error variable

Returns

The existing Tx Phase Rotation

3.11.2.18 rfeCfg_chirpProfile_getTxPower()

```
int16_t rfeCfg_chirpProfile_getTxPower (
    const uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_error_t * rfe__error__pointer )
```

Get the current [txPower](#) from the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>rfe__error__pointer</i>	= pointer to the error variable

Returns

The existing Tx Power

3.11.2.19 rfeCfg_chirpProfile_getTxTransmissionEnable()

```
uint8_t rfeCfg_chirpProfile_getTxTransmissionEnable (
    const uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_error_t * rfe__error__pointer )
```

Get the current [txTransmissionEnable](#) from the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>rfe__error__pointer</i>	= pointer to the error variable

Returns

The existing Tx Transmission Enable

3.11.2.20 rfeCfg_chirpProfile_getTxTransmissionReferenceTime()

```
uint8_t rfeCfg_chirpProfile_getTxTransmissionReferenceTime (
    const uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_error_t * rfe__error__pointer )
```

Get the current Tx Transmission Reference Time.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>rfe__error__pointer</i>	= pointer to the error variable

Returns

The existing Tx Transmission Reference Time

3.11.2.21 rfeCfg_chirpProfile_getTxTransmissionTimeOffset()

```
uint16_t rfeCfg_chirpProfile_getTxTransmissionTimeOffset (
    const uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_error_t * rfe__error__pointer )
```

Get the current Tx Transmission Time Offset.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>rfe__error__pointer</i>	= pointer to the error variable

Returns

The existing Tx Transmission Time Offset

3.11.2.22 rfeCfg_chirpProfile_getVirtualChannel()

```
uint8_t rfeCfg_chirpProfile_getVirtualChannel (
    const uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_error_t * rfe__error__pointer )
```

Get the current [virtualChannel](#) from the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>rfe__error__pointer</i>	= pointer to the error variable

Returns

The existing Virtual Channel

3.11.2.23 rfeCfg_chirpProfile_setAcquisitionTimeTicks()

```
void rfeCfg_chirpProfile_setAcquisitionTimeTicks (
    uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    uint16_t acquisitionTimeTicks,
    rfe_error_t * rfe__error__pointer )
```

Set a new value for [acquisitionTimeTicks](#) in the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>acquisitionTimeTicks</i>	= the new value for Acquisition Time Ticks
<i>rfe__error__pointer</i>	= pointer to the error variable

3.11.2.24 rfeCfg_chirpProfile_setCenterFrequency()

```
void rfeCfg_chirpProfile_setCenterFrequency (
    uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    uint32_t centerFrequency,
    rfe_error_t * rfe__error__pointer )
```

Set a new value for [centerFrequency](#) in the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>centerFrequency</i>	= the new value for Center Frequency
<i>rfe__error__pointer</i>	= pointer to the error variable

3.11.2.25 rfeCfg_chirpProfile_setChirpIntervalTimeTicks()

```
void rfeCfg_chirpProfile_setChirpIntervalTimeTicks (
    uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    uint32_t chirpIntervalTimeTicks,
    rfe_error_t * rfe__error__pointer )
```

Set a new value for [chirpIntervalTimeTicks](#) in the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>chirpIntervalTimeTicks</i>	= the new value for Chirp Interval Time Ticks
<i>rfe__error__pointer</i>	= pointer to the error variable

3.11.2.26 rfeCfg_chirpProfile_setChirpPllLoopFilterBandwidth()

```
void rfeCfg_chirpProfile_setChirpPllLoopFilterBandwidth (
    uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_chirpPllLoopFilterBandwidth_t chirpPllLoopFilterBandwidth,
    rfe_error_t * rfe__error__pointer )
```

Set a new value for [chirpPllLoopFilterBandwidth](#) in the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>chirpPllLoopFilterBandwidth</i>	= the new value for Chirp Pll Loop Filter Bandwidth
<i>rfe__error__pointer</i>	= pointer to the error variable

3.11.2.27 rfeCfg_chirpProfile_setChirpPllVco()

```
void rfeCfg_chirpProfile_setChirpPllVco (
    uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_chirpPllVco_t chirpPllVcoSelect,
    rfe_error_t * rfe__error__pointer )
```

Set a new value for [chirpPllVcoSelect](#) in the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>chirpPllVcoSelect</i>	= the new value for Chirp Pll Vco
<i>rfe__error__pointer</i>	= pointer to the error variable

3.11.2.28 rfeCfg_chirpProfile_setChirpSlopeDirection()

```
void rfeCfg_chirpProfile_setChirpSlopeDirection (
    uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_chirpSlopeDirection_t chirpSlopeDirection,
    rfe_error_t * rfe__error__pointer )
```

Set a new value for [chirpSlopeDirection](#) in the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>chirpSlopeDirection</i>	= the new value for Chirp Slope Direction
<i>rfe__error__pointer</i>	= pointer to the error variable

3.11.2.29 rfeCfg_chirpProfile_setDwellTimeTicks()

```
void rfeCfg_chirpProfile_setDwellTimeTicks (
    uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    uint16_t dwellTimeTicks,
    rfe_error_t * rfe__error__pointer )
```

Set a new value for [dwellTimeTicks](#) in the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>dwellTimeTicks</i>	= the new value for Dwell Time Ticks
<i>rfe__error__pointer</i>	= pointer to the error variable

3.11.2.30 rfeCfg_chirpProfile_setEffectiveChirpBandwidth()

```
void rfeCfg_chirpProfile_setEffectiveChirpBandwidth (
    uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    uint32_t effectiveChirpBandwidth,
    rfe_error_t * rfe__error__pointer )
```

Set a new value for [effectiveChirpBandwidth](#) in the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>effectiveChirpBandwidth</i>	= the new value for Effective Chirp Bandwidth
<i>rfe__error__pointer</i>	= pointer to the error variable

3.11.2.31 rfeCfg_chirpProfile_setEffectiveSamplingFrequency()

```
void rfeCfg_chirpProfile_setEffectiveSamplingFrequency (
    uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_effectiveSamplingFrequency_t effectiveSamplingFrequency,
    rfe_error_t * rfe__error__pointer )
```

Set a new value for [effectiveSamplingFrequency](#) in the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>effectiveSamplingFrequency</i>	= the new value for Effective Sampling Frequency
<i>rfe__error__pointer</i>	= pointer to the error variable

3.11.2.32 rfeCfg_chirpProfile_setResetTimeTicks()

```
void rfeCfg_chirpProfile_setResetTimeTicks (
    uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    uint16_t resetTimeTicks,
    rfe_error_t * rfe__error__pointer )
```

Set a new value for [resetTimeTicks](#) in the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>resetTimeTicks</i>	= the new value for Reset Time Ticks
<i>rfe__error__pointer</i>	= pointer to the error variable

3.11.2.33 rfeCfg_chirpProfile_setRxGain()

```
void rfeCfg_chirpProfile_setRxGain (
    uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_rxGain_t rxGain,
    rfe_error_t * rfe__error__pointer )
```

Set a new value for [rxGain](#) in the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>rxGain</i>	= the new value for Rx Gain
<i>rfe__error__pointer</i>	= pointer to the error variable

3.11.2.34 rfeCfg_chirpProfile_setRxHpfCutOffFrequency()

```
void rfeCfg_chirpProfile_setRxHpfCutOffFrequency (
    uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_rxHpfCutOffFrequency_t rxHpfCutOffFrequency,
    rfe_error_t * rfe__error__pointer )
```

Set a new value for **rxHpfCutOffFrequency** in the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>rxHpfCutOffFrequency</i>	= the new value for Rx Hpf CutOff Frequency
<i>rfe__error__pointer</i>	= pointer to the error variable

3.11.2.35 rfeCfg_chirpProfile_setRxLpfCutOffFrequency()

```
void rfeCfg_chirpProfile_setRxLpfCutOffFrequency (
    uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_rxLpfCutOffFrequency_t rxLpfCutOffFrequency,
    rfe_error_t * rfe__error__pointer )
```

Set a new value for **rxLpfCutOffFrequency** in the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>rxLpfCutOffFrequency</i>	= the new value for Rx Lpf CutOff Frequency
<i>rfe__error__pointer</i>	= pointer to the error variable

3.11.2.36 rfeCfg_chirpProfile_setSettleTimeTicks()

```
void rfeCfg_chirpProfile_setSettleTimeTicks (
    uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    uint16_t settleTimeTicks,
    rfe_error_t * rfe__error__pointer )
```

Set a new value for [settleTimeTicks](#) in the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>settleTimeTicks</i>	= the new value for Settle Time Ticks
<i>rfe__error__pointer</i>	= pointer to the error variable

3.11.2.37 rfeCfg_chirpProfile_setTxPhaseRotation()

```
void rfeCfg_chirpProfile_setTxPhaseRotation (
    uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_txIndex_t tx,
    rfe_phaseRotation_t txPhaseRotation,
    rfe_error_t * rfe__error__pointer )
```

Set a new value for [txPhaseRotation](#) in the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>tx</i>	= the index value for Tx Phase Rotation
<i>txPhaseRotation</i>	= the new value for Tx Phase Rotation
<i>rfe__error__pointer</i>	= pointer to the error variable

3.11.2.38 rfeCfg_chirpProfile_setTxPower()

```
void rfeCfg_chirpProfile_setTxPower (
    uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    int16_t txPower,
    rfe_error_t * rfe__error__pointer )
```

Set a new value for [txPower](#) in the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>txPower</i>	= the new value for Tx Power
<i>rfe__error__pointer</i>	= pointer to the error variable

3.11.2.39 rfeCfg_chirpProfile_setTxTransmissionEnable()

```
void rfeCfg_chirpProfile_setTxTransmissionEnable (
    uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_txSelect_t txTransmissionEnable,
    rfe_error_t * rfe__error__pointer )
```

Set a new value for **txTransmissionEnable** in the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>txTransmissionEnable</i>	= the new value for Tx Transmission Enable
<i>rfe__error__pointer</i>	= pointer to the error variable

3.11.2.40 rfeCfg_chirpProfile_setTxTransmissionReferenceTime()

```
void rfeCfg_chirpProfile_setTxTransmissionReferenceTime (
    uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_txTransmissionReferenceTime_t txTransmissionReferenceTime,
    rfe_error_t * rfe__error__pointer )
```

Set a new value for Tx Transmission Reference Time.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>txTransmissionReferenceTime</i>	= the new value for Tx Transmission Reference Time
<i>rfe__error__pointer</i>	= pointer to the error variable

3.11.2.41 rfeCfg_chirpProfile_setTxTransmissionTimeOffset()

```
void rfeCfg_chirpProfile_setTxTransmissionTimeOffset (
    uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    uint16_t txTransmissionTimeOffset,
    rfe_error_t * rfe___error___pointer )
```

Set a new value for Tx Transmission Time Offset.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>txTransmissionTimeOffset</i>	= the new value for Tx Transmission Time Offset
<i>rfe___error___pointer</i>	= pointer to the error variable

3.11.2.42 rfeCfg_chirpProfile_setVirtualChannel()

```
void rfeCfg_chirpProfile_setVirtualChannel (
    uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_virtualChannel_t virtualChannel,
    rfe_error_t * rfe___error___pointer )
```

Set a new value for [virtualChannel](#) in the selected profile.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpProfileIndex</i>	= the chirp profile index
<i>virtualChannel</i>	= the new value for Virtual Channel
<i>rfe___error___pointer</i>	= pointer to the error variable

3.11.2.43 rfeCfg_chirpSequence_getChirpCount()

```
uint16_t rfeCfg_chirpSequence_getChirpCount (
    const uint8_t * pConfig,
    rfe_chirpSequenceIndex_t chirpSequenceConfigIndex,
    rfe_error_t * rfe___error___pointer )
```

Gets the current [chirpCount](#) for the selected sequence.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpSequenceConfigIndex</i>	= the index of the sequence to be checked
<i>rfe___error___pointer</i>	= pointer to the error variable

Returns

The existing [chirpCount](#), as uint16_t

3.11.2.44 rfeCfg_chirpSequence_getChirpFrequencyDriftSteps()

```
int32_t rfeCfg_chirpSequence_getChirpFrequencyDriftSteps (
    const uint8_t * pConfig,
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_error_t * rfe__error__pointer )
```

This function gets the [chirpFrequencyDriftSteps](#).

Parameters

in	<i>pConfig</i>	- Pointer to the Configuration Data Memory.
in	<i>chirpProfileIndex</i>	- The chirp profile config index.
in, out	<i>rfe__error__pointer</i>	- fault handling parameter: On success *rfe__error__pointer == rfe_error_none_e is true, On failure *rfe__error__pointer != rfe_error_none_e is true

Returns

The Chirp Frequency Drift.

3.11.2.45 rfeCfg_chirpSequence_getChirpProfileRepeatCount()

```
uint8_t rfeCfg_chirpSequence_getChirpProfileRepeatCount (
    const uint8_t * pConfig,
    rfe_chirpSequenceIndex_t chirpSequenceConfigIndex,
    rfe_error_t * rfe__error__pointer )
```

Get the [chirpProfileRepeatCount](#) for the selected sequence.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpSequenceConfigIndex</i>	= the index of the sequence to be checked
<i>rfe__error__pointer</i>	= pointer to the error variable

Returns

The existing sequence length

3.11.2.46 rfeCfg_chirpSequence_getChirpProfileSequence()

```
uint8_t rfeCfg_chirpSequence_getChirpProfileSequence (
    const uint8_t * pConfig,
    rfe_chirpSequenceIndex_t chirpSequenceConfigIndex,
    uint8_t chirpProfileSequenceIndex,
    rfe_error_t * rfe___error___pointer )
```

Gets which profiles are used in the selected sequence.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpSequenceConfigIndex</i>	= the sequence to be checked
<i>chirpProfileSequenceIndex</i>	= the index from which to read, within the list of profiles
<i>rfe___error___pointer</i>	= pointer to the error variable

Returns

The existing Chirp Profile Sequence

3.11.2.47 rfeCfg_chirpSequence_getChirpProfileSequenceLength()

```
uint8_t rfeCfg_chirpSequence_getChirpProfileSequenceLength (
    const uint8_t * pConfig,
    rfe_chirpSequenceIndex_t chirpSequenceConfigIndex,
    rfe_error_t * rfe___error___pointer )
```

Gets the number of profiles used for the selected sequence (see [chirpProfileSequenceLength](#)).

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpSequenceConfigIndex</i>	= the index of the sequence to be checked
<i>rfe___error___pointer</i>	= pointer to the error variable

Returns

The existing sequence length

3.11.2.48 rfeCfg_chirpSequence_getDynamicTableIndex()

```
uint8_t rfeCfg_chirpSequence_getDynamicTableIndex (
    const uint8_t * pConfig,
    rfe_chirpSequenceIndex_t chirpSequenceConfigIndex,
    rfe_error_t * rfe___error___pointer )
```

Get the [dynamicTableIndex](#) for the selected sequence.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpSequenceConfigIndex</i>	= the index of the sequence to be checked
<i>rfe__error__pointer</i>	= pointer to the error variable

Returns

The existing dynamic table index

3.11.2.49 rfeCfg_chirpSequence_getDynamicUpdatesEnabled()

```
bool rfeCfg_chirpSequence_getDynamicUpdatesEnabled (
    const uint8_t * pConfig,
    rfe_chirpSequenceIndex_t chirpSequenceConfigIndex,
    rfe_error_t * rfe__error__pointer )
```

Get the existing setting for [dynamicUpdatesEnabled](#).

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpSequenceConfigIndex</i>	= the index of the sequence to be checked
<i>rfe__error__pointer</i>	= pointer to the error variable

Returns

The dynamic update current status

3.11.2.50 rfeCfg_chirpSequence_getRxEnable()

```
uint8_t rfeCfg_chirpSequence_getRxEnable (
    const uint8_t * pConfig,
    rfe_chirpSequenceIndex_t chirpSequenceConfigIndex,
    rfe_error_t * rfe__error__pointer )
```

Gets which RX antennas are enabled/disabled for the selected sequence (see [rxEnable](#))

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpSequenceConfigIndex</i>	= the index of the sequence to be checked
<i>rfe__error__pointer</i>	= pointer to the error variable

Returns

The existing Rx Enablement

3.11.2.51 rfeCfg_chirpSequence_setChirpCount()

```
void rfeCfg_chirpSequence_setChirpCount (
    uint8_t * pConfig,
    rfe_chirpSequenceIndex_t chirpSequenceConfigIndex,
    uint16_t chirpCount,
    rfe_error_t * rfe___error___pointer )
```

Sets a different [chirpCount](#) for the selected sequence.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpSequenceConfigIndex</i>	= the index of the sequence to be changed
<i>chirpCount</i>	= the new chir count
<i>rfe___error___pointer</i>	= pointer to the error variable

3.11.2.52 rfeCfg_chirpSequence_setChirpProfileRepeatCount()

```
void rfeCfg_chirpSequence_setChirpProfileRepeatCount (
    uint8_t * pConfig,
    rfe_chirpSequenceIndex_t chirpSequenceConfigIndex,
    uint8_t chirpProfileRepeatCount,
    rfe_error_t * rfe___error___pointer )
```

Set the [chirpProfileRepeatCount](#) for the selected sequence.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpSequenceConfigIndex</i>	= the index of the sequence to be changed
<i>chirpProfileRepeatCount</i>	= the new repeat count
<i>rfe___error___pointer</i>	= pointer to the error variable

3.11.2.53 rfeCfg_chirpSequence_setChirpProfileSequence()

```
void rfeCfg_chirpSequence_setChirpProfileSequence (
    uint8_t * pConfig,
```

```

rfe_chirpSequenceIndex_t chirpSequenceConfigIndex,
uint8_t chirpProfileSequenceIndex,
rfe_chirpProfileIndex_t chirpProfile,
rfe_error_t * rfe__error__pointer )

```

Sets which profiles to be used in the selected sequence.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpSequenceConfigIndex</i>	= the sequence to be updated
<i>chirpProfileSequenceIndex</i>	= the index where to add the new profile, within the list
<i>chirpProfile</i>	= the profile to add to the list
<i>rfe__error__pointer</i>	= pointer to the error variable

Remarks

The value range for *chirpProfileSequenceIndex*, should be [0 ... *chirpProfileSequenceLength* - 1] For example, if *chirpProfileSequenceLength* = 3, and this function wants to set for *rfeCfg_param_chirpSequence_chirpProfileSequence_5_e*, it will have no effect, since only indexes [0,1,2] will be used from the list:

- *rfeCfg_param_chirpSequence_chirpProfileSequence_0_e*
- *rfeCfg_param_chirpSequence_chirpProfileSequence_1_e*
- *rfeCfg_param_chirpSequence_chirpProfileSequence_2_e*

3.11.2.54 rfeCfg_chirpSequence_setChirpProfileSequenceLength()

```

void rfeCfg_chirpSequence_setChirpProfileSequenceLength (
    uint8_t * pConfig,
    rfe_chirpSequenceIndex_t chirpSequenceConfigIndex,
    uint8_t chirpProfileSequenceLength,
    rfe_error_t * rfe__error__pointer )

```

Sets the number of profiles used for the selected sequence (see *chirpProfileSequenceLength*). Useful for adding or removing profiles that will be used in the selected sequence.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpSequenceConfigIndex</i>	= the index of the sequence to be changed
<i>chirpProfileSequenceLength</i>	= the new sequence length
<i>rfe__error__pointer</i>	= pointer to the error variable

Remarks

By increasing the *chirpProfileSequenceLength*, it is important that the newly added indexes from the list, will point to a valid chirp profile (see *rfeCfg_chirpSequence_setChirpProfileSequence*) For example, if *chirpProfileSequenceLength* = 3, but want to increase it to 5 by calling this function, then indexes 3 & 4 must be initialized with a profile index, otherwise profile "0" will be used by default:

- `rfeCfg_param_chirpSequence_chirpProfileSequence_0_e` - *previously initialized*
- `rfeCfg_param_chirpSequence_chirpProfileSequence_1_e` - *previously initialized*
- `rfeCfg_param_chirpSequence_chirpProfileSequence_2_e` - *previously initialized*
- `rfeCfg_param_chirpSequence_chirpProfileSequence_3_e` - **0!**
- `rfeCfg_param_chirpSequence_chirpProfileSequence_4_e` - **0!**

3.11.2.55 `rfeCfg_chirpSequence_setDynamicTableIndex()`

```
void rfeCfg_chirpSequence_setDynamicTableIndex (
    uint8_t * pConfig,
    rfe_chirpSequenceIndex_t chirpSequenceConfigIndex,
    uint8_t dynamicTableIndex,
    rfe_error_t * rfe__error__pointer )
```

Set the `dynamicTableIndex` for the selected sequence.

Parameters

<code>pConfig</code>	= pointer to the blob configuration
<code>chirpSequenceConfigIndex</code>	= the index of the sequence to be set
<code>dynamicTableIndex</code>	= the new dynamic table index
<code>rfe__error__pointer</code>	= pointer to the error variable

3.11.2.56 `rfeCfg_chirpSequence_setDynamicUpdatesEnabled()`

```
void rfeCfg_chirpSequence_setDynamicUpdatesEnabled (
    uint8_t * pConfig,
    rfe_chirpSequenceIndex_t chirpSequenceConfigIndex,
    bool dynamicUpdatesEnabled,
    rfe_error_t * rfe__error__pointer )
```

Sets whether the dynamic updates are enabled or disabled for the selected sequence. (see `dynamicUpdatesEnabled`)

Parameters

<code>pConfig</code>	= pointer to the blob configuration
<code>chirpSequenceConfigIndex</code>	= the index of the sequence to be changed
<code>dynamicUpdatesEnabled</code>	= the new dynamic update enablement
<code>rfe__error__pointer</code>	= pointer to the error variable

3.11.2.57 rfeCfg_chirpSequence_setRxEnable()

```
void rfeCfg_chirpSequence_setRxEnable (
    uint8_t * pConfig,
    rfe_chirpSequenceIndex_t chirpSequenceConfigIndex,
    rfe_rxSelect_t rxEnable,
    rfe_error_t * rfe__error__pointer )
```

Sets which RX antennas to be enabled/disabled for the selected sequence (see [rxEnable](#))

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpSequenceConfigIndex</i>	= the index of the sequence to be set
<i>rxEnable</i>	= the new value for Rx Enablement
<i>rfe__error__pointer</i>	= pointer to the error variable

3.11.2.58 rfeCfg_general_getJumpbackTimeTicks()

```
static uint8_t rfeCfg_general_getJumpbackTimeTicks (
    const uint8_t * pConfig,
    rfe_error_t * rfe__error__pointer ) [inline], [static]
```

This function gets the [jumpbackTimeTicks](#) .

Parameters

in	<i>pConfig</i>	- Pointer to the Configuration Data Memory.
in, out	<i>rfe__error__pointer</i>	- Error handling parameter: On success *rfe__error__pointer == rfe_error_none_e is true, On failure *rfe__error__pointer != rfe_error_none_e is true

Returns

[jumpbackTimeTicks](#)

3.11.2.59 rfeCfg_general_getPdcBitwidth()

```
bool rfeCfg_general_getPdcBitwidth (
    const uint8_t * pConfig,
    rfe_error_t * rfe__error__pointer )
```

This function gets the [pdcDecimationFilter](#) stored in the Configuration Data Memory.

Parameters

in	<i>pConfig</i>	- Pointer to the Configuration Data Memory.
in, out	<i>rfe__error__pointer</i>	Error handling parameter:

Returns

pdcDecimationFilter

3.11.2.60 rfeCfg_general_getPowerMode()

```
uint8_t rfeCfg_general_getPowerMode (
    const uint8_t * pConfig,
    rfe_error_t * rfe__error__pointer )
```

This function gets the [powerMode](#) .

Parameters

in	<i>pConfig</i>	- Pointer to the Configuration Data Memory.
in, out	<i>rfe__error__pointer</i>	- pointer to the error status On success *rfe__error__pointer == rfe_error_none_e is true, On failure *rfe__error__pointer != rfe_error_none_e is true

Returns

[powerMode](#)

3.11.2.61 rfeCfg_general_getTxCalibrationMode()

```
uint8_t rfeCfg_general_getTxCalibrationMode (
    const uint8_t * pConfig,
    rfe_error_t * rfe__error__pointer )
```

3.11.2.62 rfeCfg_general_setPowerMode()

```
void rfeCfg_general_setPowerMode (
    uint8_t * pConfig,
    rfe_powerMode_t powerMode,
    rfe_error_t * rfe__error__pointer )
```

This function sets the [powerMode](#) .

Parameters

in	<i>pConfig</i>	- Pointer to the Configuration Data Memory.
in	<i>powerMode</i>	- powerMode
in, out	<i>rfe__error__pointer</i>	- pointer to the error status On success *rfe__error__pointer == rfe_error_none_e is true, On failure *rfe__error__pointer != rfe_error_none_e is true

3.11.2.63 rfeCfg_metadata_getChirpProfileCount()

```
static uint8_t rfeCfg_metadata_getChirpProfileCount (
    const uint8_t * pConfig,
    rfe_error_t * rfe__error__pointer ) [inline], [static]
```

This function gets the [chirpProfileCount](#) .

Parameters

in	<i>pConfig</i>	- Pointer to the Configuration Data Memory.
in, out	<i>rfe__error__pointer</i>	- Error handling parameter: On success *rfe__error__pointer == rfe_error_none_e is true, On failure *rfe__error__pointer != rfe_error_none_e is true

Returns

[chirpProfileCount](#)

3.11.2.64 rfeCfg_metadata_getChirpSequenceConfigCount()

```
static uint8_t rfeCfg_metadata_getChirpSequenceConfigCount (
    const uint8_t * pConfig,
    rfe_error_t * rfe__error__pointer ) [inline], [static]
```

This function gets the [chirpSequenceConfigCount](#) .

Parameters

in	<i>pConfig</i>	- Pointer to the Configuration Data Memory.
in, out	<i>rfe__error__pointer</i>	- Error handling parameter: On success *rfe__error__pointer == rfe_error_none_e is true, On failure *rfe__error__pointer != rfe_error_none_e is true

Returns

[chirpSequenceConfigCount](#)

3.11.2.65 rfeCfg_monitorAndSafety_getAdcClippingCountResetEveryChirpSequence()

```
bool rfeCfg_monitorAndSafety_getAdcClippingCountResetEveryChirpSequence (
    const uint8_t * pConfig,
    rfe_error_t * rfe__error__pointer )
```

This function gets the [adcClippingCountResetEveryChirpSequence](#) .

Parameters

in	<i>pConfig</i>	- Pointer to the Configuration Data Memory.
in, out	<i>rfe__error__pointer</i>	- fault handling parameter: On success * <i>rfe__error__pointer</i> == <i>rfe_error_none_e</i> is true, On failure * <i>rfe__error__pointer</i> != <i>rfe_error_none_e</i> is true

Returns

[rfe_paramUpdate_monitorAndSafety_adcClippingCountResetEveryChirpSequence_e](#)

3.11.2.66 rfeCfg_monitorAndSafety_getBist()

```
int16_t rfeCfg_monitorAndSafety_getBist (
    const uint8_t * pConfig,
    rfeCfg_param_t param,
    rfe_error_t * rfe__error__pointer )
```

This function gets the following values, from the in-memory blob:

- [zeroHrRefForRxPhaseDiffLna](#)
- [rxPhaseDiffLnaThresholdTolerance](#)
- [zeroHourReferenceForRxGainDiffLna](#)
- [rxGainDiffLnaThresholdTolerance](#)
- [zeroHrRefForRxPhaseDiffMixer](#)
- [rxPhaseDiffMixerThresholdTolerance](#)
- [zeroHourReferenceForRxGainDiffMixer](#)
- [rxGainDiffMixerThresholdTolerance](#)

Parameters

in	<i>pConfig</i>	- Pointer to the Configuration Data Memory.
in	<i>param</i>	- The BIST parameter to get, like rfeCfg_param_monitorAndSafety_zeroHrRefForRxPhaseDiffLna_rx1_rx2_e
in, out	<i>rfe__error__pointer</i>	- Error handling parameter: On success * <i>rfe__error__pointer</i> == <i>rfe_error_none_e</i> is true, On failure * <i>rfe__error__pointer</i> != <i>rfe_error_none_e</i> is true

Returns

the current value for the required parameter

3.11.2.67 rfeCfg_monitorAndSafety_getFrequencyForBist()

```
uint32_t rfeCfg_monitorAndSafety_getFrequencyForBist (
    const uint8_t * pConfig,
    rfe_error_t * rfe___error___pointer )
```

This function gets the [frequencyForBist](#) .

Parameters

in	<i>pConfig</i>	Pointer to the Configuration Data Memory.
in, out	<i>rfe___error___pointer</i>	Error handling parameter.

Returns

[frequencyForBist](#).

3.11.2.68 rfeCfg_monitorAndSafety_getFuSaFaultMask()

```
uint8_t rfeCfg_monitorAndSafety_getFuSaFaultMask (
    const uint8_t * pConfig,
    rfe_fuSaFaultMaskIndex_t fuSaFaultMaskIndex,
    rfe_error_t * rfe___error___pointer )
```

This function gets the [fuSaFaultMask](#) .

Parameters

in	<i>pConfig</i>	- Pointer to the Configuration Data Memory.
in	<i>fuSaFaultMaskIndex</i>	- The FuSa fault mask
in, out	<i>rfe___error___pointer</i>	- fault handling parameter: On success *rfe___error___pointer == rfe_error_none_e is true, On failure *rfe___error___pointer != rfe_error_none_e is true

Returns

[fuSaFaultMask](#)

3.11.2.69 rfeCfg_monitorAndSafety_getInjectTestTone()

```
uint8_t rfeCfg_monitorAndSafety_getInjectTestTone (
    const uint8_t * pConfig,
    rfe_error_t * rfe__error__pointer )
```

This function gets the [injectTestTone](#) .

Parameters

in	<i>pConfig</i>	- Pointer to the Configuration Data Memory.
in, out	<i>rfe__error__pointer</i>	- Error handling parameter: On success *rfe__error__pointer == rfe_error_none_e is true, On failure *rfe__error__pointer != rfe_error_none_e is true

Returns

[rfeCfg_param_monitorAndSafety_injectTestTone_e](#).

3.11.2.70 rfeCfg_monitorAndSafety_getLoiBBD()

```
uint8_t rfeCfg_monitorAndSafety_getLoiBBD (
    const uint8_t * pConfig,
    rfe_error_t * rfe__error__pointer )
```

This function gets the [checkBbd_loi](#) .

Parameters

in	<i>pConfig</i>	- Pointer to the Configuration Data Memory.
in, out	<i>rfe__error__pointer</i>	- Error handling parameter:

Returns

Loi BBD parameters

3.11.2.71 rfeCfg_monitorAndSafety_getRxBBD()

```
uint8_t rfeCfg_monitorAndSafety_getRxBBD (
    const uint8_t * pConfig,
    rfe_error_t * rfe__error__pointer )
```

This function gets the [checkBbd_rx](#) .

Parameters

in	<i>pConfig</i>	- Pointer to the Configuration Data Memory.
in, out	<i>rfe__error__pointer</i>	- Error handling parameter:

Returns

Loi BBD parameters

3.11.2.72 rfeCfg_monitorAndSafety_getRxSatCountResetEveryChirpSequence()

```
bool rfeCfg_monitorAndSafety_getRxSatCountResetEveryChirpSequence (
    const uint8_t * pConfig,
    rfe_error_t * rfe__error__pointer )
```

This function gets the [rxSatCountResetEveryChirpSequence](#) .

Parameters

in	<i>pConfig</i>	- Pointer to the Configuration Data Memory.
in, out	<i>rfe__error__pointer</i>	- fault handling parameter: On success *rfe__error__pointer == rfe_error_none_e is true, On failure *rfe__error__pointer != rfe_error_none_e is true

Returns

[rfe_paramUpdate_monitorAndSafety_rxSatCountResetEveryChirpSequence_e](#)

3.11.2.73 rfeCfg_monitorAndSafety_getRxSaturationCountLimitStage()

```
uint16_t rfeCfg_monitorAndSafety_getRxSaturationCountLimitStage (
    const uint8_t * pConfig,
    uint8_t rxStage,
    uint8_t rxIndex,
    rfe_error_t * rfe__error__pointer )
```

This function gets the [rxSaturationCountLimit](#).

Parameters

in	<i>pConfig</i>	- Pointer to the Configuration Data Memory.
in	<i>rxStage</i>	- The Rx stage.
in	<i>rxIndex</i>	- The Rx index
in, out	<i>rfe__error__pointer</i>	- fault handling parameter: On success *rfe__error__pointer == rfe_error_none_e is true, On failure *rfe__error__pointer != rfe_error_none_e is true

Returns

rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage1_rx1_e or similar

3.11.2.74 rfeCfg_monitorAndSafety_getRxSaturationThresholdStage()

```
uint8_t rfeCfg_monitorAndSafety_getRxSaturationThresholdStage (
    const uint8_t * pConfig,
    uint8_t rxStage,
    uint8_t rxIndex,
    rfe_error_t * rfe__error__pointer )
```

This function gets the rxSaturationThreshold .

Parameters

in	pConfig	- Pointer to the Configuration Data Memory.
in	rxStage	- The Rx stage.
in	rxIndex	- The Rx index
in, out	rfe__error__pointer	- fault handling parameter: On success *rfe__error__pointer == rfe_error_none_e is true, On failure *rfe__error__pointer != rfe_error_none_e is true

Returns

rxSaturationThreshold

3.11.2.75 rfeCfg_monitorAndSafety_getTxBBD()

```
uint8_t rfeCfg_monitorAndSafety_getTxBBD (
    const uint8_t * pConfig,
    rfe_error_t * rfe__error__pointer )
```

This function gets the checkBbd_tx .

Parameters

in	pConfig	- Pointer to the Configuration Data Memory.
in, out	rfe__error__pointer	- Error handling parameter:

Returns

Loi BBD parameters

3.11.2.76 rfeCfg_monitorAndSafety_setBist()

```
void rfeCfg_monitorAndSafety_setBist (
    uint8_t * pConfig,
    rfeCfg_param_t param,
    int16_t value,
    rfe_error_t * rfe___error___pointer )
```

This function updates values, for any of the following parameters, in the in-memory blob:

- [zeroHrRefForRxPhaseDiffLna](#)
- [rxPhaseDiffLnaThresholdTolerance](#)
- [zeroHourReferenceForRxGainDiffLna](#)
- [rxGainDiffLnaThresholdTolerance](#)
- [zeroHrRefForRxPhaseDiffMixer](#)
- [rxPhaseDiffMixerThresholdTolerance](#)
- [zeroHourReferenceForRxGainDiffMixer](#)
- [rxGainDiffMixerThresholdTolerance](#)

Parameters

in	<i>pConfig</i>	- Pointer to the Configuration Data Memory.
in	<i>param</i>	- The BIST parameter to set, like rfeCfg_param_monitorAndSafety_zeroHrRefForRxPhaseDiffLna_rx1_rx2_e
in	<i>value</i>	- the new value
in, out	<i>rfe___error___pointer</i>	- Error handling parameter: On success *rfe___error___pointer == rfe_error_none_e is true, On failure *rfe___error___pointer != rfe_error_none_e is true

3.11.2.77 rfeCfg_monitorAndSafety_setFrequencyForBist()

```
void rfeCfg_monitorAndSafety_setFrequencyForBist (
    uint8_t * pConfig,
    uint32_t frequencyForBist,
    rfe_error_t * rfe___error___pointer )
```

This function sets the [frequencyForBist](#) .

Parameters

in	<i>pConfig</i>	Pointer to the Configuration Data Memory.
in	<i>frequencyForBist</i>	Configuration value for frequencyForBist
in, out	<i>rfe___error___pointer</i>	Error handling parameter.

3.11.2.78 rfeCfg_monitorAndSafety_setInjectTestTone()

```
void rfeCfg_monitorAndSafety_setInjectTestTone (
    uint8_t * pConfig,
    rfeSwBist_InjectToneMode_t injectTestToneMode,
    rfe_error_t * rfe__error__pointer )
```

This function sets the [injectTestTone](#) .

Parameters

in	<i>pConfig</i>	- Pointer to the Configuration Data Memory.
in	<i>injectTestToneMode</i>	- rfeCfg_param_monitorAndSafety_injectTestTone_e
in, out	<i>rfe__error__pointer</i>	- Error handling parameter: On success *rfe__error__pointer == rfe_error_none_e is true, On failure *rfe__error__pointer != rfe_error_none_e is true

3.11.2.79 rfeCfg_monitorAndSafety_setLoiBBD()

```
void rfeCfg_monitorAndSafety_setLoiBBD (
    const uint8_t * pConfig,
    uint8_t val,
    rfe_error_t * rfe__error__pointer )
```

This function sets the [checkBbd_loi](#) .

Parameters

in	<i>pConfig</i>	- Pointer to the Configuration Data Memory.
in	<i>val</i>	- The new value
in, out	<i>rfe__error__pointer</i>	- Error handling parameter:

3.11.2.80 rfeCfg_radarCycle_getBistInterval()

```
static uint8_t rfeCfg_radarCycle_getBistInterval (
    const uint8_t * pConfig,
    rfe_error_t * rfe__error__pointer ) [inline], [static]
```

This function gets the [bistInterval](#) .

Parameters

in	<i>pConfig</i>	- Pointer to the Configuration Data Memory.
in, out	<i>rfe__error__pointer</i>	- Error handling parameter:

Returns

[bistInterval](#)

3.11.2.81 rfeCfg_radarCycle_getChirpSequence()

```
uint8_t rfeCfg_radarCycle_getChirpSequence (
    const uint8_t * pConfig,
    uint8_t chirpSequenceIndex,
    rfe_error_t * rfe___error___pointer )
```

This function gets the sequence configuration, that is used at the selected index.

Parameters

in	<i>pConfig</i>	- Pointer to the Configuration Data Memory.
in	<i>chirpSequenceIndex</i>	- The chirp sequence index
in, out	<i>rfe___error___pointer</i>	- pointer to the error status On success *rfe___error___pointer is no_error, On failure *rfe___error___pointer is not no_error

Returns

[rfeCfg_param_radarCycle_chirpSequence_0_e](#) or similar

Remarks

Example: If the XML (blob) was generated with a Radar Cycle, that uses sequences [3,1,2], then↵
: [chirpSequenceCount](#) = 3:

- [rfeCfg_param_radarCycle_chirpSequence_0_e](#) = 3
- [rfeCfg_param_radarCycle_chirpSequence_1_e](#) = 1
- [rfeCfg_param_radarCycle_chirpSequence_2_e](#) = 2

3.11.2.82 rfeCfg_radarCycle_getChirpSequenceCount()

```
static uint8_t rfeCfg_radarCycle_getChirpSequenceCount (
    const uint8_t * pConfig,
    rfe_error_t * rfe___error___pointer ) [inline], [static]
```

Gets the [chirpSequenceCount](#). This is useful in combination with [rfeCfg_radarCycle_getChirpSequence](#), for determining how many sequence configurations are utilized in the configured Radar Cycle and in what order they are used.

Parameters

in	<i>pConfig</i>	- Pointer to the Configuration Data Memory.
in, out	<i>rfe___error___pointer</i>	- Error handling parameter: On success *rfe___error___pointer == rfe_error_none_e is true, On failure *rfe___error___pointer != rfe_error_none_e is true

Returns

[chirpSequenceCount](#)

3.11.2.83 [rfeCfg_radarCycle_getChirpSequenceStartTimeOffset\(\)](#)

```
uint32_t rfeCfg_radarCycle_getChirpSequenceStartTimeOffset (
    const uint8_t * pConfig,
    uint8_t chirpSequenceIndex,
    rfe_error_t * rfe__error__pointer )
```

This function gets the corresponding [chirpSequenceStartTimeOffset](#) .

Parameters

in	<i>pConfig</i>	- Pointer to the Configuration Data Memory.
in	<i>chirpSequenceIndex</i>	- The chirp sequence index
in, out	<i>rfe__error__pointer</i>	- Error handling parameter: On success *rfe__error__pointer == rfe_error_none_e is true, On failure *rfe__error__pointer != rfe_error_none_e is true

Returns

[rfeCfg_param_radarCycle_chirpSequenceStartTimeOffset_0_e](#) or similar

3.11.2.84 [rfeCfg_radarCycle_getRadarCycleDuration\(\)](#)

```
uint32_t rfeCfg_radarCycle_getRadarCycleDuration (
    const uint8_t * pConfig,
    rfe_error_t * rfe__error__pointer )
```

Get the current [radarCycleDuration](#).

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>rfe__error__pointer</i>	= pointer to the error variable

Returns

The existing Radar Cycle Duration

3.11.2.85 rfeCfg_radarCycle_setRadarCycleDuration()

```
void rfeCfg_radarCycle_setRadarCycleDuration (
    uint8_t * pConfig,
    uint32_t radarCycleDuration,
    rfe_error_t * rfe___error___pointer )
```

Set a new value for [radarCycleDuration](#).

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>radarCycleDuration</i>	= the new value for Radar Cycle Duration
<i>rfe___error___pointer</i>	= pointer to the error variable

3.11.2.86 rfeCfg_read()

```
uint32_t rfeCfg_read (
    const uint8_t * pConfig,
    rfeCfg_section_t section,
    rfeCfg_param_t param,
    rfe_error_t * rfe___error___pointer )
```

This function reads a value from the configuration.

Parameters

in	<i>pConfig</i>	- Pointer to the Configuration Data Memory.
in	<i>section</i>	- the required section
in	<i>param</i>	- the required parameter
in, out	<i>rfe___error___pointer</i>	- Error handling parameter: On success <code>#*rfe___error___pointer == rfe_error_none_e</code> is true, On failure <code>#*rfe___error___pointer != rfe_error_none_e</code> is true

Returns

the requested parameter in a `uint32_t` value

3.11.2.87 rfeCfg_readB()

```
uint8_t rfeCfg_readB (
    const uint8_t * pConfig,
    rfeCfg_section_t section,
    rfeCfg_param_t param,
    rfe_error_t * rfe___error___pointer )
```

This function reads a `uint8_t` value from the configuration.

Parameters

in	<i>pConfig</i>	- Pointer to the Configuration Data Memory.
in	<i>section</i>	- the required section
in	<i>param</i>	- the required parameter
in, out	<i>rfe__error__pointer</i>	- Error handling parameter: On success <code>#*rfe__error__pointer == rfe_error_none_e</code> is true, On failure <code>#*rfe__error__pointer != rfe_error_none_e</code> is true

Returns

uint8_t value

3.11.2.88 rfeCfg_readDW()

```
uint32_t rfeCfg_readDW (
    const uint8_t * pConfig,
    rfeCfg_section_t section,
    rfeCfg_param_t param,
    rfe_error_t * rfe__error__pointer )
```

This function reads a uint32_t value from the configuration.

Parameters

in	<i>pConfig</i>	- Pointer to the Configuration Data Memory.
in	<i>section</i>	- the required section
in	<i>param</i>	- the required parameter
in, out	<i>rfe__error__pointer</i>	- Error handling parameter: On success <code>#*rfe__error__pointer == rfe_error_none_e</code> is true, On failure <code>#*rfe__error__pointer != rfe_error_none_e</code> is true

Returns

uint32_t value

3.11.2.89 rfeCfg_readW()

```
uint16_t rfeCfg_readW (
    const uint8_t * pConfig,
    rfeCfg_section_t section,
    rfeCfg_param_t param,
    rfe_error_t * rfe__error__pointer )
```

This function reads a uint16_t value from the configuration.

Parameters

in	<i>pConfig</i>	- Pointer to the Configuration Data Memory.
in	<i>section</i>	- the required section
in	<i>param</i>	- the required parameter
in, out	<i>rfe__error__pointer</i>	- Error handling parameter: On success <code>#*rfe__error__pointer == rfe_error_none_e</code> is true, On failure <code>#*rfe__error__pointer != rfe_error_none_e</code> is true

Returns

uint16_t value

3.11.2.90 rfeCfg_sequence_getFinalPcGenMode()

```
uint8_t rfeCfg_sequence_getFinalPcGenMode (
    const uint8_t * pConfig,
    rfe_chirpSequenceIndex_t chirpSequenceConfigIndex,
    rfe_txIndex_t tx,
    rfe_error_t * rfe__error__pointer )
```

3.11.2.91 rfeCfg_sequence_getPhaseRotationDdmaMode()

```
uint8_t rfeCfg_sequence_getPhaseRotationDdmaMode (
    const uint8_t * pConfig,
    rfe_chirpSequenceIndex_t chirpSequenceConfigIndex,
    rfe_error_t * rfe__error__pointer )
```

3.11.2.92 rfeCfg_sequence_getPhaseRotationTxUseDdma()

```
uint8_t rfeCfg_sequence_getPhaseRotationTxUseDdma (
    const uint8_t * pConfig,
    rfe_chirpSequenceIndex_t chirpSequenceConfigIndex,
    rfe_error_t * rfe__error__pointer )
```

3.11.2.93 rfeCfg_sequence_getTxCalibrationThresholdHigh()

```
uint16_t rfeCfg_sequence_getTxCalibrationThresholdHigh (
    const uint8_t * pConfig,
    rfe_chirpSequenceIndex_t chirpSequenceConfigIndex,
    rfe_error_t * rfe__error__pointer )
```

Get the [txCalibration_thresholdHigh](#) for the selected sequence.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpSequenceConfigIndex</i>	= the Sequence index
<i>rfe__error__pointer</i>	= pointer to the error variable

3.11.2.94 rfeCfg_sequence_getTxCalibrationThresholdLow()

```
uint16_t rfeCfg_sequence_getTxCalibrationThresholdLow (
    const uint8_t * pConfig,
    rfe_chirpSequenceIndex_t chirpSequenceConfigIndex,
    rfe_error_t * rfe__error__pointer )
```

Get the [txCalibration_thresholdLow](#) for the selected sequence.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpSequenceConfigIndex</i>	= the Sequence index
<i>rfe__error__pointer</i>	= pointer to the error variable

3.11.2.95 rfeCfg_sequence_getTxDdmaInitPhase()

```
uint8_t rfeCfg_sequence_getTxDdmaInitPhase (
    const uint8_t * pConfig,
    rfe_chirpSequenceIndex_t chirpSequenceConfigIndex,
    rfe_txIndex_t tx,
    rfe_error_t * rfe__error__pointer )
```

3.11.2.96 rfeCfg_sequence_getTxDdmaUpdatePhase()

```
uint8_t rfeCfg_sequence_getTxDdmaUpdatePhase (
    const uint8_t * pConfig,
    rfe_chirpSequenceIndex_t chirpSequenceConfigIndex,
    rfe_txIndex_t tx,
    rfe_error_t * rfe__error__pointer )
```

3.11.2.97 rfeCfg_sequence_setPhaseRotationDdmaMode()

```
void rfeCfg_sequence_setPhaseRotationDdmaMode (
    uint8_t * pConfig,
    rfe_chirpSequenceIndex_t chirpSequenceConfigIndex,
    uint8_t enable,
    rfe_error_t * rfe___error___pointer )
```

Set [txPhaseRotation_ddma](#) for the selected sequence.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpSequenceConfigIndex</i>	= the Sequence index
<i>enable</i>	= enable/disable
<i>rfe__error__pointer</i>	= pointer to the error variable

3.11.2.98 rfeCfg_sequence_setTxDdmaInitPhase()

```
void rfeCfg_sequence_setTxDdmaInitPhase (
    uint8_t * pConfig,
    rfe_chirpSequenceIndex_t chirpSequenceConfigIndex,
    rfe_txIndex_t tx,
    rfe_phaseRotation_t txDdmaInitPhase,
    rfe_error_t * rfe__error__pointer )
```

Set a new value for [txPhaseRotation_ddma_tx1_init](#) for the selected sequence.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpSequenceConfigIndex</i>	= the Sequence index
<i>tx</i>	= the tx index
<i>txDdmaInitPhase</i>	= the new value for Tx Ddma_init_phase
<i>rfe__error__pointer</i>	= pointer to the error variable

3.11.2.99 rfeCfg_sequence_setTxDdmaUpdatePhase()

```
void rfeCfg_sequence_setTxDdmaUpdatePhase (
    uint8_t * pConfig,
    rfe_chirpSequenceIndex_t chirpSequenceConfigIndex,
    rfe_txIndex_t tx,
    rfe_phaseRotation_t txDdmaUpdatePhase,
    rfe_error_t * rfe__error__pointer )
```

Set a new value for [txPhaseRotation_ddma_tx1_update](#) for the selected sequence.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpSequenceConfigIndex</i>	= the Sequence index
<i>tx</i>	= the tx index
<i>txDdmaUpdatePhase</i>	= the new value for Tx Ddma_update_phase
<i>rfe__error__pointer</i>	= pointer to the error variable

3.11.2.100 rfeCfg_sequence_setTxFinalPcGenMode()

```
void rfeCfg_sequence_setTxFinalPcGenMode (
    uint8_t * pConfig,
    rfe_chirpSequenceIndex_t chirpSequenceConfigIndex,
    rfe_txIndex_t tx,
    rfe_PRPCGenMode_e txFinalPcGenMode,
    rfe_error_t * rfe__error__pointer )
```

Set a new value for [txPhaseRotation_ddma_tx1_mode](#) for the selected sequence.

Parameters

<i>pConfig</i>	= pointer to the blob configuration
<i>chirpSequenceConfigIndex</i>	= the Sequence index
<i>tx</i>	= the tx index
<i>txFinalPcGenMode</i>	= the new value for Tx final pc gen mode
<i>rfe__error__pointer</i>	= pointer to the error variable

3.11.2.101 rfeCfg_util_chirpProfileIndex()

```
rfeCfg_section_t rfeCfg_util_chirpProfileIndex (
    rfe_chirpProfileIndex_t chirpProfileIndex,
    rfe_error_t * rfe__error__pointer )
```

Writes a configuration item.

Parameters

in	<i>chirpProfileIndex</i>	The chirp profile index.
in, out	<i>rfe__error__pointer</i>	Error handling parameter:

Returns

The corresponding section.

3.11.2.102 rfeCfg_util_chirpProfileSequence()

```
rfeCfg_param_t rfeCfg_util_chirpProfileSequence (
    uint8_t index,
    rfe_error_t * rfe__error__pointer )
```

Maps chirp profile sequence index to the corresponding param.

Parameters

in	<i>index</i>	The chirp profile sequence index.
in, out	<i>rfe__error__pointer</i>	Error handling parameter:

Returns

The corresponding param.

3.11.2.103 rfeCfg_util_chirpSequenceConfigIndex()

```
rfeCfg_section_t rfeCfg_util_chirpSequenceConfigIndex (
    rfe_chirpSequenceIndex_t chirpSequenceIndex,
    rfe_error_t * rfe__error__pointer )
```

Maps chirp sequence config index to the corresponding section.

Parameters

in	<i>chirpSequenceIndex</i>	- The chirp sequence config index.
in, out	<i>rfe__error__pointer</i>	- pointer to the error status

Returns

The corresponding section.

3.11.2.104 rfeCfg_util_radarCycleChirpSequence()

```
rfeCfg_param_t rfeCfg_util_radarCycleChirpSequence (
    uint8_t index,
    rfe_error_t * rfe__error__pointer )
```

Maps radar cycle chirp sequence index to the corresponding param.

Parameters

in	<i>index</i>	The radar cycle chirp sequence index.
in, out	<i>rfe__error__pointer</i>	Error handling parameter:

Returns

The corresponding param.

3.11.2.105 rfeCfg_write()

```
void rfeCfg_write (
    uint8_t * pConfig,
    rfeCfg_section_t section,
    rfeCfg_param_t param,
    uint32_t value,
    rfe_error_t * rfe___error___pointer )
```

This function writes a value to the configuration blob, according to the provided parameters. The provided value must be a uint32_t value, but the written value is according to the size of the parameter definition (see [RFE Abstract 2.0 BLOB - Constants](#)).

Parameters

in	<i>pConfig</i>	- Pointer to the Configuration Data Memory
in	<i>section</i>	- the required section
in	<i>param</i>	- the required parameter
in	<i>value</i>	- the value to be written - aligned to 32 bits
in, out	<i>rfe___error___pointer</i>	- Error handling parameter: On success *rfe___error___pointer == rfe_error_none_e is true, On failure *rfe___error___pointer != rfe_error_none_e is true

3.12 RFE Abstract 2.0 - IPC Interface

Defines the external function API required by RFE Abstract 2.0 for driver and firmware process communication.

Functions

- void * [rfelpc_srvRspBuffAcquire](#) (void)
Acquire command response buffer.
- void * [rfelpc_srvStsBuffAcquire](#) (void)
Acquire status buffer.
- int32_t [rfelpc_srvStsTx](#) (void *msg, uint32_t msgLen)
Transmit status API.
- int32_t [rfelpc_srvRspTx](#) (void *msg, uint32_t msgLen)
Transmit response API.
- void * [rfelpc_srvCmdPoll](#) (int32_t *err)
Poll Commands channel API.
- void * [rfelpc_clientCmdBuffAcquire](#) (void)
Acquire cmd buffer.
- int32_t [rfelpc_clientCmdTx](#) (void *msg, uint32_t msgLen)
Transmit command API.
- void * [rfelpc_clientStsPoll](#) (int32_t *err)
Poll State channel API.
- void * [rfelpc_clientStsWait](#) (int32_t *err)
Blocking poll on state channel API.
- void * [rfelpc_clientRspWait](#) (int32_t *err)
Blocking poll on response channel API.

3.12.1 Detailed Description

Defines the external function API required by RFE Abstract 2.0 for driver and firmware process communication.

3.12.2 Function Documentation

3.12.2.1 rfelpc_clientCmdBuffAquire()

```
void * rfeIpc_clientCmdBuffAquire (
    void )
```

Acquire cmd buffer.

The function must provide message container used by the transport protocol to communicate between customer application and RFE-Fw Used for initial initialization synchronization.

Returns

*void - pointer to a transport buffer

3.12.2.2 rfelpc_clientCmdTx()

```
int32_t rfeIpc_clientCmdTx (
    void * msg,
    uint32_t msgLen )
```

Transmit command API.

The function must transmit a command messages between Driver and RFE-Fw

Parameters

in	<i>msg</i>	- pointer to message that needs to be transmitted
in	<i>msgLen</i>	- message length

Returns

int32_t - status code indicating success (0) or failure (!=0)

3.12.2.3 rfelpc_clientRspWait()

```
void * rfeIpc_clientRspWait (
    int32_t * err )
```

Blocking poll on response channel API.

The function must poll in blocking manner on response channel. Used to get the command acknowledge from RFE-Fw side

Parameters

out	*err	- status code indicating success (0) or failure (!=0)
-----	------	---

Returns

*void – pointer to the receive message

3.12.2.4 rfelpc_clientStsPoll()

```
void * rfeIpc_clientStsPoll (
    int32_t * err )
```

Poll State channel API.

The function must poll state channel used to update RFE-Fw state for the driver running on application side This must be a nonblocking function

Returns

*void - pointer to a transport buffer

3.12.2.5 rfelpc_clientStsWait()

```
void * rfeIpc_clientStsWait (
    int32_t * err )
```

Blocking poll on state channel API.

The function must poll in blocking manner on state channel. Used to synchronize application and RFE-Fw

Parameters

out	*err	- status code indicating success (0) or failure (!=0)
-----	------	---

Returns

*void – pointer to the receive message

3.12.2.6 rfeIpc_srvCmdPoll()

```
void * rfeIpc_srvCmdPoll (
    int32_t * err )
```

Poll Commands channel API.

The function must poll the command channel used by RFE-Fw to receive commands from Driver This must be a nonblocking function

Parameters

out	*err	- status code indicating success (0) or failure (!=0)
-----	------	---

Returns

*void - pointer to the received buffer

3.12.2.7 rfeIpc_srvRspBuffAcquire()

```
void * rfeIpc_srvRspBuffAcquire (
    void )
```

Acquire command response buffer.

The function must provide message container used by the transport protocol to communicate between RFE-Fw and Driver. Used for initial initialization synchronization.

Returns

*void - pointer to a transport buffer

3.12.2.8 rfeIpc_srvRspTx()

```
int32_t rfeIpc_srvRspTx (
    void * msg,
    uint32_t msgLen )
```

Transmit response API.

The function must transmit a command response messages between RFE-Fw and Driver

Parameters

in	msg	- pointer to message that needs to be transmitted
in	msgLen	- message length

Returns

int32_t - status code indicating success (0) or failure (!=0)

3.12.2.9 rfeIpc_srvStsBuffAcquire()

```
void * rfeIpc_srvStsBuffAcquire (
    void )
```

Acquire status buffer.

The function must provide message container used by the transport protocol to communicate between RFE-Fw and Driver. Used for initial initialization synchronization.

Returns

*void - pointer to a transport buffer

3.12.2.10 rfeIpc_srvStsTx()

```
int32_t rfeIpc_srvStsTx (
    void * msg,
    uint32_t msgLen )
```

Transmit status API.

The function is used transmit a status update messages between RFE-Fw and Driver. It is used in an asynchronous mode

Parameters

in	<i>msg</i>	- pointer to message that needs to be transmitted
in	<i>msgLen</i>	- message length

Returns

int32_t - status code indicating success (0) or failure (!=0)

3.13 RFE Abstract 2.0 - Peripheral Access Interface

Defines the external function API required by RFE Abstract 2.0 to access different peripherals:

Modules

- [RFE Abstract 2.0 - GPIO Interface](#)
Defines the external function API required by RFE Abstract 2.0 for GPIO.
- [RFE Abstract 2.0 - SPI Interface](#)
Defines the external function API required by RFE Abstract 2.0 for SPI.
- [RFE Abstract 2.0 - Timers Interface](#)
Defines the external function API required by RFE Abstract 2.0 for Timers.
- [RFE Abstract 2.0 - Interrupt Interface](#)
Defines the external function API required by RFE Abstract 2.0 for interrupts.
- [RFE Abstract 2.0 - Debug Interface](#)
Defines the external function API required by RFE Abstract 2.0 for debug.

3.13.1 Detailed Description

Defines the external function API required by RFE Abstract 2.0 to access different peripherals:

- GPIO
- SPI
- Timers
- Debug(uart)

3.14 RFE Abstract 2.0 - GPIO Interface

Defines the external function API required by RFE Abstract 2.0 for GPIO.

Enumerations

- enum [TEF82XX_ExtPinID_e](#) {
[e_CHIRPSTART_IN_PAD](#) = 0x01u , [e_MCUINT_CHIRPSTART_OUT_PAD](#) = 0x02u , [e_ERRORN](#) = 0x03u ,
[e_ERRORRESET](#) = 0x04 ,
[e_TX1_I_PS](#) = 0x05u , [e_TX2_I_PS](#) = 0x06u , [e_TX3_I_PS](#) = 0x07u , [e_TX1_Q_PS](#) = 0x08u ,
[e_TX2_Q_PS](#) = 0x09u , [e_TX3_Q_PS](#) = 0x0Au , [e_HRESET](#) = 0x0Fu , [e_ALL_BC_PINS](#) = 0x10u ,
[e_PINID_INVALID](#) }

Functions

- void [RfeHwGpioInit](#) (void)
- void [RfeHwGpioSet](#) (uint32_t frontendId, [TEF82XX_ExtPinID_e](#) HostPinID, uint8_t pinState)
Set the status for an output GPIO pin.
- void [RfeHwGpioGet](#) (uint32_t frontendId, [TEF82XX_ExtPinID_e](#) HostPinID, uint8_t *pinState)
Get the status for an output GPIO pin.

3.14.1 Detailed Description

Defines the external function API required by RFE Abstract 2.0 for GPIO.

3.14.2 Enumeration Type Documentation

3.14.2.1 TEF82XX_ExtPinID_e

enum `TEF82XX_ExtPinID_e`

External Pin Id of the TEF82XX.

Enumerator

<code>e_CHIRPSTART_IN_PAD</code>	The CHIRP_START pin provides precise timing of the actual chirp. The precise timing information is available in two output waveforms, which is chosen by <code>sel_mode_for_chirp_start_out</code> bitfield.
<code>e_MCUINT_CHIRPSTART_OUT_PAD</code>	This pin can be configured as output pad with different functionalities like MCU_INT, Chirp start trigger or Chirp signal indicating busy/silent period of the chirp based on the configuration.
<code>e_ERRORN</code>	ERROR_N pin is a generic error interrupt which indicates the MCU that there has a safety-related error has occurred.
<code>e_ERRORRESET</code>	When a safety sensor is enabled and configured, then any error reported is latched within the module. The latched error is also propagated to inner safety monitor. The latched error is only reset by asserting the <code>ERROR_RESET</code> or via SPI.
<code>e_TX1_I_PS</code>	When the QPSK I/Os controls is set to asynchronous mode, the value of the TXn_PS_I and TXn_PS_Q I/Os for each TX is sampled at a moment in time defined.
<code>e_TX2_I_PS</code>	
<code>e_TX3_I_PS</code>	
<code>e_TX1_Q_PS</code>	
<code>e_TX2_Q_PS</code>	
<code>e_TX3_Q_PS</code>	
<code>e_HRESET</code>	HRSET_N resets all modules to their default state and register values, except the OTP module. The OTP module is insensitive to <code>HRESET_N</code> . Application need to set this pin high.
<code>e_ALL_BC_PINS</code>	Option to select all external pins of TEF82XX.
<code>e_PINID_INVALID</code>	

3.14.3 Function Documentation

3.14.3.1 RfeHwGpioGet()

```
void RfeHwGpioGet (
    uint32_t frontendId,
    TEF82XX_ExtPinID_e HostPinID,
    uint8_t * pinState )
```

Get the status for an output GPIO pin.

The function must do all necessary steps to correctly update the GPIO status :

- GPIO enable
- GPIO set status

Note

The function call use only the symbolic name of the outputs from SYS_ExtPinID_e

Parameters

in	<i>frontend↔ Id</i>	- single, leader or follower
in	<i>HostPinID</i>	- generic name for GPIO from TEF82XX_ExtPinID_e
out	<i>pinState</i>	- the state to be set, !=0 => high, 0 => low

3.14.3.2 RfeHwGpioInit()

```
void RfeHwGpioInit (  
    void )
```

3.14.3.3 RfeHwGpioSet()

```
void RfeHwGpioSet (  
    uint32_t frontendId,  
    TEF82XX_ExtPinID_e HostPinID,  
    uint8_t pinState )
```

Set the status for an output GPIO pin.

The function must do all necessary steps to correctly update the GPIO status :

- GPIO enable
- GPIO set status

Note

The function call use only the symbolic name of the outputs from SYS_ExtPinID_e

Parameters

in	<i>frontend↔ Id</i>	- single, leader or follower
in	<i>HostPinID</i>	- generic name for GPIO from TEF82XX_ExtPinID_e
in	<i>pinState</i>	- the state to be set, !=0 => high, 0 => low

3.15 RFE Abstract 2.0 - SPI Interface

Defines the external function API required by RFE Abstract 2.0 for SPI.

Functions

- void [RfeHwSpiInitialize](#) (uint32_t frontEndId, [rfe_error_t](#) *rfe___error___pointer)
RFE function that initializes the SPI interface.
- uint32_t [RfeHwSpiTransfer](#) (uint32_t frontEndId, uint8_t *pTxData, uint8_t *pRxData, uint16_t length)
RFE uses to communicate with the frontend using SPI.

3.15.1 Detailed Description

Defines the external function API required by RFE Abstract 2.0 for SPI.

3.15.2 Function Documentation

3.15.2.1 RfeHwSpiInitialize()

```
void RfeHwSpiInitialize (
    uint32_t frontEndId,
    rfe\_error\_t * rfe___error___pointer )
```

RFE function that initializes the SPI interface.

This function will be called to initialize the SPI interface. The SPI peripheral can't be shared by multiple external devices

Parameters

in	<i>frontEndId</i>	- Unique identifier associated with SPI used to communicate with the frontend.
in, out	<i>rfe___error___pointer</i>	Error parameter.

3.15.2.2 RfeHwSpiTransfer()

```
uint32_t RfeHwSpiTransfer (
    uint32_t frontEndId,
    uint8_t * pTxData,
    uint8_t * pRxData,
    uint16_t length )
```

RFE uses to communicate with the frontend using SPI.

This function will be called to communicate with the frontend using SPI transfers.

Parameters

in	<i>front↔ EndId</i>	- Unique identifier associated with SPI used to communicate with the frontend.
in	<i>pTxData</i>	- pointer to the data to be transfered
out	<i>pRxData</i>	- pointer to the received data
in	<i>length</i>	- the length of the transfer in bytes

Returns

uint32_t - status code indicating success (0) or failure (!=0)

Precondition

RfeHwSpilnit() must be invoked prior to communicating over SPI

3.16 RFE Abstract 2.0 - Timers Interface

Defines the external function API required by RFE Abstract 2.0 for Timers.

Functions

- void [RfeHwTimersInit](#) ([rfe_error_t](#) *rfe___error___pointer)
Init timers.
- uint32_t [RfeHwHeartBeatTimerGetValue](#) ([rfe_error_t](#) *rfe___error___pointer)
Returns heart beat value that represents 25ns system ticks.
- void [RfeHwDelayUs](#) (uint32_t us)
Application provided function to delay execution with us microseconds.

3.16.1 Detailed Description

Defines the external function API required by RFE Abstract 2.0 for Timers.

3.16.2 Function Documentation

3.16.2.1 RfeHwDelayUs()

```
void RfeHwDelayUs (
    uint32_t us )
```

Application provided function to delay execution with us microseconds.

Parameters

in	us	delay time in microseconds
----	----	----------------------------

Precondition

Invoke [RfeHwTimersInit\(\)](#) as initialization code, prior to calling this function for the first time

3.16.2.2 RfeHwHeartBeatTimerGetValue()

```
uint32_t RfeHwHeartBeatTimerGetValue (
    rfe_error_t * rfe__error__pointer )
```

Returns heart beat value that represents 25ns system ticks.

Parameters

in, out	rfe__error__pointer	Error parameter.
---------	---------------------	------------------

Returns

- value Timer value in 25ns units.

3.16.2.3 RfeHwTimersInit()

```
void RfeHwTimersInit (
    rfe_error_t * rfe__error__pointer )
```

Init timers.

Init all hw timers for RFE FSM good functionality: 25ns tick timer and others.

Parameters

in, out	rfe__error__pointer	Error parameter.
---------	---------------------	------------------

3.17 RFE Abstract 2.0 - Interrupt Interface

Defines the external function API required by RFE Abstract 2.0 for interrupts.

Functions

- void [RfeHwSetIRQ](#) (void)
- void [RfeHwRaiseIRQ](#) (uint32_t apilrq, uint32_t eventlrq)
Raise IRQ to specified core.

3.17.1 Detailed Description

Defines the external function API required by RFE Abstract 2.0 for interrupts.

3.17.2 Function Documentation

3.17.2.1 RfeHwRaiseIRQ()

```
void RfeHwRaiseIRQ (
    uint32_t apilrq,
    uint32_t eventlrq )
```

Raise IRQ to specified core.

Call any necessary low level or IPCF routine in order to signal an event or command completion

Note

Parameters

in	<i>apilrq</i>	- command id or 0xFFFFFFFF if event
in	<i>eventlrq</i>	- event id or 0xFF if command

3.17.2.2 RfeHwSetIRQ()

```
void RfeHwSetIRQ (
    void )
```

3.18 RFE Abstract 2.0 - Debug Interface

Defines the external function API required by RFE Abstract 2.0 for debug.

Functions

- void [RfeInitDebug](#) ([rfe_error_t](#) *rfe___error___pointer)
Initialize debug mechanism for RFE.
- void [RfeDbgPrintMsg](#) (const char *fmt,...)
Perform simple printf()-like functionality over UART console.

3.18.1 Detailed Description

Defines the external function API required by RFE Abstract 2.0 for debug.

3.18.2 Function Documentation

3.18.2.1 RfeDbgPrintMsg()

```
void RfeDbgPrintMsg (  
    const char * fmt,  
    ... )
```

Perform simple printf()-like functionality over UART console.

Precondition

[RfeInitDebug\(\)](#) must be called before using this function. The code must be built with a valid combination of preprocessor macros

Parameters

in	<i>fmt</i>	- String to be output
----	------------	-----------------------

3.18.2.2 RfeInitDebug()

```
void RfeInitDebug (  
    rfe\_error\_t * rfe___error___pointer )
```

Initialize debug mechanism for RFE.

The function must do all necessary steps to correctly setup the debug UART console

Parameters

in, out	<i>rfe___error___pointer</i>	Error parameter.
---------	------------------------------	------------------

3.19 RFE Abstract 2.0 - Firmware Constants

Defines the constants used to initialize the firmware finite state machine(FSM)

Macros

- `#define RFE_SW_MAIN_FSM_MAIN_STATE_COUNT ( 9UL )`

3.19.1 Detailed Description

Defines the constants used to initialize the firmware finite state machine(FSM)

3.19.2 Macro Definition Documentation

3.19.2.1 RFE_SW_MAIN_FSM_MAIN_STATE_COUNT

```
#define RFE_SW_MAIN_FSM_MAIN_STATE_COUNT ( 9UL )
```

This macro defines the number of main states.

3.20 RFE Abstract 2.0 - Firmware Interface

RFE Abstract 2.0 firmware finite state machine(FSM) specific structures and functions.

Enumerations

- enum `rfeSwMainFsm_mainState_t` {
`rfeSwMainFsm_mainState_initializing_e` = 0U , `rfeSwMainFsm_mainState_waitingForSync_e` = 1U ,
`rfeSwMainFsm_mainState_initialized_e` = 2U , `rfeSwMainFsm_mainState_configuring_e` = 3U ,
`rfeSwMainFsm_mainState_configured_e` = 4U , `rfeSwMainFsm_mainState_startRadarCycles_e` = 5U ,
`rfeSwMainFsm_mainState_radarCycles_e` = 6U , `rfeSwMainFsm_mainState_testContinuousWaveTransmission_e`
= 7U ,
`rfeSwMainFsm_mainState_fuSaFault_e` = 8U }
- enum `rfeFrontEnd_Configuration_t` { `rfeFrontEnd_Single_e` = 0U , `rfeFrontEnd_Cascaded_e` = 1U }

Functions

- `rfeSwMainFsm_mainState_t` `rfeMainFsm` (`rfeSwMainFsm_mainState_t` state, `rfe_error_t` *`rfe___error___`↔ pointer)
This function performs the main finite-state-machine.
- void `rfeSetFrontendsConfiguration` (`rfeFrontEnd_Configuration_t` feConfigType, uint8_t numConnected↔ Frontends, `rfe_error_t` *`rfe___error___` pointer)
This function set the number of connected frontends and the configuration used.
- void `rfeSetSharedAddresses` (uint32_t rfeRfBistBufferAddressParam)
Set the shared addresses for the cores intercommunication.

3.20.1 Detailed Description

RFE Abstract 2.0 firmware finite state machine(FSM) specific structures and functions.

3.20.2 Enumeration Type Documentation

3.20.2.1 rfeFrontEnd_Configuration_t

```
enum rfeFrontEnd_Configuration_t
```

This enumerated type defines the possible TEF82xx configurations.

Enumerator

rfeFrontEnd_Single_e	
rfeFrontEnd_Cascaded↔ _e	

3.20.2.2 rfeSwMainFsm_mainState_t

```
enum rfeSwMainFsm_mainState_t
```

This enumerated type defines the main states.

Enumerator

rfeSwMainFsm_mainState_initializing_e	
rfeSwMainFsm_mainState_waitingForSync_e	
rfeSwMainFsm_mainState_initialized_e	
rfeSwMainFsm_mainState_configuring_e	
rfeSwMainFsm_mainState_configured_e	
rfeSwMainFsm_mainState_startRadarCycles_e	
rfeSwMainFsm_mainState_radarCycles_e	
rfeSwMainFsm_mainState_testContinuousWaveTransmission↔ _e	
rfeSwMainFsm_mainState_fuSaFault_e	

3.20.3 Function Documentation

3.20.3.1 rfeMainFsm()

```
rfeSwMainFsm_mainState_t rfeMainFsm (
    rfeSwMainFsm_mainState_t state,
    rfe_error_t * rfe__error__pointer )
```

This function performs the main finite-state-machine.

This function shall be called by the RFE abstraction 2 main app. This function will perform all needed function on the TEF82xx frontend.

Precondition

Call rfeSetSharedAdresses for shared memory addresses settings. Call rfeSetFrontendsConfiguration or default will be used rfeFrontEnd_Cascaded_e with RFE_MAX_SUPPORTED_DEVICES

Parameters

in	state	- Start state or the state at which to continue.
in, out	rfe__error__pointer	- Error handling parameter.

Returns

rfeSwMainFsm_mainState_t - The next state. This return value can be used as argument to the next invocation of this function. In this way we can step through the state machine, state by state.

Postcondition

NIL

3.20.3.2 rfeSetFrontendsConfiguration()

```
void rfeSetFrontendsConfiguration (
    rfeFrontEnd_Configuration_t feConfigType,
    uint8_t numConnectedFrontends,
    rfe_error_t * rfe__error__pointer )
```

This function set the number of connected frontends and the configuration used.

This function must be called before the start of the main fsm, before the TEF82xx init. Default configuration is rfeFrontEnd_Cascaded_e with RFE_MAX_SUPPORTED_DEVICES. Note that single mode on multiple TEF82xx is not supported e.g. rfeFrontEnd_Single_e with NUM_FRONTENDS_MAX

Precondition

NIL

Parameters

in	<i>feConfigType</i>	- Frontend configuration type.
in	<i>numConnectedFrrontends</i>	- Number of frontends that are connected to the SOC.
in, out	<i>rfe___error___pointer</i>	- Error handling parameter.

Postcondition

NIL

3.20.3.3 rfeSetSharedAddresses()

```
void rfeSetSharedAddresses (
    uint32_t rfeRfBistBufferAddressParam )
```

Set the shared addresses for the cores intercommunication.

Precondition

NIL

Parameters

in	<i>rfeRfBistBufferAddressParam</i>	- the location for the MIPI CSI2 data buffers.
----	------------------------------------	--

Chapter 4

Data Structure Documentation

4.1 ATB_ADCVoltages_t Struct Reference

```
#include <rfe_tef82xx_voltage.h>
```

Data Fields

- uint16_t [ADC_NODE_1V1_SAFETY_SUPPLY](#)
- uint16_t [ADC_NODE_1V1_SUPPLY](#)
- uint16_t [ADC_NODE_1V8_SUPPLY](#)

4.1.1 Detailed Description

Structure for ADC voltages

4.1.2 Field Documentation

4.1.2.1 ADC_NODE_1V1_SAFETY_SUPPLY

```
uint16_t ADC_NODE_1V1_SAFETY_SUPPLY
```

ball N3 VDDD_SAFETY_1V1

4.1.2.2 ADC_NODE_1V1_SUPPLY

```
uint16_t ADC_NODE_1V1_SUPPLY
```

internally generated

4.1.2.3 ADC_NODE_1V8_SUPPLY

```
uint16_t ADC_NODE_1V8_SUPPLY
```

ball ADC12=C12 VDDA_ADC12_1V8, ADC34=N12 VDDA_ADC34_1V8

4.2 ATB_ATBVoltages_t Struct Reference

```
#include <rfe_tef82xx_voltage.h>
```

Data Fields

- uint16_t [ATB_NODE_VDDD_SAFETY_1V1](#)
- uint16_t [ATB_NODE_VDDA_1V1](#)
- uint16_t [ATB_NODE_VDDA_1V8](#)

4.2.1 Detailed Description

Structure for ATB voltages

4.2.2 Field Documentation

4.2.2.1 ATB_NODE_VDDA_1V1

```
uint16_t ATB_NODE_VDDA_1V1
```

internally generated

4.2.2.2 ATB_NODE_VDDA_1V8

```
uint16_t ATB_NODE_VDDA_1V8
```

ball C12 VDDA_ADC12_1V8

4.2.2.3 ATB_NODE_VDDD_SAFETY_1V1

```
uint16_t ATB_NODE_VDDD_SAFETY_1V1
```

ball N3 VDDD_SAFETY_1V1

4.3 ATB_CAFCVoltages_t Struct Reference

```
#include <rfe_tef82xx_voltage.h>
```

Data Fields

- uint16_t [CAFC_NODE_VDD_PLL_1V1_FILTERED](#)
- uint16_t [CAFC_NODE_LDO_CPPFD_VDD](#)
- uint16_t [CAFC_NODE_LDO_DIVIDER_VDD](#)
- uint16_t [CAFC_NODE_LDO_VCO_VDD](#)
- uint16_t [CAFC_NODE_VDD_PLL_1V8](#)
- uint16_t [CAFC_NODE_VDD_VCO_1V8](#)

4.3.1 Detailed Description

Structure for CAFC voltages

4.3.2 Field Documentation

4.3.2.1 CAFC_NODE_LDO_CPPFD_VDD

```
uint16_t CAFC_NODE_LDO_CPPFD_VDD
```

internally generated

4.3.2.2 CAFC_NODE_LDO_DIVIDER_VDD

```
uint16_t CAFC_NODE_LDO_DIVIDER_VDD
```

internally generated

4.3.2.3 CAFC_NODE_LDO_VCO_VDD

```
uint16_t CAFC_NODE_LDO_VCO_VDD
```

internally generated

4.3.2.4 CAFC_NODE_VDD_PLL_1V1_FILTERED

```
uint16_t CAFC_NODE_VDD_PLL_1V1_FILTERED
```

ball N3 filtered VDDD_SAFETY_1V1

4.3.2.5 CAFC_NODE_VDD_PLL_1V8

```
uint16_t CAFC_NODE_VDD_PLL_1V8
```

ball B8 VDDA_PLL_26G_1V8

4.3.2.6 CAFC_NODE_VDD_VCO_1V8

```
uint16_t CAFC_NODE_VDD_VCO_1V8
```

ball A7 VDDA_VCO_26G_1V8

4.4 ATB_GBIASt Voltages_t Struct Reference

```
#include <rfe_tef82xx_voltage.h>
```

Data Fields

- `uint16_t` [GBIAS_NODE_VDDA_GLOBAL_BIAS_1V8](#)

4.4.1 Detailed Description

Structure for GBIAS voltages

4.4.2 Field Documentation

4.4.2.1 GBIAS_NODE_VDDA_GLOBAL_BIAS_1V8

```
uint16_t GBIAS_NODE_VDDA_GLOBAL_BIAS_1V8
```

ball C6 & B6 & B7 VDDA_BIAS_1V8

4.5 ATB_GLDOVoltages_t Struct Reference

```
#include <rfe_tef82xx_voltage.h>
```

Data Fields

- uint16_t [GLDO_NODE_VDDD_SAFETY_1V1](#)
- uint16_t [GLDO_NODE_VDDA_LDO_1V1](#)
- uint16_t [GLDO_NODE_VDDA_LDO_1V8](#)
- uint16_t [GLDO_NODE_VDDA_LDO_3V3](#)

4.5.1 Detailed Description

Structure for GLDOV voltages

4.5.2 Field Documentation

4.5.2.1 GLDO_NODE_VDDA_LDO_1V1

uint16_t GLDO_NODE_VDDA_LDO_1V1

ball B4 VDDA_LDO_1V1

4.5.2.2 GLDO_NODE_VDDA_LDO_1V8

uint16_t GLDO_NODE_VDDA_LDO_1V8

ball C6 & B6 & B7 VDDA_BIAS_1V8

4.5.2.3 GLDO_NODE_VDDA_LDO_3V3

uint16_t GLDO_NODE_VDDA_LDO_3V3

ball A1 & B2 VDDA_LDO_3V3

4.5.2.4 GLDO_NODE_VDDD_SAFETY_1V1

uint16_t GLDO_NODE_VDDD_SAFETY_1V1

ball N3 VDDD_SAFETY_1V1

4.6 ATB_LOIVoltages_t Struct Reference

```
#include <rfe_tef82xx_voltage.h>
```

Data Fields

- uint16_t [LOI_NODE_VDDD_SAFETY_1V1](#)
- uint16_t [LOI_NODE_VDDA_INTERFACE_1V1](#)
- uint16_t [LOI_NODE_VDDA_1V8](#)

4.6.1 Detailed Description

Structure for LOI voltages

4.6.2 Field Documentation

4.6.2.1 LOI_NODE_VDDA_1V8

```
uint16_t LOI_NODE_VDDA_1V8
```

ball C6 & B6 & B7 VDDA_BIAS_1V8

4.6.2.2 LOI_NODE_VDDA_INTERFACE_1V1

```
uint16_t LOI_NODE_VDDA_INTERFACE_1V1
```

ball H13 VDDA_LOI_1V1

4.6.2.3 LOI_NODE_VDDD_SAFETY_1V1

```
uint16_t LOI_NODE_VDDD_SAFETY_1V1
```

all N3 VDDD_SAFETY_1V1

4.7 ATB_MCLKVoltages_t Struct Reference

```
#include <rfe_tef82xx_voltage.h>
```

Data Fields

- uint16_t [MCLK_NODE_VDDD_SAFETY_1V1](#)
- uint16_t [MCLK_NODE_VDDA_1V1](#)
- uint16_t [MCLK_NODE_VDDA_1V8](#)

4.7.1 Detailed Description

Structure for MCLK voltages

4.7.2 Field Documentation

4.7.2.1 MCLK_NODE_VDDA_1V1

uint16_t MCLK_NODE_VDDA_1V1

internally generated

4.7.2.2 MCLK_NODE_VDDA_1V8

uint16_t MCLK_NODE_VDDA_1V8

ball B9 VDDA_MASTERCLK_1V8

4.7.2.3 MCLK_NODE_VDDD_SAFETY_1V1

uint16_t MCLK_NODE_VDDD_SAFETY_1V1

ball N3 VDDD_SAFETY_1V1

4.8 ATB_MIPIVoltages_t Struct Reference

```
#include <rfe_tef82xx_voltage.h>
```

Data Fields

- uint16_t [MIPI_NODE_SUPPLY1V1_CSI2](#)
- uint16_t [MIPI_NODE_SUPPLY1V1_SER](#)
- uint16_t [MIPI_NODE_SUPPLY1V8](#)
- uint16_t [MIPI_NODE_SUPPLYIO_1v8_3v3](#)
- uint16_t [MIPI_NODE_SUPPLY_MIPI_3v3](#)

4.8.1 Detailed Description

Structure for MIPI voltages

4.8.2 Field Documentation

4.8.2.1 MIPI_NODE_SUPPLY1V1_CSI2

```
uint16_t MIPI_NODE_SUPPLY1V1_CSI2
```

internally generated

4.8.2.2 MIPI_NODE_SUPPLY1V1_SER

```
uint16_t MIPI_NODE_SUPPLY1V1_SER
```

internally generated

4.8.2.3 MIPI_NODE_SUPPLY1V8

```
uint16_t MIPI_NODE_SUPPLY1V8
```

ball N9 VDDD_SER_1V8

4.8.2.4 MIPI_NODE_SUPPLY_MIPI_3v3

```
uint16_t MIPI_NODE_SUPPLY_MIPI_3v3
```

ball N7 VDDD_IO_LVDS_3V3

4.8.2.5 MIPI_NODE_SUPPLYIO_1v8_3v3

```
uint16_t MIPI_NODE_SUPPLYIO_1v8_3v3
```

ball N8 VDDD_IO_1V8_3V3

4.9 ATB_RXVoltages_t Struct Reference

```
#include <rfe_tef82xx_voltage.h>
```


Data Fields

- uint16_t [RX_NODE_VDDD_1V1](#)
- uint16_t [RX_NODE_VDDA_1V1](#)
- uint16_t [RX_NODE_VDDA_1V8](#)

4.9.1 Detailed Description

Structure for RX voltages

4.9.2 Field Documentation

4.9.2.1 RX_NODE_VDDA_1V1

```
uint16_t RX_NODE_VDDA_1V1
```

ball RX1=E13 VDDA_RX1_1V1, RX2=F13 VDDA_RX2_1V1, RX3=K13 VDDA_RX3_1V1, RX4=L13 VDDA_RX4_1V1

4.9.2.2 RX_NODE_VDDA_1V8

```
uint16_t RX_NODE_VDDA_1V8
```

ball RX1=C13 VDDA_RX1_1V8, RX2=G13 VDDA_RX2_1V8, RX3=J13 VDDA_RX3_1V8, RX4=N13 VDDA_RX4_1V8

4.9.2.3 RX_NODE_VDDD_1V1

```
uint16_t RX_NODE_VDDD_1V1
```

ball N3 VDDD_SAFETY_1V1

4.10 ATB_SSBVoltages_t Struct Reference

```
#include <rfe_tef82xx_voltage.h>
```

Data Fields

- uint16_t [SSB_NODE_VDDD_1V1](#)
- uint16_t [SSB_NODE_VDDA_1V1](#)
- uint16_t [SSB_NODE_VDDA_1V8](#)

4.10.1 Detailed Description

Structure for SSB voltages

4.10.2 Field Documentation

4.10.2.1 SSB_NODE_VDDA_1V1

`uint16_t SSB_NODE_VDDA_1V1`

internally generated

4.10.2.2 SSB_NODE_VDDA_1V8

`uint16_t SSB_NODE_VDDA_1V8`

ball N11 VDDA_BIST_1V8

4.10.2.3 SSB_NODE_VDDD_1V1

`uint16_t SSB_NODE_VDDD_1V1`

ball N3 VDDD_SAFETY_1V1

4.11 ATB_TXVoltages_t Struct Reference

```
#include <rfe_tef82xx_voltage.h>
```

Data Fields

- `uint16_t TX_NODE_VDDD_1V1_FS`
- `uint16_t TX_NODE_VDDA_1V1`
- `uint16_t TX_NODE_VDDA_1V8`

4.11.1 Detailed Description

Structure for TX voltages

4.11.2 Field Documentation

4.11.2.1 TX_NODE_VDDA_1V1

uint16_t TX_NODE_VDDA_1V1

ball M3 VDDA_TX_1V1

4.11.2.2 TX_NODE_VDDA_1V8

uint16_t TX_NODE_VDDA_1V8

ball TX1=F3&D3 VDDA_TX1_1V8, TX2=H3&G3 VDDA_TX2_1V8, TX3=L3&J3 VDDA_TX3_1V8 VDDA_TX3_1V8

4.11.2.3 TX_NODE_VDDD_1V1_FS

uint16_t TX_NODE_VDDD_1V1_FS

ball N3 VDDD_SAFETY_1V1

4.12 rfe_monitorValues_t Struct Reference

```
#include <rfe_types.h>
```

Data Fields

- uint32_t rxSaturationClippingCount [RFE_RX_COUNT *RFE_MAX_SUPPORTED_DEVICES]
- uint32_t pdcClippingCount [RFE_RX_COUNT *RFE_MAX_SUPPORTED_DEVICES]
- rfe_temperature_t temperature_beforeChirpSequence [RFE_TEMPERATURE_SENSOR_COUNT *RFE_MAX_SUPPORTED_DEVICES]
- rfe_temperature_t temperature_afterChirpSequence [RFE_TEMPERATURE_SENSOR_COUNT *RFE_MAX_SUPPORTED_DEVICES]
- rfe_temperature_t temperature_immediately [RFE_TEMPERATURE_SENSOR_COUNT *RFE_MAX_SUPPORTED_DEVICES]
- int16_t txPower [RFE_CHIRP_PROFILES_MAX][RFE_TX_COUNT *RFE_MAX_SUPPORTED_DEVICES]

4.12.1 Detailed Description

This structure is a container for all RFE-Fw monitors to be read via [rfe_monitorRead\(\)](#).

4.12.2 Field Documentation

4.12.2.1 pdcClippingCount

```
uint32_t pdcClippingCount[RFE_RX_COUNT * RFE_MAX_SUPPORTED_DEVICES]
```

The clipping count of each programmable decimation chain.

4.12.2.2 rxSaturationClippingCount

```
uint32_t rxSaturationClippingCount[RFE_RX_COUNT * RFE_MAX_SUPPORTED_DEVICES]
```

The clipping count of each rx saturation.

4.12.2.3 temperature_afterChirpSequence

```
rfe_temperature_t temperature_afterChirpSequence[RFE_TEMPERATURE_SENSOR_COUNT * RFE_MAX_SUPPORTED_DEVICES]
```

The temperature measured after the chirp sequence for each temperature sensor in SQ6 format.

4.12.2.4 temperature_beforeChirpSequence

```
rfe_temperature_t temperature_beforeChirpSequence[RFE_TEMPERATURE_SENSOR_COUNT * RFE_MAX_SUPPORTED_DEVICES]
```

The temperature measured before the chirp sequence for each temperature sensor in SQ6 format.

4.12.2.5 temperature_immediately

```
rfe_temperature_t temperature_immediately[RFE_TEMPERATURE_SENSOR_COUNT * RFE_MAX_SUPPORTED_DEVICES]
```

The temperature measured at the time [rfe_monitorRead\(\)](#) is executed in SQ6 format.

4.12.2.6 txPower

```
int16_t txPower[RFE_CHIRP_PROFILES_MAX][RFE_TX_COUNT * RFE_MAX_SUPPORTED_DEVICES]
```

The measured output power in 0.1 [dBm] for each transmitter and profile. -100 means given profile for given TX is not calibrated. -128 means that the front end cannot measure the output power

4.13 rfe_radarCycleCount_t Struct Reference

```
#include <rfe_types.h>
```

Data Fields

- uint16_t [radarCycleCount](#)
- uint16_t [chirpSequenceCount](#)

4.13.1 Detailed Description

This structure contains the radar cycle and chirp sequence count. It can be retrieved via [rfe_getRadarCycleCount\(\)](#).

4.13.2 Field Documentation

4.13.2.1 chirpSequenceCount

```
uint16_t chirpSequenceCount
```

Number of chirp sequences that have been completed since [rfe_radarCycleStart\(\)](#).

4.13.2.2 radarCycleCount

```
uint16_t radarCycleCount
```

Number of radar cycles that have been completed since [rfe_radarCycleStart\(\)](#).

4.14 rfe_registerDumpValues_t Struct Reference

```
#include <rfe_register_dump.h>
```

Data Fields

- uint8_t [frontEndId](#)
- uint8_t [moduleId](#)
- uint16_t [registerAddress](#)
- uint32_t [value](#)

4.14.1 Detailed Description

This structure is a container for BCD register dump to be read via [rfe_registerDumpRead\(\)](#).

4.14.2 Field Documentation

4.14.2.1 frontEndId

```
uint8_t frontEndId
```

4.14.2.2 moduleId

```
uint8_t moduleId
```

4.14.2.3 registerAddress

```
uint16_t registerAddress
```

4.14.2.4 value

```
uint32_t value
```

4.15 rfe_tef82xxVoltages_t Struct Reference

```
#include <rfe_tef82xx_voltage.h>
```

Data Fields

- [ATB_ADCVoltages_t ADC12Voltages](#)
- [ATB_ADCVoltages_t ADC34Voltages](#)
- [ATB_ATBVoltages_t ATBVoltages](#)
- [ATB_CAFCVoltages_t CAFCVoltages](#)
- [ATB_GBIASVoltages_t GBIASVoltages](#)
- [ATB_GLDOVoltages_t GLDOVoltages](#)
- [ATB_LOIVoltages_t LOIVoltages](#)
- [ATB_MCLKVoltages_t MCLKVoltages](#)
- [ATB_MIPIVoltages_t MIPIVoltages](#)
- [ATB_RXVoltages_t RX1Voltages](#)
- [ATB_RXVoltages_t RX2Voltages](#)
- [ATB_RXVoltages_t RX3Voltages](#)
- [ATB_RXVoltages_t RX4Voltages](#)
- [ATB_SSBVoltages_t SSBVoltages](#)
- [ATB_TXVoltages_t TX1Voltages](#)
- [ATB_TXVoltages_t TX2Voltages](#)
- [ATB_TXVoltages_t TX3Voltages](#)

4.15.1 Detailed Description

This type defines the container for ATB based voltage read-out (in mV) of all modules.

4.15.2 Field Documentation

4.15.2.1 ADC12Voltages

`ATB_ADCVoltages_t` ADC12Voltages

4.15.2.2 ADC34Voltages

`ATB_ADCVoltages_t` ADC34Voltages

4.15.2.3 ATBVoltages

`ATB_ATBVoltages_t` ATBVoltages

4.15.2.4 CAFCVoltages

`ATB_CAFCVoltages_t` CAFCVoltages

4.15.2.5 GBIASVoltages

`ATB_GBIASVoltages_t` GBIASVoltages

4.15.2.6 GLDOVoltages

`ATB_GLDOVoltages_t` GLDOVoltages

4.15.2.7 LOIVoltages

`ATB_LOIVoltages_t` LOIVoltages

4.15.2.8 MCLKVoltages

`ATB_MCLKVoltages_t` MCLKVoltages

4.15.2.9 MIPIVoltages

`ATB_MIPIVoltages_t` MIPIVoltages

4.15.2.10 RX1Voltages

`ATB_RXVoltages_t` RX1Voltages

4.15.2.11 RX2Voltages

`ATB_RXVoltages_t` RX2Voltages

4.15.2.12 RX3Voltages

`ATB_RXVoltages_t` RX3Voltages

4.15.2.13 RX4Voltages

`ATB_RXVoltages_t` RX4Voltages

4.15.2.14 SSBVoltages

`ATB_SSBVoltages_t` SSBVoltages

4.15.2.15 TX1Voltages

[ATB_TXVoltages_t](#) TX1Voltages

4.15.2.16 TX2Voltages

[ATB_TXVoltages_t](#) TX2Voltages

4.15.2.17 TX3Voltages

[ATB_TXVoltages_t](#) TX3Voltages

4.16 rfe_version_t Struct Reference

```
#include <rfe_types.h>
```

Data Fields

- [uint32_t hwType](#)
- [uint32_t hwVariant](#)
- [uint32_t hwVersion](#)
- [uint32_t hwUniqueId](#)
- [uint32_t fwVariant](#)
- [bool fwVersionReleased](#)
- [uint8_t fwVersionMajor](#)
- [uint8_t fwVersionMinor](#)
- [uint8_t fwVersionPatch](#)
- [uint32_t fwHash](#)

4.16.1 Detailed Description

This structure contains the version and variant details of the RFE-Fw:

- Hardware and firmware variants
- Hardware and firmware versions

4.16.2 Field Documentation

4.16.2.1 fwHash

```
uint32_t fwHash
```

Defines the firmware hash for identifying unreleased versions.

4.16.2.2 fwVariant

```
uint32_t fwVariant
```

Defines the firmware variant 1: E2 T0 trim, evaluation sample 2: E2 OTP trim, evaluation sample 3: E2, E5 OTP trim, evaluation sample 4: A1-E5-T2MF

4.16.2.3 fwVersionMajor

```
uint8_t fwVersionMajor
```

Defines the major firmware version.

4.16.2.4 fwVersionMinor

```
uint8_t fwVersionMinor
```

Defines the minor firmware version.

4.16.2.5 fwVersionPatch

```
uint8_t fwVersionPatch
```

Defines the patch firmware version.

4.16.2.6 fwVersionReleased

```
bool fwVersionReleased
```

Defines whether the firmware has been officially released. In case of an un official release, major, minor and patch indicate the latest officially released version which is the basis for the firmware. The firmware hash can be used to identify the firmware in that case.

4.16.2.7 hwType

```
uint32_t hwType
```

Defines the IC type SAF85xx: 100 S32R41: 200

4.16.2.8 hwUniqueId

uint32_t hwUniqueId

Defines 32-bit Unique ID of RFE device connected effectively CRC32 of BC OTP ChipInfo structure of Leader Device

4.16.2.9 hwVariant

uint32_t hwVariant

Defines the commercial IC variant xx : 0 (no variant defined, evaluation sample)

4.16.2.10 hwVersion

uint32_t hwVersion

Defines the IC version ES1 : 1 ES1.1 (A1MF) : 2

4.17 rfeCfg_rxBistReferenceData_t Struct Reference

Struct to hold value of RX Bist reference data.

```
#include <rfe_blob_access.h>
```

Data Fields

- int16_t [refMag](#) [[RFE_RX_BIST_COMBINATIONS](#)][[TEF82XX_RFBIST_TESTS](#)]
- int16_t [refPh](#) [[RFE_RX_BIST_COMBINATIONS](#)][[TEF82XX_RFBIST_TESTS](#)]

4.17.1 Detailed Description

Struct to hold value of RX Bist reference data.

refMag - Array of reference signal magnitudes difference vs 1st RX for all RX channels measured in [dB] for LNA and mixer input. refPh - Array of reference signal phase difference vs 1st RX for all RX channels measured in [rad] for LNA and mixer input.

4.17.2 Field Documentation

4.17.2.1 refMag

```
int16_t refMag[RFE_RX_BIST_COMBINATIONS][TEF82XX_RFBIST_TESTS]
```

4.17.2.2 refPh

```
int16_t refPh[RFE_RX_BIST_COMBINATIONS][TEF82XX_RFBIST_TESTS]
```

4.18 rfeCmdClient_update_t Struct Reference

```
#include <rfe_cmd_client.h>
```

Data Fields

- uint16_t [section](#)
- uint16_t [param](#)
- uint32_t [value](#)

4.18.1 Detailed Description

This structure defines a structured parameter update

4.18.2 Field Documentation

4.18.2.1 param

```
uint16_t param
```

4.18.2.2 section

```
uint16_t section
```

4.18.2.3 value

```
uint32_t value
```

Index

ADC12Voltages
 rfe_tef82xxVoltages_t, [267](#)
ADC34Voltages
 rfe_tef82xxVoltages_t, [267](#)
ADC_NODE_1V1_SAFETY_SUPPLY
 ATB_ADCVoltages_t, [253](#)
ADC_NODE_1V1_SUPPLY
 ATB_ADCVoltages_t, [253](#)
ADC_NODE_1V8_SUPPLY
 ATB_ADCVoltages_t, [253](#)
ATB_ADCVoltages_t, [253](#)
 ADC_NODE_1V1_SAFETY_SUPPLY, [253](#)
 ADC_NODE_1V1_SUPPLY, [253](#)
 ADC_NODE_1V8_SUPPLY, [253](#)
ATB_ATBVoltages_t, [254](#)
 ATB_NODE_VDDA_1V1, [254](#)
 ATB_NODE_VDDA_1V8, [254](#)
 ATB_NODE_VDDD_SAFETY_1V1, [254](#)
ATB_CAFCVoltages_t, [255](#)
 CAFC_NODE_LDO_CPPFD_VDD, [255](#)
 CAFC_NODE_LDO_DIVIDER_VDD, [255](#)
 CAFC_NODE_LDO_VCO_VDD, [255](#)
 CAFC_NODE_VDD_PLL_1V1_FILTERED, [255](#)
 CAFC_NODE_VDD_PLL_1V8, [256](#)
 CAFC_NODE_VDD_VCO_1V8, [256](#)
ATB_GBIASVoltages_t, [256](#)
 GBIAS_NODE_VDDA_GLOBAL_BIAS_1V8, [256](#)
ATB_GLDOVoltages_t, [257](#)
 GLDO_NODE_VDDA_LDO_1V1, [257](#)
 GLDO_NODE_VDDA_LDO_1V8, [257](#)
 GLDO_NODE_VDDA_LDO_3V3, [257](#)
 GLDO_NODE_VDDD_SAFETY_1V1, [257](#)
ATB_LOIVoltages_t, [258](#)
 LOI_NODE_VDDA_1V8, [258](#)
 LOI_NODE_VDDA_INTERFACE_1V1, [258](#)
 LOI_NODE_VDDD_SAFETY_1V1, [258](#)
ATB_MCLKVoltages_t, [258](#)
 MCLK_NODE_VDDA_1V1, [259](#)
 MCLK_NODE_VDDA_1V8, [259](#)
 MCLK_NODE_VDDD_SAFETY_1V1, [259](#)
ATB_MIPIVoltages_t, [259](#)
 MIPI_NODE_SUPPLY1V1_CSI2, [260](#)
 MIPI_NODE_SUPPLY1V1_SER, [260](#)
 MIPI_NODE_SUPPLY1V8, [260](#)
 MIPI_NODE_SUPPLY_MIPI_3v3, [260](#)
 MIPI_NODE_SUPPLYIO_1v8_3v3, [260](#)
ATB_NODE_VDDA_1V1
 ATB_ATBVoltages_t, [254](#)
ATB_NODE_VDDA_1V8
 ATB_ATBVoltages_t, [254](#)
ATB_NODE_VDDD_SAFETY_1V1
 ATB_ATBVoltages_t, [254](#)
ATB_RXVoltages_t, [260](#)
 RX_NODE_VDDA_1V1, [261](#)
 RX_NODE_VDDA_1V8, [261](#)
 RX_NODE_VDDD_1V1, [261](#)
ATB_SSBVoltages_t, [261](#)
 SSB_NODE_VDDA_1V1, [262](#)
 SSB_NODE_VDDA_1V8, [262](#)
 SSB_NODE_VDDD_1V1, [262](#)
ATB_TXVoltages_t, [262](#)
 TX_NODE_VDDA_1V1, [263](#)
 TX_NODE_VDDA_1V8, [263](#)
 TX_NODE_VDDD_1V1_FS, [263](#)
ATBVoltages
 rfe_tef82xxVoltages_t, [267](#)
CAFC_NODE_LDO_CPPFD_VDD
 ATB_CAFCVoltages_t, [255](#)
CAFC_NODE_LDO_DIVIDER_VDD
 ATB_CAFCVoltages_t, [255](#)
CAFC_NODE_LDO_VCO_VDD
 ATB_CAFCVoltages_t, [255](#)
CAFC_NODE_VDD_PLL_1V1_FILTERED
 ATB_CAFCVoltages_t, [255](#)
CAFC_NODE_VDD_PLL_1V8
 ATB_CAFCVoltages_t, [256](#)
CAFC_NODE_VDD_VCO_1V8
 ATB_CAFCVoltages_t, [256](#)
CAFCVoltages
 rfe_tef82xxVoltages_t, [267](#)
CDD_RFE_CFG_HALT_ON_ERROR
 RFE Abstract 2.0 Constants, [8](#)
CDD_RFE_CONFIGURE
 RFE Abstract 2.0 Constants, [9](#)
CDD_Rfe_Configure
 RFE Abstract 2.0 Interface, [27](#)
CDD_Rfe_ConfigureInterrupt
 RFE Abstract 2.0 Interface, [29](#)
CDD_RFE_DEV_ERROR_DETECT
 RFE Abstract 2.0 Constants, [9](#)
CDD_RFE_E_CMD_LENGTH
 RFE Abstract 2.0 Constants, [9](#)
CDD_RFE_E_CMD_NOT_IMPLEMENTED
 RFE Abstract 2.0 Constants, [9](#)
CDD_RFE_E_FUSA_ERROR
 RFE Abstract 2.0 Constants, [9](#)
CDD_RFE_E_FW_ERROR
 RFE Abstract 2.0 Constants, [10](#)

- CDD_RFE_E_LIMIT_EXCEED
 - RFE Abstract 2.0 Constants, [10](#)
- CDD_RFE_E_PARAM_POINTER
 - RFE Abstract 2.0 Constants, [10](#)
- CDD_RFE_E_PARAM_SIZE
 - RFE Abstract 2.0 Constants, [10](#)
- CDD_RFE_E_REDUNDANCY_CHECK_FAILED
 - RFE Abstract 2.0 Constants, [10](#)
- CDD_RFE_E_RESPONSE_ERROR
 - RFE Abstract 2.0 Constants, [10](#)
- CDD_RFE_E_SETUP_FAILED
 - RFE Abstract 2.0 Constants, [11](#)
- CDD_RFE_E_UNRESPONSIVE
 - RFE Abstract 2.0 Constants, [11](#)
- CDD_RFE_FRONT_END_FOLLOWER_0
 - RFE Abstract 2.0 Interface, [25](#)
- CDD_RFE_FRONT_END_FOLLOWER_1
 - RFE Abstract 2.0 Interface, [25](#)
- CDD_RFE_FRONT_END_FOLLOWER_2
 - RFE Abstract 2.0 Interface, [25](#)
- CDD_RFE_FRONT_END_FOLLOWER_MAX
 - RFE Abstract 2.0 Interface, [25](#)
- CDD_RFE_FRONT_END_LEADER
 - RFE Abstract 2.0 Interface, [25](#)
- CDD_RFE_FRONT_END_MASK_FOLLOWER_0
 - RFE Abstract 2.0 Interface, [26](#)
- CDD_RFE_FRONT_END_MASK_FOLLOWER_1
 - RFE Abstract 2.0 Interface, [26](#)
- CDD_RFE_FRONT_END_MASK_FOLLOWER_2
 - RFE Abstract 2.0 Interface, [26](#)
- CDD_RFE_FRONT_END_MASK_LEADER
 - RFE Abstract 2.0 Interface, [26](#)
- CDD_RFE_GET_DATA
 - RFE Abstract 2.0 Constants, [11](#)
- CDD_Rfe_GetBistZeroHourReferenceData
 - RFE Abstract 2.0 Interface, [30](#)
- CDD_Rfe_GetBlobValue
 - RFE Abstract 2.0 Interface, [30](#)
- CDD_Rfe_GetError
 - RFE Abstract 2.0 Interface, [31](#)
- CDD_Rfe_GetFrontEnd
 - RFE Abstract 2.0 Interface, [31](#)
- CDD_Rfe_GetFuSaFaults
 - RFE Abstract 2.0 Interface, [32](#)
- CDD_Rfe_GetFuSaFaultStatistics
 - RFE Abstract 2.0 Interface, [33](#)
- CDD_Rfe_GetNextRadarCycleStartTime
 - RFE Abstract 2.0 Interface, [34](#)
- CDD_Rfe_GetRadarCycleCount
 - RFE Abstract 2.0 Interface, [35](#)
- CDD_Rfe_GetRegisterDump
 - RFE Abstract 2.0 Interface, [35](#)
- CDD_Rfe_GetState
 - RFE Abstract 2.0 Interface, [36](#)
- CDD_Rfe_GetTime
 - RFE Abstract 2.0 Interface, [37](#)
- CDD_Rfe_GetVersionInfo
 - RFE Abstract 2.0 Interface, [37](#)
- CDD_RFE_HALT_ON_ERROR
 - RFE Abstract 2.0 Constants, [11](#)
- CDD_Rfe_Init
 - RFE Abstract 2.0 Interface, [38](#)
- CDD_Rfe_MonitorRead
 - RFE Abstract 2.0 Interface, [38](#)
- CDD_Rfe_RadarCycleStart
 - RFE Abstract 2.0 Interface, [39](#)
- CDD_Rfe_RadarCycleStop
 - RFE Abstract 2.0 Interface, [40](#)
- CDD_RFE_SET_DATA
 - RFE Abstract 2.0 Constants, [11](#)
- CDD_Rfe_SetBlobValue
 - RFE Abstract 2.0 Interface, [41](#)
- CDD_Rfe_SetFrontEnd
 - RFE Abstract 2.0 Interface, [41](#)
- CDD_Rfe_SetNextRadarCycleStartTime
 - RFE Abstract 2.0 Interface, [42](#)
- CDD_RFE_SID_CONFIGURE
 - RFE Abstract 2.0 Constants, [12](#)
- CDD_RFE_SID_GET_ERROR
 - RFE Abstract 2.0 Constants, [12](#)
- CDD_RFE_SID_GET_VERSION_INFO
 - RFE Abstract 2.0 Constants, [12](#)
- CDD_RFE_SID_GETFUSAERRORS
 - RFE Abstract 2.0 Constants, [12](#)
- CDD_RFE_SID_GETNEXTRADARCYCLESTARTTIME
 - RFE Abstract 2.0 Constants, [12](#)
- CDD_RFE_SID_GETRADARCYCLECOUNT
 - RFE Abstract 2.0 Constants, [12](#)
- CDD_RFE_SID_GETSTATE
 - RFE Abstract 2.0 Constants, [12](#)
- CDD_RFE_SID_GETTIME
 - RFE Abstract 2.0 Constants, [13](#)
- CDD_RFE_SID_GETVERSION
 - RFE Abstract 2.0 Constants, [13](#)
- CDD_RFE_SID_INIT
 - RFE Abstract 2.0 Constants, [13](#)
- CDD_RFE_SID_INVALID
 - RFE Abstract 2.0 Constants, [13](#)
- CDD_RFE_SID_MONITORREAD
 - RFE Abstract 2.0 Constants, [13](#)
- CDD_RFE_SID_RADARCYCLESTART
 - RFE Abstract 2.0 Constants, [13](#)
- CDD_RFE_SID_RADARCYCLESTOP
 - RFE Abstract 2.0 Constants, [13](#)
- CDD_RFE_SID_SETNEXTRADARCYCLESTARTTIME
 - RFE Abstract 2.0 Constants, [14](#)
- CDD_RFE_SID_TESTCONTINUOUSWAVETRANSMISSIONSTART
 - RFE Abstract 2.0 Constants, [14](#)
- CDD_RFE_SID_TESTCONTINUOUSWAVETRANSMISSIONSTOP
 - RFE Abstract 2.0 Constants, [14](#)
- CDD_RFE_SID_TESTSETPARAM
 - RFE Abstract 2.0 Constants, [14](#)
- CDD_RFE_SID_UPDATEBEGIN
 - RFE Abstract 2.0 Constants, [14](#)
- CDD_RFE_SID_UPDATEDYNAMICTABLE
 - RFE Abstract 2.0 Constants, [14](#)

- CDD_RFE_SID_UPDATEPARAM
 - RFE Abstract 2.0 Constants, [14](#)
- CDD_RFE_SID_UPDATEPUSH
 - RFE Abstract 2.0 Constants, [14](#)
- CDD_RFE_SINGLE_MANAGEMENT_THREADS
 - RFE Abstract 2.0 Constants, [15](#)
- CDD_RFE_START_EXECUTION
 - RFE Abstract 2.0 Constants, [15](#)
- CDD_RFE_STOP_EXECUTION
 - RFE Abstract 2.0 Constants, [15](#)
- CDD_Rfe_SwHwGetVersion
 - RFE Abstract 2.0 Interface, [43](#)
- CDD_Rfe_TestContinuousWaveTransmissionStart
 - RFE Abstract 2.0 Interface, [43](#)
- CDD_Rfe_TestContinuousWaveTransmissionStop
 - RFE Abstract 2.0 Interface, [44](#)
- CDD_Rfe_TestGetInternalError
 - RFE Abstract 2.0 Interface, [45](#)
- CDD_Rfe_TestSetParam
 - RFE Abstract 2.0 Interface, [45](#)
- CDD_Rfe_UpdateBegin
 - RFE Abstract 2.0 Interface, [46](#)
- CDD_Rfe_UpdateDynamicTable
 - RFE Abstract 2.0 Interface, [47](#)
- CDD_Rfe_UpdateParam
 - RFE Abstract 2.0 Interface, [47](#)
- CDD_Rfe_UpdatePush
 - RFE Abstract 2.0 Interface, [48](#)
- CDD_RFE_VERSION_INFO_API
 - RFE Abstract 2.0 Constants, [15](#)
- CDD_RFE_VERSION_INFO_CHECK
 - RFE Abstract 2.0 Constants, [15](#)
- CDD_RFE_ZERO
 - RFE Abstract 2.0 Constants, [15](#)
- Cdd_RfeApiIRQDataType
 - RFE Abstract 2.0 Interface, [22](#)
- Cdd_RfeApiWaitForInterruptDataType
 - RFE Abstract 2.0 Interface, [22](#)
- Cdd_RfeCfgParamType
 - RFE Abstract 2.0 Interface, [22](#)
- Cdd_RfeCfgSectionType
 - RFE Abstract 2.0 Interface, [23](#)
- Cdd_RfeChirpProfileIndexType
 - RFE Abstract 2.0 Interface, [23](#)
- Cdd_RfeCoreIdDataType
 - RFE Abstract 2.0 Interface, [23](#)
- Cdd_RfeDynParamsSelectType
 - RFE Abstract 2.0 Interface, [23](#)
- Cdd_RfeErrorType
 - RFE Abstract 2.0 Interface, [23](#)
- Cdd_RfeEventsIRQDataType
 - RFE Abstract 2.0 Interface, [23](#)
- CDD_RfeFrontEndHierarchyType
 - RFE Abstract 2.0 Interface, [25](#)
- CDD_RfeFrontEndIdMaskType
 - RFE Abstract 2.0 Interface, [26](#)
- Cdd_RfeFusaFaultType
 - RFE Abstract 2.0 Interface, [23](#)
- Cdd_RfeMonitorSelectType
 - RFE Abstract 2.0 Interface, [24](#)
- Cdd_RfeMonitorValuesType
 - RFE Abstract 2.0 Interface, [24](#)
- Cdd_RfeParamUpdateType
 - RFE Abstract 2.0 Interface, [24](#)
- Cdd_RfeRadarCycleCountType
 - RFE Abstract 2.0 Interface, [24](#)
- Cdd_RfeRxBistZeroHourRefDataType
 - RFE Abstract 2.0 Interface, [24](#)
- Cdd_RfeStateType
 - RFE Abstract 2.0 Interface, [24](#)
- Cdd_RfeSysMemAddressType
 - RFE Abstract 2.0 Interface, [25](#)
- Cdd_RfeTestParamType
 - RFE Abstract 2.0 Interface, [25](#)
- Cdd_RfeVersionType
 - RFE Abstract 2.0 Interface, [25](#)
- chirpSequenceCount
 - rfe_radarCycleCount_t, [265](#)
- e_ALL_BC_PINS
 - RFE Abstract 2.0 - GPIO Interface, [240](#)
- e_CHIRPSTART_IN_PAD
 - RFE Abstract 2.0 - GPIO Interface, [240](#)
- e_ERRORN
 - RFE Abstract 2.0 - GPIO Interface, [240](#)
- e_ERRORRESET
 - RFE Abstract 2.0 - GPIO Interface, [240](#)
- e_HRESET
 - RFE Abstract 2.0 - GPIO Interface, [240](#)
- e_MCUINT_CHIRPSTART_OUT_PAD
 - RFE Abstract 2.0 - GPIO Interface, [240](#)
- e_PINID_INVALID
 - RFE Abstract 2.0 - GPIO Interface, [240](#)
- e_TX1_I_PS
 - RFE Abstract 2.0 - GPIO Interface, [240](#)
- e_TX1_Q_PS
 - RFE Abstract 2.0 - GPIO Interface, [240](#)
- e_TX2_I_PS
 - RFE Abstract 2.0 - GPIO Interface, [240](#)
- e_TX2_Q_PS
 - RFE Abstract 2.0 - GPIO Interface, [240](#)
- e_TX3_I_PS
 - RFE Abstract 2.0 - GPIO Interface, [240](#)
- e_TX3_Q_PS
 - RFE Abstract 2.0 - GPIO Interface, [240](#)
- frontEndId
 - rfe_registerDumpValues_t, [266](#)
- fwHash
 - rfe_version_t, [269](#)
- fwVariant
 - rfe_version_t, [270](#)
- fwVersionMajor
 - rfe_version_t, [270](#)
- fwVersionMinor
 - rfe_version_t, [270](#)
- fwVersionPatch

- rfe_version_t, [270](#)
- fwVersionReleased
 - rfe_version_t, [270](#)
- GBIAS_NODE_VDDA_GLOBAL_BIAS_1V8
 - ATB_GBIASVoltages_t, [256](#)
- GBIASVoltages
 - rfe_tef82xxVoltages_t, [267](#)
- GLDO_NODE_VDDA_LDO_1V1
 - ATB_GLDOVoltages_t, [257](#)
- GLDO_NODE_VDDA_LDO_1V8
 - ATB_GLDOVoltages_t, [257](#)
- GLDO_NODE_VDDA_LDO_3V3
 - ATB_GLDOVoltages_t, [257](#)
- GLDO_NODE_VDDD_SAFETY_1V1
 - ATB_GLDOVoltages_t, [257](#)
- GLDOVoltages
 - rfe_tef82xxVoltages_t, [267](#)
- gsCddRfeLoopExit
 - RFE Abstract 2.0 Constants, [18](#)
- hwType
 - rfe_version_t, [270](#)
- hwUniqueld
 - rfe_version_t, [270](#)
- hwVariant
 - rfe_version_t, [271](#)
- hwVersion
 - rfe_version_t, [271](#)
- InjectToneMode_Invalid
 - RFE Abstract 2.0 - Low Level Interface, [121](#)
- LOI_NODE_VDDA_1V8
 - ATB_LOIVoltages_t, [258](#)
- LOI_NODE_VDDA_INTERFACE_1V1
 - ATB_LOIVoltages_t, [258](#)
- LOI_NODE_VDDD_SAFETY_1V1
 - ATB_LOIVoltages_t, [258](#)
- LOIVoltages
 - rfe_tef82xxVoltages_t, [267](#)
- MCLK_NODE_VDDA_1V1
 - ATB_MCLKVoltages_t, [259](#)
- MCLK_NODE_VDDA_1V8
 - ATB_MCLKVoltages_t, [259](#)
- MCLK_NODE_VDDD_SAFETY_1V1
 - ATB_MCLKVoltages_t, [259](#)
- MCLKVoltages
 - rfe_tef82xxVoltages_t, [268](#)
- MIPI_NODE_SUPPLY1V1_CSI2
 - ATB_MIPIVoltages_t, [260](#)
- MIPI_NODE_SUPPLY1V1_SER
 - ATB_MIPIVoltages_t, [260](#)
- MIPI_NODE_SUPPLY1V8
 - ATB_MIPIVoltages_t, [260](#)
- MIPI_NODE_SUPPLY_MIPI_3v3
 - ATB_MIPIVoltages_t, [260](#)
- MIPI_NODE_SUPPLYIO_1v8_3v3
 - ATB_MIPIVoltages_t, [260](#)
- MIPIVoltages
 - rfe_tef82xxVoltages_t, [268](#)
- moduleId
 - rfe_registerDumpValues_t, [266](#)
- param
 - rfeCmdClient_update_t, [272](#)
- pdcClippingCount
 - rfe_monitorValues_t, [263](#)
- radarCycleCount
 - rfe_radarCycleCount_t, [265](#)
- refMag
 - rfeCfg_rxBistReferenceData_t, [271](#)
- refPh
 - rfeCfg_rxBistReferenceData_t, [272](#)
- REG_MAX_COUNT_TO_SEND
 - RFE Abstract 2.0 Constants, [15](#)
- registerAddress
 - rfe_registerDumpValues_t, [266](#)
- RFE Abstract 2.0, [5](#)
- RFE Abstract 2.0 - Debug Interface, [246](#)
 - RfeDbgPrintMsg, [247](#)
 - RfeInitDebug, [247](#)
- RFE Abstract 2.0 - Firmware Constants, [248](#)
 - RFE_SW_MAIN_FSM_MAIN_STATE_COUNT, [248](#)
- RFE Abstract 2.0 - Firmware Interface, [248](#)
 - rfeFrontEnd_Cascaded_e, [249](#)
 - rfeFrontEnd_Configuration_t, [249](#)
 - rfeFrontEnd_Single_e, [249](#)
 - rfeMainFsm, [249](#)
 - rfeSetFrontendsConfiguration, [250](#)
 - rfeSetSharedAdresses, [251](#)
 - rfeSwMainFsm_mainState_configured_e, [249](#)
 - rfeSwMainFsm_mainState_configuring_e, [249](#)
 - rfeSwMainFsm_mainState_fuSaFault_e, [249](#)
 - rfeSwMainFsm_mainState_initialized_e, [249](#)
 - rfeSwMainFsm_mainState_initializing_e, [249](#)
 - rfeSwMainFsm_mainState_radarCycles_e, [249](#)
 - rfeSwMainFsm_mainState_startRadarCycles_e, [249](#)
 - rfeSwMainFsm_mainState_t, [249](#)
 - rfeSwMainFsm_mainState_testContinuousWaveTransmission_e, [249](#)
 - rfeSwMainFsm_mainState_waitingForSync_e, [249](#)
- RFE Abstract 2.0 - Front-End configuration structure :
 - BLOB, [156](#)
- RFE Abstract 2.0 - GPIO Interface, [239](#)
 - e_ALL_BC_PINS, [240](#)
 - e_CHIRPSTART_IN_PAD, [240](#)
 - e_ERRORN, [240](#)
 - e_ERRORRESET, [240](#)
 - e_HRESET, [240](#)
 - e_MCUINT_CHIRPSTART_OUT_PAD, [240](#)
 - e_PINID_INVALID, [240](#)
 - e_TX1_I_PS, [240](#)
 - e_TX1_Q_PS, [240](#)

- e_TX2_I_PS, [240](#)
- e_TX2_Q_PS, [240](#)
- e_TX3_I_PS, [240](#)
- e_TX3_Q_PS, [240](#)
- RfeHwGpioGet, [240](#)
- RfeHwGpioInit, [241](#)
- RfeHwGpioSet, [241](#)
- TEF82XX_ExtPinID_e, [240](#)
- RFE Abstract 2.0 - Interrupt Interface, [245](#)
 - RfeHwRaiseIRQ, [246](#)
 - RfeHwSetIRQ, [246](#)
- RFE Abstract 2.0 - IPC Interface, [234](#)
 - rfelpc_clientCmdBuffAcquire, [235](#)
 - rfelpc_clientCmdTx, [235](#)
 - rfelpc_clientRspWait, [235](#)
 - rfelpc_clientStsPoll, [236](#)
 - rfelpc_clientStsWait, [236](#)
 - rfelpc_srvCmdPoll, [236](#)
 - rfelpc_srvRspBuffAcquire, [237](#)
 - rfelpc_srvRspTx, [237](#)
 - rfelpc_srvStsBuffAcquire, [238](#)
 - rfelpc_srvStsTx, [238](#)
- RFE Abstract 2.0 - Low Level Constants, [49](#)
 - RFE_API_IRQ_ALL, [51](#)
 - RFE_API_IRQ_MAX_NUM, [51](#)
 - RFE_CMD_CLIENT_FUSA_FAULT_BYTE_COUNT, [51](#)
 - RFE_CMD_CLIENT_UPDATE_SECTION_DYNAMIC_TABLE, [51](#)
 - RFE_DATA_OUT_DEST_CSI2, [51](#)
 - RFE_DATA_OUT_DEST_PACKET_PROCESSOR, [52](#)
 - RFE_DYN_PARAMS_SELECT_ALL, [52](#)
 - RFE_DYN_PARAMS_SELECT_CHIRP_FREQUENCY_DRIVE_STEPS, [52](#)
 - RFE_DYN_PARAMS_SELECT_CHIRP_PROFILE, [52](#)
 - RFE_DYN_PARAMS_SELECT_CIT_TICKS, [52](#)
 - RFE_DYN_PARAMS_SELECT_DWELL_TIME_TICKS, [52](#)
 - RFE_DYN_PARAMS_SELECT_TX1_PHASE_ROTATION, [52](#)
 - RFE_DYN_PARAMS_SELECT_TX2_PHASE_ROTATION, [53](#)
 - RFE_DYN_PARAMS_SELECT_TX3_PHASE_ROTATION, [53](#)
 - RFE_DYN_PARAMS_SELECT_TX4_PHASE_ROTATION, [53](#)
 - RFE_DYN_PARAMS_SELECT_TX_TRANSMISSION_ENABLE, [53](#)
 - RFE_ERROR_RFE_INTERNAL_ERROR_BIT, [53](#)
 - RFE_EVENTS_IRQ_ALL, [53](#)
 - RFE_FUSA_R1_FAULT_COUNT_MAX, [53](#)
 - RFE_FUSA_R1_R2_FAULT_COUNT_MAX, [54](#)
 - RFE_FUSA_R1_R2_MASK_UNMASK_FAULT_BYTE_COUNT, [54](#)
 - RFE_FUSA_R2_FAULT_COUNT_MAX, [54](#)
 - RFE_KELVIN_TO_CELSIUS, [54](#)
 - RFE_MAX_SUPPORTED_DEVICES, [54](#)
 - RFE_MONITOR_SELECT_ALL, [54](#)
 - RFE_MONITOR_SELECT_PDC_CLIPPING_COUNT, [54](#)
 - RFE_MONITOR_SELECT_RX_SAT_COUNT_STAGE_1I, [55](#)
 - RFE_MONITOR_SELECT_TEMP_AFTER_CHIRP_SEQ, [55](#)
 - RFE_MONITOR_SELECT_TEMP_BEFORE_CHIRP_SEQ, [55](#)
 - RFE_MONITOR_SELECT_TEMP_IMMEDIATELY, [55](#)
 - RFE_MONITOR_SELECT_TX_POWER, [55](#)
 - RFE_NS_PER_TIME_TICK, [55](#)
 - RFE_RX_BIST_COMBINATIONS, [55](#)
 - RFE_RX_CLIPPING_RESET, [56](#)
 - RFE_RX_COUNT, [56](#)
 - RFE_RX_SELECT_1, [56](#)
 - RFE_RX_SELECT_2, [56](#)
 - RFE_RX_SELECT_3, [56](#)
 - RFE_RX_SELECT_4, [56](#)
 - RFE_RX_SELECT_ALL, [56](#)
 - RFE_RX_SELECT_NONE, [56](#)
 - RFE_RX_STAGE, [57](#)
 - RFE_TEMPERATURE_1_DEG_CELSIUS, [57](#)
 - RFE_TEMPERATURE_SENSOR_COUNT, [57](#)
 - RFE_TIME_TICKS_PER_US, [57](#)
 - RFE_TX_BIST_COMBINATIONS, [57](#)
 - RFE_TX_POWER_ERR, [57](#)
 - RFE_TX_SELECT_1, [57](#)
 - RFE_TX_SELECT_2, [58](#)
 - RFE_TX_SELECT_3, [58](#)
 - RFE_TX_SELECT_4, [58](#)
 - RFE_TX_SELECT_ALL, [58](#)
 - RFE_TX_SELECT_NONE, [58](#)
 - RFE_UPDATE_COUNT_MAX, [58](#)
 - RFE Abstract 2.0 - Low Level Interface, [58](#)
 - InjectToneMode_Invalid, [121](#)
 - RFE_API_IRQ_CONFIGURE, [82](#)
 - RFE_API_IRQ_CONTINUOUS_WAVE_TRANSMISSION_START, [83](#)
 - RFE_API_IRQ_CONTINUOUS_WAVE_TRANSMISSION_STOP, [83](#)
 - RFE_API_IRQ_GET_BIST_ZERO_HOUR_REFERENCE_DATA, [83](#)
 - RFE_API_IRQ_GET_FE_ID, [83](#)
 - RFE_API_IRQ_GET_FUSA_FAULT_STATISTICS, [83](#)
 - RFE_API_IRQ_GET_FUSA_FAULTS, [82](#)
 - RFE_API_IRQ_GET_NEXT_RADAR_CYCLE_START_TIME, [83](#)
 - RFE_API_IRQ_GET_TIME, [82](#)
 - RFE_API_IRQ_GET_VERSION, [82](#)
 - RFE_API_IRQ_MAX, [83](#)
 - RFE_API_IRQ_MONITOR_READ, [82](#)
 - RFE_API_IRQ_NONE, [82](#)
 - RFE_API_IRQ_RADAR_CYCLE_START, [82](#)
 - RFE_API_IRQ_RADAR_CYCLE_STOP, [82](#)

- RFE_API_IRQ_REG_DUMP, 83
- RFE_API_IRQ_SET_FE_ID, 83
- RFE_API_IRQ_SET_NEXT_RADAR_CYCLE_START_TIME, 86
- RFE_API_IRQ_TEST_GET_INTERNAL_ERROR, 83
- RFE_API_IRQ_TEST_SET_PARAM, 83
- RFE_API_IRQ_UPDATE_PUSH, 83
- rfe_apiIRQ_t, 82
- rfe_apiWaitForInterrupt_f, 81
- rfe_bistInterval_everyRadarCycle_e, 83
- rfe_bistInterval_none_e, 83
- rfe_bistInterval_onceIn2RadarCycles_e, 83
- rfe_bistInterval_onceIn4RadarCycles_e, 83
- rfe_bistInterval_t, 83
- rfe_chirpPllVco_1GHz_e, 84
- rfe_chirpPllVco_2GHz_e, 84
- rfe_chirpPllVco_4GHz_e, 84
- rfe_chirpPllVco_t, 84
- rfe_chirpProfileIndex_0_e, 84
- rfe_chirpProfileIndex_1_e, 84
- rfe_chirpProfileIndex_2_e, 84
- rfe_chirpProfileIndex_3_e, 84
- rfe_chirpProfileIndex_4_e, 84
- rfe_chirpProfileIndex_5_e, 84
- rfe_chirpProfileIndex_6_e, 84
- rfe_chirpProfileIndex_7_e, 84
- rfe_chirpProfileIndex_invalid_e, 84
- rfe_chirpProfileIndex_t, 84
- rfe_chirpSequenceIndex_0_e, 84
- rfe_chirpSequenceIndex_1_e, 84
- rfe_chirpSequenceIndex_2_e, 85
- rfe_chirpSequenceIndex_3_e, 85
- rfe_chirpSequenceIndex_4_e, 85
- rfe_chirpSequenceIndex_5_e, 85
- rfe_chirpSequenceIndex_6_e, 85
- rfe_chirpSequenceIndex_7_e, 85
- rfe_chirpSequenceIndex_invalid_e, 85
- rfe_chirpSequenceIndex_t, 84
- rfe_chirpSlopeDirection_falling_e, 85
- rfe_chirpSlopeDirection_rising_e, 85
- rfe_chirpSlopeDirection_t, 85
- rfe_configure, 121
- rfe_configureInterrupt, 122
- rfe_coreId_a53_e, 85
- rfe_coreId_appM7_e, 85
- rfe_coreId_et, 85
- rfe_coreId_max, 85
- rfe_coreId_none_e, 85
- rfe_coreId_t, 81
- rfe_dataOutDest_t, 81
- rfe_dynParamsSelect_t, 81
- rfe_error_1_e, 87
- rfe_error_2_e, 87
- rfe_error_3_e, 87
- rfe_error_addressOutOfValidSharedMemory_e, 87
- rfe_error_api_blob_crc_wrong_e, 87
- rfe_error_api_busy_e, 86
- rfe_error_api_cfgInvalidParameterSize_e, 89
- rfe_error_api_cfgInvalidParameterSize_rfeDriverOnly_e, 86
- rfe_error_api_cfgSectionMismatch_e, 89
- rfe_error_api_cfgSectionMismatch_external_e, 86
- rfe_error_api_configParamCrossCheck_acqJumpbackResetTimeTooShort_e, 86
- rfe_error_api_configParamCrossCheck_chirpIntervalTimeTooShort_e, 86
- rfe_error_api_configParamCrossCheck_dwellSettleTimeTooShort_e, 86
- rfe_error_api_configParamCrossCheck_unconfiguredChirpProfile_e, 87
- rfe_error_api_configParamCrossCheck_unconfiguredChirpSequence_e, 86
- rfe_error_api_configParamCrossCheck_unconfiguredDynamicTable_e, 87
- rfe_error_api_dynamicTablesNotConfigured_e, 86
- rfe_error_api_fsm_was_started_e, 87
- rfe_error_api_function_not_implemented_e, 87
- rfe_error_api_invalid_frontends_conf_e, 87
- rfe_error_api_invalid_num_frontends_e, 87
- rfe_error_api_invalidArgumentValue_e, 86
- rfe_error_api_invalidConfigurationParameterValue_e, 86
- rfe_error_api_invalidConfigurationSize_e, 86
- rfe_error_api_invalidConfigurationVersion_e, 87
- rfe_error_api_invalidDynamicTableSize_e, 86
- rfe_error_api_invalidDynamicTableValue_e, 86
- rfe_error_api_invalidFrontendId_e, 87
- rfe_error_api_invalidParameterForUpdate_e, 86
- rfe_error_api_invalidRadarCycleTimeSchedule_e, 86
- rfe_error_api_maxUpdateCountExceeded_e, 86
- rfe_error_api_noUpdatesBuffered_e, 86
- rfe_error_api_operationNotAllowed_e, 86
- rfe_error_api_unresponsive_e, 86
- rfe_error_api_unsynchronized_e, 86
- rfe_error_api_updateAlreadyPending_e, 86
- rfe_error_atbAdcInvalidAveragingSamples_e, 90
- rfe_error_atbAdcNoCalibrated_e, 92
- rfe_error_atbAdcNotConfiguredToSample_e, 90
- rfe_error_atbAdcNotInSingleMode_e, 89
- rfe_error_atbBandGapInvalidVout_e, 89
- rfe_error_atbBandGapNotStarted_e, 89
- rfe_error_atbCrossSwitchNotEnabled_e, 89
- rfe_error_atbCurrentMeasurementWithoutResistors_e, 89
- rfe_error_atbInvalidAdcInputDiffMode_e, 89
- rfe_error_atbInvalidAdcSelSingleMode_e, 89
- rfe_error_atbInvalidAdcSingleModeGround_e, 89
- rfe_error_atbInvalidAtbAdc_e, 91
- rfe_error_atbInvalidAtbAdcGainMode_e, 91
- rfe_error_atbInvalidAtbMaster_e, 89
- rfe_error_atbInvalidInputSource_e, 92
- rfe_error_atbInvalidOperationalMode_e, 89
- rfe_error_atbInvalidPullDownInput_e, 89
- rfe_error_atbInvalidTrimmingSelectedResistor_e, 89

- 89
- rfe_error_atbModeOfOperationNotSet_e, 89
- rfe_error_atbNotSupported_e, 89
- rfe_error_bc_adcCalibrationCheckNotDone, 97
- rfe_error_bc_adcCalibrationFailed, 96
- rfe_error_bc_adcCalibrationTimeout, 97
- rfe_error_bc_algDummyError, 95
- rfe_error_bc_atbBistAdcTimeout, 99
- rfe_error_bc_atbWrongAtbNoSelected, 99
- rfe_error_bc_cafcAafcKvCodVdtTimeout, 96
- rfe_error_bc_cafcBwOutOfRange, 96
- rfe_error_bc_cafcConfigLoopFilterFail, 96
- rfe_error_bc_cafcDvdtCountWrong, 96
- rfe_error_bc_cafcFreqCntTimeout, 96
- rfe_error_bc_cafcInputOutOfRange, 96
- rfe_error_bc_cafcKvcoCountStatusErr, 96
- rfe_error_bc_cafcLoopBwOutOfRange, 96
- rfe_error_bc_cafcNoStoreInProfileSelected, 96
- rfe_error_bc_cafcPtrAddrNull, 96
- rfe_error_bc_cafcSbOutOfRange, 96
- rfe_error_bc_cafcVcoCalibAafcFail, 96
- rfe_error_bc_cafcVcoCalibKvCodVdtFail, 96
- rfe_error_bc_cafcWrongAcquisitionDuration, 96
- rfe_error_bc_cafcWrongKvco, 96
- rfe_error_bc_cafcWrongProfileId, 96
- rfe_error_bc_ccPtrAddrNull, 95
- rfe_error_bc_ccSysConfNotSet, 95
- rfe_error_bc_failToInjectCrcErr, 93
- rfe_error_bc_funcNoExist, 93
- rfe_error_bc_gbiasDummy, 97
- rfe_error_bc_gldoSnsError, 99
- rfe_error_bc_gldoSnsForceError, 99
- rfe_error_bc_gldoSnsResetError, 99
- rfe_error_bc_inputOutOfRange, 93
- rfe_error_bc_ipDisabled, 93
- rfe_error_bc_ismCalBusy, 100
- rfe_error_bc_ismCalError, 100
- rfe_error_bc_ismCalSuccess, 100
- rfe_error_bc_ismDisableLbToggleTestFailed, 100
- rfe_error_bc_ismFitAdcStatusFailed, 100
- rfe_error_bc_ismFitChirpStatusFailed, 100
- rfe_error_bc_ismFitErrorN_High, 99
- rfe_error_bc_ismFitFailed, 99
- rfe_error_bc_ismFitForceErrorFailed, 99
- rfe_error_bc_ismFitForceStateChangeFailed, 99
- rfe_error_bc_ismFitMclkLolismOtpCcStatusFailed, 100
- 100
- rfe_error_bc_ismFitResetErrorFailed, 99
- rfe_error_bc_ismFitResetStateChangeFailed, 99
- rfe_error_bc_ismFitRxStatusFailed, 100
- rfe_error_bc_ismFitSsbSerGbGldoAAtbStatusFailed, 100
- 100
- rfe_error_bc_ismFitStatusFailed, 99
- rfe_error_bc_ismFitTxStatusFailed, 99
- rfe_error_bc_ismInputModeToggleTestFailed, 99
- rfe_error_bc_ismInvalidInput, 99
- rfe_error_bc_ismOutputModeToggleTestFailed, 99
- rfe_error_bc_ismPtrAddrNull, 99
- rfe_error_bc_ismStartupSeqConfigFailed, 99
- rfe_error_bc_loiCalBusy, 97
- rfe_error_bc_loiCalError, 97
- rfe_error_bc_loiLutGainCodeOutOfRange, 97
- rfe_error_bc_loiPtrAddrNull, 97
- rfe_error_bc_lvdsAdcClkFromInvalid, 95
- rfe_error_bc_lvdsClkEdgeInvalid, 95
- rfe_error_bc_lvdsCountPeriodInvalid, 95
- rfe_error_bc_lvdsFreqCounterDeviation, 95
- rfe_error_bc_lvdsFreqCountTimeout, 95
- rfe_error_bc_lvdsInvalidInput, 95
- rfe_error_bc_lvdsPtrAddrNull, 95
- rfe_error_bc_lvdsrestoreFreqCounter, 95
- rfe_error_bc_lvdsSamplesShiftInvalid, 95
- rfe_error_bc_mclkMspcCalibrationError, 97
- rfe_error_bc_mclkMspcTimeout, 97
- rfe_error_bc_mclkPllCalibrationFailed, 97
- rfe_error_bc_mclkPllCalibrationNotFinished, 97
- rfe_error_bc_mclkPtrAddrNull, 97
- rfe_error_bc_mipiDeviceReadyTimeout, 95
- rfe_error_bc_mipiFreqCntOutOfRange, 95
- rfe_error_bc_mipiFreqCountTimeout, 95
- rfe_error_bc_mipiPtrAddrNull, 95
- rfe_error_bc_mipiRestoreFreqCounter, 95
- rfe_error_bc_otpAllMemDisabled, 99
- rfe_error_bc_otpCrcCheckFailure, 99
- rfe_error_bc_otpFailToRead, 99
- rfe_error_bc_otpPlpsDisabled, 99
- rfe_error_bc_otpOtplsNotLoaded, 99
- rfe_error_bc_readFail, 95
- rfe_error_bc_rx1BbdEnableWritecheck, 98
- rfe_error_bc_rx1CalBusy, 97
- rfe_error_bc_rx1CalError, 97
- rfe_error_bc_rx1FusaEnableWritecheck, 97
- rfe_error_bc_rx1Rx2BbdEnableWritecheck, 98
- rfe_error_bc_rx1Rx2CalError, 97
- rfe_error_bc_rx1Rx2FusaEnableWritecheck, 97
- rfe_error_bc_rx1Rx2Rx3BbdEnableWritecheck, 98
- rfe_error_bc_rx1Rx2Rx3CalError, 97
- rfe_error_bc_rx1Rx2Rx3FusaEnableWritecheck, 97
- rfe_error_bc_rx1Rx2Rx3Rx4BbdEnableWritecheck, 98
- rfe_error_bc_rx1Rx2Rx3Rx4CalError, 97
- rfe_error_bc_rx1Rx2Rx3Rx4FusaEnableWritecheck, 98
- rfe_error_bc_rx1Rx2Rx4BbdEnableWritecheck, 98
- rfe_error_bc_rx1Rx2Rx4CalError, 97
- rfe_error_bc_rx1Rx2Rx4FusaEnableWritecheck, 97
- rfe_error_bc_rx1Rx3BbdEnableWritecheck, 98
- rfe_error_bc_rx1Rx3CalError, 97
- rfe_error_bc_rx1Rx3FusaEnableWritecheck, 97
- rfe_error_bc_rx1Rx3Rx4BbdEnableWritecheck, 98
- rfe_error_bc_rx1Rx3Rx4CalError, 97
- rfe_error_bc_rx1Rx3Rx4FusaEnableWritecheck, 98
- rfe_error_bc_rx1Rx4BbdEnableWritecheck, 98

- rfe_error_bc_rx1Rx4CalError, 97
- rfe_error_bc_rx1Rx4FusaEnableWritecheck, 97
- rfe_error_bc_rx2BbdEnableWritecheck, 98
- rfe_error_bc_rx2CalBusy, 97
- rfe_error_bc_rx2CalError, 97
- rfe_error_bc_rx2FusaEnableWritecheck, 97
- rfe_error_bc_rx2Rx3BbdEnableWritecheck, 98
- rfe_error_bc_rx2Rx3CalError, 97
- rfe_error_bc_rx2Rx3FusaEnableWritecheck, 97
- rfe_error_bc_rx2Rx3Rx4BbdEnableWritecheck, 98
- rfe_error_bc_rx2Rx3Rx4CalError, 97
- rfe_error_bc_rx2Rx3Rx4FusaEnableWritecheck, 98
- rfe_error_bc_rx2Rx4BbdEnableWritecheck, 98
- rfe_error_bc_rx2Rx4CalError, 97
- rfe_error_bc_rx2Rx4FusaEnableWritecheck, 97
- rfe_error_bc_rx3BbdEnableWritecheck, 98
- rfe_error_bc_rx3CalBusy, 97
- rfe_error_bc_rx3CalError, 97
- rfe_error_bc_rx3FusaEnableWritecheck, 97
- rfe_error_bc_rx3Rx4BbdEnableWritecheck, 98
- rfe_error_bc_rx3Rx4CalError, 97
- rfe_error_bc_rx3Rx4FusaEnableWritecheck, 98
- rfe_error_bc_rx4BbdEnableWritecheck, 98
- rfe_error_bc_rx4CalBusy, 97
- rfe_error_bc_rx4CalError, 97
- rfe_error_bc_rx4FusaEnableWritecheck, 97
- rfe_error_bc_rxBbdEnableWritecheckBase, 98
- rfe_error_bc_rxCalBusy, 97
- rfe_error_bc_rxCalError, 97
- rfe_error_bc_rxFusaEnableWriteCheckBase, 97
- rfe_error_bc_rxInputOutOfRange, 97
- rfe_error_bc_rxPtrAddrNull, 97
- rfe_error_bc_rxWrngCalTargetOffset, 97
- rfe_error_bc_sclInputFreqDriftInvalid, 96
- rfe_error_bc_scOperatingFreqOutOfRange, 96
- rfe_error_bc_scProfile, 96
- rfe_error_bc_scPtrAddrNull, 96
- rfe_error_bc_scRampTimeOutOfRange, 96
- rfe_error_bc_scResetTimeOutOfRange, 96
- rfe_error_bc_scUserBwOutOfRange, 96
- rfe_error_bc_spiWriteCheckFail, 95
- rfe_error_bc_spiWriteReadCrcError, 95
- rfe_error_bc_ssbInputOutOfRange, 99
- rfe_error_bc_ssbSignalNotExistOnSelectedAtb, 99
- rfe_error_bc_sys_aafcAutoNotSupported, 94
- rfe_error_bc_sys_adclInvalid, 93
- rfe_error_bc_sys_cafcIntegrityCheckFailed, 94
- rfe_error_bc_sys_cafcIntegrityCheckLevel, 94
- rfe_error_bc_sys_cafcIntegrityCheckUnlock, 94
- rfe_error_bc_sys_cafcTempReadTimeout, 94
- rfe_error_bc_sys_cannotFindCafcDacCode, 93
- rfe_error_bc_sys_error_N, 94
- rfe_error_bc_sys_errorRecoveryFailed, 93
- rfe_error_bc_sys_faultInjectionFail, 94
- rfe_error_bc_sys_fileError, 93
- rfe_error_bc_sys_invalidInput, 93
- rfe_error_bc_sys_invalidProfile, 94
- rfe_error_bc_sys_invalidResetTime, 94
- rfe_error_bc_sys_ipHasNoAtbConnection, 93
- rfe_error_bc_sys_jsonStringNoExisted, 93
- rfe_error_bc_sys_loilnBallBreakDetected, 95
- rfe_error_bc_sys_loiOutBallBreakDetected, 95
- rfe_error_bc_sys_mclkIntegrityFailed, 93
- rfe_error_bc_sys_ptrAddrNull, 94
- rfe_error_bc_sys_rawNumSampleInvalid, 94
- rfe_error_bc_sys_regCrcIsModuleErrorSet, 94
- rfe_error_bc_sys_regCrcTriggerFailed, 94
- rfe_error_bc_sys_restoreChirpGlobalFailed, 94
- rfe_error_bc_sys_slaveWrongAppConfig, 94
- rfe_error_bc_sys_spiAccessCheckFail, 94
- rfe_error_bc_sys_subBandIncOutOfRange, 94
- rfe_error_bc_sys_totalBwOutOfRange, 94
- rfe_error_bc_sys_tx1CtrlEnableWriteCheck, 94
- rfe_error_bc_sys_tx1Tx2CtrlEnableWriteCheck, 94
- rfe_error_bc_sys_tx1Tx2Tx3CtrlEnableWriteCheck, 95
- rfe_error_bc_sys_tx1Tx3CtrlEnableWriteCheck, 95
- rfe_error_bc_sys_tx2CtrlEnableWriteCheck, 94
- rfe_error_bc_sys_tx2Tx3CtrlEnableWriteCheck, 95
- rfe_error_bc_sys_tx3CtrlEnableWriteCheck, 94
- rfe_error_bc_sys_txCtrlEnableWriteCheckBase, 94
- rfe_error_bc_sys_txTempReadTimeout, 93
- rfe_error_bc_sys_wrongAppconfig, 93
- rfe_error_bc_sys_wrongDataInterfaceConfig, 94
- rfe_error_bc_sys_wrongMidNoRecovery, 94
- rfe_error_bc_sys_wrongProfileNoRecovery, 93
- rfe_error_bc_teBiasDelay, 95
- rfe_error_bc_teChirpEnableWriteCheckFailed, 96
- rfe_error_bc_teChirpInterval, 96
- rfe_error_bc_teChirpSeqInterval, 95
- rfe_error_bc_teChirpSequence, 96
- rfe_error_bc_teChirpTrgMode, 96
- rfe_error_bc_teDcPowOnDelay, 95
- rfe_error_bc_teDcSafetyDelay, 95
- rfe_error_bc_teDdmaMode, 95
- rfe_error_bc_teDdmaSetting, 95
- rfe_error_bc_teDelayFromRept, 96
- rfe_error_bc_teDwllTime, 96
- rfe_error_bc_teFineGrpDelay, 95
- rfe_error_bc_teFreqCntTimeout, 96
- rfe_error_bc_teHpfResetDelay, 95
- rfe_error_bc_teHpfResetWrongTriggerPoint, 96
- rfe_error_bc_teInputNull, 95
- rfe_error_bc_teJumpBack, 95
- rfe_error_bc_teOutputSampRate, 96
- rfe_error_bc_tePhaseOutOfRange, 96
- rfe_error_bc_tePrCalibDelay, 95
- rfe_error_bc_tePrCalTimeout, 95
- rfe_error_bc_tePrfRepeatCnt, 95
- rfe_error_bc_teProfileList, 95
- rfe_error_bc_tePrSafetyDelay, 95
- rfe_error_bc_teRxActiveDelay, 95
- rfe_error_bc_teSettleTime, 95
- rfe_error_bc_teTx1CalEnableWriteCheck, 96

- rfe_error_bc_tetx1Tx2CalEnableWriteCheck, 96
- rfe_error_bc_tetx1Tx2Tx3CalEnableWriteCheck, 96
- rfe_error_bc_tetx1Tx3CalEnableWriteCheck, 96
- rfe_error_bc_tetx2CalEnableWriteCheck, 96
- rfe_error_bc_tetx2Tx3CalEnableWriteCheck, 96
- rfe_error_bc_tetx3CalEnableWriteCheck, 96
- rfe_error_bc_tetxCalEnableWriteCheckBase, 96
- rfe_error_bc_tetxRxGrpDelay, 95
- rfe_error_bc_tetVirtualChannel, 96
- rfe_error_bc_tx1BbdEnableWritecheck, 99
- rfe_error_bc_tx1CalBusy, 98
- rfe_error_bc_tx1CalError, 98
- rfe_error_bc_tx1FusaEnableWriteCheck, 99
- rfe_error_bc_tx1SafetyBusy, 98
- rfe_error_bc_tx1SafetyError, 98
- rfe_error_bc_tx1tx2BbdEnableWritecheck, 99
- rfe_error_bc_tx1Tx2CalError, 98
- rfe_error_bc_tx1Tx2FusaEnableWriteCheck, 99
- rfe_error_bc_tx1Tx2SafetyError, 98
- rfe_error_bc_tx1Tx2Tx3BbdEnableWritecheck, 99
- rfe_error_bc_tx1Tx2Tx3CalError, 98
- rfe_error_bc_tx1Tx2Tx3FusaEnableWriteCheck, 99
- rfe_error_bc_tx1Tx2Tx3SafetyError, 98
- rfe_error_bc_tx1Tx3BbdEnableWritecheck, 99
- rfe_error_bc_tx1Tx3CalError, 98
- rfe_error_bc_tx1Tx3FusaEnableWriteCheck, 99
- rfe_error_bc_tx1Tx3SafetyError, 98
- rfe_error_bc_tx2BbdEnableWritecheck, 99
- rfe_error_bc_tx2CalBusy, 98
- rfe_error_bc_tx2CalError, 98
- rfe_error_bc_tx2FusaEnableWriteCheck, 99
- rfe_error_bc_tx2SafetyBusy, 98
- rfe_error_bc_tx2SafetyError, 98
- rfe_error_bc_tx2Tx3BbdEnableWritecheck, 99
- rfe_error_bc_tx2Tx3CalError, 98
- rfe_error_bc_tx2Tx3FusaEnableWriteCheck, 99
- rfe_error_bc_tx2Tx3SafetyError, 98
- rfe_error_bc_tx3BbdEnableWritecheck, 99
- rfe_error_bc_tx3CalBusy, 98
- rfe_error_bc_tx3CalError, 98
- rfe_error_bc_tx3FusaEnableWriteCheck, 99
- rfe_error_bc_tx3SafetyBusy, 98
- rfe_error_bc_tx3SafetyError, 98
- rfe_error_bc_txBbdEnableWritecheckBase, 99
- rfe_error_bc_txCalBusy, 98
- rfe_error_bc_txCalError, 98
- rfe_error_bc_txCalibBiasCurrWriteFailed, 99
- rfe_error_bc_txCalibTimeout, 99
- rfe_error_bc_txFusaEnableWriteCheckBase, 99
- rfe_error_bc_txInputOutOfRange, 98
- rfe_error_bc_txNoStoreInProfileSelected, 99
- rfe_error_bc_txPoutVgaReduceTxNotSupported, 99
- rfe_error_bc_txPrcAdcTimeout, 98
- rfe_error_bc_txPtrAddrNull, 98
- rfe_error_bc_txRfMinCodeOutOfRange, 99
- rfe_error_bc_txSafetyBusy, 98
- rfe_error_bc_txSafetyError, 98
- rfe_error_bc_txSensorFitFail, 98
- rfe_error_bc_wrongInterface, 93
- rfe_error_bistZeroHourDataNotGenerated_e, 92
- rfe_error_boundedWaitNotValidResource_e, 90
- rfe_error_boundedWaitResourcesExhausted_e, 89
- rfe_error_chirpPllBandgapNotOk_e, 91
- rfe_error_chirpPllHvLdoCpNotOk_e, 91
- rfe_error_chirpPllHvLdoVcoNotOk_e, 91
- rfe_error_chirpPllLoopfilterC1NotOk_e, 91
- rfe_error_chirpPllLoopfilterC2FastNotOk_e, 91
- rfe_error_chirpPllLoopfilterC2SlowNotOk_e, 91
- rfe_error_chirpPllLoopfilterC3FastNotOk_e, 91
- rfe_error_chirpPllLoopfilterC3SlowNotOk_e, 91
- rfe_error_chirpPllLoopfilterCpNotOk_e, 91
- rfe_error_chirpPllLoopfilterR1NotOk_e, 91
- rfe_error_chirpPllLoopfilterR2NotOk_e, 91
- rfe_error_chirpPllLvLdoNotOk_e, 91
- rfe_error_chirpPllPdivLdoNotOk_e, 91
- rfe_error_chirpPllVcoCalAmpOutOfRange_e, 92
- rfe_error_chirpPllVcoCalibrationFailed_e, 89
- rfe_error_chirpStartedTooEarly_e, 91
- rfe_error_cmdInterface_invalidCmdId_e, 87
- rfe_error_cmdInterface_invalidCmdLength_e, 87
- rfe_error_cmdInterface_invalidConfigCmdSequencing_e, 87
- rfe_error_cmdInterface_invalidCrc_e, 87
- rfe_error_cmdInterface_invalidResponse_e, 87
- rfe_error_cmdInterface_invalidRspLength_e, 87
- rfe_error_cmdServerDft_invalidParam_e, 87
- rfe_error_cmdServerInternal_cmdbuf_overflow_e, 88
- rfe_error_cmdServerInternal_no_cmd_received_e, 87
- rfe_error_cmdServerInternal_out_of_seq_e, 88
- rfe_error_cmdServerInternal_resplen_wrong_e, 88
- rfe_error_cmdServerInternal_undefined_cmd_handler_e, 87
- rfe_error_csi2PllcalibrationTimeOut_e, 92
- rfe_error_csi2PllLockTimeOut_e, 92
- rfe_error_dftInvalidOutputSelected_e, 88
- rfe_error_dftNotSupported_e, 89
- rfe_error_dftNumberOfAveragesOutOfBounds_e, 88
- rfe_error_dftOnlySingleBinSupported_e, 89
- rfe_error_dftRealOnlySupported_e, 89
- rfe_error_dftSamplesCountAndNumberOfAveragesNotMultiples_e, 89
- rfe_error_dftSamplesCountNot2Multiple_e, 88
- rfe_error_didt_test_failure_e, 92
- rfe_error_distrGroupDoesNotExist_e, 90
- rfe_error_dynamicProgrammingDuringAcquisition_e, 90
- rfe_error_fcc_e, 92
- rfe_error_fftInvalidOutputSelected_e, 88
- rfe_error_fftNotSupported_e, 88

- rfe_error_fftNumberOfAveragesOutOfBounds_e, 88
- rfe_error_fftSamplesCountAndNumberOfAveragesNotMultiple_e, 88
- rfe_error_fftSamplesCountNot2Multiple_e, 88
- rfe_error_fic1ProcedureFailed_e, 88
- rfe_error_fic2ProcedureFailed_e, 88
- rfe_error_fit1ProcedureFailed_e, 88
- rfe_error_fit2ProcedureFailed_e, 88
- rfe_error_histogramWrongOutputSize_e, 88
- rfe_error_histogramWrongXAxisLimit_e, 88
- rfe_error_hwPitNotInitialized_e, 90
- rfe_error_HwRegister_lock_e, 92
- rfe_error_ic_wrg_input_params_e, 100
- rfe_error_initLldoPdcPowerNotOk_e, 92
- rfe_error_initMcgen0v9AnaNotOk_e, 92
- rfe_error_initRtmErrorReported, 92
- rfe_error_invalidADCNumberOfBits_e, 88
- rfe_error_invalidFlexIOConfig_e, 88
- rfe_error_invalidFractionBits_e, 88
- rfe_error_invalidPointer_e, 87
- rfe_error_invalidRadarCycleTiming_e, 90
- rfe_error_invalidRfeAccessCsrConfig_e, 88
- rfe_error_invalidSamplesNumber_e, 87
- rfe_error_invalidShiftingParam_e, 88
- rfe_error_invalidValue_e, 88
- rfe_error_lpUartNotActive_e, 92
- rfe_error_mainFsm_chirpAcquisitionStartInterruptTimeout_e, 91
- rfe_error_mainFsm_chirpSequenceEndInterruptTimeout_e, 91
- rfe_error_mainFsm_chirpStartInterruptTimeout_e, 91
- rfe_error_mainFsm_eventAlreadyActive_e, 90
- rfe_error_mainFsm_eventNotAllowed_e, 90
- rfe_error_mainFsm_mcgenAdpllNotLocked_e, 91
- rfe_error_mcgen0v9AnaNotOk_e, 92
- rfe_error_mcgenAdpllLockDetNotLocked_e, 92
- rfe_error_mcgenAdpllOutVdd0v9DigNotOk_e, 92
- rfe_error_mcgenXoDetPllClkError_e, 92
- rfe_error_none_e, 86
- rfe_error_noSpaceLeftInSharedMemory_e, 88
- rfe_error_notAnExternalErrorCode_e, 88
- rfe_error_notYetImplemented_e, 88
- rfe_error_nullPointer_e, 88
- rfe_error_param_invalidParam_e, 89
- rfe_error_param_invalidValue_e, 89
- rfe_error_parameterOutOfRange_e, 87
- rfe_error_pcdCorrelatorIntegrationTimeInvalid_e, 90
- rfe_error_pcdIndexInvalid_e, 90
- rfe_error_pcdPhaseCorrectionInvalid_e, 90
- rfe_error_pcdProfileIndexInvalid_e, 90
- rfe_error_pcdSamplingFrequencyInvalid_e, 90
- rfe_error_pcdSlectedInputNotImplemented_e, 90
- rfe_error_ppdVoutAboveMax_e, 92
- rfe_error_radarCycleFsm_stateEndTimeExceeded_e, 90
- rfe_error_registerComparisonFailed_e, 92
- rfe_error_rfeFuSaFault_e, 87
- rfe_error_rfeInErrorState_e, 87
- rfe_error_rx1SaturationStatusReadingFailure_e, 90
- rfe_error_rx2SaturationStatusReadingFailure_e, 90
- rfe_error_rx3SaturationStatusReadingFailure_e, 90
- rfe_error_rx4SaturationStatusReadingFailure_e, 90
- rfe_error_rxAdcInputUnidentified_e, 91
- rfe_error_rxAdcModuleIdUnidentified_e, 91
- rfe_error_rxbist_frequency_fail, 100
- rfe_error_rxbist_Ina_gain_fail_e, 100
- rfe_error_rxbist_Ina_phase_fail_e, 100
- rfe_error_rxbist_missing_signal, 100
- rfe_error_rxbist_mixer_gain_fail_e, 100
- rfe_error_rxbist_mixer_phase_fail_e, 100
- rfe_error_rxbistInvalidSsbCode_e, 93
- rfe_error_rxbistParameterOutOfRange_e, 92
- rfe_error_rxCalibrationIfStageInvalid_e, 92
- rfe_error_rxCalibrationTargetInvalid_e, 92
- rfe_error_rxlIfCalibrationFailed, 93
- rfe_error_rxlIfCalibrationInvalidChannel, 93
- rfe_error_rxlIfCalibrationInvalidRcCode_e, 93
- rfe_error_rxlIfCalibrationInvalidSettings, 93
- rfe_error_rxInvalidRcCode_e, 93
- rfe_error_rxTargetSettingsCombinationIsNotSupported_e, 92
- rfe_error_saturationInOperation_e, 87
- rfe_error_sharedData_invalidIndex_e, 87
- rfe_error_sharedData_redundancyCheckFailed_e, 87
- rfe_error_sharedMemoryAdresOverlapping_e, 87
- rfe_error_spiTransactionError_e, 92
- rfe_error_spiTransactionTimeout_e, 92
- rfe_error_startup_seq_failed_e, 92
- rfe_error_sys_FE_chip_es2_not_supported, 100
- rfe_error_sys_spi_invalid_len, 100
- rfe_error_sys_spi_wrg_unit_id, 100
- rfe_error_sys_timer_cm7_init, 100
- rfe_error_t, 85
- rfe_error_testBufferOverflow_e, 92
- rfe_error_timerInUse_e, 89
- rfe_error_tSenseConversionInProgress_e, 91
- rfe_error_tSenseDivisionByZero_e, 91
- rfe_error_tSenseInvalidHysteresisThreshold_e, 91
- rfe_error_tSenseInvalidModeOfOperation_e, 91
- rfe_error_tSenseInvalidOverUndeThresholds_e, 91
- rfe_error_tSenseInvalidResolutionTime_e, 91
- rfe_error_tSenseInvalidWarningThreshold_e, 91
- rfe_error_txGainBuf2AExceedRange_e, 90
- rfe_error_txGainBuf2BCExceedRange_e, 90
- rfe_error_txIndexInvalid_e, 90
- rfe_error_txInvalidAdcConnection_e, 90
- rfe_error_txInvalidCalibrationFSMState_e, 90

- rfe_error_txInvalidPowerProfile_e, 90
- rfe_error_txInvalidSubcomponent_e, 90
- rfe_error_txPaGainCExceedRange_e, 90
- rfe_error_txPrCalibrationTimeout_e, 92
- rfe_error_txPRPhaseExceedRange_e, 90
- rfe_error_undefined_e, 87
- rfe_error_uninitialized_e, 88
- rfe_error_unknown_ip_e, 87
- rfe_error_utils_memcpy_range_error_e, 88
- rfe_error_wdmaAxiBurstLengthNotSet_e, 93
- rfe_error_wdmaAxiBurstModeNotSet_e, 93
- rfe_error_wdmaBufferAddressNotAligned_e, 92
- rfe_error_wdmaBufferAddressNotSet_e, 93
- rfe_error_wdmaBufferLengthNotSet_e, 93
- rfe_error_wdmaBufferOutOfBounds_e, 92
- rfe_error_wdmaBufferOverwriteModeNotSet_e, 93
- rfe_error_wdmaBurstLengthOutOfBounds_e, 92
- rfe_error_wdmaIncorrectInterruptSourceSelected_e, 93
- rfe_error_wdmaIncorrectNumberOfStreamsSelected_e, 93
- rfe_error_wdmaIncorrectStreamSelected_e, 92
- rfe_error_wdmaWrongValueForaxiBurstModeField_e, 93
- RFE_EVENTS_IRQ_CHIRP_SEQUENCE_CHANGE, 100
- RFE_EVENTS_IRQ_MAX, 100
- RFE_EVENTS_IRQ_NONE, 100
- RFE_EVENTS_IRQ_RADAR_CYCLE_CHANGE, 100
- RFE_EVENTS_IRQ_STATE_CHANGE, 100
- rfe_eventsIRQ_t, 100
- rfe_fuSaFault_R1_hardClippingFlagPersistentAdc1_e, 101
- rfe_fuSaFault_R1_hardClippingFlagPersistentAdc2_e, 101
- rfe_fuSaFault_R1_hardClippingFlagPersistentAdc3_e, 101
- rfe_fuSaFault_R1_hardClippingFlagPersistentAdc4_e, 101
- rfe_fuSaFault_R1_levelHighErrorFlagChirp_e, 101
- rfe_fuSaFault_R1_levelLowErrorFlagChirp_e, 101
- rfe_fuSaFault_R1_loiLevelHighErrorFlagInterface_e, 101
- rfe_fuSaFault_R1_loiLevelLowErrorFlagInterface_e, 101
- rfe_fuSaFault_R1_loLevelHighErrorFlagRx1_e, 101
- rfe_fuSaFault_R1_loLevelHighErrorFlagRx2_e, 101
- rfe_fuSaFault_R1_loLevelHighErrorFlagRx3_e, 101
- rfe_fuSaFault_R1_loLevelHighErrorFlagRx4_e, 101
- rfe_fuSaFault_R1_loLevelLowErrorFlagRx1_e, 101
- rfe_fuSaFault_R1_loLevelLowErrorFlagRx2_e, 101
- rfe_fuSaFault_R1_loLevelLowErrorFlagRx3_e, 101
- rfe_fuSaFault_R1_loLevelLowErrorFlagRx4_e, 101
- rfe_fuSaFault_R1_pllFreqCalErrorFlagMc_e, 101
- rfe_fuSaFault_R1_rfLevelHighErrorFlagTx1_e, 101
- rfe_fuSaFault_R1_rfLevelHighErrorFlagTx2_e, 101
- rfe_fuSaFault_R1_rfLevelHighErrorFlagTx3_e, 101
- rfe_fuSaFault_R1_rfLevelLowErrorFlagTx1_e, 101
- rfe_fuSaFault_R1_rfLevelLowErrorFlagTx2_e, 101
- rfe_fuSaFault_R1_rfLevelLowErrorFlagTx3_e, 101
- rfe_fusaFault_R1_size_e, 101
- rfe_fusaFault_R1_t, 100
- rfe_fuSaFault_R1_tempErrorFlagChirp_e, 101
- rfe_fuSaFault_R1_tempErrorFlagTx1_e, 101
- rfe_fuSaFault_R1_tempErrorFlagTx2_e, 101
- rfe_fuSaFault_R1_tempErrorFlagTx3_e, 101
- rfe_fuSaFault_R1_vtuneHighErrorFlagChirp_e, 101
- rfe_fuSaFault_R1_vtuneLowErrorFlagChirp_e, 101
- rfe_fuSaFault_R2_sr11_xo_no_clock_mclk_e, 104
- rfe_fuSaFault_R2_sr13_input_mode_toggle_test_sw_e, 105
- rfe_fuSaFault_R2_sr14_disable_lb_toggle_test_sw_e, 105
- rfe_fuSaFault_R2_sr17_bb_lo_in_loi_e, 104
- rfe_fuSaFault_R2_sr18_bb_lo_out_loi_e, 104
- rfe_fuSaFault_R2_sr19_rd_spi_crc_e, 104
- rfe_fuSaFault_R2_sr19_wr_spi_crc_e, 104
- rfe_fuSaFault_R2_sr27_bb_tx1_e, 103
- rfe_fuSaFault_R2_sr27_bb_tx2_e, 103
- rfe_fuSaFault_R2_sr27_bb_tx3_e, 103
- rfe_fuSaFault_R2_sr31_bist_lna_gaindiff_sw_e, 105
- rfe_fuSaFault_R2_sr31_bist_lna_phasediff_sw_e, 105
- rfe_fuSaFault_R2_sr31_bist_mixer_gaindiff_sw_e, 105
- rfe_fuSaFault_R2_sr31_bist_mixer_phasediff_sw_e, 105
- rfe_fuSaFault_R2_sr34_cal_adc1_e, 105
- rfe_fuSaFault_R2_sr34_cal_adc2_e, 105
- rfe_fuSaFault_R2_sr34_cal_adc3_e, 105
- rfe_fuSaFault_R2_sr34_cal_adc4_e, 104
- rfe_fuSaFault_R2_sr35_pll_lock_mclk_e, 104
- rfe_fuSaFault_R2_sr36_mspc_mclk_e, 104
- rfe_fuSaFault_R2_sr37_pll_level_mclk_e, 105
- rfe_fuSaFault_R2_sr39_dig_freq_mclk_e, 104
- rfe_fuSaFault_R2_sr3_reg_crc_e, 105
- rfe_fuSaFault_R2_sr42_vco_freq_480_chirp_e, 104
- rfe_fuSaFault_R2_sr43_unlock_chirp_e, 104
- rfe_fuSaFault_R2_sr44_cafc_integrity_level_sw_e, 105
- rfe_fuSaFault_R2_sr44_cafc_integrity_sw_e, 105
- rfe_fuSaFault_R2_sr44_cafc_integrity_unlock_sw_e, 105
- rfe_fuSaFault_R2_sr45_level_max_chirp_e, 104
- rfe_fuSaFault_R2_sr45_level_min_chirp_e, 104
- rfe_fuSaFault_R2_sr46_aafc_cal_timeout_chirp_e, 103
- rfe_fuSaFault_R2_sr46_aafc_chirp_e, 103

- rfe_fuSaFault_R2_sr47_lo_level_max_rx1_e, [103](#)
- rfe_fuSaFault_R2_sr47_lo_level_max_rx2_e, [103](#)
- rfe_fuSaFault_R2_sr47_lo_level_max_rx3_e, [103](#)
- rfe_fuSaFault_R2_sr47_lo_level_max_rx4_e, [103](#)
- rfe_fuSaFault_R2_sr47_lo_level_min_rx1_e, [103](#)
- rfe_fuSaFault_R2_sr47_lo_level_min_rx2_e, [103](#)
- rfe_fuSaFault_R2_sr47_lo_level_min_rx3_e, [103](#)
- rfe_fuSaFault_R2_sr47_lo_level_min_rx4_e, [103](#)
- rfe_fuSaFault_R2_sr48_level_max_loi_e, [104](#)
- rfe_fuSaFault_R2_sr48_level_min_loi_e, [104](#)
- rfe_fuSaFault_R2_sr57_supply_high_1v1_gldo_e, [104](#)
- rfe_fuSaFault_R2_sr59_supply_low_1v1_gldo_e, [104](#)
- rfe_fuSaFault_R2_sr61_supply_high_1v8_gldo_e, [104](#)
- rfe_fuSaFault_R2_sr63_supply_low_1v8_gldo_e, [104](#)
- rfe_fuSaFault_R2_sr65_67_supply_low_1v1_csi2_e, [104](#)
- rfe_fuSaFault_R2_sr65_67_supply_low_1v1_ser_e, [104](#)
- rfe_fuSaFault_R2_sr65_67_supply_low_1v8_adc12_e, [104](#)
- rfe_fuSaFault_R2_sr65_67_supply_low_1v8_adc34_e, [104](#)
- rfe_fuSaFault_R2_sr65_67_supply_low_1v8_gbias_e, [104](#)
- rfe_fuSaFault_R2_sr65_67_supply_low_1v8_ser_e, [104](#)
- rfe_fuSaFault_R2_sr65_67_supply_low_1v8_ssb_e, [104](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_chirp_e, [104](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_loi_e, [104](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_rx1_e, [103](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_rx2_e, [103](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_rx3_e, [103](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_rx4_e, [103](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_tx1_e, [102](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_tx2_e, [102](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_tx3_e, [102](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_mclk_e, [104](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_rx1_e, [103](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_rx2_e, [103](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_rx3_e, [103](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_rx4_e, [103](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_tx1_e, [102](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_tx2_e, [102](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_tx3_e, [102](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_pll_1v8_chirp_e, [104](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_vco_1v8_chirp_e, [104](#)
- rfe_fuSaFault_R2_sr69_rf_level_max_tx1_e, [103](#)
- rfe_fuSaFault_R2_sr69_rf_level_max_tx2_e, [103](#)
- rfe_fuSaFault_R2_sr69_rf_level_max_tx3_e, [102](#)
- rfe_fuSaFault_R2_sr69_rf_level_min_tx1_e, [103](#)
- rfe_fuSaFault_R2_sr69_rf_level_min_tx2_e, [103](#)
- rfe_fuSaFault_R2_sr69_rf_level_min_tx3_e, [103](#)
- rfe_fuSaFault_R2_sr70_tx_integrity_sw_e, [105](#)
- rfe_fuSaFault_R2_sr72_pr_tx1_e, [102](#)
- rfe_fuSaFault_R2_sr72_pr_tx2_e, [102](#)
- rfe_fuSaFault_R2_sr72_pr_tx3_e, [102](#)
- rfe_fuSaFault_R2_sr72_tx1_pr_cal_sw_e, [105](#)
- rfe_fuSaFault_R2_sr72_tx2_pr_cal_sw_e, [105](#)
- rfe_fuSaFault_R2_sr72_tx3_pr_cal_sw_e, [105](#)
- rfe_fuSaFault_R2_sr72_tx_pr_cal_sw_e, [105](#)
- rfe_fuSaFault_R2_sr7_interface_stuck_chirp_e, [104](#)
- rfe_fuSaFault_R2_sr7_lock_step_chirp_e, [104](#)
- rfe_fuSaFault_R2_sr9_bb_rx1_e, [103](#)
- rfe_fuSaFault_R2_sr9_bb_rx2_e, [103](#)
- rfe_fuSaFault_R2_sr9_bb_rx3_e, [103](#)
- rfe_fuSaFault_R2_sr9_bb_rx4_e, [103](#)
- rfe_fuSaFault_t, [102](#)
- rfe_fuSaFaultMaskIndex_0_e, [105](#)
- rfe_fuSaFaultMaskIndex_10_e, [105](#)
- rfe_fuSaFaultMaskIndex_11_e, [105](#)
- rfe_fuSaFaultMaskIndex_1_e, [105](#)
- rfe_fuSaFaultMaskIndex_2_e, [105](#)
- rfe_fuSaFaultMaskIndex_3_e, [105](#)
- rfe_fuSaFaultMaskIndex_4_e, [105](#)
- rfe_fuSaFaultMaskIndex_5_e, [105](#)
- rfe_fuSaFaultMaskIndex_6_e, [105](#)
- rfe_fuSaFaultMaskIndex_7_e, [105](#)
- rfe_fuSaFaultMaskIndex_8_e, [105](#)
- rfe_fuSaFaultMaskIndex_9_e, [105](#)
- rfe_fuSaFaultMaskIndex_t, [105](#)
- rfe_generic_sw_e, [105](#)
- rfe_getBistZeroHourReferenceData, [124](#)
- rfe_getFrontEnd, [125](#)
- rfe_getFuSaFaults, [125](#)
- rfe_getFuSaFaultStatistics, [126](#)
- rfe_getNextRadarCycleStartTime, [127](#)
- rfe_getRadarCycleCount, [128](#)
- rfe_getRegisterDump, [129](#)
- rfe_getState, [129](#)
- rfe_getTime, [130](#)
- rfe_getVersion, [131](#)

rfe_monitorRead, 132	rfe_paramUpdate_chirpSequence_chirpProfileSequence_5_e, 108
rfe_monitorSelect_t, 81	rfe_paramUpdate_chirpSequence_chirpProfileSequence_6_e, 108
rfe_paramUpdate_chirpProfile_acquisitionTimeTicks_e, 109	rfe_paramUpdate_chirpSequence_chirpProfileSequence_7_e, 108
rfe_paramUpdate_chirpProfile_centerFrequency_e, 109	rfe_paramUpdate_chirpSequence_chirpProfileSequenceLength_e, 108
rfe_paramUpdate_chirpProfile_chirpFrequencyDriftSteps_e, 109	rfe_paramUpdate_chirpSequence_dynamicTableIndex_e, 108
rfe_paramUpdate_chirpProfile_chirpIntervalTimeTicks_e, 109	rfe_paramUpdate_chirpSequence_dynamicUpdatesEnabled_e, 108
rfe_paramUpdate_chirpProfile_chirpPllLoopFilterBandwidth_e, 109	rfe_paramUpdate_chirpSequence_pdcNotchFilterEnable_e, 108
rfe_paramUpdate_chirpProfile_chirpPllVcoSelect_e, 109	rfe_paramUpdate_chirpSequence_rxEnable_e, 108
rfe_paramUpdate_chirpProfile_chirpSlopeDirection_e, 109	rfe_paramUpdate_chirpSequence_txCalibration_thresholdHigh_e, 108
rfe_paramUpdate_chirpProfile_dwellTimeTicks_e, 109	rfe_paramUpdate_chirpSequence_txCalibration_thresholdLow_e, 109
rfe_paramUpdate_chirpProfile_effectiveChirpBandwidth_e, 109	rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_e, 108
rfe_paramUpdate_chirpProfile_effectiveSamplingFrequency_e, 109	rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_tx1_init_e, 108
rfe_paramUpdate_chirpProfile_resetTimeTicks_e, 109	rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_tx1_mod_e, 108
rfe_paramUpdate_chirpProfile_rxGain_e, 109	rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_tx1_upd_e, 108
rfe_paramUpdate_chirpProfile_rxHpfcutOffFrequency_e, 109	rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_tx2_init_e, 108
rfe_paramUpdate_chirpProfile_rxLpfCutOffFrequency_e, 109	rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_tx2_mod_e, 108
rfe_paramUpdate_chirpProfile_settleTimeTicks_e, 109	rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_tx2_upd_e, 108
rfe_paramUpdate_chirpProfile_txPhaseRotation_1_e, 109	rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_tx3_init_e, 108
rfe_paramUpdate_chirpProfile_txPhaseRotation_2_e, 109	rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_tx3_mod_e, 108
rfe_paramUpdate_chirpProfile_txPhaseRotation_3_e, 109	rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_tx3_upd_e, 108
rfe_paramUpdate_chirpProfile_txPower_e, 109	rfe_paramUpdate_chirpSequence_txPhaseRotation_ddma_tx_e, 108
rfe_paramUpdate_chirpProfile_txTransmissionEnable_e, 109	rfe_paramUpdate_general_chirpSequenceActiveSignalGpio_e, 106
rfe_paramUpdate_chirpProfile_txTransmissionReferenceTime_e, 109	rfe_paramUpdate_general_clkloConfig_e, 106
rfe_paramUpdate_chirpProfile_txTransmissionTimeOffset_e, 109	rfe_paramUpdate_general_dataOutConfig_e, 106
rfe_paramUpdate_chirpProfile_virtualChannel_e, 109	rfe_paramUpdate_general_jumpbackTimeTicks_e, 106
rfe_paramUpdate_chirpSequence_chirpCount_e, 108	rfe_paramUpdate_general_metadataPacketSwContent_e, 106
rfe_paramUpdate_chirpSequence_chirpProfileSequence_0_e, 108	rfe_paramUpdate_general_metadataPacketVirtualChannel_e, 106
rfe_paramUpdate_chirpSequence_chirpProfileSequence_1_e, 108	rfe_paramUpdate_general_pdcDecimationFilter_e, 106
rfe_paramUpdate_chirpSequence_chirpProfileSequence_2_e, 108	rfe_paramUpdate_general_powerMode_e, 106
rfe_paramUpdate_chirpSequence_chirpProfileSequence_3_e, 108	rfe_paramUpdate_general_radarCycleStartSignalGpio_e, 106
rfe_paramUpdate_chirpSequence_chirpProfileSequence_4_e, 108	rfe_paramUpdate_general_txCalibration_mode_e, 106

- 107
- rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxGainDiff108_r_x1_r_x3_e, 107
- rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxGainDiff108_r_x1_r_x4_e, 107
- rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxPhaseDiff108_r_x1_r_x2_e, 107
- rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxPhaseDiff108_r_x1_r_x3_e, 107
- rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxPhaseDiff108_r_x1_r_x4_e, 107
- rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxPhaseDiff108_r_x1_r_x2_e, 107
- rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxPhaseDiff108_r_x1_r_x3_e, 107
- rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxPhaseDiff108_r_x1_r_x4_e, 107
- rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxPhaseDiff108_r_x1_r_x2_e, 107
- rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxPhaseDiff108_r_x1_r_x3_e, 107
- rfe_paramUpdate_monitorAndSafety_zeroHrRefForRxPhaseDiff108_r_x1_r_x4_e, 107
- rfe_paramUpdate_radarCycle_bistInterval_e, 108
- rfe_paramUpdate_radarCycle_chirpSequence_0_e, 107
- rfe_paramUpdate_radarCycle_chirpSequence_1_e, 107
- rfe_paramUpdate_radarCycle_chirpSequence_2_e, 107
- rfe_paramUpdate_radarCycle_chirpSequence_3_e, 107
- rfe_paramUpdate_radarCycle_chirpSequence_4_e, 107
- rfe_paramUpdate_radarCycle_chirpSequence_5_e, 107
- rfe_paramUpdate_radarCycle_chirpSequence_6_e, 108
- rfe_paramUpdate_radarCycle_chirpSequence_7_e, 108
- rfe_paramUpdate_radarCycle_chirpSequenceCount_e, 107
- rfe_paramUpdate_radarCycle_chirpSequenceStartTimeOffset_0_e, 108
- rfe_paramUpdate_radarCycle_chirpSequenceStartTimeOffset_1_e, 108
- rfe_paramUpdate_radarCycle_chirpSequenceStartTimeOffset_2_e, 108
- rfe_paramUpdate_radarCycle_chirpSequenceStartTimeOffset_3_e, 108
- rfe_paramUpdate_radarCycle_chirpSequenceStartTimeOffset_4_e, 108
- rfe_paramUpdate_radarCycle_chirpSequenceStartTimeOffset_5_e, 108
- rfe_paramUpdate_radarCycle_chirpSequenceStartTimeOffset_6_e, 108
- rfe_paramUpdate_radarCycle_chirpSequenceStartTimeOffset_7_e, 108
- rfe_paramUpdate_radarCycle_radarCycleDuration_e, 107
- rfe_paramUpdate_radarCycle_recalibrateProfileDependent_0_e, 108
- rfe_paramUpdate_radarCycle_recalibrateProfileDependent_1_e, 108
- rfe_paramUpdate_radarCycle_recalibrateProfileDependent_2_e, 108
- rfe_paramUpdate_radarCycle_recalibrateProfileDependent_3_e, 108
- rfe_paramUpdate_radarCycle_recalibrateProfileDependent_4_e, 108
- rfe_paramUpdate_radarCycle_recalibrateProfileDependent_5_e, 108
- rfe_paramUpdate_radarCycle_recalibrateProfileDependent_6_e, 108
- rfe_paramUpdate_radarCycle_recalibrateProfileDependent_7_e, 108
- rfe_paramUpdate_radarCycle_recalibrateProfileIndependent_e, 108
- rfe_paramUpdate_t, 106
- rfe_powerMode_active_e, 109
- rfe_powerMode_inactive_e, 109
- rfe_powerMode_t, 109
- rfe_PR_ADD_QPSK_TO_MSB, 110
- rfe_PR_INVALID, 110
- rfe_PR_NO_CHANGE_e, 110
- rfe_PR_REPLACE_MSB_WITH_QPSK, 110
- rfe_PRPCGenMode_e, 109
- rfe_radarCycleStart, 132
- rfe_radarCycleStop, 133
- rfe_rx1234Reset_e, 110
- rfe_rx1Reset_e, 110
- rfe_rx2Reset_e, 110
- rfe_rx3Reset_e, 110
- rfe_rx4Reset_e, 110
- rfe_rxClippingReset_t, 110
- rfe_rxGain_25dB_e, 110
- rfe_rxGain_28dB_e, 110
- rfe_rxGain_31dB_e, 110
- rfe_rxGain_34dB_e, 110
- rfe_rxGain_37dB_e, 110
- rfe_rxGain_40dB_e, 110
- rfe_rxGain_43dB_e, 110
- rfe_rxGain_46dB_e, 110
- rfe_rxGain_invalid_e, 110
- rfe_rxGain_t, 110
- rfe_rxHpfCutOffFrequency_1600kHz_e, 111
- rfe_rxHpfCutOffFrequency_200kHz_e, 111
- rfe_rxHpfCutOffFrequency_300kHz_e, 111
- rfe_rxHpfCutOffFrequency_3200kHz_e, 111
- rfe_rxHpfCutOffFrequency_400kHz_e, 111
- rfe_rxHpfCutOffFrequency_6400kHz_e, 111
- rfe_rxHpfCutOffFrequency_800kHz_e, 111
- rfe_rxHpfCutOffFrequency_invalid_e, 111
- rfe_rxHpfCutOffFrequency_t, 110
- rfe_rxIndex_1_e, 111
- rfe_rxIndex_2_e, 111
- rfe_rxIndex_3_e, 111
- rfe_rxIndex_4_e, 111
- rfe_rxIndex_t, 111
- rfe_rxLpfCutOffFrequency_12P5MHz_e, 112
- rfe_rxLpfCutOffFrequency_15MHz_e, 112
- rfe_rxLpfCutOffFrequency_20MHz_e, 112

- rfe_rxLpfCutOffFrequency_25MHz_e, 112
- rfe_rxLpfCutOffFrequency_40MHz_e, 112
- rfe_rxLpfCutOffFrequency_invalid_e, 112
- rfe_rxLpfCutOffFrequency_t, 111
- rfe_rxSaturationThreshold_0dB_e, 112
- rfe_rxSaturationThreshold_min0_87dB_e, 112
- rfe_rxSaturationThreshold_min10_44dB_e, 112
- rfe_rxSaturationThreshold_min11_31dB_e, 112
- rfe_rxSaturationThreshold_min12_18dB_e, 112
- rfe_rxSaturationThreshold_min13_05dB_e, 112
- rfe_rxSaturationThreshold_min1_74dB_e, 112
- rfe_rxSaturationThreshold_min2_61dB_e, 112
- rfe_rxSaturationThreshold_min3_48dB_e, 112
- rfe_rxSaturationThreshold_min4_35dB_e, 112
- rfe_rxSaturationThreshold_min5_22dB_e, 112
- rfe_rxSaturationThreshold_min6_09dB_e, 112
- rfe_rxSaturationThreshold_min6_96dB_e, 112
- rfe_rxSaturationThreshold_min7_83dB_e, 112
- rfe_rxSaturationThreshold_min8_70dB_e, 112
- rfe_rxSaturationThreshold_min9_57dB_e, 112
- rfe_rxSaturationThreshold_t, 112
- rfe_rxSelect_t, 81
- rfe_rxStage_1_e, 113
- rfe_rxStage_2_e, 113
- rfe_rxStage_t, 112
- rfe_setFrontEnd, 134
- rfe_setNextRadarCycleStartTime, 135
- rfe_state_busy_e, 113
- rfe_state_configured_e, 113
- rfe_state_fuSaFault_e, 113
- rfe_state_fuSaFaultRecovery_e, 113
- rfe_state_initialized_e, 113
- rfe_state_radarCycleIdle_e, 113
- rfe_state_t, 113
- rfe_state_testContinuousWaveTransmission_e, 113
- rfe_sync, 136
- rfe_sysMemAddress_t, 81
- rfe_temperature_t, 82
- rfe_temperatureSensorIndex_chirp_e, 114
- rfe_temperatureSensorIndex_t, 113
- rfe_temperatureSensorIndex_tx1_e, 114
- rfe_temperatureSensorIndex_tx2_e, 114
- rfe_temperatureSensorIndex_tx3_e, 114
- rfe_testContinuousWaveTransmissionStart, 136
- rfe_testContinuousWaveTransmissionStop, 137
- rfe_testGetInternalError, 138
- rfe_testParam_chirpPIITestPinEnable_e, 114
- rfe_testParam_enableClockRetuning_e, 114
- rfe_testParam_keepTxTransmissionEnabled_e, 114
- rfe_testParam_outputDataTestPattern_e, 114
- rfe_testParam_t, 114
- rfe_testSetParam, 138
- rfe_txIndex_1_e, 114
- rfe_txIndex_2_e, 114
- rfe_txIndex_3_e, 114
- rfe_txIndex_4_e, 114
- rfe_txIndex_invalid_e, 114
- rfe_txIndex_t, 114
- rfe_txPowerErrorFlag_t, 115
- rfe_txPowerHighErrorFlag, 115
- rfe_txPowerLowErrorFlag, 115
- rfe_txPowerMaxErrorFlag, 115
- rfe_txPowerMinErrorFlag, 115
- rfe_txSelect_t, 82
- rfe_updateBegin, 139
- rfe_updateDynamicTable, 140
- rfe_updateParam, 140
- rfe_updatePush, 141
- rfe_virtualChannel_0_e, 115
- rfe_virtualChannel_1_e, 115
- rfe_virtualChannel_2_e, 115
- rfe_virtualChannel_3_e, 115
- rfe_virtualChannel_invalid_e, 115
- rfe_virtualChannel_t, 115
- rfeCfg_param_chirpProfile_acquisitionTimeTicks_e, 119
- rfeCfg_param_chirpProfile_centerFrequency_e, 119
- rfeCfg_param_chirpProfile_chirpFrequencyDriftSteps_e, 119
- rfeCfg_param_chirpProfile_chirpIntervalTimeTicks_e, 119
- rfeCfg_param_chirpProfile_chirpPIILoopFilterBandwidth_e, 119
- rfeCfg_param_chirpProfile_chirpPIIVcoSelect_e, 119
- rfeCfg_param_chirpProfile_chirpSlopeDirection_e, 119
- rfeCfg_param_chirpProfile_dwellTimeTicks_e, 119
- rfeCfg_param_chirpProfile_effectiveChirpBandwidth_e, 119
- rfeCfg_param_chirpProfile_effectiveSamplingFrequency_e, 119
- rfeCfg_param_chirpProfile_resetTimeTicks_e, 119
- rfeCfg_param_chirpProfile_rxGain_e, 119
- rfeCfg_param_chirpProfile_rxHpfCutOffFrequency_e, 119
- rfeCfg_param_chirpProfile_rxLpfCutOffFrequency_e, 119
- rfeCfg_param_chirpProfile_settleTimeTicks_e, 119
- rfeCfg_param_chirpProfile_txPhaseRotation_1_e, 119
- rfeCfg_param_chirpProfile_txPhaseRotation_2_e, 119
- rfeCfg_param_chirpProfile_txPhaseRotation_3_e, 119
- rfeCfg_param_chirpProfile_txPower_e, 119
- rfeCfg_param_chirpProfile_txTransmissionEnable_e, 119
- rfeCfg_param_chirpProfile_txTransmissionReferenceTime_e, 119
- rfeCfg_param_chirpProfile_txTransmissionTimeOffset_e, 119
- rfeCfg_param_chirpProfile_virtualChannel_e, 119

rfeCfg_param_chirpSequence_chirpCount_e, [118](#)
 rfeCfg_param_chirpSequence_chirpProfileRepeatCount_e, [118](#)
 rfeCfg_param_chirpSequence_chirpProfileSequence_0_e, [118](#)
 rfeCfg_param_chirpSequence_chirpProfileSequence_1_e, [118](#)
 rfeCfg_param_chirpSequence_chirpProfileSequence_2_e, [118](#)
 rfeCfg_param_chirpSequence_chirpProfileSequence_3_e, [118](#)
 rfeCfg_param_chirpSequence_chirpProfileSequence_4_e, [118](#)
 rfeCfg_param_chirpSequence_chirpProfileSequence_5_e, [118](#)
 rfeCfg_param_chirpSequence_chirpProfileSequence_6_e, [118](#)
 rfeCfg_param_chirpSequence_chirpProfileSequence_7_e, [118](#)
 rfeCfg_param_chirpSequence_chirpProfileSequenceLength_e, [118](#)
 rfeCfg_param_chirpSequence_dynamicTableIndex_e, [118](#)
 rfeCfg_param_chirpSequence_dynamicUpdatesEnabled_e, [118](#)
 rfeCfg_param_chirpSequence_pdcNotchFilterEnable_e, [118](#)
 rfeCfg_param_chirpSequence_rxEnable_e, [118](#)
 rfeCfg_param_chirpSequence_txCalibration_thresholdHigh_e, [118](#)
 rfeCfg_param_chirpSequence_txCalibration_thresholdLow_e, [119](#)
 rfeCfg_param_chirpSequence_txPhaseRotation_ddma_e, [118](#)
 rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx1_e, [118](#)
 rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx16_e, [118](#)
 rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx160_e, [118](#)
 rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx2_e, [118](#)
 rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx20_e, [118](#)
 rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx200_e, [118](#)
 rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx2000_e, [118](#)
 rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx3_e, [118](#)
 rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx30_e, [118](#)
 rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx300_e, [118](#)
 rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx3000_e, [118](#)
 rfeCfg_param_general_chirpSequenceActiveSignalGpio_e, [116](#)
 rfeCfg_param_general_clkloConfig_e, [116](#)
 rfeCfg_param_general_dataOutConfig_e, [116](#)
 rfeCfg_param_general_jumpbackTimeTicks_e,
 rfeCfg_param_general_metadataPacketSwContent_e,
 rfeCfg_param_general_metadataPacketVirtualChannel_e,
 rfeCfg_param_general_pdcDecimationFilter_e,
 rfeCfg_param_general_powerMode_e, [115](#)
 rfeCfg_param_general_radarCycleStartSignalGpio_e, [116](#)
 rfeCfg_param_general_txCalibration_mode_e, [116](#)
 rfeCfg_param_metadata_chirpProfileCount_e, [115](#)
 rfeCfg_param_metadata_chirpSequenceConfigCount_e, [115](#)
 rfeCfg_param_metadata_crc, [119](#)
 rfeCfg_param_metadata_hashVersion_e, [115](#)
 rfeCfg_param_metadata_majorVersion_e, [115](#)
 rfeCfg_param_metadata_minorVersion_e, [115](#)
 rfeCfg_param_metadata_patchVersion_e, [115](#)
 rfeCfg_param_monitorAndSafety_adcClippingCountLimit_adc1_e,
 rfeCfg_param_monitorAndSafety_adcClippingCountLimit_adc2_e,
 rfeCfg_param_monitorAndSafety_adcClippingCountLimit_adc3_e,
 rfeCfg_param_monitorAndSafety_adcClippingCountLimit_adc4_e,
 rfeCfg_param_monitorAndSafety_adcClippingCountResetEveryChirp_e,
 rfeCfg_param_monitorAndSafety_checkBbd_loi_e, [117](#)
 rfeCfg_param_monitorAndSafety_checkBbd_rx_e, [117](#)
 rfeCfg_param_monitorAndSafety_checkBbd_tx_e, [117](#)
 rfeCfg_param_monitorAndSafety_frequencyForBist_e, [116](#)
 rfeCfg_param_monitorAndSafety_fuSaFaultMask_0_e, [117](#)
 rfeCfg_param_monitorAndSafety_fuSaFaultMask_10_e, [117](#)
 rfeCfg_param_monitorAndSafety_fuSaFaultMask_11_e, [117](#)
 rfeCfg_param_monitorAndSafety_fuSaFaultMask_1_e, [117](#)
 rfeCfg_param_monitorAndSafety_fuSaFaultMask_2_e, [117](#)
 rfeCfg_param_monitorAndSafety_fuSaFaultMask_3_e, [117](#)
 rfeCfg_param_monitorAndSafety_fuSaFaultMask_4_e, [117](#)
 rfeCfg_param_monitorAndSafety_fuSaFaultMask_5_e, [117](#)
 rfeCfg_param_monitorAndSafety_fuSaFaultMask_6_e, [117](#)
 rfeCfg_param_monitorAndSafety_fuSaFaultMask_7_e, [117](#)

rfeCfg_param_monitorAndSafety_fuSaFaultMask_8_e, 117	rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage2_rx2_ 116
rfeCfg_param_monitorAndSafety_fuSaFaultMask_9_e, 117	rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage2_rx3_ 116
rfeCfg_param_monitorAndSafety_injectTestTone_e, 117	rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage2_rx4_ 116
rfeCfg_param_monitorAndSafety_ppdThreshold_profile0_ 117	rfeCfg_param_monitorAndSafety_temperatureSensorThresholdHigh_ 116
rfeCfg_param_monitorAndSafety_ppdThreshold_profile1_ 117	rfeCfg_param_monitorAndSafety_temperatureSensorThresholdHigh_ 116
rfeCfg_param_monitorAndSafety_ppdThreshold_profile2_ 117	rfeCfg_param_monitorAndSafety_temperatureSensorThresholdHigh_ 116
rfeCfg_param_monitorAndSafety_ppdThreshold_profile3_ 117	rfeCfg_param_monitorAndSafety_temperatureSensorThresholdOver_ 116
rfeCfg_param_monitorAndSafety_ppdThreshold_profile4_ 117	rfeCfg_param_monitorAndSafety_temperatureSensorThresholdOver_ 116
rfeCfg_param_monitorAndSafety_ppdThreshold_profile5_ 117	rfeCfg_param_monitorAndSafety_temperatureSensorThresholdOver_ 116
rfeCfg_param_monitorAndSafety_ppdThreshold_profile6_ 117	rfeCfg_param_monitorAndSafety_temperatureSensorThresholdUnder_ 116
rfeCfg_param_monitorAndSafety_ppdThreshold_profile7_ 117	rfeCfg_param_monitorAndSafety_temperatureSensorThresholdUnder_ 116
rfeCfg_param_monitorAndSafety_rxGainDiffLnaThreshold_ 117	rfeCfg_param_monitorAndSafety_temperatureSensorThresholdUnder_ 116
rfeCfg_param_monitorAndSafety_rxGainDiffMixerThreshold_ 117	rfeCfg_param_monitorAndSafety_thresholdValueToPromoteR1Faults_ 116
rfeCfg_param_monitorAndSafety_rxPhaseDiffLnaThreshold_ 117	rfeCfg_param_monitorAndSafety_zeroHrRefForRxGainDiffLna_rx1_ 117
rfeCfg_param_monitorAndSafety_rxPhaseDiffMixerThreshold_ 117	rfeCfg_param_monitorAndSafety_zeroHrRefForRxGainDiffLna_rx1_ 117
rfeCfg_param_monitorAndSafety_rxSatCountResetEveryChirpSequence_ 116	rfeCfg_param_monitorAndSafety_zeroHrRefForRxGainDiffLna_rx1_ 117
rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage0_ 116	rfeCfg_param_monitorAndSafety_zeroHrRefForRxGainDiffMixer_rx1_ 117
rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage1_ 116	rfeCfg_param_monitorAndSafety_zeroHrRefForRxGainDiffMixer_rx1_ 117
rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage2_ 116	rfeCfg_param_monitorAndSafety_zeroHrRefForRxGainDiffMixer_rx1_ 117
rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage3_ 116	rfeCfg_param_monitorAndSafety_zeroHrRefForRxPhaseDiffLna_rx1_ 116
rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage4_ 116	rfeCfg_param_monitorAndSafety_zeroHrRefForRxPhaseDiffLna_rx1_ 116
rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage5_ 116	rfeCfg_param_monitorAndSafety_zeroHrRefForRxPhaseDiffLna_rx1_ 116
rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage6_ 116	rfeCfg_param_monitorAndSafety_zeroHrRefForRxPhaseDiffMixer_rx1_ 117
rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage7_ 116	rfeCfg_param_monitorAndSafety_zeroHrRefForRxPhaseDiffMixer_rx1_ 117
rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage0_ 116	rfeCfg_param_monitorAndSafety_zeroHrRefForRxPhaseDiffMixer_rx1_ 117
rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage1_ 116	rfeCfg_param_radarCycle_bistInterval_e, 118
rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage2_ 116	rfeCfg_param_radarCycle_chirpSequence_0_e, 117
rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage3_ 116	rfeCfg_param_radarCycle_chirpSequence_1_e, 117
rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage4_ 116	rfeCfg_param_radarCycle_chirpSequence_2_e, 117
rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage5_ 116	rfeCfg_param_radarCycle_chirpSequence_3_e, 117

- 117
- rfeCfg_param_radarCycle_chirpSequence_4_e, 117
- rfeCfg_param_radarCycle_chirpSequence_5_e, 117
- rfeCfg_param_radarCycle_chirpSequence_6_e, 117
- rfeCfg_param_radarCycle_chirpSequence_7_e, 118
- rfeCfg_param_radarCycle_chirpSequenceCount_e, 117
- rfeCfg_param_radarCycle_chirpSequenceStartTimeOffset_e, 118
- rfeCfg_param_radarCycle_chirpSequenceStartTimeOffset_t, 118
- rfeCfg_param_radarCycle_chirpSequenceStartTimeOffset_t, 118
- rfeCfg_param_radarCycle_chirpSequenceStartTimeOffset_t, 118
- rfeCfg_param_radarCycle_chirpSequenceStartTimeOffset_t, 118
- rfeCfg_param_radarCycle_chirpSequenceStartTimeOffset_t, 118
- rfeCfg_param_radarCycle_chirpSequenceStartTimeOffset_t, 118
- rfeCfg_param_radarCycle_radarCycleDuration_e, 117
- rfeCfg_param_radarCycle_recalibrateProfileDependent_0_e, 118
- rfeCfg_param_radarCycle_recalibrateProfileDependent_1_e, 118
- rfeCfg_param_radarCycle_recalibrateProfileDependent_2_e, 118
- rfeCfg_param_radarCycle_recalibrateProfileDependent_3_e, 118
- rfeCfg_param_radarCycle_recalibrateProfileDependent_4_e, 118
- rfeCfg_param_radarCycle_recalibrateProfileDependent_5_e, 118
- rfeCfg_param_radarCycle_recalibrateProfileDependent_6_e, 118
- rfeCfg_param_radarCycle_recalibrateProfileDependent_7_e, 118
- rfeCfg_param_radarCycle_recalibrateProfileIndependent_e, 118
- rfeCfg_param_t, 115
- rfeCfg_section_chirpProfile_0_e, 119
- rfeCfg_section_chirpProfile_1_e, 119
- rfeCfg_section_chirpProfile_2_e, 120
- rfeCfg_section_chirpProfile_3_e, 120
- rfeCfg_section_chirpProfile_4_e, 120
- rfeCfg_section_chirpProfile_5_e, 120
- rfeCfg_section_chirpProfile_6_e, 120
- rfeCfg_section_chirpProfile_7_e, 120
- rfeCfg_section_chirpSequenceConfig_0_e, 119
- rfeCfg_section_chirpSequenceConfig_1_e, 119
- rfeCfg_section_chirpSequenceConfig_2_e, 120
- rfeCfg_section_chirpSequenceConfig_3_e, 120
- rfeCfg_section_chirpSequenceConfig_4_e, 120
- rfeCfg_section_chirpSequenceConfig_5_e, 120
- rfeCfg_section_chirpSequenceConfig_6_e, 120
- rfeCfg_section_chirpSequenceConfig_7_e, 120
- rfeCfg_section_crc, 120
- rfeCfg_section_general_e, 119
- rfeCfg_section_metadata_e, 119
- rfeCfg_section_monitorAndSafety_e, 119
- rfeCfg_section_radarCycle_e, 119
- rfeCfg_section_t, 119
- rfeCfg_size_16_bit_e, 120
- rfeCfg_size_32_bit_e, 120
- rfeCfg_size_8_bit_e, 120
- rfeCfg_size_invalid_e, 120
- rfeCfg_size_t, 120
- rfeCmdClient_call, 142
- rfeCmdClient_checkNotBusy, 143
- rfeCmdClient_cmdParamAddBool, 143
- rfeCmdClient_cmdParamAddBuffer, 144
- rfeCmdClient_cmdParamAddInt16, 144
- rfeCmdClient_cmdParamAddInt32, 145
- rfeCmdClient_cmdParamAddInt8, 146
- rfeCmdClient_cmdParamAddPointer, 146
- rfeCmdClient_cmdParamAddUint16, 147
- rfeCmdClient_cmdParamAddUint32, 147
- rfeCmdClient_cmdParamAddUint8, 148
- rfeCmdClient_cmdParamStart, 149
- rfeCmdClient_cmdRespGetBool, 149
- rfeCmdClient_cmdRespGetBuf, 150
- rfeCmdClient_cmdRespGetInt16, 150
- rfeCmdClient_cmdRespGetInt32, 151
- rfeCmdClient_cmdRespGetInt8, 151
- rfeCmdClient_cmdRespGetUint16, 152
- rfeCmdClient_cmdRespGetUint32, 153
- rfeCmdClient_cmdRespGetUint8, 153
- rfeCmdClient_configureInterrupt, 154
- rfeCmdClient_getCurrentRespLen, 154
- rfeCmdClient_init, 154
- rfeCmdClient_sharedDataRead, 155
- rfeCmdClient_sharedParameter_chirpSequenceCount_e, 121
- rfeCmdClient_sharedParameter_fw_ready_e, 121
- rfeCmdClient_sharedParameter_radarCycleCount_e, 121
- rfeCmdClient_sharedParameter_state_e, 121
- rfeCmdClient_sharedParameter_t, 120
- rfeCrc_calBuf, 156
- rfeError_error_api_boundedWaitResourcesExhausted_e, 90
- rfeError_error_api_TimerInUse_e, 90
- rfeSwBist_InjectToneMode_t, 121
- rfeTef82XXRfBistTest_t, 121
- TEF82XX_alternate_lna_mixer, 121
- TEF82XX_both_lna_mixer, 121
- TEF82XX_only_mixer, 121
- TEF82XX_RFBIST_LNA_TEST, 121

- TEF82XX_RFBIST_MIXER_TEST, 121
- TEF82XX_RFBIST_TESTS, 121
- RFE Abstract 2.0 - Peripheral Access Interface, 238
- RFE Abstract 2.0 - SPI Interface, 242
 - RfeHwSpiInitialize, 242
 - RfeHwSpiTransfer, 242
- RFE Abstract 2.0 - Timers Interface, 244
 - RfeHwDelayUs, 244
 - RfeHwHeartBeatTimerGetValue, 245
 - RfeHwTimersInit, 245
- RFE Abstract 2.0 API, 5
- RFE Abstract 2.0 BLOB - Constants, 156
 - RFE_CFG_MAX_DATA_LENGTH, 173
 - RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_0, 174
 - RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_1, 174
 - RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_2, 174
 - RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_3, 174
 - RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_4, 174
 - RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_5, 174
 - RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_6, 174
 - RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_7, 175
 - RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_0, 175
 - RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_1, 175
 - RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_2, 175
 - RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_3, 175
 - RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_4, 175
 - RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_5, 175
 - RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_6, 176
 - RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_7, 176
 - RFE_CFG_OFFSET_SECTION_CRC, 176
 - RFE_CFG_OFFSET_SECTION_GENERAL, 176
 - RFE_CFG_OFFSET_SECTION_METADATA, 176
 - RFE_CFG_OFFSET_SECTION_MONITOR_AND_SAFETY, 176
 - RFE_CFG_OFFSET_SECTION_RADAR_CYCLE, 176
 - RFE_CFG_PARAM_OFFSET_MSK, 177
 - RFE_CFG_PARAM_OFFSET_SHF, 177
 - RFE_CFG_PARAM_SECTION_CHIRP_PROFILE, 177
 - RFE_CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG, 177
 - RFE_CFG_PARAM_SECTION_CRC, 177
 - RFE_CFG_PARAM_SECTION_GENERAL, 177
 - RFE_CFG_PARAM_SECTION_METADATA, 177
 - RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY, 178
 - RFE_CFG_PARAM_SECTION_MSK, 178
 - RFE_CFG_PARAM_SECTION_RADAR_CYCLE, 178
 - RFE_CFG_PARAM_SIZE_16_BIT, 178
 - RFE_CFG_PARAM_SIZE_32_BIT, 178
 - RFE_CFG_PARAM_SIZE_32_BIT_FLOAT, 178
 - RFE_CFG_PARAM_SIZE_8_BIT, 178
 - RFE_CFG_PARAM_SIZE_MSK, 179
 - RFE_CFG_SIZE_SECTION_CHIRP_PROFILE, 179
 - RFE_CFG_SIZE_SECTION_CHIRP_SEQUENCE, 179
 - RFE_CFG_SIZE_SECTION_CRC, 179
 - RFE_CFG_SIZE_SECTION_GENERAL, 179
 - RFE_CFG_SIZE_SECTION_METADATA, 179
 - RFE_CFG_SIZE_SECTION_MONITOR_AND_SAFETY, 179
 - RFE_CFG_SIZE_SECTION_RADAR_CYCLE, 179
 - RFE_CFG_SIZE_TOTAL, 180
 - RFE_CHIRP_PROFILE_SEQUENCE_LENGTH_MAX, 180
 - RFE_CHIRP_PROFILES_MAX, 180
 - RFE_CHIRP_SEQUENCE_CONFIGS_MAX, 180
 - RFE_CHIRP_SEQUENCES_PER_RADAR_CYCLE_MAX, 180
 - rfe_chirpPllLoopFilterBandwidth_1000kHz_e, 183
 - rfe_chirpPllLoopFilterBandwidth_1050kHz_e, 183
 - rfe_chirpPllLoopFilterBandwidth_1100kHz_e, 183
 - rfe_chirpPllLoopFilterBandwidth_1150kHz_e, 183
 - rfe_chirpPllLoopFilterBandwidth_1200kHz_e, 183
 - rfe_chirpPllLoopFilterBandwidth_1250kHz_e, 183
 - rfe_chirpPllLoopFilterBandwidth_1300kHz_e, 183
 - rfe_chirpPllLoopFilterBandwidth_1350kHz_e, 183
 - rfe_chirpPllLoopFilterBandwidth_1400kHz_e, 183
 - rfe_chirpPllLoopFilterBandwidth_1450kHz_e, 183
 - rfe_chirpPllLoopFilterBandwidth_1500kHz_e, 183
 - rfe_chirpPllLoopFilterBandwidth_1550kHz_e, 183
 - rfe_chirpPllLoopFilterBandwidth_1600kHz_e, 183
 - rfe_chirpPllLoopFilterBandwidth_1650kHz_e, 183
 - rfe_chirpPllLoopFilterBandwidth_1700kHz_e, 183
 - rfe_chirpPllLoopFilterBandwidth_200kHz_e, 182
 - rfe_chirpPllLoopFilterBandwidth_250kHz_e, 182
 - rfe_chirpPllLoopFilterBandwidth_300kHz_e, 182
 - rfe_chirpPllLoopFilterBandwidth_350kHz_e, 182
 - rfe_chirpPllLoopFilterBandwidth_400kHz_e, 182
 - rfe_chirpPllLoopFilterBandwidth_450kHz_e, 182
 - rfe_chirpPllLoopFilterBandwidth_500kHz_e, 182
 - rfe_chirpPllLoopFilterBandwidth_550kHz_e, 182
 - rfe_chirpPllLoopFilterBandwidth_600kHz_e, 182
 - rfe_chirpPllLoopFilterBandwidth_650kHz_e, 182
 - rfe_chirpPllLoopFilterBandwidth_700kHz_e, 182
 - rfe_chirpPllLoopFilterBandwidth_750kHz_e, 182
 - rfe_chirpPllLoopFilterBandwidth_800kHz_e, 183

- rfe_chirpPllLoopFilterBandwidth_850kHz_e, 183
- rfe_chirpPllLoopFilterBandwidth_900kHz_e, 183
- rfe_chirpPllLoopFilterBandwidth_950kHz_e, 183
- rfe_chirpPllLoopFilterBandwidth_invalid_e, 183
- rfe_chirpPllLoopFilterBandwidth_t, 182
- rfe_effectiveSamplingFrequency_10MHz_e, 184
- rfe_effectiveSamplingFrequency_20MHz_e, 183
- rfe_effectiveSamplingFrequency_40MHz_e, 183
- rfe_effectiveSamplingFrequency_5MHz_e, 184
- rfe_effectiveSamplingFrequency_invalid_e, 184
- rfe_effectiveSamplingFrequency_t, 183
- RFE_FUSA_FAULT_MASK_PARAM_COUNT, 180
- rfe_phaseRotation_t, 182
- RFE_SW_CFG_MNGR_CHIRP_PROFILE_PARAMS, 180
- RFE_SW_CFG_MNGR_CHIRP_SEQUENCE_PARAMS, 181
- RFE_SW_CFG_MNGR_GENRAL_PARAMS, 181
- RFE_SW_CFG_MNGR_MONITOR_AND_SAFETY_PARAMS, 181
- RFE_SW_CFG_MNGR_NO_OF_SECTION, 181
- RFE_SW_CFG_MNGR_NON_UPDATABLE_PARAMS, 181
- RFE_SW_CFG_MNGR_RADAR_CYCLE_PARAMS, 181
- RFE_SW_CFG_MNGR_UPDATABLE_PARAMS, 181
- RFE_TX_COUNT, 182
- rfe_txTransmissionReferenceTime_startOfDwellTime_e, 184
- rfe_txTransmissionReferenceTime_startOfSettleTime_e, 184
- rfe_txTransmissionReferenceTime_t, 184
- RFE Abstract 2.0 BLOB - Low Level Interface, 184
- rfeCfg_calcAndWriteCrc, 190
- rfeCfg_CheckBlobCompatibility, 190
- rfeCfg_chirpProfile_getAcquisitionTimeTicks, 191
- rfeCfg_chirpProfile_getCenterFrequency, 191
- rfeCfg_chirpProfile_getChirpIntervalTimeTicks, 192
- rfeCfg_chirpProfile_getChirpPllLoopFilterBandwidth, 192
- rfeCfg_chirpProfile_getChirpPllVco, 192
- rfeCfg_chirpProfile_getChirpSlopeDirection, 193
- rfeCfg_chirpProfile_getDwellTimeTicks, 193
- rfeCfg_chirpProfile_getEffectiveChirpBandwidth, 194
- rfeCfg_chirpProfile_getEffectiveSamplingFrequency, 194
- rfeCfg_chirpProfile_getResetTimeTicks, 195
- rfeCfg_chirpProfile_getRxGain, 195
- rfeCfg_chirpProfile_getRxHpfCutOffFrequency, 195
- rfeCfg_chirpProfile_getRxLpfCutOffFrequency, 196
- rfeCfg_chirpProfile_getSettleTimeTicks, 196
- rfeCfg_chirpProfile_getTxPhaseRotation, 197
- rfeCfg_chirpProfile_getTxPower, 197
- rfeCfg_chirpProfile_getTxTransmissionEnable, 198
- rfeCfg_chirpProfile_getTxTransmissionReferenceTime, 198
- rfeCfg_chirpProfile_getTxTransmissionTimeOffset, 198
- rfeCfg_chirpProfile_getVirtualChannel, 199
- rfeCfg_chirpProfile_setAcquisitionTimeTicks, 199
- rfeCfg_chirpProfile_setCenterFrequency, 200
- rfeCfg_chirpProfile_setChirpIntervalTimeTicks, 200
- rfeCfg_chirpProfile_setChirpPllLoopFilterBandwidth, 200
- rfeCfg_chirpProfile_setChirpPllVco, 201
- rfeCfg_chirpProfile_setChirpSlopeDirection, 201
- rfeCfg_chirpProfile_setDwellTimeTicks, 202
- rfeCfg_chirpProfile_setEffectiveChirpBandwidth, 202
- rfeCfg_chirpProfile_setEffectiveSamplingFrequency, 203
- rfeCfg_chirpProfile_setResetTimeTicks, 203
- rfeCfg_chirpProfile_setRxGain, 203
- rfeCfg_chirpProfile_setRxHpfCutOffFrequency, 204
- rfeCfg_chirpProfile_setRxLpfCutOffFrequency, 204
- rfeCfg_chirpProfile_setSettleTimeTicks, 204
- rfeCfg_chirpProfile_setTxPhaseRotation, 205
- rfeCfg_chirpProfile_setTxPower, 205
- rfeCfg_chirpProfile_setTxTransmissionEnable, 206
- rfeCfg_chirpProfile_setTxTransmissionReferenceTime, 206
- rfeCfg_chirpProfile_setTxTransmissionTimeOffset, 206
- rfeCfg_chirpProfile_setVirtualChannel, 207
- rfeCfg_chirpSequence_getChirpCount, 207
- rfeCfg_chirpSequence_getChirpFrequencyDriftSteps, 208
- rfeCfg_chirpSequence_getChirpProfileRepeatCount, 208
- rfeCfg_chirpSequence_getChirpProfileSequence, 208
- rfeCfg_chirpSequence_getChirpProfileSequenceLength, 209
- rfeCfg_chirpSequence_getDynamicTableIndex, 209
- rfeCfg_chirpSequence_getDynamicUpdatesEnabled, 210
- rfeCfg_chirpSequence_getRxEnable, 210
- rfeCfg_chirpSequence_setChirpCount, 211
- rfeCfg_chirpSequence_setChirpProfileRepeatCount, 211
- rfeCfg_chirpSequence_setChirpProfileSequence, 211
- rfeCfg_chirpSequence_setChirpProfileSequenceLength, 212
- rfeCfg_chirpSequence_setDynamicTableIndex, 213
- rfeCfg_chirpSequence_setDynamicUpdatesEnabled, 213

- 213
- rfeCfg_chirpSequence_setRxEnable, 213
- rfeCfg_general_getJumpbackTimeTicks, 214
- rfeCfg_general_getPdcBitwidth, 214
- rfeCfg_general_getPowerMode, 215
- rfeCfg_general_getTxCalibrationMode, 215
- rfeCfg_general_setPowerMode, 215
- rfeCfg_metadata_getChirpProfileCount, 216
- rfeCfg_metadata_getChirpSequenceConfigCount, 216
- rfeCfg_monitorAndSafety_getAdcClippingCountResetEveryChirpSequence, 217
- rfeCfg_monitorAndSafety_getBist, 217
- rfeCfg_monitorAndSafety_getFrequencyForBist, 218
- rfeCfg_monitorAndSafety_getFuSaFaultMask, 218
- rfeCfg_monitorAndSafety_getInjectTestTone, 218
- rfeCfg_monitorAndSafety_getLoiBBD, 219
- rfeCfg_monitorAndSafety_getRxBBD, 219
- rfeCfg_monitorAndSafety_getRxSatCountResetEveryChirpSequence, 220
- rfeCfg_monitorAndSafety_getRxSaturationCountLimitStage, 220
- rfeCfg_monitorAndSafety_getRxSaturationThresholdStage, 221
- rfeCfg_monitorAndSafety_getTxBBD, 221
- rfeCfg_monitorAndSafety_setBist, 221
- rfeCfg_monitorAndSafety_setFrequencyForBist, 222
- rfeCfg_monitorAndSafety_setInjectTestTone, 223
- rfeCfg_monitorAndSafety_setLoiBBD, 223
- rfeCfg_radarCycle_getBistInterval, 223
- rfeCfg_radarCycle_getChirpSequence, 224
- rfeCfg_radarCycle_getChirpSequenceCount, 224
- rfeCfg_radarCycle_getChirpSequenceStartTimeOffset, 225
- rfeCfg_radarCycle_getRadarCycleDuration, 225
- rfeCfg_radarCycle_setRadarCycleDuration, 225
- rfeCfg_read, 226
- rfeCfg_readB, 226
- rfeCfg_readDW, 227
- rfeCfg_readW, 227
- rfeCfg_sequence_getFinalPcGenMode, 228
- rfeCfg_sequence_getPhaseRotationDdmaMode, 228
- rfeCfg_sequence_getPhaseRotationTxUseDdma, 228
- rfeCfg_sequence_getTxCalibrationThresholdHigh, 228
- rfeCfg_sequence_getTxCalibrationThresholdLow, 229
- rfeCfg_sequence_getTxDdmaInitPhase, 229
- rfeCfg_sequence_getTxDdmaUpdatePhase, 229
- rfeCfg_sequence_setPhaseRotationDdmaMode, 229
- rfeCfg_sequence_setTxDdmaInitPhase, 231
- rfeCfg_sequence_setTxDdmaUpdatePhase, 231
- rfeCfg_sequence_setTxFinalPcGenMode, 232
- rfeCfg_util_chirpProfileIndex, 232
- rfeCfg_util_chirpProfileSequence, 232
- rfeCfg_util_chirpSequenceConfigIndex, 233
- rfeCfg_util_radarCycleChirpSequence, 233
- rfeCfg_write, 233
- RFE Abstract 2.0 Constants, 7
- CDD_RFE_CFG_HALT_ON_ERROR, 8
- CDD_RFE_CONFIGURE, 9
- CDD_RFE_DEV_ERROR_DETECT, 9
- CDD_RFE_E_CMD_LENGTH, 9
- CDD_RFE_E_CMD_NOT_IMPLEMENTED, 9
- CDD_RFE_E_FUSA_ERROR, 9
- CDD_RFE_E_FW_ERROR, 10
- CDD_RFE_E_LIMIT_EXCEED, 10
- CDD_RFE_E_PARAM_POINTER, 10
- CDD_RFE_E_PARAM_SIZE, 10
- CDD_RFE_E_REDUNDANCY_CHECK_FAILED, 10
- CDD_RFE_E_RESPONSE_ERROR, 10
- CDD_RFE_E_SETUP_FAILED, 11
- CDD_RFE_E_UNRESPONSIVE, 11
- CDD_RFE_GET_DATA, 11
- CDD_RFE_HALT_ON_ERROR, 11
- CDD_RFE_SET_DATA, 11
- CDD_RFE_SID_CONFIGURE, 12
- CDD_RFE_SID_GET_ERROR, 12
- CDD_RFE_SID_GET_VERSION_INFO, 12
- CDD_RFE_SID_GETFUSAERRORS, 12
- CDD_RFE_SID_GETNEXTRADARCYCLESTARTTIME, 12
- CDD_RFE_SID_GETRADARCYCLECOUNT, 12
- CDD_RFE_SID_GETSTATE, 12
- CDD_RFE_SID_GETTIME, 13
- CDD_RFE_SID_GETVERSION, 13
- CDD_RFE_SID_INIT, 13
- CDD_RFE_SID_INVALID, 13
- CDD_RFE_SID_MONITORREAD, 13
- CDD_RFE_SID_RADARCYCLESTART, 13
- CDD_RFE_SID_RADARCYCLESTOP, 13
- CDD_RFE_SID_SETNEXTRADARCYCLESTARTTIME, 14
- CDD_RFE_SID_TESTCONTINUOUSWAVETRANSMISSIONSTART, 14
- CDD_RFE_SID_TESTCONTINUOUSWAVETRANSMISSIONSTOP, 14
- CDD_RFE_SID_TESTSETPARAM, 14
- CDD_RFE_SID_UPDATEBEGIN, 14
- CDD_RFE_SID_UPDATEDYNAMICTABLE, 14
- CDD_RFE_SID_UPDATEPARAM, 14
- CDD_RFE_SID_UPDATEPUSH, 14
- CDD_RFE_SINGLE_MANAGEMENT_THREADS, 15
- CDD_RFE_START_EXECUTION, 15
- CDD_RFE_STOP_EXECUTION, 15
- CDD_RFE_VERSION_INFO_API, 15
- CDD_RFE_VERSION_INFO_CHECK, 15
- CDD_RFE_ZERO, 15
- gsCddRfeLoopExit, 18

- REG_MAX_COUNT_TO_SEND, 15
- RFE_ADC_REG_NUMBER, 16
- RFE_ATB_VOLTAGE_TABLE_NUMBER, 16
- RFE_CAFC_REG_NUMBER, 16
- RFE_CC_REG_NUMBER, 16
- RFE_CSI2_REG_NUMBER, 16
- RFE_GB_REG_NUMBER, 16
- RFE_GLDO_REG_NUMBER, 16
- RFE_ISM_REG_NUMBER, 17
- RFE_LDVS_REG_NUMBER, 17
- RFE_LOI_REG_NUMBER, 17
- RFE_MCLK_REG_NUMBER, 17
- RFE_OTP_REG_NUMBER, 17
- RFE_OTP_TABLE_NUMBER, 17
- RFE_RX_REG_NUMBER, 17
- RFE_SC_REG_NUMBER, 18
- RFE_SSBMOD_REG_NUMBER, 18
- RFE_TE_REG_NUMBER, 18
- RFE_TX_REG_NUMBER, 18
- RFE Abstract 2.0 External Software Dependencies, 6
- RFE Abstract 2.0 Firmware API, 6
- RFE Abstract 2.0 Interface, 18
 - CDD_Rfe_Configure, 27
 - CDD_Rfe_ConfigureInterrupt, 29
 - CDD_RFE_FRONT_END_FOLLOWER_0, 25
 - CDD_RFE_FRONT_END_FOLLOWER_1, 25
 - CDD_RFE_FRONT_END_FOLLOWER_2, 25
 - CDD_RFE_FRONT_END_FOLLOWER_MAX, 25
 - CDD_RFE_FRONT_END_LEADER, 25
 - CDD_RFE_FRONT_END_MASK_FOLLOWER_0, 26
 - CDD_RFE_FRONT_END_MASK_FOLLOWER_1, 26
 - CDD_RFE_FRONT_END_MASK_FOLLOWER_2, 26
 - CDD_RFE_FRONT_END_MASK_LEADER, 26
 - CDD_Rfe_GetBistZeroHourReferenceData, 30
 - CDD_Rfe_GetBlobValue, 30
 - CDD_Rfe_GetError, 31
 - CDD_Rfe_GetFrontEnd, 31
 - CDD_Rfe_GetFuSaFaults, 32
 - CDD_Rfe_GetFuSaFaultStatistics, 33
 - CDD_Rfe_GetNextRadarCycleStartTime, 34
 - CDD_Rfe_GetRadarCycleCount, 35
 - CDD_Rfe_GetRegisterDump, 35
 - CDD_Rfe_GetState, 36
 - CDD_Rfe_GetTime, 37
 - CDD_Rfe_GetVersionInfo, 37
 - CDD_Rfe_Init, 38
 - CDD_Rfe_MonitorRead, 38
 - CDD_Rfe_RadarCycleStart, 39
 - CDD_Rfe_RadarCycleStop, 40
 - CDD_Rfe_SetBlobValue, 41
 - CDD_Rfe_SetFrontEnd, 41
 - CDD_Rfe_SetNextRadarCycleStartTime, 42
 - CDD_Rfe_SwHwGetVersion, 43
 - CDD_Rfe_TestContinuousWaveTransmissionStart, 43
 - CDD_Rfe_TestContinuousWaveTransmissionStop, 44
 - CDD_Rfe_TestGetInternalError, 45
 - CDD_Rfe_TestSetParam, 45
 - CDD_Rfe_UpdateBegin, 46
 - CDD_Rfe_UpdateDynamicTable, 47
 - CDD_Rfe_UpdateParam, 47
 - CDD_Rfe_UpdatePush, 48
 - Cdd_RfeApiIRQDataType, 22
 - Cdd_RfeApiWaitForInterruptDataType, 22
 - Cdd_RfeCfgParamType, 22
 - Cdd_RfeCfgSectionType, 23
 - Cdd_RfeChirpProfileIndexType, 23
 - Cdd_RfeCoreIdDataType, 23
 - Cdd_RfeDynParamsSelectType, 23
 - Cdd_RfeErrorType, 23
 - Cdd_RfeEventsIRQDataType, 23
 - CDD_RfeFrontEndHierarchyType, 25
 - CDD_RfeFrontEndIdMaskType, 26
 - Cdd_RfeFusaFaultType, 23
 - Cdd_RfeMonitorSelectType, 24
 - Cdd_RfeMonitorValuesType, 24
 - Cdd_RfeParamUpdateType, 24
 - Cdd_RfeRadarCycleCountType, 24
 - Cdd_RfeRxBistZeroHourRefDataType, 24
 - Cdd_RfeStateType, 24
 - Cdd_RfeSysMemAddressType, 25
 - Cdd_RfeTestParamType, 25
 - Cdd_RfeVersionType, 25
 - rfe_bcd_adc12_e, 26
 - rfe_bcd_adc34_e, 26
 - rfe_bcd_atb_voltage_e, 27
 - rfe_bcd_cafc_e, 26
 - rfe_bcd_cc_e, 26
 - rfe_bcd_csi2_e, 26
 - rfe_bcd_gb_e, 26
 - rfe_bcd_gldo_e, 26
 - rfe_bcd_ism_e, 27
 - rfe_bcd_loi_e, 26
 - rfe_bcd_lvds_e, 26
 - rfe_bcd_mclk_e, 26
 - rfe_bcd_otp_e, 27
 - rfe_bcd_otp_table_e, 27
 - rfe_bcd_rx1_e, 26
 - rfe_bcd_rx2_e, 26
 - rfe_bcd_rx3_e, 26
 - rfe_bcd_rx4_e, 26
 - rfe_bcd_sc_e, 26
 - rfe_bcd_ssbmod_e, 26
 - rfe_bcd_te_e, 26
 - rfe_bcd_tx1_e, 26
 - rfe_bcd_tx2_e, 26
 - rfe_bcd_tx3_e, 26
 - rfe_bcdModuleId_t, 26
 - rfe_moduleDumpSelect_t, 27
 - rfe_registerDump_ADC12, 27
 - rfe_registerDump_ADC34, 27
 - rfe_registerDump_ATB_VOLTAGE_READOUT, 27

- rfe_registerDump_CAFC, [27](#)
- rfe_registerDump_CC, [27](#)
- rfe_registerDump_CSI2, [27](#)
- rfe_registerDump_GB, [27](#)
- rfe_registerDump_GLDO, [27](#)
- rfe_registerDump_ISM, [27](#)
- rfe_registerDump_LDVS, [27](#)
- rfe_registerDump_LOI, [27](#)
- rfe_registerDump_MCLK, [27](#)
- rfe_registerDump_OTP, [27](#)
- rfe_registerDump_OTP_TABLE, [27](#)
- rfe_registerDump_RX1, [27](#)
- rfe_registerDump_RX2, [27](#)
- rfe_registerDump_RX3, [27](#)
- rfe_registerDump_RX4, [27](#)
- rfe_registerDump_SC, [27](#)
- rfe_registerDump_SSBMOD, [27](#)
- rfe_registerDump_TE, [27](#)
- rfe_registerDump_TX1, [27](#)
- rfe_registerDump_TX2, [27](#)
- rfe_registerDump_TX3, [27](#)
- RFE_ADC_REG_NUMBER
 - RFE Abstract 2.0 Constants, [16](#)
- RFE_API_IRQ_ALL
 - RFE Abstract 2.0 - Low Level Constants, [51](#)
- RFE_API_IRQ_CONFIGURE
 - RFE Abstract 2.0 - Low Level Interface, [82](#)
- RFE_API_IRQ_CONTINUOUS_WAVE_TRANSMISSION_START
 - RFE Abstract 2.0 - Low Level Interface, [83](#)
- RFE_API_IRQ_CONTINUOUS_WAVE_TRANSMISSION_STOP
 - RFE Abstract 2.0 - Low Level Interface, [83](#)
- RFE_API_IRQ_GET_BIST_ZERO_HOUR_REFERENCE_DATA
 - RFE Abstract 2.0 - Low Level Interface, [83](#)
- RFE_API_IRQ_GET_FE_ID
 - RFE Abstract 2.0 - Low Level Interface, [83](#)
- RFE_API_IRQ_GET_FUSA_FAULT_STATISTICS
 - RFE Abstract 2.0 - Low Level Interface, [83](#)
- RFE_API_IRQ_GET_FUSA_FAULTS
 - RFE Abstract 2.0 - Low Level Interface, [82](#)
- RFE_API_IRQ_GET_NEXT_RADAR_CYCLE_START_TIME
 - RFE Abstract 2.0 - Low Level Interface, [83](#)
- RFE_API_IRQ_GET_TIME
 - RFE Abstract 2.0 - Low Level Interface, [82](#)
- RFE_API_IRQ_GET_VERSION
 - RFE Abstract 2.0 - Low Level Interface, [82](#)
- RFE_API_IRQ_MAX
 - RFE Abstract 2.0 - Low Level Interface, [83](#)
- RFE_API_IRQ_MAX_NUM
 - RFE Abstract 2.0 - Low Level Constants, [51](#)
- RFE_API_IRQ_MONITOR_READ
 - RFE Abstract 2.0 - Low Level Interface, [82](#)
- RFE_API_IRQ_NONE
 - RFE Abstract 2.0 - Low Level Interface, [82](#)
- RFE_API_IRQ_RADAR_CYCLE_START
 - RFE Abstract 2.0 - Low Level Interface, [82](#)
- RFE_API_IRQ_RADAR_CYCLE_STOP
 - RFE Abstract 2.0 - Low Level Interface, [82](#)
- RFE_API_IRQ_REG_DUMP
 - RFE Abstract 2.0 - Low Level Interface, [83](#)
- RFE_API_IRQ_SET_FE_ID
 - RFE Abstract 2.0 - Low Level Interface, [83](#)
- RFE_API_IRQ_SET_NEXT_RADAR_CYCLE_START_TIME
 - RFE Abstract 2.0 - Low Level Interface, [83](#)
- RFE_API_IRQ_TEST_GET_INTERNAL_ERROR
 - RFE Abstract 2.0 - Low Level Interface, [83](#)
- RFE_API_IRQ_TEST_SET_PARAM
 - RFE Abstract 2.0 - Low Level Interface, [83](#)
- RFE_API_IRQ_UPDATE_PUSH
 - RFE Abstract 2.0 - Low Level Interface, [83](#)
- rfe_apiIRQ_t
 - RFE Abstract 2.0 - Low Level Interface, [82](#)
- rfe_apiWaitForInterrupt_f
 - RFE Abstract 2.0 - Low Level Interface, [81](#)
- RFE_ATB_VOLTAGE_TABLE_NUMBER
 - RFE Abstract 2.0 Constants, [16](#)
- rfe_bcd_adc12_e
 - RFE Abstract 2.0 Interface, [26](#)
- rfe_bcd_adc34_e
 - RFE Abstract 2.0 Interface, [26](#)
- rfe_bcd_atb_voltage_e
 - RFE Abstract 2.0 Interface, [27](#)
- rfe_bcd_cafc_e
 - RFE Abstract 2.0 Interface, [26](#)
- rfe_bcd_cc_e
 - RFE Abstract 2.0 Interface, [26](#)
- rfe_bcd_csi2_e
 - RFE Abstract 2.0 Interface, [26](#)
- rfe_bcd_gb_e
 - RFE Abstract 2.0 Interface, [26](#)
- rfe_bcd_gldo_e
 - RFE Abstract 2.0 Interface, [26](#)
- rfe_bcd_ism_e
 - RFE Abstract 2.0 Interface, [27](#)
- rfe_bcd_loi_e
 - RFE Abstract 2.0 Interface, [26](#)
- rfe_bcd_lvds_e
 - RFE Abstract 2.0 Interface, [26](#)
- rfe_bcd_mclk_e
 - RFE Abstract 2.0 Interface, [26](#)
- rfe_bcd_otp_e
 - RFE Abstract 2.0 Interface, [27](#)
- rfe_bcd_otp_table_e
 - RFE Abstract 2.0 Interface, [27](#)
- rfe_bcd_rx1_e
 - RFE Abstract 2.0 Interface, [26](#)
- rfe_bcd_rx2_e
 - RFE Abstract 2.0 Interface, [26](#)
- rfe_bcd_rx3_e
 - RFE Abstract 2.0 Interface, [26](#)
- rfe_bcd_rx4_e
 - RFE Abstract 2.0 Interface, [26](#)
- rfe_bcd_sc_e
 - RFE Abstract 2.0 Interface, [26](#)
- rfe_bcd_ssbmod_e
 - RFE Abstract 2.0 Interface, [26](#)
- rfe_bcd_te_e

- RFE Abstract 2.0 Interface, [26](#)
- rfe_bcd_tx1_e
 - RFE Abstract 2.0 Interface, [26](#)
- rfe_bcd_tx2_e
 - RFE Abstract 2.0 Interface, [26](#)
- rfe_bcd_tx3_e
 - RFE Abstract 2.0 Interface, [26](#)
- rfe_bcdModuleId_t
 - RFE Abstract 2.0 Interface, [26](#)
- rfe_bistInterval_everyRadarCycle_e
 - RFE Abstract 2.0 - Low Level Interface, [83](#)
- rfe_bistInterval_none_e
 - RFE Abstract 2.0 - Low Level Interface, [83](#)
- rfe_bistInterval_onceIn2RadarCycles_e
 - RFE Abstract 2.0 - Low Level Interface, [83](#)
- rfe_bistInterval_onceIn4RadarCycles_e
 - RFE Abstract 2.0 - Low Level Interface, [83](#)
- rfe_bistInterval_t
 - RFE Abstract 2.0 - Low Level Interface, [83](#)
- RFE_CAFC_REG_NUMBER
 - RFE Abstract 2.0 Constants, [16](#)
- RFE_CC_REG_NUMBER
 - RFE Abstract 2.0 Constants, [16](#)
- RFE_CFG_MAX_DATA_LENGTH
 - RFE Abstract 2.0 BLOB - Constants, [173](#)
- RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_0
 - RFE Abstract 2.0 BLOB - Constants, [174](#)
- RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_1
 - RFE Abstract 2.0 BLOB - Constants, [174](#)
- RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_2
 - RFE Abstract 2.0 BLOB - Constants, [174](#)
- RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_3
 - RFE Abstract 2.0 BLOB - Constants, [174](#)
- RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_4
 - RFE Abstract 2.0 BLOB - Constants, [174](#)
- RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_5
 - RFE Abstract 2.0 BLOB - Constants, [174](#)
- RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_6
 - RFE Abstract 2.0 BLOB - Constants, [174](#)
- RFE_CFG_OFFSET_SECTION_CHIRP_PROFILE_7
 - RFE Abstract 2.0 BLOB - Constants, [175](#)
- RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_0
 - RFE Abstract 2.0 BLOB - Constants, [175](#)
- RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_1
 - RFE Abstract 2.0 BLOB - Constants, [175](#)
- RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_2
 - RFE Abstract 2.0 BLOB - Constants, [175](#)
- RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_3
 - RFE Abstract 2.0 BLOB - Constants, [175](#)
- RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_4
 - RFE Abstract 2.0 BLOB - Constants, [175](#)
- RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_5
 - RFE Abstract 2.0 BLOB - Constants, [175](#)
- RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_6
 - RFE Abstract 2.0 BLOB - Constants, [176](#)
- RFE_CFG_OFFSET_SECTION_CHIRP_SEQUENCE_7
 - RFE Abstract 2.0 BLOB - Constants, [176](#)
- RFE_CFG_OFFSET_SECTION_CRC
 - RFE Abstract 2.0 BLOB - Constants, [176](#)
- RFE_CFG_OFFSET_SECTION_GENERAL
 - RFE Abstract 2.0 BLOB - Constants, [176](#)
- RFE_CFG_OFFSET_SECTION_METADATA
 - RFE Abstract 2.0 BLOB - Constants, [176](#)
- RFE_CFG_OFFSET_SECTION_MONITOR_AND_SAFETY
 - RFE Abstract 2.0 BLOB - Constants, [176](#)
- RFE_CFG_OFFSET_SECTION_RADAR_CYCLE
 - RFE Abstract 2.0 BLOB - Constants, [176](#)
- RFE_CFG_PARAM_OFFSET_MSK
 - RFE Abstract 2.0 BLOB - Constants, [177](#)
- RFE_CFG_PARAM_OFFSET_SHF
 - RFE Abstract 2.0 BLOB - Constants, [177](#)
- RFE_CFG_PARAM_SECTION_CHIRP_PROFILE
 - RFE Abstract 2.0 BLOB - Constants, [177](#)
- RFE_CFG_PARAM_SECTION_CHIRP_SEQUENCE_CONFIG
 - RFE Abstract 2.0 BLOB - Constants, [177](#)
- RFE_CFG_PARAM_SECTION_CRC
 - RFE Abstract 2.0 BLOB - Constants, [177](#)
- RFE_CFG_PARAM_SECTION_GENERAL
 - RFE Abstract 2.0 BLOB - Constants, [177](#)
- RFE_CFG_PARAM_SECTION_METADATA
 - RFE Abstract 2.0 BLOB - Constants, [177](#)
- RFE_CFG_PARAM_SECTION_MONITOR_AND_SAFETY
 - RFE Abstract 2.0 BLOB - Constants, [178](#)
- RFE_CFG_PARAM_SECTION_MSK
 - RFE Abstract 2.0 BLOB - Constants, [178](#)
- RFE_CFG_PARAM_SECTION_RADAR_CYCLE
 - RFE Abstract 2.0 BLOB - Constants, [178](#)
- RFE_CFG_PARAM_SIZE_16_BIT
 - RFE Abstract 2.0 BLOB - Constants, [178](#)
- RFE_CFG_PARAM_SIZE_32_BIT
 - RFE Abstract 2.0 BLOB - Constants, [178](#)
- RFE_CFG_PARAM_SIZE_32_BIT_FLOAT
 - RFE Abstract 2.0 BLOB - Constants, [178](#)
- RFE_CFG_PARAM_SIZE_8_BIT
 - RFE Abstract 2.0 BLOB - Constants, [178](#)
- RFE_CFG_PARAM_SIZE_MSK
 - RFE Abstract 2.0 BLOB - Constants, [179](#)
- RFE_CFG_SIZE_SECTION_CHIRP_PROFILE
 - RFE Abstract 2.0 BLOB - Constants, [179](#)
- RFE_CFG_SIZE_SECTION_CHIRP_SEQUENCE
 - RFE Abstract 2.0 BLOB - Constants, [179](#)
- RFE_CFG_SIZE_SECTION_CRC
 - RFE Abstract 2.0 BLOB - Constants, [179](#)
- RFE_CFG_SIZE_SECTION_GENERAL
 - RFE Abstract 2.0 BLOB - Constants, [179](#)
- RFE_CFG_SIZE_SECTION_METADATA
 - RFE Abstract 2.0 BLOB - Constants, [179](#)
- RFE_CFG_SIZE_SECTION_MONITOR_AND_SAFETY
 - RFE Abstract 2.0 BLOB - Constants, [179](#)
- RFE_CFG_SIZE_SECTION_RADAR_CYCLE
 - RFE Abstract 2.0 BLOB - Constants, [179](#)
- RFE_CFG_SIZE_TOTAL
 - RFE Abstract 2.0 BLOB - Constants, [180](#)
- RFE_CHIRP_PROFILE_SEQUENCE_LENGTH_MAX
 - RFE Abstract 2.0 BLOB - Constants, [180](#)
- RFE_CHIRP_PROFILES_MAX

- RFE Abstract 2.0 BLOB - Constants, [180](#)
- RFE_CHIRP_SEQUENCE_CONFIGS_MAX
 - RFE Abstract 2.0 BLOB - Constants, [180](#)
- RFE_CHIRP_SEQUENCES_PER_RADAR_CYCLE_MAX
 - RFE Abstract 2.0 BLOB - Constants, [180](#)
- rfe_chirpPllLoopFilterBandwidth_1000kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [183](#)
- rfe_chirpPllLoopFilterBandwidth_1050kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [183](#)
- rfe_chirpPllLoopFilterBandwidth_1100kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [183](#)
- rfe_chirpPllLoopFilterBandwidth_1150kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [183](#)
- rfe_chirpPllLoopFilterBandwidth_1200kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [183](#)
- rfe_chirpPllLoopFilterBandwidth_1250kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [183](#)
- rfe_chirpPllLoopFilterBandwidth_1300kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [183](#)
- rfe_chirpPllLoopFilterBandwidth_1350kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [183](#)
- rfe_chirpPllLoopFilterBandwidth_1400kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [183](#)
- rfe_chirpPllLoopFilterBandwidth_1450kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [183](#)
- rfe_chirpPllLoopFilterBandwidth_1500kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [183](#)
- rfe_chirpPllLoopFilterBandwidth_1550kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [183](#)
- rfe_chirpPllLoopFilterBandwidth_1600kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [183](#)
- rfe_chirpPllLoopFilterBandwidth_1650kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [183](#)
- rfe_chirpPllLoopFilterBandwidth_1700kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [183](#)
- rfe_chirpPllLoopFilterBandwidth_200kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [182](#)
- rfe_chirpPllLoopFilterBandwidth_250kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [182](#)
- rfe_chirpPllLoopFilterBandwidth_300kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [182](#)
- rfe_chirpPllLoopFilterBandwidth_350kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [182](#)
- rfe_chirpPllLoopFilterBandwidth_400kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [182](#)
- rfe_chirpPllLoopFilterBandwidth_450kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [182](#)
- rfe_chirpPllLoopFilterBandwidth_500kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [182](#)
- rfe_chirpPllLoopFilterBandwidth_550kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [182](#)
- rfe_chirpPllLoopFilterBandwidth_600kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [182](#)
- rfe_chirpPllLoopFilterBandwidth_650kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [182](#)
- rfe_chirpPllLoopFilterBandwidth_700kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [182](#)
- rfe_chirpPllLoopFilterBandwidth_750kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [182](#)
- rfe_chirpPllLoopFilterBandwidth_800kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [183](#)
- rfe_chirpPllLoopFilterBandwidth_850kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [183](#)
- rfe_chirpPllLoopFilterBandwidth_900kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [183](#)
- rfe_chirpPllLoopFilterBandwidth_950kHz_e
 - RFE Abstract 2.0 BLOB - Constants, [183](#)
- rfe_chirpPllLoopFilterBandwidth_invalid_e
 - RFE Abstract 2.0 BLOB - Constants, [183](#)
- rfe_chirpPllLoopFilterBandwidth_t
 - RFE Abstract 2.0 BLOB - Constants, [182](#)
- rfe_chirpPllVco_1GHz_e
 - RFE Abstract 2.0 - Low Level Interface, [84](#)
- rfe_chirpPllVco_2GHz_e
 - RFE Abstract 2.0 - Low Level Interface, [84](#)
- rfe_chirpPllVco_4GHz_e
 - RFE Abstract 2.0 - Low Level Interface, [84](#)
- rfe_chirpPllVco_t
 - RFE Abstract 2.0 - Low Level Interface, [84](#)
- rfe_chirpProfileIndex_0_e
 - RFE Abstract 2.0 - Low Level Interface, [84](#)
- rfe_chirpProfileIndex_1_e
 - RFE Abstract 2.0 - Low Level Interface, [84](#)
- rfe_chirpProfileIndex_2_e
 - RFE Abstract 2.0 - Low Level Interface, [84](#)
- rfe_chirpProfileIndex_3_e
 - RFE Abstract 2.0 - Low Level Interface, [84](#)
- rfe_chirpProfileIndex_4_e
 - RFE Abstract 2.0 - Low Level Interface, [84](#)
- rfe_chirpProfileIndex_5_e
 - RFE Abstract 2.0 - Low Level Interface, [84](#)
- rfe_chirpProfileIndex_6_e
 - RFE Abstract 2.0 - Low Level Interface, [84](#)
- rfe_chirpProfileIndex_7_e
 - RFE Abstract 2.0 - Low Level Interface, [84](#)
- rfe_chirpProfileIndex_invalid_e
 - RFE Abstract 2.0 - Low Level Interface, [84](#)
- rfe_chirpProfileIndex_t
 - RFE Abstract 2.0 - Low Level Interface, [84](#)
- rfe_chirpSequenceIndex_0_e
 - RFE Abstract 2.0 - Low Level Interface, [84](#)
- rfe_chirpSequenceIndex_1_e
 - RFE Abstract 2.0 - Low Level Interface, [84](#)
- rfe_chirpSequenceIndex_2_e
 - RFE Abstract 2.0 - Low Level Interface, [85](#)
- rfe_chirpSequenceIndex_3_e
 - RFE Abstract 2.0 - Low Level Interface, [85](#)
- rfe_chirpSequenceIndex_4_e
 - RFE Abstract 2.0 - Low Level Interface, [85](#)
- rfe_chirpSequenceIndex_5_e
 - RFE Abstract 2.0 - Low Level Interface, [85](#)
- rfe_chirpSequenceIndex_6_e
 - RFE Abstract 2.0 - Low Level Interface, [85](#)
- rfe_chirpSequenceIndex_7_e
 - RFE Abstract 2.0 - Low Level Interface, [85](#)
- rfe_chirpSequenceIndex_invalid_e
 - RFE Abstract 2.0 - Low Level Interface, [85](#)

- RFE Abstract 2.0 - Low Level Interface, 85
- rfe_chirpSequenceIndex_t
 - RFE Abstract 2.0 - Low Level Interface, 84
- rfe_chirpSlopeDirection_falling_e
 - RFE Abstract 2.0 - Low Level Interface, 85
- rfe_chirpSlopeDirection_rising_e
 - RFE Abstract 2.0 - Low Level Interface, 85
- rfe_chirpSlopeDirection_t
 - RFE Abstract 2.0 - Low Level Interface, 85
- RFE_CMD_CLIENT_FUSA_FAULT_BYTE_COUNT
 - RFE Abstract 2.0 - Low Level Constants, 51
- RFE_CMD_CLIENT_UPDATE_SECTION_DYNAMIC_TABLE_e
 - RFE Abstract 2.0 - Low Level Constants, 51
- rfe_configure
 - RFE Abstract 2.0 - Low Level Interface, 121
- rfe_configureInterrupt
 - RFE Abstract 2.0 - Low Level Interface, 122
- rfe_coreId_a53_e
 - RFE Abstract 2.0 - Low Level Interface, 85
- rfe_coreId_appM7_e
 - RFE Abstract 2.0 - Low Level Interface, 85
- rfe_coreId_et
 - RFE Abstract 2.0 - Low Level Interface, 85
- rfe_coreId_max
 - RFE Abstract 2.0 - Low Level Interface, 85
- rfe_coreId_none_e
 - RFE Abstract 2.0 - Low Level Interface, 85
- rfe_coreId_t
 - RFE Abstract 2.0 - Low Level Interface, 81
- RFE_CS12_REG_NUMBER
 - RFE Abstract 2.0 Constants, 16
- RFE_DATA_OUT_DEST_CS12
 - RFE Abstract 2.0 - Low Level Constants, 51
- RFE_DATA_OUT_DEST_PACKET_PROCESSOR
 - RFE Abstract 2.0 - Low Level Constants, 52
- rfe_dataOutDest_t
 - RFE Abstract 2.0 - Low Level Interface, 81
- RFE_DYN_PARAMS_SELECT_ALL
 - RFE Abstract 2.0 - Low Level Constants, 52
- RFE_DYN_PARAMS_SELECT_CHIRP_FREQUENCY_DRIVE_STEPS_e
 - RFE Abstract 2.0 - Low Level Constants, 52
- RFE_DYN_PARAMS_SELECT_CHIRP_PROFILE
 - RFE Abstract 2.0 - Low Level Constants, 52
- RFE_DYN_PARAMS_SELECT_CIT_TICKS
 - RFE Abstract 2.0 - Low Level Constants, 52
- RFE_DYN_PARAMS_SELECT_DWELL_TIME_TICKS
 - RFE Abstract 2.0 - Low Level Constants, 52
- RFE_DYN_PARAMS_SELECT_TX1_PHASE_ROTATION
 - RFE Abstract 2.0 - Low Level Constants, 52
- RFE_DYN_PARAMS_SELECT_TX2_PHASE_ROTATION
 - RFE Abstract 2.0 - Low Level Constants, 53
- RFE_DYN_PARAMS_SELECT_TX3_PHASE_ROTATION
 - RFE Abstract 2.0 - Low Level Constants, 53
- RFE_DYN_PARAMS_SELECT_TX4_PHASE_ROTATION
 - RFE Abstract 2.0 - Low Level Constants, 53
- RFE_DYN_PARAMS_SELECT_TX_TRANSMISSION_ENABLE_e
 - RFE Abstract 2.0 - Low Level Constants, 53
- rfe_dynParamsSelect_t
 - RFE Abstract 2.0 - Low Level Interface, 81
- rfe_effectiveSamplingFrequency_10MHz_e
 - RFE Abstract 2.0 BLOB - Constants, 184
- rfe_effectiveSamplingFrequency_20MHz_e
 - RFE Abstract 2.0 BLOB - Constants, 183
- rfe_effectiveSamplingFrequency_40MHz_e
 - RFE Abstract 2.0 BLOB - Constants, 183
- rfe_effectiveSamplingFrequency_5MHz_e
 - RFE Abstract 2.0 BLOB - Constants, 184
- rfe_effectiveSamplingFrequency_invalid_e
 - RFE Abstract 2.0 BLOB - Constants, 184
- rfe_effectiveSamplingFrequency_t
 - RFE Abstract 2.0 BLOB - Constants, 183
- rfe_error_1_e
 - RFE Abstract 2.0 - Low Level Interface, 87
- rfe_error_2_e
 - RFE Abstract 2.0 - Low Level Interface, 87
- rfe_error_3_e
 - RFE Abstract 2.0 - Low Level Interface, 87
- rfe_error_addressOutOfValidSharedMemory_e
 - RFE Abstract 2.0 - Low Level Interface, 87
- rfe_error_api_blob_crc_wrong_e
 - RFE Abstract 2.0 - Low Level Interface, 87
- rfe_error_api_busy_e
 - RFE Abstract 2.0 - Low Level Interface, 86
- rfe_error_api_cfgInvalidParameterSize_e
 - RFE Abstract 2.0 - Low Level Interface, 89
- rfe_error_api_cfgInvalidParameterSize_rfeDriverOnly_e
 - RFE Abstract 2.0 - Low Level Interface, 86
- rfe_error_api_cfgSectionMismatch_e
 - RFE Abstract 2.0 - Low Level Interface, 89
- rfe_error_api_cfgSectionMismatch_external_e
 - RFE Abstract 2.0 - Low Level Interface, 86
- rfe_error_api_configParamCrossCheck_acqJumpbackResetTimeTooShort_e
 - RFE Abstract 2.0 - Low Level Interface, 86
- rfe_error_api_configParamCrossCheck_chirpIntervalTimeTooShort_e
 - RFE Abstract 2.0 - Low Level Interface, 86
- rfe_error_api_configParamCrossCheck_dwellsSettleTimeTooShort_e
 - RFE Abstract 2.0 - Low Level Interface, 86
- rfe_error_api_configParamCrossCheck_unconfiguredChirpProfile_e
 - RFE Abstract 2.0 - Low Level Interface, 87
- rfe_error_api_configParamCrossCheck_unconfiguredChirpSequence_e
 - RFE Abstract 2.0 - Low Level Interface, 86
- rfe_error_api_configParamCrossCheck_unconfiguredDynamicTable_e
 - RFE Abstract 2.0 - Low Level Interface, 87
- rfe_error_api_dynamicTablesNotConfigured_e
 - RFE Abstract 2.0 - Low Level Interface, 86
- rfe_error_api_fsm_was_started_e
 - RFE Abstract 2.0 - Low Level Interface, 87
- rfe_error_api_function_not_implemented_e
 - RFE Abstract 2.0 - Low Level Interface, 87
- rfe_error_api_invalid_frontends_conf_e
 - RFE Abstract 2.0 - Low Level Interface, 87
- rfe_error_api_invalid_num_frontends_e
 - RFE Abstract 2.0 - Low Level Interface, 87
- rfe_error_api_invalidArgumentValue_e
 - RFE Abstract 2.0 - Low Level Interface, 86
- rfe_error_api_invalidConfigurationParameterValue_e
 - RFE Abstract 2.0 - Low Level Interface, 86

- RFE Abstract 2.0 - Low Level Interface, [86](#)
- rfe_error_api_invalidConfigurationSize_e
 - RFE Abstract 2.0 - Low Level Interface, [86](#)
- rfe_error_api_invalidConfigurationVersion_e
 - RFE Abstract 2.0 - Low Level Interface, [87](#)
- rfe_error_api_invalidDynamicTableSize_e
 - RFE Abstract 2.0 - Low Level Interface, [86](#)
- rfe_error_api_invalidDynamicTableValue_e
 - RFE Abstract 2.0 - Low Level Interface, [86](#)
- rfe_error_api_invalidFrontendId_e
 - RFE Abstract 2.0 - Low Level Interface, [87](#)
- rfe_error_api_invalidParameterForUpdate_e
 - RFE Abstract 2.0 - Low Level Interface, [86](#)
- rfe_error_api_invalidRadarCycleTimeSchedule_e
 - RFE Abstract 2.0 - Low Level Interface, [86](#)
- rfe_error_api_maxUpdateCountExceeded_e
 - RFE Abstract 2.0 - Low Level Interface, [86](#)
- rfe_error_api_noUpdatesBuffered_e
 - RFE Abstract 2.0 - Low Level Interface, [86](#)
- rfe_error_api_operationNotAllowed_e
 - RFE Abstract 2.0 - Low Level Interface, [86](#)
- rfe_error_api_unresponsive_e
 - RFE Abstract 2.0 - Low Level Interface, [86](#)
- rfe_error_api_unsynchronized_e
 - RFE Abstract 2.0 - Low Level Interface, [86](#)
- rfe_error_api_updateAlreadyPending_e
 - RFE Abstract 2.0 - Low Level Interface, [86](#)
- rfe_error_atbAdcInvalidAveragingSamples_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_atbAdcNoCalibrated_e
 - RFE Abstract 2.0 - Low Level Interface, [92](#)
- rfe_error_atbAdcNotConfiguredToSample_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_atbAdcNotInSingleMode_e
 - RFE Abstract 2.0 - Low Level Interface, [89](#)
- rfe_error_atbBandGapInvalidVout_e
 - RFE Abstract 2.0 - Low Level Interface, [89](#)
- rfe_error_atbBandGapNotStarted_e
 - RFE Abstract 2.0 - Low Level Interface, [89](#)
- rfe_error_atbCrossSwitchNotEnabled_e
 - RFE Abstract 2.0 - Low Level Interface, [89](#)
- rfe_error_atbCurrentMeasurementWithoutResistors_e
 - RFE Abstract 2.0 - Low Level Interface, [89](#)
- rfe_error_atbInvalidAdcInputDiffMode_e
 - RFE Abstract 2.0 - Low Level Interface, [89](#)
- rfe_error_atbInvalidAdcSelSingleMode_e
 - RFE Abstract 2.0 - Low Level Interface, [89](#)
- rfe_error_atbInvalidAdcSingleModeGround_e
 - RFE Abstract 2.0 - Low Level Interface, [89](#)
- rfe_error_atbInvalidAtbAdc_e
 - RFE Abstract 2.0 - Low Level Interface, [91](#)
- rfe_error_atbInvalidAtbAdcGainMode_e
 - RFE Abstract 2.0 - Low Level Interface, [91](#)
- rfe_error_atbInvalidAtbMaster_e
 - RFE Abstract 2.0 - Low Level Interface, [89](#)
- rfe_error_atbInvalidInputSource_e
 - RFE Abstract 2.0 - Low Level Interface, [92](#)
- rfe_error_atbInvalidOperationalMode_e
 - RFE Abstract 2.0 - Low Level Interface, [89](#)
- rfe_error_atbInvalidPullDownInput_e
 - RFE Abstract 2.0 - Low Level Interface, [89](#)
- rfe_error_atbInvalidTrimmingSelectedResistor_e
 - RFE Abstract 2.0 - Low Level Interface, [89](#)
- rfe_error_atbModeOfOperationNotSet_e
 - RFE Abstract 2.0 - Low Level Interface, [89](#)
- rfe_error_atbNotSupported_e
 - RFE Abstract 2.0 - Low Level Interface, [89](#)
- rfe_error_bc_adcCalibrationCheckNotDone
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_adcCalibrationFailed
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_adcCalibrationTimeout
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_algDummyError
 - RFE Abstract 2.0 - Low Level Interface, [95](#)
- rfe_error_bc_atbBistAdcTimeout
 - RFE Abstract 2.0 - Low Level Interface, [99](#)
- rfe_error_bc_atbWrongAtbNoSelected
 - RFE Abstract 2.0 - Low Level Interface, [99](#)
- rfe_error_bc_cafcAafcKvCodVdtTimeout
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_cafcBwOutOfRange
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_cafcConfigLoopFilterFail
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_cafcDvdtCountWrong
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_cafcFreqCntTimeout
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_cafcInputOutOfRange
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_cafcKvcoCountStatusErr
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_cafcLoopBwOutOfRange
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_cafcNoStoreInProfileSelected
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_cafcPtrAddrNull
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_cafcSbOutOfRange
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_cafcVcoCalibAafcFail
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_cafcVcoCalibKvCodVdtFail
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_cafcWrongAcquisitionDuration
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_cafcWrongKvco
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_cafcWrongProfileId
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_ccPtrAddrNull
 - RFE Abstract 2.0 - Low Level Interface, [95](#)
- rfe_error_bc_ccSysConfNotSet
 - RFE Abstract 2.0 - Low Level Interface, [95](#)
- rfe_error_bc_failToInjectCrcErr

- RFE Abstract 2.0 - Low Level Interface, 93
- rfe_error_bc_funcNoExist
 - RFE Abstract 2.0 - Low Level Interface, 93
- rfe_error_bc_gbiasDummy
 - RFE Abstract 2.0 - Low Level Interface, 97
- rfe_error_bc_gldoSnsError
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_gldoSnsForceError
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_gldoSnsResetError
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_inputOutOfRange
 - RFE Abstract 2.0 - Low Level Interface, 93
- rfe_error_bc_ipDisabled
 - RFE Abstract 2.0 - Low Level Interface, 93
- rfe_error_bc_ismCalBusy
 - RFE Abstract 2.0 - Low Level Interface, 100
- rfe_error_bc_ismCalError
 - RFE Abstract 2.0 - Low Level Interface, 100
- rfe_error_bc_ismCalSuccess
 - RFE Abstract 2.0 - Low Level Interface, 100
- rfe_error_bc_ismDisableLbToggleTestFailed
 - RFE Abstract 2.0 - Low Level Interface, 100
- rfe_error_bc_ismFitAdcStatusFailed
 - RFE Abstract 2.0 - Low Level Interface, 100
- rfe_error_bc_ismFitChirpStatusFailed
 - RFE Abstract 2.0 - Low Level Interface, 100
- rfe_error_bc_ismFitErrorN_High
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_ismFitFailed
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_ismFitForceErrorFailed
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_ismFitForceStateChangeFailed
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_ismFitMclkLolismOtpCcStatusFailed
 - RFE Abstract 2.0 - Low Level Interface, 100
- rfe_error_bc_ismFitResetErrorFailed
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_ismFitResetStateChangeFailed
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_ismFitRxStatusFailed
 - RFE Abstract 2.0 - Low Level Interface, 100
- rfe_error_bc_ismFitSsbSerGbGldoAAtbStatusFailed
 - RFE Abstract 2.0 - Low Level Interface, 100
- rfe_error_bc_ismFitStatusFailed
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_ismFitTxStatusFailed
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_ismInputModeToggleTestFailed
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_ismInvalidInput
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_ismOutputModeToggleTestFailed
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_ismPtrAddrNull
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_ismStartupSeqConfigFailed
 - RFE Abstract 2.0 - Low Level Interface, 99
- RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_loiCalBusy
 - RFE Abstract 2.0 - Low Level Interface, 97
- rfe_error_bc_loiCalError
 - RFE Abstract 2.0 - Low Level Interface, 97
- rfe_error_bc_loiLutGainCodeOutOfRange
 - RFE Abstract 2.0 - Low Level Interface, 97
- rfe_error_bc_loiPtrAddrNull
 - RFE Abstract 2.0 - Low Level Interface, 97
- rfe_error_bc_lvdsAdcClkFromInvalid
 - RFE Abstract 2.0 - Low Level Interface, 95
- rfe_error_bc_lvdsClkEdgeInvalid
 - RFE Abstract 2.0 - Low Level Interface, 95
- rfe_error_bc_lvdsCountPeriodInvalid
 - RFE Abstract 2.0 - Low Level Interface, 95
- rfe_error_bc_lvdsFreqCounterDeviation
 - RFE Abstract 2.0 - Low Level Interface, 95
- rfe_error_bc_lvdsFreqCountTimeout
 - RFE Abstract 2.0 - Low Level Interface, 95
- rfe_error_bc_lvdsInvalidInput
 - RFE Abstract 2.0 - Low Level Interface, 95
- rfe_error_bc_lvdsPtrAddrNull
 - RFE Abstract 2.0 - Low Level Interface, 95
- rfe_error_bc_lvdsrestoreFreqCounter
 - RFE Abstract 2.0 - Low Level Interface, 95
- rfe_error_bc_lvdsSamplesShiftInvalid
 - RFE Abstract 2.0 - Low Level Interface, 95
- rfe_error_bc_mclkMspcCalibrationError
 - RFE Abstract 2.0 - Low Level Interface, 97
- rfe_error_bc_mclkMspcTimeout
 - RFE Abstract 2.0 - Low Level Interface, 97
- rfe_error_bc_mclkPIICalibrationFailed
 - RFE Abstract 2.0 - Low Level Interface, 97
- rfe_error_bc_mclkPIICalibrationNotFinished
 - RFE Abstract 2.0 - Low Level Interface, 97
- rfe_error_bc_mclkPtrAddrNull
 - RFE Abstract 2.0 - Low Level Interface, 97
- rfe_error_bc_mipiDeviceReadyTimeout
 - RFE Abstract 2.0 - Low Level Interface, 95
- rfe_error_bc_mipiFreqCntOutOfRange
 - RFE Abstract 2.0 - Low Level Interface, 95
- rfe_error_bc_mipiFreqCountTimeout
 - RFE Abstract 2.0 - Low Level Interface, 95
- rfe_error_bc_mipiPtrAddrNull
 - RFE Abstract 2.0 - Low Level Interface, 95
- rfe_error_bc_mipiRestoreFreqCounter
 - RFE Abstract 2.0 - Low Level Interface, 95
- rfe_error_bc_otpAllMemDisabled
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_otpCrcCheckFailure
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_otpFailToRead
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_otplplsDisabled
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_otpOtplsNotLoaded
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_readFail

- RFE Abstract 2.0 - Low Level Interface, [95](#)
- rfe_error_bc_rx1BbdEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_rx1CalBusy
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx1CalError
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx1FusaEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx1Rx2BbdEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_rx1Rx2CalError
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx1Rx2FusaEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx1Rx2Rx3BbdEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_rx1Rx2Rx3CalError
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx1Rx2Rx3FusaEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx1Rx2Rx3Rx4BbdEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_rx1Rx2Rx3Rx4CalError
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx1Rx2Rx3Rx4FusaEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_rx1Rx2Rx4BbdEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_rx1Rx2Rx4CalError
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx1Rx2Rx4FusaEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx1Rx3BbdEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_rx1Rx3CalError
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx1Rx3FusaEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx1Rx3Rx4BbdEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_rx1Rx3Rx4CalError
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx1Rx3Rx4FusaEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_rx1Rx4BbdEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_rx1Rx4CalError
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx1Rx4FusaEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx2BbdEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_rx2CalBusy
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx2CalError
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx2FusaEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx2Rx3BbdEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_rx2Rx3CalError
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx2Rx3FusaEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx2Rx3Rx4BbdEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_rx2Rx3Rx4CalError
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx2Rx3Rx4FusaEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_rx2Rx4BbdEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_rx2Rx4CalError
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx2Rx4FusaEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx3BbdEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_rx3CalBusy
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx3CalError
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx3FusaEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx3Rx4BbdEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_rx3Rx4CalError
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx3Rx4FusaEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_rx4BbdEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_rx4CalBusy
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx4CalError
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rx4FusaEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rxBbdEnableWritecheckBase
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_rxCalBusy
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rxCalError
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rxFusaEnableWritecheckBase
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rxInputOutOfRange
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rxPrtAddrNull
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_rxWrngCalTargetOffset
 - RFE Abstract 2.0 - Low Level Interface, [97](#)
- rfe_error_bc_scInputFreqDriftInvalid
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_scOperatingFreqOutOfRange
 - RFE Abstract 2.0 - Low Level Interface, [97](#)

- RFE Abstract 2.0 - Low Level Interface, 96
- rfe_error_bc_scProfile
 - RFE Abstract 2.0 - Low Level Interface, 96
- rfe_error_bc_scPtrAddrNull
 - RFE Abstract 2.0 - Low Level Interface, 96
- rfe_error_bc_scRampTimeOutOfRange
 - RFE Abstract 2.0 - Low Level Interface, 96
- rfe_error_bc_scResetTimeOutOfRange
 - RFE Abstract 2.0 - Low Level Interface, 96
- rfe_error_bc_scUserBwOutOfRange
 - RFE Abstract 2.0 - Low Level Interface, 96
- rfe_error_bc_spiWriteCheckFail
 - RFE Abstract 2.0 - Low Level Interface, 95
- rfe_error_bc_spiWriteReadCrcError
 - RFE Abstract 2.0 - Low Level Interface, 95
- rfe_error_bc_ssbInInputOutOfRange
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_ssbSignalNotExistOnSelectedAtb
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_sys_aafcAutoNotSupported
 - RFE Abstract 2.0 - Low Level Interface, 94
- rfe_error_bc_sys_adclInvalid
 - RFE Abstract 2.0 - Low Level Interface, 93
- rfe_error_bc_sys_cafcIntegrityCheckFailed
 - RFE Abstract 2.0 - Low Level Interface, 94
- rfe_error_bc_sys_cafcIntegrityCheckLevel
 - RFE Abstract 2.0 - Low Level Interface, 94
- rfe_error_bc_sys_cafcIntegrityCheckUnlock
 - RFE Abstract 2.0 - Low Level Interface, 94
- rfe_error_bc_sys_cafcTempReadTimeout
 - RFE Abstract 2.0 - Low Level Interface, 94
- rfe_error_bc_sys_cannotFindCafcDacCode
 - RFE Abstract 2.0 - Low Level Interface, 93
- rfe_error_bc_sys_error_N
 - RFE Abstract 2.0 - Low Level Interface, 94
- rfe_error_bc_sys_errorRecoveryFailed
 - RFE Abstract 2.0 - Low Level Interface, 93
- rfe_error_bc_sys_faultInjectionFail
 - RFE Abstract 2.0 - Low Level Interface, 94
- rfe_error_bc_sys_fileError
 - RFE Abstract 2.0 - Low Level Interface, 93
- rfe_error_bc_sys_invalidInput
 - RFE Abstract 2.0 - Low Level Interface, 93
- rfe_error_bc_sys_invalidProfile
 - RFE Abstract 2.0 - Low Level Interface, 94
- rfe_error_bc_sys_invalidResetTime
 - RFE Abstract 2.0 - Low Level Interface, 94
- rfe_error_bc_sys_ipHasNoAtbConnection
 - RFE Abstract 2.0 - Low Level Interface, 93
- rfe_error_bc_sys_jsonStringNoExisted
 - RFE Abstract 2.0 - Low Level Interface, 93
- rfe_error_bc_sys_loiInBallBreakDetected
 - RFE Abstract 2.0 - Low Level Interface, 95
- rfe_error_bc_sys_loiOutBallBreakDetected
 - RFE Abstract 2.0 - Low Level Interface, 95
- rfe_error_bc_sys_mclkIntegrityFailed
 - RFE Abstract 2.0 - Low Level Interface, 93
- rfe_error_bc_sys_ptrAddrNull
 - RFE Abstract 2.0 - Low Level Interface, 94
- rfe_error_bc_sys_rawNumSampleInvalid
 - RFE Abstract 2.0 - Low Level Interface, 94
- rfe_error_bc_sys_regCrcIsModuleErrorSet
 - RFE Abstract 2.0 - Low Level Interface, 94
- rfe_error_bc_sys_regCrcTriggerFailed
 - RFE Abstract 2.0 - Low Level Interface, 94
- rfe_error_bc_sys_restoreChirpGlobalFailed
 - RFE Abstract 2.0 - Low Level Interface, 94
- rfe_error_bc_sys_slaveWrongAppConfig
 - RFE Abstract 2.0 - Low Level Interface, 94
- rfe_error_bc_sys_spiAccessCheckFail
 - RFE Abstract 2.0 - Low Level Interface, 94
- rfe_error_bc_sys_subBandIncOutOfRange
 - RFE Abstract 2.0 - Low Level Interface, 94
- rfe_error_bc_sys_totalBwOutOfRange
 - RFE Abstract 2.0 - Low Level Interface, 94
- rfe_error_bc_sys_tx1CtrlEnableWriteCheck
 - RFE Abstract 2.0 - Low Level Interface, 94
- rfe_error_bc_sys_tx1Tx2CtrlEnableWriteCheck
 - RFE Abstract 2.0 - Low Level Interface, 94
- rfe_error_bc_sys_tx1Tx2Tx3CtrlEnableWriteCheck
 - RFE Abstract 2.0 - Low Level Interface, 95
- rfe_error_bc_sys_tx1Tx3CtrlEnableWriteCheck
 - RFE Abstract 2.0 - Low Level Interface, 95
- rfe_error_bc_sys_tx2CtrlEnableWriteCheck
 - RFE Abstract 2.0 - Low Level Interface, 94
- rfe_error_bc_sys_tx2Tx3CtrlEnableWriteCheck
 - RFE Abstract 2.0 - Low Level Interface, 95
- rfe_error_bc_sys_tx3CtrlEnableWriteCheck
 - RFE Abstract 2.0 - Low Level Interface, 94
- rfe_error_bc_sys_txCtrlEnableWriteCheckBase
 - RFE Abstract 2.0 - Low Level Interface, 94
- rfe_error_bc_sys_txTempReadTimeout
 - RFE Abstract 2.0 - Low Level Interface, 93
- rfe_error_bc_sys_wrongAppconfig
 - RFE Abstract 2.0 - Low Level Interface, 93
- rfe_error_bc_sys_wrongDataInterfaceConfig
 - RFE Abstract 2.0 - Low Level Interface, 94
- rfe_error_bc_sys_wrongMidNoRecovery
 - RFE Abstract 2.0 - Low Level Interface, 94
- rfe_error_bc_sys_wrongProfileNoRecovery
 - RFE Abstract 2.0 - Low Level Interface, 93
- rfe_error_bc_teBiasDelay
 - RFE Abstract 2.0 - Low Level Interface, 95
- rfe_error_bc_teChirpEnableWriteCheckFailed
 - RFE Abstract 2.0 - Low Level Interface, 96
- rfe_error_bc_teChirpInterval
 - RFE Abstract 2.0 - Low Level Interface, 96
- rfe_error_bc_teChirpSeqInterval
 - RFE Abstract 2.0 - Low Level Interface, 95
- rfe_error_bc_teChirpSequence
 - RFE Abstract 2.0 - Low Level Interface, 96
- rfe_error_bc_teChirpTrgMode
 - RFE Abstract 2.0 - Low Level Interface, 96
- rfe_error_bc_teDcPowOnDelay
 - RFE Abstract 2.0 - Low Level Interface, 95
- rfe_error_bc_teDcSafetyDelay
 - RFE Abstract 2.0 - Low Level Interface, 95

- RFE Abstract 2.0 - Low Level Interface, [95](#)
- rfe_error_bc_teDdmaMode
 - RFE Abstract 2.0 - Low Level Interface, [95](#)
- rfe_error_bc_teDdmaSetting
 - RFE Abstract 2.0 - Low Level Interface, [95](#)
- rfe_error_bc_teDelayFromRefpt
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_teDwlTime
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_teFineGrpDelay
 - RFE Abstract 2.0 - Low Level Interface, [95](#)
- rfe_error_bc_teFreqCntTimeout
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_teHpfResetDelay
 - RFE Abstract 2.0 - Low Level Interface, [95](#)
- rfe_error_bc_teHpfResetWrongTriggerPoint
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_teInputNull
 - RFE Abstract 2.0 - Low Level Interface, [95](#)
- rfe_error_bc_teJumpBack
 - RFE Abstract 2.0 - Low Level Interface, [95](#)
- rfe_error_bc_teOutputSampRate
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_tePhaseOutOfRange
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_tePrCalibDelay
 - RFE Abstract 2.0 - Low Level Interface, [95](#)
- rfe_error_bc_tePrCalTimeout
 - RFE Abstract 2.0 - Low Level Interface, [95](#)
- rfe_error_bc_tePrfRepeatCnt
 - RFE Abstract 2.0 - Low Level Interface, [95](#)
- rfe_error_bc_teProfileList
 - RFE Abstract 2.0 - Low Level Interface, [95](#)
- rfe_error_bc_tePrSafetyDelay
 - RFE Abstract 2.0 - Low Level Interface, [95](#)
- rfe_error_bc_teRxActiveDelay
 - RFE Abstract 2.0 - Low Level Interface, [95](#)
- rfe_error_bc_teSettleTime
 - RFE Abstract 2.0 - Low Level Interface, [95](#)
- rfe_error_bc_teTx1CalEnableWriteCheck
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_teTx1Tx2CalEnableWriteCheck
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_teTx1Tx2Tx3CalEnableWriteCheck
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_teTx1Tx3CalEnableWriteCheck
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_teTx2CalEnableWriteCheck
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_teTx2Tx3CalEnableWriteCheck
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_teTx3CalEnableWriteCheck
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_teTxCalEnableWriteCheckBase
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_teTxRxGrpDelay
 - RFE Abstract 2.0 - Low Level Interface, [95](#)
- rfe_error_bc_teVirtualChannel
 - RFE Abstract 2.0 - Low Level Interface, [96](#)
- rfe_error_bc_tx1BbdEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [99](#)
- rfe_error_bc_tx1CalBusy
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_tx1CalError
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_tx1FusaEnableWriteCheck
 - RFE Abstract 2.0 - Low Level Interface, [99](#)
- rfe_error_bc_tx1SafetyBusy
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_tx1SafetyError
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_tx1tx2BbdEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [99](#)
- rfe_error_bc_tx1Tx2CalError
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_tx1Tx2FusaEnableWriteCheck
 - RFE Abstract 2.0 - Low Level Interface, [99](#)
- rfe_error_bc_tx1Tx2SafetyError
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_tx1Tx2Tx3BbdEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [99](#)
- rfe_error_bc_tx1Tx2Tx3CalError
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_tx1Tx2Tx3FusaEnableWriteCheck
 - RFE Abstract 2.0 - Low Level Interface, [99](#)
- rfe_error_bc_tx1Tx2Tx3SafetyError
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_tx1Tx3BbdEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [99](#)
- rfe_error_bc_tx1Tx3CalError
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_tx1Tx3FusaEnableWriteCheck
 - RFE Abstract 2.0 - Low Level Interface, [99](#)
- rfe_error_bc_tx1Tx3SafetyError
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_tx2BbdEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [99](#)
- rfe_error_bc_tx2CalBusy
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_tx2CalError
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_tx2FusaEnableWriteCheck
 - RFE Abstract 2.0 - Low Level Interface, [99](#)
- rfe_error_bc_tx2SafetyBusy
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_tx2SafetyError
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_tx2Tx3BbdEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [99](#)
- rfe_error_bc_tx2Tx3CalError
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_tx2Tx3FusaEnableWriteCheck
 - RFE Abstract 2.0 - Low Level Interface, [99](#)
- rfe_error_bc_tx2Tx3SafetyError
 - RFE Abstract 2.0 - Low Level Interface, [98](#)
- rfe_error_bc_tx3BbdEnableWritecheck
 - RFE Abstract 2.0 - Low Level Interface, [98](#)

- RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_tx3CalBusy
 - RFE Abstract 2.0 - Low Level Interface, 98
- rfe_error_bc_tx3CalError
 - RFE Abstract 2.0 - Low Level Interface, 98
- rfe_error_bc_tx3FusaEnableWriteCheck
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_tx3SafetyBusy
 - RFE Abstract 2.0 - Low Level Interface, 98
- rfe_error_bc_tx3SafetyError
 - RFE Abstract 2.0 - Low Level Interface, 98
- rfe_error_bc_txBbdEnableWritecheckBase
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_txCalBusy
 - RFE Abstract 2.0 - Low Level Interface, 98
- rfe_error_bc_txCalError
 - RFE Abstract 2.0 - Low Level Interface, 98
- rfe_error_bc_txCalibBiasCurrWriteFailed
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_txCalibTimeout
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_txFusaEnableWriteCheckBase
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_txInputOutOfRange
 - RFE Abstract 2.0 - Low Level Interface, 98
- rfe_error_bc_txNoStoreInProfileSelected
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_txPoutVgaReduceTxNotSupported
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_txPrcAdcTimeout
 - RFE Abstract 2.0 - Low Level Interface, 98
- rfe_error_bc_txPtrAddrNull
 - RFE Abstract 2.0 - Low Level Interface, 98
- rfe_error_bc_txRfMinCodeOutOfRange
 - RFE Abstract 2.0 - Low Level Interface, 99
- rfe_error_bc_txSafetyBusy
 - RFE Abstract 2.0 - Low Level Interface, 98
- rfe_error_bc_txSafetyError
 - RFE Abstract 2.0 - Low Level Interface, 98
- rfe_error_bc_txSensorFitFail
 - RFE Abstract 2.0 - Low Level Interface, 98
- rfe_error_bc_wrongInterface
 - RFE Abstract 2.0 - Low Level Interface, 93
- rfe_error_bistZeroHourDataNotGenerated_e
 - RFE Abstract 2.0 - Low Level Interface, 92
- rfe_error_boundedWaitNotValidResource_e
 - RFE Abstract 2.0 - Low Level Interface, 90
- rfe_error_boundedWaitResourcesExhausted_e
 - RFE Abstract 2.0 - Low Level Interface, 89
- rfe_error_chirpPllBandgapNotOk_e
 - RFE Abstract 2.0 - Low Level Interface, 91
- rfe_error_chirpPllHvLdoCpNotOk_e
 - RFE Abstract 2.0 - Low Level Interface, 91
- rfe_error_chirpPllHvLdoVcoNotOk_e
 - RFE Abstract 2.0 - Low Level Interface, 91
- rfe_error_chirpPllLoopfilterC1NotOk_e
 - RFE Abstract 2.0 - Low Level Interface, 91
- rfe_error_chirpPllLoopfilterC2FastNotOk_e
 - RFE Abstract 2.0 - Low Level Interface, 91
- rfe_error_chirpPllLoopfilterC2SlowNotOk_e
 - RFE Abstract 2.0 - Low Level Interface, 91
- rfe_error_chirpPllLoopfilterC3FastNotOk_e
 - RFE Abstract 2.0 - Low Level Interface, 91
- rfe_error_chirpPllLoopfilterC3SlowNotOk_e
 - RFE Abstract 2.0 - Low Level Interface, 91
- rfe_error_chirpPllLoopfilterCpNotOk_e
 - RFE Abstract 2.0 - Low Level Interface, 91
- rfe_error_chirpPllLoopfilterR1NotOk_e
 - RFE Abstract 2.0 - Low Level Interface, 91
- rfe_error_chirpPllLoopfilterR2NotOk_e
 - RFE Abstract 2.0 - Low Level Interface, 91
- rfe_error_chirpPllLvLdoNotOk_e
 - RFE Abstract 2.0 - Low Level Interface, 91
- rfe_error_chirpPllPdivLdoNotOk_e
 - RFE Abstract 2.0 - Low Level Interface, 91
- rfe_error_chirpPllVcoCalAmpOutOfRange_e
 - RFE Abstract 2.0 - Low Level Interface, 92
- rfe_error_chirpPllVcoCalibrationFailed_e
 - RFE Abstract 2.0 - Low Level Interface, 89
- rfe_error_chirpStartedTooEarly_e
 - RFE Abstract 2.0 - Low Level Interface, 91
- rfe_error_cmdInterface_invalidCmdId_e
 - RFE Abstract 2.0 - Low Level Interface, 87
- rfe_error_cmdInterface_invalidCmdLength_e
 - RFE Abstract 2.0 - Low Level Interface, 87
- rfe_error_cmdInterface_invalidConfigCmdSequencing_e
 - RFE Abstract 2.0 - Low Level Interface, 87
- rfe_error_cmdInterface_invalidCrc_e
 - RFE Abstract 2.0 - Low Level Interface, 87
- rfe_error_cmdInterface_invalidResponse_e
 - RFE Abstract 2.0 - Low Level Interface, 87
- rfe_error_cmdInterface_invalidRspLength_e
 - RFE Abstract 2.0 - Low Level Interface, 87
- rfe_error_cmdServerDft_invalidParam_e
 - RFE Abstract 2.0 - Low Level Interface, 87
- rfe_error_cmdServerInternal_cmdbuf_overflow_e
 - RFE Abstract 2.0 - Low Level Interface, 88
- rfe_error_cmdServerInternal_no_cmd_received_e
 - RFE Abstract 2.0 - Low Level Interface, 87
- rfe_error_cmdServerInternal_out_of_seq_e
 - RFE Abstract 2.0 - Low Level Interface, 88
- rfe_error_cmdServerInternal_resplen_wrong_e
 - RFE Abstract 2.0 - Low Level Interface, 88
- rfe_error_cmdServerInternal_undefined_cmd_handler_e
 - RFE Abstract 2.0 - Low Level Interface, 87
- rfe_error_csi2PllcalibrationTimeOut_e
 - RFE Abstract 2.0 - Low Level Interface, 92
- rfe_error_csi2PllLockTimeOut_e
 - RFE Abstract 2.0 - Low Level Interface, 92
- rfe_error_dftInvalidOutputSelected_e
 - RFE Abstract 2.0 - Low Level Interface, 88
- rfe_error_dftNotSupported_e
 - RFE Abstract 2.0 - Low Level Interface, 89
- rfe_error_dftNumberOfAveragesOutOfBounds_e
 - RFE Abstract 2.0 - Low Level Interface, 88
- rfe_error_dftOnlySingleBinSupported_e

- RFE Abstract 2.0 - Low Level Interface, [89](#)
- rfe_error_dftRealOnlySupported_e
 - RFE Abstract 2.0 - Low Level Interface, [89](#)
- rfe_error_dftSamplesCountAndNumberOfAveragesNotMultiple_e
 - RFE Abstract 2.0 - Low Level Interface, [89](#)
- rfe_error_dftSamplesCountNot2Multiple_e
 - RFE Abstract 2.0 - Low Level Interface, [88](#)
- rfe_error_didt_test_failure_e
 - RFE Abstract 2.0 - Low Level Interface, [92](#)
- rfe_error_distrGroupDoesNotExist_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_dynamicProgrammingDuringAcquisition_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_fcc_e
 - RFE Abstract 2.0 - Low Level Interface, [92](#)
- rfe_error_fftInvalidOutputSelected_e
 - RFE Abstract 2.0 - Low Level Interface, [88](#)
- rfe_error_fftNotSupported_e
 - RFE Abstract 2.0 - Low Level Interface, [88](#)
- rfe_error_fftNumberOfAveragesOutOfBounds_e
 - RFE Abstract 2.0 - Low Level Interface, [88](#)
- rfe_error_fftSamplesCountAndNumberOfAveragesNotMultiple_e
 - RFE Abstract 2.0 - Low Level Interface, [88](#)
- rfe_error_fftSamplesCountNot2Multiple_e
 - RFE Abstract 2.0 - Low Level Interface, [88](#)
- rfe_error_fic1ProcedureFailed_e
 - RFE Abstract 2.0 - Low Level Interface, [88](#)
- rfe_error_fic2ProcedureFailed_e
 - RFE Abstract 2.0 - Low Level Interface, [88](#)
- rfe_error_fit1ProcedureFailed_e
 - RFE Abstract 2.0 - Low Level Interface, [88](#)
- rfe_error_fit2ProcedureFailed_e
 - RFE Abstract 2.0 - Low Level Interface, [88](#)
- rfe_error_histogramWrongOutputSize_e
 - RFE Abstract 2.0 - Low Level Interface, [88](#)
- rfe_error_histogramWrongXAxisLimit_e
 - RFE Abstract 2.0 - Low Level Interface, [88](#)
- rfe_error_hwPitNotInitialized_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_HwRegister_lock_e
 - RFE Abstract 2.0 - Low Level Interface, [92](#)
- rfe_error_ic_wrg_input_params_e
 - RFE Abstract 2.0 - Low Level Interface, [100](#)
- rfe_error_initLldoPdcPowerNotOk_e
 - RFE Abstract 2.0 - Low Level Interface, [92](#)
- rfe_error_initMcgen0v9AnaNotOk_e
 - RFE Abstract 2.0 - Low Level Interface, [92](#)
- rfe_error_initRtmErrorReported
 - RFE Abstract 2.0 - Low Level Interface, [92](#)
- rfe_error_invalidADCNumberOfBits_e
 - RFE Abstract 2.0 - Low Level Interface, [88](#)
- rfe_error_invalidFlexIOConfig_e
 - RFE Abstract 2.0 - Low Level Interface, [88](#)
- rfe_error_invalidFractionBits_e
 - RFE Abstract 2.0 - Low Level Interface, [88](#)
- rfe_error_invalidPointer_e
 - RFE Abstract 2.0 - Low Level Interface, [87](#)
- rfe_error_invalidRadarCycleTiming_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_invalidRfeAccessCsrConfig_e
 - RFE Abstract 2.0 - Low Level Interface, [88](#)
- rfe_error_invalidSamplesNumber_e
 - RFE Abstract 2.0 - Low Level Interface, [87](#)
- rfe_error_invalidShiftingParam_e
 - RFE Abstract 2.0 - Low Level Interface, [88](#)
- rfe_error_invalidValue_e
 - RFE Abstract 2.0 - Low Level Interface, [88](#)
- rfe_error_IpUartNotActive_e
 - RFE Abstract 2.0 - Low Level Interface, [92](#)
- rfe_error_mainFsm_chirpAcquisitionStartInterruptTimeout_e
 - RFE Abstract 2.0 - Low Level Interface, [91](#)
- rfe_error_mainFsm_chirpSequenceEndInterruptTimeout_e
 - RFE Abstract 2.0 - Low Level Interface, [91](#)
- rfe_error_mainFsm_chirpStartInterruptTimeout_e
 - RFE Abstract 2.0 - Low Level Interface, [91](#)
- rfe_error_mainFsm_eventAlreadyActive_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_mainFsm_eventNotAllowed_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_mainFsm_mcgenAdpIILocked_e
 - RFE Abstract 2.0 - Low Level Interface, [91](#)
- rfe_error_mcgen0v9AnaNotOk_e
 - RFE Abstract 2.0 - Low Level Interface, [92](#)
- rfe_error_mcgenAdpIILockDetNotLocked_e
 - RFE Abstract 2.0 - Low Level Interface, [92](#)
- rfe_error_mcgenAdpIILOutVdd0v9DigNotOk_e
 - RFE Abstract 2.0 - Low Level Interface, [92](#)
- rfe_error_mcgenXoDetPILClkError_e
 - RFE Abstract 2.0 - Low Level Interface, [92](#)
- rfe_error_none_e
 - RFE Abstract 2.0 - Low Level Interface, [86](#)
- rfe_error_noSpaceLeftInSharedMemory_e
 - RFE Abstract 2.0 - Low Level Interface, [88](#)
- rfe_error_notAnExternalErrorCode_e
 - RFE Abstract 2.0 - Low Level Interface, [88](#)
- rfe_error_notYetImplemented_e
 - RFE Abstract 2.0 - Low Level Interface, [88](#)
- rfe_error_nullPointer_e
 - RFE Abstract 2.0 - Low Level Interface, [88](#)
- rfe_error_param_invalidParam_e
 - RFE Abstract 2.0 - Low Level Interface, [89](#)
- rfe_error_param_invalidValue_e
 - RFE Abstract 2.0 - Low Level Interface, [89](#)
- rfe_error_parameterOutOfRange_e
 - RFE Abstract 2.0 - Low Level Interface, [87](#)
- rfe_error_pcdCorrelatorIntegrationTimeInvalid_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_pcdIndexInvalid_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_pcdPhaseCorrectionInvalid_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_pcdProfileIndexInvalid_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_pcdSamplingFrequencyInvalid_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_pcdSlectedInputNotImplemented_e

- RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_ppdVoutAboveMax_e
 - RFE Abstract 2.0 - Low Level Interface, [92](#)
- rfe_error_radarCycleFsm_stateEndTimeExceeded_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_registerComparisonFailed_e
 - RFE Abstract 2.0 - Low Level Interface, [92](#)
- RFE_ERROR_RFE_INTERNAL_ERROR_BIT
 - RFE Abstract 2.0 - Low Level Constants, [53](#)
- rfe_error_rfeFuSaFault_e
 - RFE Abstract 2.0 - Low Level Interface, [87](#)
- rfe_error_rfeInErrorState_e
 - RFE Abstract 2.0 - Low Level Interface, [87](#)
- rfe_error_rx1SaturationStatusReadingFailure_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_rx2SaturationStatusReadingFailure_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_rx3SaturationStatusReadingFailure_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_rx4SaturationStatusReadingFailure_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_rxAdcInputUnidentified_e
 - RFE Abstract 2.0 - Low Level Interface, [91](#)
- rfe_error_rxAdcModuleIdUnidentified_e
 - RFE Abstract 2.0 - Low Level Interface, [91](#)
- rfe_error_rxbist_frequency_fail
 - RFE Abstract 2.0 - Low Level Interface, [100](#)
- rfe_error_rxbist_lna_gain_fail_e
 - RFE Abstract 2.0 - Low Level Interface, [100](#)
- rfe_error_rxbist_lna_phase_fail_e
 - RFE Abstract 2.0 - Low Level Interface, [100](#)
- rfe_error_rxbist_missing_signal
 - RFE Abstract 2.0 - Low Level Interface, [100](#)
- rfe_error_rxbist_mixer_gain_fail_e
 - RFE Abstract 2.0 - Low Level Interface, [100](#)
- rfe_error_rxbist_mixer_phase_fail_e
 - RFE Abstract 2.0 - Low Level Interface, [100](#)
- rfe_error_rxbistInvalidSsbCode_e
 - RFE Abstract 2.0 - Low Level Interface, [93](#)
- rfe_error_rxbistParameterOutOfRange_e
 - RFE Abstract 2.0 - Low Level Interface, [92](#)
- rfe_error_rxCalibrationIfStageInvalid_e
 - RFE Abstract 2.0 - Low Level Interface, [92](#)
- rfe_error_rxCalibrationTargetInvalid_e
 - RFE Abstract 2.0 - Low Level Interface, [92](#)
- rfe_error_rxIfCalibrationFailed
 - RFE Abstract 2.0 - Low Level Interface, [93](#)
- rfe_error_rxIfCalibrationInvalidChannel
 - RFE Abstract 2.0 - Low Level Interface, [93](#)
- rfe_error_rxIfCalibrationInvalidRcCode_e
 - RFE Abstract 2.0 - Low Level Interface, [93](#)
- rfe_error_rxIfCalibrationInvalidSettings
 - RFE Abstract 2.0 - Low Level Interface, [93](#)
- rfe_error_rxInvalidRcCode_e
 - RFE Abstract 2.0 - Low Level Interface, [93](#)
- rfe_error_rxTargetSettingsCombinationIsNotSupported_e
 - RFE Abstract 2.0 - Low Level Interface, [92](#)
- rfe_error_saturationInOperation_e
 - RFE Abstract 2.0 - Low Level Interface, [87](#)
- rfe_error_sharedData_invalidIndex_e
 - RFE Abstract 2.0 - Low Level Interface, [87](#)
- rfe_error_sharedData_redundancyCheckFailed_e
 - RFE Abstract 2.0 - Low Level Interface, [87](#)
- rfe_error_sharedMemoryAddresOverlapping_e
 - RFE Abstract 2.0 - Low Level Interface, [87](#)
- rfe_error_spiTransactionError_e
 - RFE Abstract 2.0 - Low Level Interface, [92](#)
- rfe_error_spiTransactionTimeout_e
 - RFE Abstract 2.0 - Low Level Interface, [92](#)
- rfe_error_startup_seq_failed_e
 - RFE Abstract 2.0 - Low Level Interface, [92](#)
- rfe_error_sys_FE_chip_es2_not_supported
 - RFE Abstract 2.0 - Low Level Interface, [100](#)
- rfe_error_sys_spi_invalid_len
 - RFE Abstract 2.0 - Low Level Interface, [100](#)
- rfe_error_sys_spi_wrg_unit_id
 - RFE Abstract 2.0 - Low Level Interface, [100](#)
- rfe_error_sys_timer_cm7_init
 - RFE Abstract 2.0 - Low Level Interface, [100](#)
- rfe_error_t
 - RFE Abstract 2.0 - Low Level Interface, [85](#)
- rfe_error_testBufferOverflow_e
 - RFE Abstract 2.0 - Low Level Interface, [92](#)
- rfe_error_timerInUse_e
 - RFE Abstract 2.0 - Low Level Interface, [89](#)
- rfe_error_tSenseConversionInProgress_e
 - RFE Abstract 2.0 - Low Level Interface, [91](#)
- rfe_error_tSenseDivisionByZero_e
 - RFE Abstract 2.0 - Low Level Interface, [91](#)
- rfe_error_tSenseInvalidHysteresisThreshold_e
 - RFE Abstract 2.0 - Low Level Interface, [91](#)
- rfe_error_tSenseInvalidModeOfOperation_e
 - RFE Abstract 2.0 - Low Level Interface, [91](#)
- rfe_error_tSenseInvalidOverUndeThresholds_e
 - RFE Abstract 2.0 - Low Level Interface, [91](#)
- rfe_error_tSenseInvalidResolutionTime_e
 - RFE Abstract 2.0 - Low Level Interface, [91](#)
- rfe_error_tSenseInvalidWarningThreshold_e
 - RFE Abstract 2.0 - Low Level Interface, [91](#)
- rfe_error_txGainBuf2AExceedRange_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_txGainBuf2BCExceedRange_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_txIndexInvalid_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_txInvalidAdcConnection_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_txInvalidCalibrationFSMState_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_txInvalidPowerProfile_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_txInvalidSubcomponent_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_txPaGainCExceedRange_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_txPrCalibrationTimeout_e

- RFE Abstract 2.0 - Low Level Interface, [92](#)
- rfe_error_txPRPhaseExceedRange_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfe_error_undefined_e
 - RFE Abstract 2.0 - Low Level Interface, [87](#)
- rfe_error_uninitialized_e
 - RFE Abstract 2.0 - Low Level Interface, [88](#)
- rfe_error_unknown_ip_e
 - RFE Abstract 2.0 - Low Level Interface, [87](#)
- rfe_error_utils_memcpy_range_error_e
 - RFE Abstract 2.0 - Low Level Interface, [88](#)
- rfe_error_wdmaAxiBurstLengthNotSet_e
 - RFE Abstract 2.0 - Low Level Interface, [93](#)
- rfe_error_wdmaAxiBurstModeNotSet_e
 - RFE Abstract 2.0 - Low Level Interface, [93](#)
- rfe_error_wdmaBufferAddressNotAligned_e
 - RFE Abstract 2.0 - Low Level Interface, [92](#)
- rfe_error_wdmaBufferAddressNotSet_e
 - RFE Abstract 2.0 - Low Level Interface, [93](#)
- rfe_error_wdmaBufferLengthNotSet_e
 - RFE Abstract 2.0 - Low Level Interface, [93](#)
- rfe_error_wdmaBufferOutOfBounds_e
 - RFE Abstract 2.0 - Low Level Interface, [92](#)
- rfe_error_wdmaBufferOverwriteModeNotSet_e
 - RFE Abstract 2.0 - Low Level Interface, [93](#)
- rfe_error_wdmaBurstLengthOutOfBounds_e
 - RFE Abstract 2.0 - Low Level Interface, [92](#)
- rfe_error_wdmaIncorrectInterruptSourceSelected_e
 - RFE Abstract 2.0 - Low Level Interface, [93](#)
- rfe_error_wdmaIncorrectNumberOfStreamsSelected_e
 - RFE Abstract 2.0 - Low Level Interface, [93](#)
- rfe_error_wdmaIncorrectStreamSelected_e
 - RFE Abstract 2.0 - Low Level Interface, [92](#)
- rfe_error_wdmaWrongValueForaxiBurstModeField_e
 - RFE Abstract 2.0 - Low Level Interface, [93](#)
- RFE_EVENTS_IRQ_ALL
 - RFE Abstract 2.0 - Low Level Constants, [53](#)
- RFE_EVENTS_IRQ_CHIRP_SEQUENCE_CHANGE
 - RFE Abstract 2.0 - Low Level Interface, [100](#)
- RFE_EVENTS_IRQ_MAX
 - RFE Abstract 2.0 - Low Level Interface, [100](#)
- RFE_EVENTS_IRQ_NONE
 - RFE Abstract 2.0 - Low Level Interface, [100](#)
- RFE_EVENTS_IRQ_RADAR_CYCLE_CHANGE
 - RFE Abstract 2.0 - Low Level Interface, [100](#)
- RFE_EVENTS_IRQ_STATE_CHANGE
 - RFE Abstract 2.0 - Low Level Interface, [100](#)
- rfe_eventsIRQ_t
 - RFE Abstract 2.0 - Low Level Interface, [100](#)
- RFE_FUSA_FAULT_MASK_PARAM_COUNT
 - RFE Abstract 2.0 BLOB - Constants, [180](#)
- RFE_FUSA_R1_FAULT_COUNT_MAX
 - RFE Abstract 2.0 - Low Level Constants, [53](#)
- RFE_FUSA_R1_R2_FAULT_COUNT_MAX
 - RFE Abstract 2.0 - Low Level Constants, [54](#)
- RFE_FUSA_R1_R2_MASK_UNMASK_FAULT_BYTE_COUNT
 - RFE Abstract 2.0 - Low Level Constants, [54](#)
- RFE_FUSA_R2_FAULT_COUNT_MAX
 - RFE Abstract 2.0 - Low Level Constants, [54](#)
- rfe_fuSaFault_R1_hardClippingFlagPersistentAdc1_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R1_hardClippingFlagPersistentAdc2_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R1_hardClippingFlagPersistentAdc3_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R1_hardClippingFlagPersistentAdc4_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R1_levelHighErrorFlagChirp_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R1_levelLowErrorFlagChirp_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R1_loiLevelHighErrorFlagInterface_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R1_loiLevelLowErrorFlagInterface_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R1_loLevelHighErrorFlagRx1_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R1_loLevelHighErrorFlagRx2_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R1_loLevelHighErrorFlagRx3_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R1_loLevelHighErrorFlagRx4_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R1_loLevelLowErrorFlagRx1_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R1_loLevelLowErrorFlagRx2_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R1_loLevelLowErrorFlagRx3_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R1_loLevelLowErrorFlagRx4_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R1_pllFreqCalErrorFlagMc_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R1_rfLevelHighErrorFlagTx1_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R1_rfLevelHighErrorFlagTx2_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R1_rfLevelHighErrorFlagTx3_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R1_rfLevelLowErrorFlagTx1_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R1_rfLevelLowErrorFlagTx2_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R1_rfLevelLowErrorFlagTx3_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fusaFault_R1_size_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fusaFault_R1_t
 - RFE Abstract 2.0 - Low Level Interface, [100](#)
- rfe_fuSaFault_R1_tempErrorFlagChirp_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R1_tempErrorFlagTx1_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R1_tempErrorFlagTx2_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R1_tempErrorFlagTx3_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)

- RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R1_vtuneHighErrorFlagChirp_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R1_vtuneLowErrorFlagChirp_e
 - RFE Abstract 2.0 - Low Level Interface, [101](#)
- rfe_fuSaFault_R2_sr11_xo_no_clock_mclk_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr13_input_mode_toggle_test_sw_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFault_R2_sr14_disable_lb_toggle_test_sw_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFault_R2_sr17_bb_lo_in_loi_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr18_bb_lo_out_loi_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr19_rd_spi_crc_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr19_wr_spi_crc_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr27_bb_tx1_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr27_bb_tx2_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr27_bb_tx3_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr31_bist_lna_gaindiff_sw_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFault_R2_sr31_bist_lna_phasediff_sw_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFault_R2_sr31_bist_mixer_gaindiff_sw_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFault_R2_sr31_bist_mixer_phasediff_sw_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFault_R2_sr34_cal_adc1_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFault_R2_sr34_cal_adc2_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFault_R2_sr34_cal_adc3_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFault_R2_sr34_cal_adc4_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr35_pll_lock_mclk_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr36_mspc_mclk_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr37_pll_level_mclk_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFault_R2_sr39_dig_freq_mclk_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr3_reg_crc_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFault_R2_sr42_vco_freq_480_chirp_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr43_unlock_chirp_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr44_cafc_integrity_level_sw_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFault_R2_sr44_cafc_integrity_sw_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFault_R2_sr44_cafc_integrity_unlock_sw_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFault_R2_sr45_level_max_chirp_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr45_level_min_chirp_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr46_aafc_cal_timeout_chirp_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr46_aafc_chirp_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr47_lo_level_max_rx1_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr47_lo_level_max_rx2_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr47_lo_level_max_rx3_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr47_lo_level_max_rx4_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr47_lo_level_min_rx1_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr47_lo_level_min_rx2_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr47_lo_level_min_rx3_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr47_lo_level_min_rx4_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr48_level_max_loi_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr48_level_min_loi_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr57_supply_high_1v1_gldo_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr59_supply_low_1v1_gldo_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr61_supply_high_1v8_gldo_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr63_supply_low_1v8_gldo_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr65_67_supply_low_1v1_csi2_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr65_67_supply_low_1v1_ser_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr65_67_supply_low_1v8_adc12_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr65_67_supply_low_1v8_adc34_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr65_67_supply_low_1v8_gbias_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr65_67_supply_low_1v8_ser_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr65_67_supply_low_1v8_ssb_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_chirp_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_loi_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_rx1_e

- RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_rx2_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_rx3_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_rx4_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_tx1_e
 - RFE Abstract 2.0 - Low Level Interface, [102](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_tx2_e
 - RFE Abstract 2.0 - Low Level Interface, [102](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v1_tx3_e
 - RFE Abstract 2.0 - Low Level Interface, [102](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_mclk_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_rx1_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_rx2_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_rx3_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_rx4_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_tx1_e
 - RFE Abstract 2.0 - Low Level Interface, [102](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_tx2_e
 - RFE Abstract 2.0 - Low Level Interface, [102](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_1v8_tx3_e
 - RFE Abstract 2.0 - Low Level Interface, [102](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_pll_1v8_chirp_erfe
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr65_sr67_supply_low_vco_1v8_chirp_efe
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr69_rf_level_max_tx1_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr69_rf_level_max_tx2_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr69_rf_level_max_tx3_e
 - RFE Abstract 2.0 - Low Level Interface, [102](#)
- rfe_fuSaFault_R2_sr69_rf_level_min_tx1_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr69_rf_level_min_tx2_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr69_rf_level_min_tx3_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr70_tx_integrity_sw_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFault_R2_sr72_pr_tx1_e
 - RFE Abstract 2.0 - Low Level Interface, [102](#)
- rfe_fuSaFault_R2_sr72_pr_tx2_e
 - RFE Abstract 2.0 - Low Level Interface, [102](#)
- rfe_fuSaFault_R2_sr72_pr_tx3_e
 - RFE Abstract 2.0 - Low Level Interface, [102](#)
- rfe_fuSaFault_R2_sr72_tx1_pr_cal_sw_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFault_R2_sr72_tx2_pr_cal_sw_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFault_R2_sr72_tx3_pr_cal_sw_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFault_R2_sr72_tx_pr_cal_sw_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFault_R2_sr7_interface_stuck_chirp_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr7_lock_step_chirp_e
 - RFE Abstract 2.0 - Low Level Interface, [104](#)
- rfe_fuSaFault_R2_sr9_bb_rx1_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr9_bb_rx2_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr9_bb_rx3_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_R2_sr9_bb_rx4_e
 - RFE Abstract 2.0 - Low Level Interface, [103](#)
- rfe_fuSaFault_t
 - RFE Abstract 2.0 - Low Level Interface, [102](#)
- rfe_fuSaFaultMaskIndex_0_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFaultMaskIndex_10_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFaultMaskIndex_11_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFaultMaskIndex_1_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFaultMaskIndex_2_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFaultMaskIndex_3_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFaultMaskIndex_4_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFaultMaskIndex_5_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFaultMaskIndex_6_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFaultMaskIndex_7_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFaultMaskIndex_8_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFaultMaskIndex_9_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_fuSaFaultMaskIndex_t
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- RFE_GB_REG_NUMBER
 - RFE Abstract 2.0 Constants, [16](#)
- rfe_generic_sw_e
 - RFE Abstract 2.0 - Low Level Interface, [105](#)
- rfe_getBistZeroHourReferenceData
 - RFE Abstract 2.0 - Low Level Interface, [124](#)
- rfe_getFrontEnd
 - RFE Abstract 2.0 - Low Level Interface, [125](#)
- rfe_getFuSaFaults
 - RFE Abstract 2.0 - Low Level Interface, [125](#)
- rfe_getFuSaFaultStatistics
 - RFE Abstract 2.0 - Low Level Interface, [126](#)
- rfe_getNextRadarCycleStartTime
 - RFE Abstract 2.0 - Low Level Interface, [127](#)
- rfe_getRadarCycleCount
 - RFE Abstract 2.0 - Low Level Interface, [105](#)

- RFE Abstract 2.0 - Low Level Interface, [128](#)
- rfe_getRegisterDump
 - RFE Abstract 2.0 - Low Level Interface, [129](#)
- rfe_getState
 - RFE Abstract 2.0 - Low Level Interface, [129](#)
- rfe_getTime
 - RFE Abstract 2.0 - Low Level Interface, [130](#)
- rfe_getVersion
 - RFE Abstract 2.0 - Low Level Interface, [131](#)
- RFE_GLDO_REG_NUMBER
 - RFE Abstract 2.0 Constants, [16](#)
- RFE_ISM_REG_NUMBER
 - RFE Abstract 2.0 Constants, [17](#)
- RFE_KELVIN_TO_CELSIUS
 - RFE Abstract 2.0 - Low Level Constants, [54](#)
- RFE_LDVS_REG_NUMBER
 - RFE Abstract 2.0 Constants, [17](#)
- RFE_LOI_REG_NUMBER
 - RFE Abstract 2.0 Constants, [17](#)
- RFE_MAX_SUPPORTED_DEVICES
 - RFE Abstract 2.0 - Low Level Constants, [54](#)
- RFE_MCLK_REG_NUMBER
 - RFE Abstract 2.0 Constants, [17](#)
- rfe_moduleDumpSelect_t
 - RFE Abstract 2.0 Interface, [27](#)
- RFE_MONITOR_SELECT_ALL
 - RFE Abstract 2.0 - Low Level Constants, [54](#)
- RFE_MONITOR_SELECT_PDC_CLIPPING_COUNT
 - RFE Abstract 2.0 - Low Level Constants, [54](#)
- RFE_MONITOR_SELECT_RX_SAT_COUNT_STAGE_11
 - RFE Abstract 2.0 - Low Level Constants, [55](#)
- RFE_MONITOR_SELECT_TEMP_AFTER_CHIRP_SEQ
 - RFE Abstract 2.0 - Low Level Constants, [55](#)
- RFE_MONITOR_SELECT_TEMP_BEFORE_CHIRP_SEQ
 - RFE Abstract 2.0 - Low Level Constants, [55](#)
- RFE_MONITOR_SELECT_TEMP_IMMEDIATELY
 - RFE Abstract 2.0 - Low Level Constants, [55](#)
- RFE_MONITOR_SELECT_TX_POWER
 - RFE Abstract 2.0 - Low Level Constants, [55](#)
- rfe_monitorRead
 - RFE Abstract 2.0 - Low Level Interface, [132](#)
- rfe_monitorSelect_t
 - RFE Abstract 2.0 - Low Level Interface, [81](#)
- rfe_monitorValues_t, [263](#)
 - pdcClippingCount, [263](#)
 - rxSaturationClippingCount, [264](#)
 - temperature_afterChirpSequence, [264](#)
 - temperature_beforeChirpSequence, [264](#)
 - temperature_immediately, [264](#)
 - txPower, [264](#)
- RFE_NS_PER_TIME_TICK
 - RFE Abstract 2.0 - Low Level Constants, [55](#)
- RFE_OTP_REG_NUMBER
 - RFE Abstract 2.0 Constants, [17](#)
- RFE_OTP_TABLE_NUMBER
 - RFE Abstract 2.0 Constants, [17](#)
- rfe_paramUpdate_chirpProfile_acquisitionTimeTicks_e
 - RFE Abstract 2.0 - Low Level Interface, [109](#)
- rfe_paramUpdate_chirpProfile_centerFrequency_e
 - RFE Abstract 2.0 - Low Level Interface, [109](#)
- rfe_paramUpdate_chirpProfile_chirpFrequencyDriftSteps_e
 - RFE Abstract 2.0 - Low Level Interface, [109](#)
- rfe_paramUpdate_chirpProfile_chirpIntervalTimeTicks_e
 - RFE Abstract 2.0 - Low Level Interface, [109](#)
- rfe_paramUpdate_chirpProfile_chirpPllLoopFilterBandwidth_e
 - RFE Abstract 2.0 - Low Level Interface, [109](#)
- rfe_paramUpdate_chirpProfile_chirpPllVcoSelect_e
 - RFE Abstract 2.0 - Low Level Interface, [109](#)
- rfe_paramUpdate_chirpProfile_chirpSlopeDirection_e
 - RFE Abstract 2.0 - Low Level Interface, [109](#)
- rfe_paramUpdate_chirpProfile_dwellTimeTicks_e
 - RFE Abstract 2.0 - Low Level Interface, [109](#)
- rfe_paramUpdate_chirpProfile_effectiveChirpBandwidth_e
 - RFE Abstract 2.0 - Low Level Interface, [109](#)
- rfe_paramUpdate_chirpProfile_effectiveSamplingFrequency_e
 - RFE Abstract 2.0 - Low Level Interface, [109](#)
- rfe_paramUpdate_chirpProfile_resetTimeTicks_e
 - RFE Abstract 2.0 - Low Level Interface, [109](#)
- rfe_paramUpdate_chirpProfile_rxGain_e
 - RFE Abstract 2.0 - Low Level Interface, [109](#)
- rfe_paramUpdate_chirpProfile_rxHpfCutOffFrequency_e
 - RFE Abstract 2.0 - Low Level Interface, [109](#)
- rfe_paramUpdate_chirpProfile_rxLpfCutOffFrequency_e
 - RFE Abstract 2.0 - Low Level Interface, [109](#)
- rfe_paramUpdate_chirpProfile_settleTimeTicks_e
 - RFE Abstract 2.0 - Low Level Interface, [109](#)
- rfe_paramUpdate_chirpProfile_txPhaseRotation_1_e
 - RFE Abstract 2.0 - Low Level Interface, [109](#)
- rfe_paramUpdate_chirpProfile_txPhaseRotation_2_e
 - RFE Abstract 2.0 - Low Level Interface, [109](#)
- rfe_paramUpdate_chirpProfile_txPhaseRotation_3_e
 - RFE Abstract 2.0 - Low Level Interface, [109](#)
- rfe_paramUpdate_chirpProfile_txPower_e
 - RFE Abstract 2.0 - Low Level Interface, [109](#)
- rfe_paramUpdate_chirpProfile_txTransmissionEnable_e
 - RFE Abstract 2.0 - Low Level Interface, [109](#)
- rfe_paramUpdate_chirpProfile_txTransmissionReferenceTime_e
 - RFE Abstract 2.0 - Low Level Interface, [109](#)
- rfe_paramUpdate_chirpProfile_txTransmissionTimeOffset_e
 - RFE Abstract 2.0 - Low Level Interface, [109](#)
- rfe_paramUpdate_chirpProfile_virtualChannel_e
 - RFE Abstract 2.0 - Low Level Interface, [109](#)
- rfe_paramUpdate_chirpSequence_chirpCount_e
 - RFE Abstract 2.0 - Low Level Interface, [108](#)
- rfe_paramUpdate_chirpSequence_chirpProfileSequence_0_e
 - RFE Abstract 2.0 - Low Level Interface, [108](#)
- rfe_paramUpdate_chirpSequence_chirpProfileSequence_1_e
 - RFE Abstract 2.0 - Low Level Interface, [108](#)
- rfe_paramUpdate_chirpSequence_chirpProfileSequence_2_e
 - RFE Abstract 2.0 - Low Level Interface, [108](#)
- rfe_paramUpdate_chirpSequence_chirpProfileSequence_3_e
 - RFE Abstract 2.0 - Low Level Interface, [108](#)
- rfe_paramUpdate_chirpSequence_chirpProfileSequence_4_e
 - RFE Abstract 2.0 - Low Level Interface, [108](#)
- rfe_paramUpdate_chirpSequence_chirpProfileSequence_5_e
 - RFE Abstract 2.0 - Low Level Interface, [108](#)

- rfe_paramUpdate_radarCycle_recalibrateProfileDependent_1_e RFE Abstract 2.0 Interface, [27](#)
RFE Abstract 2.0 - Low Level Interface, [108](#)
- rfe_paramUpdate_radarCycle_recalibrateProfileDependent_2_e RFE Abstract 2.0 Interface, [27](#)
RFE Abstract 2.0 - Low Level Interface, [108](#)
- rfe_paramUpdate_radarCycle_recalibrateProfileDependent_3_e RFE Abstract 2.0 Interface, [27](#)
RFE Abstract 2.0 - Low Level Interface, [108](#)
- rfe_paramUpdate_radarCycle_recalibrateProfileDependent_4_e RFE Abstract 2.0 Interface, [27](#)
RFE Abstract 2.0 - Low Level Interface, [108](#)
- rfe_paramUpdate_radarCycle_recalibrateProfileDependent_5_e RFE Abstract 2.0 Interface, [27](#)
RFE Abstract 2.0 - Low Level Interface, [108](#)
- rfe_paramUpdate_radarCycle_recalibrateProfileDependent_6_e RFE Abstract 2.0 Interface, [27](#)
RFE Abstract 2.0 - Low Level Interface, [108](#)
- rfe_paramUpdate_radarCycle_recalibrateProfileDependent_7_e RFE Abstract 2.0 Interface, [27](#)
RFE Abstract 2.0 - Low Level Interface, [108](#)
- rfe_paramUpdate_radarCycle_recalibrateProfileIndependent_e RFE Abstract 2.0 Interface, [27](#)
RFE Abstract 2.0 - Low Level Interface, [108](#)
- rfe_paramUpdate_t
RFE Abstract 2.0 - Low Level Interface, [106](#)
- rfe_phaseRotation_t
RFE Abstract 2.0 BLOB - Constants, [182](#)
- rfe_powerMode_active_e
RFE Abstract 2.0 - Low Level Interface, [109](#)
- rfe_powerMode_inactive_e
RFE Abstract 2.0 - Low Level Interface, [109](#)
- rfe_powerMode_t
RFE Abstract 2.0 - Low Level Interface, [109](#)
- rfe_PR_ADD_QPSK_TO_MSB
RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_PR_INVALID
RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_PR_NO_CHANGE_e
RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_PR_REPLACE_MSB_WITH_QPSK
RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_PRPCGenMode_e
RFE Abstract 2.0 - Low Level Interface, [109](#)
- rfe_radarCycleCount_t, [264](#)
chirpSequenceCount, [265](#)
radarCycleCount, [265](#)
- rfe_radarCycleStart
RFE Abstract 2.0 - Low Level Interface, [132](#)
- rfe_radarCycleStop
RFE Abstract 2.0 - Low Level Interface, [133](#)
- rfe_registerDump_ADC12
RFE Abstract 2.0 Interface, [27](#)
- rfe_registerDump_ADC34
RFE Abstract 2.0 Interface, [27](#)
- rfe_registerDump_ATB_VOLTAGE_READOUT
RFE Abstract 2.0 Interface, [27](#)
- rfe_registerDump_CAFC
RFE Abstract 2.0 Interface, [27](#)
- rfe_registerDump_CC
RFE Abstract 2.0 Interface, [27](#)
- rfe_registerDump_CSI2
RFE Abstract 2.0 Interface, [27](#)
- rfe_registerDump_GB
RFE Abstract 2.0 Interface, [27](#)
- rfe_registerDump_GLDO
RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_registerDump_ISM
RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_registerDump_LDVS
RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_registerDump_LOI
RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_registerDump_MCLK
RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_registerDump_OTP
RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_registerDump_OTP_TABLE
RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_registerDump_RX1
RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_registerDump_RX2
RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_registerDump_RX3
RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_registerDump_RX4
RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_registerDump_SC
RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_registerDump_SSBMOD
RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_registerDump_TE
RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_registerDump_TX1
RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_registerDump_TX2
RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_registerDump_TX3
RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_registerDumpValues_t, [265](#)
frontEndId, [266](#)
moduleId, [266](#)
registerAddress, [266](#)
value, [266](#)
- rfe_rx1234Reset_e
RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_rx1Reset_e
RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_rx2Reset_e
RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_rx3Reset_e
RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_rx4Reset_e
RFE Abstract 2.0 - Low Level Interface, [110](#)
- RFE_RX_BIST_COMBINATIONS
RFE Abstract 2.0 - Low Level Constants, [55](#)
- RFE_RX_CLIPPING_RESET
RFE Abstract 2.0 - Low Level Constants, [56](#)
- RFE_RX_COUNT
RFE Abstract 2.0 - Low Level Constants, [56](#)
- RFE_RX_REG_NUMBER
RFE Abstract 2.0 Constants, [17](#)
- RFE_RX_SELECT_1
RFE Abstract 2.0 - Low Level Constants, [56](#)

- RFE_RX_SELECT_2
 - RFE Abstract 2.0 - Low Level Constants, [56](#)
- RFE_RX_SELECT_3
 - RFE Abstract 2.0 - Low Level Constants, [56](#)
- RFE_RX_SELECT_4
 - RFE Abstract 2.0 - Low Level Constants, [56](#)
- RFE_RX_SELECT_ALL
 - RFE Abstract 2.0 - Low Level Constants, [56](#)
- RFE_RX_SELECT_NONE
 - RFE Abstract 2.0 - Low Level Constants, [56](#)
- RFE_RX_STAGE
 - RFE Abstract 2.0 - Low Level Constants, [57](#)
- rfe_rxClippingReset_t
 - RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_rxGain_25dB_e
 - RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_rxGain_28dB_e
 - RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_rxGain_31dB_e
 - RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_rxGain_34dB_e
 - RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_rxGain_37dB_e
 - RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_rxGain_40dB_e
 - RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_rxGain_43dB_e
 - RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_rxGain_46dB_e
 - RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_rxGain_invalid_e
 - RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_rxGain_t
 - RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_rxHpfCutOffFrequency_1600kHz_e
 - RFE Abstract 2.0 - Low Level Interface, [111](#)
- rfe_rxHpfCutOffFrequency_200kHz_e
 - RFE Abstract 2.0 - Low Level Interface, [111](#)
- rfe_rxHpfCutOffFrequency_300kHz_e
 - RFE Abstract 2.0 - Low Level Interface, [111](#)
- rfe_rxHpfCutOffFrequency_3200kHz_e
 - RFE Abstract 2.0 - Low Level Interface, [111](#)
- rfe_rxHpfCutOffFrequency_400kHz_e
 - RFE Abstract 2.0 - Low Level Interface, [111](#)
- rfe_rxHpfCutOffFrequency_6400kHz_e
 - RFE Abstract 2.0 - Low Level Interface, [111](#)
- rfe_rxHpfCutOffFrequency_800kHz_e
 - RFE Abstract 2.0 - Low Level Interface, [111](#)
- rfe_rxHpfCutOffFrequency_invalid_e
 - RFE Abstract 2.0 - Low Level Interface, [111](#)
- rfe_rxHpfCutOffFrequency_t
 - RFE Abstract 2.0 - Low Level Interface, [110](#)
- rfe_rxIndex_1_e
 - RFE Abstract 2.0 - Low Level Interface, [111](#)
- rfe_rxIndex_2_e
 - RFE Abstract 2.0 - Low Level Interface, [111](#)
- rfe_rxIndex_3_e
 - RFE Abstract 2.0 - Low Level Interface, [111](#)
- rfe_rxIndex_4_e
 - RFE Abstract 2.0 - Low Level Interface, [111](#)
- rfe_rxIndex_t
 - RFE Abstract 2.0 - Low Level Interface, [111](#)
- rfe_rxLpfCutOffFrequency_12P5MHz_e
 - RFE Abstract 2.0 - Low Level Interface, [112](#)
- rfe_rxLpfCutOffFrequency_15MHz_e
 - RFE Abstract 2.0 - Low Level Interface, [112](#)
- rfe_rxLpfCutOffFrequency_20MHz_e
 - RFE Abstract 2.0 - Low Level Interface, [112](#)
- rfe_rxLpfCutOffFrequency_25MHz_e
 - RFE Abstract 2.0 - Low Level Interface, [112](#)
- rfe_rxLpfCutOffFrequency_40MHz_e
 - RFE Abstract 2.0 - Low Level Interface, [112](#)
- rfe_rxLpfCutOffFrequency_invalid_e
 - RFE Abstract 2.0 - Low Level Interface, [112](#)
- rfe_rxLpfCutOffFrequency_t
 - RFE Abstract 2.0 - Low Level Interface, [111](#)
- rfe_rxSaturationThreshold_0dB_e
 - RFE Abstract 2.0 - Low Level Interface, [112](#)
- rfe_rxSaturationThreshold_min0_87dB_e
 - RFE Abstract 2.0 - Low Level Interface, [112](#)
- rfe_rxSaturationThreshold_min10_44dB_e
 - RFE Abstract 2.0 - Low Level Interface, [112](#)
- rfe_rxSaturationThreshold_min11_31dB_e
 - RFE Abstract 2.0 - Low Level Interface, [112](#)
- rfe_rxSaturationThreshold_min12_18dB_e
 - RFE Abstract 2.0 - Low Level Interface, [112](#)
- rfe_rxSaturationThreshold_min13_05dB_e
 - RFE Abstract 2.0 - Low Level Interface, [112](#)
- rfe_rxSaturationThreshold_min1_74dB_e
 - RFE Abstract 2.0 - Low Level Interface, [112](#)
- rfe_rxSaturationThreshold_min2_61dB_e
 - RFE Abstract 2.0 - Low Level Interface, [112](#)
- rfe_rxSaturationThreshold_min3_48dB_e
 - RFE Abstract 2.0 - Low Level Interface, [112](#)
- rfe_rxSaturationThreshold_min4_35dB_e
 - RFE Abstract 2.0 - Low Level Interface, [112](#)
- rfe_rxSaturationThreshold_min5_22dB_e
 - RFE Abstract 2.0 - Low Level Interface, [112](#)
- rfe_rxSaturationThreshold_min6_09dB_e
 - RFE Abstract 2.0 - Low Level Interface, [112](#)
- rfe_rxSaturationThreshold_min6_96dB_e
 - RFE Abstract 2.0 - Low Level Interface, [112](#)
- rfe_rxSaturationThreshold_min7_83dB_e
 - RFE Abstract 2.0 - Low Level Interface, [112](#)
- rfe_rxSaturationThreshold_min8_70dB_e
 - RFE Abstract 2.0 - Low Level Interface, [112](#)
- rfe_rxSaturationThreshold_min9_57dB_e
 - RFE Abstract 2.0 - Low Level Interface, [112](#)
- rfe_rxSaturationThreshold_t
 - RFE Abstract 2.0 - Low Level Interface, [112](#)
- rfe_rxSelect_t
 - RFE Abstract 2.0 - Low Level Interface, [81](#)
- rfe_rxStage_1_e
 - RFE Abstract 2.0 - Low Level Interface, [113](#)
- rfe_rxStage_2_e
 - RFE Abstract 2.0 - Low Level Interface, [113](#)

- rfe_rxStage_t
 - RFE Abstract 2.0 - Low Level Interface, [112](#)
- RFE_SC_REG_NUMBER
 - RFE Abstract 2.0 Constants, [18](#)
- rfe_setFrontEnd
 - RFE Abstract 2.0 - Low Level Interface, [134](#)
- rfe_setNextRadarCycleStartTime
 - RFE Abstract 2.0 - Low Level Interface, [135](#)
- RFE_SSBMOD_REG_NUMBER
 - RFE Abstract 2.0 Constants, [18](#)
- rfe_state_busy_e
 - RFE Abstract 2.0 - Low Level Interface, [113](#)
- rfe_state_configured_e
 - RFE Abstract 2.0 - Low Level Interface, [113](#)
- rfe_state_fuSaFault_e
 - RFE Abstract 2.0 - Low Level Interface, [113](#)
- rfe_state_fuSaFaultRecovery_e
 - RFE Abstract 2.0 - Low Level Interface, [113](#)
- rfe_state_initialized_e
 - RFE Abstract 2.0 - Low Level Interface, [113](#)
- rfe_state_radarCycleIdle_e
 - RFE Abstract 2.0 - Low Level Interface, [113](#)
- rfe_state_t
 - RFE Abstract 2.0 - Low Level Interface, [113](#)
- rfe_state_testContinuousWaveTransmission_e
 - RFE Abstract 2.0 - Low Level Interface, [113](#)
- RFE_SW_CFG_MNGR_CHIRP_PROFILE_PARAMS
 - RFE Abstract 2.0 BLOB - Constants, [180](#)
- RFE_SW_CFG_MNGR_CHIRP_SEQUENCE_PARAMS
 - RFE Abstract 2.0 BLOB - Constants, [181](#)
- RFE_SW_CFG_MNGR_GENRAL_PARAMS
 - RFE Abstract 2.0 BLOB - Constants, [181](#)
- RFE_SW_CFG_MNGR_MONITOR_AND_SAFETY_PARAMS
 - RFE Abstract 2.0 BLOB - Constants, [181](#)
- RFE_SW_CFG_MNGR_NO_OF_SECTION
 - RFE Abstract 2.0 BLOB - Constants, [181](#)
- RFE_SW_CFG_MNGR_NON_UPDATABLE_PARAMS
 - RFE Abstract 2.0 BLOB - Constants, [181](#)
- RFE_SW_CFG_MNGR_RADAR_CYCLE_PARAMS
 - RFE Abstract 2.0 BLOB - Constants, [181](#)
- RFE_SW_CFG_MNGR_UPDATABLE_PARAMS
 - RFE Abstract 2.0 BLOB - Constants, [181](#)
- RFE_SW_MAIN_FSM_MAIN_STATE_COUNT
 - RFE Abstract 2.0 - Firmware Constants, [248](#)
- rfe_sync
 - RFE Abstract 2.0 - Low Level Interface, [136](#)
- rfe_sysMemAddress_t
 - RFE Abstract 2.0 - Low Level Interface, [81](#)
- RFE_TE_REG_NUMBER
 - RFE Abstract 2.0 Constants, [18](#)
- rfe_tef82xxVoltages_t, [266](#)
 - ADC12Voltages, [267](#)
 - ADC34Voltages, [267](#)
 - ATBVoltages, [267](#)
 - CAFCVoltages, [267](#)
 - GBIASVoltages, [267](#)
 - GLDOVoltages, [267](#)
 - LOIVoltages, [267](#)
 - MCLKVoltages, [268](#)
 - MIPIVoltages, [268](#)
 - RX1Voltages, [268](#)
 - RX2Voltages, [268](#)
 - RX3Voltages, [268](#)
 - RX4Voltages, [268](#)
 - SSBVoltages, [268](#)
 - TX1Voltages, [268](#)
 - TX2Voltages, [269](#)
 - TX3Voltages, [269](#)
- RFE_TEMPERATURE_1_DEG_CELSIUS
 - RFE Abstract 2.0 - Low Level Constants, [57](#)
- RFE_TEMPERATURE_SENSOR_COUNT
 - RFE Abstract 2.0 - Low Level Constants, [57](#)
- rfe_temperature_t
 - RFE Abstract 2.0 - Low Level Interface, [82](#)
- rfe_temperatureSensorIndex_chirp_e
 - RFE Abstract 2.0 - Low Level Interface, [114](#)
- rfe_temperatureSensorIndex_t
 - RFE Abstract 2.0 - Low Level Interface, [113](#)
- rfe_temperatureSensorIndex_tx1_e
 - RFE Abstract 2.0 - Low Level Interface, [114](#)
- rfe_temperatureSensorIndex_tx2_e
 - RFE Abstract 2.0 - Low Level Interface, [114](#)
- rfe_temperatureSensorIndex_tx3_e
 - RFE Abstract 2.0 - Low Level Interface, [114](#)
- rfe_testContinuousWaveTransmissionStart
 - RFE Abstract 2.0 - Low Level Interface, [136](#)
- rfe_testContinuousWaveTransmissionStop
 - RFE Abstract 2.0 - Low Level Interface, [137](#)
- rfe_testGetInternalError
 - RFE Abstract 2.0 - Low Level Interface, [138](#)
- rfe_testParam_chirpPILTestPinEnable_e
 - RFE Abstract 2.0 - Low Level Interface, [114](#)
- rfe_testParam_enableClockRetuning_e
 - RFE Abstract 2.0 - Low Level Interface, [114](#)
- rfe_testParam_keepTxTransmissionEnabled_e
 - RFE Abstract 2.0 - Low Level Interface, [114](#)
- rfe_testParam_outputDataTestPattern_e
 - RFE Abstract 2.0 - Low Level Interface, [114](#)
- rfe_testParam_t
 - RFE Abstract 2.0 - Low Level Interface, [114](#)
- rfe_testSetParam
 - RFE Abstract 2.0 - Low Level Interface, [138](#)
- RFE_TIME_TICKS_PER_US
 - RFE Abstract 2.0 - Low Level Constants, [57](#)
- RFE_TX_BIST_COMBINATIONS
 - RFE Abstract 2.0 - Low Level Constants, [57](#)
- RFE_TX_COUNT
 - RFE Abstract 2.0 BLOB - Constants, [182](#)
- RFE_TX_POWER_ERR
 - RFE Abstract 2.0 - Low Level Constants, [57](#)
- RFE_TX_REG_NUMBER
 - RFE Abstract 2.0 Constants, [18](#)
- RFE_TX_SELECT_1
 - RFE Abstract 2.0 - Low Level Constants, [57](#)
- RFE_TX_SELECT_2
 - RFE Abstract 2.0 - Low Level Constants, [58](#)

- RFE_TX_SELECT_3
 - RFE Abstract 2.0 - Low Level Constants, [58](#)
- RFE_TX_SELECT_4
 - RFE Abstract 2.0 - Low Level Constants, [58](#)
- RFE_TX_SELECT_ALL
 - RFE Abstract 2.0 - Low Level Constants, [58](#)
- RFE_TX_SELECT_NONE
 - RFE Abstract 2.0 - Low Level Constants, [58](#)
- rfe_txIndex_1_e
 - RFE Abstract 2.0 - Low Level Interface, [114](#)
- rfe_txIndex_2_e
 - RFE Abstract 2.0 - Low Level Interface, [114](#)
- rfe_txIndex_3_e
 - RFE Abstract 2.0 - Low Level Interface, [114](#)
- rfe_txIndex_4_e
 - RFE Abstract 2.0 - Low Level Interface, [114](#)
- rfe_txIndex_invalid_e
 - RFE Abstract 2.0 - Low Level Interface, [114](#)
- rfe_txIndex_t
 - RFE Abstract 2.0 - Low Level Interface, [114](#)
- rfe_txPowerErrorFlag_t
 - RFE Abstract 2.0 - Low Level Interface, [115](#)
- rfe_txPowerHighErrorFlag
 - RFE Abstract 2.0 - Low Level Interface, [115](#)
- rfe_txPowerLowErrorFlag
 - RFE Abstract 2.0 - Low Level Interface, [115](#)
- rfe_txPowerMaxErrorFlag
 - RFE Abstract 2.0 - Low Level Interface, [115](#)
- rfe_txPowerMinErrorFlag
 - RFE Abstract 2.0 - Low Level Interface, [115](#)
- rfe_txSelect_t
 - RFE Abstract 2.0 - Low Level Interface, [82](#)
- rfe_txTransmissionReferenceTime_startOfDwellTime_e
 - RFE Abstract 2.0 BLOB - Constants, [184](#)
- rfe_txTransmissionReferenceTime_startOfSettleTime_e
 - RFE Abstract 2.0 BLOB - Constants, [184](#)
- rfe_txTransmissionReferenceTime_t
 - RFE Abstract 2.0 BLOB - Constants, [184](#)
- RFE_UPDATE_COUNT_MAX
 - RFE Abstract 2.0 - Low Level Constants, [58](#)
- rfe_updateBegin
 - RFE Abstract 2.0 - Low Level Interface, [139](#)
- rfe_updateDynamicTable
 - RFE Abstract 2.0 - Low Level Interface, [140](#)
- rfe_updateParam
 - RFE Abstract 2.0 - Low Level Interface, [140](#)
- rfe_updatePush
 - RFE Abstract 2.0 - Low Level Interface, [141](#)
- rfe_version_t, [269](#)
 - fwHash, [269](#)
 - fwVariant, [270](#)
 - fwVersionMajor, [270](#)
 - fwVersionMinor, [270](#)
 - fwVersionPatch, [270](#)
 - fwVersionReleased, [270](#)
 - hwType, [270](#)
 - hwUniqueId, [270](#)
 - hwVariant, [271](#)
 - hwVersion, [271](#)
- rfe_virtualChannel_0_e
 - RFE Abstract 2.0 - Low Level Interface, [115](#)
- rfe_virtualChannel_1_e
 - RFE Abstract 2.0 - Low Level Interface, [115](#)
- rfe_virtualChannel_2_e
 - RFE Abstract 2.0 - Low Level Interface, [115](#)
- rfe_virtualChannel_3_e
 - RFE Abstract 2.0 - Low Level Interface, [115](#)
- rfe_virtualChannel_invalid_e
 - RFE Abstract 2.0 - Low Level Interface, [115](#)
- rfe_virtualChannel_t
 - RFE Abstract 2.0 - Low Level Interface, [115](#)
- rfeCfg_calcAndWriteCrc
 - RFE Abstract 2.0 BLOB - Low Level Interface, [190](#)
- rfeCfg_CheckBlobCompatibility
 - RFE Abstract 2.0 BLOB - Low Level Interface, [190](#)
- rfeCfg_chirpProfile_getAcquisitionTimeTicks
 - RFE Abstract 2.0 BLOB - Low Level Interface, [191](#)
- rfeCfg_chirpProfile_getCenterFrequency
 - RFE Abstract 2.0 BLOB - Low Level Interface, [191](#)
- rfeCfg_chirpProfile_getChirpIntervalTimeTicks
 - RFE Abstract 2.0 BLOB - Low Level Interface, [192](#)
- rfeCfg_chirpProfile_getChirpPllLoopFilterBandwidth
 - RFE Abstract 2.0 BLOB - Low Level Interface, [192](#)
- rfeCfg_chirpProfile_getChirpPllVco
 - RFE Abstract 2.0 BLOB - Low Level Interface, [192](#)
- rfeCfg_chirpProfile_getChirpSlopeDirection
 - RFE Abstract 2.0 BLOB - Low Level Interface, [193](#)
- rfeCfg_chirpProfile_getDwellTimeTicks
 - RFE Abstract 2.0 BLOB - Low Level Interface, [193](#)
- rfeCfg_chirpProfile_getEffectiveChirpBandwidth
 - RFE Abstract 2.0 BLOB - Low Level Interface, [194](#)
- rfeCfg_chirpProfile_getEffectiveSamplingFrequency
 - RFE Abstract 2.0 BLOB - Low Level Interface, [194](#)
- rfeCfg_chirpProfile_getResetTimeTicks
 - RFE Abstract 2.0 BLOB - Low Level Interface, [195](#)
- rfeCfg_chirpProfile_getRxGain
 - RFE Abstract 2.0 BLOB - Low Level Interface, [195](#)
- rfeCfg_chirpProfile_getRxHpfCutOffFrequency
 - RFE Abstract 2.0 BLOB - Low Level Interface, [195](#)
- rfeCfg_chirpProfile_getRxLpfCutOffFrequency
 - RFE Abstract 2.0 BLOB - Low Level Interface, [196](#)
- rfeCfg_chirpProfile_getSettleTimeTicks
 - RFE Abstract 2.0 BLOB - Low Level Interface, [196](#)
- rfeCfg_chirpProfile_getTxPhaseRotation
 - RFE Abstract 2.0 BLOB - Low Level Interface, [197](#)
- rfeCfg_chirpProfile_getTxPower
 - RFE Abstract 2.0 BLOB - Low Level Interface, [197](#)
- rfeCfg_chirpProfile_getTxTransmissionEnable
 - RFE Abstract 2.0 BLOB - Low Level Interface, [198](#)
- rfeCfg_chirpProfile_getTxTransmissionReferenceTime
 - RFE Abstract 2.0 BLOB - Low Level Interface, [198](#)
- rfeCfg_chirpProfile_getTxTransmissionTimeOffset
 - RFE Abstract 2.0 BLOB - Low Level Interface, [198](#)
- rfeCfg_chirpProfile_getVirtualChannel
 - RFE Abstract 2.0 BLOB - Low Level Interface, [199](#)
- rfeCfg_chirpProfile_setAcquisitionTimeTicks

- RFE Abstract 2.0 BLOB - Low Level Interface, [199](#)
- rfeCfg_chirpProfile_setCenterFrequency
 - RFE Abstract 2.0 BLOB - Low Level Interface, [200](#)
- rfeCfg_chirpProfile_setChirpIntervalTimeTicks
 - RFE Abstract 2.0 BLOB - Low Level Interface, [200](#)
- rfeCfg_chirpProfile_setChirpPllLoopFilterBandwidth
 - RFE Abstract 2.0 BLOB - Low Level Interface, [200](#)
- rfeCfg_chirpProfile_setChirpPllVco
 - RFE Abstract 2.0 BLOB - Low Level Interface, [201](#)
- rfeCfg_chirpProfile_setChirpSlopeDirection
 - RFE Abstract 2.0 BLOB - Low Level Interface, [201](#)
- rfeCfg_chirpProfile_setDwellTimeTicks
 - RFE Abstract 2.0 BLOB - Low Level Interface, [202](#)
- rfeCfg_chirpProfile_setEffectiveChirpBandwidth
 - RFE Abstract 2.0 BLOB - Low Level Interface, [202](#)
- rfeCfg_chirpProfile_setEffectiveSamplingFrequency
 - RFE Abstract 2.0 BLOB - Low Level Interface, [202](#)
- rfeCfg_chirpProfile_setResetTimeTicks
 - RFE Abstract 2.0 BLOB - Low Level Interface, [203](#)
- rfeCfg_chirpProfile_setRxGain
 - RFE Abstract 2.0 BLOB - Low Level Interface, [203](#)
- rfeCfg_chirpProfile_setRxHpfCutOffFrequency
 - RFE Abstract 2.0 BLOB - Low Level Interface, [204](#)
- rfeCfg_chirpProfile_setRxLpfCutOffFrequency
 - RFE Abstract 2.0 BLOB - Low Level Interface, [204](#)
- rfeCfg_chirpProfile_setSettleTimeTicks
 - RFE Abstract 2.0 BLOB - Low Level Interface, [204](#)
- rfeCfg_chirpProfile_setTxPhaseRotation
 - RFE Abstract 2.0 BLOB - Low Level Interface, [205](#)
- rfeCfg_chirpProfile_setTxPower
 - RFE Abstract 2.0 BLOB - Low Level Interface, [205](#)
- rfeCfg_chirpProfile_setTxTransmissionEnable
 - RFE Abstract 2.0 BLOB - Low Level Interface, [206](#)
- rfeCfg_chirpProfile_setTxTransmissionReferenceTime
 - RFE Abstract 2.0 BLOB - Low Level Interface, [206](#)
- rfeCfg_chirpProfile_setTxTransmissionTimeOffset
 - RFE Abstract 2.0 BLOB - Low Level Interface, [206](#)
- rfeCfg_chirpProfile_setVirtualChannel
 - RFE Abstract 2.0 BLOB - Low Level Interface, [207](#)
- rfeCfg_chirpSequence_getChirpCount
 - RFE Abstract 2.0 BLOB - Low Level Interface, [207](#)
- rfeCfg_chirpSequence_getChirpFrequencyDriftSteps
 - RFE Abstract 2.0 BLOB - Low Level Interface, [208](#)
- rfeCfg_chirpSequence_getChirpProfileRepeatCount
 - RFE Abstract 2.0 BLOB - Low Level Interface, [208](#)
- rfeCfg_chirpSequence_getChirpProfileSequence
 - RFE Abstract 2.0 BLOB - Low Level Interface, [208](#)
- rfeCfg_chirpSequence_getChirpProfileSequenceLength
 - RFE Abstract 2.0 BLOB - Low Level Interface, [209](#)
- rfeCfg_chirpSequence_getDynamicTableIndex
 - RFE Abstract 2.0 BLOB - Low Level Interface, [209](#)
- rfeCfg_chirpSequence_getDynamicUpdatesEnabled
 - RFE Abstract 2.0 BLOB - Low Level Interface, [210](#)
- rfeCfg_chirpSequence_getRxEnable
 - RFE Abstract 2.0 BLOB - Low Level Interface, [210](#)
- rfeCfg_chirpSequence_setChirpCount
 - RFE Abstract 2.0 BLOB - Low Level Interface, [211](#)
- rfeCfg_chirpSequence_setChirpProfileRepeatCount
 - RFE Abstract 2.0 BLOB - Low Level Interface, [211](#)
- rfeCfg_chirpSequence_setChirpProfileSequence
 - RFE Abstract 2.0 BLOB - Low Level Interface, [211](#)
- rfeCfg_chirpSequence_setChirpProfileSequenceLength
 - RFE Abstract 2.0 BLOB - Low Level Interface, [212](#)
- rfeCfg_chirpSequence_setDynamicTableIndex
 - RFE Abstract 2.0 BLOB - Low Level Interface, [213](#)
- rfeCfg_chirpSequence_setDynamicUpdatesEnabled
 - RFE Abstract 2.0 BLOB - Low Level Interface, [213](#)
- rfeCfg_chirpSequence_setRxEnable
 - RFE Abstract 2.0 BLOB - Low Level Interface, [213](#)
- rfeCfg_general_getJumpbackTimeTicks
 - RFE Abstract 2.0 BLOB - Low Level Interface, [214](#)
- rfeCfg_general_getPdcBitwidth
 - RFE Abstract 2.0 BLOB - Low Level Interface, [214](#)
- rfeCfg_general_getPowerMode
 - RFE Abstract 2.0 BLOB - Low Level Interface, [215](#)
- rfeCfg_general_getTxCalibrationMode
 - RFE Abstract 2.0 BLOB - Low Level Interface, [215](#)
- rfeCfg_general_setPowerMode
 - RFE Abstract 2.0 BLOB - Low Level Interface, [215](#)
- rfeCfg_metadata_getChirpProfileCount
 - RFE Abstract 2.0 BLOB - Low Level Interface, [216](#)
- rfeCfg_metadata_getChirpSequenceConfigCount
 - RFE Abstract 2.0 BLOB - Low Level Interface, [216](#)
- rfeCfg_monitorAndSafety_getAdcClippingCountResetEveryChirpSequence
 - RFE Abstract 2.0 BLOB - Low Level Interface, [217](#)
- rfeCfg_monitorAndSafety_getBist
 - RFE Abstract 2.0 BLOB - Low Level Interface, [217](#)
- rfeCfg_monitorAndSafety_getFrequencyForBist
 - RFE Abstract 2.0 BLOB - Low Level Interface, [218](#)
- rfeCfg_monitorAndSafety_getFuSaFaultMask
 - RFE Abstract 2.0 BLOB - Low Level Interface, [218](#)
- rfeCfg_monitorAndSafety_getInjectTestTone
 - RFE Abstract 2.0 BLOB - Low Level Interface, [218](#)
- rfeCfg_monitorAndSafety_getLoiBBB
 - RFE Abstract 2.0 BLOB - Low Level Interface, [219](#)
- rfeCfg_monitorAndSafety_getRxBBB
 - RFE Abstract 2.0 BLOB - Low Level Interface, [219](#)
- rfeCfg_monitorAndSafety_getRxSatCountResetEveryChirpSequence
 - RFE Abstract 2.0 BLOB - Low Level Interface, [220](#)
- rfeCfg_monitorAndSafety_getRxSaturationCountLimitStage
 - RFE Abstract 2.0 BLOB - Low Level Interface, [220](#)
- rfeCfg_monitorAndSafety_getRxSaturationThresholdStage
 - RFE Abstract 2.0 BLOB - Low Level Interface, [221](#)
- rfeCfg_monitorAndSafety_getTxBBB
 - RFE Abstract 2.0 BLOB - Low Level Interface, [221](#)
- rfeCfg_monitorAndSafety_setBist
 - RFE Abstract 2.0 BLOB - Low Level Interface, [221](#)
- rfeCfg_monitorAndSafety_setFrequencyForBist
 - RFE Abstract 2.0 BLOB - Low Level Interface, [222](#)
- rfeCfg_monitorAndSafety_setInjectTestTone
 - RFE Abstract 2.0 BLOB - Low Level Interface, [223](#)
- rfeCfg_monitorAndSafety_setLoiBBB
 - RFE Abstract 2.0 BLOB - Low Level Interface, [223](#)
- rfeCfg_param_chirpProfile_acquisitionTimeTicks_e
 - RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_param_chirpProfile_centerFrequency_e

- RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_param_chirpProfile_chirpFrequencyDriftSteps_e RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_param_chirpProfile_chirpIntervalTimeTicks_e RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_param_chirpProfile_chirpPllLoopFilterBandwidth_e RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_param_chirpProfile_chirpPllVcoSelect_e RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_param_chirpProfile_chirpSlopeDirection_e RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_param_chirpProfile_dwellTimeTicks_e RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_param_chirpProfile_effectiveChirpBandwidth_e RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_param_chirpProfile_effectiveSamplingFrequency_e RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_param_chirpProfile_resetTimeTicks_e RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_param_chirpProfile_rxGain_e RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_param_chirpProfile_rxHpfCutOffFrequency_e RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_param_chirpProfile_rxLpfCutOffFrequency_e RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_param_chirpProfile_settleTimeTicks_e RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_param_chirpProfile_txPhaseRotation_1_e RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_param_chirpProfile_txPhaseRotation_2_e RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_param_chirpProfile_txPhaseRotation_3_e RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_param_chirpProfile_txPower_e RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_param_chirpProfile_txTransmissionEnable_e RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_param_chirpProfile_txTransmissionReferenceTime_e RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_param_chirpProfile_txTransmissionTimeOffset_e RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_param_chirpProfile_virtualChannel_e RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_param_chirpSequence_chirpCount_e RFE Abstract 2.0 - Low Level Interface, [118](#)
- rfeCfg_param_chirpSequence_chirpProfileRepeatCount_e RFE Abstract 2.0 - Low Level Interface, [118](#)
- rfeCfg_param_chirpSequence_chirpProfileSequence_0_e RFE Abstract 2.0 - Low Level Interface, [118](#)
- rfeCfg_param_chirpSequence_chirpProfileSequence_1_e RFE Abstract 2.0 - Low Level Interface, [118](#)
- rfeCfg_param_chirpSequence_chirpProfileSequence_2_e RFE Abstract 2.0 - Low Level Interface, [118](#)
- rfeCfg_param_chirpSequence_chirpProfileSequence_3_e RFE Abstract 2.0 - Low Level Interface, [118](#)
- rfeCfg_param_chirpSequence_chirpProfileSequence_4_e RFE Abstract 2.0 - Low Level Interface, [118](#)
- rfeCfg_param_chirpSequence_chirpProfileSequence_5_e RFE Abstract 2.0 - Low Level Interface, [118](#)
- RFE Abstract 2.0 - Low Level Interface, [118](#)
- rfeCfg_param_chirpSequence_chirpProfileSequence_6_e RFE Abstract 2.0 - Low Level Interface, [118](#)
- rfeCfg_param_chirpSequence_chirpProfileSequence_7_e RFE Abstract 2.0 - Low Level Interface, [118](#)
- rfeCfg_param_chirpSequence_chirpProfileSequenceLength_e RFE Abstract 2.0 - Low Level Interface, [118](#)
- rfeCfg_param_chirpSequence_dynamicTableIndex_e RFE Abstract 2.0 - Low Level Interface, [118](#)
- rfeCfg_param_chirpSequence_dynamicUpdatesEnabled_e RFE Abstract 2.0 - Low Level Interface, [118](#)
- rfeCfg_param_chirpSequence_pdcNotchFilterEnable_e RFE Abstract 2.0 - Low Level Interface, [118](#)
- rfeCfg_param_chirpSequence_rxEnable_e RFE Abstract 2.0 - Low Level Interface, [118](#)
- rfeCfg_param_chirpSequence_txCalibration_thresholdHigh_e RFE Abstract 2.0 - Low Level Interface, [118](#)
- rfeCfg_param_chirpSequence_txCalibration_thresholdLow_e RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_param_chirpSequence_txPhaseRotation_ddma_e RFE Abstract 2.0 - Low Level Interface, [118](#)
- rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx1_init_e RFE Abstract 2.0 - Low Level Interface, [118](#)
- rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx1_mode_e RFE Abstract 2.0 - Low Level Interface, [118](#)
- rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx1_update_e RFE Abstract 2.0 - Low Level Interface, [118](#)
- rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx2_init_e RFE Abstract 2.0 - Low Level Interface, [118](#)
- rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx2_mode_e RFE Abstract 2.0 - Low Level Interface, [118](#)
- rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx2_update_e RFE Abstract 2.0 - Low Level Interface, [118](#)
- rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx3_init_e RFE Abstract 2.0 - Low Level Interface, [118](#)
- rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx3_mode_e RFE Abstract 2.0 - Low Level Interface, [118](#)
- rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx3_update_e RFE Abstract 2.0 - Low Level Interface, [118](#)
- rfeCfg_param_chirpSequence_txPhaseRotation_ddma_tx_e RFE Abstract 2.0 - Low Level Interface, [118](#)
- rfeCfg_param_general_chirpSequenceActiveSignalGpio_e RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_general_clkloConfig_e RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_general_dataOutConfig_e RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_general_jumpbackTimeTicks_e RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_general_metadataPacketSwContent_e RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_general_metadataPacketVirtualChannel_e RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_general_pdcDecimationFilter_e RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_general_powerMode_e RFE Abstract 2.0 - Low Level Interface, [115](#)
- rfeCfg_param_general_radarCycleStartSignalGpio_e

- RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_general_txCalibration_mode_e
 - RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_metadata_chirpProfileCount_e
 - RFE Abstract 2.0 - Low Level Interface, [115](#)
- rfeCfg_param_metadata_chirpSequenceConfigCount_e
 - RFE Abstract 2.0 - Low Level Interface, [115](#)
- rfeCfg_param_metadata_crc
 - RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_param_metadata_hashVersion_e
 - RFE Abstract 2.0 - Low Level Interface, [115](#)
- rfeCfg_param_metadata_majorVersion_e
 - RFE Abstract 2.0 - Low Level Interface, [115](#)
- rfeCfg_param_metadata_minorVersion_e
 - RFE Abstract 2.0 - Low Level Interface, [115](#)
- rfeCfg_param_metadata_patchVersion_e
 - RFE Abstract 2.0 - Low Level Interface, [115](#)
- rfeCfg_param_monitorAndSafety_adcClippingCountLimit_adcClippingCountLimit_e
 - RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_monitorAndSafety_adcClippingCountLimit_adcClippingCountLimit_e
 - RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_monitorAndSafety_adcClippingCountLimit_adcClippingCountLimit_e
 - RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_monitorAndSafety_adcClippingCountLimit_adcClippingCountLimit_e
 - RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_monitorAndSafety_adcClippingCountResetEveryChirpSequence_e
 - RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_monitorAndSafety_checkBbd_loi_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_checkBbd_rx_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_checkBbd_tx_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_frequencyForBist_e
 - RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_monitorAndSafety_fuSaFaultMask_0_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_fuSaFaultMask_10_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_fuSaFaultMask_11_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_fuSaFaultMask_1_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_fuSaFaultMask_2_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_fuSaFaultMask_3_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_fuSaFaultMask_4_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_fuSaFaultMask_5_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_fuSaFaultMask_6_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_fuSaFaultMask_7_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_fuSaFaultMask_8_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_fuSaFaultMask_9_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_injectTestTone_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_ppdThreshold_profile0_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_ppdThreshold_profile1_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_ppdThreshold_profile2_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_ppdThreshold_profile3_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_ppdThreshold_profile4_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_ppdThreshold_profile5_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_ppdThreshold_profile6_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_ppdThreshold_profile7_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_rxGainDiffLnaThresholdTolerance_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_rxGainDiffMixerThresholdTolerance_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_rxPhaseDiffLnaThresholdTolerance_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_rxPhaseDiffMixerThresholdTolerance_e
 - RFE Abstract 2.0 - Low Level Interface, [117](#)
- rfeCfg_param_monitorAndSafety_rxSatCountResetEveryChirpSequence_e
 - RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage1_rx1_e
 - RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage1_rx2_e
 - RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage1_rx3_e
 - RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage1_rx4_e
 - RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage2_rx1_e
 - RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage2_rx2_e
 - RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage2_rx3_e
 - RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_monitorAndSafety_rxSaturationCountLimitStage2_rx4_e
 - RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage1_rx1_e
 - RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage1_rx2_e
 - RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage1_rx3_e
 - RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage1_rx4_e
 - RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage2_rx1_e
 - RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage2_rx2_e
 - RFE Abstract 2.0 - Low Level Interface, [116](#)
- rfeCfg_param_monitorAndSafety_rxSaturationThresholdStage2_rx3_e
 - RFE Abstract 2.0 - Low Level Interface, [116](#)

© 2024 NXP Semiconductors - BFE for S32B41 - rev.0.8.10/16.12.2024

- RFE Abstract 2.0 BLOB - Low Level Interface, [225](#)
- rfeCfg_read
 - RFE Abstract 2.0 BLOB - Low Level Interface, [226](#)
- rfeCfg_readB
 - RFE Abstract 2.0 BLOB - Low Level Interface, [226](#)
- rfeCfg_readDW
 - RFE Abstract 2.0 BLOB - Low Level Interface, [227](#)
- rfeCfg_readW
 - RFE Abstract 2.0 BLOB - Low Level Interface, [227](#)
- rfeCfg_rxBistReferenceData_t, [271](#)
 - refMag, [271](#)
 - refPh, [272](#)
- rfeCfg_section_chirpProfile_0_e
 - RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_section_chirpProfile_1_e
 - RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_section_chirpProfile_2_e
 - RFE Abstract 2.0 - Low Level Interface, [120](#)
- rfeCfg_section_chirpProfile_3_e
 - RFE Abstract 2.0 - Low Level Interface, [120](#)
- rfeCfg_section_chirpProfile_4_e
 - RFE Abstract 2.0 - Low Level Interface, [120](#)
- rfeCfg_section_chirpProfile_5_e
 - RFE Abstract 2.0 - Low Level Interface, [120](#)
- rfeCfg_section_chirpProfile_6_e
 - RFE Abstract 2.0 - Low Level Interface, [120](#)
- rfeCfg_section_chirpProfile_7_e
 - RFE Abstract 2.0 - Low Level Interface, [120](#)
- rfeCfg_section_chirpSequenceConfig_0_e
 - RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_section_chirpSequenceConfig_1_e
 - RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_section_chirpSequenceConfig_2_e
 - RFE Abstract 2.0 - Low Level Interface, [120](#)
- rfeCfg_section_chirpSequenceConfig_3_e
 - RFE Abstract 2.0 - Low Level Interface, [120](#)
- rfeCfg_section_chirpSequenceConfig_4_e
 - RFE Abstract 2.0 - Low Level Interface, [120](#)
- rfeCfg_section_chirpSequenceConfig_5_e
 - RFE Abstract 2.0 - Low Level Interface, [120](#)
- rfeCfg_section_chirpSequenceConfig_6_e
 - RFE Abstract 2.0 - Low Level Interface, [120](#)
- rfeCfg_section_chirpSequenceConfig_7_e
 - RFE Abstract 2.0 - Low Level Interface, [120](#)
- rfeCfg_section_crc
 - RFE Abstract 2.0 - Low Level Interface, [120](#)
- rfeCfg_section_general_e
 - RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_section_metadata_e
 - RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_section_monitorAndSafety_e
 - RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_section_radarCycle_e
 - RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_section_t
 - RFE Abstract 2.0 - Low Level Interface, [119](#)
- rfeCfg_sequence_getFinalPcGenMode
 - RFE Abstract 2.0 BLOB - Low Level Interface, [228](#)
- rfeCfg_sequence_getPhaseRotationDdmaMode
 - RFE Abstract 2.0 BLOB - Low Level Interface, [228](#)
- rfeCfg_sequence_getPhaseRotationTxUseDdma
 - RFE Abstract 2.0 BLOB - Low Level Interface, [228](#)
- rfeCfg_sequence_getTxCalibrationThresholdHigh
 - RFE Abstract 2.0 BLOB - Low Level Interface, [228](#)
- rfeCfg_sequence_getTxCalibrationThresholdLow
 - RFE Abstract 2.0 BLOB - Low Level Interface, [229](#)
- rfeCfg_sequence_getTxDdmaInitPhase
 - RFE Abstract 2.0 BLOB - Low Level Interface, [229](#)
- rfeCfg_sequence_getTxDdmaUpdatePhase
 - RFE Abstract 2.0 BLOB - Low Level Interface, [229](#)
- rfeCfg_sequence_setPhaseRotationDdmaMode
 - RFE Abstract 2.0 BLOB - Low Level Interface, [229](#)
- rfeCfg_sequence_setTxDdmaInitPhase
 - RFE Abstract 2.0 BLOB - Low Level Interface, [231](#)
- rfeCfg_sequence_setTxDdmaUpdatePhase
 - RFE Abstract 2.0 BLOB - Low Level Interface, [231](#)
- rfeCfg_sequence_setTxFinalPcGenMode
 - RFE Abstract 2.0 BLOB - Low Level Interface, [232](#)
- rfeCfg_size_16_bit_e
 - RFE Abstract 2.0 - Low Level Interface, [120](#)
- rfeCfg_size_32_bit_e
 - RFE Abstract 2.0 - Low Level Interface, [120](#)
- rfeCfg_size_8_bit_e
 - RFE Abstract 2.0 - Low Level Interface, [120](#)
- rfeCfg_size_invalid_e
 - RFE Abstract 2.0 - Low Level Interface, [120](#)
- rfeCfg_size_t
 - RFE Abstract 2.0 - Low Level Interface, [120](#)
- rfeCfg_util_chirpProfileIndex
 - RFE Abstract 2.0 BLOB - Low Level Interface, [232](#)
- rfeCfg_util_chirpProfileSequence
 - RFE Abstract 2.0 BLOB - Low Level Interface, [232](#)
- rfeCfg_util_chirpSequenceConfigIndex
 - RFE Abstract 2.0 BLOB - Low Level Interface, [233](#)
- rfeCfg_util_radarCycleChirpSequence
 - RFE Abstract 2.0 BLOB - Low Level Interface, [233](#)
- rfeCfg_write
 - RFE Abstract 2.0 BLOB - Low Level Interface, [233](#)
- rfeCmdClient_call
 - RFE Abstract 2.0 - Low Level Interface, [142](#)
- rfeCmdClient_checkNotBusy
 - RFE Abstract 2.0 - Low Level Interface, [143](#)
- rfeCmdClient_cmdParamAddBool
 - RFE Abstract 2.0 - Low Level Interface, [143](#)
- rfeCmdClient_cmdParamAddBuffer
 - RFE Abstract 2.0 - Low Level Interface, [144](#)
- rfeCmdClient_cmdParamAddInt16
 - RFE Abstract 2.0 - Low Level Interface, [144](#)
- rfeCmdClient_cmdParamAddInt32
 - RFE Abstract 2.0 - Low Level Interface, [145](#)
- rfeCmdClient_cmdParamAddInt8
 - RFE Abstract 2.0 - Low Level Interface, [146](#)
- rfeCmdClient_cmdParamAddPointer
 - RFE Abstract 2.0 - Low Level Interface, [146](#)
- rfeCmdClient_cmdParamAddUInt16
 - RFE Abstract 2.0 - Low Level Interface, [147](#)

- rfeCmdClient_cmdParamAddUInt32
 - RFE Abstract 2.0 - Low Level Interface, [147](#)
- rfeCmdClient_cmdParamAddUInt8
 - RFE Abstract 2.0 - Low Level Interface, [148](#)
- rfeCmdClient_cmdParamStart
 - RFE Abstract 2.0 - Low Level Interface, [149](#)
- rfeCmdClient_cmdRespGetBool
 - RFE Abstract 2.0 - Low Level Interface, [149](#)
- rfeCmdClient_cmdRespGetBuf
 - RFE Abstract 2.0 - Low Level Interface, [150](#)
- rfeCmdClient_cmdRespGetInt16
 - RFE Abstract 2.0 - Low Level Interface, [150](#)
- rfeCmdClient_cmdRespGetInt32
 - RFE Abstract 2.0 - Low Level Interface, [151](#)
- rfeCmdClient_cmdRespGetInt8
 - RFE Abstract 2.0 - Low Level Interface, [151](#)
- rfeCmdClient_cmdRespGetUInt16
 - RFE Abstract 2.0 - Low Level Interface, [152](#)
- rfeCmdClient_cmdRespGetUInt32
 - RFE Abstract 2.0 - Low Level Interface, [153](#)
- rfeCmdClient_cmdRespGetUInt8
 - RFE Abstract 2.0 - Low Level Interface, [153](#)
- rfeCmdClient_configureInterrupt
 - RFE Abstract 2.0 - Low Level Interface, [154](#)
- rfeCmdClient_getCurrentRespLen
 - RFE Abstract 2.0 - Low Level Interface, [154](#)
- rfeCmdClient_init
 - RFE Abstract 2.0 - Low Level Interface, [154](#)
- rfeCmdClient_sharedDataRead
 - RFE Abstract 2.0 - Low Level Interface, [155](#)
- rfeCmdClient_sharedParameter_chirpSequenceCount_e
 - RFE Abstract 2.0 - Low Level Interface, [121](#)
- rfeCmdClient_sharedParameter_fw_ready_e
 - RFE Abstract 2.0 - Low Level Interface, [121](#)
- rfeCmdClient_sharedParameter_radarCycleCount_e
 - RFE Abstract 2.0 - Low Level Interface, [121](#)
- rfeCmdClient_sharedParameter_state_e
 - RFE Abstract 2.0 - Low Level Interface, [121](#)
- rfeCmdClient_sharedParameter_t
 - RFE Abstract 2.0 - Low Level Interface, [120](#)
- rfeCmdClient_update_t, [272](#)
 - param, [272](#)
 - section, [272](#)
 - value, [272](#)
- rfeCrc_calBuf
 - RFE Abstract 2.0 - Low Level Interface, [156](#)
- RfeDbgPrintMsg
 - RFE Abstract 2.0 - Debug Interface, [247](#)
- rfeError_error_api_boundedWaitResourcesExhausted_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfeError_error_api_TimerInUse_e
 - RFE Abstract 2.0 - Low Level Interface, [90](#)
- rfeFrontEnd_Cascaded_e
 - RFE Abstract 2.0 - Firmware Interface, [249](#)
- rfeFrontEnd_Configuration_t
 - RFE Abstract 2.0 - Firmware Interface, [249](#)
- rfeFrontEnd_Single_e
 - RFE Abstract 2.0 - Firmware Interface, [249](#)
- RfeHwDelayUs
 - RFE Abstract 2.0 - Timers Interface, [244](#)
- RfeHwGpioGet
 - RFE Abstract 2.0 - GPIO Interface, [240](#)
- RfeHwGpioInit
 - RFE Abstract 2.0 - GPIO Interface, [241](#)
- RfeHwGpioSet
 - RFE Abstract 2.0 - GPIO Interface, [241](#)
- RfeHwHeartBeatTimerGetValue
 - RFE Abstract 2.0 - Timers Interface, [245](#)
- RfeHwRaiseIRQ
 - RFE Abstract 2.0 - Interrupt Interface, [246](#)
- RfeHwSetIRQ
 - RFE Abstract 2.0 - Interrupt Interface, [246](#)
- RfeHwSpiInitialize
 - RFE Abstract 2.0 - SPI Interface, [242](#)
- RfeHwSpiTransfer
 - RFE Abstract 2.0 - SPI Interface, [242](#)
- RfeHwTimersInit
 - RFE Abstract 2.0 - Timers Interface, [245](#)
- RfeInitDebug
 - RFE Abstract 2.0 - Debug Interface, [247](#)
- rfelpc_clientCmdBuffAcquire
 - RFE Abstract 2.0 - IPC Interface, [235](#)
- rfelpc_clientCmdTx
 - RFE Abstract 2.0 - IPC Interface, [235](#)
- rfelpc_clientRspWait
 - RFE Abstract 2.0 - IPC Interface, [235](#)
- rfelpc_clientStsPoll
 - RFE Abstract 2.0 - IPC Interface, [236](#)
- rfelpc_clientStsWait
 - RFE Abstract 2.0 - IPC Interface, [236](#)
- rfelpc_srvCmdPoll
 - RFE Abstract 2.0 - IPC Interface, [236](#)
- rfelpc_srvRspBuffAcquire
 - RFE Abstract 2.0 - IPC Interface, [237](#)
- rfelpc_srvRspTx
 - RFE Abstract 2.0 - IPC Interface, [237](#)
- rfelpc_srvStsBuffAcquire
 - RFE Abstract 2.0 - IPC Interface, [238](#)
- rfelpc_srvStsTx
 - RFE Abstract 2.0 - IPC Interface, [238](#)
- rfeMainFsm
 - RFE Abstract 2.0 - Firmware Interface, [249](#)
- rfeSetFrontendsConfiguration
 - RFE Abstract 2.0 - Firmware Interface, [250](#)
- rfeSetSharedAdresses
 - RFE Abstract 2.0 - Firmware Interface, [251](#)
- rfeSwBist_InjectToneMode_t
 - RFE Abstract 2.0 - Low Level Interface, [121](#)
- rfeSwMainFsm_mainState_configured_e
 - RFE Abstract 2.0 - Firmware Interface, [249](#)
- rfeSwMainFsm_mainState_configuring_e
 - RFE Abstract 2.0 - Firmware Interface, [249](#)
- rfeSwMainFsm_mainState_fuSaFault_e
 - RFE Abstract 2.0 - Firmware Interface, [249](#)
- rfeSwMainFsm_mainState_initialized_e
 - RFE Abstract 2.0 - Firmware Interface, [249](#)

- rfeSwMainFsm_mainState_initializing_e
 - RFE Abstract 2.0 - Firmware Interface, [249](#)
- rfeSwMainFsm_mainState_radarCycles_e
 - RFE Abstract 2.0 - Firmware Interface, [249](#)
- rfeSwMainFsm_mainState_startRadarCycles_e
 - RFE Abstract 2.0 - Firmware Interface, [249](#)
- rfeSwMainFsm_mainState_t
 - RFE Abstract 2.0 - Firmware Interface, [249](#)
- rfeSwMainFsm_mainState_testContinuousWaveTransmission_e
 - RFE Abstract 2.0 - Firmware Interface, [249](#)
- rfeSwMainFsm_mainState_waitingForSync_e
 - RFE Abstract 2.0 - Firmware Interface, [249](#)
- rfeTef82XXRfBistTest_t
 - RFE Abstract 2.0 - Low Level Interface, [121](#)
- RX1Voltages
 - rfe_tef82xxVoltages_t, [268](#)
- RX2Voltages
 - rfe_tef82xxVoltages_t, [268](#)
- RX3Voltages
 - rfe_tef82xxVoltages_t, [268](#)
- RX4Voltages
 - rfe_tef82xxVoltages_t, [268](#)
- RX_NODE_VDDA_1V1
 - ATB_RXVoltages_t, [261](#)
- RX_NODE_VDDA_1V8
 - ATB_RXVoltages_t, [261](#)
- RX_NODE_VDDD_1V1
 - ATB_RXVoltages_t, [261](#)
- rxSaturationClippingCount
 - rfe_monitorValues_t, [264](#)
- section
 - rfeCmdClient_update_t, [272](#)
- SSB_NODE_VDDA_1V1
 - ATB_SSBVoltages_t, [262](#)
- SSB_NODE_VDDA_1V8
 - ATB_SSBVoltages_t, [262](#)
- SSB_NODE_VDDD_1V1
 - ATB_SSBVoltages_t, [262](#)
- SSBVoltages
 - rfe_tef82xxVoltages_t, [268](#)
- TEF82XX_alternate_Ina_mixer
 - RFE Abstract 2.0 - Low Level Interface, [121](#)
- TEF82XX_both_Ina_mixer
 - RFE Abstract 2.0 - Low Level Interface, [121](#)
- TEF82XX_ExtPinID_e
 - RFE Abstract 2.0 - GPIO Interface, [240](#)
- TEF82XX_only_mixer
 - RFE Abstract 2.0 - Low Level Interface, [121](#)
- TEF82XX_RFBIST_LNA_TEST
 - RFE Abstract 2.0 - Low Level Interface, [121](#)
- TEF82XX_RFBIST_MIXER_TEST
 - RFE Abstract 2.0 - Low Level Interface, [121](#)
- TEF82XX_RFBIST_TESTS
 - RFE Abstract 2.0 - Low Level Interface, [121](#)
- temperature_afterChirpSequence
 - rfe_monitorValues_t, [264](#)
- temperature_beforeChirpSequence
 - rfe_monitorValues_t, [264](#)
- temperature_immediately
 - rfe_monitorValues_t, [264](#)
- TX1Voltages
 - rfe_tef82xxVoltages_t, [268](#)
- TX2Voltages
 - rfe_tef82xxVoltages_t, [269](#)
- TX3Voltages
 - rfe_tef82xxVoltages_t, [269](#)
- TX_NODE_VDDA_1V1
 - ATB_TXVoltages_t, [263](#)
- TX_NODE_VDDA_1V8
 - ATB_TXVoltages_t, [263](#)
- TX_NODE_VDDD_1V1_FS
 - ATB_TXVoltages_t, [263](#)
- txPower
 - rfe_monitorValues_t, [264](#)
- value
 - rfe_registerDumpValues_t, [266](#)
 - rfeCmdClient_update_t, [272](#)

How to Reach Us:**Home Page:**nxp.com**Web Support:**nxp.com/support

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: nxp.com/SalesTermsandConditions.

NXP, the NXP logo, NXP SECURE CONNECTIONS FOR A SMARTER WORLD, COOLFLUX, EMBRACE, GREENCHIP, HITAG, I2C BUS, ICODE, JCOP, LIFE VIBES, MIFARE, MIFARE CLASSIC, MIFARE DESFire, MIFARE PLUS, MIFARE FLEX, MANTIS, MIFARE ULTRALIGHT, MIFARE4MOBILE, MIGLO, NTAG, ROADLINK, SMARTLX, SMARTMX, STARPLUG, TOPFET, TRENCHMOS, UCODE, Freescale, the Freescale logo, AltiVec, C-5, CodeTEST, CodeWarrior, ColdFire, ColdFire+, C-Ware, the Energy Efficient Solutions logo, Kinetis, Layerscape, MagniV, mobileGT, PEG, PowerQUICC, Processor Expert, QorIQ, QorIQ Converge, Ready Play, SafeAssure, the SafeAssure logo, StarCore, Symphony, VortiQa, Vybrid, Airfast, BeeKit, BeeStack, CoreNet, Flexis, MXC, Platform in a Package, QUICC Engine, SMARTMOS, Tower, TurboLink, and UMEMS are trademarks of NXP B.V. All other product or service names are the property of their respective owners. ARM, AMBA, ARM Powered, Artisan, Cortex, Jazelle, Keil, SecurCore, Thumb, TrustZone, and μ Vision are registered trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. ARM7, ARM9, ARM11, big.LITTLE, CoreLink, CoreSight, DesignStart, Mali, mbed, NEON, POP, Sensinode, Socrates, ULINK and Versatile are trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. All rights reserved. Oracle and Java are registered trademarks of Oracle and/or its affiliates. The Power Architecture and Power.org word marks and the Power and Power.org logos and related marks are trademarks and service marks licensed by Power.org.

© 2024 NXP B.V.

